

课前须知

课程讲授的内容

包括：MySQL 基础、MySQL 中表结构设计和数据类型优化、高性能索引的创建、MySQL 性能优化、InnoDB 引擎底层解析、优化背后的原理、事务的原理、MySQL 中的锁。

MySQL 的版本众多，目前最新版本为 MySQL8，考虑到实际的情况，本次课程以 CentOS7 下 MySQL5.7.32 社区 版本进行讲解，但是所讲述的内容基本在 MySQL8 中亦可适用。

下载地址：<https://downloads.mysql.com/archives/community/>

同时学习的过程中可以多参考官方文档：

官方文档地址：

<https://dev.mysql.com/doc/refman/5.7/en/>

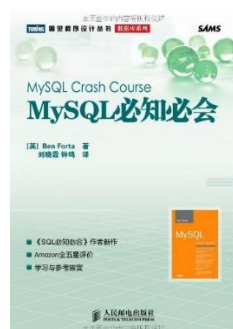
<https://dev.mysql.com/doc/mysql-errors/5.7/en/>

请注意，本课程的目的是培养 Java 架构师应该掌握的 MySQL 的知识，通过这些知识，旨在解决工作和面试中作为 Java 架构师可能会遇到的问题，比如合理的设计表结构、索引优化、SQL 语句优化等等，而不是培养 MySQL 的 DBA !!!

本课程推荐书籍：



对 MySQL 非常不熟悉的同学请先参阅：



MySQL DBA 需要关注的方面包括的数据库的原理和实战、主机硬件、OS、网络、安全、NOSQL，以及一些 MySQL 运维自动化开发。一线的大型互联网公司里，还区分运维 DBA、开发 DBA、DB 架构师等不同岗位。有志成为 MySQL DBA 的同学除了前面推荐的书籍：

原理类：

《数据库系统概念 原书第六版》、《数据库系统实现/概念》、《数据库与事务处理》、《数据库索引设计与优化》、《事务处理 概念与技术》

操作运维、源码类：

《MySQL 官方文档》、《MySQL 技术内幕 第 5 版》、《深入理解 MySQL》、《深入理解 MySQL 核心技术》、《MySQL 性能调优与架构设计》、《高可用 MySQL——构建健壮的数据中心》、《MySQL 技术内幕：InnoDB 存储引擎 第 2 版》、《数据库查询优化器的艺术》和《MySQL 运维内参》、《高性能 MySQL》、PYTHON（PERL）、Linux 运维和脚本编写等。

MySQL 的分支、变种与替代

MySQL 变种有好几个，主要有三个久经考验的主流变种：Percona Server，MariaDB 和 Drizzle。它们都有活跃的用户社区和某种程度上的商业支持，均由独立的服务供应商支持。

同时还有几个优秀的开源关系数据库，值得我们了解一下。

Drizzle

Drizzle 是真正的 MySQL 分支，而且是完全开源的产品，而非只是个变种或增强版本。它并不与 MySQL 兼容不能简单地将 MySQL 后端替换为 Drizzle。

Drizzle 与 MySQL 有很大差别，进行了一些重大更改，甚至 SQL 语法的变化都非常大，设计目标之一是提供一种出色的解决方案来解决高可用性問題。在实现上，Drizzle 清除了一些表现不佳和不必要的功能，将很多代码重写，对它们进行了优化，甚至将所用语言从 C 换成了 C++。

此外，Drizzle 另一个设计目标是能很好的适应具有大量内容的多核服务器、运行 Linux 的 64 位机器、云计算中使用的服务器、托管网站的服务器和每分钟接收数以万计点击率的服务器并且大幅度的削减服务器成本。

MariaDB

在 Sun 收购 MySQL 后，Monty Widenius，这位 MySQL 的创建者，因不认同 MySQL 开发流程而离开 Sun。他成立了 Monty 程序公司，创立了 MariaDB。MariaDB 的目标是社区开发，Bug 修复和许多的新特性实际上，可以将 MariaDB 视为 MySQL 的扩展集，它不仅提供 MySQL 提供的所有功能，还提供其他功能。MariaDB 是原版 MySQL 的超集，因此已有的系统不需要任何修改就可以运行。

诸如 Google、Facebook、维基百科等公司或者网站所使用了 MariaDB。不过 Monty 公司不是以赢利为目的，而是由产品驱动的，这可能会带来问题，因为没有赢利的公司不一定能长久维持下去。

Percona Server

由领先的 MySQL 咨询公司 Percona 发布，Percona 公司的口号就是 “The Database Performance Experts”，Percona 的创始人也就是《高性能 MySQL》书的作者。

Percona Server 是个与 MySQL 向后兼容的替代品，它尽可能不改变 SQL 语法、客户端/服务器协议和磁盘上的文件格式。任何运行在 MySQL 上的都可以运行在 Percona Server 上而不需要修改。切换到 Percona Server 只需要关闭 MySQL 和启动 PerconaServer，不需要导出和重新导入数据。

Percona Server 有三个主要的目标：透明，增加允许用户更紧密地查看服务器内部信息和行为的方法。比如慢查询日志中特别增加的详细信息；性能，Percona Server 包含许多性能和可扩展性方面的改进，还加强了性能的可预测性和稳定性。其中主要集中于 InnoDB；操作灵活性，Percona Server 使操作人员和系统管理员在让 MySQL 作为架构的一部分而可靠并稳定运行时提供了很多便利。

一般来说，Percona Server 中的许多特性会在后来的标准 MySQL 中出现。

国内公司阿里内部就运行了上千个 Percona Server 的实例。

Postgre SQL

PostgreSQL 称自己是世界上最先进的开源数据库，同时也是个一专多长的全栈数据库。最初是 1985 年在加利福尼亚大学伯克利分校开发的。

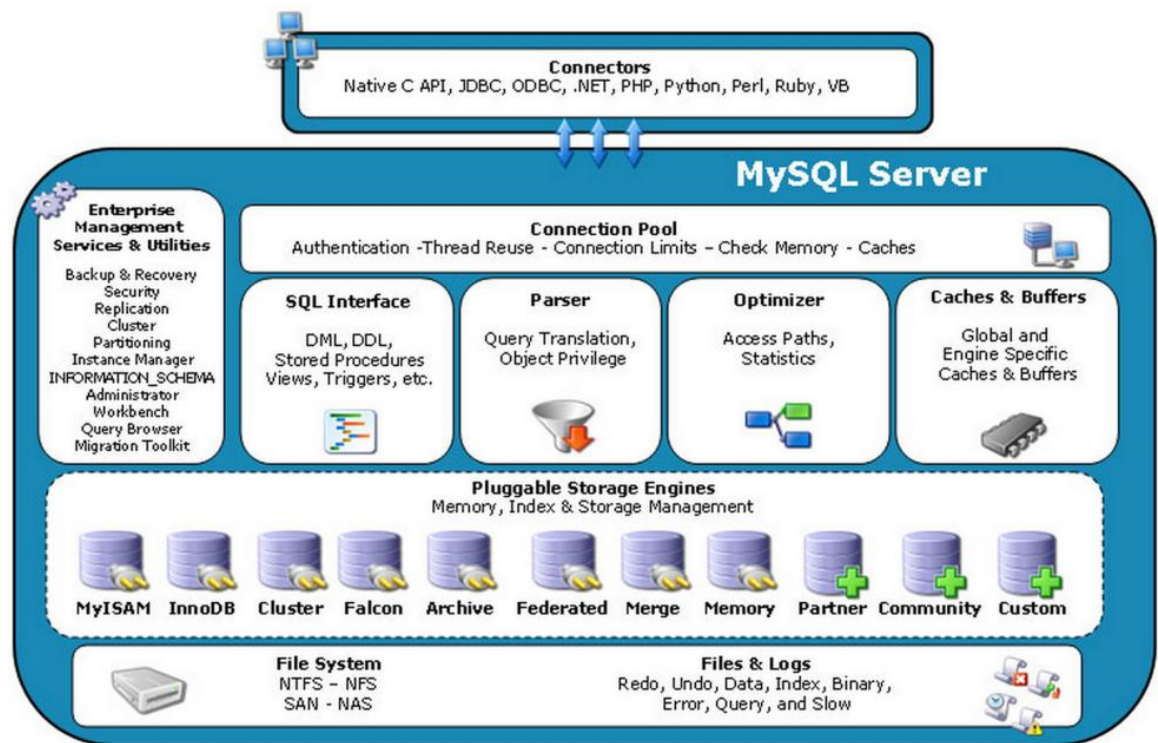
PostgreSQL 的稳定性极强，在崩溃、断电之类的灾难场景下依然可以保证数据的正确；在高并发读写，负载逼近极限下，PG 的性能指标仍可以维持双曲线甚至对数曲线，到顶峰之后不再下降，表现的非常稳定，而 MySQL 明显出现一个波峰后下滑；PG 多年来在 GIS 领域处于优势地位，因为它有丰富的几何类型，实际上不止几何类型，PG 有大量字典、数组、bitmap 等数据类型，相比之下 mysql 就差很多。所以总的来说，PostgreSQL 更学术化一些，在绝对需要可靠性和数据完整性的时候，PostgreSQL 是更好的选择。但是从商业支持、文档资料、易用性，第三方支持来说，MySQL 无疑更好些。

SQLite

SQLite 是世界上部署最广泛的数据库引擎，为物联网（IoT）下的数据库首选，并且是手机，PDA，甚至 MP3 播放器的下的首选。SQLite 代码占用空间小，并且不需要数据库管理员的维护。SQLite 没有单独的服务器进程，提供的事务也基本符合 ACID。当然，简单也就意味着功能和性能受限。

1.MySql 基础

1.1. MySQL 体系架构



可以看出 MySQL 最上层是连接组件。下面服务器是由连接池、管理工具和服务、SQL 接口、解析器、优化器、缓存、存储引擎、文件系统组成。

连接池：由于每次建立连接需要消耗很多时间，连接池的作用就是将这些连接缓存下来，下次可以直接用已经建立好的连接，提升服务器性能。

管理工具和服务：系统管理和控制工具，例如备份恢复、Mysql 复制、集群等

SQL 接口：接受用户的 SQL 命令，并且返回用户需要查询的结果。比如 select from 就是调用 SQL Interface

解析器：SQL 命令传递到解析器的时候会被解析器验证和解析。解析器主要功能：

a. 将 SQL 语句分解成数据结构，并将这个结构传递到后续步骤，以后 SQL 语句的传递和处理就是基于这个结构的

b. 如果在分解构成中遇到错误，那么就说明这个 sql 语句是不合理的

优化器：查询优化器，SQL 语句在查询之前会使用查询优化器对查询进行优化。

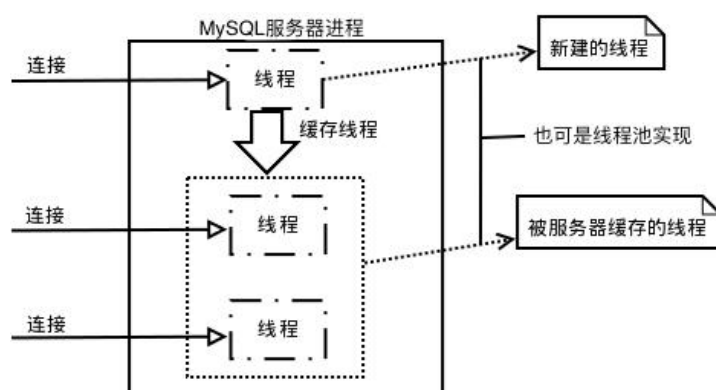
缓存器： 查询缓存，如果查询缓存有命中的查询结果，查询语句就可以直接去查询缓存中取数据。

这个缓存机制是由一系列小缓存组成的。比如表缓存，记录缓存，key 缓存，权限缓存等。

存储引擎、文件系统的作用，后面会细讲。

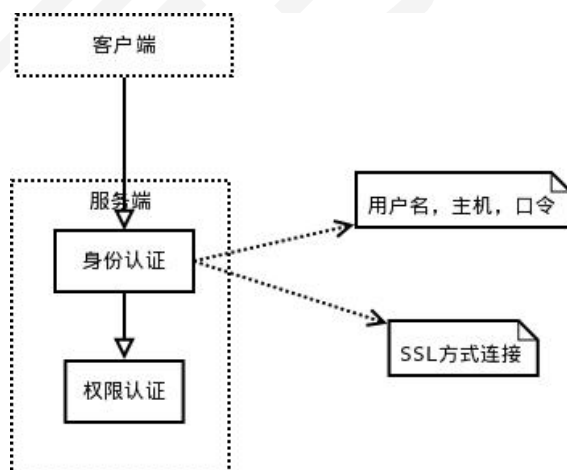
1.1.1. 连接层

1.1.1.1. 服务器处理客户端请求



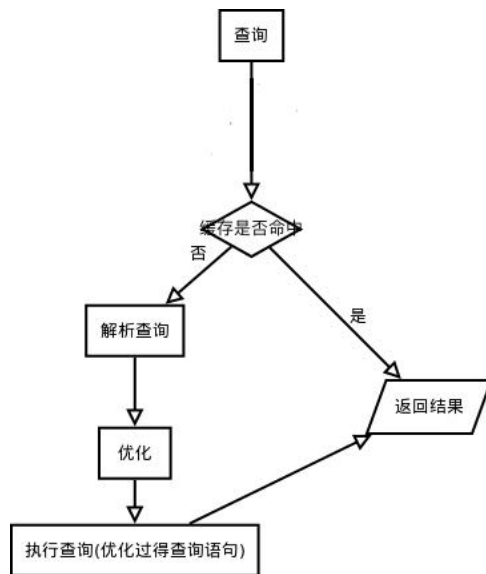
当 MySQL 启动（MySQL 服务器就是一个进程），等待客户端连接，每一个客户端连接请求，服务器都会新建一个线程处理（如果是线程池的话，则是分配一个空的线程），每个线程独立，拥有各自的内存处理空间

show VARIABLES like '%max_connections%'



连接到服务器，服务器需要对其进行验证，也就是用户名、IP、密码验证，一旦连接成功，还要验证是否具有执行某个特定查询的权限（例如，是否允许客户端对某个数据库某个表的某个操作）

1.1.2. Server 层(SQL 处理层)



这一层主要功能有：SQL 语句的解析、优化，缓存的查询，MySQL 内置函数的实现，跨存储引擎功能（所谓跨存储引擎就是说每个引擎都需提供的功能（引擎需对外提供接口）），例如：存储过程、触发器、视图等。

1.如果是查询语句（select 语句），首先会查询缓存是否已有相应结果，有则返回结果，无则进行下一步（如果不是查询语句，同样调到下一步）

2.解析查询，创建一个内部数据结构（解析树），这个解析树主要用来 SQL 语句的语义与语法解析；

3.优化：优化 SQL 语句，例如重写查询，决定表的读取顺序，以及选择需要的索引等。这一阶段用户是可以查询的，查询服务器优化器是如何进行优化的，便于用户重构查询和修改相关配置，达到最优化。这一阶段还涉及到存储引擎，优化器会询问存储引擎，比如某个操作的开销信息、是否对特定索引有查询优化等。

1.1.2.1. 缓存

```
show variables like '%query_cache_type%' -- 默认不开启
```

```
show variables like '%query_cache_size%' --默认值 1M
```

```
SET GLOBAL query_cache_type = 1; --会报错
```

query_cache_type 只能配置在 my.cnf 文件中，这大大限制了 qc 的作用

在生产环境建议不开启，除非经常有 sql 完全一模一样的查询

QC 严格要求 2 次 SQL 请求要完全一样，包括 SQL 语句，连接的数据库、协议版本、字符集等因素都会影响

从 8.0 开始，MySQL 不再使用查询缓存，那么放弃它的原因是什么呢？

MySQL 查询缓存是查询结果缓存。它将以 `SEL` 开头的查询与哈希表进行比较，如果匹配，则返回上一次查询的结果。进行匹配时，查询必须逐字节匹配，例如 `SELECT * FROM e1;` 不等于 `select * from e1;`，此外，一些不确定的查询结果无法被缓存，任何对表的修改都会导致这些表的所有缓存无效。因此，适用于查询缓存的最理想的方案是只读，特别是需要检查数百万行后仅返回数行的复杂查询。如果你的查询符合这样一个特点，开启查询缓存会提升你的查询性能。

随着技术的进步，经过时间的考验，MySQL 的工程团队发现启用缓存的好处并不多。

首先，查询缓存的效果取决于缓存的命中率，只有命中缓存的查询效果才能有改善，因此无法预测其性能。

其次，查询缓存的另一个大问题是它受到单个互斥锁的保护。在具有多个内核的服务器上，大量查询会导致大量的互斥锁争用。

通过基准测试发现，大多数工作负载最好禁用查询缓存(5.6 的默认设置)：按照官方所说的：造成的问题比它解决问题要多的多，弊大于利就直接砍掉了

1.1.3. 存储引擎层

从体系结构图中可以发现，MySQL 数据库区别于其他数据库的最重要的一个特点就是其插件式的表存储引擎。MySQL 插件式的存储引擎架构提供了一系列标准的管理和服务支持，这些标准与存储引擎本身无关，可能是每个数据库系统本身都必需的，如 SQL 分析器和优化器等，而存储引擎是底层物理结构和实际文件读写的实现，每个存储引擎开发者可以按照自己的意愿来进行开发。需要特别注意的是，存储引擎是基于表的，而不是数据库。

插件式存储引擎的好处是，每个存储引擎都有各自的特点，能够根据具体的应用建立不同存储引擎表。由于 MySQL 数据库的开源特性，用户可以根据 MySQL 预定义的存储引擎接口编写自己的存储引擎。若用户对某一种存储引擎的性能或功能不满意，可以通过修改源码来得到想要的特性，这就是开源带给我们的方便与力量。

由于 MySQL 数据库开源特性，存储引擎可以分为 MySQL 官方存储引擎和第三方存储引擎。有些第三方存储引擎很强大，如大名鼎鼎的 InnoDB 存储引擎（最早是第三方存储引擎，后被 Oracle 收购），其应用就极其广泛，甚至是 MySQL 数据库 OLTP(Online Transaction Processing 在线事务处理)应用中使用最广泛的存储引擎。

1.1.3.1. MySQL 官方引擎概要

InnoDB 存储引擎

InnoDB 是 MySQL 的默认事务型引擎，也是最重要、使用最广泛的存储引擎。它被设计用来处理大量的短期(short-lived)事务，短期事务大部分情况是正常提交的，很少会被回滚。InnoDB 的性能和自动崩溃恢复特性，使得它在非事务型存储的需求中也很流行。除非有非常特别的原因需要使用其他的存储引擎，否则应该优先考虑 InnoDB 引擎。如果要学习存储引擎，InnoDB 也是一个非常好的值得

花最多的时间去深入学习的对象，收益肯定比将时间平均花在每个存储引擎的学习上要高得多。所以 InnoDB 引擎也将是我们学习的重点。

MyISAM 存储引擎

在 MySQL 5.1 及之前的版本，MyISAM 是默认的存储引擎。MyISAM 提供了大量的特性，包括全文索引、压缩、空间函数（GIS）等，但 MyISAM 不支持事务和行级锁，而且有一个毫无疑问的缺陷就是崩溃后无法安全恢复。尽管 MyISAM 引擎不支持事务、不支持崩溃后的安全恢复，但它绝不是一无是处的。对于只读的数据，或者表比较小、可以忍受修复（repair）操作，则依然可以继续使用 MyISAM（但请不要默认使用 MyISAM，而是应当默认使用 InnoDB）。但是 MyISAM 对整张表加锁，而不是针对行。读取时会对需要读到的所有表加共享锁，写入时则对表加排他锁。MyISAM 很容易因为表锁的问题导致典型的性能问题。

Mrg_MyISAM

Merge 存储引擎，是一组 MyISAM 的组合，也就是说，他将 MyISAM 引擎的多个表聚合起来，但是他的内部没有数据，真正的数据依然是 MyISAM 引擎的表中，但是可以直接进行查询、删除更新等操作。

Archive 引擎

Archive 存储引擎只支持 INSERT 和 SELECT 操作，在 MySQL 5.1 之前也不支持索引。Archive 引擎会缓存所有的写并利用 zlib 对插入的行进行压缩，所以比 MyISAM 表的磁盘 I/O 更少。但是每次 SELECT 查询都需要执行全表扫描。所以 Archive 表适合日志和数据采集类应用，这类应用做数据分析时往往需要全表扫描。或者在一些需要更快速的 INSERT 操作的场合下也可以使用。Archive 引擎不是一个事务型的引擎，而是一个针对高速插入和压缩做了优化的简单引擎。

Blackhole 引擎

Blackhole 引擎没有实现任何的存储机制，它会丢弃所有插入的数据，不做任何保存。但是服务器会记录 Blackhole 表的日志，所以可以用于复制数据到备库，或者只是简单地记录到日志。这种特殊的存储引擎可以在一些特殊的复制架构和日志审核时发挥作用。但这种引擎在应用方式上有很多问题，因此并不推荐。

CSV 引擎

CSV 引擎可以将普通的 CSV 文件（逗号分割值的文件）作为 MySQL 的表来处理，但这种表不支持索引。CSV 引擎可以在数据库运行时拷入或者拷出文件。可以将 Excel 等的数据存储为 CSV 文件，然后复制到 MySQL 数据目录下，就能在 MySQL 中打开使用。同样，如果将数据写入到一个 CSV 引擎表，其他的外部程序也能立即从表的数据文件中读取 CSV 格式的数据。因此 CSV 引擎可以作为一种数据交换的机制，非常有用。

Federated 引擎

Federated 引擎是访问其他 MySQL 服务器的一个代理，它会创建一个到远程 MySQL 服务器的客户端连接，并将查询传输到远程服务器执行，然后提取或者发送需要的数据。最初设计该存储引擎是为了和企业级数据库如 Microsoft SQL Server 和 Oracle 的类似特性竞争的，可以说更多的是一种市场行为。尽管该引

引擎看起来提供了一种很好的跨服务器的灵活性，但也经常带来问题，因此默认是禁用的。

Memory 引擎

如果需要快速地访问数据，并且这些数据不会被修改，重启以后丢失也没有关系，那么使用 Memory 表(以前也叫做 HEAP 表)是非常有用的。Memory 表至少比 MyISAM 表要快一个数量级，因为每个基于 MEMORY 存储引擎的表实际对应一个磁盘文件。该文件的文件名与表名相同，类型为 frm 类型。该文件中只存储表的结构。而其数据文件，都是存储在内存中，这样有利于数据的快速处理，提高整个表的效率，不需要进行磁盘 I/O。所以 Memory 表的结构在重启以后还会保留，但数据会丢失。

Memory 表在很多场景可以发挥好的作用：

用于查找(lookup)或者映射(mapping)表，例如将邮编和州名映射的表。

用于缓存周期性聚合数据(periodically aggregated data)的结果。

用于保存数据分析中产生的中间数据。

Memory 表支持 Hash 索引，因此查找操作非常快。虽然 Memory 表的速度非常快，但还是无法取代传统的基于磁盘的表。Memory 表是表级锁，因此并发写入的性能较低。它不支持 BLOB 或 TEXT 类型的列，并且每行的长度是固定的，所以即使指定了 VARCHAR 列，实际存储时也会转换成 CHAR，这可能导致部分内存的浪费。

NDB 集群引擎

使用 MySQL 服务器、NDB 集群存储引擎，以及分布式的、share-nothing 的、容灾的、高可用的 NDB 数据库的组合，被称为 MySQL 集群 (MySQL Cluster)。

1.1.3.2. 值得了解的第三方引擎

Percona 的 XtraDB 存储引擎

基于 InnoDB 引擎的一个改进版本，已经包含在 Percona Server 和 MariaDB 中，它的改进点主要集中在性能、可测量性和操作灵活性方面。XtraDB 可以作为 InnoDB 的一个完全的替代产品，甚至可以兼容地读写 InnoDB 的数据文件，并支持 InnoDB 的所有查询。

TokuDB 引擎

使用了一种新的叫做分形树(Fractal Trees)的索引数据结构。该结构是缓存无关的，因此即使其大小超过内存性能也不会下降，也就没有内存生命周期和碎片的问题。TokuDB 是一种大数据 (Big Data)存储引擎，因为其拥有很高的压缩比，可以在很大的数据量上创建大量索引。现在该引擎也被 Percona 公司收购。

Tips: 分形树，是一种写优化的磁盘索引数据结构。在一般情况下，分形树的写操作 (Insert/Update/Delete) 性能比较好，同时它还能保证读操作近似于 B+树的读性能。据测试结果显示，TokuDB 分形树的写性能优于 InnoDB 的 B+树，读性能略低于 B+树。分形树核心思想是利用节点的 MessageBuffer 缓存更新操

作，充分利用数据局部性原理，将随机写转换为顺序写，这样极大的提高了随机写的效率。

Infobright

MySQL 默认是面向行的，每一行的数据是一起存储的，服务器的查询也是以行为单位处理的。而在大数据量处理时，面向列的方式可能效率更高，比如 HBASE 就是面向列存储的。

Infobright 是最有名的面向列的存储引擎。在非常大的数据量（数十 TB）时，该引擎工作良好。Infobright 是为数据分析和数据仓库应用设计的。数据高度压缩，按照块进行排序，每个块都对应有一组元数据。在处理查询时，访问元数据可决定跳过该块，甚至可能只需要元数据即可满足查询的需求。但该引擎不支持索引，不过在这么大的数据量级，即使有索引也很难发挥作用，而且块结构也是一种准索引 (quasi-index)。Infobright 需要对 MySQL 服务器做定制，因为一些地方需要修改以适应面向列存储的需要。如果查询无法在存储层使用面向列的模式执行，则需要在服务器层转换成按行处理，这个过程会很慢。Infobright 有社区版和商业版两个版本。

其他

针对图操作，全文检索，MySQL 下都有对应的存储引擎，大家可以自行查阅。

1.1.3.3. 选择合适的引擎

这么多存储引擎，我们怎么选择？大部分情况下，InnoDB 都是正确的选择，所以在 MySQL 5.5 版本将 InnoDB 作为默认的存储引擎了。对于如何选择存储引擎，可以简单地归纳为一句话：“除非需要用到某些 InnoDB 不具备的特性，并且没有其他办法可以替代，否则都应该优先选择 InnoDB 引擎”。比如，MySQL 中只有 MyISAM 支持地理空间搜索。

当然，如果不需要用到 InnoDB 的特性，同时其他引擎的特性能够更好地满足需求，也可以考虑一下其他存储引擎。举个例子，如果不在乎可扩展能力和并发能力，也不在乎崩溃后的数据丢失问题，却对 InnoDB 的空间占用过多比较敏感，这种场合下选择 MyISAM 就比较合适。

除非万不得已，否则建议不要混合使用多种存储引擎，否则可能带来一系列复杂的问题，以及一些潜在的 bug 和边界问题。存储引擎层和服务器层的交互已经比较复杂，更不用说混合多个存储引擎了。至少，混合存储对一致性备份和服务器参数配置都带来了一些困难。

1.1.3.4. 表引擎的转换

有很多种方法可以将表的存储引擎转换成另外一种引擎。每种方法都有其优点和缺点。常用的有三种方法

ALTER TABLE

将表从一个引擎修改为另一个引擎最简单的办法是使用 ALTER TABLE 语句。下面的语句将 mytable 的引擎修改为 InnoDB：

```
mysql> ALTER TABLE mytable ENGINE = InnoDB;
```

上述语法可以适用任何存储引擎。但需要执行很长时间，在实现上，MySQL 会按行将数据从原表复制到一张新的表中，在复制期间可能会消耗系统所有的 I/O 能力，同时原表上会加上读锁。所以，在繁忙的表上执行此操作要特别小心。

如果转换表的存储引擎，将会失去和原引擎相关的所有特性。

导出与导入

还可以使用 `mysqldump` 工具将数据导出到文件，然后修改文件中 `CREATE TABLE` 语句的存储引擎选项，注意同时修改表名，因为同一个数据库中不能存在相同的表名，即使它们使用的是不同的存储引擎。

CREATE 和 SELECT

先创建一个新的存储引擎的表，然后利用 `INSERT...SELECT` 语法来导出数据：

```
mysql>CREATE TABLE innodb_table LIKE myisam_table;
```

```
mysql>ALTER TABLE innodb_table ENGINE=InnoDB;
```

```
mysql>INSERT INTO innodb_table SELECT * FROM myisam_table;
```

如果数据量很大，则可以考虑做分批处理，针对每一段数据执行事务提交操作。

1.1.3.5. 检查我的 MySQL 的引擎

#看我的 MySQL 现在已提供什么存储引擎：

```
mysql> show engines;
```

```
mysql> show engines;
```

Engine	Support	Comment	Transactions	XA	Savepoints
InnoDB	DEFAULT	Supports transactions, row-level locking, and foreign keys	YES	YES	YES
MRG_MYISAM	YES	Collection of identical MyISAM tables	NO	NO	NO
MEMORY	YES	Hash based, stored in memory, useful for temporary tables	NO	NO	NO
BLACKHOLE	YES	/dev/null storage engine (anything you write to it disappears)	NO	NO	NO
MyISAM	YES	MyISAM storage engine	NO	NO	NO
CSV	YES	CSV storage engine	NO	NO	NO
ARCHIVE	YES	Archive storage engine	NO	NO	NO
PERFORMANCE_SCHEMA	YES	Performance Schema	NO	NO	NO
FEDERATED	NO	Federated MySQL storage engine	NULL	NULL	NULL

9 rows in set (0.00 sec)

#看我的 MySQL 当前默认的存储引擎：

```
mysql> show variables like '%storage_engine%';
```

```
mysql> show variables like '%storage_engine%';
```

Variable_name	Value
default_storage_engine	InnoDB
default_tmp_storage_engine	InnoDB
disabled_storage_engines	
internal_tmp_disk_storage_engine	InnoDB

4 rows in set (0.00 sec)

1.1.3.6. MyISAM 和 InnoDB 比较

对比项	MyISAM	InnoDB
主外键	不支持	支持
事务	不支持	支持
行表锁	表锁,即使操作一条记录也会锁住整个表 不适合高并发的操作	行锁,操作时只锁某一行,不对其它行有影响 适合高并发的操作
缓存	只缓存索引,不缓存真实数据	不仅缓存索引还要缓存真实数据,对内存要求较高,而且内存大小对性能有决

1.2. MySQL 中的目录和文件

1.2.1. bin 目录

在 MySQL 的安装目录下有一个特别特别重要的 bin 目录,这个目录下存放着许多可执行文件。

```
[mysql@i2wz9j203itc4guluwb2w2 bin]$ pwd
/home/mysql/mysql57/bin
[mysql@i2wz9j203itc4guluwb2w2 bin]$ ls
innosetup      myisampack      mysqlcheck      mysqld-debug    mysql_embedded  mysql_secure_installation  mysql_tzinfo_to_sql  resolveip
lib4_decompress  my_print_defaults  mysql_client_test_embedded  mysqld_multi    mysqlimport     mysqlshow          mysql_upgrade        resolve_stack_dump
myisamchk       mysql            mysql_config     mysqld_safe     mysql_install_db  mysqlslap          mysqlxtest           zlib_decompress
myisam_ftdump   mysqladmin       mysql_config_editor  mysqldump       mysql_plugin     mysql_ssl_rsa_setup  perror               replace
myisamlog       mysqlbinlog      mysqld           mysqldumpslow   mysqlpump        mysqltest_embedded
```

其他系统中的可执行文件与此的类似。这些可执行文件都是与服务器程序和客户端程序相关的。

1.2.1.1. 启动 MySQL 服务器程序

在类 UNIX 系统中用来启动 MySQL 服务器程序的可执行文件有很多,大多在 MySQL 安装目录的 bin 目录下。

mysqld

mysqld 这个可执行文件就代表着 MySQL 服务器程序,运行这个可执行文件就可以直接启动一个服务器进程。但这个命令不常用。

mysqld_safe

mysqld_safe 是一个启动脚本,它会间接的调用 mysqld,而且还顺便启动了另外一个监控进程,这个监控进程在服务器进程挂了的时候,可以帮助重启它。另外,使用 mysqld_safe 启动服务器程序时,它会将服务器程序的出错信息和其他诊断信息重定向到某个文件中,产生出错日志,这样可以方便我们找出发生错误的原因。

mysql.server

mysql.server 也是一个启动脚本,它会间接的调用 mysqld_safe,在调用 mysql.server 时在后边指定 start 参数就可以启动服务器程序了

就像这样:

```
mysql.server start
```


需要注意的是，这个 `mysql.server` 文件其实是一个链接文件，它的实际文件是 `support-files/mysql.server`，所以如果在 `bin` 目录找不到，到 `support-files` 下去找找，而且如果你愿意的话，自行用 `ln` 命令在 `bin` 创建一个链接。

另外，我们还可以使用 `mysql.server` 命令来关闭正在运行的服务器程序，只要把 `start` 参数换成 `stop` 就好了：

```
mysql.server stop
```

`mysqld_multi`

其实我们一台计算机上也可以运行多个服务器实例，也就是运行多个 MySQL 服务器进程。`mysql_multi` 可执行文件可以对每一个服务器进程的启动或停止进行监控。

1.2.1.2. 客户端程序

在我们成功启动 MySQL 服务器程序后，就可以接着启动客户端程序来连接到这个服务器喽，`bin` 目录下有许多客户端程序，比方说 `mysqladmin`、`mysqldump`、`mysqlcheck` 等等。

我们常用的是可执行文件 `mysql`，通过这个可执行文件可以让我们和服务器程序进程交互，也就是发送请求，接收服务器的处理结果。

`mysqladmin` 执行管理操作的工具，检查服务器配置、当前运行状态，创建、删除数据库、设置新密码。

`mysqldump` 数据库逻辑备份程序。

`mysqlbackup` 备份数据表、整个数据库、所有数据库，一般来说 `mysqldump` 备份、`mysql` 还原。

1.2.2. 启动选项和参数

1.2.2.1. 配置参数文件

当 MySQL 实例启动时，数据库会先去读一个配置参数文件，用来寻找数据库的各种文件所在位置以及指定某些初始化参数，这些参数通常定义了某种内存结构有多大等。在默认情况下，MySQL 实例会按照一定的顺序在指定的位置进行读取，用户只需通过命令 `mysql --help|grep my.cnf` 来寻找即可。

```
[mysql@i2wz9j203ithc4guluvb2w2 mysql57]$ ./bin/mysql --help|grep my.cnf
      order of preference, my.cnf, $MYSQL_TCP_PORT,
/etc/my.cnf /etc/mysql/my.cnf /usr/local/mysql/etc/my.cnf ~/.my.cnf
```

当然，也可以在启动 MySQL 时，指定配置文件：

```
[mysql@i2wz9j203ithc4guluvb2w2 mysql57]$ ./bin/mysqld_safe --defaults-file=/home/mysql/mysql57/conf/my.cnf --user=mysql &
[1] 18210
[mysql@i2wz9j203ithc4guluvb2w2 mysql57]$ 2021-04-05T02:52:39.268767Z mysqld_safe Logging to '/home/mysql/mysql57/log/error.log'.
2021-04-05T02:52:39.302753Z mysqld_safe Starting mysqld daemon with databases from /home/mysql/mysql57/data
```



```
[mysql@i2wz9j203ithc4guluvb2wZ mysql57]$ ps -ef|grep mysqld
mysql      18210 18090  0 10:52 pts/1    00:00:00 /bin/sh ./bin/mysqld_safe --defaults-file=/home/mysql/mysql57/conf/my.cnf -
--user=mysql
mysql      18379 18210  0 10:52 pts/1    00:00:00 /home/mysql/mysql57/bin/mysqld --defaults-file=/home/mysql/mysql57/conf/my.
cnf --basedir=/home/mysql/mysql57 --datadir=/home/mysql/mysql57/data --plugin-dir=/home/mysql/mysql57/lib/plugin --log-erro
r=/home/mysql/mysql57/log/error.log --pid-file=/home/mysql/mysql57/log/mysql.pid --socket=/home/mysql/mysql57/log/mysql.soc
k --port=3306
```

这个时候，就会以启动时指定的配置文件为准。

MySQL 数据库参数文件的作用和 Oracle 数据库的参数文件极其类似，不同的是，Oracle 实例在启动时若找不到参数文件，是不能进行装载(mount)操作的。MySQL 稍微有所不同，MySQL 实例可以不需要参数文件，这时所有的参数值取决于编译 MySQL 时指定的默认值和源代码中指定参数的默认值。

MySQL 数据库的参数文件是以文本方式进行存储的。可以直接通过一些常用的文本编辑软件进行参数的修改。

1.2.2.2. 参数的查看和修改

可以通过命令 `show variables` 查看数据库中的所有参数，也可以通过 `LIKE` 来过滤参数名，前面查找数据库引擎时已经展示过了。从 MySQL 5.1 版本开始，还可以通过 `information_schema` 架构下的 `GLOBAL_VARIABLES` 视图来进行查找，推荐使用命令 `show variables`，使用更为简单，且各版本的 MySQL 数据库都支持。

参数的具体含义可以参考 MySQL 官方手册：

<https://dev.mysql.com/doc/refman/5.7/en/server-system-variables.html>

但是课程中遇到的参数会进行讲解。

MySQL 数据库中的参数可以分为两类：动态(dynamic)参数和静态(static)参数。同时从作用范围又可以分为全局变量和会话变量。

动态参数意味着可以在 MySQL 实例运行中进行更改，静态参数说明在整个实例生命周期内都不得进行更改，就好像是只读(read only)的。

全局变量（GLOBAL）影响服务器的整体操作。

会话变量（SESSION/LOCAL）影响某个客户端连接的操作。

举个例子，用 `default_storage_engine` 来说明，在服务器启动时会初始化一个名为 `default_storage_engine`，作用范围为 GLOBAL 的系统变量。之后每当有一个客户端连接到该服务器时，服务器都会单独为该客户端分配一个名为 `default_storage_engine`，作用范围为 SESSION 的系统变量，该作用范围为 SESSION 的系统变量值按照当前作用范围为 GLOBAL 的同名系统变量值进行初始化。

可以通过 `SET` 命令对动态的参数值进行修改。

SET 的语法如下：

SET

| [global | session] system_var_name= expr

| [@@global.| @@session.] system_var_name= expr

比如：

SET read_buffer_size=524288;

```
SET @@global.read_buffer_size=524288;
```

MySQL 所有动态变量的可修改范围，可以参考 MySQL 官方手册的 [Dynamic System Variables](#) 的相关内容：

<https://dev.mysql.com/doc/refman/5.7/en/dynamic-system-variables.html>

对于静态变量，若对其进行修改，会得到类似如下错误：

```
mysql> set global datadir='/db/mysql';  
ERROR 1238 (HY000): Variable 'datadir' is a read only variable
```

1.2.3. 数据目录

我们知道像 InnoDB、MyISAM 这样的存储引擎都是把表存储在磁盘上的，而操作系统用来管理磁盘的那个东东又被称为文件系统，所以用专业一点的话来表述就是：像 InnoDB、MyISAM 这样的存储引擎都是把表存储在文件系统上的。当我们想读取数据的时候，这些存储引擎会从文件系统中把数据读出来返回给我们，当我们想写入数据的时候，这些存储引擎会把这些数据又写回文件系统。

1.2.3.1. 确定 MySQL 中的数据目录

那说了半天，到底 MySQL 把数据都存到哪个路径下呢？其实数据目录对应着一个系统变量 `datadir`，我们在使用客户端与服务器建立连接之后查看这个系统变量的值就可以了：

```
show variables like 'datadir';
```

```
mysql> show variables like 'datadir';  
+-----+-----+  
| Variable_name | Value                               |  
+-----+-----+  
| datadir       | /home/mysql/mysql57/data/         |  
+-----+-----+  
1 row in set (0.00 sec)
```

当然这个目录可以通过配置文件进行修改，由我们自己进行指定。

1.2.3.2. 数据目录中放些什么？

MySQL 在运行过程中都会产生哪些数据呢？当然会包含我们创建的数据库、表、视图和触发器等用户数据，除了这些用户数据，为了程序更好的运行，MySQL 也会创建一些其他的额外数据

数据库在文件系统中的表示

每当我们使用 `CREATE DATABASE` 语句创建一个数据库的时候，在文件系统上实际发生了什么呢？其实很简单，每个数据库都对应数据目录下的一个子目录，或者说对应一个文件夹，我们每当我们新建一个数据库时，MySQL 会帮我们做这两件事儿：

1. 在数据目录下创建一个和数据库名同名的子目录（或者说是文件夹）。

2. 在该与数据库名同名的子目录下创建一个名为 `db.opt` 的文件，这个文件中包含了该数据库的各种属性，比方说该数据库的字符集和比较规则是个啥。

比方说我们查看一下在我的计算机上当前有哪些数据库：

```
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| mysqladv |
| performance_schema |
| sys |
+-----+
5 rows in set (0.00 sec)
```

可以看到在当前有 5 个数据库，其中 `mysqladv` 数据库是我们自定义的，其余 4 个数据库是属于 MySQL 自带的系统数据库。我们再看一下数据目录下的内容：

```
[mysql@iZwz9j203ithc4guluwb2wZ data]$ pwd
/home/mysql/mysql57/data
[mysql@iZwz9j203ithc4guluwb2wZ data]$ ls
auto.cnf  ca.pem  client-key.pem  ibdata1  ib_logfil
ca-key.pem  client-cert.pem  ib_buffer_pool  ib_logfile0  ibtmp1
[mysql@iZwz9j203ithc4guluwb2wZ data]$ ls -l
total 122944
-rw-r----- 1 mysql mysql      56 Mar 19 16:19 auto.cnf
-rw-r----- 1 mysql mysql    1680 Mar 19 16:19 ca-key.pem
-rw-r--r-- 1 mysql mysql    1112 Mar 19 16:19 ca.pem
-rw-r--r-- 1 mysql mysql    1112 Mar 19 16:19 client-cert.pem
-rw-r----- 1 mysql mysql    1680 Mar 19 16:19 client-key.pem
-rw-r----- 1 mysql mysql     431 Mar 19 16:19 ib_buffer_pool
-rw-r----- 1 mysql mysql 12582912 Mar 19 16:20 ibdata1
-rw-r----- 1 mysql mysql 50331648 Mar 19 16:20 ib_logfile0
-rw-r----- 1 mysql mysql 50331648 Mar 19 16:19 ib_logfile1
-rw-r----- 1 mysql mysql 12582912 Mar 19 16:20 ibtmp1
drwxr-x--- 2 mysql mysql    4096 Mar 19 16:19 mysql
drwxr-x--- 2 mysql mysql    4096 Mar 20 17:07 mysqladv
drwxr-x--- 2 mysql mysql    4096 Mar 19 16:19 performance_schema
-rw-r----- 1 mysql mysql    1680 Mar 19 16:19 private_key.pem
-rw-r--r-- 1 mysql mysql     452 Mar 19 16:19 public_key.pem
-rw-r--r-- 1 mysql mysql    1112 Mar 19 16:19 server-cert.pem
-rw-r----- 1 mysql mysql    1680 Mar 19 16:19 server-key.pem
drwxr-x--- 2 mysql mysql   12288 Mar 19 16:19 sys
```

当然这个数据目录下的文件和子目录比较多，但是如果仔细看的话，除了 `information_schema` 这个系统数据库外，其他的数据库在数据目录下都有对应的子目录。这个 `information_schema` 比较特殊，我们后面再讲它的作用。

表在文件系统中的表示

我们的数据其实都是以记录的形式插入到表中的，每个表的信息其实可以分为两种：

1. 表结构的定义
2. 表中的数据

表结构就是该表的名称是啥，表里边有多少列，每个列的数据类型是啥，有啥约束条件和索引，用的是啥字符集和比较规则各种信息，这些信息都体现在了我们的建表语句中了。为了保存这些信息，InnoDB 和 MyISAM 这两种存储引擎都在数据目录下对应的数据库子目录下创建了一个专门用于描述表结构的文件，文件名是这样：表名.frm

比方说我们在 `mysqladv` 数据库下创建一个名为 `test` 的表:

```
mysql> use mysqladv
Database changed
mysql> CREATE TABLE test (c1 INT);
Query OK, 0 rows affected (0.03 sec)
```

```
[mysql@iZwz9j203ithc4guluvb2wZ mysqladv]$ ls
db.opt test.frm test.ibd
```

那在数据库 `mysqladv` 对应的子目录下就会创建一个名为 `test.frm` 的用于描述表结构的文件。这个后缀名为 `.frm` 是以二进制格式存储的。

那表中的数据存到什么文件中了呢?在这个问题上,不同的存储引擎就产生了有所不同,下边我们分别看一下 `InnoDB` 和 `MyISAM` 是用什么文件来保存表中数据的。

InnoDB 是如何存储表数据的

`InnoDB` 的数据会放在一个表空间或者文件空间(英文名: `table space` 或者 `file space`)的概念,这个表空间是一个抽象的概念,它可以对应文件系统上一个或多个真实文件(不同表空间对应的文件数量可能不同)。每一个表空间可以被划分为很多很多很多个页,我们的表数据就存放在某个表空间下的某些页里。表空间有好几种类型。

系统表空间(system tablespace)

这个所谓的系统表空间可以对应文件系统上一个或多个实际的文件,默认情况下, `InnoDB` 会在数据目录下创建一个名为 `ibdata1`(在你的数据目录下找找看有木有)、大小为 12M 的文件,这个文件就是对应的系统表空间在文件系统上的表示。

```
[mysql@iZwz9j203ithc4guluvb2wZ data]$ ls -l
total 122944
-rw-r----- 1 mysql mysql      56 Mar 19 16:19 auto.cnf
-rw-r----- 1 mysql mysql    1680 Mar 19 16:19 ca-key.pem
-rw-r--r-- 1 mysql mysql    1112 Mar 19 16:19 ca.pem
-rw-r--r-- 1 mysql mysql    1112 Mar 19 16:19 client-cert.pem
-rw-r----- 1 mysql mysql    1680 Mar 19 16:19 client-key.pem
-rw-r----- 1 mysql mysql     431 Mar 19 16:19 ib_buffer_pool
-rw-r----- 1 mysql mysql 12582912 Mar 19 16:20 ibdata1
-rw-r----- 1 mysql mysql 50331648 Mar 19 16:20 ib_logfile0
-rw-r----- 1 mysql mysql 50331648 Mar 19 16:19 ib_logfile1
-rw-r----- 1 mysql mysql 12582912 Mar 19 16:20 ibtmp1
drwxr-x--- 2 mysql mysql    4096 Mar 19 16:19 mysql
drwxr-x--- 2 mysql mysql    4096 Mar 20 17:07 mysqladv
drwxr-x--- 2 mysql mysql    4096 Mar 19 16:19 performance_schema
```

这个文件是所谓的自扩展文件,也就是当不够用的时候它会自己增加文件大小,当然,如果你想让系统表空间对应文件系统上多个实际文件,或者仅仅觉得原来的 `ibdata1` 这个文件名难听,那可以在 `MySQL` 启动时配置对应的文件路径以及它们的大小,我们也可以把系统表空间对应的文件路径不配置到数据目录下,甚至可以配置到单独的磁盘分区上。

需要注意的一点是，在一个 MySQL 服务器中，系统表空间只有一份。从 MySQL5.5.7 到 MySQL5.6.6 之间的各个版本中，我们表中的数据都会被默认存储到这个系统表空间。

独立表空间(file-per-table tablespace)

在 MySQL5.6.6 以及之后的版本中，InnoDB 并不会默认的把各个表的数据存储到系统表空间中，而是为每一个表建立一个独立表空间，也就是说我们创建了多少个表，就有多少个独立表空间。使用独立表空间来存储表数据的话，会在该表所属数据库对应的子目录下创建一个表示该独立表空间的文件，文件名和表名相同，只不过添加了一个.ibd 的扩展名而已，所以完整的文件名称长这样:表名.ibd。

比方说假如我们使用了独立表空间去存储 xiaohaizi 数据库下的 test 表的话，那么在该表所在数据库对应的 xiaohaizi 目录下会为 test 表创建这两个文件：

test.frm 和 test.ibd

```
[mysql@izwz9j203ithc4guluvb2wZ mysqladv]$ ls -l
total 112
-rw-r----- 1 mysql mysql    65 Mar 20 17:07 db.opt
-rw-r----- 1 mysql mysql 8556 Mar 20 17:50 test.frm
-rw-r----- 1 mysql mysql 98304 Mar 20 17:50 test.ibd
```

其中 test.ibd 文件就用来存储 test 表中的数据和索引。当然我们也可以自己指定使用系统表空间还是独立表空间来存储数据，这个功能由启动参数

innodb_file_per_table 控制，比如说我们想刻意将表数据都存储到系统表空间时，可以在启动 MySQL 服务器的时候这样配置：

[server]

innodb_file_per_table=0

当 innodb_file_per_table 的值为 0 时，代表使用系统表空间；当 innodb_file_per_table 的值为 1 时，代表使用独立表空间。不过 innodb_file_per_table 参数只对新建的表起作用，对于已经分配了表空间的表并不起作用。

其他类型的表空间

随着 MySQL 的发展，除了上述两种老牌表空间之外，现在还新提出了一些不同类型的表空间，比如通用表空间(general tablespace)、undo 表空间(undotablespace)、临时表空间(temporary tablespace)等。

MyISAM 是如何存储表数据的

在 MyISAM 中的数据和索引是分开存放的。所以在文件系统中也是使用不同的文件来存储数据文件和索引文件。而且和 InnoDB 不同的是，MyISAM 并没有什么所谓的表空间一说，表数据都存放到对应的数据库子目录下。

```
mysql> CREATE TABLE test_myisam (c1 INT) ENGINE=MyISAM;
Query OK, 0 rows affected (0.01 sec)
```

test_myisam 表使用 MyISAM 存储引擎的话，那么在它所在数据库对应的 xieotaizi 目录下会为 myisam 表创建三个文件：


```
[mysql@iZwz9j203ithc4guluvb2wZ mysqladv]$ ls -l
total 128
-rw-r----- 1 mysql mysql    65 Mar 20 17:07 db.opt
-rw-r----- 1 mysql mysql  8556 Mar 20 17:50 test.frm
-rw-r----- 1 mysql mysql 98304 Mar 20 17:50 test.ibd
-rw-r----- 1 mysql mysql  8556 Mar 20 23:27 test_myisam.frm
-rw-r----- 1 mysql mysql    0 Mar 20 23:27 test_myisam.MYD
-rw-r----- 1 mysql mysql  1024 Mar 20 23:27 test_myisam.MYI
```

其中 test_myisam.MYD 代表表的数据文件，也就是我们插入的用户记录；test_myisam.MYI 代表表的索引文件，我们为表创建的索引都会放到这个文件中。

1.2.3.3. 日志文件

在服务器运行过程中，会产生各种各样的日志，比如常规的查询日志、错误日志、二进制日志、redo 日志、Undo 日志等等，日志文件记录了影响 MySQL 数据库的各种类型活动。

常见的日志文件有：错误日志（error log）、慢查询日志（slow query log）、查询日志（query log）、二进制文件（bin log）。

错误日志

错误日志文件对 MySQL 的启动、运行、关闭过程进行了记录。遇到问题时应该首先查看该文件以便定位问题。该文件不仅记录了所有的错误信息，也记录一些警告信息或正确的信息

用户可以通过下面命令来查看错误日志文件的位置：

show variables like 'log_error'\G;

```
mysql> show variables like 'log_error'\G
***** 1. row *****
Variable_name: log_error
Value: /home/mysql/mysql57/log/error.log
1 row in set (0.00 sec)
```

当 MySQL 不能正常启动时，第一个必须查找的文件应该就是错误日志文件，该文件记录了错误信息。

慢查询日志

慢查询日志可以帮助定位可能存在问题的 SQL 语句，从而进行 SQL 语句层面的优化，具体的使用我们以后再说。

```
[mysql@iZwz9j203ithc4guluvb2wZ data]$ ls
auto.cnf          client-key.pem    ib_logfile1      mysql            public_key.pem
ca-key.pem        ib_buffer_pool    ibtmp1           mysqladv         server-cert.pem
ca.pem            ibdata1           iZwz9j203ithc4guluvb2wZ.log  performance_schema  server-key.pem
client-cert.pem   ib_logfile0       iZwz9j203ithc4guluvb2wZ-slow.log  private_key.pem    sys
```

查询日志

查询日志记录了所有对 MySQL 数据库请求的信息，无论这些请求是否得到了正确的执行。

默认文件名：主机名.log

```
[mysql@iZwz9j203ithc4guluvb2wZ data]$ ls
auto.cnf          client-key.pem    ib_logfile1      mysql             public_key.pem
ca-key.pem        ib_buffer_pool   ibtmp1           mysqladv          server-cert.pem
ca.pem            ibdata1          iZwz9j203ithc4guluvb2wZ.log  performance_schema server-key.pem
client-cert.pem   ib_logfile0      iZwz9j203ithc4guluvb2wZ-slow.log private_key.pem  sys
```

从 MySQL 5.1 开始，可以将查询日志的记录放入 mysql 架构下的 `general_log` 表

```
[mysql@iZwz9j203ithc4guluvb2wZ data]$ tail -f iZwz9j203ithc4guluvb2wZ.log
2021-03-22T11:01:10.395195Z      32 Query      SELECT * FROM `mysqladv`.`order_exp` LIMIT 0,1000
2021-03-22T11:01:10.464485Z      63 Init DB    mysqladv
2021-03-22T11:01:10.487734Z      63 Query      SHOW COLUMNS FROM `mysqladv`.`order_exp`
2021-03-22T11:01:10.512007Z      32 Query      SHOW TABLE STATUS LIKE 'order_exp'
2021-03-22T11:01:10.529946Z      32 Query      SHOW CREATE TABLE `mysqladv`.`order_exp`
2021-03-22T11:01:14.457541Z      20 Query      select * from general_log
2021-03-22T11:01:18.848060Z      20 Query      select * from general_log
2021-03-22T11:01:21.599884Z      20 Query      select * from general_log
2021-03-22T11:01:46.035133Z      20 Query      show variables like 'general_log'
2021-03-22T11:02:10.313622Z      20 Query      set global log_output='TABLE'
```

二进制日志（binlog）

二进制日志记录了对 MySQL 数据库执行更改的所有操作，若操作本身没有导致数据库发生变化，该操作可能也会写入二进制文件。但是不包括 `select` 和 `show` 这类操作（因为这些操作对数据本身不会进行修改）

二进制日志的几种作用：

恢复（recovery）：某些数据的恢复需要二进制日志，例如，在一个数据库全备文件恢复后，用户可以通过二进制文件进行 `point-in-time` 的恢复

复制（replication）：其原理与恢复类似，通过复制和执行二进制日志使一台远程的 MySQL 数据库（一般称为 `slave` 或 `standby`）与一台 MySQL 数据库（一般称为 `master` 或 `primary`）进行实时同步

审计（audit）：用户可以通过二进制日志中的信息来进行审计，判断是否有对数据库进行注入的攻击

log-bin 参数

该参数用来控制是否开启二进制日志，默认为关闭

如果想要开启二进制日志的功能，可以在 MySQL 的配置文件中指定如下的格式：

“name”为二进制日志文件的名称

如果不提供 `name`，那么数据库会使用默认的日志文件名（文件名为主机名，后缀名为二进制日志的序列号），且文件保存在数据库所在的目录（`datadir` 下）

--启用/设置二进制日志文件(name 可省略)

`log-bin=name;`

配置以后，就会在数据目录下产生类似于：

`bin_log.00001` 即为二进制日志文件；`bin_log.index` 为二进制的索引文件，用来存储过往产生的二进制日志序号，通常情况下，不建议手动修改这个文件。

二进制日志文件在默认情况下并没有启动，需要手动指定参数来启动。开启这个选项会对 MySQL 的性能造成影响，但是性能损失十分有限。根据 MySQL 官方手册中的测试指明，开启二进制日志会使性能下降 1%。

1.2.3.3. 其他的数据文件

除了我们上边说的这些用户自己存储的数据以外，数据文件下还包括为了更好地运行程序的一些额外文件，当然这些文件不一定会放在数据目录下，而且可以在配置文件或者启动时另外指定存放目录。

主要包括这几种类型的文件：

- 服务器进程文件。

我们知道每运行一个 MySQL 服务器程序，都意味着启动一个进程。MySQL 服务器会把自己的进程 ID 写入到一个 pid 文件中。

socket 文件

当用 UNIX 域套接字方式进行连接时需要的文件。

```
[mysql@iZwz9j203ithc4guluvb2wZ log]$ ls -l
total 80
-rw-r----- 1 mysql mysql 68593 Apr  5 10:52 error.log
-rw-r----- 1 mysql mysql    6 Apr  5 10:52 mysql.pid
srwxrwxrwx 1 mysql mysql    0 Apr  5 10:52 mysql.sock
-rw-r----- 1 mysql mysql    6 Apr  5 10:52 mysql.sock.lock
[mysql@iZwz9j203ithc4guluvb2wZ log]$ pwd
/home/mysql/mysql57/log
[mysql@iZwz9j203ithc4guluvb2wZ log]$
```

- 默认/自动生成的 SSL 和 RSA 证书和密钥文件。

```
-rw----- 1 mysql mysql 1680 Mar 19 16:19 ca-key.pem
-rw-r--r-- 1 mysql mysql 1112 Mar 19 16:19 ca.pem
-rw-r--r-- 1 mysql mysql 1112 Mar 19 16:19 client-cert.pem
-rw----- 1 mysql mysql 1680 Mar 19 16:19 client-key.pem
```

```
-rw----- 1 mysql mysql 1680 Mar 19 16:19 private_key.pem
-rw-r--r-- 1 mysql mysql  452 Mar 19 16:19 public_key.pem
-rw-r--r-- 1 mysql mysql 1112 Mar 19 16:19 server-cert.pem
-rw----- 1 mysql mysql 1680 Mar 19 16:19 server-key.pem
```

1.3. MySQL 中的系统库

1.3.1. 系统库简介

以下系统库的介绍，主要是扩展大家的知识面，因为系统库中的数据很重要，没有对 MySQL 有全面通盘的了解，不宜去操作其下的数据，而且对系统库的管理维护是 DBA 的职责。

MySQL 有几个系统数据库，这几个数据库包含了 MySQL 服务器运行过程中所需的一些信息以及一些运行状态信息，我们现在稍微了解一下。

performance_schema

这个数据库里主要保存 MySQL 服务器运行过程中的一些状态信息，算是对 MySQL 服务器的一个性能监控。包括统计最近执行了哪些语句，在执行过程的每个阶段都花费了多长时间，内存的使用情况等等信息。

sys

这个数据库主要是通过视图的形式把 `information_schema` 和 `performance_schema` 结合起来，让程序员可以更方便的了解 MySQL 服务器的一些性能信息。

`information_schema`

这个数据库保存着 MySQL 服务器维护的所有其他数据库的信息，比如有哪些表、哪些视图、哪些触发器、哪些列、哪些索引。这些信息并不是真实的用户数据，而是一些描述性信息，有时候也称之为元数据。

`mysql`

这个数据库核心，它存储了 MySQL 的用户账户和权限信息，一些存储过程、事件的定义信息，一些运行过程中产生的日志信息，一些帮助信息以及时区信息等。

1.3.2. `performance_schema`

1.3.2.1. 什么是 `performance_schema`

MySQL 的 `performance_schema` 是运行在较低级别的用于监控 MySQL Server 运行过程中的资源消耗、资源等待等情况的一个功能特性，它具有以下特点。

- `performance_schema` 提供了一种在数据库运行时实时检查 Server 内部执行情况的方法。`performance_schema` 数据库中的表使用 `performance_schema` 存储引擎。该数据库主要关注数据库运行过程中的性能相关数据。
- `performance_schema` 通过监视 Server 的事件来实现监视其内部执行情况，“事件”就是在 Server 内部活动中所做的任何事情以及对应的时间消耗，利用这些信息来判断 Server 中的相关资源被消耗在哪里。一般来说，事件可以是函数调用、操作系统的等待、SQL 语句执行的阶段[如 SQL 语句执行过程中的 `parsing`（解析）或 `sorting`（排序）阶段]或者整个 SQL 语句的集合。采集事件可以方便地提供 Server 中的相关存储引擎对磁盘文件、表 I/O、表锁等资源的同步调用信息。
- 当前活跃事件、历史事件和事件摘要相关表中记录的信息，能提供某个事件的执行次数、使用时长，进而可用于分析与某个特定线程、特定对象（如 `mutex` 或 `file`）相关联的活动。
- `performance_schema` 存储引擎使用 Server 源代码中的“检测点”来实现事件数据的收集。对于 `performance_schema` 实现机制本身的代码没有相关的单独线程来检测，这与其他功能（如复制或事件计划程序）不同。

收集到的事件数据被存储在 `performance_schema` 数据库的表中。对于这些表可以使用 `SELECT` 语句查询，也可以使用 SQL 语句更新 `performance_schema` 数据库中的表记录（比如动态修改 `performance_schema` 的以“`setup_`”开头的配置表，但要注意，配置表的更改会立即生效，这会影响数据收集）。

- `performance_schema` 的表中数据不会持久化存储在磁盘中，而是保存在内存中，一旦服务器重启，这些数据就会丢失（包括配置表在内的整个 `performance_schema` 下的所有数据）。

1.3.2.2. performance_schema 使用

通过上面介绍，相信你对于什么是 performance_schema 这个问题了解得更清晰了。下面开始介绍 performance_schema 的使用。

检查当前数据库版本是否支持

performance_schema 被视为存储引擎，如果该引擎可用，则应该在

INFORMATION_SCHEMA.ENGINES 表或 show engines 语句的输出中可以看到它的 Support 字段值为 YES，如下所示。

```
mysql> show engines;
+-----+-----+
| Engine          | Support |
+-----+-----+
| InnoDB          | DEFAULT |
| MRG_MYISAM      | YES     |
| MEMORY          | YES     |
| BLACKHOLE       | YES     |
| MyISAM          | YES     |
| CSV             | YES     |
| ARCHIVE         | YES     |
| PERFORMANCE_SCHEMA | YES     |
| FEDERATED       | NO      |
+-----+-----+
```

当我们看到 performance_schema 对应的 Support 字段值为 YES 时，就表示当前的数据库版本是支持 performance_schema 的。但确认了数据库实例支持 performance_schema 存储引擎就可以使用了吗？NO，很遗憾，performance_schema 在 MySQL 5.6 及之前的版本中默认没有启用，在 MySQL 5.7 及之后的版本中才修改为默认启用。

启用 performance_schema

如果要显式启用或关闭 performance_schema，则需要使用参数 performance_schema=ON/OFF 来设置，并在 my.cnf 中进行配置。注意：该参数为只读参数，需要在实例启动之前设置才生效

mysqld 启动之后，通过如下语句查看 performance_schema 启用是否生效（值为 ON 表示 performance_schema 已初始化成功且可以使用了；值为 OFF 表示在启用 performance_schema 时发生某些错误，可以查看错误日志进行排查）。

```
mysql> show variables like 'performance_schema';
+-----+-----+
| Variable_name      | Value |
+-----+-----+
| performance_schema | ON    |
+-----+-----+
```

现在，可以通过查询 INFORMATION_SCHEMA.TABLES 表中与 performance_schema 存储引擎相关的元数据，或者在 performance_schema 库下使用 show tables 语句来了解其存在哪些表。

使用 show tables 语句来查询有哪些 performance_schema 引擎表。

```
mysql> use performance_schema ;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
+-----+
| Tables_in_performance_schema |
+-----+
| accounts                     |
| cond_instances               |
| events_stages_current        |
| events_stages_history        |
| events_stages_history_long   |
| events_stages_summary_by_account_by_event_name |
| table_io_waits_summary_by_index_usage |
| table_io_waits_summary_by_table |
| table_lock_waits_summary_by_table |
| threads                      |
| user_variables_by_thread     |
| users                        |
| variables_by_thread          |
+-----+
87 rows in set (0.00 sec)
```

现在，我们知道了在当前版本中，performance_schema 库下一共有 87 个表，那么这些表都用于存放什么数据呢？我们如何使用它们来查询数据呢？先来看看这些表是如何分类的。

performance_schema 表的分类

performance_schema 库下的表可以按照监视的不同维度进行分组，例如：按照不同的数据库对象进行分组、按照不同的事件类型进行分组，或者按照事件类型分组之后，再进一步按照账号、主机、程序、线程、用户等进行细分。

下面介绍按照事件类型分组记录性能事件数据的表。

- 语句事件记录表：记录语句事件信息的表，包括：
events_statements_current（当前语句事件表）、events_statements_history（历史语句事件表）、events_statements_history_long（长语句历史事件表）以及一些 summary 表（聚合后的摘要表）。其中，summary 表还可以根据账号（account）、主机（host）、程序（program）、线程（thread）、用户（user）和全局（global）再进行细分。

show tables like 'events_statement%';

```
mysql> show tables like 'events_statement%';
+-----+
| Tables_in_performance_schema (events_statement%) |
+-----+
| events_statements_current                        |
| events_statements_history                       |
| events_statements_history_long                  |
| events_statements_summary_by_account_by_event_name |
| events_statements_summary_by_digest             |
| events_statements_summary_by_host_by_event_name |
| events_statements_summary_by_program            |
| events_statements_summary_by_thread_by_event_name |
| events_statements_summary_by_user_by_event_name |
| events_statements_summary_global_by_event_name   |
+-----+
10 rows in set (0.00 sec)
```

- 等待事件记录表：与语句事件记录表类似。

show tables like 'events_wait%';

```
mysql> show tables like 'events_wait%';
+-----+
| Tables_in_performance_schema (events_wait%) |
+-----+
| events_waits_current                          |
| events_waits_history                         |
| events_waits_history_long                    |
| events_waits_summary_by_account_by_event_name |
| events_waits_summary_by_host_by_event_name   |
| events_waits_summary_by_instance             |
| events_waits_summary_by_thread_by_event_name |
| events_waits_summary_by_user_by_event_name   |
| events_waits_summary_global_by_event_name    |
+-----+
9 rows in set (0.00 sec)
```

- 阶段事件记录表：记录语句执行阶段事件的表，与语句事件记录表类似。

show tables like 'events_stage%';

```
mysql> show tables like 'events_stage%';
+-----+
| Tables_in_performance_schema (events_stage%) |
+-----+
| events_stages_current                        |
| events_stages_history                       |
| events_stages_history_long                  |
| events_stages_summary_by_account_by_event_name |
| events_stages_summary_by_host_by_event_name  |
| events_stages_summary_by_thread_by_event_name |
| events_stages_summary_by_user_by_event_name  |
| events_stages_summary_global_by_event_name   |
+-----+
8 rows in set (0.01 sec)
```

- 事务事件记录表：记录与事务相关的事件的表，与语句事件记录表类似。

show tables like 'events_transaction%';

- 监视文件系统层调用的表：

show tables like '%file%';

- 监视内存使用的表：

show tables like '%memory%';

- 动态对 performance_schema 进行配置的配置表：

show tables like '%setup%';

现在，我们已经大概知道了 performance_schema 中主要表的分类，但如何使用这些表来提供性能事件数据呢？

performance_schema 简单配置与使用

当数据库初始化完成并启动时，并非所有的 instruments（在采集配置项的配置表中，每一项都有一个开关字段，或为 YES，或为 NO）和 consumers（与采集配置项类似，也有一个对应的事件类型保存表配置项，为 YES 表示对应的表保存性能数据，为 NO 表示对应的表不保存性能数据）都启用了，所以默认不会收集所有的事件。

可能你想检测的事件并没有打开，需要进行设置。可以使用如下两条语句打开对应的 instruments 和 consumers，我们以配置监测等待事件数据为例进行说明。

打开等待事件的采集器配置项开关，需要修改 setup_instruments 配置表中对应的采集器配置项。

update setup_instruments set enabled='yes',timed='yes' where name like 'wait%';

```
mysql> update setup_instruments set enabled='yes',timed='yes' where name like 'wait%';
Query OK, 267 rows affected (0.01 sec)
Rows matched: 321  Changed: 267  Warnings: 0
```


打开等待事件的保存表配置项开关，修改 setup_consumers 配置表中对应的配置项。

update setup_consumers set enabled='yes' where name like 'wait%';

```
mysql> update setup_consumers set enabled='yes' where name like 'wait%';
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0
```

配置好之后，我们就可以查看 Server 当前正在做什么了。可以通过查询 events_waits_current 表来得知，该表中每个线程只包含一行数据，用于显示每个线程的最新监视事件（正在做的事情）。

```
mysql> select * from events_waits_current\G
Empty set (0.00 sec)
```

*_current 表中每个线程只保留一条记录，且一旦线程完成工作，该表中就不会再记录该线程的事件信息了。*_history 表中记录每个线程已经执行完成的事件信息，但每个线程的事件信息只记录 10 条，再多就会被覆盖掉。*_history_long 表中记录所有线程的事件信息，但总记录数量是 10000 行，超过会被覆盖掉。

summary 表提供所有事件的汇总信息。该组中的表以不同的方式汇总事件数据（如：按用户、按主机、按线程等汇总）。

查看最近执行失败的 SQL 语句

使用代码对数据库的某些操作（比如：使用 Java 的 ORM 框架操作数据库）报出语法错误，但是代码并没有记录 SQL 语句文本的功能，在 MySQL 数据库层能否查看到具体的 SQL 语句文本，看看是否哪里写错了？这个时候，大多数人首先想到的就是去查看错误日志。很遗憾，对于 SQL 语句的语法错误，错误日志并不会记录。

实际上，在 performance_schema 的语句事件记录表中针对每一条语句的执行状态都记录了较为详细的信息，例如：events_statements_表和 events_statements_summary_by_digest 表（events_statements_表记录了语句所有的执行错误信息，而 events_statements_summary_by_digest 表只记录了语句在执行过程中发生错误的语句记录统计信息，不记录具体的错误类型，例如：不记录语法错误类的信息）。下面看看如何使用这两个表查询语句发生错误的语句信息。

首先，我们模拟一条语法错误的 SQL 语句，使用 events_statements_history_long 表或者 events_statements_history 表查询发生语法错误的 SQL 语句：

```
mysql> select * from;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your
```

然后，查询 events_statements_history 表中错误号为 1064 的记录

select * from events_statements_history where mysql_errno=1064\G

```
mysql> select * from events_statements_history where mysql_errno=1064\G
***** 1. row *****
      THREAD_ID: 45
      EVENT_ID: 168
    END_EVENT_ID: 168
      EVENT_NAME: statement/sql/error
        SOURCE:
      TIMER_START: 250860217657115000
      TIMER_END: 250860217726053000
      TIMER_WAIT: 68938000
      LOCK_TIME: 0
      SQL_TEXT: select * from
        DIGEST: NULL
      DIGEST_TEXT: NULL
    CURRENT_SCHEMA: performance_schema
      OBJECT_TYPE: NULL
      OBJECT_SCHEMA: NULL
      OBJECT_NAME: NULL
    OBJECT_INSTANCE_BEGIN: NULL
      MYSQL_ERRNO: 1064
    RETURNED_SQLSTATE: 42000
      MESSAGE_TEXT: You have an error in your SQL syntax; check the ma
        ERRORS: 1
      WARNINGS: 0
```

不知道错误号是多少，可以查询发生错误次数不为 0 的语句记录，在里边找到 SQL_TEXT 和 MESSAGE_TEXT 字段（提示信息为语法错误的就是它）。

查看最近的事务执行信息

我们可以通过慢查询日志查询到一条语句的执行总时长，但是如果数据库中存在一些大事务在执行过程中回滚了，或者在执行过程中异常中止，这个时候慢查询日志就爱莫能助了，这时我们可以借助 performance_schema 的 events_transactions_* 表来查看与事务相关的记录，在这些表中详细记录了是否有事务被回滚、活跃（长时间未提交的事务也属于活跃事务）或已提交等信息。

首先需要进行配置启用，事务事件默认并未启用

update setup_instruments set enabled='yes',timed='yes' where name like 'transaction%';

update setup_consumers set enabled='yes' where name like '%transaction%';

```
mysql> update setup_consumers set enabled='yes' where name like 'transaction%';
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0

mysql> update setup_consumers set enabled='yes' where name like '%transaction%';
Query OK, 3 rows affected (0.00 sec)
Rows matched: 3  Changed: 3  Warnings: 0
```

开启一个新会话（会话 2）用于执行事务，并模拟事务回滚。

```
mysql> begin
-> ;
Query OK, 0 rows affected (0.00 sec)

mysql> insert test values(100);
Query OK, 1 row affected (0.00 sec)

mysql> █
```

查询活跃事务，活跃事务表示当前正在执行的事务事件，需要从 events_transactions_current 表中查询。

```
mysql> select * from events_transactions_current\G
***** 1. row *****
      THREAD_ID: 49
      EVENT_ID: 15
      END_EVENT_ID: NULL
      EVENT_NAME: transaction
      STATE: ACTIVE
      TRX_ID: NULL
      GTID: AUTOMATIC
      XID_FORMAT_ID: NULL
      XID_GTRID: NULL
      XID_BQUAL: NULL
      XA_STATE: NULL
      SOURCE:
      TIMER_START: 252805882964220000
      TIMER_END: 252825964355824000
      TIMER_WAIT: 20081391604000
      ACCESS_MODE: READ WRITE
      ISOLATION_LEVEL: REPEATABLE READ
      AUTOCOMMIT: NO
      NUMBER_OF_SAVEPOINTS: 0
      NUMBER_OF_ROLLBACK_TO_SAVEPOINT: 0
      NUMBER_OF_RELEASE_SAVEPOINT: 0
      OBJECT_INSTANCE_BEGIN: NULL
      NESTING_EVENT_ID: 14
      NESTING_EVENT_TYPE: STATEMENT
1 row in set (0.00 sec)
```

会话 2 中回滚事务：

```
mysql> begin
-> ;
Query OK, 0 rows affected (0.00 sec)

mysql> insert test values(100);
Query OK, 1 row affected (0.00 sec)

mysql> rollback;
Query OK, 0 rows affected (0.00 sec)
```

查询事务事件当前表和事务事件历史记录表，可以看到在两表中都记录了一行事务事件信息，线程ID为49的线程执行了一个事务，事务状态为ROLLED BACK。

```
mysql> select * from events_transactions_current\G
***** 1. row *****
      THREAD_ID: 49
      EVENT_ID: 15
      END_EVENT_ID: 17
      EVENT_NAME: transaction
      STATE: ROLLED BACK
      TRX_ID: NULL
      GTID: AUTOMATIC
      XID_FORMAT_ID: NULL
      XID_GTRID: NULL
      XID_BQUAL: NULL
      XA_STATE: NULL
      SOURCE:
      TIMER_START: 252805882964220000
      TIMER_END: 252917927250422000
      TIMER_WAIT: 112044286202000
      ACCESS_MODE: READ WRITE
      ISOLATION_LEVEL: REPEATABLE READ
      AUTOCOMMIT: NO
      NUMBER_OF_SAVEPOINTS: 0
      NUMBER_OF_ROLLBACK_TO_SAVEPOINT: 0
      NUMBER_OF_RELEASE_SAVEPOINT: 0
      OBJECT_INSTANCE_BEGIN: NULL
      NESTING_EVENT_ID: 14
      NESTING_EVENT_TYPE: STATEMENT
1 row in set (0.00 sec)

mysql> select * from events_transactions_history_long\G
***** 1. row *****
      THREAD_ID: 49
      EVENT_ID: 15
      END_EVENT_ID: 17
      EVENT_NAME: transaction
      STATE: ROLLED BACK
      TRX_ID: NULL
      GTID: AUTOMATIC
      XID_FORMAT_ID: NULL
      XID_GTRID: NULL
      XID_BQUAL: NULL
      XA_STATE: NULL
      SOURCE:
      TIMER_START: 252805882964220000
      TIMER_END: 252917927250422000
      TIMER_WAIT: 112044286202000
      ACCESS_MODE: READ WRITE
      ISOLATION_LEVEL: REPEATABLE READ
      AUTOCOMMIT: NO
      NUMBER_OF_SAVEPOINTS: 0
      NUMBER_OF_ROLLBACK_TO_SAVEPOINT: 0
      NUMBER_OF_RELEASE_SAVEPOINT: 0
      OBJECT_INSTANCE_BEGIN: NULL
      NESTING_EVENT_ID: 14
      NESTING_EVENT_TYPE: STATEMENT
1 row in set (0.00 sec)
```

但是当我们关闭会话 2 以后，事务事件当前表中的记录就消失了。

```
mysql> select * from events_transactions_history_long\G
***** 1. row *****
      THREAD_ID: 49
      EVENT_ID: 15
      END_EVENT_ID: 17
      EVENT_NAME: transaction
      STATE: ROLLED BACK
      TRX_ID: NULL
      GTID: AUTOMATIC
      XID_FORMAT_ID: NULL
      XID_GTRID: NULL
      XID_BQUAL: NULL
      XA_STATE: NULL
      SOURCE:
      TIMER_START: 252805882964220000
      TIMER_END: 252917927250422000
      TIMER_WAIT: 112044286202000
      ACCESS_MODE: READ WRITE
      ISOLATION_LEVEL: REPEATABLE READ
      AUTOCOMMIT: NO
      NUMBER_OF_SAVEPOINTS: 0
      NUMBER_OF_ROLLBACK_TO_SAVEPOINT: 0
      NUMBER_OF_RELEASE_SAVEPOINT: 0
      OBJECT_INSTANCE_BEGIN: NULL
      NESTING_EVENT_ID: 14
      NESTING_EVENT_TYPE: STATEMENT
1 row in set (0.00 sec)

mysql> select * from events_transactions_current\G
Empty set (0.00 sec)
```

小结

当然 `performance_schema` 的用途不止我们上面说到过的这些，它还能提供比如查看 SQL 语句执行阶段和进度信息、MySQL 集群下复制功能查看复制报错详情等等，更多的功能请大家自行挖掘，并欢迎大家以文章的形式发表在大家共有的 VIP 论坛 <http://bbs.xiangxueketang.cn/article/article> 上。

1.3.3. sys 系统库

1.3.3.1. sys 使用须知

`sys` 系统库通常都是提供给专业的 DBA 人员排查一些特定问题使用的，其下所涉及的各项查询或多或少都会对性能有一定的影响。所以这个库的配置、触发器等等我们都不做学习。

同时在使用上：

`sys` 系统库支持 MySQL 5.6 或更高版本，不支持 MySQL 5.5.x 及以下版本。

因为 `sys` 系统库提供了一些代替直接访问 `performance_schema` 的视图，所以必须启用 `performance_schema`（将 `performance_schema` 系统参数设置为 ON），`sys` 系统库的大部分功能才能正常使用。

同时要完全访问 `sys` 系统库，用户必须具有以下数据库的管理员权限。

如果要充分使用 `sys` 系统库的功能，则必须启用某些 `performance_schema` 的功能。当然 `sys` 系统库本身已经提供了启用所有需要的功能的存储过程，比如：

启用所有的 wait instruments：

```
CALL sys.ps_setup_enable_instrument('wait');
```

启用所有事件类型的 current 表：

```
CALL sys.ps_setup_enable_consumer('current');
```

注意：performance_schema 的默认配置就可以满足 sys 系统库的大部分数据收集功能。启用所有需要功能会对性能产生一定的影响，因此最好仅启用所需的配置。

1.3.3.2. sys 系统库使用

如果使用了 USE 语句切换默认数据库，那么就可以直接使用 `sys` 系统库下的视图进行查询，就像查询某个库下的表一样操作。也可以使用 `b_name.view_name`、`db_name.procedure_name`、`db_name.func_name` 等方式，在不指定默认数据库的情况下访问 `sys` 系统库中的对象（这叫作名称限定对象引用）。

```
mysql> use sys;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> select * from version;
+-----+-----+
| sys_version | mysql_version |
+-----+-----+
| 1.5.2       | 5.7.32       |
+-----+-----+
1 row in set (0.00 sec)

mysql> select * from sys.version;
+-----+-----+
| sys_version | mysql_version |
+-----+-----+
| 1.5.2       | 5.7.32       |
+-----+-----+
1 row in set (0.00 sec)
```

在 `sys` 系统库下包含很多视图，它们以各种方式对 `performance_schema` 表进行聚合计算展示。这些视图大部分是成对出现的，两个视图名称相同，但有一个视图是带 “`x$`” 前缀的，例如：`host_summary_by_file_io` 和 `x$host_summary_by_file_io`，代表按照主机进行汇总统计的文件 I/O 性能数据，两个视图访问的数据源是相同的，但是在创建视图的语句中，不带 “`x$`” 前缀的视图显示的是相关数值经过单位换算后的数据（单位是毫秒、秒、分钟、小时、天等），带 “`x$`” 前缀的视图显示的是原始的数据（单位是皮秒）。

```
mysql> show tables like '%session';
+-----+
| Tables_in_sys (%session) |
+-----+
| session                  |
| x$session                |
+-----+
2 rows in set (0.00 sec)
```

查看慢 SQL 语句慢在哪里

如果我们频繁地在慢查询日志中发现某个语句执行缓慢，且在表结构、索引结构、统计信息中都无法找出原因时，则可以利用 `sys` 系统库中的撒手锏：`sys.session` 视图结合 `performance_schema` 的等待事件来找出症结所在。那么 `session` 视图有什么用呢？使用它可以查看当前用户会话的进程列表信息，看看当前进程到底再干什么，注意，这个视图在 MySQL 5.7.9 中才出现。

首先需要启用与等待事件相关功能：

```
mysql> call sys.ps_setup_enable_instrument('wait');
+-----+
| summary |
+-----+
| Enabled 49 instruments |
+-----+
1 row in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> call sys.ps_setup_enable_consumer('wait');
+-----+
| summary |
+-----+
| Enabled 3 consumers |
+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

call sys.ps_setup_enable_instrument('wait');

call sys.ps_setup_enable_consumer('wait');

然后模拟一下:

```
mysql> select sleep(30);
```

```
mysql> select * from session where command='query' and conn_id!=connection_id()\G
***** 1. row *****
      thd_id: 52
      conn_id: 27
        user: root@localhost
          db: NULL
      command: Query
        state: User sleep
         time: 4
current_statement: select sleep(30)
statement_latency: 4.19 s
  progress: NULL
lock_latency: 0 ps
rows_examined: 0
   rows_sent: 0
rows_affected: 0
   tmp_tables: 0
tmp_disk_tables: 0
   full_scan: NO
last_statement: NULL
last_statement_latency: NULL
current_memory: 0 bytes
last_wait: wait/synch/cond/sql/Item_func_sleep::cond
last_wait_latency: Still Waiting
      source:
        trx_latency: NULL
        trx_state: NULL
    trx_autocommit: NULL
          pid: 27347
    program_name: mysql
1 row in set (0.05 sec)
```

查询表的增、删、改、查数据量和 I/O 耗时统计

select * from schema_table_statistics_with_buffer\G

```
mysql> select * from schema_table_statistics_with_buffer\G
***** 1. row *****
      table_schema: mysqladv
        table_name: test
      rows_fetched: 1
    fetch_latency: 31.70 us
    rows_inserted: 3
insert_latency: 170.84 us
   rows_updated: 0
update_latency: 0 ps
   rows_deleted: 0
delete_latency: 0 ps
io_read_requests: 17
      io_read: 986 bytes
io_read_latency: 48.51 us
io_write_requests: 32
      io_write: 228.47 KiB
io_write_latency: 332.43 us
io_misc_requests: 37
   io_misc_latency: 36.15 ms
innodb buffer allocated: 16.00 KiB
innodb buffer data: 29 bytes
innodb buffer free: 15.97 KiB
innodb_buffer_pages: 1
innodb_buffer_pages_hashed: 0
innodb_buffer_pages_old: 0
innodb_buffer_rows_cached: 1
***** 2. row *****
      table_schema: sys
        table_name: sys_config
      rows_fetched: 1
```

小结

除此之外，通过 `sys` 还可以查询查看 InnoDB 缓冲池中的热点数据、查看是否有事务锁等待、查看未使用的，冗余索引、查看哪些语句使用了全表扫描等等。

更多的功能请大家自行挖掘，并欢迎大家以文章的形式发表在大家共有的 VIP 论坛 <http://bbs.xiangxueketang.cn/article/article> 上。

1.3.4. information_schema

1.3.4.1. 什么是 information_schema

`information_schema` 提供了对数据库元数据、统计信息以及有关 MySQL Server 信息的访问（例如：数据库名或表名、字段的数据类型和访问权限等）。该库中保存的信息也可以称为 MySQL 的数据字典或系统目录。

在每个 MySQL 实例中都有一个独立的 `information_schema`，用来存储 MySQL 实例中所有其他数据库的基本信息。`information_schema` 库下包含多个只读表（非持久表），所以在磁盘中的数据目录下没有对应的关联文件，且不能对这些表设置触发器。虽然在查询时可以使用 `USE` 语句将默认数据库设置为 `information_schema`，但该库下的所有表是只读的，不能执行 `INSERT`、`UPDATE`、`DELETE` 等数据变更操作。

针对 `information_schema` 下的表的查询操作可以替代一些 `SHOW` 查询语句（例如：`SHOW DATABASES`、`SHOW TABLES` 等）。

注意：根据 MySQL 版本的不同，表的个数和存放是有所不同的。在 MySQL 5.6 版本中总共有 59 个表，在 MySQL 5.7 版本中，该 schema 下总共有 61 个表，

在 MySQL 8.0 版本中，该 schema 下的数据字典表（包含部分原 Memory 引擎临时表）都迁移到了 `mysql schema` 下，且在 `mysql schema` 下这些数据字典表被隐藏，无法直接访问，需要通过 `information_schema` 下的同名表进行访问。

`information_schema` 下的所有表使用的都是 Memory 和 InnoDB 存储引擎，且都是临时表，不是持久表，在数据库重启之后这些数据会丢失。在 MySQL 的 4 个系统库中，`information_schema` 也是唯一一个在文件系统上没有对应库表的目录和文件的系统库。

1.3.4.2. information_schema 表分类

Server 层的统计信息字典表

(1) COLUMNS

- 提供查询表中的列（字段）信息。

(2) KEY_COLUMN_USAGE

- 提供查询哪些索引列存在约束条件。
- 该表中的信息包含主键、唯一索引、外键等约束信息，例如：所在的库表列名、引用的库表列名等。该表中的信息与 `TABLE_CONSTRAINTS` 表中记录的信息有些类似，但 `TABLE_CONSTRAINTS` 表中没有记录约束引用的库表列信息，

而 KEY_COLUMN_USAGE 表中却记录了 TABLE_CONSTRAINTS 表中所没有的约束类型。

(3) REFERENTIAL_CONSTRAINTS

- 提供查询关于外键约束的一些信息。

(4) STATISTICS

- 提供查询关于索引的一些统计信息，一个索引对应一行记录。

(5) TABLE_CONSTRAINTS

- 提供查询与表相关的约束信息。

(6) FILES

- 提供查询与 MySQL 的数据表空间文件相关的信息。

(7) ENGINES

- 提供查询 MySQL Server 支持的引擎相关信息。

(8) TABLESPACES

- 提供查询关于活跃表空间的相关信息（主要记录的是 NDB 存储引擎的表空间信息）。

- 注意：该表不提供有关 InnoDB 存储引擎的表空间信息。对于 InnoDB 表空间的元数据信息，请查询 INNODB_SYS_TABLESPACES 表和 INNODB_SYS_DATAFILES 表。另外，从 MySQL 5.7.8 开始，INFORMATION_SCHEMA.FILES 表也提供查询 InnoDB 表空间的元数据信息。

(9) SCHEMATA

- 提供查询 MySQL Server 中的数据库列表信息，一个 schema 就代表一个数据库。

Server 层的表级别对象字典表

(1) VIEWS

- 提供查询数据库中的视图相关信息。查询该表的账户需要拥有 show view 权限。

(2) TRIGGERS

- 提供查询关于某个数据库下的触发器相关信息。

(3) TABLES

- 提供查询与数据库内的表相关的基本信息。

(4) ROUTINES

- 提供查询关于存储过程和存储函数的信息（不包括用户自定义函数）。该表中的信息与 mysql.proc 中记录的信息相对应（如果该表中有值的话）。

(5) PARTITIONS

- 提供查询关于分区表的信息。

(6) EVENTS

- 提供查询与计划任务事件相关的信息。

(7) PARAMETERS

- 提供有关存储过程和函数的参数信息，以及有关存储函数的返回值信息。这些参数信息与 `mysql.proc` 表中的 `param_list` 列记录的内容类似。

Server 层的混杂信息字典表

(1) GLOBAL_STATUS、GLOBAL_VARIABLES、SESSION_STATUS、

SESSION_VARIABLES

- 提供查询全局、会话级别的状态变量与系统变量信息。

(2) OPTIMIZER_TRACE

- 提供优化程序跟踪功能产生的信息。
- 跟踪功能默认是关闭的，使用 `optimizer_trace` 系统变量启用跟踪功能。如果开启该功能，则每个会话只能跟踪它自己执行的语句，不能看到其他会话执行的语句，且每个会话只能记录最后一条跟踪的 SQL 语句。

(3) PLUGINS

- 提供查询关于 MySQL Server 支持哪些插件的信息。

(4) PROCESSLIST

- 提供查询一些关于线程运行过程中的状态信息。

(5) PROFILING

- 提供查询关于语句性能分析的信息。其记录内容对应于 `SHOW PROFILES` 和 `SHOW PROFILE` 语句产生的信息。该表只有在会话变量 `profiling=1` 时才会记录语句性能分析信息，否则该表不记录。
- 注意：从 MySQL 5.7.2 开始，此表不再推荐使用，在未来的 MySQL 版本中删除，改用 Performance Schema 代替。

(6) CHARACTER_SETS

- 提供查询 MySQL Server 支持的可用字符集。

(7) COLLATIONS

- 提供查询 MySQL Server 支持的可用校对规则。

(8) COLLATION_CHARACTER_SET_APPLICABILITY

- 提供查询 MySQL Server 中哪种字符集适用于什么校对规则。查询结果集相当于从 `SHOW COLLATION` 获得的结果集的前两个字段值。目前并没有发现该表有太大的作用。

(9) COLUMN_PRIVILEGES

- 提供查询关于列（字段）的权限信息，表中的内容来自 `mysql.column_priv` 列权限表（需要针对一个表的列单独授权之后才会有内容）。

(10) SCHEMA_PRIVILEGES

- 提供查询关于库级别的权限信息，每种类型的库级别权限记录一行信息，该表中的信息来自 `mysql.db` 表。

(11) TABLE_PRIVILEGES

-
- 提供查询关于表级别的权限信息，该表中的内容来自 `mysql.tables_priv` 表。

(12) USER_PRIVILEGES

- 提供查询全局权限的信息，该表中的信息来自 `mysql.user` 表。

10.2.4 InnoDB 层的系统字典表

(1) INNODB_SYS_DATAFILES

- 提供查询 InnoDB 所有表空间类型文件的元数据（内部使用的表空间 ID 和表空间文件的路径信息），包括独立表空间、常规表空间、系统表空间、临时表空间和 undo 空间（如果开启了独立 undo 空间的话）。

- 该表中的信息等同于 InnoDB 数据字典内部 `SYS_DATAFILES` 表的信息。

(2) INNODB_SYS_VIRTUAL

- 提供查询有关 InnoDB 虚拟生成列和与之关联的列的元数据信息，等同于 InnoDB 数据字典内部 `SYS_VIRTUAL` 表的信息。该表中展示的行信息是与虚拟生成列相关联列的每个列的信息。

(3) INNODB_SYS_INDEXES

- 提供查询有关 InnoDB 索引的元数据信息，等同于 InnoDB 数据字典内部 `SYS_INDEXES` 表中的信息。

(4) INNODB_SYS_TABLES

- 提供查询有关 InnoDB 表的元数据信息，等同于 InnoDB 数据字典内部 `SYS_TABLES` 表的信息。

(5) INNODB_SYS_FIELDS

- 提供查询有关 InnoDB 索引键列（字段）的元数据信息，等同于 InnoDB 数据字典内部 `SYS_FIELDS` 表的信息。

(6) INNODB_SYS_TABLESPACES

- 提供查询有关 InnoDB 独立表空间和普通表空间的元数据信息（也包含了全文索引表空间），等同于 InnoDB 数据字典内部 `SYS_TABLESPACES` 表的信息。

(7) INNODB_SYS_FOREIGN_COLS

- 提供查询有关 InnoDB 外键列的状态信息，等同于 InnoDB 数据字典内部 `SYS_FOREIGN_COLS` 表的信息。

(8) INNODB_SYS_COLUMNS

- 提供查询有关 InnoDB 表列的元数据信息，等同于 InnoDB 数据字典内部 `SYS_COLUMNS` 表的信息。

(9) INNODB_SYS_FOREIGN

- 提供查询有关 InnoDB 外键的元数据信息，等同于 InnoDB 数据字典内部 `SYS_FOREIGN` 表的信息。

(10) INNODB_SYS_TABLESTATS

- 提供查询有关 InnoDB 表的较低级别的状态信息视图。MySQL 优化器会使用这些统计信息数据来计算并确定在查询 InnoDB 表时要使用哪个索引。这些

信息保存在内存中的数据结构中，与存储在磁盘上的数据无对应关系。在 InnoDB 内部也无对应的系统表。

InnoDB 层的锁、事务、统计信息字典表

(1) INNODB_LOCKS

- 提供查询 InnoDB 引擎中事务正在请求的且同时被其他事务阻塞的锁信息（即没有发生不同事务之间锁等待的锁信息，在这里是查看不到的。例如，当只有一个事务时，无法查看到该事务所加的锁信息）。该表中的内容可用于诊断高并发下的锁争用信息。

(2) INNODB_TRX

- 提供查询当前在 InnoDB 引擎中执行的每个事务（不包括只读事务）的信息，包括事务是否正在等待锁、事务什么时间点开始，以及事务正在执行的 SQL 语句文本信息等（如果有 SQL 语句的话）。

(3) INNODB_BUFFER_PAGE_LRU

- 提供查询缓冲池中的页面信息。与 INNODB_BUFFER_PAGE 表不同，INNODB_BUFFER_PAGE_LRU 表保存有关 InnoDB 缓冲池中的页如何进入 LRU 链表，以及在缓冲池不够用时确定需要从中逐出哪些页的信息。

(4) INNODB_LOCK_WAITS

- 提供查询 InnoDB 事务的锁等待信息。如果查询该表为空，则表示无锁等待信息；如果查询该表中有记录，则说明存在锁等待，表中的每一行记录表示一个锁等待关系。在一个锁等待关系中包含：一个等待锁（即，正在请求获得锁）的事务及其正在等待的锁等信息、一个持有锁（这里指的是发生锁等待事务正在请求的锁）的事务及其所持有的锁等信息。

(5) INNODB_TEMP_TABLE_INFO

- 提供查询有关在 InnoDB 实例中当前处于活动状态的用户（只对已建立连接的用户有效，断开的用户连接对应的临时表会被自动删除）创建的 InnoDB 临时表的信息。它不提供查询优化器使用的内部 InnoDB 临时表的信息。该表在首次查询时创建。

(6) INNODB_BUFFER_PAGE

- 提供查询关于缓冲池中的页相关信息。

(7) INNODB_METRICS

- 提供查询 InnoDB 更为详细的性能信息，是对 InnoDB 的 `performance_schema` 的补充。通过对该表的查询，可用于检查 InnoDB 的整体健康状况，也可用于诊断性能瓶颈、资源短缺和应用程序的问题等。

(8) INNODB_BUFFER_POOL_STATS

- 提供查询一些 InnoDB 缓冲池中的状态信息，该表中记录的信息与 `SHOW ENGINEINNODB STATUS` 语句输出的缓冲池统计部分信息类似。另外，InnoDB 缓冲池的一些状态变量也提供了部分相同的值。

InnoDB 层的全文索引字典表

(1) INNODB_FT_CONFIG

- (2) INNODB_FT_BEING_DELETED
- (3) INNODB_FT_DELETED
- (4) INNODB_FT_DEFAULT_STOPWORD
- (5) INNODB_FT_INDEX_TABLE

InnoDB 层的压缩相关字典表

- (1) INNODB_CMP 和 INNODB_CMP_RESET

• 这两个表中的数据包含了与压缩的 InnoDB 表页有关的操作状态信息。表中记录的数据为测量数据库中的 InnoDB 表压缩的有效性提供参考。

- (2) INNODB_CMP_PER_INDEX 和 INNODB_CMP_PER_INDEX_RESET

• 这两个表中记录了与 InnoDB 压缩表数据和索引相关的操作状态信息，对数据库、表、索引的每个组合使用不同的统计信息，以便为评估特定表的压缩性能和实用性提供参考数据。

- (3) INNODB_CMPMEM 和 INNODB_CMPMEM_RESET

• 这两个表中记录了 InnoDB 缓冲池中压缩页的状态信息，为测量数据库中 InnoDB 表压缩的有效性提供参考。

1.3.4.3. information_schema 应用

查看索引列的信息

INNODB_SYS_FIELDS 表提供查询有关 InnoDB 索引列（字段）的元数据信息，等同于 InnoDB 数据字典中 SYS_FIELDS 表的信息。

INNODB_SYS_INDEXES 表提供查询有关 InnoDB 索引的元数据信息，等同于 InnoDB 数据字典内部 SYS_INDEXES 表中的信息。

INNODB_SYS_TABLES 表提供查询有关 InnoDB 表的元数据信息，等同于 InnoDB 数据字典中 SYS_TABLES 表的信息。

假设需要查询 mysqladv 库下的 InnoDB 表 order_exp 的索引列名称、组成和索引列顺序等相关信息，

```
CREATE TABLE `order_exp` (
  `id` bigint(22) NOT NULL AUTO INCREMENT COMMENT '订单的主键',
  `order_no` varchar(50) CHARACTER SET utf8 COLLATE utf8_general_ci NOT NULL COMMENT '订单的编号',
  `order_note` varchar(100) CHARACTER SET utf8 COLLATE utf8_general_ci NOT NULL COMMENT '订单的说明',
  `insert_time` datetime(0) NOT NULL COMMENT '插入订单的时间',
  `expire_duration` bigint(22) NOT NULL COMMENT '订单的过期时长，单位秒',
  `expire_time` datetime(0) NOT NULL COMMENT '订单的过期时间',
  `order_status` smallint(6) NOT NULL DEFAULT 0 COMMENT
  '订单的状态，0：未支付；1：已支付；-1：已过期，关闭',
  PRIMARY KEY (`id`) USING BTREE,
  UNIQUE INDEX `idx_day_status`(`insert_time`, `order_status`) USING BTREE
) ENGINE = InnoDB AUTO_INCREMENT = 16 CHARACTER SET = utf8 COLLATE = utf8_general_ci ROW_FORMAT =
```

则可以使用如下 SQL 语句进行查询

```
select t.name as d_t_name,i.name as i_name ,i.type as i_type,i.N_FIELDS as
i_column_numbers,f.name as i_column_name,f.pos as i_position from
INNODB_SYS_TABLES as t join INNODB_SYS_INDEXES as i on
t.TABLE_ID=i.TABLE_ID left join INNODB_SYS_FIELDS as f on
i.INDEX_ID=f.INDEX_ID where t.name='mysqladv/order_exp';
```

```
mysql> select t.name as d t name,i.name as i name ,i.type as i type,i.N_FIELDS as i_column_numbers,f.name
as t join INNODB_SYS_INDEXES as i on t.TABLE_ID=i.TABLE_ID left join INNODB_SYS_FIELDS as f on i.INDEX_ID=f.INDEX_ID
+-----+-----+-----+-----+-----+-----+
| d t name | i name | i type | i_column_numbers | i_column_name | i_position |
+-----+-----+-----+-----+-----+-----+
| mysqladv/order_exp | PRIMARY | 3 | 1 | id | 0 |
| mysqladv/order_exp | idx_day_status | 2 | 2 | insert_time | 0 |
| mysqladv/order_exp | idx_day_status | 2 | 2 | order_status | 1 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

结果中的列都很好理解，唯一需要解释的是 `i_type(INNODB_SYS_INDEXES.type)`，它是表示索引类型的数字 ID，0 = 二级索引、1 = 集群索引、2 = 唯一索引、3 = 主键索引、32 = 全文索引、64 = 空间索引、128 = 包含虚拟生成列的二级索引。

1.3.5. Mysql 中 mysql 系统库

1.3.5.1. 权限系统表

因为权限管理是 DBA 的职责，所以对于这个部分的表，我们大概了解下即可。在 mysql 系统库中，MySQL 访问权限系统表，放在 mysql 库中，主要包含如下几个表。

- **user:** 包含用户账户、全局权限和其他非权限列表（安全配置字段和资源控制字段）。
- **db:** 数据库级别的权限表。该表中记录的权限信息代表用户是否可以使用这些权限来访问被授予访问的数据库下的所有对象（表或存储程序）。
- **tables_priv:** 表级别的权限表。
- **columns_priv:** 字段级别的权限表。
- **procs_priv:** 存储过程和函数权限表。
- **proxies_priv:** 代理用户权限表。

提示：

要更改权限表的内容，应该使用账号管理语句（如：**CREATE USER**、**GRANT**、**REVOKE** 等）来间接修改，不建议直接使用 **DML** 语句修改权限表。

1.3.5.2. 统计信息表

持久化统计功能是通过将内存中的统计数据存储到磁盘中，使其在数据库重启时可以快速重新读入这些统计信息而不用重新执行统计，从而使得查询优化器可以利用这些持久化的统计信息准确地选择执行计划（如果没有这些持久化的统计信息，那么数据库重启之后内存中的统计信息将会丢失，下一次访问到某库某表时，需要重新计算统计信息，并且重新计算可能会因为估算值的差异导致查询计划发生变更，从而导致查询性能发生变化）。

如何启用统计信息的持久化功能呢？当 `innodb_stats_persistent = ON` 时全局的开启统计信息的持久化功能，默认是开启的，

```
show variables like 'innodb_stats_persistent';
```

```
mysql> show variables like 'innodb_stats_persistent';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_stats_persistent | ON |
+-----+-----+
1 row in set (0.00 sec)
```

如果要单独关闭某个表的持久化统计功能，则可以通过 ALTER TABLE tbl_name STATS_PERSISTENT = 0 语句来修改。

innodb_table_stats

innodb_table_stats 表提供查询与表数据相关的统计信息。

select * from innodb_table_stats where table_name = 'order_exp'\G

```
mysql> select * from innodb_table_stats where table_name = 'order_exp'\G
***** 1. row *****
      database_name: mysqladv
      table_name: order_exp
      last_update: 2021-03-22 17:26:35
      n_rows: 0
      clustered_index_size: 1
sum_of_other_index_sizes: 1
1 row in set (0.00 sec)
```

database_name: 数据库名称。

- table_name: 表名、分区名或子分区名。
- last_update: 表示 InnoDB 上次更新统计信息行的时间。
- n_rows: 表中的估算数据记录行数。
- clustered_index_size: 主键索引的大小，以页为单位的估算数值。
- sum_of_other_index_sizes: 其他（非主键）索引的总大小，以页为单位的估算数值。

其中的估算数据记录行数，页为单位的估算数值等等，将在我们的课程中学习到。

innodb_index_stats

innodb_index_stats 表提供查询与索引相关的统计信息。

select * from innodb_index_stats where table_name = 'order_exp';

```
mysql> select * from innodb_index_stats where table_name = 'order_exp';
+-----+-----+-----+-----+-----+-----+-----+-----+
| database_name | table_name | index_name | last_update | stat_name | stat_value | sample_size | stat_description |
+-----+-----+-----+-----+-----+-----+-----+-----+
| mysqladv | order_exp | PRIMARY | 2021-03-22 18:44:01 | n_diff_pfx01 | 862 | 7 | id |
| mysqladv | order_exp | PRIMARY | 2021-03-22 18:44:01 | n_leaf_pages | 7 | NULL | Number of leaf pages in the index |
| mysqladv | order_exp | PRIMARY | 2021-03-22 18:44:01 | size | 8 | NULL | Number of pages in the index |
| mysqladv | order_exp | idx_order_no | 2021-03-22 18:44:01 | n_diff_pfx01 | 520 | 2 | order_no |
| mysqladv | order_exp | idx_order_no | 2021-03-22 18:44:01 | n_diff_pfx02 | 862 | 2 | order_no,id |
| mysqladv | order_exp | idx_order_no | 2021-03-22 18:44:01 | n_leaf_pages | 2 | NULL | Number of leaf pages in the index |
| mysqladv | order_exp | idx_order_no | 2021-03-22 18:44:01 | size | 3 | NULL | Number of pages in the index |
| mysqladv | order_exp | u_idx_day_status | 2021-03-22 18:44:01 | n_diff_pfx01 | 80 | 2 | insert_time |
| mysqladv | order_exp | u_idx_day_status | 2021-03-22 18:44:01 | size | 80 | 2 | insert_time,order_status |
| mysqladv | order_exp | u_idx_day_status | 2021-03-22 18:44:01 | n_diff_pfx03 | 862 | 2 | insert_time,order_status,expire_time |
| mysqladv | order_exp | u_idx_day_status | 2021-03-22 18:44:01 | n_leaf_pages | 2 | NULL | Number of leaf pages in the index |
| mysqladv | order_exp | u_idx_day_status | 2021-03-22 18:44:01 | size | 3 | NULL | Number of pages in the index |
12 rows in set (0.00 sec)
```

表字段含义如下。

- database_name: 数据库名称。

- `table_name`: 表名、分区表名、子分区表名。
 - `index_name`: 索引名称。
 - `last_update`: 表示 InnoDB 上次更新统计信息行的时间。
 - `stat_name`: 统计信息名称，其对应的统计信息值保存在 `stat_value` 字段中。
 - `stat_value`: 保存统计信息名称 `stat_name` 字段对应的统计信息值。
 - `sample_size`: `stat_value` 字段中提供的统计信息估计值的采样页数。
 - `stat_description`: 统计信息名称 `stat_name` 字段中指定的统计信息的说明。
- 从表的查询数据中可以看到:

- `stat_name` 字段一共有如下几个统计值。

■ `size`: 当 `stat_name` 字段为 `size` 值时, `stat_value` 字段值表示索引中的总页数数量。

■ `n_leaf_pages`: 当 `stat_name` 字段为 `n_leaf_pages` 值时, `stat_value` 字段值表示索引叶子页的数量。

■ `n_diff_pfxNN`: `NN` 代表数字 (例如 `01`、`02` 等)。当 `stat_name` 字段为 `n_diff_pfxNN` 值时, `stat_value` 字段值表示索引的 `first column` (即索引的最前索引列, 从索引定义顺序的第一个列开始) 列的唯一值数量。例如: 当 `NN` 为 `01` 时, `stat_value` 字段值就表示索引的第一个列的唯一值数量; 当 `NN` 为 `02` 时, `stat_value` 字段值就表示索引的第一个和第二个列组合的唯一值数量, 依此类推。此外, 在 `stat_name = n_diff_pfxNN` 的情况下, `stat_description` 字段显示一个以逗号分隔的计算索引统计信息字段的列表。

- 从 `index_name` 字段值为 `PRIMARY` 数据行的 `stat_description` 字段的描述信息 “`id`” 中可以看出, 主键索引的统计信息只包括创建主键索引时显式指定的列。

- 从 `index_name` 字段值为 `u_idx_day_status` 数据行的 `stat_description` 字段的描述信息 “`insert_time,order_status,expire_time`” 中可以看出, 唯一索引的统计信息只包括创建唯一索引时显式指定的列。

- 从 `index_name` 字段值为 `idx_order_no` 数据行的 `stat_description` 字段的描述信息 “`order_no,id`” 中可以看出, 普通索引 (非唯一的辅助索引) 的统计信息包括了显式定义的列和主键列。

注意, 上述的描述中出现的诸如叶子页, 索引的最前索引列等等, 以及这个统计表的具体作用, 将在我们后面的课程中学到, 现在不必深究。

1.3.5.3. 日志记录表

MySQL 的日志系统包含: 普通查询日志、慢查询日志、错误日志 (记录服务器启动时、运行中、停止时的错误信息)、二进制日志 (记录服务器运行过程中数据变更的逻辑日志)、中继日志 (记录从库 I/O 线程从主库获取的主库数据变更日志)、DDL 日志 (记录 DDL 语句执行时的元数据变更信息。在 MySQL 5.7 中只支持写入文件中, 在 MySQL 8.0 中支持写入 `innodb_ddl_log` 表中。在 MySQL 5.7 中, 只有普通查询日志、慢查询日志支持写入表中 (也支持写入文件中), 可以

通过 `log_output=TABLE` 设置保存到 `mysql.general_log` 表和 `mysql.slow_log` 表中，其他日志类型在 MySQL 5.7 中只支持写入文件中。

general_log

`general_log` 表提供查询普通 SQL 语句的执行记录信息，用于查看客户端到底在服务器上执行了什么 SQL 语句。

缺省不开启

show variables like 'general_log';

```
mysql> show variables like 'general_log';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| general_log   | OFF   |
+-----+-----+
```

开启

set global log_output='TABLE'; -- 'TABLE,FILE'表示同时输出到表和文件

set global general_log=on;

show variables like 'general_log';

```
mysql> set global log_output='TABLE';
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> set global general_log=on;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> show variables like 'general_log';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| general_log   | ON    |
+-----+-----+
1 row in set (0.00 sec)
```

任意执行一个查询后

select * from mysql.general_log\G

```
mysql> select * from general_log\G
***** 1. row *****
event_time: 2021-03-22 19:02:16.234447
user_host: sqlper[sqlper] @ [175.9.122.171]
thread_id: 32
server_id: 100
command_type: Init DB
argument: mysqladv
***** 2. row *****
event_time: 2021-03-22 19:02:16.253963
user_host: sqlper[sqlper] @ [175.9.122.171]
thread_id: 32
server_id: 100
command_type: Query
argument: SELECT * FROM `mysqladv`.`order_exp` LIMIT 0,1000
***** 3. row *****
event_time: 2021-03-22 19:02:16.323385
user_host: sqlper[sqlper] @ [175.9.122.171]
thread_id: 63
server_id: 100
command_type: Init DB
argument: mysqladv
***** 4. row *****
event_time: 2021-03-22 19:02:16.346691
user_host: sqlper[sqlper] @ [175.9.122.171]
thread_id: 63
server_id: 100
command_type: Query
argument: SHOW COLUMNS FROM `mysqladv`.`order_exp`
```

slow_log

slow_log 表提供查询执行时间超过 long_query_time 设置值的 SQL 语句、未使用索引的语句（需要开启参数 log_queries_not_using_indexes=ON）或者管理语句（需要开启参数 log_slow_admin_statements=ON）。

更详细的信息，我们在 SQL 优化的部分再了解。

1.3.5.4. 复制信息表

复制信息表在从库复制主库的数据期间，用于保存从主库转发到从库的 binlog（二进制日志）事件，记录有关 relay log（中继日志）当前状态和位置的信息。我们大致了解即可。

master.info 文件或者 mysql.slave_master_info 表：用于保存从库的 I/O 线程连接主库的连接状态、账号、IP 地址、端口、密码，以及 I/O 线程当前读取主库 binlog 的文件和位置信息（称为 I/O 线程信息日志）。在默认情况下，I/O 线程的连接信息和状态保存在 master.info 文件中（默认位置在 datadir 下，可以使用 master_info_file 参数指定 master.info 文件路径。注：在 MySQL 5.7.x 较新的版本以及 8.0.x 版本中该参数已经被移除）。如果需要保存在 mysql.slave_master_info 表中，则需要在服务器启动之前设置 master_info_repository=TABLE。

- relay_log.info 文件或者 mysql.slave_relay_log_info 表：当从库的 I/O 线程从主库获取到最新的 binlog 事件信息后会先写入从库本地的 relay log 中，然后 SQL 线程再去读取 relay log 解析并重放。relay_log.info 文件或者 mysql.slave_relay_log_info 表就是用于记录最新的 relay log 的文件和位置，以及

SQL 线程当前重放的事件对应的主库 binlog 的文件和位置信息的（SQL 线程位置被称为 SQL 线程信息日志）。在默认情况下，relay log 的位置信息和 SQL 线程的位置信息保存在 relay-log.info 文件中（默认位置在 datadir 下，可以使用 relay_log_info_file 选项指定 relay-log.info 文件路径）。如果需要保存在 mysql.slave_relay_log_info 表中，则需要在服务器启动之前设置 relay_log_info_repository=TABLE。

1.4. 事务和事务的隔离级别

1.4.1. 为什么需要事务

事务是数据库管理系统（DBMS）执行过程中的一个逻辑单位（不可再进行分割），由一个有限的数据库操作序列构成（多个 DML 语句，select 语句不包含事务），要不全部成功，要不全部不成功。

A 给 B 要划钱，A 的账户-1000 元，B 的账户就要+1000 元，这两个 update 语句必须作为一个整体来执行，不然 A 扣钱了，B 没有加钱这种情况就是错误的。那么事务就可以保证 A、B 账户的变动要么全部一起发生，要么全部一起不发生。

1.4.2. 事务特性

事务应该具有 4 个属性：原子性、一致性、隔离性、持久性。这四个属性通常称为 ACID 特性。

- 原子性（atomicity）
- 一致性（consistency）
- 隔离性（isolation）
- 持久性（durability）

1.4.2.1. 原子性 (atomicity)

一个事务必须被视为一个不可分割的最小单元，整个事务中的所有操作要么全部提交成功，要么全部失败，对于一个事务来说，不能只执行其中的一部分操作。比如：

King 老师借给大飞老师生活费：

- 1.King 老师工资卡扣除 500 元
- 2.大飞老师工资卡增加 500

整个事务的操作要么全部成功，要么全部失败，不能出现 King 老师工资卡扣除，但是大飞老师工资卡不增加的情况。如果原子性不能保证，就会很自然的出现一致性问题。

1.4.2.2. 一致性 (consistency)

一致性是指事务将数据库从一种一致性转换到另外一种一致性状态，在事务开始之前和事务结束之后数据库中数据的完整性没有被破坏。

King 老师借给大飞老师生活费：

1.King 老师工资卡扣除 500 元

2.大飞老师工资卡增加 500

扣除的钱（-500）与增加的钱（500）相加应该为 0，或者说 King 老师和大飞老师的账户的钱加起来，前后应该不变。

1.4.2.3. 持久性 (durability)

一旦事务提交，则其所做的修改就会永久保存到数据库中。此时即使系统崩溃，已经提交的修改数据也不会丢失。

1.4.2.4. 隔离性 (isolation)

一个事务的执行不能被其他事务干扰。即一个事务内部的操作及使用的数据对并发的其他事务是隔离的，并发执行的各个事务之间不能互相干扰。

如果隔离性不能保证，会导致什么问题？

King 老师借给大飞老师生活费，借了两次，每次都是 500，King 老师的卡里开始有 1200，大飞老师的卡里开始有 300，从理论上，借完后，King 老师的卡里有 200，大飞老师的卡里应该有 1300。

我们将 King 老师向大飞老师同时进行的两次转账操作分别称为 T1 和 T2，在现实世界中 T1 和 T2 是应该没有关系的，可以先执行完 T1，再执行 T2，或者先执行完 T2，再执行 T1，结果都是一样的。但是很不幸，真实的数据库中 T1 和 T2 的操作可能交替执行的，执行顺序就有可能是：

T1	T2
读_King(return King=1200)	
	读_King (return King=1200)
King-500	
写_King(return King=700)	
读_大飞(return 大飞=300)	
大飞+500	
写_大飞(return 大飞=800)	
	King-500
	写_King(return King=700)
	读_大飞(return 大飞=800)
	大飞+500
	写_大飞(return 大飞=1300)

如果按照上图中的执行顺序来进行两次转账的话，最终我们看到，King 老师的账户里还剩 700 元钱，相当于只扣了 500 元钱，但是大飞老师的账户里却成了 1300 元钱，多出现了 500 元，这银行岂不是要亏死了？

所以对于现实世界中状态转换对应的某些数据库操作来说，不仅要保证这些操作以原子性的方式执行完成，而且要保证其它的状态转换不会影响到本次状态转换，这个规则被称之为隔离性。

1.4.3. 事务并发引发的问题

我们知道 MySQL 是一个客户端 / 服务器架构的软件，对于同一个服务器来说，可以有若干个客户端与之连接，每个客户端与服务器连接上之后，就可以称之为一个会话（Session）。每个客户端都可以在自己的会话中向服务器发出请求语句，一个请求语句可能是某个事务的一部分，也就是对于服务器来说可能同时处理多个事务。

在上面我们说过事务有一个称之为隔离性的特性，理论上在某个事务对某个数据进行访问时，其他事务应该进行排队，当该事务提交之后，其他事务才可以继续访问这个数据，这样的话并发事务的执行就变成了串行化执行。

但是对串行化执行性能影响太大，我们既想保持事务的一定的隔离性，又想让服务器在处理访问同一数据的多个事务时性能尽量高些，当我们舍弃隔离性的时候，可能会带来什么样的数据问题呢？

1.4.3.1. 脏读

当一个事务读取到了另外一个事务修改但未提交的数据，被称为脏读。

T1	T2
begin	
select money from t1 where id = King(return 1200)	begin
	Update t1 set money = 1500 where id = King
select money from t1 where id = King(return 1500)	
	rollback;

事务 2 修改了一行记录，但是没有提交。然后事务 1 读取到了未提交的数据，如果事务 2 回滚其更改的数据或者再次更新，那么在事务 1 中看到的记录可能就是错误的。事务 1 读取到了 King 老师余额为 1500 的记录，但是事务 2 执行了回滚操作，这时并不存在 King 老师余额为 1500 记录。

1.4.3.2. 不可重复读

当事务内相同的记录被检索两次，且两次得到的结果不同时，此现象称为不可重复读。

事务1	事务2
begin	
select money from t1 where id = King(return 1200)	begin
	update t1 set money = 1500 where id = King
	commit;
select money from t1 where id = King(return 1500)	
commit;	

事务 2 对记录做了修改并提交成功，这意味着修改的记录对其他事务是可见的，因此事务 1 两次读取的 money 值不同。

1.4.3.3. 幻读

在事务执行过程中，另一个事务将新记录添加到正在读取的事务中时，会发生幻读。

当事务 1 重复执行 `SELECT... WHERE` 语句时，在这期间事务 2 执行 `INSERT` 语句插入了满足 `where` 条件的新记录。于是事务 1 执行两次一模一样的 `SELECT... WHERE`，返回却是两组不同的记录。

事务1	事务2
begin	
select money from t1 where id = King (return 1200)	begin
	insert into t1(id,money)values(king,500)
	commit;
select money from t1 where id = King (return [1200,500])	
commit;	

有的同学会有疑问，那如果事务 2 中是删除了符合的记录而不是插入新记录，那事务 1 中之后再根据条件读取的记录变少了，这种现象算不算幻读呢？明确说一下，在 MySQL 中这种现象不属于幻读，幻读强调的是在一个事务按照某个相同条件多次读取记录时，后读取时读到了之前没有读到的记录。

那对于先前已经读到的记录，之后又读取不到这种情况，算啥呢？其实这相当于对每一条记录都发生了不可重复读的现象。幻读只是重点强调了读取到了之前读取没有获取到的记录。

1.4.4. SQL 标准中的四种隔离级别

我们上边介绍了几种并发事务执行过程中可能遇到的一些问题，这些问题也有轻重缓急之分，我们给这些问题按照严重性来排一下序：

脏读 > 不可重复读 > 幻读

我们上边所说的舍弃一部分隔离性来换取一部分性能在这里就体现在：设立一些隔离级别，隔离级别越低，越严重的问题就越可能发生。有一帮人（并不是设计 MySQL 的大叔们）制定了一个所谓的 SQL 标准，在标准中设立了 4 个隔离级别：

READ UNCOMMITTED：未提交读。

READ COMMITTED：已提交读。

REPEATABLE READ：可重复读。

SERIALIZABLE：可串行化。

SQL 标准中规定，针对不同的隔离级别，并发事务可以发生不同严重程度的问题，具体情况如下：

隔离级别	脏读	不可重复读	幻读
READ UNCOMMITTED 未提交读	可能	可能	可能
READ COMMITTED 已提交读	-	可能	可能
REPEATABLE READ 可重复读	-	-	可能
SERIALIZABLE 可串行化	-	-	-

也就是说：

READ UNCOMMITTED 隔离级别下，可能发生脏读、不可重复读和幻读问题。

READ COMMITTED 隔离级别下，可能发生不可重复读和幻读问题，但是不可以发生脏读问题。

REPEATABLE READ 隔离级别下，可能发生幻读问题，但是不可以发生脏读和不可重复读的问题。

SERIALIZABLE 隔离级别下，各种问题都不可以发生。

1.4.5. MySQL 中的隔离级别

不同的数据库厂商对 SQL 标准中规定的四种隔离级别支持不一样，比方说 Oracle 就只支持 READ COMMITTED 和 SERIALIZABLE 隔离级别。本书中所讨论的 MySQL 虽然支持 4 种隔离级别，但与 SQL 标准中所规定的各级隔离级别允许发生的问题却有些出入，MySQL 在 REPEATABLE READ 隔离级别下，是可以禁止幻读问题的发生的。

隔离级别	脏读	不可重复读	幻读
READ UNCOMMITTED 未提交读	可能	可能	可能
READ COMMITTED 已提交读	-	可能	可能
REPEATABLE READ 可重复读	-	-	-
SERIALIZABLE 可串行化	-	-	-

MySQL 的默认隔离级别为 REPEATABLE READ，我们可以手动修改事务的隔离级别。

1.4.5.1. 如何设置事务的隔离级别

我们可以通过下边的语句修改事务的隔离级别：

```
SET [GLOBAL|SESSION] TRANSACTION ISOLATION LEVEL level;
```

其中的 level 可选值有 4 个：

```
level: {  
    REPEATABLE READ  
    | READ COMMITTED  
    | READ UNCOMMITTED  
    | SERIALIZABLE  
}
```

设置事务的隔离级别的语句中，在 SET 关键字后可以放置 GLOBAL 关键字、SESSION 关键字或者什么都不放，这样会对不同范围的事务产生不同的影响，具体如下：

使用 GLOBAL 关键字（在全局范围影响）：

比方说这样：

```
SET GLOBAL TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

则：

只对执行完该语句之后产生的会话起作用。当前已经存在的会话无效。

使用 SESSION 关键字（在会话范围影响）：

比方说这样：

```
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

则：对当前会话的所有后续的事务有效

该语句可以在已经开启的事务中间执行，但不会影响当前正在执行的事务。

如果在事务之间执行，则对后续的事务有效。

上述两个关键字都不用（只对执行语句后的下一个事务产生影响）：

比方说这样：

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

则：

只对当前会话中下一个即将开启的事务有效。下一个事务执行完后，后续事务将恢复到之前的隔离级别。该语句不能在已经开启的事务中间执行，会报错的。

如果我们在服务器启动时想改变事务的默认隔离级别，可以修改启动参数 transaction-isolation 的值，比方说我们在启动服务器时指定了 --transaction-isolation=SERIALIZABLE，那么事务的默认隔离级别就从原来的 REPEATABLE READ 变成了 SERIALIZABLE。

想要查看当前会话默认的隔离级别可以通过查看系统变量 transaction_isolation 的值来确定：

SHOW VARIABLES LIKE 'transaction_isolation';

```
mysql> SHOW VARIABLES LIKE 'transaction_isolation';
+-----+-----+
| Variable_name | Value          |
+-----+-----+
| transaction_isolation | REPEATABLE-READ |
+-----+-----+
1 row in set (0.00 sec)
```

或者使用更简便的写法:

SELECT @@transaction_isolation;

```
mysql> SELECT @@transaction_isolation;
+-----+
| @@transaction_isolation |
+-----+
| REPEATABLE-READ        |
+-----+
1 row in set (0.00 sec)
```

3. 注意: transaction_isolation 是在 MySQL 5.7.20 的版本中引入来替换 tx_isolation 的, 如果你使用的是之前版本的 MySQL, 请将上述用到系统变量 transaction_isolation 的地方替换为 tx_isolation。

1.4.6. MySQL 事务

1.4.6.1. 事务基本语法

事务开始

- 1、begin
- 2、START TRANSACTION (推荐)
- 3、begin work

事务回滚

rollback

事务提交

commit

1.4.6.2. 保存点

如果你开启了一个事务, 执行了很多语句, 忽然发现某条语句有点问题, 你只好使用 ROLLBACK 语句来让数据库状态恢复到事务执行之前的样子, 然后一切从头再来, 但是可能根据业务和数据的变化, 不需要全部回滚。所以 MySQL 里提出了一个保存点 (英文: savepoint) 的概念, 就是在事务对应的数据库语句中打几个点, 我们在调用 ROLLBACK 语句时可以指定会滚到哪个点, 而不是回到最初的原点。定义保存点的语法如下:

SAVEPOINT 保存点名称;

当我们想回滚到某个保存点时，可以使用下边这个语句（下边语句中的单词 WORK 和 SAVEPOINT 是可有可无的）：

ROLLBACK TO [SAVEPOINT] 保存点名称;

不过如果 ROLLBACK 语句后边不跟随保存点名称的话，会直接回滚到事务执行之前的状态。

如果我们想删除某个保存点，可以使用这个语句：

RELEASE SAVEPOINT 保存点名称;

savepoint

show variables like '%autocommit%'; 自动提交事务是开启的

set autocommit=0;

insert into testdemo values(5,5,5);

savepoint order_exp;

insert into testdemo values(6,6,6);

savepoint order_exp_2;

insert into testdemo values(7,7,7);

savepoint s3;

select * from testdemo

rollback to savepoint order_exp_2

rollback

1.4.6.3. 隐式提交

当我们使用 START TRANSACTION 或者 BEGIN 语句开启了一个事务，或者把系统变量 autocommit 的值设置为 OFF 时，事务就不会进行自动提交，但是如果我们输入了某些语句之后就会悄悄的提交掉，就像我们输入了 COMMIT 语句了一样，这种因为某些特殊的语句而导致事务提交的情况称为隐式提交，这些会导致事务隐式提交的语句包括：

执行 DDL

定义或修改数据库对象的数据定义语言（Data definition language，缩写为：DDL）。

所谓的数据库对象，指的就是数据库、表、视图、存储过程等等这些东西。当我们使用 CREATE、ALTER、DROP 等语句去修改这些所谓的数据库对象时，就会隐式的提交前边语句所属于的事务，就像这样：

BEGIN;

SELECT ... # 事务中的一条语句

UPDATE ... # 事务中的一条语句

... # 事务中的其它语句

CREATE TABLE ... # 此语句会隐式的提交前边语句所属于的事务

隐式使用或修改 mysql 数据库中的表

当我们使用 ALTER USER、CREATE USER、DROP USER、GRANT、RENAME USER、REVOKE、SET PASSWORD 等语句时也会隐式的提交前边语句所属于的事务。

事务控制或关于锁定的语句

当我们在一个会话里，一个事务还没提交或者回滚时就又使用 START TRANSACTION 或者 BEGIN 语句开启了另一个事务时，会隐式的提交上一个事务，比如这样：

```
BEGIN;
```

```
SELECT ... # 事务中的一条语句
```

```
UPDATE ... # 事务中的一条语句
```

```
... # 事务中的其它语句
```

```
BEGIN; # 此语句会隐式的提交前边语句所属于的事务
```

或者当前的 autocommit 系统变量的值为 OFF，我们手动把它调为 ON 时，也会隐式的提交前边语句所属的事务。

或者使用 LOCK TABLES、UNLOCK TABLES 等关于锁定的语句也会隐式的提交前边语句所属的事务。

加载数据的语句

比如我们使用 LOAD DATA 语句来批量往数据库中导入数据时，也会隐式的提交前边语句所属的事务。

关于 MySQL 复制的一些语句

使用 START SLAVE、STOP SLAVE、RESET SLAVE、CHANGE MASTER TO 等语句时也会隐式的提交前边语句所属的事务。

其它的一些语句

使用 ANALYZE TABLE、CACHE INDEX、CHECK TABLE、FLUSH、LOAD INDEX INTO CACHE、OPTIMIZE TABLE、REPAIR TABLE、RESET 等语句也会隐式的提交前边语句所属的事务。

2. 表结构设计和数据类型优化

2.1. 数据库表设计

良好的表结构设计是高性能的基石，应该根据系统将要执行的业务查询来设计，这往往需要权衡各种因素。糟糕的表结构设计，会浪费大量的开发时间，严重延误项目开发周期，让人痛苦万分，而且直接影响到数据库的性能，并需要花费大量不必要的优化时间，效果往往还不怎么样。

在数据库表设计上有个很重要的设计准则，称为范式设计。

2.1.1. 范式设计

2.1.1.1. 什么是范式？

范式来自英文 **Normal Form**，简称 **NF**。要想设计一个好的关系，必须使关系满足一定的约束条件，此约束已经形成了规范，分成几个等级，一级比一级要求得严格。满足这些规范的数据库是简洁的、结构明晰的，同时，不会发生插入(insert)、删除(delete)和更新(update)操作异常。反之则是乱七八糟，不仅给数据库的编程人员制造麻烦，而且面目可憎，可能存储了大量不需要的冗余信息。

目前关系数据库有六种范式：第一范式（1NF）、第二范式（2NF）、第三范式（3NF）、巴斯-科德范式（BCNF）、第四范式(4NF)和第五范式（5NF，又称完美范式）。满足最低要求的范式是第一范式（1NF）。在第一范式的基础上进一步满足更多规范要求的称为第二范式（2NF），其余范式以次类推。一般说来，数据库只需满足第三范式(3NF)就行了。

2.1.1.2. 数据库设计的第一范式

定义： 属于第一范式关系的所有属性都不可再分，即数据项不可分。

理解： 第一范式强调数据表的原子性，是其他范式的基础。例如下表

id	name-age
1	张三-23

name-age 列具有两个属性，一个 name,一个 age 不符合第一范式，把它拆分成两列

id	name	age
1	张三	23

上表就符合第一范式关系。但日常生活中仅用第一范式来规范表格是远远不够的，依然会存在数据冗余过大、删除异常、插入异常、修改异常的问题，此时就需要引入规范化概念，将其转化为更标准化的表格，减少数据依赖。

实际上，1NF 是所有关系型数据库的最基本要求，你在关系型数据库管理系统（RDBMS），例如 SQL Server，Oracle，MySQL 中创建数据表的时候，如果数据表的设计不符合这个最基本要求，那么操作一定是不能成功的。也就是说，只要在 RDBMS 中已经存在的数据表，一定是符合 1NF 的。

2.1.1.3. 数据库设计的第二范式

第二范式（2NF）是在第一范式（1NF）的基础上建立起来的，即满足第二范式（2NF）必须先满足第一范式（1NF）。

第二范式（2NF）要求数据库表中的每个实例或行必须可以被惟一地区分。通常在实现来说，需要为表加上一个列，以存储各个实例的惟一标识。例如员工信息表中加上了员工编号（emp_id）列，因为每个员工的员工编号是惟一的，因此每个员工可以被惟一区分。这个惟一属性列被称为主关键字或主键、主码。

也就是说要求表中只具有一个业务主键，而且第二范式（2NF）要求实体的属性完全依赖于主关键字。所谓完全依赖是指不能存在仅依赖主关键字一部分的属性。什么意思呢？

有两张表：订单表，产品表

订单表ID (主键)	订单时间	产品ID
1	2018-12-12	3
1	2018-12-12	4

产品表ID	产品名称
2	娃娃
3	飞机
4	java入门

一个订单有多个产品，所以订单的主键为【订单 ID】和【产品 ID】组成的联合主键，这样 2 个组件不符合第二范式，而且产品 ID 和订单 ID 没有强关联，故，把订单表进行拆分为订单表与订单与商品的中间表

订单表ID	订单时间
1	2018-12-12

订单-商品中间表ID	订单ID	产品ID
1	1	3
2	1	4

产品表ID	产品名称
2	娃娃
3	飞机
4	java入门

2.1.1.4. 数据库设计的第三范式

指每一个非主属性既不部分依赖于也不传递依赖于业务主键，也就是在第二范式的基础上消除了非主键对主键的传递依赖。例如，存在一个部门信息表，其中每个部门有部门编号（dept_id）、部门名称、部门简介等信息。那么在员工信息表中列出部门编号后就不能再将部门名称、部门简介等与部门有关的信息再加入员工信息表中。如果不存在部门信息表，则根据第三范式（3NF）也应该构建它，否则就会有大量的数据冗余。

订单表

ID-订单编号 (主键)	订单时间	产品ID	产品名称
1	2020-01-01	1	机械键盘
2	2020-01-02	2	蓝牙音响

产品ID	产品名称
1	机械键盘
2	蓝牙音响
3	蓝牙鼠标

其中

产品 ID 与订单编号存在关联关系

产品名称与订单编号存在关联关系

产品 ID 与产品名称存在关联关系

如果产品 ID 发生改变，产品姓名也会改变；产品名称发生变化，这样不符合第三范式，应该把产品名称这一列从订单表中删除

2.1.1.5. 范式说明

真正的数据库范式定义上，相当难懂，比如第二范式（2NF）的定义“若某关系 R 属于第一范式，且每一个非主属性完全函数依赖于任何一个候选码，则关系 R 属于第二范式。”，这里面有着大堆专业术语的堆叠，比如“函数依赖”、“码”、“非主属性”、与“完全函数依赖”等等，而且有完备的公式定义，需要仔细研究的同学，请参考这本书：



2.1.2. 反范式设计

2.1.2.1. 什么叫反范式化设计

完全符合范式化的设计真的完美无缺吗？很明显在实际的业务查询中会大量存在着表的关联查询，而大量的表关联很多的时候非常影响查询的性能。

所谓得反范式化就是为了性能和读取效率得考虑而适当得对数据库设计范式得要求进行违反。允许存在少量得冗余，换句话说来说反范式化就是使用空间来换取时间。

2.1.2.2. 反范式设计-商品信息

下面是范式设计的商品信息表

商品信息					
ID	商品名称	出版社名称	图书价格	图书表述	作者
分类信息					
分类名称	分类描述				
商品分类对应关系表					
商品名称	分类名称				

商品信息和分类信息经常一起查询，所以把分类信息也放到商品表里面，冗余存放。

商品信息反范式设计

ID	商品名称	分类名称	出版社名称	图书价格	图书表述	作者
----	------	------	-------	------	------	----

2.1.2.3. 范式化和反范式总结

范式化设计优缺点

- 1、范式化的更新操作通常比反范式化要快。
- 2、当数据较好地范式化时，就只有很少或者没有重复数据，所以只需要修改更少的数据。
- 3、范式化的表通常更小，可以更好地放在内存里，所以执行操作会更快。

4、很少有多余的数据意味着检索列表数据时更少需要 **DISTINCT** 或者 **GROUP BY** 语句。在非范式化的结构中必须使用 **DISTINCT** 或者 **GROUPBY** 才能获得一份唯一的列表，但是如果是一张单独的表，很可能则只需要简单的查询这张表就行了。

范式化设计的缺点是通常需要关联。稍微复杂一些的查询语句在符合范式的表上都可能需要至少一次关联，也许更多。这不但代价昂贵，也可能使一些索引策略无效。例如，范式化可能将列存放在不同的表中，而这些列如果在一个表中本可以属于同一个索引。

反范式化设计优缺点

- 1、反范式设计可以减少表的关联
- 2、可以更好的进行索引优化。

反范式设计缺点也很明显，1、存在数据冗余及数据维护异常，2、对数据的修改需要更多的成本。

2.1.3. 实际工作中的反范式实现

2.1.3.1. 性能提升-缓存和汇总

范式化和反范式化的各有优劣，怎么选择最佳的设计？

请记住：小孩子才做选择，我们全都要；小孩才分对错，大人只看利弊。

而现实也是，完全的范式化和完全的反范式化设计都是实验室里才有的东西，在真实世界中很少会这么极端地使用。在实际应用中经常需要混用。

最常见的反范式化数据的方法是复制或者缓存，在不同的表中存储相同的特定列。

比如从父表冗余一些数据到子表的。前面我们看到的分类信息放到商品表里面进行冗余存放就是典型的例子。

缓存衍生值也是有用的。如果需要显示每个用户发了多少消息，可以每次执行一个对用户发送消息进行 **count** 的子查询来计算并显示它，也可以在 **user** 表用户中建一个消息发送数目的专门列，每当用户发新消息时更新这个值。

有需要时创建一张完全独立的汇总表或缓存表也是提升性能的好办法。“缓存表”来表示存储那些可以比较简单地从其他表获取（但是每次获取的速度比较慢）数据的表（例如，逻辑上冗余的数据）。而“汇总表”时，则保存的是使用 **GROUP BY** 语句聚合数据的表。

在使用缓存表和汇总表时，有个关键点是如何维护缓存表和汇总表中的数据，常用的有两种方式，实时维护数据和定期重建，这个取决于应用程序，不过一般来说，缓存表用实时维护数据更多点，往往在一个事务中同时更新数据本表和缓存表，汇总表则用定期重建更多，使用定时任务对汇总表进行更新。

2.1.3.2. 性能提升-计数器表

计数器表在 Web 应用中很常见。比如网站点击数、用户的朋友数、文件下载次数等。对于高并发下的处理，首先可以创建一张独立的表存储计数器，这样可使计数器表小且快，并且可以使用一些更高级的技巧。

比如假设有一个计数器表，只有一行数据，记录网站的点击次数，网站的每次点击都会导致对计数器进行更新，问题在于，对于任何想要更新这一行的事务来说，这条记录上都有一个全局的互斥锁(mutex)。这会使得这些事务只能串行执行，会严重限制系统的并发能力。

怎么改进呢？可以将计数器保存在多行中，每次随机选择一行进行更新。在具体实现上，可以增加一个槽 (slot) 字段，然后预先在这张表增加 100 行或者更多数据，当对计数器更新时，选择一个随机的槽 (slot) 进行更新即可。

这种解决思路其实就是写热点的分散，在 JDK 的 JDK1.8 中新出的原子类 LongAdder 也是这种处理方式，而我们在实际的缓冲中间件 Redis 等的使用、架构设计中，可以采用这种写热点的分散的方式，当然架构设计中对于写热点还有削峰填谷的处理方式，这种在 MySQL 的实现中也有体现，我们后面会讲到。

2.1.3.3. 反范式设计-分库分表中的查询

例如,用户购买了商品,需要将交易记录保存下来,那么如果按照买家的纬度分表,则每个买家的交易记录都被保存在同一表中, 我们可以很快、很方便地查到某个买家的购买情况, 但是某个商品被购买的交易数据很有可能分布在多张表中, 查找起来比较麻烦。 反之, 按照商品维度分表, 则可以很方便地查找到该商品的购买情况, 但若要查找到买家的交易记录, 则会比较麻烦。

所以常见的解决方式如下。

- (1) 在多个分片表查询后合并数据集, 这种方式的效率很低。
- (2) 记录两份数据, 一份按照买家纬度分表, 一份按照商品维度分表,,
- (3) 通过搜索引擎解决, 但如果实时性要求很高, 就需要实现实时搜索

在某电商交易平台下, 可能有买家查询自己在某一时时间段的订单, 也可能有卖家查询自己在某一时时间段的订单, 如果使用了分库分表方案, 则这两个需求是难以满足的, 因此, 通用的解决方案是, 在交易生成时生成一份按照买家分片的数据副本和一份按照卖家分片的数据副本, 查询时分别满足之前的两个需求, 因此, 查询的数据和交易的数据可能是分别存储的, 并从不同的系统提供接口。

2.2. 字段数据类型优化

MySQL 支持的数据类型非常多, 选择正确的数据类型对于获得高性能至关重要。不管存储哪种类型的数据, 下面几个简单的原则都有助于做出更好的选择。

2.2.1. 基本原则

2.2.1.1. 更小的通常更好

一般情况下,应该尽量使用可以正确存储数据的最小数据类型。更小的数据类型通常更快,因为它们占用更少的磁盘、内存和 CPU 缓存,并且处理时需要的 CPU 周期也更少。

但是要确保没有低估需要存储的值的范围,因为在的多个地方增加数据类型的范围是一个非常耗时和痛苦的操作。如果无法确定哪个数据类型是最好的,就选择你认为不会超过范围的最小类型。

2.2.1.2. 简单就好

简单数据类型的操作通常需要更少的 CPU 周期。例如,整型比字符操作代价更低,因为字符集和校对规则(排序规则)使字符比较比整型比较更复杂。比如应该使用 MySQL 内建的类型而不是字符串来存储日期和时间。

2.2.1.3. 尽量避免 NULL

很多表都包含可为 NULL(空值)的列,即使应用程序并不需要保存 NULL 也是如此,这是因为可为 NULL 是列的默认属性。通常情况下最好指定列为 NOT NULL,除非真的需要存储 NULL 值。

如果查询中包含可为 NULL 的列,对 MySQL 来说更难优化,因为可为 NULL 的列使得索引、索引统计和值比较都更复杂。可为 NULL 的列会使用更多的存储空间,在 MySQL 里也需要特殊处理。当可为 NULL 的列被索引时,每个索引记录需要一个额外的字节。

通常把可为 NULL 的列改为 NOT NULL 带来的性能提升比较小,所以(调优时)没有必要首先在现有 schema 中查找并修改掉这种情况,除非确定这会导致问题。但是,如果计划在列上建索引,就应该尽量避免设计成可为 NULL 的列。

2.2.2. Int/整数类型

存储整数,可以使用这几种整数类型:TINYINT, SMALLINT, MEDIUMINT, INT, BIGINT。分别使用 8, 16, 24, 32, 64 位存储空间,也就是 1、2、3、4、8 个字节。它们可以存储的值的范围请自行计算。

同时整数类型有可选的 UNSIGNED 属性,表示不允许负值,这大致可以使正数的上限提高一倍。例如 TINYINT UNSIGNED 可以存储的范围是 0~255,而 TINYINT 的存储范围是-128~127。

有符号和无符号类型使用相同的存储空间,并具有相同的性能,因此可以根据实际情况选择合适的类型。

MySQL 可以为整数类型指定宽度,例如 INT(11),对大多数应用这是没有意义的,它不会限制值的合法范围,只是规定了 MySQL 的一些交互工具(例如 MySQL

命令行客户端)用来显示字符的个数。对于存储和计算来说，INT(1)和 INT(20)是相同的。

在选择上，遵循着更小的通常更好的原则，在业务许可的情况下，尽量选择位数小的。

2.2.3. 实数类型

实数是带有小数部分的数字。MySQL 既支持精确类型的存储 DECIMAL 类型，也支持不精确类型存储 FLOAT 和 DOUBLE 类型。

DECIMAL 类型用于存储精确的小数，本质上 MySQL 是以字符串形式存放的。所以 CPU 不支持对 DECIMAL 的直接计算，所以在 MySQL 中自身实现了 DECIMAL 的高精度计算。相对而言，CPU 直接支持原生浮点计算，所以浮点运算明显更快。

浮点和 DECIMAL 类型都可以指定精度。对于 DECIMAL 列，可以指定小数点前后所允许的最大位数。这会影响列的空间消耗。MySQL 5.0 和更高版本将数字打包保存到一个二进制字符串中（每 4 个字节存 9 个数字）。例如，DECIMAL(18,9) 小数点两边将各存储 9 个数字，一共使用 9 个字节：小数点前的数字用 4 个字节，小数点后的数字用 4 个字节，小数点本身占 1 个字节。

MySQL 5.0 和更高版本中的 DECIMAL 类型允许最多 65 个数字。

浮点类型在存储同样范围的值时，通常比 DECIMAL 使用更少的空间。FLOAT 使用 4 个字节存储，DOUBLE 占用 8 个字节，所以 DOUBLE 比 FLOAT 有更高的精度和更大的范围。

因为需要额外的空间和计算开销，所以应该尽量只在对小数进行精确计算时才使用 DECIMAL，例如存储财务或金融数据，在精度不敏感和需要快速运算的时候，选择 FLOAT 和 DOUBLE。

但在数据量比较大的而且要求精度时，可以考虑使用 BIGINT 代替 DECIMAL，将需要存储的货币单位根据小数的位数乘以相应的倍数即可。假设要存储财务数据精确到万分之一分，则可以把所有金额乘以一百万，然后将结果存储在 BIGINT 里，这样可以同时避免浮点存储计算不精确和 DECIMAL 精确计算代价高的问题。

2.2.4. 字符串类型

MySQL 支持多种字符串类型，包括 VARCHAR 和 CHAR 类型、BLOB 和 TEXT 类型、ENUM（枚举）和 SET 类型。

2.2.4.1. VARCHAR 和 CHAR 类型

VARCHAR 和 CHAR 是两种最主要的字符串类型。

VARCHAR

VARCHAR 类型用于存储可变长字符串，是最常见的字符串数据类型。它比定长类型更节省空间，因为它仅使用必要的空间（例如，越短的字符串使用越少的空间）。

在内部实现上，既然是变长，**VARCHAR** 需要使用 1 或 2 个额外字节记录字符串的长度，如果列的最大长度小于或等于 255 字节，则只使用 1 个字节表示，否则使用 2 个字节。

VARCHAR 节省了存储空间，所以对性能也有帮助。但是，由于行是变长的，在 **UPDATE** 时新值比旧值长时，使行变得比原来更长，这就可能导致需要做额外的工作。如果一个行占用的空间增长，并且在页内没有更多的空间可以存储，在这种情况下，**MyISAM** 会将行拆成不同的片段存储，**InnoDB** 则需要分裂页来使行可以放进页内。

CHAR

CHAR 类型是定长的，**MySQL** 总是根据定义的字符串长度分配足够的空间。当存储 **CHAR** 值时，**MySQL** 会删除所有的末尾空格，**CHAR** 值会根据需要采用空格进行填充以方便比较。

小结

在 **CHAR** 和 **VARCHAR** 的选择上，这些情况下使用 **VARCHAR** 是合适的：

字符串列的最大长度比平均长度大很多；列的更新很少；使用了像 **UTF-8** 这样复杂的字符集，每个字符都使用不同的字节数进行存储。

CHAR 适合存储很短的字符串，或者所有值定长或都接近同一个长度。例如，**CHAR** 非常适合存储密码的 **MD5** 值，因为这是一个定长的值。对于经常变更的数据，**CHAR** 也比 **VARCHAR** 更好，因为定长的 **CHAR** 类型不容易产生碎片。

对于非常短的列，**CHAR** 比 **VARCHAR** 在存储空间上也更有效率。例如用 **CHAR(1)** 来存储只有 **Y** 和 **N** 的值，如果采用单字节字符集只需要一个字节，但是 **VARCHAR(1)** 却需要两个字节，因为还有一个记录长度的额外字节。

另外，使用 **VARCHAR(5)** 和 **VARCHAR(200)** 存储 'hello' 在磁盘空间上开销是一样的。我们随便选择一个就好？应该使用更短的列，为什么？

事实证明有很大的优势。更长的列会消耗更多的内存，因为 **MySQL** 通常会分配固定大小的内存块来保存内部值。尤其是使用内存临时表进行排序或操作时会特别糟糕。在利用磁盘临时表进行排序时也同样糟糕。

所以最好的策略是只分配真正需要的空间。

2.2.4.2. BLOB 和 TEXT 类型

BLOB 和 **TEXT** 都是为存储很大的数据而设计的字符串数据类型，分别采用二进制和字符方式存储。

与其他类型不同，**MySQL** 把每个 **BLOB** 和 **TEXT** 值当作一个独立的对象处理。存储引擎在存储时通常会做特殊处理。当 **BLOB** 和 **TEXT** 值太大时，**InnoDB** 会使用专门的“外部”存储区域来进行存储，此时每个值在行内需要 1~4 个字节存储一个指针，然后在外部存储区域存储实际的值。

BLOB 和 **TEXT** 家族之间仅有的不同是 **BLOB** 类型存储的是二进制数据，没有排序规则或字符集，而 **TEXT** 类型有字符集和排序规则。

使用 **BLOB** 和 **TEXT** 要慎重：

(1) BLOB 和 TEXT 值会引起一些性能问题,所以尽量避免使用 BLOB 和 TEXT 类型;

(2) 一定要用, 建议把 BLOB 或 TEXT 列分离到单独的表中;

(3) 在不必要的时候避免检索大型的 BLOB 或 TEXT 值。例如, SELECT * 查询就不是很好的想法, 除非能够确定作为约束条件的 WHERE 子句只会找到所需要的数据行。否则, 很可能毫无目的地在网络上传输大量的值。建议可以搜索索引列, 决定需要的哪些数据行, 然后从符合条件的数据行中检索 BLOB 或 TEXT 值;

(4) 还可以使用合成的(Synthetic)索引来提高大文本字段(BLOB 或 TEXT)的查询性能。简单来说, 合成索引就是根据大文本字段的内容建立一个散列值, 并把这个值存储在单独的数据列中, 接下来就可以通过检索散列值找到数据行了。但是, 要注意这种技术只能用于精确匹配的查询(散列值对于类似“<”或“>=”等范围搜索操作符是没有用处的)。可以使用 MD5 函数生成散列值, 也可以使用 SHA1(或 CRC32), 或者使用自己的应用程序逻辑来计算散列值。

2.2.4.3. 必要时, 使用枚举代替字符串

如果表中的字段的取值是固定几个字符串, 可以使用枚举列代替常用的字符串类型。

枚举列可以把一些不重复的字符串存储成一个预定义的集合。MySQL 在存储枚举时非常紧凑, 会根据列表值的数量压缩到一个或者两个字节中, MySQL 在内部会将每个值在列表中的位置保存为整数, 这样的话可以让表的大小大为缩小。

```
CREATE TABLE enum_test(e ENUM(' fish', 'apple', 'dog') NOT NULL);
```

```
INSERT INTO enum_test(e) VALUES('fish'),('dog'),('apple');
```

但是要注意,

1) 因为枚举列实际存储为整数, 而不是字符串, 所以不要使用数字作为 ENUM 枚举常量, 这种双重性很容易导致混乱, 例如 ENUN(' 1','2', '3')。

2) 枚举字段是按照内部存储的整数而不是定义的字符串进行排序的, 所以尽量按照需要的顺序来定义枚举列。

2.2.5. 日期和时间类型

日期类型	占用空间	表示范围
DATETIME	8	1000-01-01 00:00:00 ~ 9999-12-31 23:59:59
DATE	3	1000-01-01 ~ 9999-12-31
TIMESTAMP	4	1970-01-01 00:00:00UTC ~ 2038-01-19 03:14:07UTC
YEAR	1	YEAR(2):1970-2070, YEAR(4):1901-2155
TIME	3	-838:59:59 ~ 838:59:59

MySQL 可以使用许多类型来保存日期和时间值，例如 YEAR 和 DATE。MySQL 能存储的最小时间粒度为秒。

大部分时间类型都没有替代品，因此没有什么什么是最佳选择的问题。唯一的问题 MySQL 提供两种相似的日期类型:DATETIME 和 TIMESTAMP。对于很多应用程序，它们都能工作，但是在某些场景，需要做些适当选择。

DATETIME

这个类型能保存大范围的值，从 1001 年到 9999 年，精度为秒。它把日期和时间封装到格式为 YYYMMDDHHMMSS 的整数中，与时区无关。使用 8 个字节的存储空间。

TIMESTAMP

TIMESTAMP 类型保存了从 1970 年 1 月 1 日午夜（格林尼治标准时间）以来的秒数，它和 UNIX 时间戳相同。TIMESTAMP 只使用 4 个字节的存储空间，因此它的范围比 DATETIME 小得多；只能表示从 1970 年到 2038 年。TIMESTAMP 显示的值也依赖于时区。从空间效率来说，当然 TIMESTAMP 比 DATETIME 更高。

如果需要存储比秒更小粒度的日期和时间值怎么办？MySQL 目前没有提供合适的数据类型，但是可以使用自己的存储格式：可以使用 BIGINT 类型存储微秒级别的时间戳，或者使用 DOUBLE 存储秒之后的小数部分。

2.3. 工程实践-命名规范

1、数据库、表、字段的命名要遵守可读性原则，尽可能少使用或者不使用缩写

2、表名、字段名必须使用小写字母或数字，禁止出现数字开头，禁止两个下划线中间只出现数字。数据库字段名的修改代价很大，因为无法进行预发布，所以字段名称需要慎重考虑。说明：MySQL 在 Windows 下不区分大小写，但在 Linux 下默认是区分大小写。因此，数据库名、表名、字段名，都不允许出现任何大写字母，避免节外生枝。

3、表名不使用复数名词

4、数据库、表、字段的命名禁用保留字，如 desc、range、match 之类

5、对象的名字应该能够描述它所表示的对象

例如：

表的名称应该能够体现表中存储的数据内容，最好是遵循“业务名称_表的作用”；对于存储过程存储过程应该能够体现存储过程的功能。库名与应用名称尽量一致。

6、主键索引名为 pk_字段名；唯一索引名为 uk_字段名；普通索引名则为 idx_字段名。

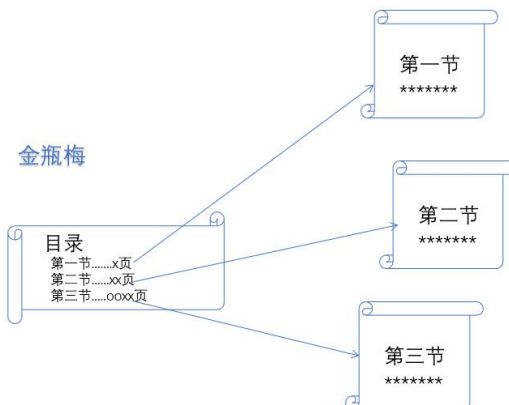
7、表达是与否概念的字段，应该使用 is_xxx 的方式命名，数据类型是 unsigned tinyint（1 表示是，0 表示否）。

3. 高性能索引

3.1. 生活中的索引

MySQL 官方对索引的定义为：索引（Index）是帮助 MySQL 高效获取数据的数据结构。可以得到索引的本质：**索引是数据结构**。

上面的理解比较抽象，举一个例子，平时看任何一本书，首先看到的都是目录，通过目录去查询书籍里面的内容会非常的迅速。

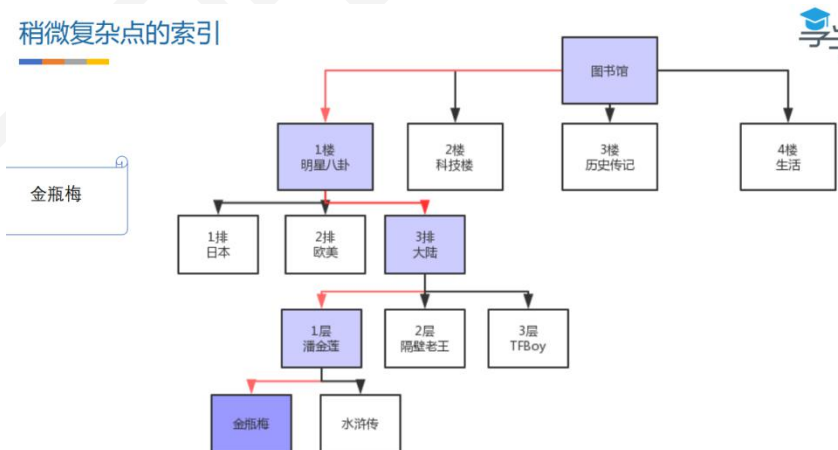


上图就是一本金瓶梅的书，书籍的目录是按顺序放置的，有第一节，第二节它本身就是一种顺序存放的数据结构，是一种顺序结构。

另外通过目录（索引），可以快速查询到目录里面的内容，它能高效获取数据，通过这个简单的案例可以理解所以就是**高效获取数据的数据结构**

再来看一个复杂的情况，

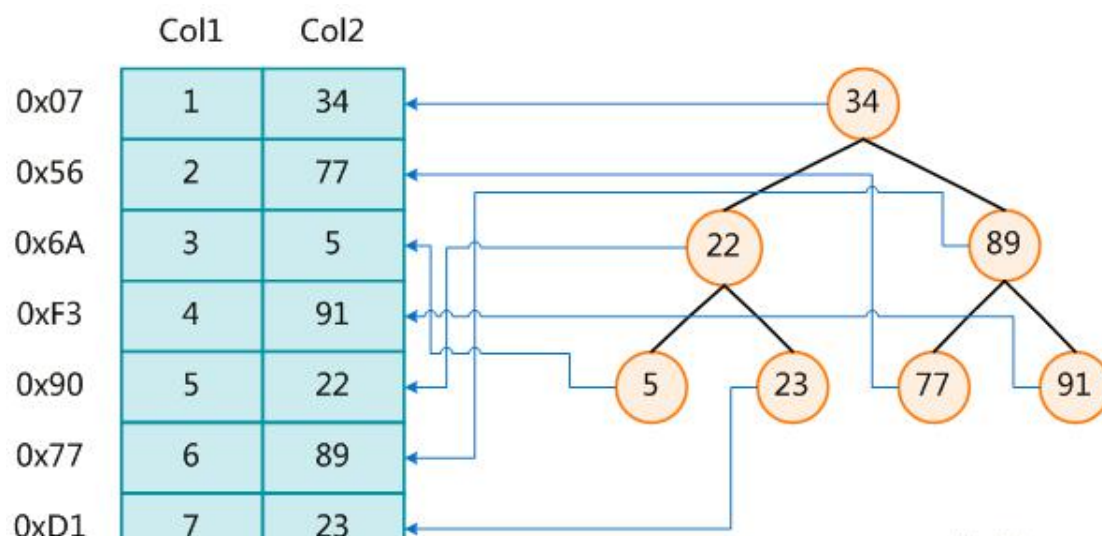
稍微复杂点的索引



我们要去图书馆找一本书，这图书馆的书肯定不是线性存放的，它对不同的书籍内容进行了分类存放，整索引由于一个个节点组成，根节点有中间节点，中间节点下面又由子节点，最后一层是叶子节点，

可见，整个索引结构是一棵倒挂着的树，其实它就是一种数据结构，这种数据结构比前面讲到的线性目录更好的增加了查询的速度。

3.2. MySQL 中的索引



MySQL 中的索引其实也是这么一回事，我们可以在数据库中建立一系列的索引，比如创建主键的时候默认会创建主键索引，上图是一种 **BTREE** 的索引。每一个节点都是主键的 ID

当我们通过 ID 来查询内容的时候，首先去查索引库，查到索引后能快速的根据索引定位数据的具体位置。

所以索引是应用程序设计和开发的一个重要方面。若索引太多，应用程序的性能可能会受到影响。而索引太少，对查询性能又会产生影响。要找到一个合适的平衡点，这对应用程序的性能至关重要。

InnoDB 存储引擎支持以下几种常见的索引：**B+树索引**、全文索引、哈希索引，其中比较关键的是 **B+树索引**，其他的索引我们也会在本课程中穿插讲解。

HashMap 适合做数据库索引吗？

- 1、hash 表只能匹配是否相等，不能实现范围查找；
- 2、当需要按照索引进行 **order by** 时，hash 值没办法支持排序；
- 3、组合索引可以支持部分索引查询，如(a,b,c)的组合索引，查询中只用到了阿和 b 也可以查询的，如果使用 hash 表，组合索引会将几个字段合并 hash，没办法支持部分索引；
- 4、当数据量很大时，hash 冲突的概率也会非常大。

3.2.1. B+Tree

B+树索引就是传统意义上的索引，这是目前关系型数据库系统中查找最常用和最为有效的索引。**B+树索引**的构造类似于二叉树，根据键值 (**Key Value**) 快速找到数据。注意 **B+树**中的 **B** 不是代表二叉(binary)，而是代表平衡(balance)，因为 **B+树**是从最早的平衡二叉树演化而来，但是 **B+树**不是一个二叉树。

在了解 B+树索引之前先要知道与之密切相关的一些算法与数据结构，这有助于我们更好的理解 B+树索引的工作方式，因为 B+树是通过二叉查找树，再由平衡二叉树，B 树演化而来。

3.2.1.1. 二分查找

二分查找法 (binary search) 也称为折半查找法，用来查找一组有序的记录数组中的某一记录。

其基本思想是：将记录按有序化（递增或递减）排列，在查找过程中采用跳跃式方式查找，即先以有序数列的中点位置作为比较对象，如果要找的元素值小于该中点元素，则将待查序列缩小为左半部分，否则为右半部分。通过一次比较，将查找区间缩小一半。

给出一个例子，注意该例子已经是升序排序的，且查找 数字 48

数据：5, 10, 19, 21, 31, 37, 42, 48, 50, 52

下标：0, 1, 2, 3, 4, 5, 6, 7, 8, 9

- 步骤一：设 low 为下标最小值 0，high 为下标最大值 9；
- 步骤二：通过 low 和 high 得到 mid， $mid = (low + high) / 2$ ，初始时 mid 为下标 4 (也可以=5，看具体算法实现)；
- 步骤三：mid=4 对应的数据值是 31， $31 < 48$ （我们要找的数字）；
- 步骤四：通过二分查找的思路，将 low 设置为 31 对应的下标 4，high 保持不变为 9，此时 mid 为 6；
- 步骤五：mid=6 对应的数据值是 42， $42 < 48$ （我们要找的数字）；
- 步骤六：通过二分查找的思路，将 low 设置为 42 对应的下标 6，high 保持不变为 9，此时 mid 为 7；
- 步骤七：mid=7 对应的数据值是 48， $48 == 48$ （我们要找的数字），查找结束；

通过 3 次二分查找 就找到了我们所要的数字，而顺序查找需 8 次。

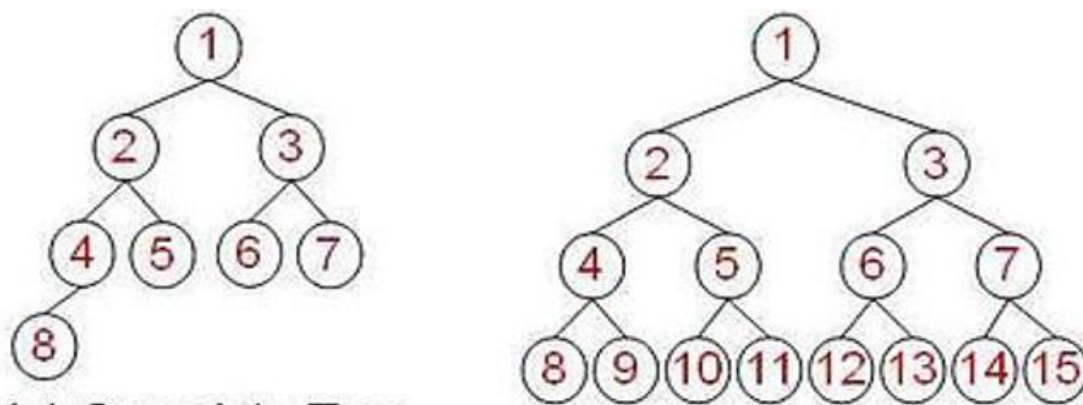
因此二分查找法的效率比顺序查找法要好（平均地来说）。如果说查 5 这条记录，顺序查找只需 1 次，而二分查找法需要 4 次。我们来看，对于上面 10 个数来说，顺序查找平均查找次数为 $(1+2+3+4+5+6+7+8+9+10)/10=5.5$ 次。而二分查找法为 $(4+3+2+4+3+1+4+3+2+3)/10=2.9$ 次。在最坏的情况下，顺序查找的次数为 10，而二分查找的次数为 4。

3.2.1.2. 二叉树系列

根据数组的特性我们知道，有最优的查找效率和空间需求；能够进行有序性相关操作，缺点也很明显，插入和删除操作很慢，所以为了结合链表插入的灵活性和有序数组查找的高效性，我们引入了二叉查找树。那么首先来看看什么是二叉树。**注意，本节课不是数据结构和算法的专门课，所以对牵涉到的数据结构和算法不会深入讲解，只讲解我们可能用到的部分。**

二叉树

每个节点至多只有二棵子树，左子树和右子树，次序不能颠倒。逻辑上二叉树有五种基本形态：空二叉树、只有一个根结点的二叉树、只有左子树、只有右子树、完全二叉树（特例为满二叉树）。遍历是对树的一种最基本的运算，所谓遍历二叉树，就是按一定的规则和顺序走遍二叉树的所有结点，使每一个结点都被访问一次，而且只被访问一次，有前序、中序、后序遍历。



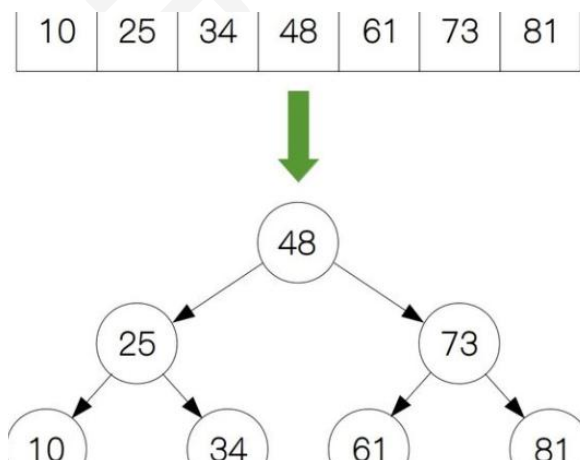
二叉查找(搜索)树

二叉查找树首先肯定是个二叉树，除此之外，还规定，在二叉查找树中，左子树的节点值总是小于根的节点值，右子树的节点值总是大于根的节点值，也就是左子树节点值<根的节点值<右子树节点值。因此可以通过中序遍历得到节点值的排序输出。以一个排序好的数组为例：

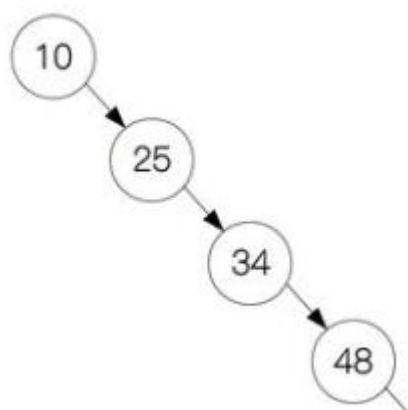
10、25、34、48、61、73、81

可以将这个数组转为二叉树结构，其中数组的中间元素作为树的根节点，左半部分的中间元素作为根节点的左孩子，右半部分的中间元素作为根节点的右孩子，第二层的节点以此类推，最后树的层数就会越来越多，这样我们就构建好了一颗二叉树。

同时这棵树，对于每个节点而言，其左孩子都小于它的父节点，右孩子都大于等于它的父节点。所以我们把这样的二叉树也是一个二叉查找树(binary search tree)，它的查找时间复杂度和二分查找一样， $O(\log N)$ 。



但是二叉查找树，如果设计不良，完全可以变成一颗极不平衡的二叉查找树：



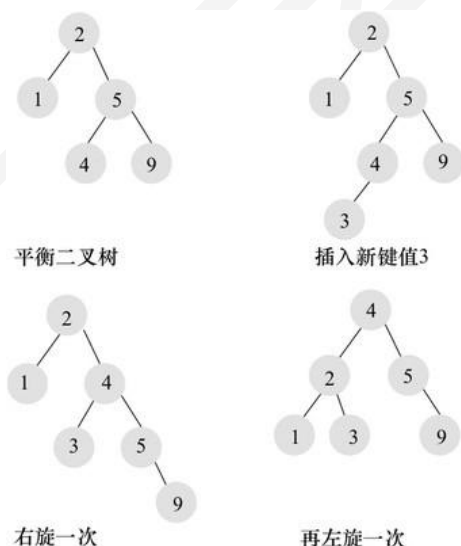
查找效率和顺序查找基本没差别了。因此若想最大性能地构造一棵二叉查找树，需要这棵二叉查找树是平衡的，从而引出了新的定义——平衡二叉树，或称为 AVL 树。

平衡二叉树（AVL-树）

平衡二叉树的定义如下：首先符合二叉查找树的定义，其次必须满足任何节点的两个子树的高度最大差为 1。

平衡二叉树的查找性能是比较高的，但不是最高的，只是接近最高性能。最好的性能需要建立一棵最优二叉树，但是最优二叉树的建立和维护需要大量的操作，因此，用户一般只需建立一棵平衡二叉树即可。

平衡二叉树的查询速度的确很快，但是维护一棵平衡二叉树的代价是非常大的。通常来说，需要 1 次或多次左旋和右旋来得到插入、更新和删除后树的平衡性。



平衡二叉树对一个数据库来说，有什么问题？因为二叉树每个节点最多只有两个直属子节点，所以当节点数比较多时，二叉树的高度增长很快，比如 1000 个节点时，树的高度差不多有 9 到 10 层。我们知道数据库是持久化的，数据是要从磁盘上读取的，一般的机械磁盘每秒至少可以做 100 次 IO，一次 IO 的时间基本上在 0.01 秒，1000 个节点在查找时就需要 0.1 秒，如果是 10000 个节点，

100000 个节点呢？所以对数据库来说，为了减少树的高度，提出了 B+树的数据结构。

3.2.1.3. B+树

B+树的定义

B+树和二叉树、平衡二叉树一样，都是经典的数据结构。B+树由 B 树和索引顺序访问方法演化而来，但是在现实使用过程中几乎已经没有使用 B 树的情况了。

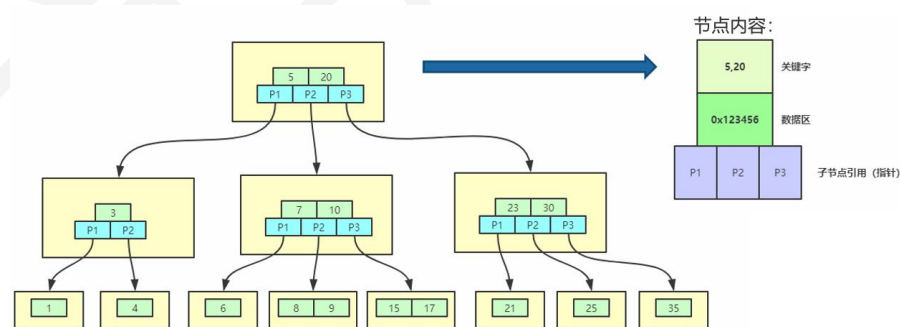
B+树的定义在很多数据结构书中都能找到，非常复杂，我们概略它的定义：

B+树是 B 树的一种变形形式，B+树上的叶子结点存储关键字以及相应记录的地址，叶子结点以上各层作为索引使用。一棵 m 阶的 B+树定义如下：

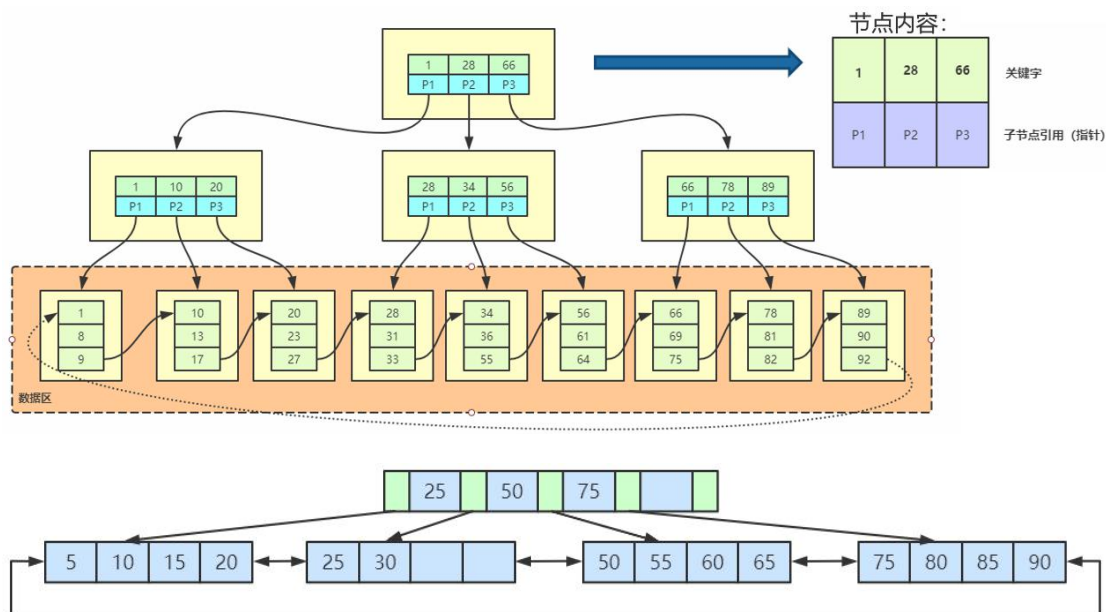
- 每个节点最多可以有 m 个元素；
- 除了根节点外，每个节点最少有 $(m/2)$ 个元素；
- 如果根节点不是叶节点，那么它最少有 2 个孩子节点；
- 所有的叶子节点都在同一层；
- 一个有 k 个孩子节点的非叶子节点有 $(k-1)$ 个元素，按升序排列；
- 某个元素的左子树中的元素都比它小，右子树的元素都大于或等于它；
- 非叶子节点只存放关键字和指向下一个孩子节点的索引，记录只存放在叶子节点中；
- 相邻的叶子节点之间用指针相连。

B+树的变体为 B*树，在 B+树的非根和非叶子结点再增加指向兄弟的指针；B*树定义了非叶子结点关键字个数至少为 $(2/3)*M$ ，即块的最低使用率为 $2/3$ （代替 B+树的 $1/2$ ）。

我们概要的了解下 B 树和 B+树。



B+树是为磁盘或其他直接存取辅助设备设计的一种平衡查找树。在 B+树中，所有记录节点都是按键值的大小顺序存放在同一层的叶子节点上，由各叶子节点指针进行连接。比如：



从上图我们可以归纳出 B+ 树的几个特征，在 B+ 树的简要定义中其实已经包括了：

- 1、相同节点数量的情况下，B+ 树高度远低于平衡二叉树；
- 2、非叶子节点只保存索引信息和下一层节点的指针信息，不保存实际数据记录；
- 3、每个叶子页（LeafPage）存储了实际的数据，比如上图中每个叶子页就存放了 4 条数据记录，当然可以更多，叶子节点由小到大（有序）串联在一起，叶子页中的数据也是排好序的；
- 4、索引节点指示该节点的左子树比这个索引值小，而右子树大于等于这个索引值。

注意：叶子节点中的数据在物理存储上完全可以是无序的，仅仅是在逻辑上有序（通过指针串在一起）。

B+ 树的插入

B+ 树的插入必须保证插入后叶子节点中的记录依然排序，同时需要考虑插入到 B+ 树的三种情况，每种情况都可能会导致不同的插入算法。

Leaf Page 不满，Index Page 不满，直接将记录插入到叶子节点。

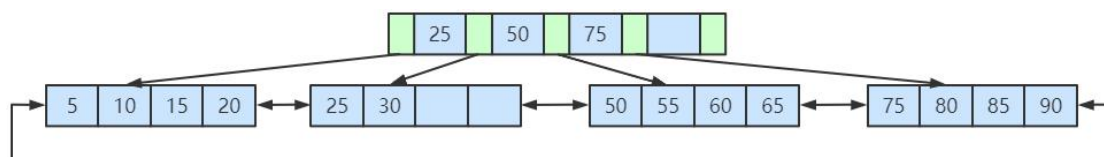
Leaf Page 满，Index Page 不满，

- 1) 拆分 Leaf Page
- 2) 将中间的节点放入到 Index Page 中
- 3) 小于中间节点的记录放左边
- 4) 大于或等于中间节点的记录放右边。

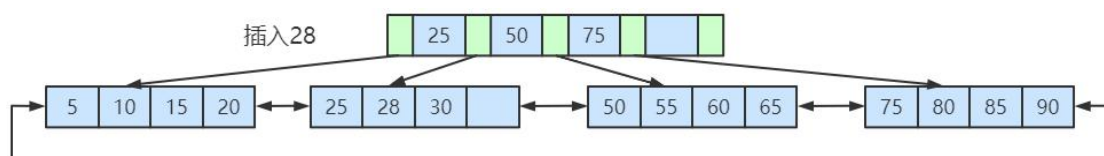
Leaf Page 满，Index Page 满，

- 1) 拆分 Leaf Page
- 2) 小于中间节点的记录放左边
- 3) 大于或等于中间节点的记录放右边
- 4) 拆分 Index Page
- 5) 小于中间节点的记录放左边

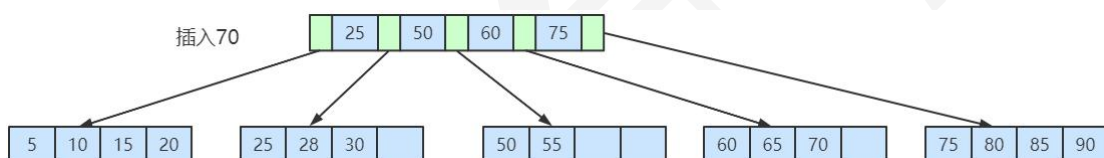
- 6)大于中间节点的记录放右边
7)中间节点放入上一层 Index Page。
比如已有 B+树:



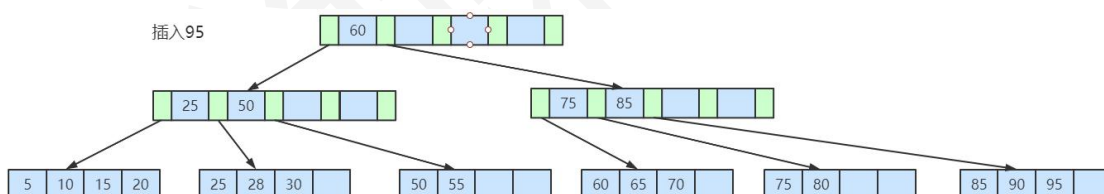
插入 28，发现当前 Leaf Page 和 Index Page 都没有满，直接进行插入即可：



接着再插入 70 这个键值，这时原先的 Leaf Page 已经满了，但是 Index Page 还没有满，符合表 5-1 的第二种情况，这时插入 Leaf Page 后的情况为 55、55、60、65、70，并根据中间的值 60 来拆分叶子节点：

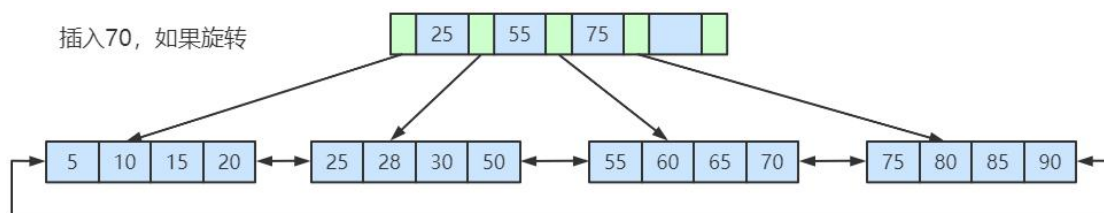


最后插入键值 95，这时 Leaf Page 和 Index Page 都满了，这时需要做两次拆分：



可以看到，不管怎么变化，B+树总是会保持平衡。但是为了保持平衡对于新插入的键值可能需要做大量的拆分页(split)操作。因为 B+树结构主要用于磁盘，页的拆分意味着磁盘的操作，所以应该在可能的情况下尽量减少页的拆分操作。因此，B+树同样提供了类似于平衡二叉树的旋转（Rotation）功能。

旋转发生在 Leaf Page 已经满，但是其的左右兄弟节点没有满的情况下。这时 B+树并不会急于去做拆分页的操作，而是将记录移到所在页的兄弟节点上。比如前面的插入键值 70，其实 B+树并不会急于去拆分叶子节点，而是去做旋转操作：



从上图可以看到，采用旋转操作使 B+树减少了一次页的拆分操作，同时这棵 B+树的高度依然还是 2。

B+树的删除

B+树使用填充因子（fill factor）来控制树的删除变化，50%是填充因子可设的最小值。B+树的删除操作同样必须保证删除后叶子节点中的记录依然排序，同插入一样，B+树的删除操作同样需要考虑三种情况，与插入不同的是，删除会加入填充因子的考量。

叶子节点大于填充因子，中间节点大于填充因子，直接将记录从叶子节点删除，如果该节点还是 IndexPage 的节点，用该节点的右节点代替。

叶子节点大于填充因子，中间节点小于填充因子，合并叶子节点和它的兄弟节点，同时更新 Index Page。

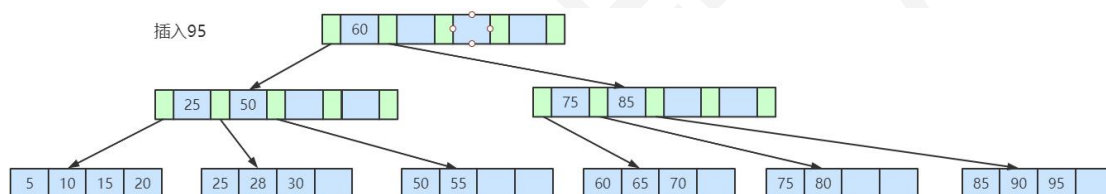
叶子节点小于填充因子，中间节点小于填充因子，

1)合并叶子节点和它的兄弟节点

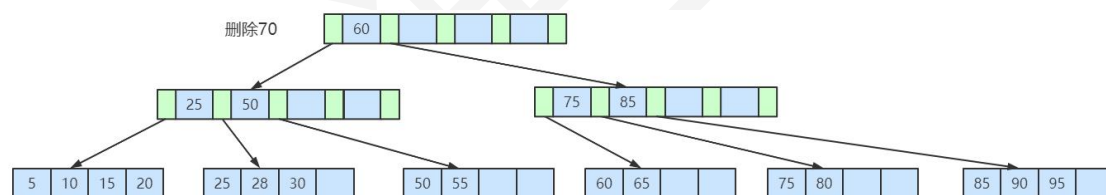
2)更新 Index Page

3)合并 Index Page 和它的兄弟节点。

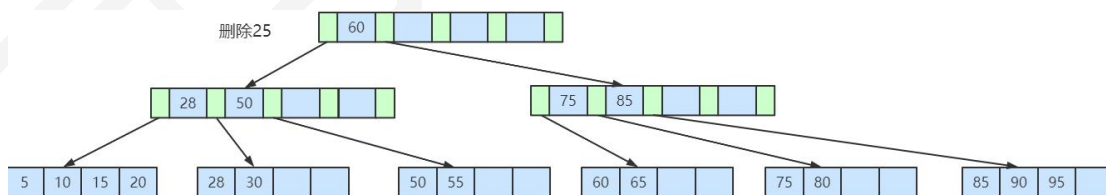
用我们最后生成的 B+树来演示：



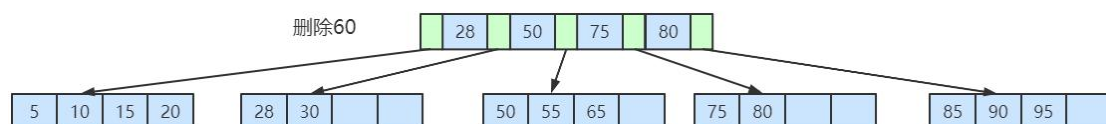
首先删除键值为 70 的这条记录，B+树变为：



删除键值为 25 的记录，但是该值还是 Index Page 中的值，因此在删除 Leaf Page 中的 25 后，还应将 25 的右兄弟节点的 28 更新到 Page Index 中，B+树变为：



删除 Leaf Page 中键值为 60 的记录后，Fill Factor 小于 50%，这时需要做合并操作，同样，在删除 Index Page 中相关记录后需要做 IndexPage 的合并操作，B+树变为：

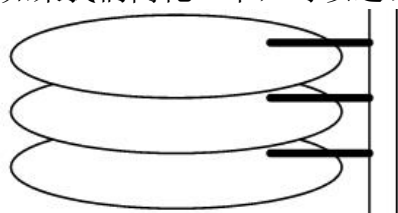


磁盘和 B+树

为什么关系型数据库都选择了 B+树,这个和磁盘的特性有着非常大的关系。



如果我们简化一下，可以这么看



一个磁盘由大小相同且同轴的圆形盘片组成，磁盘可以转动（各个磁盘必须同步转动）。在磁盘的一侧有磁头支架，磁头支架固定了一组磁头，每个磁头负责存取一个磁盘的内容。磁头不能转动，但是可以沿磁盘半径方向运动。

盘片被划分成一系列同心环，圆心是盘片中心，每个同心环叫做一个磁道，所有半径相同的磁道组成一个柱面。磁道被沿半径线划分成一个个小的段，每个段叫做一个扇区，每个扇区是磁盘的最小存储单元也是最小读写单元。现在磁盘扇区一般是 512 个字节~4k 个字节。

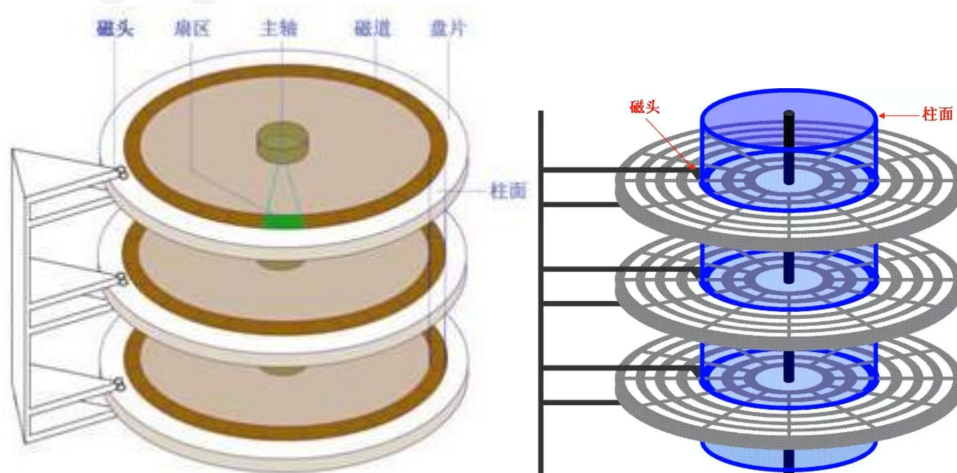
磁盘上数据必须用一个三维地址唯一标示：柱面号、盘面号、扇区号。

读/写磁盘上某一指定数据需要下面步骤：

(1) 首先移动臂根据柱面号使磁头移动到所需要的柱面上，这一过程被称为定位或查找。

(2) 所有磁头都定位到磁道上后，这时根据盘面号来确定指定盘面上的具体磁道。

(3) 盘面确定以后，盘片开始旋转，将指定块号的磁道段移动至磁头下。经过上面步骤，指定数据的存储位置就被找到。这时就可以开始读/写操作了。



可以看见，磁盘读取依靠的是机械运动，分为寻道时间、旋转延迟、传输时间三个部分，这三个部分耗时相加就是一次磁盘 IO 的时间，一般大概 9ms 左右。寻道时间（seek）是将读写磁头移动至正确的磁道上所需要的时间，这部分时间代价最高；旋转延迟时间（rotation）是磁盘旋转将目标扇区移动到读写磁头下方所需的时间，取决于磁盘转速；数据传输时间（transfer）是完成传输数据所需要的时间，取决于接口的数据传输率，在纳秒级，远小于前两部分消耗时间。磁盘读取时间成本是访问内存的几百倍到几万倍之间。

为了提高效率，要尽量减少磁盘 I/O。为了达到这个目的，磁盘往往不是严格按需读取，而是每次都会预读，即使只需要一个字节，磁盘也会从这个位置开始，顺序向后读取一定长度的数据放入内存，这个称之为**预读**。这样做的理论依据是计算机科学中著名的局部性原理：

当一个数据被用到时，其附近的数据也通常会马上被使用。

程序运行期间所需要的数据通常比较集中。

由于磁盘顺序读取的效率很高（不需要寻道时间，只需很少的旋转时间），一般来说，磁盘的顺序读的效率是随机读的 40 到 400 倍都有可能，顺序写是随机写的 10 到 100 倍（SSD 盘则差距要小的多，顺序读写的效率是随机读写的 7 到 10 倍，但是有评测表明机械硬盘的顺序写性能稍优于 SSD。总的来说 Mysql 数据库如果由硬盘由机械的换成 SSD 的，性能会有很大的提升），因此对于具有局部性的程序来说，预读可以提高 I/O 效率。

预读的长度一般为页（page）的整倍数。页是计算机管理存储器的逻辑块，硬件及操作系统往往将主存和磁盘存储区分割为连续的大小相等的块，每个存储块称为一页，页大小通常为 4k 当然也有 16K 的，主存和磁盘以页为单位交换数据。当程序要读取的数据不在主存中时，会触发一个缺页异常，此时系统会向磁盘发出读盘信号，磁盘会找到数据的起始位置并向后连续读取一页或几页载入内存中，然后异常返回，程序继续运行。

按照磁盘的这种性质，如果是一个页存放一个 B+树的节点，自然是可存放很多的数据的，比如 InnoDB 里，默认定义的 B+树的节点大小是 16KB，这就是说，假如一个 Key 是 8 个字节，那么一个节点可以存放大约 1000 个 Key，意味着 B+数可以有 1000 个分叉。同时 InnoDB 每一次磁盘 I/O，读取的都是 16KB 的整数倍的数据。也就是说 InnoDB 在节点的读写上是可以充分利用磁盘顺序 IO 的高速读写特性。

同时按照 B+树逻辑结构来说，在叶子节点一层，所有记录的主键按照从小到大的顺序排列，并且形成了一个双向链表。同一层的非叶子节点也互相串联，形成了一个双向链表。那么在实际读写的时候，很大的概率相邻的节点会放在相邻的页上，又可以充分利用磁盘顺序 IO 的高速读写特性。所以我们对 MySQL 优化的一大方向就是尽可能的多让数据顺序读写，少让数据随机读写。

总结 B+树的作用

- 在块设备上，通过 B+树可以有效的存储数据；
- 所有记录都存储在叶子节点上，非叶子(non-leaf)存储索引(keys)信息；而且记录按照索引列的值由小到大排好了序。
- B+树含有非常高的扇出（fanout），通常超过 100，在查找一个记录时，可以有效的减少 IO 操作；

Tips:

扇出 是每个索引节点 (Non-LeafPage) 指向每个叶子节点 (LeafPage) 的指

针;

扇出数 = 索引节点 (Non-LeafPage) 可存储的最大关键字个数 + 1

3.2.2. InnoDB 中的索引

InnoDB 中的索引自然也是按照 B+树来组织的, 前面我们说过 B+树的叶子节点用来放数据的, 但是放什么数据呢? 索引自然是要放的, 因为 B+树的作用本来就是就是为了快速检索数据而提出的一种数据结构, 不放索引放什么呢? 但是数据库中的表, 数据才是我们真正需要的数据, 索引只是辅助数据, 甚至于一个表可以没有自定义索引。InnoDB 中的数据到底是如何组织的?

3.2.2.1. 聚集索引/聚簇索引

InnoDB 中使用了聚集索引, 就是将表的主键用来构造一棵 B+树, 并且将整张表的行记录数据存放在该 B+树的叶子节点中。也就是所谓的索引即数据, 数据即索引。由于聚集索引是利用表的主键构建的, 所以每张表只能拥有一个聚集索引。

聚集索引的叶子节点就是数据页。换句话说, 数据页上存放的是完整的每行记录。因此聚集索引的一个优点就是: 通过过聚集索引能获取完整的整行数据。另一个优点是: 对于主键的排序查找和范围查找速度非常快。

如果我们没有定义主键呢? MySQL 会使用唯一性索引, 没有唯一性索引, MySQL 也会创建一个隐含列 RowID 来做主键, 然后用这个主键来建立聚集索引。

主键列	1	2	3	4	5	6	...	...	...	...	...	99
	其他列	其他列	其他列	其他列	其他列	其他列	其他列	其他列	其他列	其他列	其他列	其他列

3.2.2.2. 辅助索引/二级索引

上边介绍的聚簇索引只能在搜索条件是主键值时才能发挥作用, 因为 B+树中的数据都是按照主键进行排序的, 那如果我们想以别的列作为搜索条件怎么办? 我们一般会建立多个索引, 这些索引被称为辅助索引/二级索引。

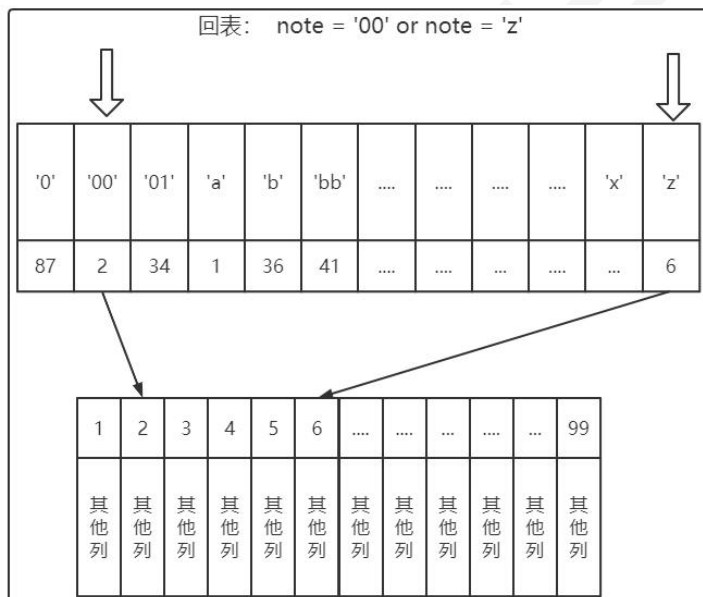
对于辅助索引 (Secondary Index, 也称二级索引、非聚集索引), 叶子节点并不包含行记录的全部数据。叶子节点除了包含键值以外, 每个叶子节点中的索引行中还包含了一个书签 (bookmark)。该书签用来告诉 InnoDB 存储引擎哪里可以找到与索引相对应的行数据。因此 InnoDB 存储引擎的辅助索引的书签就是相应行数据的聚集索引键。

索引列 note	'0'	'00'	'01'	'a'	'b'	'bb'					'x'	'z'
主键列	87	2	34	1	36	41					...	10

比如辅助索引 index(note)，那么叶子节点中包含的数据就包括了(主键、note)。

回表

辅助索引的存在并不影响数据在聚集索引中的组织，因此每张表上可以有多个辅助索引。当通过辅助索引来寻找数据时，InnoDB 存储引擎会遍历辅助索引并通过叶级别的指针获得指向主键索引的主键，然后再通过主键索引(聚集索引)来找到一个完整的行记录。这个过程也被称为**回表**。也就是根据辅助索引的值查询一条完整的用户记录需要使用到 2 棵 B+树——一次辅助索引，一次聚集索引。



为什么我们还需要一次回表操作呢?直接把完整的用户记录放到辅助索引 d 的叶子节点不就好了么?如果把完整的用户记录放到叶子节点是可以不用回表，但是太占地地方了，相当于每建立一棵 B+树都需要把所有的用户记录再都拷贝一遍，这就有点太浪费存储空间了。而且每次对数据的变化要在所有包含数据的索引中全部都修改一次，性能也非常低下。

很明显，回表的记录越少，性能提升就越高，需要回表的记录越多，使用二级索引的性能就越低，甚至让某些查询宁愿使用全表扫描也不使用二级索引。

那什么时候采用全表扫描的方式，什么时候使用采用二级索引 + 回表的方式去执行查询呢?这个就是查询优化器做的工作，查询优化器会事先对表中的记录计算一些统计数据，然后再利用这些统计数据根据查询的条件来计算出需要回表的记录数，需要回表的记录数越多，就越倾向于使用全表扫描，反之倾向于使用二级索引 + 回表的方式。具体怎么算的，我们后面会详细说到。

3.2.2.3. 联合索引/复合索引

前面我们对索引的描述,隐含了一个条件,那就是构建索引的字段只有一个,但实践工作中构建索引的完全可以是多个字段。所以,将表上的多个列组合起来进行索引我们称之为联合索引或者复合索引,比如 `index(a, b)` 就是将 `a, b` 两个列组合起来构成一个索引。

千万要注意一点,建立联合索引只会建立 1 棵 B+树,多个列分别建立索引会分别以每个列则建立 B+树,有几个列就有几个 B+树,比如, `index(note)`、`index(b)`,就分别对 `note, b` 两个列各构建了一个索引。

`index(note, b)` 在索引构建上,包含了两个意思:

- 1、先把各个记录按照 `note` 列进行排序。
- 2、在记录的 `note` 列相同的情况下,采用 `b` 列进行排序

索引列 note	'01'	'01'	'01'	'02'	'03'	'03'					'30'	'30'
索引列 b	'0'	'00'	'aa'	'00'	'00'	'bb'					'x'	'z'
主键列	87	2	34	1	36	41			...		...	10

3.2.2.4. 覆盖索引/索引覆盖

既然多个列可以组合起来构建为联合索引,那么辅助索引自然也可以由多个列组成。

InnoDB 存储引擎支持覆盖索引(covering index, 或称索引覆盖),即从辅助索引中就可以得到查询的记录,而不需要查询聚集索引中的记录。使用覆盖索引的一个好处是辅助索引不包含整行记录的所有信息,故其大小要远小于聚集索引,因此可以减少大量的 I/O 操作。所以记住,覆盖索引并不是索引类型的一种。


```
mysql> show engine innodb status\G
***** 1. row *****
  Type: InnoDB
  Name:
  Status:
=====
2021-03-25 14:57:00 0xf3dfc116700 INNODB MONITOR OUTPUT
=====
Per second averages calculated from the last 32 seconds
-----
BACKGROUND THREAD
-----
srv_master_thread loops: 134 srv_active, 0 srv_shutdown, 513170 srv_idle
srv_master_thread log flush and writes: 513304
-----
SEMAPHORES
-----
OS WAIT ARRAY INFO: reservation count 81
OS WAIT ARRAY INFO: signal count 80
RW-shared spins 0, rounds 142, OS waits 71
RW-excl spins 0, rounds 90, OS waits 2
RW-sx spins 5, rounds 150, OS waits 5
Spin rounds per wait: 142.00 RW-shared, 90.00 RW-excl, 30.00 RW-sx
-----
TRANSACTIONS
=====
```

```
-----
INSERT BUFFER AND ADAPTIVE HASH INDEX
-----
Ibuf: size 1, free list len 0, seg size 2, 0 merges
merged operations:
  insert 0, delete mark 0, delete 0
discarded operations:
  insert 0, delete mark 0, delete 0
Hash table size 34673, node heap has 0 buffer(s)
Hash table size 34673, node heap has 1 buffer(s)
Hash table size 34673, node heap has 0 buffer(s)
Hash table size 34673, node heap has 0 buffer(s)
Hash table size 34673, node heap has 1 buffer(s)
Hash table size 34673, node heap has 1 buffer(s)
Hash table size 34673, node heap has 0 buffer(s)
Hash table size 34673, node heap has 2 buffer(s)
0.00 hash searches/s, 0.00 non-hash searches/s
```

哈希索引只能用来搜索等值的查询, 如 `SELECT* FROM table WHERE index co=xxx`。而对于其他查找类型, 如范围查找, 是不能使用哈希索引的,

因此这里出现了 `non-hash searches/s` 的情况。通过 `hash searches: non-hash searches` 可以大概了解使用哈希索引后的效率。

由于 AHI 是由 InnoDB 存储引擎控制的, 因此这里的信息只供我们参考。不过我们可以通过观察 `SHOW ENGINE INNODB STATUS` 的结果及参数 `innodb_adaptive_hash_index` 来考虑是禁用或启动此特性, 默认 AHI 为开启状态。

什么时候需要禁用呢？如果发现监视索引查找和维护哈希索引结构的额外开销远远超过了自适应哈希索引带来的性能提升就需要关闭这个功能。

同时在 MySQL 5.7 中，自适应哈希索引搜索系统被分区。每个索引都绑定到一个特定的分区，每个分区都由一个单独的 latch 锁保护。分区由 innodb_adaptive_hash_index_parts 配置选项控制。在早期版本中，自适应哈希索引搜索系统受到单个 latch 锁的保护，这可能成为繁重工作负载下的争用点。innodb_adaptive_hash_index_parts 默认情况下，该选项设置为 8。最大设置为 512。当然禁用或启动此特性和调整分区个数这个应该是 DBA 的工作，我们了解即可。

3.2.2.6. 全文检索之倒排索引

什么是全文检索（Full-Text Search）？它是将存储于数据库中的整本书或整篇文章中的任意内容信息查找出来的技术。它可以根据需要获得全文中有关章、节、段、句、词等信息，也可以进行各种统计和分析。我们比较熟知的 Elasticsearch、Solr 等就是全文检索引擎，底层都是基于 Apache Lucene 的。

举个例子，现在我们要保存唐宋诗词，数据库中我们会怎么设计？诗词表我们可能的设计如下：

朝代	作者	诗词年代	标题	诗词全文
唐	李白		静夜思	床前明月光，疑是地上霜。 举头望明月，低头思故乡。
宋	李清照		如梦令	常记溪亭日暮，沉醉不知归路，兴尽晚回舟，误入藕花深处。争渡，争渡，惊起一滩鸥鹭。
....		...		

要根据朝代或者作者寻找诗，都很简单，比如“select 诗词全文 from 诗词表 where 作者=‘李白’”，如果数据很多，查询速度很慢，怎么办？我们可以在对应的查询字段上建立索引加速查询。

但是如果我们现在有个需求：要求找到包含“望”字的诗词怎么办？用

“select 诗词全文 from 诗词表 where 诗词全文 like ‘%望%’”，这个意味着要扫描库中的诗词全文字段，逐条比对，找出所有包含关键词“望”字的记录。基本上，数据库中一般的 SQL 优化手段都是用不上的。数量少，大概性能还能接受，如果数据量稍微大点，就完全无法接受了，更何况在互联网这种海量数据的情况下呢？怎么解决这个问题呢，用倒排索引。

比如现在有：

蜀道难（唐）李白 蜀道之难难于上青天，侧身西望长咨嗟。

静夜思（唐）李白 举头望明月，低头思故乡。

春台望（唐）李隆基 暇景属三春，高台聊四望。

鹤冲天(宋)柳永 黄金榜上，偶失龙头望。明代暂遗贤，如何向？未遂风云便，争不恣狂荡。何须论得丧？才子词人，自是白衣卿相。烟花巷陌，依约丹青屏障。幸有意中人，堪寻访。且恁偎红翠，风流事，平生畅。青春都一饷。忍把浮名，换了浅斟低唱！

都有望字，于是我们可以这么保存

序号	关键字	蜀道难	静夜思	春台望	鹤冲天
1	望	有	有	有	有

如果查哪个诗词中包含上，怎么办，上述的表格可以继续填入新的记录

序号	关键字	蜀道难	静夜思	春台望	鹤冲天
1	望	有	有	有	有
2	上	有			有

其实，上述诗词的中每个字都可以作为关键字，然后建立关键字和文档之间的对应关系，也就是标识关键字被哪些文档包含。

所以，倒排索引就是，将文档中包含的关键字全部提取处理，然后再将关键字和文档之间的对应关系保存起来，最后再对关键字本身做索引排序。用户在检索某一个关键字是，先对关键字的索引进行查找，再通过关键字与文档的对应关系找到所在文档。

在存储在关系型数据库中的数据，需要我们事先分析将数据拆分为不同的字段，而在 es 这类的存储中，需要应用程序根据规则自动提取关键字，并形成对应关系。

这些预先提取的关键字，在全文检索领域一般被称为 term（词项），文档的词项提取在 es 中被称为文档分析，这是全文检索很核心的过程，必须要区分哪些是词项，哪些不是，比如很多场景下，apple 和 apples 是同一个东西，望和看其实是同一个动作。

从 InnoDB 1.2.x 版本开始，InnoDB 存储引擎开始支持全文检索，对应的 MySQL 版本是 5.6.x 系列。不过 MySQL 从设计之初就是关系型数据库，存储引擎虽然支持全文检索，整体架构上对全文检索支持并不好而且限制很多，比如每张表只能有一个全文检索的索引，不支持没有单词界定符（delimiter）的语言，如中文、日语、韩语等。

所以如果有大批量或者专门的全文检索需求，还是应该选择专门的全文检索引擎，毕竟 Elastic 靠着全文检索起家，然后产品化、公司化后依赖全文检索不断扩充产品线和应用场景，并推出商业版本的解决方案然后融资上市，现在的市值已达 100 亿美元（2021/03/24-纽约证券交易所中的市值 99.84 亿美元）。

具体如何使用 InnoDB 存储引擎的全文检索，请自行查阅相关官方文档或者书籍，我们不做任何讲解和技术支持。

Elastic 发家小故事

多年前，一个叫做 Shay Banon 的刚结婚不久的失业开发者，由于妻子要去伦敦学习厨师，他便跟着也去了。在他找工作的过程中，为了给妻子构建一个食谱的搜索引擎，他开始构建一个早期版本的 Lucene。

直接基于 Lucene 工作会比较困难，所以 Shay 开始抽象 Lucene 代码以便 Java 程序员可以在应用中添加搜索功能。他发布了他的第一个开源项目，叫做“Compass”。

后来 Shay 找到一份工作，这份工作处在高性能和内存数据网络的分布式环境中，因此高性能的、实时的、分布式的搜索引擎也是理所当然需要的。然后他决定重写 Compass 库使其成为一个独立的服务叫做 Elasticsearch。

第一个公开版本出现在 2010 年 2 月，在那之后 Elasticsearch 已经成为 Github 上最受欢迎的项目之一，代码贡献者超过 300 人。一家主营 Elasticsearch 的公司就此成立，他们一边提供商业支持一边开发新功能，不过 Elasticsearch 将永远开源且对所有人可用。

3.2.3. 深入思考索引在查询中的使用

索引在查询中的作用到底是什么？在我们的查询中发挥着什么样的作用呢？

请记住：

1、一个索引就是一个 B+树，索引让我们的查询可以快速定位和扫描到我们需要的数据记录上，加快查询的速度。

2、一个 select 查询语句在执行过程中一般最多能使用一个二级索引，即使在 where 条件中用了多个二级索引。

3.2.3.1. 扫描区间

对于某个查询来说，最简单粗暴的执行方案就是扫描表中的所有记录，判断每一条记录是否符合搜索条件。如果符合，就将其发送到客户端，否则就跳过该记录。这就是全表扫描。

对于使用 InnoDB 存储引擎的表来说，全表扫描意味着从聚簇索引第一个叶子节点的第一条记录开始，沿着记录所在的单向链表向后扫描，直到最后一个叶子节点的最后一条记录。虽然全表扫描是一种很笨的执行方案，但却是一种万能的执行方案，所有的查询都可以使用这种方案来执行，只是效率不高。

我们有了索引，利用 B+树查找索引列值等于某个值的记录，这样可以明显减少需要扫描的记录数量。由于 B+树叶子节点中的记录是按照索引列值由小到大的顺序排序的，所以即使只扫描某个区间或者某些区间中的记录也可以明显减少需要扫描的记录数量。比如下面这个查询语句：

```
SELECT * FROM order_exp WHERE id >= 3 AND id <= 99;
```

这个语句其实是想查找 id 值在[3,99]区间中的所有聚簇索引记录。我们可以通过聚簇索引对应的 B+树快速地定位到 id 值为 3 的那条聚簇索引记录，然后沿

着记录所在的单向链表向后扫描,直到某条聚簇索引记录的 id 值不在[3,99]区间中为止。

与全表扫描相比,扫描 id 值在[3,99]区间中的记录已经很大程度地减少了需要扫描的记录数量,所以提升了查询效率。其实所谓的全表扫描,我们可以理解为扫描的区间是[负无穷, 正无穷]或者[第一条记录, 最后一条记录]。

再看下面这个查询语句:

```
SELECT * FROM order_exp WHERE id in(3,9) OR (id>=23 AND id<= 99);
```

这里有几个扫描区间? 三个, 两个单独扫描区间[3,3]、[9,9], 一个范围扫描区间[23,99]。

再看下面这个查询语句:

```
SELECT * FROM order_exp WHERE order_no <'DD00_10S' AND expire_time>
'2021-03-22 18:28:28' AND order_note > '7 排';
```

这个语句里, order_no 和 expire_time 都有索引, order_note 没有索引, 那会有两个扫描区间吗? 并不会, 请记住, 一个 Select 查询语句在执行过程中一般最多能使用一个二级索引。那么也就是说:

如果用 idx_order_no 执行查询, 那扫描区间就是[第一条记录, 'DD00_10S'], expire_time> '2021-03-22 18:28:28' AND order_note > '7 排'只能成为普通的搜索或者说判定条件。

如果说用 idx_expire_time 执行查询, 那扫描区间就是['2021-03-22 18:28:28', 最后一条记录], order_no <'DD00_10S' AND order_note > '7 排'只能成为普通的搜索或者说判定条件。

无论用哪个索引执行查询, 都需要获取到索引中的记录后, 进行回表, 获取到完整的用户记录后再根据判定条件判断这条记录是否满足 SQL 语句的要求。

3.2.3.2. 范围区间扫描

其实对于 B+树索引来说, 只要索引列和常数使用=、<=>、IN、NOT IN、IS NULL、IS NOT NULL、>、<、>=、<=、BETWEEN、!= (不等于也可以写成<>) 或者 LIKE 操作符连接起来, 就可以产生一个区间。

1、IN 操作符的效果和若干个等值匹配操作符`= `之间用`OR`连接起来是一样的, 也就是说会产生多个单点区间, 比如下边这两个语句的效果是一样的:

```
SELECT * FROM order_exp WHERE insert_time IN (2021-03-22 18:23:42, yyyy);
SELECT * FROM order_exp WHERE insert_time= 2021-03-22 18:23:42 OR
insert_time = yyyy;
```

2、!=产生的扫描区间呢? 比如

```
SELECT * FROM order_exp WHERE order_no != 'DD00_9S'
```

此时使用 idx_expire_time 执行查询时对应的扫描区间就是[第一条记录 , 'DD00_9S']和['DD00_9S',最后一条记录]。

3、LIKE 操作符比较特殊, 只有在匹配完整的字符串或者匹配字符串前缀时才产生合适的扫描区间。

对于某个索引列来说，字符串前缀相同的记录在由记录组成的单向链表中肯定是相邻的。比如我们有一个搜索条件是 `note LIKE 'b%'`，对于二级索引 `idx_note` 来说，所有字符串前缀为 'b' 的二级索引记录肯定是相邻的。这也就意味着我们只要定位到 `idx_note` 值的字符串前缀为 'b' 的第一条记录，就可以沿着记录所在的单向链表向后扫描，直到某条二级索引记录的字符串前缀不为 'b' 为止。

索引列 note	'0'	'00'	'01'	'a'	'b'	'bb'	'bc'	'c'			'x'	'z'
主键列	87	2	34	1	36	41	52	25	...		...	10

很显然，`note LIKE 'b%'` 形成的扫描区间相当于 ['b', 'c')。

不过在日常的工作中，一个查询的 `WHERE` 子句可能有很多个小的搜索条件，这些搜索条件需要使用 `AND` 或者 `OR` 操作符连接起来，我们来看看怎么从由 `AND` 或 `OR` 组成的复杂搜索条件中提取出正确的范围区间。

3.2.3.3. 所有搜索条件都可以使用某个索引的情况

有时候每个搜索条件都可以使用到某个索引，比如下边这个查询语句：

```
SELECT * FROM order_exp WHERE order_no > 'DD00_6S' AND order_no > 'DD00_9S';
```

这个查询中的搜索条件都可以使用到 `idx_order_no`，也就是说每个搜索条件都对应着一个 `idx_order_no` 的范围区间。这两个小的搜索条件使用 `AND` 连接起来，也就是要取两个范围区间的交集，两者交集当然就是 `order_no > 'DD00_9S'` 了，也就是说上边这个查询使用 `idx_order_no` 的范围区间就是 ('DD00_9S', 最后一条记录)。

再看一下使用 `OR` 将多个搜索条件连接在一起的情况：

```
SELECT * FROM order_exp WHERE order_no > 'DD00_6S' OR order_no > 'DD00_9S';
```

`OR` 意味着需要取各个范围区间的并集，所以上边这个查询使用 `idx_expire_time` 的范围区间就是 ('DD00_6S', 最后一条记录)。

3.2.3.4. 有的搜索条件无法使用索引的情况

比如下边这个查询：

```
SELECT * FROM order_exp WHERE expire_time > '2021-03-22 18:35:09' AND order_note = 'abc';
```

请注意，这个查询语句中能利用的索引只有 `idx_expire_time` 一个，而 `idx_expire_time` 这个二级索引的记录中又不包含 `order_note` 这个字段，所以在使用二级索引 `idx_expire_time` 定位记录的阶段用不到 `order_note = 'abc'` 这个条件，

这个条件是在回表获取了完整的用户记录后才使用的，而范围区间是为了到索引中取记录中提出的概念，所以在确定范围区间的时候不需要考虑 `order_note = 'abc'` 这个条件。

我们把上边的查询中用不到 `idx_expire_time` 的搜索条件化简之后就是这样：

```
SELECT * FROM order_exp WHERE expire_time > '2021-03-22 18:35:09';
```

也就是说最上边那个查询使用 `idx_expire_time` 的范围区间就是：('2021-03-22 18:35:09', 最后一条记录)。

再来看一下使用 OR 的情况：

```
SELECT * FROM order_exp WHERE expire_time > '2021-03-22 18:35:09' OR  
order_note = 'abc';
```

这条语句在搜索时可以化简为：

```
SELECT * FROM order_exp ;
```

这也就是说如果我们使用 `idx_expire_time` 执行查询的话，对应的范围区间就是 [第一条记录, 最后一条记录]，也就是需要将全部二级索引的记录进行回表，这个代价肯定比直接全表扫描都大了。也就是说一个使用到索引的搜索条件和没有使用该索引的搜索条件使用 OR 连接起来后是无法使用该索引的。为什么？道理很简单，`idx_expire_time` 这个二级索引的记录中不包含 `order_note` 这个字段，那就说，即使二级索引 `idx_expire_time` 中找到了满足 `expire_time > '2021-03-22 18:35:09'` 的记录，是无法判定 `order_note` 是否满足 `order_note = 'abc'` 的，又因为 OR 条件，所以必须要在主键索引中从第一条记录到最后一条记录逐条判定 `order_note` 是否等于 'abc'。

3.2.3.5. 复杂搜索条件下找出范围匹配的区间

有的查询的搜索条件可能特别复杂，比方说下边这个：

```
SELECT * FROM order_exp WHERE (order_no > 'DD00_9S' AND expire_time =  
'2021-03-22 18:35:09') OR (order_no < 'DD00_6S' AND order_no > 'DD00_9S') OR  
(order_no LIKE '%0S' AND order_no > 'DD00_12S' AND (expire_time < '2021-03-22  
18:28:28' OR order_note = 'abc')) ;
```

分析一下：

首先查看 WHERE 子句中的搜索条件都涉及到了哪些列，哪些列可能使用到索引。

这个查询的搜索条件涉及到了 `order_no`、`expire_time`、`order_note` 这 3 个列，然后 `order_no` 列有二级索引 `idx_order_no`，`expire_time` 列有二级索引 `idx_expire_time`。

对于那些可能用到的索引，分析它们的范围区间。

使用 `idx_order_no` 执行查询

我们需要把那些用不到该索引的搜索条件暂时移除掉。上边的查询中除了有关 `expire_time` 和 `order_note` 列不能使用到 `idx_order_no` 索引外，`order_no LIKE '%0S'` 也使用不到索引。

如果条件太复杂，看着演化怕出错，我们可以把所有用不到的搜索条件视为 True 来进行中间替换，所以把这些搜索条件替换为 True 之后的样子就是这样：

```
(order_no > 'DD00_9S' AND TRUE ) OR  
(order_no < 'DD00_6S' AND order_no > 'DD00_9S') OR  
(TRUE AND order_no > 'DD00_12S' AND (TRUE OR TRUE))
```

再化简：

```
(order_no > 'DD00_9S') OR  
(order_no < 'DD00_6S' AND order_no > 'DD00_9S') OR  
(order_no > 'DD00_12S')
```

接下来替换掉永远为 TRUE 或 FALSE 的条件

因为符合 `order_no < 'DD00_6S' AND order_no > 'DD00_9S'` 永远为 FALSE，所以上边的搜索条件可以被写成这样：

```
(order_no > 'DD00_9S') OR (order_no > 'DD00_12S')
```

很明显，两者使用 OR 操作符连接起来的，意味着要取并集，所以最终的结果化简的到的区间就是：`order_no > 'DD00_12S'`。也就是说：上边那个复杂搜索条件的查询语句如果使用 `idx_order_no` 索引执行查询的话，需要把满足 `order_no > 'DD00_12S'` 的二级索引记录都取出来，然后拿着这些记录的 id 再进行回表，得到完整的用户记录之后再使用其他的搜索条件进行过滤。记住，我们说的是如果使用 `idx_order_no` 索引执行查询，不代表 MySQL 一定会使用，因为 MySQL 需要做整体评估，才能确定是否使用这个索引还是别的索引，或者是干脆全表扫描。

使用 `idx_expire_time` 执行查询

我们需要把那些用不到该索引的搜索条件暂时使用 TRUE 条件替换掉，其中有关 `order_no` 和 `order_note` 的搜索条件都需要被替换掉，替换结果就是：

```
(TRUE AND expire_time = '2021-03-22 18:35:09' ) OR  
(TRUE AND TRUE) OR  
(TRUE AND TRUE AND (expire_time < '2021-03-22 18:28:28' OR TRUE))
```

安装布尔运算的规则，`expire_time < '2021-03-22 18:28:28' OR TRUE` 的结果肯定是 TRUE，也就是说化简之后的搜索条件成这样了：

```
expire_time = '2021-03-22 18:35:09' OR TRUE
```

这个化简之后的结果就更简单了：TRUE

这个结果也就意味着如果我们要使用 `idx_expire_time` 索引执行查询语句的话，需要扫描 `idx_expire_time` 二级索引的所有记录，然后再回表，这种情况下为啥 MySQL 不直接全表扫描呢？所以一定不会使用 `idx_expire_time` 索引的。

Tips:上面这个 SQL 语句，执行全表扫描的代价大概是 2169，用 `idx_order_no` 索引的代价大概是 6211，所以，实际执行的时候，MySQL 会选择全表扫描。至于代价怎么算来的，我们后面会学到。

3.2.3.6. 使用联合索引执行查询时对应的扫描区间

联合索引的索引列包含多个列，B+树每一层页面以及每个页面中的记录采用的排序规则较为复杂，以 order_exp 表的 u_idx_day_status 联合索引为例，它采用的排序规则如下所示：

先按照 insert_time 列的值进行排序。

在 insert_time 列的值相同的情况下，再按照 order_status 列的值进行排序。

在 insert_time 和 order_status 列的值都相同的情况下，再按照 expire_time 列的值进行排序。

insert_time		'2021-03-22 18:34:55'	'2021-03-22 18:34:55'	'2021-03-22 18:34:55'		'2021-03-22 18:34:56'	'2021-03-22 18:34:57'	'2021-03-22 18:34:57'	
order_status		0	0	1		-1	1	1	
expire_time		'2021-03-22 18:35:13'	'2021-03-22 18:35:14'	'2021-03-22 18:35:12'		'2021-03-22 18:35:14'	'2021-03-22 18:35:01'	'2021-03-22 18:35:10'	
主键列	87	2	34	1		26	26	15	

对于下边这个查询 Q1 来说：

Q1: SELECT * FROM order_exp WHERE insert_time = '2021-03-22 18:34:55';

由于二级索引记录是先按照 insert_time 列的值进行排序的，所以所有符合 insert_time = '2021-03-22 18:34:55' 条件的记录肯定是相邻的，我们可以定位到第一条符合 insert_time = '2021-03-22 18:34:55' 条件的记录，然后沿着记录所在的单向链表向后扫描，直到某条记录不符合 insert_time = '2021-03-22 18:34:55' 条件为止(当然，对于获取到的每一条二级索引记录都要执行回表操作)。

也就是说，如果我们使用 u_idx_day_status 索引执行查询 Q1 时，对应的扫描区间就是['2021-03-22 18:34:55', '2021-03-22 18:34:55']，形成这个扫描区间的条件就是 insert_time = '2021-03-22 18:34:55'。

对于下边这个查询 Q2 来说：

Q2: SELECT * FROM order_exp WHERE insert_time = '2021-03-22 18:34:55' AND order_status = 0;

由于二级索引记录是先按照 insert_time 列的值进行排序的；在 insert_time 列的值相等的情况下，再按照 order_status 列进行排序。所以符合 insert_time = '2021-03-22 18:34:55' AND order_status = 0 条件的二级索引记录肯定是相邻的，我们可以定位到第一条符合 insert_time='2021-03-22 18:34:55' AND order_status=0 条件的记录，然后沿着记录所在的链表向后扫描，直到某条记录不符合 insert_time='2021-03-22 18:34:55' 条件或者 order_status=0 条件为止。

也就是说，如果我们使用 u_idx_day_status 索引执行查询 Q2 时，可以形成扫描区间[('2021-03-22 18:34:55', 0), ('2021-03-22 18:34:55', 0)], 形成这个扫描区间的条件就是 insert_time = '2021-03-22 18:34:55' AND order_status = 0。

对于下边这个查询 Q3 来说：

Q3: SELECT * FROM order_exp WHERE insert_time = '2021-03-22 18:34:55' AND order_status = 0 AND expire_time = '2021-03-22 18:35:13';

由于二级索引记录是先按照 insert_time 列的值进行排序的；在 insert_time 列的值相等的情况下，再按照 order_status 列进行排序；在 insert_time 和 order_status 列的值都相等的情况下，再按照 expire_time 列进行排序。所以符合 insert_time = '2021-03-22 18:34:55' AND order_status = 0 AND expire_time = '2021-03-22 18:35:13' 条件的二级索引记录肯定是相邻的，我们可以定位到第一条符合 insert_time='2021-03-22 18:34:55' AND order_status=0 AND expire_time='2021-03-22 18:35:13' 条件的记录，然后沿着记录所在的链表向后扫描，直到某条记录不符合 insert_time='2021-03-22 18:34:55' 条件或者 order_status=0 条件或者 expire_time='2021-03-22 18:35:13' 条件为止。如果我们使用 u_idx_day_status 索引执行查询 Q3 时，可以形成扫描区间[('2021-03-22 18:34:55', 0, '2021-03-22 18:35:13'), ('2021-03-22 18:34:55', 0, '2021-03-22 18:35:13')], 形成这个扫描区间的条件就是 insert_time = '2021-03-22 18:34:55' AND order_status = 0 AND expire_time = '2021-03-22 18:35:13'。

对于下边这个查询 Q4 来说：

Q4: SELECT * FROM order_exp WHERE insert_time < '2021-03-22 18:34:55';

由于二级索引记录是先按照 insert_time 列的值进行排序的，所以所有符合 insert_time < '2021-03-22 18:34:55' 条件的记录肯定是相邻的，我们可以定位到第一条符合 insert_time < '2021-03-22 18:34:55' 条件的记录(其实就是 u_idx_day_status 索引第一个叶子节点的第一条记录)，然后沿着记录所在的链表向前扫描，直到某条记录不符合 insert_time < '2021-03-22 18:34:55' 为止。

也就是说，如果我们使用 u_idx_day_status 索引执行查询 Q4 时，可以形成扫描区间(第一条记录, '2021-03-22 18:34:55')，形成这个扫描区间的条件就是 insert_time < '2021-03-22 18:34:55'。

对于下边这个查询 Q5 来说：

Q5: SELECT * FROM order_exp WHERE insert_time = '2021-03-22 18:34:55' AND order_status >= 0 ;

由于二级索引记录是先按照 insert_time 列的值进行排序的；在 insert_time 列的值相等的情况下，再按照 order_status 列进行排序。也就是说在符合 insert_time = '2021-03-22 18:34:55' 条件的二级索引记录中，是按照 order_status 列的值进行排序的，那么此时符合 insert_time = '2021-03-22 18:34:55' AND order_status >= 0 ; 条件的二级索引记录肯定是相邻的。我们可以定位到第一条符合 insert_time = '2021-03-22 18:34:55' AND order_status >= 0 ; 条件的记录，然后沿着记录所在的链表向后扫描，直到某条记录不符合 insert_time='2021-03-22 18:34:55' 条件或者 order_status >= 0 条件为止。

也就是说，如果我们使用 u_idx_day_status 索引执行查询 Q5 时，可以形成扫描区间，条件就是 insert_time = '2021-03-22 18:34:55' AND order_status >= 0 ;。

对于下边这个查询 Q6 来说：

Q6: SELECT * FROM order_exp WHERE order_status = 1;

由于二级索引记录不是直接按照 order_status 列的值排序的，所以符合 order_status = 1 的二级索引记录可能并不相邻，也就意味着我们不能通过这个

order_status = 1 搜索条件来减少需要扫描的记录数量。在这种情况下，我们是不使用 u_idx_day_status 索引执行查询的。

对于下边这个查询 Q7 来说：

```
Q7: SELECT * FROM order_exp WHERE insert_time = '2021-03-22 18:34:55' AND
expire_time = '2021-03-22 18:35:12';
```

由于二级索引记录是先按照 insert_time 列的值进行排序的，所以符合 insert_time = '2021-03-22 18:34:55' 条件的二级索引记录肯定是相邻的，但是对于符合 insert_time = '2021-03-22 18:34:55' 条件的二级索引记录来说，并不是直接按照 expire_time 列进行排序的，也就是说我们不能根据搜索条件 expire_time = '2021-03-22 18:35:12' 来进一步减少需要扫描的记录数量。那么如果我们使用 u_idx_day_status 索引执行查询的话，可以定位到第一条符合 insert_time='2021-03-22 18:34:55' 条件的记录，然后沿着记录所在的单向链表向后扫描，直到某条记录不符合 insert_time = '2021-03-22 18:34:55' 条件为止。所以在使用 u_idx_day_status 索引执行查询 Q7 的过程中，对应的扫描区间其实是 ['2021-03-22 18:34:55', '2021-03-22 18:34:55']，形成该扫描区间的搜索条件是 insert_time = '2021-03-22 18:34:55'，与 expire_time = '2021-03-22 18:35:12' 无关。

对于下边这个查询 Q8 来说：

```
Q8: SELECT * FROM order_exp WHERE insert_time < '2021-03-22 18:34:57' AND
order_status = 1;
```

由于二级索引记录是先按照 insert_time 列的值进行排序的，所以符合 insert_time < '2021-03-22 18:34:57' 条件的二级索引记录肯定是相邻的，但是对于符合 insert_time < '2021-03-22 18:34:57' 条件的二级索引记录来说，并不是直接按照 order_status 列进行排序的，也就是说我们不能根据搜索条件 order_status = 0 来进一步减少需要扫描的记录数量。那么如果我们使用 u_idx_day_status 索引执行查询的话，可以定位到第一条符合 insert_time 的记录，其实就是 u_idx_day_status 索引第一个叶子节点的第一条记录，所以在 u_idx_day_status 索引执行查询 Q8 的过程中，对应的扫描区间其实是 [第一条记录, '2021-03-22 18:34:57']。

3.2.5. 简单了解 MyISAM 中的索引

至此，我们介绍的都是 InnoDB 存储引擎中的索引方案，为了内容的完整性，以及各位可能在面试的时候遇到这类的问题，我们有必要再简单介绍一下 MyISAM 存储引擎中的索引方案。我们知道 InnoDB 中索引即数据，也就是聚簇索引的那棵 B+ 树的叶子节点中已经把所有完整的用户记录都包含了，而 MyISAM 的索引方案虽然也使用树形结构，但是却将索引和数据分开存储的。

MyISAM 将表中的记录按照记录的插入顺序单独存储在一个文件中，称之为数据文件。这个文件并不划分为若干个数据页，有多少记录就往这个文件中塞多少记录就成了。我们可以通过行号而快速访问到一条记录。

由于在插入数据的时候并没有刻意按照主键大小排序，所以我们并不能在这些数据上使用二分法进行查找。

使用 MyISAM 存储引擎的表会把索引信息另外存储到一个称为索引文件的另一个文件中。MyISAM 会单独为表的主键创建一个索引，只不过在索引的叶子节点中存储的不是完整的用户记录，而是主键值+行号的组合。也就是先通过索引找到对应的行号，再通过行号去找对应的记录！

这一点和 InnoDB 是完全不相同的，在 InnoDB 存储引擎中，我们只需要根据主键值对聚簇索引进行一次查找就能找到对应的记录，而在 MyISAM 中却需要进行一次回表操作，意味着 MyISAM 中建立的索引相当于全部都是二级索引！

如果有需要的话，我们也可以对其它的列分别建立索引或者建立联合索引，原理和 InnoDB 中的索引差不多，不过在叶子节点处存储的是相应的列+行号。这些索引也全部都是二级索引。

```
-rw-r----- 1 mysql mysql 8556 Mar 20 23:27 test_myisam.frm
-rw-r----- 1 mysql mysql 0 Mar 20 23:27 test_myisam.MYD
-rw-r----- 1 mysql mysql 1024 Mar 20 23:27 test_myisam.MYI
```

3.2.6. 创建和删除索引的语句

那我们如何使用 SQL 语句去建立这种索引呢？InnoDB 和 MyISAM 会自动为主键或者声明为 UNIQUE 的列去自动建立 B+树索引，但是如果我们想为其他的列建立索引就需要我们显式的去指明。为啥不自动为每个列都建立个索引呢？我们待会说。

查看索引

```
SHOW INDEX FROM table_name\G
```

创建修改索引

```
CREATE TABLE 表名 (
```

```
    各种列的信息 . . . ,
```

```
    [KEY|INDEX] 索引名 (需要被索引的单个列或多个列)
```

```
)
```

```
CREATE [UNIQUE ] INDEX indexName ON mytable(columnname(length));
```

```
ALTER TABLE 表名 ADD [UNIQUE ] INDEX [indexName] ON
(columnname(length))
```

删除索引

```
DROP INDEX [indexName] ON mytable;
```

```
ALTER TABLE 表名 DROP [INDEX|KEY] 索引名;
```

3.2.7. 索引的代价

世界上从来没有只有好处没有坏处的东西，如果你有，请你一定要告诉我，让我也感受一下。虽然索引是个好东西，在学习如何更好的使用索引之前先要了解一下使用它的代价，它在空间和时间上都会拖后腿。

3.2.7.1. 空间上的代价

这个是显而易见的，每建立一个索引都要为它建立一棵 B+ 树，每一棵 B+ 树的每一个节点都是一个数据页，一个页默认会占用 16KB 的存储空间，一棵很大的 B+ 树由许多数据页组成会占据很多的存储空间。

3.2.7.2. 时间上的代价

每次对表中的数据进行增、删、改操作时，都需要去修改各个 B+ 树索引。而且我们讲过，B+ 树每层节点都是按照索引列的值从小到大的顺序排序而组成了双向链表。不论是叶子节点中的记录，还是非叶子内节点中的记录都是按照索引列的值从小到大的顺序而形成了一个单向链表。

而增、删、改操作可能会对节点和记录的排序造成破坏，所以存储引擎需要额外的时间进行一些记录移位，页面分裂、页面回收的操作来维护好节点和记录的排序。如果我们建了许多索引，每个索引对应的 B+ 树都要进行相关的维护操作，这必然会对性能造成影响。

既然索引这么有用，我们是不是创建越多越好？既然索引有代价，我们还是别创建了吧？当然不是！那么我们应该怎么创建索引呢？

3.3. 高性能的索引创建策略

正确地创建和使用索引是实现高性能查询的基础。前面我们已经了解了索引相关的数据结构，各种类型的索引及其对应的优缺点。现在我们一起来看看如何真正地发挥这些索引的优势。

3.3.1. 索引列的类型尽量小

我们在定义表结构的时候要显式的指定列的类型，以整数类型为例，有 TINYINT、MEDIUMINT、INT、BIGINT 这么几种，它们占用的存储空间依次递增，我们这里所说的类型大小指的就是该类型表示的数据范围的大小。能表示的整数范围当然也是依次递增，如果我们想要对某个整数列建立索引的话，在表示的整数范围允许的情况下，尽量让索引列使用较小的类型，比如我们能使用 INT 就不要使用 BIGINT，能使用 MEDIUMINT 就不要使用 INT，这是因为：

- 数据类型越小，在查询时进行的比较操作越快（CPU 层次）
- 数据类型越小，索引占用的存储空间就越少，在一个数据页内就可以放下更多的记录，从而减少磁盘 I/O 带来的性能损耗，也就意味着可以把更多的数据页缓存在内存中，从而加快读写效率。

这个建议对于表的主键来说更加适用，因为不仅是聚簇索引中会存储主键值，其他所有的二级索引的节点处都会存储一份记录的主键值，如果主键适用更小的数据类型，也就意味着节省更多的存储空间和更高效的 I/O。

3.3.2. 索引选择性和前缀索引

3.3.2.1. 索引的选择性/离散性

创建索引应该选择选择性/离散性高的列。索引的选择性/离散性是指，不重复的索引值（也称为基数，**cardinality**）和数据表的记录总数（**N**）的比值，范围从 $1/N$ 到 1 之间。索引的选择性越高则查询效率越高，因为选择性高的索引可以让 MySQL 在查找时过滤掉更多的行。唯一索引的选择性是 1，这是最好的索引选择性，性能也是最好的。

很差的索引选择性就是列中的数据重复度很高，比如性别字段，不考虑政治正确的情况下，只有两者可能，男或女。那么我们在查询时，即使使用这个索引，从概率的角度来说，依然可能查出一半的数据出来。

比如下面这个表：

姓名	年龄	性别	区号
Peter	18	1	0731
Jack	20	1	0734
Allen	21	1	0731
Lisa	22	0	0746
Sam	23	1	0739
King	18	1	0745
James	20	1	0744
Mark	21	1	0731

哪列做为索引字段最好？当然是姓名字段，因为里面的数据没有任何重复，性别字段是最不适合做索引的，因为数据的重复度非常高。

怎么算索引的选择性/离散性？比如 `order_exp` 这个表：

```
select COUNT(DISTINCT order_no)/count(*) cnt from order_exp;
```

```
mysql> select COUNT(DISTINCT order_no)/count(*) cnt from order_exp;
+-----+
| cnt   |
+-----+
| 0.9676 |
+-----+
1 row in set (0.01 sec)
```

```
select COUNT(DISTINCT order_status)/count(*) cnt from order_exp;
```

```
mysql> select COUNT(DISTINCT order_status)/count(*) cnt from order_exp;
+-----+
| cnt   |
+-----+
| 0.0001 |
+-----+
1 row in set (0.00 sec)
```

很明显，`order_no` 列上的索引就比 `order_status` 列上的索引的选择性就要好，原因很简单，因为 `order_status` 列中的值只有 -1,0,1 三种。

3.3.2.2. 前缀索引

有时候需要索引很长的字符列，这会让索引变得大且慢。一个策略是前面提到过的模拟哈希索引。

模拟哈希索引：

`order_exp` 表中 `order_note` 字段很长，想把它作为一个索引，我们可以增加一个 `order_not_hash` 字段来存储 `order_note` 的哈希值，然后在 `order_not_hash` 上建立索引，相对于之前的索引速度会有明显提升，一个是对完整的 `order_note` 做索引，而后者则是用整数哈希值做索引，显然数字的比较比字符串的匹配要高效得多。

但是缺陷也很明显：

- 1、需要额外维护 `order_not_hash` 字段；
- 2、哈希算法的选择决定了哈希冲突的概率，不良的哈希算法会导致重复值很多；
- 3、不支持范围查找。

还可以做些什么改进呢？还可以索引开始的部分字符，这样可以大大节约索引空间，从而提高索引效率。但这样也会降低索引的选择性。一般情况下我们需要保证某个列前缀的选择性也是足够高的，以满足查询性能。（尤其对于 `BLOB`、`TEXT` 或者很长的 `VARCHAR` 类型的列，应该使用前缀索引，因为 MySQL 不允许索引这些列的完整长度）。

诀窍在于要选择足够长的前缀以保证较高的选择性，同时又不能太长（以便节约空间）。前缀应该足够长，以使得前缀索引的选择性接近于索引整个列。换句话说，前缀的“基数”应该接近于完整列的“基数”。

为了决定前缀的合适长度，可以找到最常见的值的列表，然后和最常见的前缀列表进行比较。

首先找到最常见的值的列表：

```
SELECT COUNT(*) AS cnt,order_note FROM order_exp GROUP BY order_note  
ORDER BY cnt DESC LIMIT 20;
```



```
mysql> SELECT COUNT(*) AS cnt,order_note FROM order_exp GROUP BY order_note ORDER BY cnt DESC LIMIT 20;
```

cnt	order_note
24	你好, 李焕英。7排23号,过期时长:DD00_23S
24	你好, 李焕英。7排17号,过期时长:DD00_17S
23	你好, 李焕英。7排24号,过期时长:DD00_24S
22	你好, 李焕英。7排7号,过期时长:DD00_7S
22	你好, 李焕英。7排10号,过期时长:DD00_10S
20	你好, 李焕英。7排5号,过期时长:DD00_5S
19	你好, 李焕英。7排14号,过期时长:DD00_14S
19	你好, 李焕英。7排19号,过期时长:DD00_19S
19	你好, 李焕英。7排6号,过期时长:DD00_6S
17	你好, 李焕英。7排18号,过期时长:DD00_18S
17	你好, 李焕英。7排9号,过期时长:DD00_9S
17	你好, 李焕英。7排16号,过期时长:DD00_16S
16	你好, 李焕英。7排12号,过期时长:DD00_12S
16	你好, 李焕英。7排21号,过期时长:DD00_21S
16	你好, 李焕英。7排13号,过期时长:DD00_13S
15	你好, 李焕英。7排8号,过期时长:DD00_8S
15	你好, 李焕英。7排15号,过期时长:DD00_15S
15	你好, 李焕英。7排22号,过期时长:DD00_22S
13	你好, 李焕英。7排20号,过期时长:DD00_20S
13	你好, 李焕英。7排11号,过期时长:DD00_11S

20 rows in set (0.06 sec)

通过观察数据的分布,我们可以大胆的猜测,前9个字符的选择性不会太好,从第10个开始应该还不错。试一试:

```
SELECT COUNT(DISTINCT LEFT(order_note,3))/COUNT(*) AS sel3,
COUNT(DISTINCT LEFT(order_note,4))/COUNT(*)AS sel4,
COUNT(DISTINCT LEFT(order_note,5))/COUNT(*) AS sel5,
COUNT(DISTINCT LEFT(order_note, 6))/COUNT(*) As sel6,
COUNT(DISTINCT LEFT(order_note, 7))/COUNT(*) As sel7,
COUNT(DISTINCT LEFT(order_note, 8))/COUNT(*) As sel8,
COUNT(DISTINCT LEFT(order_note, 9))/COUNT(*) As sel9,
COUNT(DISTINCT LEFT(order_note, 10))/COUNT(*) As sel10,
COUNT(DISTINCT LEFT(order_note, 11))/COUNT(*) As sel11,
COUNT(DISTINCT LEFT(order_note, 12))/COUNT(*) As sel12,
COUNT(DISTINCT LEFT(order_note, 13))/COUNT(*) As sel13,
COUNT(DISTINCT LEFT(order_note, 14))/COUNT(*) As sel14,
COUNT(DISTINCT LEFT(order_note, 15))/COUNT(*) As sel15,
COUNT(DISTINCT order_note)/COUNT(*) As total
FROM order_exp;
```

```
mysql> SELECT COUNT(DISTINCT LEFT(order_note,3))/COUNT(*) AS sel3,
-> COUNT(DISTINCT LEFT(order_note,4))/COUNT(*)AS sel4,
-> COUNT(DISTINCT LEFT(order_note,5))/COUNT(*) AS sel5,
-> COUNT(DISTINCT LEFT(order_note, 6))/COUNT(*) As sel6,
-> COUNT(DISTINCT LEFT(order_note, 7))/COUNT(*) As sel7,
-> COUNT(DISTINCT LEFT(order_note, 8))/COUNT(*) As sel8,
-> COUNT(DISTINCT LEFT(order_note, 9))/COUNT(*) As sel9,
-> COUNT(DISTINCT LEFT(order_note, 10))/COUNT(*) As sel10,
-> COUNT(DISTINCT LEFT(order_note, 11))/COUNT(*) As sel11,
-> COUNT(DISTINCT LEFT(order_note, 12))/COUNT(*) As sel12,
-> COUNT(DISTINCT LEFT(order_note, 13))/COUNT(*) As sel13,
-> COUNT(DISTINCT LEFT(order_note, 14))/COUNT(*) As sel14,
-> COUNT(DISTINCT LEFT(order_note, 15))/COUNT(*) As sel15,
-> COUNT(DISTINCT LEFT(order_note, 16))/COUNT(*) As sel16,
-> COUNT(DISTINCT order_note, 15)/COUNT(*) As total
-> FROM order_exp;
```

sel3	sel4	sel5	sel6	sel7	sel8	sel9	sel10	sel11	sel12	sel13	sel14	sel15	sel16	total
0.0008	0.0008	0.0008	0.0015	0.0107	0.0844	0.1628	0.3455	0.4723	0.6834	0.8564	0.9197	0.9592	0.9657	0.9676

1 row in set (0.14 sec)

可以看见，从第 10 个开始选择性的增加值很高，随着前缀字符的越来越多，选择度也在不断上升，但是增长到第 15 时，已经和第 14 没太大差别了，选择性提升的幅度已经很小了，都非常接近整个列的选择性了。

那么针对这个字段做前缀索引的话，从第 13 到第 15 都是不错的选择，甚至第 12 也不是不能考虑。当然不找到最常见的值的列表，直接计算前缀字符选择性也是可以的。

在上面的示例中，已经找到了合适的前缀长度，如何创建前缀索引：

```
ALTER TABLE order_exp ADD KEY (order_note(14));
```

建立前缀索引后查询语句并不需要更改：

```
select * from order_exp where order_note = 'xxxx' ;
```

前缀索引是一种能使索引更小、更快的有效办法，但另一方面也有其缺点 MySQL 无法使用前缀索引做 ORDER BY 和 GROUP BY，也无法使用前缀索引做覆盖扫描。

有时候后缀索引 (suffix index)也有用途（例如，找到某个域名的所有电子邮件地址）。MySQL 原生并不支持反向索引，但是可以把字符串反转后存储，并基于此建立前缀索引。可以通过触发器或者应用程序自行处理来维护索引。

3.3.2. 只为用于搜索、排序或分组的列创建索引

也就是说，只为出现在 WHERE 子句中的列、连接子句中的连接列创建索引，而出现在查询列表中的列一般就没必要建立索引了，除非是需要使用覆盖索引；又或者为出现在 ORDER BY 或 GROUP BY 子句中的列创建索引，这句话什么意思呢？比如：

```
SELECT * FROM order_exp ORDER BY insert_time, order_status,expire_time;
```

查询的结果集需要先按照 insert_time 值排序，如果记录的 insert_time 值相同，则需要按照 order_status 来排序，如果 order_status 的值相同，则需要按照 expire_time 排序。回顾一下联合索引的存储结构，u_idx_day_status 索引本身就是按照上述规则排好序的，所以直接从索引中提取数据，然后进行回表操作取出该索引中不包含的列就好了。

当然 ORDER BY 的子句后边的列的顺序也必须按照索引列的顺序给出，如果给出 ORDER BY order_status,expire_time, insert_time 的顺序，那也是用不了 B+树索引的，原因不用再说了吧。

```
SELECT insert_time, order_status,expire_time,count(*) FROM order_exp  
GROUP BY insert_time, order_status,expire_time;
```

这个查询语句相当于做了 3 次分组操作：

先把记录按照 insert_time 值进行分组，所有 insert_time 值相同的记录划分为一组。

将每个 insert_time 值相同的分组里的记录再按照 order_status 的值进行分组，将 order_status 值相同的记录放到一个小分组里。

再将上一步中产生的小分组按照 expire_time 的值分成更小的分组。

然后针对最后的分组进行统计，如果没有索引的话，这个分组过程全部需要在内存里实现，而如果有了索引的话，恰巧这个分组顺序又和我们的 `u_idx_day_status` 索引中的索引列的顺序是一致的，而我们的 B+树索引又是按照索引列排好序的，这不正好么，所以可以直接使用 B+树索引进行分组。和使用 B+树索引进行排序是一个道理，分组列的顺序也需要和索引列的顺序一致。

3.3.3. 多列索引

很多人对多列索引的理解都不够。一个常见的错误就是，为每个列创建独立的索引，或者按照错误的顺序创建多列索引。

我们遇到的最容易引起困惑的问题就是索引列的顺序。正确的顺序依赖于使用该索引的查询，并且同时需要考虑如何更好地满足排序和分组的需要。反复强调过，在一个多列 B-Tree 索引中，索引列的顺序意味着索引首先按照最左列进行排序，其次是第二列，等等。所以，索引可以按照升序或者降序进行扫描，以满足精确符合列顺序的 `ORDER BY`、`GROUP BY` 和 `DISTINCT` 等子句的查询需求。

所以多列索引的列顺序至关重要。对于如何选择索引的列顺序有一个经验法则：将选择性最高的列放到索引最前列。当不需要考虑排序和分组时，将选择性最高的列放在前面通常是很好的。这时候索引的作用只是用于优化 `WHERE` 条件的查找。在这种情况下，这样设计的索引确实能够最快地过滤出需要的行，对于在 `WHERE` 子句中只使用了索引部分前缀列的查询来说选择性也更高。

然而，性能不只是依赖于索引列的选择性，也和查询条件的有关。可能需要根据那些运行频率最高的查询来调整索引列的顺序，比如排序和分组，让这种情况下索引的选择性最高。

同时，在优化性能的时候，可能需要使用相同的列但顺序不同的索引来满足不同类型的查询需求。

3.3.4. 设计三星索引

3.3.4.1. 三星索引概念

对于一个查询而言，一个三星索引，可能是其最好的索引。

如果查询使用三星索引，一次查询通常只需要进行一次磁盘随机读以及一次窄索引片的扫描，因此其相应时间通常比使用一个普通索引的响应时间少几个数量级。

三星索引概念是在《*Relational Database Index Design and the optimizers*》一书中提出来的。原文如下：

The index earns one star if it places relevant rows adjacent to each other,
a second star if its rows are sorted in the order the query needs,
and a final star if it contains all the columns needed for the query.

索引将相关的记录放到一起则获得一星；

如果索引中的数据顺序和查找中的排列顺序一致则获得二星；

如果索引中的列包含了查询中需要的全部列则获得三星。

二星（排序星）：

在满足一星的情况下，当查询需要排序，`group by`、`order by`，如果查询所需的顺序与索引是一致的（索引本身是有序的），是不是就可以不用再另外排序了，一般来说排序可是影响性能的关键因素。

三星（宽索引星）：

在满足了二星的情况下，如果索引中所包含了这个查询所需的所有列（包括 `where` 子句 和 `select` 子句中所需的列，也就是覆盖索引），这样一来，查询就不再需要回表了，减少了查询的步骤和 IO 请求次数，性能几乎可以提升一倍。

一星按照原文稍微有点难以理解，其实它的意思就是：如果一个查询相关的索引行是相邻的或者至少相距足够靠近的话，必须扫描的索引片宽度就会缩至最短，也就是说，让索引片尽量变窄，也就是我们所说的索引的扫描范围越小越好。

这三颗星，哪颗最重要？第三颗星。因为将一个列排除在索引之外可能会导致很多磁盘随机读（回表操作）。第一和第二颗星重要性差不多，可以理解为第三颗星比重是 50%，第一颗星为 27%，第二颗星为 23%，所以在大部分的情况下，会先考虑第一颗星，但会根据业务情况调整这两颗星的优先度。

3.3.4.2. 达成三星索引

现在有表

```
create table customer(  
  cno int,  
  lname varchar(10),  
  fname varchar(10),  
  sex int,  
  weight int,  
  city varchar(10));
```

建立索引

```
create index idx_cust on customer(city,lname,fname,cno);
```

对于下面的 SQL 而言，这是个三星索引

```
select cno,fname from customer where lname='xx' and city='yy' order by  
fname;
```

来评估下：

第一颗星：所有等值谓词的列，是组合索引的开头的列，可以把索引片缩得很窄，符合。

第二颗星：`order by` 的 `fname` 字段在组合索引中且是索引自动排序好的，符合。

第三颗星：`select` 中的 `cno` 字段、`fname` 字段在组合索引中存在，符合。

3.3.4.3. 达不成三星索引

现在有表

```
CREATE TABLE `test` (  
    `id` int(11) NOT NULL AUTO_INCREMENT,  
    `user_name` varchar(100) DEFAULT NULL,  
    `sex` int(11) DEFAULT NULL,  
    `age` int(11) DEFAULT NULL,  
    `c_date` datetime DEFAULT NULL,  
    PRIMARY KEY (`id`),  
    ) ENGINE=InnoDB AUTO_INCREMENT=12 DEFAULT CHARSET=utf8;
```

SQL 语句如下：

```
select user_name,sex,age from test where user_name like 'test%' and sex =1  
ORDER BY age
```

如果我们建立索引(user_name,sex,age):

第三颗星，满足

第一颗星，满足

第二颗星，不满足，user_name 采用了范围匹配，sex 是过滤列，此时 age 列无法保证有序的。

上述我们看到，此时索引(user_name,sex,age)并不能满足三星索引中的第二颗星（排序）。

于是我们改改，建立索引(sex, age, user_name):

第一颗星，不满足，只可以匹配到 sex，sex 选择性很差，意味着是一个宽索引片，

第二颗星，满足，等值 sex 的情况下，age 是有序的，

第三颗星，满足，select 查询的列都在索引列中，

对于索引(sex,age, user_name)我们可以看到，此时无法满足第一颗星，窄索引片的需求。

以上 2 个索引，都是无法同时满足三星索引设计中的三个需求的，我们只能尽力满足 2 个。而在多数情况下，能够满足 2 颗星，已经能缩小很大的查询范围了，具体最终要保留那一颗星（排序星 or 窄索引片星），这个就需要看查询者自己的着重点了，无法给出标准答案。

3.3.5. 主键是很少改变的列

我们知道，行是按照聚集索引物理排序的，如果主键频繁改变(update)，物理顺序会改变，MySQL 要不断调整 B+树，并且中间可能会产生页面的分裂和合并等等，会导致性能会急剧降低。

3.3.6. 冗余和重复索引

MySQL 允许在相同列上创建多个索引，无论是有意还是无意的。MySQL 需要单独维护重复的索引，并且优化器在优化查询的时候也需要逐个地进行考虑，这会影响性能。重复索引是指在相同的列上按照相同的顺序创建的相同类型的索引。应该避免这样创建重复索引，发现以后也应该立即移除。

有时会在不经意间创建了重复索引，例如下面的代码：

```
CREATE TABLE test (  
    ID INT NOT NULL PRIMARY KEY,  
    A INT NOT NULL,  
    B INT NOT NULL,  
    UNIQUE(ID),  
    INDEX(ID)  
) ENGINE=InnoDB;
```

这里创建了一个主键，又加上唯一限制，然后再加上索引以供查询使用。事实上，MySQL 的唯一限制和主键限制都是通过索引实现的，因此，上面的写法实际上在相同的列上创建了三个重复的索引。通常并没有理由这样做，除非是在同一列上创建不同类型的索引来满足不同的查询需求。

冗余索引和重复索引有一些不同。如果创建了索引(A B)，再创建索引(A)就是冗余索引，因为这只是前一个索引的前缀索引。因此索引(AB)也可以当作索引(A)来使用（这种冗余只是对 B-Tree 索引来说的）。但是如果再创建索引 (B,A)，则不是冗余索引，索引(B)也不是，因为 B 不是索引(A,B)的最左前缀列。

已有的索引(A)，扩展为(A, ID)，其中 ID 是主键，对于 InnoDB 来说主键列已经包含在二级索引中了，所以这也是冗余的。

解决冗余索引和重复索引的方法很简单，删除这些索引就可以，但首先要做的是找出这样的索引。可以通过写一些复杂的访问 INFORMATION_SCHEMA 表的查询来找。

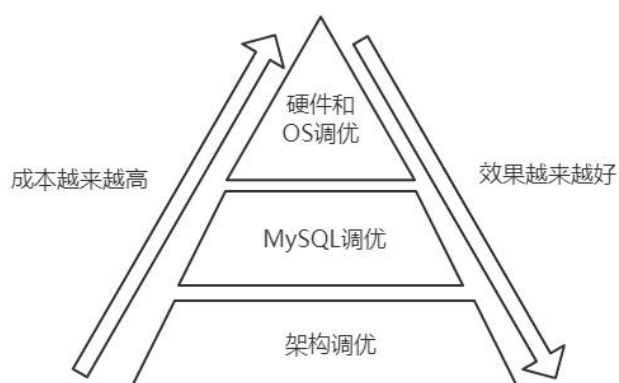
3.3.7. 删除未使用的索引

除了冗余索引和重复索引，可能还会有一些服务器永远不用的索引。这样的索引完全是累赘，建议考虑删除。

4. MySQL 性能优化

4.1. 总论

MySQL 性能优化其实是个很大的课题，在优化上存在着一个调优金字塔的说法：



很明显从图上可以看出，越往上走，难度越来越高，收益却是越来越小的。比如硬件和 OS 调优，需要对硬件和 OS 有着非常深刻的了解，仅仅就磁盘一项来说，一般非 DBA 能想到的调整就是 SSD 盘比用机械硬盘更好，但其实它至少包括了，使用什么样的磁盘阵列（RAID）级别、是否可以分散磁盘 IO、是否使用裸设备存放数据，使用哪种文件系统（目前比较推荐的是 XFS），操作系统的磁盘调度算法（目前比较推荐 deadline，对机械硬盘和 SSD 都比较合适。从内核 2.5 开始，默认的 I/O 调度算法是 Deadline，之后默认 I/O 调度算法为 Anticipatory，直到内核 2.6.17 为止，从内核 2.6.18 开始，CFQ 成为默认的 IO 调度算法，但 CFQ 并不推荐作为数据库服务器的磁盘调度算法。）选择，是否需要调整操作系统文件管理方面比如 atime 属性等等。

TIPS: 裸设备(raw device)，也叫裸分区（原始分区），是一种没有经过格式化，不被 Unix 通过文件系统来读取的特殊块设备文件。由应用程序负责对它进行读写操作。不经过文件系统的缓冲。它是不被操作系统直接管理的设备。这种设备少了操作系统这一层，I/O 效率更高。

TIPS: 小演示，查看磁盘调度算法：

```
# dmesg |grep -i scheduler
```

```
# df -m
```

```
# more /sys/block/vda/queue/scheduler
```

永久地修改 IO 调度算法，需要修改内核引导参数

```
[root@iZwz9j203ithc4guluvb2wZ ~]# dmesg |grep -i scheduler
[ 0.777484] io scheduler noop registered
[ 0.777487] io scheduler deadline registered (default)
[ 0.777507] io scheduler cfq registered
[ 0.777511] io scheduler mq-deadline registered
[ 0.777514] io scheduler kyber registered
[root@iZwz9j203ithc4guluvb2wZ ~]# df -m
Filesystem      1M-blocks  Used Available Use% Mounted on
/dev/vda1        40184    8350      29975  22% /
devtmpfs         909        0         909    0% /dev
tmpfs            920        0         920    0% /dev/shm
tmpfs            920        1         919    1% /run
tmpfs            920        0         920    0% /sys/fs/cgroup
tmpfs            184        0         184    0% /run/user/1000
tmpfs            184        0         184    0% /run/user/0
[root@iZwz9j203ithc4guluvb2wZ ~]# more /sys/block/vda/queue/scheduler
[mq-deadline] kyber none
```

所以在进行优化时，首先需要关注和优化的应该是架构，如果架构不合理，即使是 DBA 能做的事情其实也是比较有限的。

对于架构调优，在系统设计时首先需要充分考虑业务的实际情况，是否可以把不适合数据库做的事情放到数据仓库、搜索引擎或者缓存中去做；然后考虑写的并发量有多大，是否需要采用分布式；最后考虑读的压力是否很大，是否需要读写分离。对于核心应用或者金融类的应用，需要额外考虑数据安全因素，数据是否不允许丢失。

作为金字塔的底部的架构调优，采用更适合业务场景的架构能最大程度地提升系统的扩展性和可用性。在设计中进行垂直拆分能尽量解耦应用的依赖，对读压力比较大的业务进行读写分离能保证读性能线性扩展，而对于读写并发压力比较大的业务在 MySQL 上也有采用读写分离的大量案例。

作为金字塔的底部，在底层硬件系统、SQL 语句和参数都基本定型的情况下，单个 MySQL 数据库能提供的性能、扩展性等就基本定型了。但是通过架构设计和优化，却能承载几倍、几十倍甚至百倍于单个 MySQL 数据库能力的业务请求能力。

对于 MySQL 调优，需要确认业务表结构设计是否合理，SQL 语句优化是否足够，该添加的索引是否都添加了，是否可以剔除多余的索引等等。

最后确定系统、硬件有哪些地方需要优化，系统瓶颈在哪里，哪些系统参数需要调整优化，进程资源限制是否提到足够高；在硬件方面是否需要更换为具有更高 I/O 性能的存储硬件，是否需要升级内存、CPU、网络等。

如果在设计之初架构就不合理，比如没有进行读写分离，那么后期的 MySQL 和硬件、系统优化的成本就会很高，并且还不一定能最终解决问题。如果业务性能的瓶颈是由于索引等 MySQL 层的优化不够导致的，那么即使配置再高性能的 I/O 存储硬件或者 CPU 也无法支撑业务的全表扫描。

所以本章我们重点关注 MySQL 方面的调优，特别是索引。SQL/索引调优要求对业务和数据流非常清楚。在阿里巴巴内部，有三分之二的 DBA 是业务 DBA，从业务需求讨论到表结构审核、SQL 语句审核、上线、索引更新、版本迭代升级，甚至哪些数据应该放到非关系型数据库中，哪些数据放到数据仓库、搜索引擎或者缓存中，都需要这些 DBA 跟踪和复审。他们甚至可以称为数据架构师（Data Architecher）。

4.2. 查询性能优化

前面的章节我们知道如何设计最优的库表结构、如何建立最好的索引，这些对于高性能来说是必不可少的。但这些还不够—还需要合理的设计查询。如果查询写得很糟糕，即使库表结构再合理、索引再合适，也无法实现高性能。

4.2.1. 什么是慢查询

慢查询日志，顾名思义，就是查询花费大量时间的日志，是指 mysql 记录所有执行超过 `long_query_time` 参数设定的时间阈值的 SQL 语句的日志。该日志能为 SQL 语句的优化带来很好的帮助。默认情况下，慢查询日志是关闭的，要使用慢查询日志功能，首先要开启慢查询日志功能。如何开启，我们稍后再说。

4.2.2. 慢查询基础-优化数据访问

查询性能低下最基本的原因是访问的数据太多。大部分性能低下的查询都可以通过减少访问的数据量的方式进行优化。对于低效的查询，一般通过下面两个步骤来分析总是很有效：

1. 确认应用程序是否在检索大量超过需要的数据。这通常意味着访问了太多的行，但有时候也可能是访问了太多的列。
2. 确认 MySQL 服务器层是否在分析大量超过需要的数据行。

4.2.2.1. 请求了不需要的数据？

有些查询会请求超过实际需要的数据，然后这些多余的数据会被应用程序丢弃。这会给 MySQL 服务器带来额外的负担，并增加网络开销，另外也会消耗应用服务器的 CPU 和内存资源。比如：

查询不需要的记录

一个常见的错误是常常会误以为 MySQL 会只返回需要的数据，实际上 MySQL 却是先返回全部结果集再进行计算。我们经常会看到一些了解其他数据库系统的人会设计出这类应用程序。这些开发者习惯使用这样的技术，先使用 `SELECT` 语句查询大量的结果，然后获取前面的 `N` 行后关闭结果集（例如在新闻网站中取出 100 条记录，但是只是在页面上显示前面 10 条）。他们认为 MySQL 会执行查询，并只返回他们需要的 10 条数据，然后停止查询。实际情况是 MySQL 会查询出全部的结果集，客户端的应用程序会接收全部的结果集数据，然后抛弃其中大部分数据。最简单有效的解决方法就是在这样的查询后面加上 `LIMIT`。

总是取出全部列

每次看到 `SELECT *` 的时候都需要用怀疑的眼光审视，是不是真的需要返回全部的列？很可能不是必需的。取出全部列，会让优化器无法完成索引覆盖扫描这类优化，还会为服务器带来额外的 I/O、内存和 CPU 的消耗。因此，一些 DBA 是严格禁止 `SELECT *` 的写法的，这样做有时候还能避免某些列被修改带来的问题。

什么时候应该允许查询返回超过需要的数据？如果这种有点浪费数据库资源的方式可以简化开发，因为能提高相同代码片段的复用性，如果清楚这样做的性能影响，那么这种做法也是值得考虑的。如果应用程序使用了某种缓存机制，或者有其他考虑，获取超过需要的数据也可能有其好处，但不要忘记这样做的代价是什么。获取并缓存所有的列的查询，相比多个独立的只获取部分列的查询可能就更有好处。

重复查询相同的数据

不断地重复执行相同的查询，然后每次都返回完全相同的数据。比较好的方案是，当初次查询的时候将这个数据缓存起来，需要的时候从缓存中取出，这样性能显然会更好。

4.2.2.2. 是否在扫描额外的记录

在确定查询只返回需要的数据以后，接下来应该看看查询为了返回结果是否扫描了过多的数据。对于 MySQL，最简单的衡量查询开销的三个指标如下：

响应时间、扫描的行数、返回的行数

没有哪个指标能够完美地衡量查询的开销，但它们大致反映了 MySQL 在内部执行查询时需要访问多少数据，并可以大概推算出查询运行的时间。这三个指标都会记录到 MySQL 的慢日志中，所以检查慢日志记录是找出扫描行数过多的查询的好办法。

响应时间

响应时间是两个部分之和：服务时间和排队时间。

服务时间是指数据库处理这个查询真正花了多长时间。

排队时间是指服务器因为等待某些资源而没有真正执行查询的时间——可能是等 I/O 操作完成，也可能是等待行锁，等等。

当你看到一个查询的响应时间的时候，首先需要问问自己，这个响应时间是否是一个合理的值。概括地说，了解这个查询需要哪些索引以及它的执行计划是什么，然后计算大概需要多少个顺序和随机 I/O，再用其乘以在具体硬件条件下一次 I/O 的消耗时间。最后把这些消耗都加起来，就可以获得一个大概参考值来判断当前响应时间是不是一个合理的值。

扫描的行数和返回的行数

分析查询时，查看该查询扫描的行数是非常有帮助的。这在一定程度上能够说明该查询找到需要的数据的效率高不高。

理想情况下扫描的行数和返回的行数应该是相同的。但实际情况中这种“美事”并不多。例如在做一个关联查询时，服务器必须要扫描多行才能生成结果集中的一行。扫描的行数对返回的行数的比率通常很小，一般在 1:1 和 10:1 之间，不过有时候这个值也可能非常非常大。

扫描的行数和访问类型

在评估查询开销的时候，需要考虑一下从表中找到某一行数据的成本。MySQL 有好几种访问方式可以查找并返回一行结果。有些访问方式可能需要扫描很多行才能返回一行结果，也有些访问方式可能无须扫描就能返回结果。

在 EXPLAIN 语句中的 type 列反应了访问类型。访问类型有很多种，从全表扫描到索引扫描、范围扫描、唯一索引查询、常数引用等。这里列的这些，速度是从慢到快，扫描的行数也是从小到大。你不需要记住这些访问类型，但需要明白扫描表、扫描索引、范围访问和单值访问的概念。

如果查询没有办法找到合适的访问类型，那么解决的最好办法通常就是增加一个合适的索引，为什么索引对于查询优化如此重要了。索引让 MySQL 以最高效、扫描行数最少的方式找到需要的记录。

一般 MySQL 能够使用如下三种方式应用 WHERE 条件，从好到坏依次为：

- 1、在索引中使用 WHERE 条件来过滤不匹配的记录。这是在存储引擎层完成的。

2、使用索引覆盖扫描（在 Extra 列中出现了 Using index）来返回记录，直接从索引中过滤不需要的记录并返回命中的结果。这是在 MySQL 服务器层完成的，但无须再回表查询记录。

3、从数据表中返回数据，然后过滤不满足条件的记录（在 Extra 列中出现 Using Where）。这在 MySQL 服务器层完成，MySQL 需要先从数据表读出记录然后过滤。

好的索引可以让查询使用合适的访问类型，尽可能地只扫描需要的数据行。

如果发现查询需要扫描大量的数据但只返回少数的行，那么通常可以尝试下面的技巧去优化它：

1、使用索引覆盖扫描，把所有需要用的列都放到索引中，这样存储引擎无须回表获取对应行就可以返回结果了（在前面的章节中我们已经讨论过了）。

2、改变库表结构。例如使用单独的汇总表。

3、重写这个复杂的查询，让 MySQL 优化器能够以更优化的方式执行这个查询。

4.2.3. 重构查询的方式

在优化有问题的查询时，目标应该是找到一个更优的方法获得实际需要的结果——而不一定总是需要从 MySQL 获取一模一样的结果集。有时候，可以将查询转换一种写法让其返回一样的结果，但是性能更好。但也可以通过修改应用代码，用另一种方式完成查询，最终达到一样的目的。

4.2.3.1. 一个复杂查询还是多个简单查询

设计查询的时候一个需要考虑的重要问题是，是否需要将一个复杂的查询分成多个简单的查询。在传统实现中，总是强调需要数据库层完成尽可能多的工作，这样做的逻辑在于以前总是认为网络通信、查询解析和优化是一件代价很高的事情。但是这样的想法对于 MySQL 并不适用，MySQL 从设计上让连接和断开连接都很轻量级，在返回一个小的查询结果方面很高效。现代的网络速度比以前要快很多，无论是带宽还是延迟。在某些版本的 MySQL 上，即使在一个通用服务器上，也能够运行每秒超过 10 万的查询，即使是一个千兆网卡也能轻松满足每秒超过 2000 次的查询。所以运行多个小查询现在已经不是大问题了。

MySQL 内部每秒能够扫描内存中上百万行数据，相比之下，MySQL 响应数据给客户端就慢得多了。在其他条件都相同的时候，使用尽可能少的查询当然是更好的。但是有时候，将一个大查询分解为多个小查询是很有必要的。

不过，在应用设计的时候，如果一个查询能够胜任时还写成多个独立查询是不明智的。例如，应用对一个数据表做 10 次独立的查询来返回 10 行数据，每个查询返回一条结果，查询 10 次。

4.2.3.2. 切分查询

有时候对于一个大查询我们需要“分而治之”，将大查询切分成小查询，每个查询功能完全一样，只完成一小部分，每次只返回一小部分查询结果。

删除旧的数据就是一个很好的例子。定期地清除大量数据时，如果用一个大的语句一次性完成的话，则可能需要一次锁住很多数据、占满整个事务日志、耗尽系统资源、阻塞很多小的但重要的查询。将一个大的 `DELETE` 语句切分成多个较小的查询可以尽可能小地影响 MySQL 性能，同时还可以减少 MySQL 复制的延迟。

一次删除一万行数据一般来说是一个比较高效而且对服务器影响也最小的做法。同时，需要注意的是，如果每次删除数据后，都暂停一会儿再做下一次删除，这样也可以将服务器上原本一次性的压力分散到一个很长的时间段中，就可以大大降低对服务器的影响，还可以大大减少删除时锁的持有时间。

4.2.3.3. 分解关联查询

很多高性能的应用都会对关联查询进行分解。简单地，可以对每一个表进行一次单表查询，然后将结果在应用程序中进行关联。到底为什么要这样做？乍一看，这样做并没有什么好处，原本一条查询，这里却变成多条查询，返回的结果又是一模一样的。事实上，用分解关联查询的方式重构查询有如下的优势：

让缓存的效率更高。许多应用程序可以方便地缓存单表查询对应的结果对象。将查询分解后，执行单个查询可以减少锁的竞争。

在应用层做关联，可以更容易对数据库进行拆分，更容易做到高性能和可扩展。查询本身效率也可能会有所提升。

可以减少冗余记录的查询。在应用层做关联查询，意味着对于某条记录应用只需要查询一次，而在数据库中做关联查询，则可能需要重复地访问一部分数据。从这点看，这样的重构还可能减少网络和内存的消耗。

更进一步，这样做相当于在应用中实现了哈希关联，而不是使用 MySQL 的嵌套循环关联。某些场景哈希关联的效率要高很多。

在很多场景下，通过重构查询将关联放到应用程序中将会更加高效，这样的场景有很多，比如：当应用能够方便地缓存单个查询的结果的时候、当可以将数据分布到不同的 MySQL 服务器上的时候、当能够使用 `IN()` 的方式代替关联查询的时候、当查询中使用同一个数据表的时候。

4.2.4. 慢查询配置

我们已经知道慢查询日志可以帮助定位可能存在问题的 SQL 语句，从而进行 SQL 语句层面的优化。但是默认值为关闭的，需要我们手动开启。

`show VARIABLES like 'slow_query_log';`

```
mysql> show VARIABLES like 'slow_query_log';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| slow_query_log | OFF   |
+-----+-----+
1 row in set (0.00 sec)
```

开启：

set GLOBAL slow_query_log=1;

```
mysql> set GLOBAL slow_query_log = 1;
Query OK, 0 rows affected (0.00 sec)

mysql> show VARIABLES like 'slow_query_log';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| slow_query_log | ON    |
+-----+-----+
1 row in set (0.00 sec)
```

但是多慢算慢？MySQL 中可以设定一个阈值，将运行时间超过该值的所有 SQL 语句都记录到慢查询日志中。long_query_time 参数就是这个阈值。默认值为 10，代表 10 秒。

show VARIABLES like '%long_query_time%';

当然也可以设置

set global long_query_time=0; --默认 10 秒，这里为了演示方便设置为 0

```
mysql> show VARIABLES like '%long_query_time%';
+-----+-----+
| Variable_name | Value      |
+-----+-----+
| long_query_time | 10.000000 |
+-----+-----+
1 row in set (0.00 sec)
```

同时对于没有运行的 SQL 语句没有使用索引，则 MySQL 数据库也可以将这条 SQL 语句记录到慢查询日志文件，控制参数是：

show VARIABLES like '%log_queries_not_using_indexes%';

```
mysql> show VARIABLES like '%log_queries_not_using_indexes%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| log_queries_not_using_indexes | OFF   |
+-----+-----+
1 row in set (0.00 sec)
```

对于产生的慢查询日志，可以指定输出的位置，通过参数 log_output 来控制，可以输出到[TABLE][FILE][FILE, TABLE]。比如

set global log_output='FILE, TABLE'; 缺省是输出到文件，我们的配置把慢查询输出到表，不过一般不推荐输出到表。

show VARIABLES like 'log_output';ls

```
mysql> show VARIABLES like 'log_output';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| log_output    | FILE  |
+-----+-----+
1 row in set (0.00 sec)
```

小结

- `slow_query_log` 启动停止慢查询日志
- `slow_query_log_file` 指定慢查询日志得存储路径及文件（默认和数据文件放在一起）
- `long_query_time` 指定记录慢查询日志 SQL 执行时间得伐值（单位：秒，默认 10 秒）
- `log_queries_not_using_indexes` 是否记录未使用索引的 SQL
- `log_output` 日志存放的地方可以是[TABLE][FILE][FILE, TABLE]

4.2.5. 慢查询解读分析

4.2.5.1. 日志格式

开启慢查询功能以后，会根据我们的配置产生慢查询日志

```
[mysql@iZwz9j203ithc4guluvb2wZ data]$ cat iZwz9j203ithc4guluvb2wZ-slow.log
/home/mysql/mysql57/bin/mysqld, Version: 5.7.32 (MySQL Community Server (GPL)
Tcp port: 3306  Unix socket: /home/mysql/mysql57/log/mysql.sock
Time          Id Command          Argument
# Time: 2021-04-05T07:50:23.580909Z
# User@Host: root[root] @ localhost [] Id:      3
# Query_time: 0.000103  Lock_time: 0.000000 Rows_sent: 1  Rows_examined: 0
SET timestamp=1617609023;
select @@version_comment limit 1;
# Time: 2021-04-05T07:50:49.898353Z
# User@Host: root[root] @ localhost [] Id:      3
# Query_time: 0.000103  Lock_time: 0.000000 Rows_sent: 1  Rows_examined: 0
SET timestamp=1617609049;
SELECT DATABASE();
# Time: 2021-04-05T07:50:49.898603Z
# User@Host: root[root] @ localhost [] Id:      3
# Query_time: 0.000085  Lock_time: 0.000000 Rows_sent: 1  Rows_examined: 0
use mysqladv;
SET timestamp=1617609049;
```

从慢查询日志里面摘选一条慢查询日志，数据组成如下

“Time: 2021-04-05T07:50:53.243703Z”：查询执行时间

“User@Host: root[root] @ localhost [] Id: 3”：用户名、用户的 IP 信息、线程 ID 号

“Query_time: 0.000495”：执行花费的时长【单位：毫秒】

“Lock_time: 0.000170”：执行获得锁的时长

“Rows_sent”：获得的结果行数

“Rows_examined”：扫描的数据行数

“SET timestamp”：这 SQL 执行的具体时间

最后一行：执行的 SQL 语句

4.2.5.2. 慢查询分析

慢查询的日志记录非常多，要从里面找寻一条查询慢的日志并不是很容易的事情，一般来说都需要一些工具辅助才能快速定位到需要优化的 SQL 语句，下面介绍两个慢查询辅助工具

mysqldumpslow

常用的慢查询日志分析工具，汇总除查询条件外其他完全相同的 SQL，并将分析结果按照参数中所指定的顺序输出。当然它的参数不少，我们常用的也就是那么几个。

语法：

`mysqldumpslow -s r -t 10 slow-mysql.log`

`-s order (c,t,l,r,at,al,ar)`

c:总次数

t:总时间

l:锁的时间

r:获得的结果行数

at,al,ar:指 t,l,r 平均数 【例如：at = 总时间/总次数】

-s 对结果进行排序，怎么排，根据后面所带的 (c,t,l,r,at,al,ar)，缺省为 at

-t NUM just show the top n queries: 仅显示前 n 条查询

-g PATTERN grep: only consider stmts that include this string: 通过 grep 来筛选语句。

`./mysqldumpslow -s t -t 10`

`/home/mysql/mysql57/data/iZwz9j203ithc4gu1uvb2wZ-slow.log`

```
[mysql@iZwz9j203ithc4gu1uvb2wZ bin]$ ./mysqldumpslow -s t -t 10 /home/mysql/mysql57/data/iZwz9j203ithc4gu1uvb2wZ-slow.log
Reading mysql slow query log from /home/mysql/mysql57/data/iZwz9j203ithc4gu1uvb2wZ-slow.log
Count: 11 Time=0.00s (0s) Lock=0.00s (0s) Rows=0.0 (0), root[root]@localhost

Count: 1 Time=0.00s (0s) Lock=0.00s (0s) Rows=14.0 (14), root[root]@localhost
SELECT expire_time FROM s1 WHERE insert_time = 'S'

Count: 1 Time=0.00s (0s) Lock=0.00s (0s) Rows=5.0 (5), root[root]@localhost
SELECT DATABASE()
```

`./mysqldumpslow -s t -t 10`

`/home/mysql/mysql57/data/iZwz9j203ithc4gu1uvb2wZ-slow.log -g select`

```
[mysql@iZwz9j203ithc4gu1uvb2wZ bin]$ ./mysqldumpslow -s t -t 10 /home/mysql/mysql57/data/iZwz9j203ithc4gu1uvb2wZ-slow.log -g select
Reading mysql slow query log from /home/mysql/mysql57/data/iZwz9j203ithc4gu1uvb2wZ-slow.log
Count: 1 Time=0.00s (0s) Lock=0.00s (0s) Rows=14.0 (14), root[root]@localhost
SELECT expire_time FROM s1 WHERE insert_time = 'S'

Count: 1 Time=0.00s (0s) Lock=0.00s (0s) Rows=1.0 (1), root[root]@localhost
SELECT DATABASE()

Count: 1 Time=0.00s (0s) Lock=0.00s (0s) Rows=1.0 (1), root[root]@localhost
select @@version_comment limit N
```


4.2.6. Explain 执行计划

4.2.6.1. 什么是执行计划

有了慢查询语句后，就要对语句进行分析。一条查询语句在经过 MySQL 查询优化器的各种基于成本和规则的优化会后生成一个所谓的执行计划，这个执行计划展示了接下来具体执行查询的方式，比如多表连接的顺序是什么，对于每个表采用什么访问方法来具体执行查询等等。EXPLAIN 语句来帮助我们查看某个查询语句的具体执行计划，我们需要搞懂 EXPLAIN 的各个输出项都是干嘛使的，从而可以有针对性的提升我们查询语句的性能。

通过使用 EXPLAIN 关键字可以模拟优化器执行 SQL 查询语句，从而知道 MySQL 是如何处理你的 SQL 语句的。分析查询语句或是表结构的性能瓶颈，总的来说通过 EXPLAIN 我们可以：

- 表的读取顺序
- 数据读取操作的操作类型
- 哪些索引可以使用
- 哪些索引被实际使用
- 表之间的引用
- 每张表有多少行被优化器查询

4.2.6.2. 执行计划的语法

执行计划的语法其实非常简单：在 SQL 查询的前面加上 EXPLAIN 关键字就行。比如：EXPLAIN select * from table1

重点的就是 **EXPLAIN** 后面你要分析的 **SQL 语句**

除了以 SELECT 开头的查询语句，其余的 DELETE、INSERT、REPLACE 以及 UPDATE 语句前边都可以加上 EXPLAIN，用来查看这些语句的执行计划，不过我们这里对 SELECT 语句更感兴趣，所以后边只会以 SELECT 语句为例来描述 EXPLAIN 语句的用法。

4.2.6.3. 执行计划详解

为了让大家先有一个感性的认识，我们把 EXPLAIN 语句输出的各个列的作用先大致罗列一下：

explain select * from order_exp;

```
mysql> explain select * from order_exp;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | order_exp | NULL | ALL | NULL | NULL | NULL | NULL | 10248 | 100.00 | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

id: 在一个大的查询语句中每个 SELECT 关键字都对应一个唯一的 id

select_type: SELECT 关键字对应的那个查询的类型

table: 表名

partitions: 匹配的分区信息

type: 针对单表的访问方法

possible_keys: 可能用到的索引

key: 实际上使用的索引

key_len: 实际使用到的索引长度

ref: 当使用索引列等值查询时，与索引列进行等值匹配的对象信息

rows: 预估的需要读取的记录条数

filtered: 某个表经过搜索条件过滤后剩余记录条数的百分比

Extra: 一些额外的信息

看到这里，是不是一脸懵逼，这是正常的，这里把它们都列出来只是为了描述一个轮廓，随着我们课程的进行，我们会仔细讲解每个列的含义，显示值的含义。

为了方便学习，我们使用范例表 `order_exp`:

```
CREATE TABLE `order_exp` (
  `id` bigint(22) NOT NULL AUTO_INCREMENT COMMENT '订单的主键',
  `order_no` varchar(50) CHARACTER SET utf8 COLLATE utf8_general_ci NOT NULL COMMENT '订单的编号',
  `order_note` varchar(100) CHARACTER SET utf8 COLLATE utf8_general_ci NOT NULL COMMENT '订单的说明',
  `insert_time` datetime(0) NOT NULL DEFAULT CURRENT_TIMESTAMP(0) ON UPDATE CURRENT_TIMESTAMP(0) COMMENT '插入订单的时间',
  `expire_duration` bigint(22) NOT NULL COMMENT '订单的过期时长，单位秒',
  `expire_time` datetime(0) NOT NULL COMMENT '订单的过期时间',
  `order_status` smallint(6) NOT NULL DEFAULT 0 COMMENT '订单的状态，0: 未支付; 1: 已支付; -1: 已过期，关闭',
  PRIMARY KEY (`id`) USING BTREE,
  UNIQUE INDEX `u_idx_day_status` (`insert_time`, `order_status`, `expire_time`) USING BTREE,
  INDEX `idx_order_no` (`order_no`) USING BTREE,
  INDEX `idx_expire_time` (`expire_time`) USING BTREE
) ENGINE = InnoDB AUTO_INCREMENT = 10819 CHARACTER SET = utf8 COLLATE = utf8_general_ci ROW_FORMAT = Dynamic;
```

这个表在库中有个三个派生表 `s1`, `s2`, `order_exp_cut`，表结构基本一致，有少许差别:

```
CREATE TABLE `order_exp_cut` (
  `id` bigint(22) NOT NULL AUTO_INCREMENT COMMENT '订单的主键',
  `order_no` varchar(50) CHARACTER SET utf8 COLLATE utf8_general_ci NULL DEFAULT NULL COMMENT '订单的编号',
  `order_note` varchar(100) CHARACTER SET utf8 COLLATE utf8_general_ci NOT NULL COMMENT '订单的说明',
  `insert_time` datetime(0) NULL DEFAULT CURRENT_TIMESTAMP(0) ON UPDATE CURRENT_TIMESTAMP(0) COMMENT '插入订单的时间',
  `expire_duration` bigint(22) NOT NULL COMMENT '订单的过期时长，单位秒',
  `expire_time` datetime(0) NOT NULL COMMENT '订单的过期时间',
  `order_status` smallint(6) NOT NULL DEFAULT 0 COMMENT '订单的状态，0: 未支付; 1: 已支付; -1: 已过期，关闭',
  PRIMARY KEY (`id`) USING BTREE,
  UNIQUE INDEX `u_idx_day_status` (`insert_time`, `order_status`, `expire_time`) USING BTREE,
  INDEX `idx_order_no` (`order_no`) USING BTREE
)
```

```
CREATE TABLE `s2` (
  `id` bigint(22) NOT NULL AUTO_INCREMENT COMMENT '订单的主键',
  `order_no` varchar(50) CHARACTER SET utf8 COLLATE utf8_general_ci NOT NULL COMMENT '订单的编号',
  `order_note` varchar(100) CHARACTER SET utf8 COLLATE utf8_general_ci NOT NULL COMMENT '订单的说明',
  `insert_time` datetime(0) NOT NULL DEFAULT CURRENT_TIMESTAMP(0) ON UPDATE CURRENT_TIMESTAMP(0) COMMENT '插入订单的时间',
  `expire_duration` bigint(22) NOT NULL COMMENT '订单的过期时长，单位秒',
  `expire_time` datetime(0) NOT NULL COMMENT '订单的过期时间',
  `order_status` smallint(6) NOT NULL DEFAULT 0 COMMENT '订单的状态，0: 未支付; 1: 已支付; -1: 已过期，关闭',
  PRIMARY KEY (`id`) USING BTREE,
  UNIQUE INDEX `u_idx_day_status` (`insert_time`, `order_status`, `expire_time`) USING BTREE,
  INDEX `idx_order_no` (`order_no`) USING BTREE,
  INDEX `idx_insert_time` (`insert_time`) USING BTREE
) ENGINE = InnoDB AUTO_INCREMENT = 10814 CHARACTER SET = utf8 COLLATE = utf8_general_ci ROW_FORMAT = Dynamic;
```

注意：为了方便讲述，我们可能会适当调整对列的讲解顺序，不会完全按照 EXPLAIN 语句输出列顺序来讲解。

table

不论我们的查询语句有多复杂，里边包含了多少个表，到最后也是需要每个表进行单表访问的，MySQL 规定 EXPLAIN 语句输出的每条记录都对应着某个单表的访问方法，该条记录的 table 列代表着该表的表名。

```
mysql> EXPLAIN SELECT * FROM s1 INNER JOIN s2;
+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type |
+-----+-----+-----+-----+
| 1 | SIMPLE | s2 | NULL | ALL |
| 1 | SIMPLE | s1 | NULL | ALL |
+-----+-----+-----+-----+
```

```
mysql> EXPLAIN SELECT * FROM s1 WHERE order_no = 'a';
+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys |
+-----+-----+-----+-----+-----+
| 1 | SIMPLE | s1 | NULL | ref | idx_order_no |
+-----+-----+-----+-----+-----+
```

可以看见，只涉及对 s1 表的单表查询，所以 EXPLAIN 输出中只有一条记录，其中的 table 列的值是 s1，而连接查询的执行计划中有两条记录，这两条记录的 table 列分别是 s1 和 s2。

id

我们知道我们写的查询语句一般都以 SELECT 关键字开头，比较简单的查询语句里只有一个 SELECT 关键字，

稍微复杂一点连接查询中也只有一个 SELECT 关键字，比如：

```
SELECT * FROM s1 INNER JOIN s2
ON s1.id = s2.id
```

```
WHERE s1.order_status = 0;
```

但是下边两种情况下在一条查询语句中会出现多个 SELECT 关键字：

1、查询中包含子查询的情况

比如下边这个查询语句中就包含 2 个 SELECT 关键字：

```
SELECT * FROM s1 WHERE id IN ( SELECT * FROM s2);
```

2、查询中包含 UNION 语句的情况

比如下边这个查询语句中也包含 2 个 SELECT 关键字：

```
SELECT * FROM s1 UNION SELECT * FROM s2 ;
```

查询语句中每出现一个 SELECT 关键字，MySQL 就会为它分配一个唯一的 id 值。这个 id 值就是 EXPLAIN 语句的第一个列。

单 SELECT 关键字

比如下边这个查询中只有一个 SELECT 关键字，所以 EXPLAIN 的结果中也就只有一条 id 列为 1 的记录：

```
EXPLAIN SELECT * FROM s1 WHERE order_no = 'a';
```

```
mysql> EXPLAIN SELECT * FROM s1 WHERE order_no = 'a';
+----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key |
+----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE      | s1    | NULL       | ref  | idx_order_no  | idx_order_no |
+----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

连接查询

对于连接查询来说，一个 SELECT 关键字后边的 FROM 子句中可以跟随多个表，所以在连接查询的执行计划中，每个表都会对应一条记录，但是这些记录的 id 值都是相同的，比如：

```
EXPLAIN SELECT * FROM s1 INNER JOIN s2;
```

```
mysql> EXPLAIN SELECT * FROM s1 INNER JOIN s2;
+----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | extra |
+----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE      | s2    | NULL       | ALL  | NULL         | NULL | NULL |
| 1 | SIMPLE      | s1    | NULL       | ALL  | NULL         | NULL | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set, 1 warning (0.00 sec)
```

可以看到，上述连接查询中参与连接的 s1 和 s2 表分别对应一条记录，但是这两条记录对应的 id 值都是 1。这里需要大家记住的是，在连接查询的执行计划中，每个表都会对应一条记录，这些记录的 id 列的值是相同的。

包含子查询

对于包含子查询的查询语句来说，就可能涉及多个 SELECT 关键字，所以在包含子查询的查询语句的执行计划中，每个 SELECT 关键字都会对应一个唯一的 id 值，比如这样：

```
EXPLAIN SELECT * FROM s1 WHERE id IN (SELECT id FROM s2) OR order_no = 'a';
```

```
mysql> EXPLAIN SELECT * FROM s1 WHERE id IN (SELECT id FROM s2) OR order_no = 'a';
+----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | extra |
+----+-----+-----+-----+-----+-----+-----+-----+
| 1 | PRIMARY     | s1    | NULL       | ALL  | idx_order_no  | NULL | NULL |
| 2 | SUBQUERY    | s2    | NULL       | index | PRIMARY       | u_idx_day_status | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set, 1 warning (0.00 sec)
```

但是这里大家需要特别注意，查询优化器可能对涉及子查询的查询语句进行重写，从而转换为连接查询。所以如果我们想知道查询优化器对某个包含子查询的语句是否进行了重写，直接查看执行计划就好了，比如说：

```
EXPLAIN SELECT * FROM s1 WHERE id IN (SELECT id FROM s2 WHERE order_no = 'a');
```



```
mysql> EXPLAIN SELECT * FROM s1 WHERE id IN (SELECT id FROM s2 WHERE order_no = 'a');
+----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key |
+----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | s2 | NULL | ref | PRIMARY,idx_order_no | idx_order_no |
| 1 | SIMPLE | s1 | NULL | eq_ref | PRIMARY | PRIMARY |
+----+-----+-----+-----+-----+-----+-----+
2 rows in set, 1 warning (0.00 sec)
```

可以看到，虽然我们的查询语句是一个子查询，但是执行计划中 `s1` 和 `s2` 表对应的记录的 `id` 值全部是 1，这就表明了查询优化器将子查询转换为了连接查询，

包含 UNION 子句

对于包含 UNION 子句的查询语句来说，每个 SELECT 关键字对应一个 `id` 值也是没错的，不过还是有点儿特别的东西，比方说下边这个查询：

EXPLAIN SELECT * FROM s1 UNION SELECT * FROM s2;

```
mysql> EXPLAIN SELECT * FROM s1 UNION SELECT * FROM s2;
+----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key |
+----+-----+-----+-----+-----+-----+-----+
| 1 | PRIMARY | s1 | NULL | ALL | NULL | NULL |
| 2 | UNION | s2 | NULL | ALL | NULL | NULL |
| NULL | UNION RESULT | <union1,2> | NULL | ALL | NULL | NULL |
+----+-----+-----+-----+-----+-----+-----+
3 rows in set, 1 warning (0.00 sec)
```

这个语句的执行计划的第三条记录为什么这样？UNION 子句会把多个查询的结果集合并起来并对结果集中的记录进行去重，怎么去重呢？MySQL 使用的是内部的临时表。正如上边的查询计划中所示，UNION 子句是为了把 `id` 为 1 的查询和 `id` 为 2 的查询的结果集合并起来并去重，所以在内部创建了一个名为 `<union1,2>` 的临时表（就是执行计划第三条记录的 `table` 列的名称），`id` 为 NULL 表明这个临时表是为了合并两个查询的结果集而创建的。

跟 UNION 对比起来，UNION ALL 就不需要为最终的结果集进行去重，它只是单纯的把多个查询的结果集中的记录合并成一个并返回给用户，所以也就不需要使用临时表。所以在包含 UNION ALL 子句的查询的执行计划中，就没有那个 `id` 为 NULL 的记录，如下所示：

EXPLAIN SELECT * FROM s1 UNION ALL SELECT * FROM s2;

```
mysql> EXPLAIN SELECT * FROM s1 UNION ALL SELECT * FROM s2;
+----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key |
+----+-----+-----+-----+-----+-----+-----+
| 1 | PRIMARY | s1 | NULL | ALL | NULL | NULL |
| 2 | UNION | s2 | NULL | ALL | NULL | NULL |
+----+-----+-----+-----+-----+-----+-----+
2 rows in set, 1 warning (0.00 sec)
```

select_type 列

通过上边的内容我们知道，一条大的查询语句里边可以包含若干个 SELECT 关键字，每个 SELECT 关键字代表着一个小的查询语句，而每个 SELECT 关键字的 From 子句中都可以包含若干张表（这些表用来做连接查询），每一张表都对应着

执行计划输出中的一条记录，对于在同一个 SELECT 关键字中的表来说，它们的 id 值是相同的。

MySQL 为每一个 SELECT 关键字代表的小查询都定义了一个称之为 select_type 的属性，意思是我们只要知道了某个小查询的 select_type 属性，就知道了这个小查询在整个大查询中扮演了一个什么角色，select_type 取值如下：

SIMPLE: 简单的 select 查询,不使用 union 及子查询

PRIMARY: 最外层的 select 查询

UNION: UNION 中的第二个或随后的 select 查询,不依赖于外部查询的结果集

UNION RESULT: UNION 结果集

SUBQUERY: 子查询中的第一个 select 查询,不依赖于外部查询的结果集

DEPENDENT UNION: UNION 中的第二个或随后的 select 查询,依赖于外部查询的结果集

DEPENDENT SUBQUERY: 子查询中的第一个 select 查询,依赖于外部查询的结果集

DERIVED: 用于 from 子句里有子查询的情况。MySQL 会递归执行这些子查询，把结果放在临时表里。

MATERIALIZED: 物化子查询

UNCACHEABLE SUBQUERY: 结果集不能被缓存的子查询,必须重新为外层查询的每一行进行评估，出现极少。

UNCACHEABLE UNION: UNION 中的第二个或随后的 select 查询,属于不可缓存的子查询，出现极少。

SIMPLE

EXPLAIN SELECT * FROM s1 WHERE order_no = 'a';

简单的 select 查询，查询中不包含子查询或者 UNION

```
mysql> EXPLAIN SELECT * FROM s1 WHERE order_no = 'a';
+----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys |
+----+-----+-----+-----+-----+-----+
| 1 | SIMPLE      | s1    | NULL       | ref  | idx_order_no  |
+----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

连接查询也算是 SIMPLE 类型

EXPLAIN SELECT * FROM s1 INNER JOIN s2;

```
mysql> EXPLAIN SELECT * FROM s1 INNER JOIN s2;
+----+-----+-----+-----+-----+
| id | select_type | table | partitions | type |
+----+-----+-----+-----+-----+
| 1  | SIMPLE      | s2    | NULL        | ALL  |
| 1  | SIMPLE      | s1    | NULL        | ALL  |
+----+-----+-----+-----+-----+
2 rows in set, 1 warning (0.00 sec)
```

PRIMARY

对于包含 UNION、UNION ALL 或者子查询的大查询来说，它是由几个小查询组成的，其中最左边的那个查询的 select_type 值就是 PRIMARY，比方说：

EXPLAIN SELECT * FROM s1 UNION SELECT * FROM s2;

```
mysql> EXPLAIN SELECT * FROM s1 UNION SELECT * FROM s2;
+----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key |
+----+-----+-----+-----+-----+-----+-----+
| 1  | PRIMARY     | s1    | NULL        | ALL  | NULL          | NULL |
| 2  | UNION       | s2    | NULL        | ALL  | NULL          | NULL |
| NULL | UNION RESULT | <union1,2> | NULL        | ALL  | NULL          | NULL |
+----+-----+-----+-----+-----+-----+-----+
3 rows in set, 1 warning (0.00 sec)
```

从结果中可以看到，最左边的小查询 SELECT * FROM s1 对应的是执行计划中的第一条记录，它的 select_type 值就是 PRIMARY。

UNION

对于包含 UNION 或者 UNION ALL 的大查询来说，它是由几个小查询组成的，其中除了最左边的那个小查询以外，其余的查询的 select_type 值就是 UNION，可以对比上一个例子的效果。

UNION RESULT

MySQL 选择使用临时表来完成 UNION 查询的去重工作，针对该临时表的查询的 select_type 就是 UNION RESULT，例子上边有。

SUBQUERY

如果包含子查询的查询语句不能够转为对应的 semi-join 的形式，并且该子查询是不相关子查询，并且查询优化器决定采用将该子查询物化的方案来执行该子查询时，该子查询的第一个 SELECT 关键字代表的那个查询的 select_type 就是 SUBQUERY，比如下边这个查询：

EXPLAIN SELECT * FROM s1 WHERE id IN (SELECT id FROM s2) OR order_no = 'a';

```
mysql> EXPLAIN SELECT * FROM s1 WHERE id IN (SELECT id FROM s2) OR order_no = 'a';
+----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key |
+----+-----+-----+-----+-----+-----+-----+
| 1  | PRIMARY     | s1    | NULL        | ALL  | idx_order_no | NULL |
| 2  | SUBQUERY    | s2    | NULL        | index | PRIMARY       | u_idx_day_status |
+----+-----+-----+-----+-----+-----+-----+
2 rows in set, 1 warning (0.00 sec)
```

可以看到，外层查询的 `select_type` 就是 `PRTIMARY`，子查询的 `select_type` 就是 `SUBOUERY`。需要大家注意的是，由于 `select_type` 为 `SUBQUERY` 的子查询由于会被物化，所以只需要执行一遍。

TIPS

上面的说明里出现了几个新名字，这里稍微解释下，以后会有详解讲解。

semi-join：半连接优化技术，本质上是把子查询上拉到父查询中，与父查询的表做 `join` 操作。关键词是“上拉”。对于子查询，其子查询部分相对于父表的每个符合条件的元组，都要把子查询执行一轮。效率低下。用半连接操作优化子查询，是把子查询上拉到父查询中，这样子查询的表和父查询中的表是并列关系，父表的每个符合条件的元组，只需要在子表中找符合条件的元组即可。简单来说，就是通过将子查询上拉对父查询中的数据进行筛选，以使获取到最少量的足以对父查询记录进行筛选的信息就足够了。

子查询物化：子查询的结果通常缓存在内存或临时表中。

关联/相关子查询：子查询的执行依赖于外部查询。多数情况下是子查询的 `WHERE` 子句中引用了外部查询的表。自然“非关联/相关子查询”的执行则不依赖与外部的查询。

DEPENDENT UNION、DEPENDENT SUBQUERY

在包含 `UNION` 或者 `UNION ALL` 的大查询中，如果各个小查询都依赖于外层查询的话，那除了最左边的那个小查询之外，其余的小查询的 `select_type` 的值就是 `DEPENDENT UNION`。比方说下边这个查询：

EXPLAIN SELECT * FROM s1 WHERE id IN (SELECT id FROM s2 WHERE id = 716 UNION SELECT id FROM s1 WHERE id = 718);

```
mysql> EXPLAIN SELECT * FROM s1 WHERE id IN (SELECT id FROM s2 WHERE id = 716 UNION SELECT id FROM s1 WHERE id = 718);
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | PRIMARY | s1 | NULL | ALL | NULL | NULL | NULL | NULL | 10664 | 100 |
| 2 | DEPENDENT SUBQUERY | s2 | NULL | const | PRIMARY | PRIMARY | 8 | const | 1 | 100 |
| 3 | DEPENDENT UNION | s1 | NULL | const | PRIMARY | PRIMARY | 8 | const | 1 | 100 |
| NULL | UNION RESULT | <union2,3> | NULL | ALL | NULL | NULL | NULL | NULL | NULL | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
4 rows in set, 1 warning (0.00 sec)
```

这个查询比较复杂，大查询里包含了一个子查询，子查询里又是由 `UNION` 连起来的两个小查询。从执行计划中可以看出，**SELECT id FROM s2 WHERE id = 716** 这个小查询由于是子查询中第一个查询，所以它的 `select_type` 是 `DEPENDENT SUBQUERY`，而 **SELECT id FROM s1 WHERE id = 718** 这个查询的 `select_type` 就是 `DEPENDENT UNION`。

是不是很奇怪这条语句并没有依赖外部的查询？MySQL 优化器对 `IN` 操作符的优化会将 `IN` 中的非关联子查询优化成一个关联子查询。我们可以在执行上面那个执行计划后，马上执行 `show warnings\G`，可以看到 MySQL 对 SQL 语句的大致改写情况：

```
mysql> EXPLAIN SELECT * FROM s1 WHERE id IN (SELECT id FROM s2 WHERE id = 716 UNION SELECT id FROM s1 WHERE id = 718);
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | PRIMARY | s1 | NULL | ALL | NULL | NULL | NULL | NULL | 10664 | 100.00 | Using where |
| 2 | DEPENDENT SUBQUERY | s2 | NULL | const | PRIMARY | PRIMARY | 8 | const | 1 | 100.00 | Using index |
| 3 | DEPENDENT UNION | s1 | NULL | const | PRIMARY | PRIMARY | 8 | const | 1 | 100.00 | Using index |
| NULL | UNION RESULT | <union2> | NULL | ALL | NULL | NULL | NULL | NULL | NULL | NULL | Using temporary |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
4 rows in set, 1 warning (0.00 sec)

mysql> show warnings\G
***** 1. row *****
Level: Note
Code: 1003
Message: /* select#1 */ select 'mysqladv`.`s1`.`id` AS `id`,`mysqladv`.`s1`.`order_no` AS `order_no`,`mysqladv`.`s1`.`order_note` AS `order_note`,`mysqladv`.`s1`.`insert_time` AS `insert_time`,`mysqladv`.`s1`.`expire_duration` AS `expire_duration`,`mysqladv`.`s1`.`expire_time` AS `expire_time`,`mysqladv`.`s1`.`order_status` AS `order_status` from `mysqladv`.`s1` where <in_optimizer>(`mysqladv`.`s1`.`id`,`exists`/* select#2 */ select 1 from `mysqladv`.`s2` where ((<cache>(`mysqladv`.`s1`.`id`) = 716)) union /* select#3 */ select 1 from `mysqladv`.`s1` where ((<cache>(`mysqladv`.`s1`.`id`) = 718)))
1 row in set (0.00 sec)
```

DERIVED

对于采用物化的方式执行的包含派生表的查询，该派生表对应的子查询的 select_type 就是 DERIVED。

EXPLAIN SELECT * FROM (SELECT id, count(*) as c FROM s1 GROUP BY id) AS derived_s1 where c >1;

```
mysql> EXPLAIN SELECT * FROM (SELECT id, count(*) as c FROM s1 GROUP BY id) AS derived_s1 where c >1;
+----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible keys | key |
+----+-----+-----+-----+-----+-----+-----+
| 1 | PRIMARY | <derived2> | NULL | ALL | NULL | NULL |
| 2 | DERIVED | s1 | NULL | index | PRIMARY,u_idx_day_status,idx_order_no | PRIMARY |
+----+-----+-----+-----+-----+-----+-----+
2 rows in set, 1 warning (0.00 sec)
```

从执行计划中可以看出，id 为 2 的记录就代表子查询的执行方式，它的 select_type 是 DERIVED，说明该子查询是以物化的方式执行的。id 为 1 的记录代表外层查询，大家注意看它的 table 列显示的是 <derived2>，表示该查询是针对将派生表物化之后的表进行查询的。

MATERIALIZED

当查询优化器在执行包含子查询的语句时，选择将子查询物化之后与外层查询进行连接查询时，该子查询对应的 select_type 属性就是 MATERIALIZED，比如下边这个查询：

EXPLAIN SELECT * FROM s1 WHERE order_no IN (SELECT order_no FROM s2);

```
mysql> EXPLAIN SELECT * FROM s1 WHERE order_no IN (SELECT order_no FROM s2);
+----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible keys | key |
+----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | s1 | NULL | ALL | idx_order_no | NULL |
| 1 | SIMPLE | <subquery2> | NULL | eq_ref | <auto_key> | <auto_key> |
| 2 | MATERIALIZED | s2 | NULL | index | idx_order_no | idx_order_no |
+----+-----+-----+-----+-----+-----+-----+
3 rows in set, 1 warning (0.00 sec)
```

执行计划的第三条记录的 id 值为 2，从它的 select_type 值为 MATERIALIZED 可以看出，查询优化器是要把子查询先转换成物化表。

然后看执行计划的前两条记录的 id 值都为 1，说明这两条记录对应的表进行连接查询，需要注意的是第二条记录的 table 列的值是 <subquery2>，说明该表其实就是 id 为 2 对应的子查询执行之后产生的物化表，然后将 s1 和该物化表进行连接查询。

UNCACHEABLE SUBQUERY、UNCACHEABLE UNION

出现极少，不做深入讲解，比如

explain select * from s1 where id = (select id from s2 where order_no=@@sql_log_bin);

```
mysql> explain select * from s1 where id = ( select id from s2 where order_no=@@sql_log_bin);
+----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_ |
+----+-----+-----+-----+-----+-----+-----+-----+
| 1 | PRIMARY | NULL | NULL | NULL | NULL | NULL | NULL |
| 2 | UNCACHEABLE SUBQUERY | s2 | NULL | index | idx_order_no | idx_order_no | 152 |
+----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set, 10565 warnings (0.00 sec)
```

partitions

和分区表有关，一般情况下我们的查询语句的执行计划的 partitions 列的值都是 NULL。

type

我们前边说过执行计划的一条记录就代表着 MySQL 对某个表的执行查询时的访问方法/访问类型，其中的 type 列就表明了这个访问方法/访问类型是个什么东西，是较为重要的一个指标，结果值从最好到最坏依次是：

system > const > eq_ref > ref > fulltext > ref_or_null > index_merge > unique_subquery > index_subquery > range > index > ALL

出现比较多的是 **system>const>eq_ref>ref>range>index>ALL**

一般来说，得保证查询至少达到 **range** 级别，最好能达到 **ref**。

system

当表中只有一条记录并且该表使用的存储引擎的统计数据是精确的，比如 MyISAM、Memory，那么对该表的访问方法就是 system。

explain select * from test_myisam;

```
mysql> explain select * from test_myisam;
+----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type |
+----+-----+-----+-----+-----+
| 1 | SIMPLE | test_myisam | NULL | system |
+----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

当然，如果改成使用 InnoDB 存储引擎，试试看执行计划的 type 列的值是什么。

```
mysql> explain select * from test;
+----+-----+-----+-----+-----+
| id | select_type | table | partitions | type |
+----+-----+-----+-----+-----+
| 1 | SIMPLE | test | NULL | ALL |
+----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```


const

就是当我们根据主键或者唯一二级索引列与常数进行等值匹配时，对单表的访问方法就是 **const**。因为只匹配一行数据，所以很快。

例如将主键置于 **where** 列表中

EXPLAIN SELECT * FROM s1 WHERE id = 716;

```
mysql> EXPLAIN SELECT * FROM s1 WHERE id = 716;
+----+-----+-----+-----+-----+
| id | select_type | table | partitions | type |
+----+-----+-----+-----+-----+
| 1 | SIMPLE      | s1    | NULL        | const |
+----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

B+树叶子节点中的记录是按照索引列排序的，对于的聚簇索引来说，它对应的 B+树叶子节点中的记录就是按照 **id** 列排序的。B+树矮胖，所以这样根据主键值定位一条记录的速度很快。类似的，我们根据唯一二级索引列来定位一条记录的速度也很快的，比如下边这个查询：

SELECT * FROM order_exp WHERE insert_time="" and order_status="" and expire_time="" ;

这个查询的执行分两步，第一步先从 **u_idx_day_status** 对应的 B+树索引中根据索引列与常数的等值比较条件定位到一条二级索引记录，然后再根据该记录的 **id** 值到聚簇索引中获取到完整的用户记录。

MySQL 把这种通过主键或者唯一二级索引列来定位一条记录的访问方法定义为：**const**，意思是常数级别的，代价是可以忽略不计的。

不过这种 **const** 访问方法只能在主键列或者唯一二级索引列和一个常数进行等值比较时才有效，如果主键或者唯一二级索引是由多个列构成的话，组成索引的每一个列都是与常数进行等值比较时，这个 **const** 访问方法才有效。

对于唯一二级索引来说，查询该列为 **NULL** 值的情况比较特殊，因为唯一二级索引列并不限制 **NULL** 值的数量，所以上述语句可能访问到多条记录，也就是说 **is null** 不可以使用 **const** 访问方法来执行。

eq_ref

在连接查询时，如果被驱动表是通过主键或者唯一二级索引列等值匹配的方式进行访问的（如果该主键或者唯一二级索引是联合索引的话，所有的索引列都必须进行等值比较），则对该被驱动表的访问方法就是 **eq_ref**。

TIPS

驱动表与被驱动表：A 表和 B 表 join 连接查询，如果通过 A 表的结果集作为循环基础数据，然后一条一条地通过该结果集中的数据作为过滤条件到 B 表中查询数据，然后合并结果。那么我们称 A 表为驱动表，B 表为被驱动表

比方说：

EXPLAIN SELECT * FROM s1 INNER JOIN s2 ON s1.id = s2.id;

```
mysql> EXPLAIN SELECT * FROM s1 INNER JOIN s2 ON s1.id = s2.id;
+----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_key |
+----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | s2 | NULL | ALL | PRIMARY |
| 1 | SIMPLE | s1 | NULL | eq_ref | PRIMARY |
+----+-----+-----+-----+-----+-----+
2 rows in set, 1 warning (0.00 sec)
```

从执行计划的结果中可以看出，MySQL 打算将 `s2` 作为驱动表，`s1` 作为被驱动表，重点关注 `s1` 的访问方法是 `eq_ref`，表明在访问 `s1` 表的时候可以通过主键的等值匹配来进行访问。

ref

当通过普通的二级索引与常量进行等值匹配时来查询某个表，那么对该表的访问方法就可能是 `ref`。

本质上也是一种索引访问，它返回所有匹配某个单独值的行，然而，它可能会找到多个符合条件的行，所以他属于查找和扫描的混合体

```
EXPLAIN SELECT * FROM s1 WHERE order_no = 'a';
```

```
mysql> EXPLAIN SELECT * FROM s1 WHERE order_no = 'a';
+----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_key |
+----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | s1 | NULL | ref | idx_order_no |
+----+-----+-----+-----+-----+-----+
```

对于这个查询，我们当然可以选择全表扫描来逐一对比搜索条件是否满足要求，我们也可以先使用二级索引找到对应记录的 `id` 值，然后再回表到聚簇索引中查找完整的用户记录。

由于普通二级索引并不限制索引列值的唯一性，所以可能找到多条对应的记录，也就是说使用二级索引来执行查询的代价取决于等值匹配到的二级索引记录条数。如果匹配的记录较少，则回表的代价还是比较低的，所以 MySQL 可能选择使用索引而不是全表扫描的方式来执行查询。这种搜索条件为二级索引列与常数等值比较，采用二级索引来执行查询的访问方法称为：`ref`。

对于普通的二级索引来说，通过索引列进行等值比较后可能匹配到多条连续的记录，而不是像主键或者唯一二级索引那样最多只能匹配 1 条记录，所以这种 `ref` 访问方法比 `const` 要差些，但是在二级索引等值比较时匹配的记录数较少时的效率还是很高的（如果匹配的二级索引记录太多那么回表的成本就太大了）。不过需要注意下边两种情况：

二级索引列值为 `NULL` 的情况

不论是普通的二级索引，还是唯一二级索引，它们的索引列对包含 `NULL` 值的数量并不限制，所以我们采用 `key IS NULL` 这种形式的搜索条件最多只能使用 `ref` 的访问方法，而不是 `const` 的访问方法。

对于某个包含多个索引列的二级索引来说，只要是最左边的连续索引列是与常数的等值比较就可能采用 `ref` 的访问方法，比方说下边这几个查询：

```
SELECT * FROM order_exp WHERE insert_time = '2021-03-22 18:28:23';
SELECT * FROM order_exp WHERE insert_time = '2021-03-22 18:28:23' AND
order_status = 0;
```

但是如果最左边的连续索引列并不全部是等值比较的话，它的访问方法就不能称为 *ref* 了，比方说这样：

```
SELECT * FROM order_exp WHERE insert_time = '2021-03-22 18:28:23' AND
order_status > -1;
```

fulltext

全文索引，跳过～。

ref_or_null

有时候我们不仅想找出某个二级索引列的值等于某个常数的记录，还想把该列的值为 *NULL* 的记录也找出来，就像下边这个查询：

```
explain SELECT * FROM order_exp_cut WHERE order_no= 'abc' OR order_no IS
NULL;
```

```
mysql> explain SELECT * FROM order_exp_cut WHERE order_no= 'abc' OR order_no IS NULL;
+-----+
| id | select_type | table          | partitions | type        | possible_keys | key          |
+-----+
| 1  | SIMPLE      | order_exp_cut | NULL       | ref_or_null | idx_order_no  | idx_order_no |
+-----+
1 row in set, 1 warning (0.00 sec)
```

注意，上面的表改为了 *order_exp_cut*，*order_exp_cut* 相对于 *order_exp* 就是把一些列改为了允许 *null*，其他的无变化。

id	type	length	flags	is_primary	is_unique	is_indexed	comment
id	bigint	22	0	☒	☐	☒	订单的主键
order_no	varchar	50	0	☐	☐	☒	订单的编号
order_note	varchar	100	0	☒	☐	☒	订单的说明
insert_time	datetime	0	0	☐	☐	☒	插入订单的
expire_duration	bigint	22	0	☒	☐	☒	订单的过期
expire_time	datetime	0	0	☒	☐	☒	订单的过期
order_status	smallint	6	0	☒	☐	☒	订单的状态

这个查询相当于先分别从 *order_exp_cut* 表的 *idx_order_no* 索引对应的 B+树 中找出 *order_no IS NULL* 和 *order_no= 'abc'* 的两个连续的记录范围，然后根据这些二级索引记录中的 *id* 值再回表查找完整的用户记录。

index_merge

一般情况下对于某个表的查询只能使用到一个索引，在某些场景下可以使用索引合并的方式来执行查询：

```
EXPLAIN SELECT * FROM s1 WHERE order_no = 'a' OR insert_time =
'2021-03-22 18:36:47';
```

```
mysql> EXPLAIN SELECT * FROM s1 WHERE order_no = 'a' OR insert_time = '2021-03-22 18:36:47';
+----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys |
+----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | s1 | NULL | index_merge | u_idx_day_status,idx_order_no,idx_insert_time |
+----+-----+-----+-----+-----+-----+
00 | Using union(idx_order_no,idx_insert_time); Using where |
+----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

什么是索引合并，我们以后再说。

unique_subquery

类似于两表连接中被驱动表的 `eg_ref` 访问方法，`unique_subquery` 是针对在一些包含 IN 子查询的查询语句中，如果查询优化器决定将 IN 子查询转换为 EXISTS 子查询，而且子查询可以使用到主键进行等值匹配的话，那么该子查询执行计划的 `type` 列的值就是 `unique_subquery`，比如下边的这个查询语句：

```
EXPLAIN SELECT * FROM s1 WHERE id IN (SELECT id FROM s2 where
s1.insert_time = s2.insert_time) OR order_no = 'a';
```

```
mysql> EXPLAIN SELECT * FROM s1 WHERE id IN (SELECT id FROM s2 where s1.insert_time = s2.insert_time) OR order_no = 'a';
+----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key |
+----+-----+-----+-----+-----+-----+-----+-----+
| 1 | PRIMARY | s1 | NULL | ALL | idx_order_no | NULL | NU |
+----+-----+-----+-----+-----+-----+-----+-----+
| 2 | DEPENDENT SUBQUERY | s2 | NULL | unique_subquery | PRIMARY,u_idx_day_status,idx_insert_time | PRIMARY | 8 |
+----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set, 2 warnings (0.00 sec)
```

可以看到执行计划的第二条记录的 `type` 值就是 `unique_subquery`，说明在执行子查询时会使用到 `id` 列的索引。

index_subquery

`index_subquery` 与 `unique_subquery` 类似，只不过访问子查询中的表时使用的是普通的索引：

```
EXPLAIN SELECT * FROM s1 WHERE order_no IN (SELECT order_no FROM s2
where s1.insert_time = s2.insert_time) OR order_no = 'a';
```

这个语句和 `unique_subquery` 章节中的唯一不同是什么？就是 `in` 子句的字段由 `id` 变成了 `order_no`。


```
mysql> EXPLAIN SELECT * FROM s1 WHERE order_no IN (SELECT order_no FROM
***** 1. row *****
      id: 1
    select_type: PRIMARY
      table: s1
    partitions: NULL
      type: ALL
possible_keys: idx_order_no
      key: NULL
    key_len: NULL
      ref: NULL
     rows: 10573
  filtered: 100.00
    Extra: Using where
***** 2. row *****
      id: 2
    select_type: DEPENDENT SUBQUERY
      table: s2
    partitions: NULL
      type: index subquery
possible_keys: u_idx_day_status,idx_order_no,idx_insert_time
      key: idx_order_no
    key_len: 152
      ref: func
     rows: 1
  filtered: 10.00
    Extra: Using where
2 rows in set, 2 warnings (0.00 sec)
```

range

如果使用索引获取某些范围区间的记录,那么就使用到 range 访问方法,一般就是在你的 where 语句中出现了 between、<、>、in 等的查询。

这种范围扫描索引扫描比全表扫描要好,因为它只需要开始于索引的某一点,而结束于另一点,不用扫描全部索引。

EXPLAIN SELECT * FROM s1 WHERE order_no IN ('a', 'b', 'c');

EXPLAIN SELECT * FROM s1 WHERE order_no > 'a' AND order_no < 'b';

```
mysql> EXPLAIN SELECT * FROM s1 WHERE order_no IN ('a', 'b', 'c');
+----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key |
+----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | s1 | NULL | range | idx_order_no | idx_order_no |
+----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.01 sec)

mysql> EXPLAIN SELECT * FROM s1 WHERE order_no > 'a' AND order_no < 'b';
+----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key |
+----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | s1 | NULL | range | idx_order_no | idx_order_no |
+----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

这种利用索引进行范围匹配的访问方法称之为: range。

此处所说的使用索引进行范围匹配中的‘索引’可以是聚簇索引,也可以是二级索引。

index

当我们可以使用索引覆盖,但需要扫描全部的索引记录时,该表的访问方法就是 index。

EXPLAIN SELECT insert_time FROM s1 WHERE expire_time = '2021-03-22 18:36:47';


```
mysql> EXPLAIN SELECT insert_time FROM s1 WHERE expire_time = '2021-03-22 18:36:47';
+----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key |
+----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | s1 | NULL | index | NULL | u_idx_day_status |
+----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

all

最熟悉的全表扫描，将遍历全表以找到匹配的行

EXPLAIN SELECT * FROM s1;

```
mysql> EXPLAIN SELECT * FROM s1;
+----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key |
+----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | s1 | NULL | ALL | NULL | NULL |
+----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

possible_keys 与 key

在 EXPLAIN 语句输出的执行计划中,possible_keys 列表示在某个查询语句中,对某个表执行单表查询时可能用到的索引有哪些, key 列表示实际用到的索引有哪些, 如果为 NULL, 则没有使用索引。比方说下边这个查询:

EXPLAIN SELECT order_note FROM s1 WHERE insert_time = '2021-03-22 18:36:47';

```
mysql> EXPLAIN SELECT order_note FROM s1 WHERE insert_time = '2021-03-22 18:36:47';
+----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key |
+----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | s1 | NULL | ref | u_idx_day_status,idx_insert_time | u_idx_day_status |
+----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

上述执行计划的 possible keys 列的值表示该查询可能使用到 u_idx_day_status,idx_insert_time 两个索引, 然后 key 列的值是 u_idx_day_status, 表示经过查询优化器计算使用不同索引的成本后, 最后决定使用 u_idx_day_status 来执行查询比较划算。

不过有一点比较特别, 就是在使用 index 访问方法来查询某个表时, 可能会出现 possible_keys 列是空的, 而 key 列展示的是实际使用到的索引, 比如这样:

EXPLAIN SELECT insert_time FROM s1 WHERE expire_time = '2021-03-22 18:36:47';

具体原因请自行思考。

```
mysql> EXPLAIN SELECT insert_time FROM s1 WHERE expire_time = '2021-03-22 18:36:47';
+----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key |
+----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | s1 | NULL | index | NULL | u_idx_day_status |
+----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

另外需要注意的一点是，possible keys 列中的值并不是越多越好，可能使用的索引越多，查询优化器计算查询成本时就得花费更长时间，所以如果可以的话，尽量删除那些用不到的索引。

key_len

key_len 列表示当优化器决定使用某个索引执行查询时，该索引记录的最大长度，计算方式是这样的：

对于使用固定长度类型的索引列来说，它实际占用的存储空间的最大长度就是该固定值，对于指定字符集的变长类型的索引列来说，比如某个索引列的类型是 VARCHAR(100)，使用的字符集是 utf8，那么该列实际占用的最大存储空间就是 $100 \times 3 = 300$ 个字节。

如果该索引列可以存储 NULL 值，则 key_len 比不可以存储 NULL 值时多 1 个字节。

对于变长字段来说，都会有 2 个字节的空间来存储该变长列的实际长度。

比如下边这个查询：

EXPLAIN SELECT * FROM s1 WHERE id = 718;

```
mysql> EXPLAIN SELECT * FROM s1 WHERE id = 718;
+----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len |
+----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | s1 | NULL | const | PRIMARY | PRIMARY | 8 |
+----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

由于 id 列的类型是 bigint，并且不可以存储 NULL 值，所以在使用该列的索引时 key_len 大小就是 8。

对于可变长度的索引列来说，比如下边这个查询：

EXPLAIN SELECT * FROM s1 WHERE order_no = 'a';

```
mysql> EXPLAIN SELECT * FROM s1 WHERE order_no = 'a';
+----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len |
+----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | s1 | NULL | ref | idx_order_no | idx_order_no | 152 |
+----+-----+-----+-----+-----+-----+-----+-----+
```

由于 order_no 列的类型是 VARCHAR(50)，所以该列实际最多占用的存储空间就是 50×3 字节，又因为该列是可变长度列，所以 key_len 需要加 2，所以最后 key_len 的值就是 152。

执行计划的生成是在 MySQL server 层中的功能，并不是针对具体某个存储引擎的功能，MySQL 在执行计划中输出 key_len 列主要是为了让我们区分某个使用联合索引的查询具体用了几个索引列(复合索引有最左前缀的特性，如果复合索引能全部使用上，则是复合索引字段的索引长度之和，这也可以用来判定复合索引是否部分使用，还是全部使用)，而不是为了准确的说明针对某个具体存储引擎存储变长字段的实际长度占用的空间到底是占用 1 个字节还是 2 个字节。

Key_len 表示索引中使用的字节数，可通过该列计算查询中使用的索引的长度。在不损失精确性的情况下，长度越短越好

key_len 显示的值为索引字段的最大可能长度,并非实际使用长度,即 key_len 是根据表定义计算而得,不是通过表内检索出的。

注意: char 和 varchar 跟字符编码也有密切的联系,比如 latin1 占用 1 个字节,gbk 占用 2 个字节,utf8 占用 3 个字节。

ref

当使用索引列等值匹配的条件去执行查询时,也就是在访问方法是 const、eg_ref、ref、ref_or_null、unique_subquery、index_subquery 其中之一时,ref 列展示的就是与索引列作等值匹配的是谁,比如只是一个常数或者是某个列。比如:

EXPLAIN SELECT * FROM s1 WHERE order_no = 'a';

```
mysql> EXPLAIN SELECT * FROM s1 WHERE order_no = 'a';
+----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref |
+----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | s1 | NULL | ref | idx_order_no | idx_order_no | 152 | const |
+----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

可以看到 ref 列的值是 const,表明在使用 idx_order_no 索引执行查询时,与 order_no 列作等值匹配的对象是一个常数,当然有时候更复杂一点:

EXPLAIN SELECT * FROM s1 INNER JOIN s2 ON s1.id = s2.id;

```
mysql> EXPLAIN SELECT * FROM s1 INNER JOIN s2 ON s1.id = s2.id;
+----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref |
+----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | s2 | NULL | ALL | PRIMARY | NULL | NULL | NULL |
| 1 | SIMPLE | s1 | NULL | eq_ref | PRIMARY | PRIMARY | 8 | mysqladv.s2.id |
+----+-----+-----+-----+-----+-----+-----+-----+-----+
```

可以看到对被驱动表 s2 的访问方法是 eg_ref,而对应的 ref 列的值是 mysqladv.s2.id,这说明在对被驱动表进行访问时会用到 PRIMARY 索引,也就是聚簇索引与一个列进行等值匹配的条件,与 s2 表的 id 作等值匹配的对象就是 mysqladv.s2.id 列(注意这里把数据库名也写出来了)。

有的时候与索引列进行等值匹配的对象是一个函数,比如

EXPLAIN SELECT * FROM s1 INNER JOIN s2 ON s2.order_no=UPPER(s1.order_no);

可以看到在查询计划的 ref 列里输出的是 func。

```
mysql> EXPLAIN SELECT * FROM s1 INNER JOIN s2 ON s2.order_no=UPPER(s1.order_no);
+----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows |
+----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | s1 | NULL | ALL | NULL | NULL | NULL | NULL | 10573 |
| 1 | SIMPLE | s2 | NULL | ref | idx_order_no | idx_order_no | 152 | func | 1 |
+----+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set, 1 warning (0.01 sec)
```

rows

如果查询优化器决定使用全表扫描的方式对某个表执行查询时,执行计划的 rows 列就代表预计需要扫描的行数,如果使用索引来执行查询时,执行计划的 rows 列就代表预计扫描的索引记录行数。比如下边两个个查询:

EXPLAIN SELECT * FROM s1 WHERE order_no > 'z';

EXPLAIN SELECT * FROM s1 WHERE order_no > 'a';

```
mysql> EXPLAIN SELECT * FROM s1 WHERE order_no > 'z';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	s1	NULL	range	idx_order_no	idx_order_no	152	NULL	1	100.00	Using index condition

1 row in set, 1 warning (0.00 sec)

```
mysql> EXPLAIN SELECT * FROM s1 WHERE order_no > 'a';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	s1	NULL	ALL	idx_order_no	NULL	NULL	NULL	10573	50.00	Using where

1 row in set, 1 warning (0.00 sec)

我们看到执行计划的 rows 列的值是分别是 1 和 10573，这意味着查询优化器在经过分析使用 idx_order_no 进行查询的成本之后，觉得满足 order_no > 'a' 这个条件的记录只有 1 条，觉得满足 order_no > 'a' 这个条件的记录有 10573 条。

filtered

查询优化器预测有多少条记录满足其余的搜索条件，什么意思呢？看具体的语句：

```
EXPLAIN SELECT * FROM s1 WHERE id > 5890 AND order_note = 'a';
```

```
mysql> EXPLAIN SELECT * FROM s1 WHERE order_no > 'r' AND order_note = 'a';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	s1	NULL	range	idx_order_no	idx_order_no	152	NULL	1	10.00	

从执行计划的 key 列中可以看出，该查询使用 PRIMARY 索引来执行查询，从 rows 列可以看出满足 id > 5890 的记录有 5286 条。执行计划的 filtered 列就代表查询优化器预测在这 5286 条记录中，有多少条记录满足其余的搜索条件，也就是 order_note = 'a' 这个条件的百分比。此处 filtered 列的值是 10.0，说明查询优化器预测在 5286 条记录中有 10.00% 的记录满足 order_note = 'a' 这个条件。

对于单表查询来说，这个 filtered 列的值没什么意义，我们更关注在连接查询中驱动表对应的执行计划记录的 filtered 值，比方说下边这个查询：

```
EXPLAIN SELECT * FROM s1 INNER JOIN s2 ON s1.order_no = s2.order_no
WHERE s1.order_note > '你好，李焕英';
```

```
mysql> EXPLAIN SELECT * FROM s1 INNER JOIN s2 ON s1.order_no = s2.order_no WHERE s1.order_note > '你好，李焕英';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	s1	NULL	ALL	idx_order_no	NULL	NULL	NULL	10573	33.33	Using where
1	SIMPLE	s2	NULL	ref	idx_order_no	idx_order_no	152	mysqladv.s1.order_no	1	100.00	NULL

2 rows in set, 1 warning (0.00 sec)

从执行计划中可以看出，查询优化器打算把 s1 当作驱动表，s2 当作被驱动表。我们可以看到驱动表 s1 表的执行计划的 rows 列为 10573，filtered 列为 33.33，这意味着驱动表 s1 的扇出值就是 $10573 \times 33.33\% = 3524.3$ ，这说明还要对被驱动表执行大约 3524 次查询。

Extra

顾名思义，Extra 列是用来说明一些额外信息的，我们可以通过这些额外信息来更准确的理解 MySQL 到底将如何执行给定的查询语句。MySQL 提供的额外信息很多，几十个，无法一一介绍，挑一些平时常见的或者比较重要的额外信息讲讲，同时因为数据的关系，很难实际演示出来，所以本小节的相关内容不会提供全部实际的 SQL 语句和结果画面。

No tables used

当查询语句的没有 FROM 子句时将会提示该额外信息。

Impossible WHERE

查询语句的 WHERE 子句永远为 FALSE 时将会提示该额外信息。

No matching min/max row

当查询列表处有 MIN 或者 MAX 聚集函数，但是并没有符合 WHERE 子句中的搜索条件的记录时，将会提示该额外信息。

Using index

当我们的查询列表以及搜索条件中只包含属于某个索引的列，也就是在可以使用索引覆盖的情况下，在 Extra 列将会提示该额外信息。比方说下边这个查询中只需要用到 idx_order_no 而不需要回表操作：

```
EXPLAIN SELECT expire_time FROM s1 WHERE insert_time = '2021-03-22 18:36:47';
```

select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
SIMPLE	s1	NULL	ref	u_idx_day_status,idx_insert_time	u_idx_day_status	5	const	14	100.00	Using index
1 set, 1 warning (0.00 sec)										

Using index condition

有些搜索条件中虽然出现了索引列，但却不能使用到索引，比如下边这个查询：

```
SELECT * FROM s1 WHERE order_no > 'z' AND order_no LIKE '%a';
```

其中的 order_no > 'z' 可以使用到索引，但是 order_no LIKE '%a' 却无法使用到索引，在以前版本的 MySQL 中，是按照下边步骤来执行这个查询的：

- 1、先根据 order_no > 'z' 这个条件，从二级索引 idx_order_no 中获取到对应的二级索引记录。
- 2、根据上一步骤得到的二级索引记录中的主键值进行回表(因为是 select *)，找到完整的用户记录再检测该记录是否符合 key1 LIKE '%a' 这个条件，将符合条件的记录加入到最后的结果集。

但是虽然 order_no LIKE '%a' 不能组成范围区间参与 range 访问方法的执行，但这个条件毕竟只涉及到了 order_no 列，MySQL 把上边的步骤改进了一下。

索引条件下推

- 1、先根据 order_no > 'z' 这个条件，定位到二级索引 idx_order_no 中对应的二级索引记录。
- 2、对于指定的二级索引记录，先不着急回表，而是先检测一下该记录是否满足 order_no LIKE '%a' 这个条件，如果这个条件不满足，则该二级索引记录压根儿就没必要回表。

- 3、对于满足 order_no LIKE '%a' 这个条件的二级索引记录执行回表操作。

我们说回表操作其实是一个随机 IO，比较耗时，所以上述修改可以省去很多回表操作的成本。这个改进称之为索引条件下推(英文名: ICP, Index Condition Pushdown)。

如果在查询语句的执行过程中将要使用索引条件下推这个特性，在 Extra 列中将会显示 Using index condition，比如这样：

```
mysql> EXPLAIN SELECT * FROM s1 WHERE order_no > 'z' AND order_no LIKE '%a';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | s1 | NULL | range | idx_order_no | idx_order_no | 152 | NULL | 1 | 100.00 | Using index condition |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

Using where

当我们使用全表扫描来执行对某个表的查询，并且该语句的 WHERE 子句中有针对该表的搜索条件时，在 Extra 列中会提示上述额外信息。

EXPLAIN SELECT * FROM s1 WHERE order_note = 'a';

```
mysql> EXPLAIN SELECT * FROM s1 WHERE order_note = 'a';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | s1 | NULL | ALL | NULL | NULL | NULL | NULL | 10573 | 10.00 | Using where |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

当使用索引访问来执行对某个表的查询，并且该语句的 WHERE 子句中有除了该索引包含的列之外的其他搜索条件时，在 Extra 列中也会提示上述信息。

比如下边这个查询虽然使用 idx_order_no 索引执行查询，但是搜索条件中除了包含 order_no 的搜索条件 order_no = 'a'，还有包含 order_note 的搜索条件，此时需要回表检索记录然后进行条件判断，所以 Extra 列会显示 Using where 的提示：

EXPLAIN SELECT * FROM s1 WHERE order_no = 'a' AND order_note = 'a';

```
mysql> EXPLAIN SELECT * FROM s1 WHERE order_no = 'a' AND order_note = 'a';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | s1 | NULL | ref | idx_order_no | idx_order_no | 152 | const | 1 | 10.00 | Using where |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

但是大家注意：出现了 Using where，只是表示在 server 层根据 where 条件进行了过滤，和是否全表扫描或读取了索引文件没有关系，网上有不少文章把 Using where 和是否读取索引进行关联，是不正确的，也有文章把 Using where 和回表进行了关联，这也是不对的。按照 MySQL 官方的说明：

https://dev.mysql.com/doc/refman/5.7/en/explain-output.html#explain_extra

Using where (JSON property: attached_condition)

A WHERE clause is used to restrict which rows to match against the next table or send to the client.

意思是：Extra 列中出现了 Using where 代表 WHERE 子句用于限制要与下一个表匹配或发送给客户端的行。

很明显，Using where 只是表示 MySQL 使用 where 子句中的条件对记录进行了过滤。

Using join buffer (Block Nested Loop)

在连接查询执行过程中，当被驱动表不能有效的利用索引加快访问速度，MySQL 一般会为其分配一块名叫 join buffer 的内存块来加快查询速度：

EXPLAIN SELECT * FROM s1 INNER JOIN s2 ON s1.order_note = s2.order_note;

```
mysql> EXPLAIN SELECT * FROM s1 INNER JOIN s2 ON s1.order_note = s2.order_note;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	s2	NULL	ALL	NULL	NULL	NULL	NULL	10315	100.00	NULL
1	SIMPLE	s1	NULL	ALL	NULL	NULL	NULL	NULL	10573	10.00	Using where; Using join buffer (Block Nested Loop)

2 rows in set, 1 warning (0.00 sec)

我们看到，在对 s1 表的执行计划的 Extra 列显示了两个提示：

Using join buffer (Block Nested Loop): 这是因为对表 s1 的访问不能有效利用索引，只好退而求其次，使用 join buffer 来减少对 s1 表的访问次数，从而提高性能。

Using where: 可以看到查询语句中有一个 `s1.order_note = s2.order_note` 条件，因为 `s2` 是驱动表，`s1` 是被驱动表，所以在访问 `s1` 表时，`s1.order_note` 的值已经确定下来了，所以实际上查询 `s1` 表的条件就是 `s1.order_note = 一个常数`，所以提示了 `Using where` 额外信息。

Not exists

当我们使用左（外）连接时，如果 **WHERE** 子句中包含要求被驱动表的某个列等于 **NULL** 值的搜索条件，而且那个列又是不允许存储 **NULL** 值的，那么在该表的执行计划的 **Extra** 列就会提示 **Not exists** 额外信息，比如这样：

```
EXPLAIN SELECT * FROM s1 LEFT JOIN s2 ON s1.order_no = s2.order_no
WHERE s2.id IS NULL;
```

```
mysql> EXPLAIN SELECT * FROM s1 LEFT JOIN s2 ON s1.order_no = s2.order_no WHERE s2.id IS NULL;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	s1	NULL	ALL	NULL	NULL	NULL	NULL	10573	100.00	NULL
1	SIMPLE	s2	NULL	ref	idx_order_no	idx_order_no	152	mysqgladv.s1.order_no	1	10.00	Using where; Not exists

2 rows in set, 1 warning (0.00 sec)

上述查询中 **s1** 表是驱动表，**s2** 表是被驱动表，**s2.id** 列是主键而且不允许存储 **NULL** 值的，而 **WHERE** 子句中又包含 **s2.id IS NULL** 的搜索条件。

Using `intersect(...)`、`union(...)` 和 `sort_union(...)`

如果执行计划的 Extra 列出现了 `Using intersect(...)`提示，说明准备使用 `Intersect` 索引合并的方式执行查询，括号中的...表示需要进行索引合并的索引名称；如果出现了 `Using union(...)`提示，说明准备使用 `Union` 索引合并的方式执行查询；出现了 `Using sort_union(...)`提示，说明准备使用 `Sort-Union` 索引合并的方式执行查询。什么是索引合并，我们后面会单独讲。

Zero limit

当我们的 **LIMIT** 子句的参数为 0 时,表示压根儿不打算从表中读出任何记录,将会提示该额外信息。

Using filesort

有一些情况下对结果集中的记录进行排序是可以使用到索引的,比如下边这个查询:

```
EXPLAIN SELECT * FROM s1 ORDER BY order no LIMIT 10;
```

```
mysql> EXPLAIN SELECT * FROM s1 ORDER BY order_no LIMIT 10;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	s1	NULL	index	NULL	idx_order_no	152	NULL	10	100.00	NULL

1 row in set, 1 warning (0.00 sec)

这个查询语句可以利用 `idx_order_no` 索引直接取出 `order_no` 列的 10 条记录，然后再进行回表操作就好了。但是很多情况下排序操作无法使用到索引，只能在内存中（记录较少的时候）或者磁盘中（记录较多的时候）进行排序，MySQL 把这种在内存中或者磁盘上进行排序的方式统称为文件排序。如果某个查询需要使用文件排序的方式执行查询，就会在执行计划的 `Extra` 列中显示 `Using filesort` 提示：

EXPLAIN SELECT * FROM s1 ORDER BY order_note LIMIT 10;

```
mysql> EXPLAIN SELECT * FROM s1 ORDER BY order_note LIMIT 10;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1  | SIMPLE     | s1    | NULL       | ALL  | NULL         | NULL | NULL    | NULL | 10573 | 100.00 | Using filesort |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

需要注意的是，如果查询中需要使用 `filesort` 的方式进行排序的记录非常多，那么这个过程是很耗费性能的，我们最好想办法将使用文件排序的执行方式改为使用索引进行排序。

Using temporary

在许多查询的执行过程中，MySQL 可能会借助临时表来完成一些功能，比如去重、排序之类的，比如我们在执行许多包含 `DISTINCT`、`GROUP BY`、`UNION` 等子句的查询过程中，如果不能有效利用索引来完成查询，MySQL 很有可能寻求通过建立内部的临时表来执行查询。如果查询中使用到了内部的临时表，在执行计划的 `Extra` 列将会显示 `Using temporary` 提示：

EXPLAIN SELECT DISTINCT order_note FROM s1;

```
mysql> EXPLAIN SELECT DISTINCT order_note FROM s1;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1  | SIMPLE     | s1    | NULL       | ALL  | NULL         | NULL | NULL    | NULL | 10573 | 100.00 | Using temporary |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

再比如：

EXPLAIN SELECT order_note, COUNT(*) AS amount FROM s1 GROUP BY order_note;

```
EXPLAIN SELECT order_note, COUNT(*) AS amount FROM s1 GROUP BY order_note;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1  | SIMPLE     | s1    | NULL       | ALL  | NULL         | NULL | NULL    | NULL | 10573 | 100.00 | Using temporary; Using filesort |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

上述执行计划的 `Extra` 列不仅仅包含 `Using temporary` 提示，还包含 `Using filesort` 提示，可是我们的查询语句中明明没有写 `ORDER BY` 子句呀？这是因为 MySQL 会在包含 `GROUP BY` 子句的查询中默认添加上 `ORDER BY` 子句，也就是说上述查询其实和下边这个查询等价：

EXPLAIN SELECT order_note, COUNT(*) AS amount FROM s1 GROUP BY order_note ORDER BY order_note;

如果我们并不想为包含 `GROUP BY` 子句的查询进行排序，需要我们显式的写上 `ORDER BY NULL`：

EXPLAIN SELECT order_note, COUNT(*) AS amount FROM s1 GROUP BY order_note order by null;

```
EXPLAIN SELECT order_note, COUNT(*) AS amount FROM s1 GROUP BY order_note order by null;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
SIMPLE      | s1    | NULL       | ALL  | NULL         | NULL | NULL    | NULL | 10573 | 100.00 | Using temporary |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
n set, 1 warning (0.01 sec)
```

这回执行计划中就没有 Using filesort 的提示了，也就意味着执行查询时可以省去对记录进行文件排序的成本了。

很明显，执行计划中出现 Using temporary 并不是一个好的征兆，因为建立与维护临时表要付出很大成本的，所以我们最好能使用索引来替代掉使用临时表，比方说下边这个包含 GROUP BY 子句的查询就不需要使用临时表：

EXPLAIN SELECT order_no, COUNT(*) AS amount FROM s1 GROUP BY order_no;

```
EXPLAIN SELECT order_no, COUNT(*) AS amount FROM s1 GROUP BY order_no;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
SIMPLE      | s1    | NULL       | index | idx_order_no | idx_order_no | 152    | NULL | 10573 | 100.00 | Using index |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
n set, 1 warning (0.00 sec)
```

从 Extra 的 Using index 的提示里我们可以看出，上述查询只需要扫描 idx_order_no 索引就可以搞定了，不再需要临时表了。

总的来说，发现在执行计划里面有 using filesort 或者 Using temporary 的时候，特别需要注意，这往往存在着很大优化的余地，最好进行改进，变为使用 Using index 会更好。

Start temporary, End temporary

有子查询时，查询优化器会优先尝试将 IN 子查询转换成 semi-join(半连接优化技术，本质上是把子查询上拉到父查询中，与父查询的表做 join 操作)，而 semi-join 又有好多种执行策略，当执行策略为 DuplicateWeedout 时，也就是通过建立临时表来实现为外层查询中的记录进行去重操作时，驱动表查询执行计划的 Extra 列将显示 Start temporary 提示，被驱动表查询执行计划的 Extra 列将显示 End temporary 提示。

LooseScan

在将 In 子查询转为 semi-join 时，如果采用的是 LooseScan 执行策略，则在驱动表执行计划的 Extra 列就是显示 LooseScan 提示。

FirstMatch(tbl_name)

在将 In 子查询转为 semi-join 时，如果采用的是 FirstMatch 执行策略，则在被驱动表执行计划的 Extra 列就是显示 FirstMatch(tbl_name)提示。

4.1.7.4. 复习二级索引 + 回表

一般情况下只能利用单个二级索引执行查询，比方说下边的这个查询：

```
SELECT * FROM order_exp WHERE insert_time = '2021-03-22 18:34:56' AND
order_no> '你好，李焕英。7 排 24 号,过期时长:DD00_24S';
```

查询优化器会识别到这个查询中的两个搜索条件：

`insert_time = '2021-03-22 18:34:56'`

`order_no > '你好，李焕英。7排24号,过期时长:DD00_24S'`

优化器一般会根据 `order_exp` 表的统计数据来判断到底使用哪个条件到对应的二级索引中查询扫描的行数会更少，选择那个扫描行数较少的条件到对应的二级索引中查询。

然后将从该二级索引中查询到的结果经过回表得到完整的用户记录后再根据其余的 `WHERE` 条件过滤记录。一般来说，等值查找比范围查找需要扫描的行数更少（也就是 `ref` 的访问方法一般比 `range` 好，但这也不总是一定的，也可能采用 `ref` 访问方法的那个索引列的值为特定值的行数特别多）。所以这里假设优化器决定使用 `idx_insert_time` 索引进行查询，那么整个查询过程可以分为两个步骤：

步骤 1：使用二级索引定位记录的阶段，也就是根据条件 `insert_time = '2021-03-22 18:34:56'` 从 `idx_insert_time` 索引代表的 B+ 树中找到对应的二级索引记录。

步骤 2：回表阶段，也就是根据上一步骤中找到的记录的主键值进行回表操作，也就是到聚簇索引中找到对应的完整的用户记录，再根据条件 `order_no > '你好，李焕英。7排24号,过期时长:DD00_24S'` 到完整的用户记录继续过滤。将最终符合过滤条件的记录返回给用户。

很明显，因为二级索引的节点中的记录只包含索引列和主键，所以在步骤 1 中使用 `idx_insert_time` 索引进行查询时只会用到与 `insert_time` 列有关的搜索条件，其余条件，比如 `order_no > '.....'` 这个条件在步骤 1 中是用不到的，只有在步骤 2 完成回表操作后才能继续针对完整的用户记录中继续过滤。

MRR

从上文可以看出，每次从二级索引中读取到一条记录后，就会根据该记录的主键值执行回表操作。而在某个扫描区间中的二级索引记录的主键值是无序的，也就是说这些二级索引记录对应的聚簇索引记录所在的页面的页号是无序的。

每次执行回表操作时都相当于要随机读取一个聚簇索引页面，而这些随机 IO 带来的性能开销比较大。MySQL 中提出了一个名为 `Disk-Sweep Multi-Range Read (MRR)`，多范围读取的优化措施，即先读取一部分二级索引记录，将它们的主键值排好序之后再统一执行回表操作。

相对于每读取一条二级索引记录就立即执行回表操作，这样会节省一些 IO 开销。使用这个 `MRR` 优化措施的条件比较苛刻，所以我们直接认为每读取一条二级索引记录就立即执行回表操作。`MRR` 的详细信息，可以查询官方文档。

4.1.8. 高性能的索引使用策略

4.2.8.1. 不在索引列上做任何操作

我们通常会看到一些查询不当地使用索引，或者使得 MySQL 无法使用已有的索引。如果查询中的列不是独立的，则 MySQL 就不会使用索引。“独立的列”是指索引列不能是表达式的一部分，也不能是函数的参数。

例如，我们假设 `order_status` 上有索引，但是下面这个查询无法使用 `order_status` 列的索引：

```
mysql>SELECT * FROM order_exp WHERE order_status + 1 = 1;
```

凭肉眼很容易看出 `WHERE` 中的表达式其实等价于 `order_status = 0`，但是 MySQL 无法自动解析这个方程式。这完全是用户行为。我们应该养成简化 `WHERE` 条件的习惯，始终将索引列单独放在比较符号的一侧。

下面是另一个常见的错误：

```
mysql>SELECT ... WHERE TO_DAYS(insert_time) - TO_DAYS(expire_time) <= 10;
```

在索引列上使用函数，也是无法利用索引的。

4.2.8.2. 尽量全值匹配

建立了联合索引列后，如果我们的搜索条件中的列和索引列一致的话，这种情况就称为全值匹配，比方说下边这个查找语句：

```
select * from order_exp where insert_time='2021-03-22 18:34:55' and  
order_status=0 and expire_time='2021-03-22 18:35:14';
```

我们建立的 `u_idx_day_statusr` 索引包含的 3 个列在这个查询语句中都展现出来了，联合索引中的三个列都可能被用到。

有的同学也许有个疑问，`WHERE` 子句中的几个搜索条件的顺序对查询结果有啥影响么？也就是说如果我们调换 `'insert_time'`、`'order_status'`、`'expire_time'` 这几个搜索列的顺序对查询的执行过程有影响么？比方说写成下边这样：

```
Select * from order_exp where order_status=1 and expire_time='2021-03-22  
18:35:14' and insert_time='2021-03-22 18:34:55';
```

放心，MySQL 没那么蠢，查询优化器会分析这些搜索条件并且按照可以使用的索引中列的顺序来决定先使用哪个搜索条件，后使用哪个搜索条件。

所以，当建立了联合索引列后，能在 `where` 条件中使用索引的尽量使用。

4.2.8.3. 最佳左前缀法则

建立了联合索引列，如果搜索条件不够全值匹配怎么办？在我们的搜索语句中也可以不用包含全部联合索引中的列，但要遵守最佳左前缀法则。指的是查询从索引的最左前列开始并且不跳过索引中的列。

比如：

4.2.8.5. 覆盖索引尽量用

覆盖索引是非常有用的工具，能够极大地提高性能，三星索引里最重要的那颗星就是宽索引星。考虑一下如果查询只需要扫描索引而无须回表，会带来多少好处：

索引条目通常远小于数据行大小，所以如果只需要读取索引，那 MySQL 就会极大地减少数据访问量。这对缓存的负载非常重要，因为这种情况下响应时间大部分花费在数据拷贝上。覆盖索引对于 I/O 密集型的应用也有帮助，因为索引比数据更小，更容易全部放入内存中。

因为索引是按照列值顺序存储的，所以对于 I/O 密集型的范围查询会比随机从磁盘读取每一行数据的 I/O 要少得多。

由于 InnoDB 的聚簇索引，覆盖索引对 InnoDB 表特别有用。InnoDB 的二级索引在叶子节点中保存了行的主键值，所以如果二级主键能够覆盖查询，则可以避免对主键索引的二次查询。

尽量使用覆盖索引(只访问索引的查询(索引列和查询列一致))，不是必要的情况下减少 `select *`，除非是需要将表中的全部列检索后，进行缓存。

```
select * from order_exp_cut where insert_time='2021-03-22 18:34:55' and
order_status=0 and expire_time>'2021-03-22 18:23:57' and
expire_time<'2021-03-22 18:35:00';
```

和

```
select expire_time,id from order_exp_cut where insert_time='2021-03-22
18:34:55' and order_status=0 and expire_time>'2021-03-22 18:23:57' and
expire_time<'2021-03-22 18:35:00';
```

表现肯定是不一样的。

```
mysql> explain select * from order_exp_cut where insert_time='2021-03-22 18:34:55' and order_status=0 and expire_time='2021-03-22 18:35:04';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | order_exp_cut | NULL | const | u_idx_day_status | u_idx_day_status | 12 | const,const,const | 1 | 100.00 | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> explain select expire_time,id from order_exp_cut where insert_time='2021-03-22 18:34:55' and order_status=0 and expire_time='2021-03-22 18:35:04';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | order_exp_cut | NULL | const | u_idx_day_status | u_idx_day_status | 12 | const,const,const | 1 | 100.00 | Using index |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

4.2.8.6. 不等于要慎用

mysql 在使用不等于(`!=` 或者 `<>`)的时候无法使用索引会导致全表扫描

```
SELECT * FROM order_exp WHERE order_no <> 'DD00_6S';
```

```
mysql> explain SELECT * FROM order_exp WHERE order_no <> 'DD00_6S';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | order_exp | NULL | ALL | idx_order_no | NULL | NULL | NULL | 10350 | 55.91 | Using where |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.01 sec)
```

为什么？道理是明显的，这种情况下，扫描区间是[第一条记录,'DD00_6S']和['DD00_6S',最后一条记录]，加上回表，还不如直接进行全部扫描。

4.2.8.7. Null/Not 有影响

需要注意 null/not null 对索引的可能影响

explain SELECT * FROM order_exp WHERE order_no is null;

explain SELECT * FROM order_exp WHERE order_no is not null;

```
mysql> explain SELECT * FROM order_exp WHERE order_no is null;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | NULL | NULL | NULL | NULL | NULL | NULL | NULL | NULL | NULL | Impossible WHERE |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> explain SELECT * FROM order_exp WHERE order_no is not null;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | order_exp | NULL | ALL | idx_order_no | NULL | NULL | NULL | 10350 | 90.00 | Using where |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

order_no 为索引列，同时不允许为 null，

id	bigint	22	0	☒	☐	
order_no	varchar	50	0	☒	☐	
order_note	varchar	100	0	☒	☐	
insert_time	datetime	0	0	☒	☐	
expire_duration	bigint	22	0	☒	☐	
expire_time	datetime	0	0	☒	☐	
order_status	smallint	6	0	☒	☐	

可以看见，order_no is null 的情况下，MySQL 直接表示 Impossible WHERE，对于 is not null 直接走的全表扫描。

当 order_no 允许为 null 时：

explain SELECT * FROM order_exp_cut WHERE order_no is null;

explain SELECT * FROM order_exp_cut WHERE order_no is not null;

```
mysql> explain SELECT * FROM order_exp_cut WHERE order_no is null;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | order_exp_cut | NULL | ref | idx_order_no | idx_order_no | 153 | const | 1 | 100.00 | Using index condition |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> explain SELECT * FROM order_exp_cut WHERE order_no is not null;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | order_exp_cut | NULL | ALL | idx_order_no | NULL | NULL | NULL | 10509 | 50.00 | Using where |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

is null 会走 ref 类型的索引访问，is not null;依然是全表扫描。所以总结起来：

is not null 容易导致索引失效，is null 则会区分被检索的列是否为 null，如果是 null 则会走 ref 类型的索引访问，如果不为 null，也是全表扫描。

但是当联合索引上使用时覆盖索引时，情况又不同了：

explain SELECT order_status,expire_time FROM order_exp WHERE insert_time is null;

explain SELECT order_status,expire_time FROM order_exp WHERE insert_time is not null;

```
mysql> explain SELECT order_status,expire_time FROM order_exp WHERE insert_time is null;
+----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len |
+----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | order_exp | NULL | ref | u_idx_day_status | u_idx_day_status | 5 |
+----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> explain SELECT order_status,expire_time FROM order_exp WHERE insert_time is not null;
+----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len |
+----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | order_exp | NULL | index | u_idx_day_status | u_idx_day_status | 12 |
+----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

explain SELECT order_status,expire_time FROM order_exp_cut WHERE insert_time is null;

explain SELECT order_status,expire_time FROM order_exp_cut WHERE insert_time is not null;

```
mysql> explain SELECT order_status,expire_time FROM order_exp_cut WHERE insert_time is null;
+----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len |
+----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | order_exp_cut | NULL | ref | u_idx_day_status | u_idx_day_status | 6 |
+----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> explain SELECT order_status,expire_time FROM order_exp_cut WHERE insert_time is not null;
+----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len |
+----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | order_exp_cut | NULL | range | u_idx_day_status | u_idx_day_status | 6 |
+----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

所以总的来说，在设计表时列尽可能的不要声明为 null，在章节中《2、MySQL 中表设计和数据类型优化》也强调过这一点。

4.2.8.8. Like 查询要当心

like 以通配符开头('%abc...')，mysql 索引失效会变成全表扫描的操作

explain SELECT * FROM order_exp WHERE order_no like '%_6S';

```
mysql> explain SELECT * FROM order_exp WHERE order_no like '%_6S';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | order_exp | NULL | ALL | NULL | NULL | NULL | NULL | 10350 | 11.11 | Using where |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

原因前面已经讲过了。此时如果使用覆盖索引可以改善这个问题

SELECT order_status,expire_time FROM order_exp_cut WHERE insert_time like '%18:35:09';

```
mysql> explain SELECT order_status,expire_time FROM order_exp_cut WHERE insert_time like '%18:35:09';
+----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len |
+----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | order_exp_cut | NULL | index | NULL | u_idx_day_status | 13 |
+----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 2 warnings (0.00 sec)
```

4.2.8.9. 字符类型加引号

字符串不加单引号索引失效

explain SELECT * FROM order_exp WHERE order_no = 6;

```
mysql> explain SELECT * FROM order_exp WHERE order_no = 6;
+----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key |
+----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | order_exp | NULL | ALL | idx_order_no | NULL |
+----+-----+-----+-----+-----+-----+-----+
1 row in set, 3 warnings (0.00 sec)
```

MySQL 的查询优化器，会自动的进行类型转换，比如上个语句里会尝试将 order_no 转换为数字后和 6 进行比较，自然造成索引失效。

4.2.8.10. 使用 or 关键字时要注意

explain SELECT * FROM order_exp WHERE order_no = 'DD00_6S' OR order_no = 'DD00_9S';

和

explain SELECT * FROM order_exp WHERE expire_time= '2021-03-22 18:35:09' OR order_note = 'abc';

```
mysql> explain SELECT * FROM order_exp WHERE order_no = 'DD00_6S' OR order_no = 'DD00_9S';
+----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows |
+----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | order_exp | NULL | range | idx_order_no | idx_order_no | 152 | NULL | 3 |
+----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> explain SELECT * FROM order_exp WHERE expire_time= '2021-03-22 18:35:09' OR order_note = 'abc';
+----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows |
+----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | order_exp | NULL | ALL | idx_expire_time | NULL | NULL | NULL | 10350 |
+----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

表现是不一样的，具体原因在“3.2.3. 深入思考索引在查询中的使用”中已经说明过原因了。

当然如果两个条件都是索引列，情况会有变化：

explain SELECT * FROM order_exp WHERE expire_time= '2021-03-22 18:35:09' OR order_no = 'DD00_6S';

```
mysql> explain SELECT * FROM order_exp WHERE expire_time= '2021-03-22 18:35:09' OR order_no = 'DD00_6S';
+----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key |
+----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | order_exp | NULL | index_merge | idx_order_no,idx_expire_time | idx_expire_time,idx_order_no |
+----+-----+-----+-----+-----+-----+-----+
Extra: Using union(idx_expire_time,idx_order_no); Using where
1 row in set, 1 warning (0.00 sec)
```

这也给了我们提示，如果我们将 **SELECT * FROM order_exp WHERE expire_time= '2021-03-22 18:35:09' OR order_note = 'abc';**改为：

explain SELECT * FROM order_exp WHERE expire_time= '2021-03-22 18:35:09' union all SELECT * FROM order_exp WHERE order_note = 'abc';


```
mysql> explain SELECT * FROM order_exp WHERE expire_time= '2021-03-22 18:35:09' union all SELECT * FROM order_exp WHERE order_note = 'abc';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	order_exp	NULL	ref	idx_expire_time	idx_expire_time	5	const	6	100.00	NULL
2	UNION	order_exp	NULL	ALL	NULL	NULL	NULL	NULL	10350	10.00	Using where

```
2 rows in set, 1 warning (0.00 sec)
```

可以改善语句的表现。

当然使用覆盖扫描也可以改善这个问题：

```
explain SELECT order_status,id FROM order_exp_cut WHERE  
insert_time='2021-03-22 18:34:55' or expire_time='2021-03-22 18:28:28';
```

```
mysql> explain SELECT order_status,id FROM order_exp_cut WHERE insert_time='2021-03-22 18:34:55' or expire_time='2021-03-22 18:28:28';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	order_exp_cut	NULL	index	u_idx_day_status	u_idx_day_status	13	NULL	10547	10.09	Using index

```
1 row in set, 1 warning (0.00 sec)
```

4.2.8.11. 使用索引扫描来做排序和分组

MySQL 有两种方式可以生成有序的结果；通过排序操作；或者按索引顺序扫描；如果 EXPLAIN 出来的 type 列的值为“index”，则说明 MySQL 使用了索引扫描来做排序（不要和 Extra 列的“Using index”搞混淆了）。

扫描索引本身是很快，因为只需要从一条索引记录移动到紧接着的下一条记录。但如果索引不能覆盖查询所需的全部列，那就不得不每扫描一条索引记录就都回表查询一次对应的行。这基本上都是随机 I/O，因此按索引顺序读取数据的速度通常要比顺序地全表扫描慢，尤其是在 IO 密集型的工作负载时。

MySQL 可以使用同一个索引既满足排序，又用于查找行。因此，如果可能，设计索引时应该尽可能地同时满足这两种任务，这样是最好的。

只有当索引的列顺序和 ORDER BY 子句的顺序完全一致，并且所有列的排序方向（倒序或正序）都一样时，MySQL 才能够使用索引来对结果做排序。如果查询需要关联多张表，则只有当 ORDER BY 子句引用的字段全部为第一个表时，才能使用索引做排序。

4.1.8.12. 排序要当心

ASC、DESC 别混用

对于使用联合索引进行排序的场景，我们要求各个排序列的排序顺序是一致的，也就是要么各个列都是 ASC 规则排序，要么都是 DESC 规则排序。联合索引为何是这样，前面已经说过很多次了。

排序列包含非同一个索引的列

用来排序的多个列不是一个索引里的，这种情况也不能使用索引进行排序

```
explain SELECT * FROM order_exp order by order_no,insert_time;
```



```
mysql> explain SELECT * FROM order_exp order by order no,insert time;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | order_exp | NULL | ALL | NULL | NULL | NULL | NULL | 10350 | 100.00 | Using filesort |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

4.2.8.13. 尽可能按主键顺序插入行

最好避免随机的（不连续且值的分布范围非常大）聚簇索引，特别是对于 I/O 密集型的应用。例如，从性能的角度考虑，使用 UUID 来作为聚簇索引则会很糟糕，它使得聚簇索引的插入变得完全随机，这是最坏的情况，使得数据没有任何聚集特性。

最简单的方法是使用 **AUTO_INCREMENT** 自增列。这样可以保证数据行是按顺序写入，对于根据主键做关联操作的性能也会更好。

注意到向 **UUID** 主键插入行不仅花费的时间更长，而且索引占用的空间也更大。这一方面是由于主键字段更长；另一方面毫无疑问是由于页分裂和碎片导致的。

因为主键的值是顺序的，所以 InnoDB 把每一条记录都存储在上一条记录的后面。当达到页的最大填充因子时(InnoDB 默认的最大填充因子是页大小的 15/16，留出部分空间用于以后修改)，下一条记录就会写入新的页中。一旦数据按照这种顺序的方式加载，主键页就会近似于被顺序的记录填满，这也正是所期望的结果。

如果新行的主键值不一定比之前插入的大，所以 InnoDB 无法简单地总是把新行插入到索引的最后，而是需要为新的行寻找合适的位置——通常是已有数据的中间位置——并且分配空间。这会增加很多的额外工作，并导致数据分布不够优化。下面是总结的一些缺点：

写入的目标页可能已经刷到磁盘上并从缓存中移除，或者是还没有被加载到缓存中，InnoDB 在插入之前不得不先找到并从磁盘读取目标页到内存中。这将导致大量的随机 IO。

因为写入是乱序的，InnoDB 不得不频繁地做页分裂操作，以便为新的行分配空间。页分裂会导致移动大量数据，一次插入最少需要修改三个页而不是一个页。

所以使用 InnoDB 时应该尽可能地按主键顺序插入数据，并且尽可能地使用单调增加的聚簇键的值来插入新行。

4.1.8.14. 优化 Count 查询

首先要注意，**COUNT()**是一个特殊的函数，有两种非常不同的作用：它可以统计某个列值的数量，也可以统计行数。

在统计列值时要求列值是非空的（不统计 **NULL**）。

COUNT()的另一个作用是统计结果集的行数。常用的就是当我们使用 **COUNT(*)**。实际上，它会忽略所有的列而直接统计所有的行数。

c1	2	select count(c1) from test;
1000	3	select count(*) from test;
2	4	
(Null)	信息	结果 1
(Null)	剖析	状态
	count(c1)	
	2	

3	select count(*) from test;
4	
信息	结果 1
剖析	状态
count(*)	
4	

通常来说，COUNT()都需要扫描大量的行（意味着要访问大量数据）才能获得精确的结果，因此是很难优化的。在 MySQL 层面能做的基本只有索引覆盖扫描了。如果这还不够,就需要考虑修改应用的架构，可以用估算值取代精确值，可以增加汇总表，或者增加类似 Redis 这样的外部缓存系统。

关于 Null 的特别说明

对于 Null 到底算什么，存在着分歧：

1、有的认为 NULL 值代表一个未确定的值，MySQL 认为任何和 NULL 值做比较的表达式值都为 NULL，包括 select null=null 和 select null!=null;

```
mysql> select null=null;
+-----+
| null=null |
+-----+
| NULL      |
+-----+
1 row in set (0.00 sec)

mysql> select null!=null;
+-----+
| null!=null |
+-----+
| NULL      |
+-----+
1 row in set (0.00 sec)
```

所以每一个 NULL 值都是独一无二的。

2、有的认为其实 NULL 值在业务上就是代表没有，所有的 NULL 值和起来算一份；

3、有的认为这 NULL 完全没有意义，所以在统计数量时压根儿不能把它们算进来。

假设一个表中某个列 c1 的记录为(2,1000,null,null)，在第一种情况下，表中 c1 的记录数为 4，第二种表中 c1 的记录数为 3，第三种表中 c1 的记录数为 2。

c1
(Null)
(Null)
2
1000

但是在我们后面将要学到的《优化背后的原理-InnoDB 中的统计数据》章节时，我们会学到 MySQL 会统计索引中不重复值的数量，专门用于语句的执行计划分析。在对统计索引列不重复值的数量时如何对待 NULL 值，MySQL 专门提供了一个 `innodb_stats_method` 的系统变量，

https://dev.mysql.com/doc/refman/5.7/en/innodb-parameters.html#sysvar_innodb_stats_method

这个系统变量有三个候选值：

`nulls_equal`: 认为所有 NULL 值都是相等的。这个值也是 `innodb_stats_method` 的默认值。

如果某个索引列中 NULL 值特别多的话，这种统计方式会让优化器认为某个列中平均一个值重复次数特别多，所以倾向于不使用索引进行访问。

`nulls_unequal`: 认为所有 NULL 值都是不相等的。

如果某个索引列中 NULL 值特别多的话，这种统计方式会让优化器认为某个列中平均一个值重复次数特别少，所以倾向于使用索引进行访问。

`nulls_ignored`: 直接把 NULL 值忽略掉。

而且有迹象表明，在 MySQL5.7.22 以后的版本，对这个 `innodb_stats_method` 的修改不起作用，MySQL 把这个值在代码里写死为 `nulls_equal`。也就是说 MySQL 在进行索引列的数据统计行为又把 null 视为第二种情况（NULL 值在业务上就是代表没有，所有的 NULL 值和起来算一份），看起来，MySQL 中对 Null 值的处理也很分裂。所以总的来说，对于列的声明尽可能的不要允许为 null。

4.1.8.15. 优化 limit 分页

在系统中需要进行分页操作的时候，我们通常会使用 LIMIT 加上偏移量的办法实现，同时加上合适的 ORDER BY 子句。

一个非常常见又令人头疼的问题就是，在偏移量非常大的时候，例如可能是 **`select * from order_exp limit 10000,10;`**

这样的查询，这时 MySQL 需要查询 10010 条记录然后只返回最后 10 条，前面 10 000 条记录都将被抛弃，这样的代价非常高。

优化此类分页查询的一个最简单的办法是

`SELECT * FROM (select id from order_exp limit 10000,10) b,order_exp a where a.id = b.id;`

会先查询翻页中需要的 N 条数据的主键值，然后根据主键值回表查询所需要的 N 条数据，在此过程中查询 N 条数据的主键 id 在索引中完成，所以效率会高一些。

```
mysql> explain select * from order_exp limit 10000,10;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	order_exp	NULL	ALL	NULL	NULL	NULL	NULL	10350	100.00	NULL

1 row in set, 1 warning (0.00 sec)

```
mysql> explain SELECT * FROM (select id from order_exp limit 10000,10) b,order_exp a where a.id = b.id;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	<derived2>	NULL	ALL	NULL	NULL	NULL	NULL	10010	10	
1	PRIMARY	a	NULL	eq_ref	PRIMARY	PRIMARY	8	b.id	1	10	
2	DERIVED	order_exp	NULL	index	NULL	idx_expire_time	5	NULL	10350	10	

3 rows in set, 1 warning (0.00 sec)

从执行计划中可以看出，首先执行子查询中的 `order_exp` 表，根据主键做索引全表扫描，然后与 `a` 表通过 `id` 做主键关联查询，相比传统写法中的全表扫描效率会高一些。

从两种写法上能看出性能有一定的差距，虽然并不明显，但是随着数据量的增大，两者执行的效率便会体现出来。

上面的写法虽然可以达到一定程度的优化，但还是存在性能问题。最佳的方式是在业务上进行配合修改为以下语句：

select * from order_exp where id > 67 order by id limit 10;

```
mysql> explain select * from order_exp where id > 67 order by id limit 10;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	order_exp	NULL	range	PRIMARY	PRIMARY					

1 row in set, 1 warning (0.00 sec)

采用这种写法，需要前端通过点击 **More** 来获得更多数据，而不是纯粹的翻页，因此，每次查询只需要使用上次查询出的数据中的 `id` 来获取接下来的数据即可，但这种写法需要业务配合。

4.1.8.16. 记忆总结

- 全职匹配我最爱，最左前缀要遵守；
- 带头大哥不能死，中间兄弟不能断；
- 索引列上少计算，范围之后全失效；
- LIKE 百分写最右，覆盖索引不写*；
- 不等空值还有 OR，索引影响要注意；
- VAR 引号不可丢，SQL 优化有诀窍。

4.3. 分区表

4.3.1. 简介

分区是指根据一定的规则，数据库把一个表分解成多个更小的、更容易管理的部分。就访问数据库的应用而言，逻辑上只有一个表或一个索引,但是实际上这个表可能由数 10 个物理分区对象组成，每个分区都是一个独立的对象，可

以独自处理，可以作为表的一部分进行处理。分区对应用来说是完全透明的，不影响应用的业务逻辑。

从上面的说明我们可以看到，分区表是一个独立的逻辑表，但是底层由多个物理子表组成。实现分区的代码实际上是对一组底层表的封装。对分区表的请求，都会转化成对存储引擎的接口调用。所以分区对于 SQL 层来说是一个完全封装底层实现的黑盒子，对应用是透明的，但是如果我们从底层的文件系统来看就会发现，每一个分区表都有一个使用#分隔命名的表文件。

MySQL 在创建表时使用 **PARTITION BY** 子句定义每个分区存放的数据。在执行查询的时候，优化器会根据分区定义过滤那些没有我们需要数据的分区，这样查询就无须扫描所有分区——只需要查找包含需要数据的分区就可以了。

分区的一个主要目的是将数据按照一个较粗的粒度分在不同的表中。这样做可以将相关的数据存放在一起，另外，如果想一次批量删除整个分区的数据也会变得很方便。在下面的场景中，分区可以起到非常大的作用

- 表非常大以至于无法全部都放在内存中，或者只在表的最后部分有热点数据，其他均是历史数据。
- 分区表的数据更容易维护。例如，想批量删除大量数据可以使用清除整个分区的方式。另外，还可以对一个独立分区进行优化、检查、修复等操作。
- 分区表的数据可以分布在不同的物理设备上，从而高效地利用多个硬件设备。可以使用分区表来避免某些特殊的瓶颈，例如 InnoDB 的单个索引的互斥访问、ext3 文件系统的 inode 锁竞争等。
- 如果需要，还可以备份和恢复独立的分区，这在非常大的数据集的场景下效果非常好。

分区表本身也有一些限制，下面是其中比较重要的几点：

- 一个表最多只能有 1024 个分区。
- 如果分区字段中有主键或者唯一索引的列，那么所有主键列和唯一索引列都必须包含进来。
- 分区表中无法使用外键约束。

4.3.2. 分区表的原理

如前所述，分区表由多个相关的底层表实现，这些底层表也是由句柄对象（Handlerobject）表示，所以我们可以直接访问各个分区。存储引擎管理分区的各个底层表和管理普通表一样（所有的底层表都必须使用相同的存储引擎），分区表的索引只是在各个底层表上各自加上一个完全相同的索引。从存储引擎的角度来看，底层表和一个普通表没有任何不同，存储引擎也无须知道这是一个普通表还是一个分区表的一部分。分区表上的操作按照下面的操作逻辑进行：

虽然每个操作都会“先打开并锁住所有的底层表”，但这并不是说分区表在处理过程中是锁住全表的。如果存储引擎能够自己实现行级锁，例如 InnoDB，则会在分区层释放对应表锁。这个加锁和解锁过程与普通 InnoDB 上的查询类似。

4.3.3. 分区表的类型

MySQL 支持多种分区表。

RANGE 分区：基于属于一个给定连续区间的列值，把多行分配给分区。

LIST 分区：类似于按 **RANGE** 分区，区别在于 **LIST** 分区是基于列值匹配一个离散值集合中的某个值来进行选择。

HASH 分区：基于用户定义的表达式的返回值来进行选择的分区，该表达式使用将要插入到表中的这些行的列值进行计算。这个函数可以包含 MySQL 中有效的、产生非负整数值的任何表达式。

KEY 分区：类似于按 **HASH** 分区，区别在于 **KEY** 分区只支持计算一列或多列，且 MySQL 服务器提供其自身的哈希函数。必须有一列或多列包含整数值。

复合分区/子分区：目前只支持 **RANGE** 和 **LIST** 的子分区，且子分区的类型只能为 **HASH** 和 **KEY**。

分区的基本语法如下：

```
CREATE TABLE test (  
  order_date DATETIME NOT NULL,  
) ENGINE=InnoDB  
PARTITION BY RANGE(YEAR(order_date))(  
  PARTITION p_0 VALUES LESS THAN (2010) ,  
  PARTITION p_1 VALUES LESS THAN (2011),  
  PARTITION p_2 VALUES LESS THAN (2012),  
  PARTITION p_other VALUES LESS THAN MAXVALUE );
```

按照这种分区方案，那么 **order_date** 在 2010 年以前（不包括 2010 年）的记录进入 **p_0** 分区，年份在 2010 年的进入 **p_1** 分区，年份在 2011 年的进入 **p_2** 分区，其他年份的进入 **p_other** 分区。

LIST 分区有点类似枚举，比如：

```
CREATE TABLE h2 (  
  c1 INT,  
  c2 INT  
PARTITION BY LIST(c1) (  
  PARTITION p0 VALUES IN (1, 4, 7),  
  PARTITION p1 VALUES IN (2, 5, 8));
```

意味着 insert 时，**c1** 的值只能在 1, 4, 7, 2, 5, 8 中，不在这个范围内插入就会报错。

在上面的例子中，我们看见 **range** 和 **List** 都是整数类型分区，其实 **range** 和 **List** 也支持非整数分区，但是要结合 **COLUMN** 分区，支持整形、日期、字符串，比如：

```
CREATE TABLE emp_date(  
  id INT NOT NULL,  
  ename VARCHAR (30),  
  hired DATE NOT NULL DEFAULT '1970-01-01',  
  separated DATE NOT NULL DEFAULT '9999-12-31',job VARCHAR(30) NOT  
NULL,  
  store_id INT NOT NULL)  
PARTITION BY RANGE COLUMNS (separated)(  
  PARTITION p0 VALUES LESS THAN ( '1996-01-01'),  
  PARTITION p1 VALUES LESS THAN (2001-01-01'),
```

```
PARTITION p2 VALUES LESS THAN ( '2006-01-01'));
```

和

```
CREATE TABLE expenses (  
expense_date DATE NOT NULL,  
category VARCHAR(30),  
amount DECIMAL (10,3)  
)  
PARTITION BY LIST COLUMNS (category)(  
PARTITION p0 VALUES IN ('a','b') ,  
PARTITION p1 VALUES IN('c', 'd'),  
PARTITION p2 VALUES IN('e','f'),  
PARTITION p3 VALUES IN('g'),  
PARTITION p4 VALUES IN('h'));
```

在结合 COLUMN 分区时还支持多列比如:

```
CREATE TABLE rc3(  
a INT,  
b INT  
)  
PARTITION BY RANGE COLUMNS(a,b)(  
PARTITION p01 VALUES LESS THAN (0,10),  
PARTITION p02 VALUES LESS THAN (10, 10),  
PARTITION p03 VALUES LESS THAN (10,20),  
PARTITION p04 VALUES LESS THAN (10,35),  
PARTITION p05 VALUES LESS THAN (10,MAXVALUE),  
PARTITION p06 VALUES LESS THAN (MAXVALUE,MAXVALUE));
```

Hash 分区如下:

```
CREATE TABLE emp (  
id INT NOT NULL,  
ename VARCHAR(30),  
hired DATE NOT NULL DEFAULT '1970-01-01'  
separated DATENOT NULL DEFAULT '9999-12-31',  
job VARCHAR(30) NOT NULL,  
store_id INT NOT NULL)  
PARTITION BY HASH (store_id) PARTITIONS 4;
```

这里我们创建了一个基于 store_id 列 HASH 分区的表,表被分成了 4 个分区,如果我们插入的记录 store_id=234, 则 $234 \bmod 4 = 2$, 这条记录就会保存到第二个分区。虽然我们在 HASH () 中直接使用的 store_id 列,但是 MySQL 是允许基于某列值返回一个整数值的表达式或者 MySQL 中有效的任何函数或者其他表达式都是可以的。

Key 分区语法如下:

```
CREATE TABLE emp (  
id INT NOT NULL,  
ename VARCHAR(30),  
hired DATE NOT NULL DEFAULT '1970-01-01'
```

```
separated DATENOT NULL DEFAULT '9999-12-31',
job VARCHAR(30) NOT NULL,
store_id INT NOT NULL)
PARTITION BY KEY (job) PARTITIONS 4;
```

上面创建了一个基于 job 字段进行 Key 分区的表，表被分成了 4 个分区。
KEY ()里只允许出现表中的字段。

更详细的其他的分区语法请自行查阅官方文档。因为我不建议使用 MySQL 的分区。

4.3.4. 不建议 mysql 分区表

但在实际的互联网中，MySQL 分区表用的极少，更多的是自己分库分表。

分库分表除了支持 MySQL 分区表的水平切分以外，还支持垂直切分，把一个很大的库（表）的数据分到几个库（表）中，每个库（表）的结构都相同，但他们可能分布在不同的 mysql 实例，甚至不同的物理机器上，以达到降低单库（表）数据量，提高访问性能的目的。

两者对比起来

- 1) 分区表，分区键设计不太灵活，如果不走分区键，很容易出现全表锁
- 2) 一旦数据量并发量上来，如果在分区表实施关联，就是一个灾难
- 3) 自己分库分表，自己掌控业务场景与访问模式，可控。分区表，研发写了一个 sql，都不确定 mysql 是怎么操作的，不太可控
- 4) 分区表无论怎么分，都是在一台机器上，天然就有性能的上限。

5. MySQL 的执行原理

5.1. 单表访问之索引合并

我们前边说过 MySQL 在一般情况下执行一个查询时最多只会用到单个二级索引，但存在有特殊情况，在这些特殊情况下也可能在一个查询中使用到多个二级索引，MySQL 中这种使用到多个索引来完成一次查询的执行方法称之为：索引合并/index merge，具体的索引合并算法有下边三种。

5.1.1. Intersection 合并

Intersection 翻译过来的意思是交集。这里是说某个查询可以使用多个二级索引，将从多个二级索引中查询到的结果取交集，比方说下边这个查询：

```
SELECT * FROM order_exp WHERE order_no = 'a' AND expire_time = 'b';
```

假设这个查询使用 Intersection 合并的方式执行的话，那这个过程就是这样的：

从 idx_order_no 二级索引对应的 B+树中取出 order_no='a'的相关记录。

从 idx_insert_time 二级索引对应的 B+树中取出 insert_time='b'的相关记录。

二级索引的记录都是由索引列 + 主键构成的，所以我们可以计算出这两个结果集中 id 值的交集。

按照上一步生成的 id 值列表进行回表操作，也就是从聚簇索引中把指定 id 值的完整用户记录取出来，返回给用户。

为啥不直接使用 idx_order_no 或者 idx_insert_time 只根据某个搜索条件去读取一个二级索引，然后回表后再过滤另外一个搜索条件呢？这里要分析一下两种查询执行方式之间需要的成本代价。

只读取一个二级索引的成本：

按照某个搜索条件读取一个二级索引，根据从该二级索引得到的主键值进行回表操作，然后再过滤其他的搜索条件

读取多个二级索引之后取交集成本：

按照不同的搜索条件分别读取不同的二级索引，将从多个二级索引得到的主键值取交集，然后进行回表操作。

虽然读取多个二级索引比读取一个二级索引消耗性能，但是大部分情况下读取二级索引的操作是顺序 I/O，而回表操作是随机 I/O，所以如果只读取一个二级索引时需要回表的记录数特别多，而读取多个二级索引之后取交集的记录数非常少，当节省的因为回表而造成的性能损耗比访问多个二级索引带来的性能损耗更高时，读取多个二级索引后取交集比只读取一个二级索引的成本更低。

MySQL 在某些特定的情况下才可能会使用到 Intersection 索引合并，哪些情况呢？

5.1.1.1. 情况一：等值匹配

二级索引列是等值匹配的情况，对于联合索引来说，在联合索引中的每个列都必须等值匹配，不能出现只匹配部分列的情况。

而下边这两个查询就不能进行 Intersection 索引合并：

```
SELECT * FROM order_exp WHERE order_no > 'a' AND insert_time = 'a' AND  
order_status = 'b' AND expire_time = 'c';
```

```
SELECT * FROM order_exp WHERE order_no = 'a' AND insert_time = 'a';
```

第一个查询是因为对 order_no 进行了范围匹配，第二个查询是因为联合索引 u_idx_day_status 中的 order_status 和 expire_time 列并没有出现在搜索条件中，所以这两个查询不能进行 Intersection 索引合并。

5.1.1.2. 情况二：主键列可以是范围匹配

比方说下边这个查询可能用到主键和 u_idx_day_status 进行 Intersection 索引合并的操作：

```
SELECT * FROM order_exp WHERE id > 100 AND insert_time = 'a';
```

对于 InnoDB 的二级索引来说，记录先是按照索引列进行排序，如果该二级索引是一个联合索引，那么会按照联合索引中的各个列依次排序。而二级索引的

用户记录是由索引列 + 主键构成的，二级索引列的值相同的记录可能会有好多条，这些索引列的值相同的记录又是按照主键的值进行排序的。

所以重点来了，之所以在二级索引列都是等值匹配的情况下才可能使用 Intersection 索引合并，是因为只有在这种情况下根据二级索引查询出的结果集是按照主键值排序的。

Intersection 索引合并会把从多个二级索引中查询出的主键值求交集，如果从各个二级索引中查询到的结果集本身就是已经按照主键排好序的，那么求交集的过程就很容易。

假设某个查询使用 Intersection 索引合并的方式从 `idx_order_no` 和 `idx_expire_time` 这两个二级索引中获取到的主键值分别是：

从 `idx_order_no` 中获取到已经排好序的主键值：1、3、5

从 `idx_expire_time` 中获取到已经排好序的主键值：2、3、4

那么求交集的过程就是这样：逐个取出这两个结果集中最小的主键值，如果两个值相等，则加入最后的交集结果中，否则丢弃当前较小的主键值，再取该丢弃的主键值所在结果集的后一个主键值来比较，直到某个结果集中的主键值用完了，时间复杂度是 $O(n)$ 。

但是如果从各个二级索引中查询出的结果集并不是按照主键排序的话，那就要先把结果集中的主键值排序完再来做上边的那个过程，就比较耗时了。

按照有序的主键值去回表取记录有个专有名词，叫：Rowid Ordered Retrieval，简称 **ROR**。

另外，不仅是多个二级索引之间可以采用 Intersection 索引合并，索引合并也可以有聚簇索引参加，也就是我们上边写的情况二：在搜索条件中有主键的范围匹配的情况下也可以使用 Intersection 索引合并。为啥主键这就可以范围匹配了？还是得回到应用场景里：

```
SELECT * FROM order_exp WHERE id > 100 AND order_no = 'a';
```

假设这个查询可以采用 Intersection 索引合并，我们理所当然的以为这个查询会分别按照 `id > 100` 这个条件从聚簇索引中获取一些记录，在通过 `order_no = 'a'` 这个条件从 `idx_order_no` 二级索引中获取一些记录，然后再求交集，其实这样就把问题复杂化了，没必要从聚簇索引中获取一次记录。别忘了二级索引的记录中都带有主键值的，所以可以在从 `idx_order_no` 中获取到的主键值上直接运用条件 `id > 100` 过滤就行了，这样多简单。所以涉及主键的搜索条件只不过是为了从别的二级索引得到的结果集中过滤记录罢了，是不是等值匹配不重要。

当然，上边说的情况一和情况二只是发生 Intersection 索引合并的必要条件，不是充分条件。也就是说即使情况一、情况二成立，也不一定发生 Intersection 索引合并，这得看优化器的心情。优化器只有在单独根据搜索条件从某个二级索引中获取的记录数太多，导致回表开销太大，而通过 Intersection 索引合并后需要回表的记录数大大减少时才会使用 Intersection 索引合并。

5.1.2. Union 合并

我们在写查询语句时经常想把既符合某个搜索条件的记录取出来，也把符合另外的某个搜索条件的记录取出来，我们说这些不同的搜索条件之间是 OR 关系。有时候 OR 关系的不同搜索条件会使用到不同的索引，比方说这样：

```
SELECT * FROM order_exp WHERE order_no = 'a' OR expire_time = 'b'
```

Intersection 是交集的意思，这适用于使用不同索引的搜索条件之间使用 AND 连接起来的情况；Union 是并集的意思，适用于使用不同索引的搜索条件之间使用 OR 连接起来的情况。与 Intersection 索引合并类似，MySQL 在某些特定的情况下才可能会使用到 Union 索引合并：

5.1.2.1. 情况一：等值匹配

分析同 Intersection 合并

5.1.2.2. 情况二：主键列可以是范围匹配

分析同 Intersection 合并

5.1.2.3. 情况三：使用 Intersection 索引合并的搜索条件

就是搜索条件的某些部分使用 Intersection 索引合并的方式得到的主键集合和其他方式得到的主键集合取交集，比方说这个查询：

```
SELECT * FROM order_exp WHERE insert_time = 'a' AND order_status = 'b' AND  
expire_time = 'c' OR (order_no = 'a' AND expire_time = 'b');
```

优化器可能采用这样的方式来执行这个查询：

先按照搜索条件 `order_no = 'a' AND expire_time = 'b'` 从索引 `idx_order_no` 和 `idx_expire_time` 中使用 Intersection 索引合并的方式得到一个主键集合。

再按照搜索条件 `insert_time = 'a' AND order_status = 'b' AND expire_time = 'c'` 从联合索引 `u_idx_day_status` 中得到另一个主键集合。

采用 Union 索引合并的方式把上述两个主键集合取并集，然后进行回表操作，将结果返回给用户。

当然，查询条件符合了这些情况也不一定就会采用 Union 索引合并，也得看优化器的心情。优化器只有在单独根据搜索条件从某个二级索引中获取的记录数比较少，通过 Union 索引合并后进行访问的代价比全表扫描更小时才会使用 Union 索引合并。

5.1.3. Sort-Union 合并

Union 索引合并的使用条件太苛刻，必须保证各个二级索引列在进行等值匹配的条件下才可能被用到，比方说下边这个查询就无法使用到 Union 索引合并：

```
SELECT * FROM order_exp WHERE order_no < 'a' OR expire_time > 'z'
```

这是因为根据 `order_no < 'a'` 从 `idx_order_no` 索引中获取的二级索引记录的主键值不是排好序的，根据 `expire_time > 'z'` 从 `idx_expire_time` 索引中获取的二级索引记录的主键值也不是排好序的，但是 `order_no < 'a'` 和 `expire_time > 'z'` 这两个条件又特别让我们动心，所以我们可以这样：

先根据 `order_no < 'a'` 条件从 `idx_order_no` 二级索引中获取记录，并按照记录的主键值进行排序

再根据 `expire_time > 'z'` 条件从 `idx_expire_time` 二级索引中获取记录，并按照记录的主键值进行排序

因为上述的两个二级索引主键值都是排好序的，剩下的操作和 Union 索引合并方式就一样了。

上述这种先按照二级索引记录的主键值进行排序，之后按照 Union 索引合并方式执行的方式称之为 Sort-Union 索引合并，很显然，这种 Sort-Union 索引合并比单纯的 Union 索引合并多了一步对二级索引记录的主键值排序的过程。

5.1.4. 联合索引替代 Intersection 索引合并

```
SELECT * FROM order_exp WHERE order_no= 'a' And expire_time= 'z';
```

这个查询之所以可能使用 Intersection 索引合并的方式执行，还不是因为 `idx_order_no` 和 `idx_expire_time` 是两个单独的 B+树索引，要是把这两个列搞一个联合索引，那直接使用这个联合索引就把事情搞定了，何必用啥索引合并呢，就像这样：

```
ALTER TABLE order_exp drop index idx_order_no, idx_expire_time,  
add index idx_order_no_expire_time(order_no, expire_time);
```

这样我们把 `idx_order_no`, `idx_expire_time` 都干掉，再添加一个联合索引 `idx_order_no_expire_time`，使用这个联合索引进行查询简直是又快又好，既不用多读一棵 B+树，也不用合并结果。

5.2. 连接查询

搞数据库一个避不开的概念就是 Join，翻译成中文就是连接。使用的时候常常陷入下边两种误区：

误区一：业务至上，管他三七二十一，再复杂的查询也用在在一个连接语句中搞定。

误区二：敬而远之，上次慢查询就是因为使用了连接导致的，以后也不敢用了。

所以我们来学习一下连接的原理，才能在工作中用好 SQL 连接。

5.2.1. 连接简介

5.2.1.1. 连接的本质

为了方便讲述，我们建立两个简单的演示表并给它们写入数据：

```
CREATE TABLE e1 (m1 int, n1 char(1));
CREATE TABLE e2 (m2 int, n2 char(1));
INSERT INTO e1 VALUES(1, 'a'), (2, 'b'), (3, 'c');
INSERT INTO e2 VALUES(2, 'b'), (3, 'c'), (4, 'd');
```

```
mysql> CREATE TABLE e1 (m1 int, n1 char(1));
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE e2 (m2 int, n2 char(1));
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO e1 VALUES(1, 'a'), (2, 'b'), (3, 'c');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

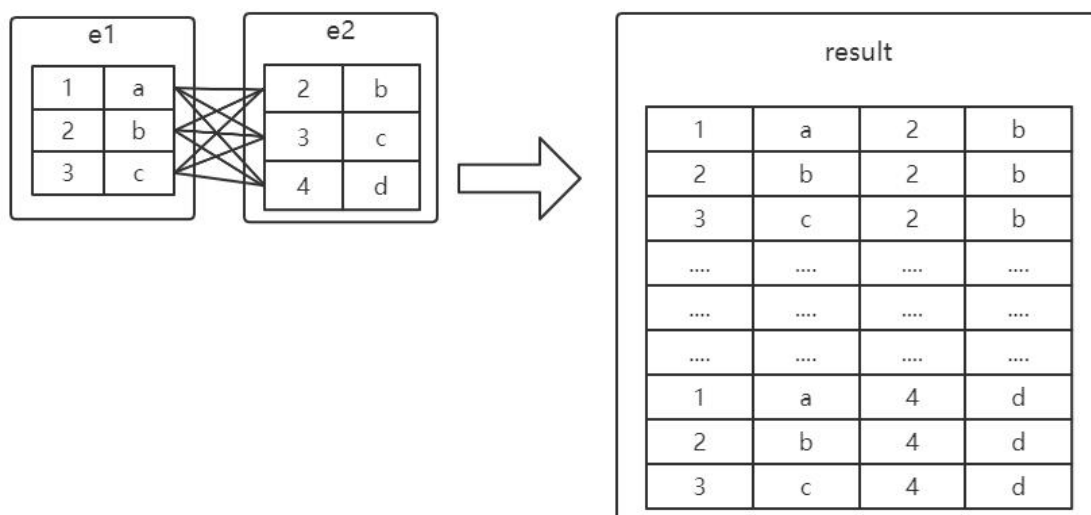
mysql> INSERT INTO e2 VALUES(2, 'b'), (3, 'c'), (4, 'd');
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM e1;
+-----+-----+
| m1 | n1 |
+-----+-----+
| 1 | a |
| 2 | b |
| 3 | c |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM e2;
+-----+-----+
| m2 | n2 |
+-----+-----+
| 2 | b |
| 3 | c |
| 4 | d |
+-----+-----+
3 rows in set (0.00 sec)
```

连接的本质就是把各个连接表中的记录都取出来依次匹配的组合加入结果集并返回给用户。

所以我们将 e1 和 e2 两个表连接起来的过程如下图所示：



这个过程看起来就是把 **e1** 表的记录和 **e2** 的记录连起来组成新的更大的记录，所以这个查询过程称之为连接查询。连接查询的结果集中包含一个表中的每一条记录与另一个表中的每一条记录相互匹配的组合，像这样的结果集就可以称之为笛卡尔积。因为表 **e1** 中有 3 条记录，表 **e2** 中也有 3 条记录，所以这两个表连接之后的笛卡尔积就有 $3 \times 3 = 9$ 行记录。

在 MySQL 中，连接查询的语法很随意，只要在 **FROM** 语句后边跟多个表名就好了，比如我们把 **e1** 表和 **e2** 表连接起来的查询语句可以写成这样：

SELECT * FROM e1, e2;

```
mysql> SELECT * FROM e1, e2;
+-----+-----+-----+-----+
| m1 | n1 | m2 | n2 |
+-----+-----+-----+-----+
| 1 | a | 2 | b |
| 2 | b | 2 | b |
| 3 | c | 2 | b |
| 1 | a | 3 | c |
| 2 | b | 3 | c |
| 3 | c | 3 | c |
| 1 | a | 4 | d |
| 2 | b | 4 | d |
| 3 | c | 4 | d |
+-----+-----+-----+-----+
9 rows in set (0.00 sec)
```

5.2.1.2. 连接过程简介

我们可以连接任意数量张表，但是如果没有任何限制条件的话，这些表连接起来产生的笛卡尔积可能是非常巨大的。比方说 3 个 100 行记录的表连接起来产生的笛卡尔积就有 $100 \times 100 \times 100 = 1000000$ 行数据！所以在连接的时候过滤掉特定记录组合是有必要的，在连接查询中的过滤条件可以分成两种，比方说下边这个查询语句：

SELECT * FROM e1, e2 WHERE e1.m1 > 1 AND e1.m1 = e2.m2 AND e2.n2 < 'd';
涉及单表的条件

比如 $e1.m1 > 1$ 是只针对 $e1$ 表的过滤条件, $e2.n2 < 'd'$ 是只针对 $e2$ 表的过滤条件。

涉及两表的条件

比如类似 $e1.m1 = e2.m2$ 、 $e1.n1 > e2.n2$ 等, 这些条件中涉及到了两个表。

看一下携带过滤条件的连接查询的大致执行过程在这个查询中我们指明了这三个过滤条件:

$e1.m1 > 1$

$e1.m1 = e2.m2$

$e2.n2 < 'd'$

那么这个连接查询的大致执行过程如下:

步骤一: 首先确定第一个需要查询的表, 这个表称之为**驱动表**。单表中执行查询语句只需要选取代价最小的那种访问方法去执行单表查询语句就好了(就是说从 `const`、`ref`、`ref_or_null`、`range`、`index`、`all` 等等这些执行方法中选取代价最小的去执行查询)。

此处假设使用 $e1$ 作为驱动表, 那么就需要到 $e1$ 表中找满足 $e1.m1 > 1$ 的记录, 因为表中的数据太少, 我们也没在表上建立二级索引, 所以此处查询 $e1$ 表的访问方法就设定为 `all`, 也就是采用全表扫描的方式执行单表查询。

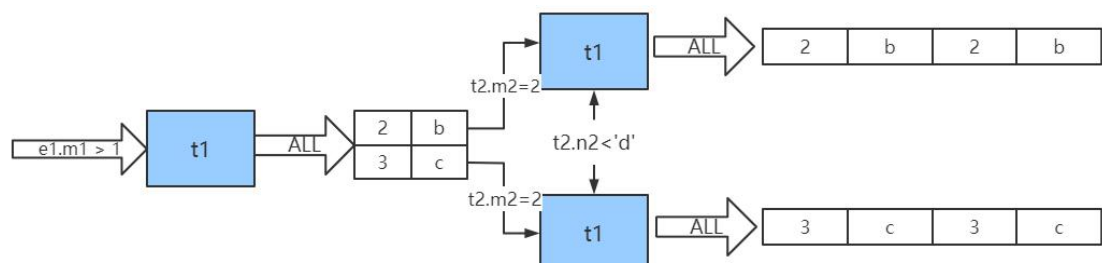
很明显, $e1$ 表中符合 $e1.m1 > 1$ 的记录有两条。

步骤二: 针对上一步骤中从驱动表产生的结果集中的每一条记录, 分别需要到 $e2$ 表中查找匹配的记录, 所谓匹配的记录, 指的是符合过滤条件的记录。因为是根据 $e1$ 表中的记录去找 $e2$ 表中的记录, 所以 $e2$ 表也可以被称之为**被驱动表**。上一步骤从驱动表中得到了 2 条记录, 所以需要查询 2 次 $e2$ 表。此时涉及两个表的列的过滤条件 $e1.m1 = e2.m2$ 就派上用场了:

当 $e1.m1 = 2$ 时, 过滤条件 $e1.m1 = e2.m2$ 就相当于 $e2.m2 = 2$, 所以此时 $e2$ 表相当于有了 $e2.m2 = 2$ 、 $e2.n2 < 'd'$ 这两个过滤条件, 然后到 $e2$ 表中执行单表查询。

当 $e1.m1 = 3$ 时, 过滤条件 $e1.m1 = e2.m2$ 就相当于 $e2.m2 = 3$, 所以此时 $e2$ 表相当于有了 $e2.m2 = 3$ 、 $e2.n2 < 'd'$ 这两个过滤条件, 然后到 $e2$ 表中执行单表查询。

所以整个连接查询的执行过程就如下图所示:



也就是说整个连接查询最后的结果只有两条符合过滤条件的记录:

```
mysql> SELECT * FROM e1, e2 WHERE e1.m1 > 1 AND e1.m1 = e2.m2 AND e2.n2 < 'd';
```

m1	n1	m2	n2
2	b	2	b
3	c	3	c

```
2 rows in set (0.00 sec)
```

从上边两个步骤可以看出来，这个两表连接查询共需要查询 1 次 e1 表，2 次 e2 表。当然这是在特定的过滤条件下的结果，如果我们把 e1.m1 > 1 这个条件去掉，那么从 e1 表中查出的记录就有 3 条，就需要查询 3 次 e2 表了。也就是说在两表连接查询中，驱动表只需要访问一次，被驱动表可能被访问多次。

5.2.1.3. 内连接和外连接

为了大家更好理解后边内容，我们创建两个有现实意义的表，并插入一些数据：

```
CREATE TABLE student (
    number INT NOT NULL AUTO_INCREMENT COMMENT '学号',
    name VARCHAR(5) COMMENT '姓名',
    major VARCHAR(30) COMMENT '专业',
    PRIMARY KEY (number)
) Engine=InnoDB CHARSET=utf8 COMMENT '客户信息表';

CREATE TABLE score (
    number INT COMMENT '学号',
    subject VARCHAR(30) COMMENT '科目',
    score TINYINT COMMENT '成绩',
    PRIMARY KEY (number, subject)
) Engine=InnoDB CHARSET=utf8 COMMENT '客户成绩表';

SELECT * FROM student; SELECT * FROM score;
```

```
mysql> SELECT * FROM student;
```

number	name	major
20200901	Jack	网络工程
20200902	Mark	计算机科学
20200903	James	计算机科学
20200904	King	软件工程

```
4 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM score;
```

number	subject	score
20200901	数据结构和算法	88
20200901	网络通信原理	90
20200902	数据结构和算法	95
20200902	网络通信原理	89
20200903	离散数学	70

```
5 rows in set (0.00 sec)
```

现在我们把每个学生的考试成绩都查询出来就需要进行两表连接了(因为 score 中没有姓名信息, 所以不能单纯只查询 score 表)。连接过程就是从 student 表中取出记录, 在 score 表中查找 number 相同的成绩记录, 所以过滤条件就是 student.number = socre.number, 整个查询语句就是这样:

SELECT s1.number, s1.name, s2.subject, s2.score FROM student AS s1, score AS s2 WHERE s1.number = s2.number;

```
mysql> SELECT s1.number, s1.name, s2.subject, s2.score FROM student AS s1, score AS s2 WHERE s1.number = s2.number;
+-----+-----+-----+-----+
| number | name  | subject          | score |
+-----+-----+-----+-----+
| 20200901 | Jack | 数据结构和算法   | 88    |
| 20200901 | Jack | 网络通信原理     | 90    |
| 20200902 | Mark | 数据结构和算法   | 95    |
| 20200902 | Mark | 网络通信原理     | 89    |
| 20200903 | James | 离散数学         | 70    |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

从上述查询结果中我们可以看到, 各个同学对应的各科成绩就都被查出来了, 可是有个问题, King 同学, 也就是学号为 20200904 的同学因为某些原因没有参加考试, 所以在 score 表中没有对应的成绩记录。

如果老师想查看所有同学的考试成绩, 即使是缺考的同学也应该展示出来, 但是到目前为止我们介绍的连接查询是无法完成这样的需求的。我们稍微思考一下这个需求, 其本质是想: **驱动表中的记录即使在被驱动表中没有匹配的记录, 也仍然需要加入到结果集**。为了解决这个问题, 就有了**内连接**和**外连接**的概念:

对于内连接的两个表, 驱动表中的记录在被驱动表中找不到匹配的记录, 该记录不会加入到最后的结果集, 我们上边提到的连接都是所谓的内连接。

对于外连接的两个表, 驱动表中的记录即使在被驱动表中没有匹配的记录, 也仍然需要加入到结果集。

在 MySQL 中, 根据选取驱动表的不同, 外连接仍然可以细分为 2 种:

左外连接, 选取左侧的表为驱动表。

右外连接, 选取右侧的表为驱动表。

可是这样仍然存在问题, 即使对于外连接来说, 有时候我们也并不想把驱动表的全部记录都加入到最后的结果集。

这就犯难了, 怎么办? 把过滤条件分为两种就可以就解决这个问题了, 所以放在不同地方的过滤条件是有不同语义的:

WHERE 子句中的过滤条件

WHERE 子句中的过滤条件就是我们平时见的那种, 不论是内连接还是外连接, 凡是不符合 WHERE 子句中的过滤条件的记录都不会被加入最后的结果集。

ON 子句中的过滤条件

对于外连接的驱动表的记录来说, 如果无法在被驱动表中找到匹配 ON 子句中的过滤条件的记录, 那么该记录仍然会被加入到结果集中, 对应的被驱动表记录的各个字段使用 NULL 值填充。

需要注意的是, 这个 ON 子句是专门为外连接驱动表中的记录在被驱动表找不到匹配记录时应不应该把该记录加入结果集这个场景下提出的, 所以如果把

ON 子句放到内连接中，MySQL 会把它和 WHERE 子句一样对待，也就是说：内连接中的 WHERE 子句和 ON 子句是等价的。

一般情况下，我们都把只涉及单表的过滤条件放到 WHERE 子句中，把涉及两表的过滤条件都放到 ON 子句中，我们也一般把放到 ON 子句中的过滤条件也称之为连接条件。

左（外）连接的语法

左（外）连接的语法还是挺简单的，比如我们要把 e1 表和 e2 表进行左外连接查询可以这么写：

SELECT * FROM e1 LEFT [OUTER] JOIN e2 ON 连接条件 [WHERE 普通过滤条件];

其中中括号里的 OUTER 单词是可以省略的。对于 LEFT JOIN 类型的连接来说，我们把放在左边的表称之为外表或者驱动表，右边的表称之为内表或者被驱动表。所以上述例子中 e1 就是外表或者驱动表，e2 就是内表或者被驱动表。需要注意的是，对于左（外）连接和右（外）连接来说，必须使用 ON 子句来指出连接条件。了解了左（外）连接的基本语法之后，再次回到我们上边那个现实问题中来，看看怎样写查询语句才能把所有的客户的成绩信息都查询出来，即使是缺考的考生也应该被放到结果集中：

SELECT s1.number, s1.name, s2.subject, s2.score FROM student AS s1 LEFT JOIN score AS s2 ON s1.number = s2.number;

```
mysql> SELECT s1.number, s1.name, s2.subject, s2.score FROM student AS s1 LEFT
+-----+-----+-----+-----+
| number | name  | subject          | score |
+-----+-----+-----+-----+
| 20200901 | Jack | 数据结构和算法   | 88    |
| 20200901 | Jack | 网络通信原理     | 90    |
| 20200902 | Mark | 数据结构和算法   | 95    |
| 20200902 | Mark | 网络通信原理     | 89    |
| 20200903 | James | 离散数学         | 70    |
| 20200904 | King  | NULL             | NULL  |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

从结果集中可以看出来，虽然 King 并没有对应的成绩记录，但是由于采用的是连接类型为左（外）连接，所以仍然把她放到了结果集中，只不过在对应的成绩记录的专栏使用 NULL 值填充而已。

右（外）连接的语法

右（外）连接和左（外）连接的原理是一样的，语法也只是把 LEFT 换成 RIGHT 而已：

SELECT * FROM e1 RIGHT [OUTER] JOIN e2 ON 连接条件 [WHERE 普通过滤条件];

只不过驱动表是右边的表 e2，被驱动表是左边的表 e1。

内连接的语法

内连接和外连接的根本区别就是在驱动表中的记录不符合 ON 子句中的连接条件时不会把该记录加入到最后的结果集，一种最简单的内连接语法，就是直接

把需要连接的多个表都放到 **FROM** 子句后边。其实针对内连接，MySQL 提供了好多不同的语法：

```
SELECT * FROM e1 [INNER | CROSS] JOIN e2 [ON 连接条件] [WHERE 普通过滤条件];
```

也就是说在 MySQL 中，下边这几种内连接的写法都是等价的：

```
SELECT * FROM e1 JOIN e2;
```

```
SELECT * FROM e1 INNER JOIN e2;
```

```
SELECT * FROM e1 CROSS JOIN e2;
```

上边的这些写法和直接把需要连接的表名放到 **FROM** 语句之后，用逗号,分隔开的写法是等价的：

```
SELECT * FROM e1, e2;
```

再说一次，由于在内连接中 **ON** 子句和 **WHERE** 子句是等价的，所以内连接中不要求强制写明 **ON** 子句。

我们前边说过，连接的本质就是把各个连接表中的记录都取出来依次匹配的组合加入结果集并返回给用户。不论哪个表作为驱动表，两表连接产生的笛卡尔积肯定是一样的。而对于内连接来说，由于凡是不符合 **ON** 子句或 **WHERE** 子句中的条件的记录都会被过滤掉，其实也就相当于从两表连接的笛卡尔积中把不符合过滤条件的记录给踢出去，所以对于内连接来说，驱动表和被驱动表是可以互换的，并不会影响最后的查询结果。

但是对于外连接来说，由于驱动表中的记录即使在被驱动表中找不到符合 **ON** 子句条件的记录时也要将其加入到结果集，所以此时驱动表和被驱动表的关系就很重要了，也就是说左外连接和右外连接的驱动表和被驱动表不能轻易互换。

5.2.2. MySQL 对连接的执行

复习了连接、内连接、外连接这些基本概念后，我们需要理解 MySQL 怎么样来进行表与表之间的连接，才能明白有的连接查询运行的快，有的却慢。

5.2.2.1. 嵌套循环连接 (Nested-Loop Join)

我们前边说过，对于两表连接来说，驱动表只会被访问一遍，但被驱动表却要访问到好多遍，具体访问几遍取决于对驱动表执行单表查询后的结果集中的记录条数。

对于内连接来说，选取哪个表为驱动表都没关系，而外连接的驱动表是固定的，也就是说左（外）连接的驱动表就是左边的那个表，右（外）连接的驱动表就是右边的那个表。

如果有 3 个表进行连接的话，那么首先两表连接得到的结果集就像是新的驱动表，然后第三个表就成为了被驱动表，可以用伪代码表示一下这个过程就是这样：

```
for each row in e1 {    #此处表示遍历满足对 e1 单表查询结果集中的每一条记录，N 条
```

```
for each row in e2 {    #此处表示对于某条 e1 表的记录来说，遍历满足
对 e2 单表查询结果集中的每一条记录，M 条
    for each row in t3 {    #此处表示对于某条 e1 和 e2 表的记录组
合来说，对 t3 表进行单表查询，L 条
        if row satisfies join conditions, send to client
    }
}
```

这个过程就像是一个嵌套的循环，所以这种驱动表只访问一次，但被驱动表却可能被多次访问，访问次数取决于对驱动表执行单表查询后的结果集中的记录条数的连接执行方式称之为**嵌套循环连接（Nested-Loop Join）**，这是最简单，也是最笨拙的一种连接查询算法，时间复杂度是 $O(N*M*L)$ 。

5.2.2.2. 使用索引加快连接速度

我们知道在嵌套循环连接的步骤 2 中可能需要访问多次被驱动表，如果访问被驱动表的方式都是全表扫描的话，那酸爽不敢想象！

但是查询 e2 表其实就相当于一次单表查询，我们可以利用索引来加快查询速度。回顾一下最开始介绍的 e1 表和 e2 表进行内连接的例子：

```
SELECT * FROM e1, e2 WHERE e1.m1 > 1 AND e1.m1 = e2.m2 AND e2.n2 < 'd';
```

我们使用的其实是嵌套循环连接算法执行的连接查询，再把上边那个查询执行过程表回顾一下：

查询驱动表 e1 后的结果集中有两条记录，嵌套循环连接算法需要对被驱动表查询 2 次：

当 e1.m1 = 2 时，去查询一遍 e2 表，对 e2 表的查询语句相当于：

```
SELECT * FROM e2 WHERE e2.m2 = 2 AND e2.n2 < 'd';
```

当 e1.m1 = 3 时，再去查询一遍 e2 表，此时对 e2 表的查询语句相当于：

```
SELECT * FROM e2 WHERE e2.m2 = 3 AND e2.n2 < 'd';
```

可以看到，原来的 e1.m1 = e2.m2 这个涉及两个表的过滤条件在针对 e2 表做查询时关于 e1 表的条件就已经确定了，所以我们只需要单单优化对 e2 表的查询了，上述两个对 e2 表的查询语句中利用到的列是 m2 和 n2 列，我们可以：

在 m2 列上建立索引，因为对 m2 列的条件是等值查找，比如 e2.m2 = 2、e2.m2 = 3 等，所以可能使用到 **ref** 的访问方法，假设使用 ref 的访问方法去执行对 e2 表的查询的话，需要回表之后再判断 e2.n2 < d 这个条件是否成立。

这里有一个比较特殊的情况，就是假设 m2 列是 e2 表的主键或者唯一二级索引列，那么使用 e2.m2 = 常数值这样的条件从 e2 表中查找记录的过程的代价就是常数级别的。我们知道在单表中使用主键值或者唯一二级索引列的值进行等值查找的方式称之为 **const**，而 MySQL 把在连接查询中对被驱动表使用主键值或者唯一二级索引列的值进行等值查找的查询执行方式称之为：**eq_ref**。

在 `n2` 列上建立索引，涉及到的条件是 `e2.n2 < 'd'`，可能用到 `range` 的访问方法，假设使用 `range` 的访问方法对 `e2` 表的查询的话，需要回表之后再判断在 `m2` 列上的条件是否成立。

假设 `m2` 和 `n2` 列上都存在索引的话，那么就需要从这两个里边儿挑一个代价更低的去执行对 `e2` 表的查询。当然，建立了索引不一定使用索引，只有在二级索引 + 回表的代价比全表扫描的代价更低时才会使用索引。

另外，有时候连接查询的查询列表和过滤条件中可能只涉及被驱动表的部分列，而这些列都是某个索引的一部分，这种情况下即使不能使用 `eq_ref`、`ref`、`ref_or_null` 或者 `range` 这些访问方法执行对被驱动表的查询的话，也可以使用索引扫描，也就是 `index`(索引覆盖)的访问方法来查询被驱动表。

5.2.2.3. 基于块的嵌套循环连接 (Block Nested-Loop Join)

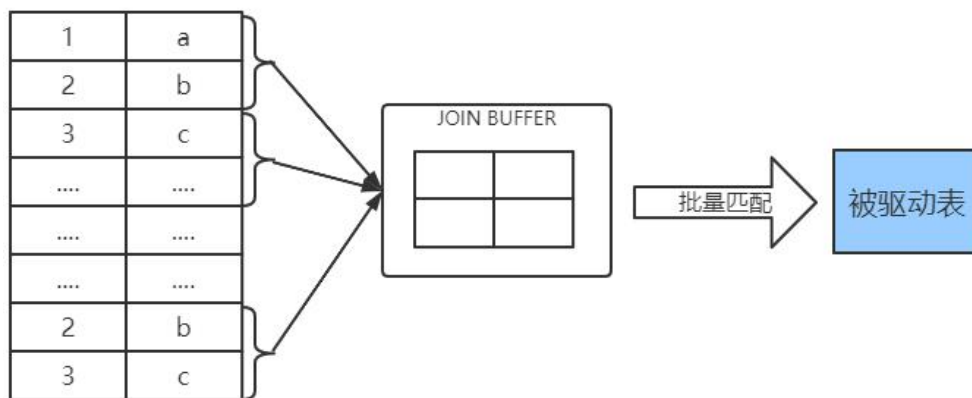
扫描一个表的过程其实是先把这个表从磁盘上加载到内存中，然后从内存中比较匹配条件是否满足。

现实生活中的表成千上万条记录都是少的，几百万、几千万甚至几亿条记录的表到处都是。内存里可能并不能完全存放的下表中所有的记录，所以在扫描表前边记录的时候后边的记录可能还在磁盘上，等扫描到后边记录的时候可能内存不足，所以需要把前边的记录从内存中释放掉。

而采用嵌套循环连接算法的两表连接过程中，被驱动表可是要被访问好多次的，如果这个被驱动表中的数据特别多而且不能使用索引进行访问，那就相当于是从磁盘上读好几次这个表，这个 I/O 代价就非常大了，所以我们得想办法：尽量减少访问被驱动表的次数。

当被驱动表中的数据非常多时，每次访问被驱动表，被驱动表的记录会被加载到内存中，在内存中的每一条记录只会和驱动表结果集的一条记录做匹配，之后就会被从内存中清除掉。然后再从驱动表结果集中拿出另一条记录，再一次把被驱动表的记录加载到内存中一遍，周而复始，驱动表结果集中有多少条记录，就得把被驱动表从磁盘上加载到内存中多少次。

所以我们可不可以在把被驱动表的记录加载到内存的时候，一次性和多条驱动表中的记录做匹配，这样就可以大大减少重复从磁盘上加载被驱动表的代价了。所以 MySQL 提出了一个 `join buffer` 的概念，`join buffer` 就是执行连接查询前申请的一块固定大小的内存，先把若干条驱动表结果集中的记录装在这个 `join buffer` 中，然后开始扫描被驱动表，每一条被驱动表的记录一次性和 `join buffer` 中的多条驱动表记录做匹配，因为匹配的过程都是在内存中完成的，所以这样可以显著减少被驱动表的 I/O 代价。使用 `join buffer` 的过程如下图所示：



最最好的情况是 join buffer 足够大，能容纳驱动表结果集中的所有记录。

这种加入了 join buffer 的嵌套循环连接算法称之为**基于块的嵌套连接(Block Nested-Loop Join)**算法。

这个 join buffer 的大小是可以通过启动参数或者系统变量 join_buffer_size 进行配置，默认大小为 262144 字节（也就是 256KB），最小可以设置为 128 字节。

show variables like 'join_buffer_size' ;

```
mysql> show variables like 'join_buffer_size';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| join_buffer_size | 262144 |
+-----+-----+
1 row in set (0.01 sec)
```

当然，对于优化被驱动表的查询来说，最好是为被驱动表加上效率高的索引，如果实在不能使用索引，并且自己的机器的内存也比较大可以尝试调大 join_buffer_size 的值来对连接查询进行优化。

另外需要注意的是，驱动表的记录并不是所有列都会被放到 join buffer 中，只有查询列表中的列和过滤条件中的列才会被放到 join buffer 中，所以再次提醒我们，最好不要把*作为查询列表，只需要把我们关心的列放到查询列表就好了，这样还可以在 join buffer 中放置更多的记录。

5.3. MySQL 的查询成本

5.3.1. 什么是成本

MySQL 执行一个查询可以有不同的执行方案，它会选择其中成本最低，或者说代价最低的那种方案去真正的执行查询。不过我们之前对成本的描述是非常模糊的，其实在 MySQL 中一条查询语句的执行成本是由下边这两个方面组成的：

I/O 成本

我们的表经常使用的 MyISAM、InnoDB 存储引擎都是将数据和索引都存储到磁盘上的，当我们想查询表中的记录时，需要先把数据或者索引加载到内存中然后再操作。这个从磁盘到内存这个加载的过程损耗的时间称之为 I/O 成本。

CPU 成本

读取以及检测记录是否满足对应的搜索条件、对结果集进行排序等这些操作损耗的时间称之为 CPU 成本。

对于 InnoDB 存储引擎来说，页是磁盘和内存之间交互的基本单位，MySQL 规定读取一个页面花费的成本默认是 1.0，读取以及检测一条记录是否符合搜索条件的成本默认是 0.2。1.0、0.2 这些数字称之为成本常数，这两个成本常数我们最常用到，当然还有其他的成本常数。

注意，不管读取记录时需不需要检测是否满足搜索条件，其成本都算是 0.2。

5.3.2. 单表查询的成本

5.3.2.1. 基于成本的优化步骤实战

在一条单表查询语句真正执行之前，MySQL 的查询优化器会找出执行该语句所有可能使用的方案，对比之后找出成本最低的方案，这个成本最低的方案就是所谓的执行计划，之后才会调用存储引擎提供的接口真正的执行查询，这个过程总结一下就是这样：

- 1、根据搜索条件，找出所有可能使用的索引
- 2、计算全表扫描的代价
- 3、计算使用不同索引执行查询的代价
- 4、对比各种执行方案的代价，找出成本最低的那一个

下边我们就以一个实例来分析一下这些步骤，单表查询语句如下：

```
SELECT * FROM order_exp WHERE order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S') AND expire_time > '2021-03-22 18:28:28' AND expire_time <= '2021-03-22 18:35:09' AND insert_time > expire_time AND order_note LIKE '%7 排 1%' AND order_status = 0;
```

乍看上去有点儿复杂，我们一步一步分析一下。

1. 根据搜索条件，找出所有可能使用的索引

我们前边说过，对于 B+ 树索引来说，只要索引列和常数使用 =、<=>、IN、NOT IN、IS NULL、IS NOT NULL、>、<、>=、<=、BETWEEN、!=（不等于也可以写成 <>）或者 LIKE 操作符连接起来，就可以产生一个所谓的范围区间（LIKE 匹配字符串前缀也行），MySQL 把一个查询中可能使用到的索引称之为 possible keys。

我们分析一下上边查询中涉及到的几个搜索条件：

order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S')，这个搜索条件可以使用二级索引 idx_order_no。

expire_time > '2021-03-22 18:28:28' AND expire_time <= '2021-03-22 18:35:09'，这个搜索条件可以使用二级索引 idx_expire_time。

insert_time > expire_time，这个搜索条件的索引列由于没有和常数比较，所以并不能使用到索引。

order_note LIKE '%hello%'，order_note 即使有索引，但是通过 LIKE 操作符和以通配符开头的字符串做比较，不可以适用索引。

`order_status = 0`，由于该列上只有联合索引，而且不符合最左前缀原则，所以不会用到索引。

综上所述，上边的查询语句可能用到的索引，也就是 `possible keys` 只有 `idx_order_no,idx_expire_time`。

```
mysql> explain SELECT * FROM order_exp WHERE order_no=
18:35:09' AND insert_time> expire_time AND order_no=
***** 1. row *****
      id: 1
    select_type: SIMPLE
        table: order_exp
    partitions: NULL
         type: range
possible_keys: idx_order_no,idx_expire_time
          key: idx_expire_time
        key_len: 5
           ref: NULL
          rows: 39
     filtered: 0.13
      Extra: Using index condition; Using where
1 row in set, 1 warning (0.00 sec)
```

2. 计算全表扫描的代价

对于 InnoDB 存储引擎来说，全表扫描的意思就是把聚簇索引中的记录都依次和给定的搜索条件做一下比较，把符合搜索条件的记录加入到结果集，所以需要把聚簇索引对应的页面加载到内存中，然后再检测记录是否符合搜索条件。由于查询成本=I/O 成本+CPU 成本，所以计算全表扫描的代价需要两个信息：

聚簇索引占用的页面数

该表中的记录数

这两个信息从哪来呢？MySQL 为每个表维护了一系列的统计信息，关于这些统计信息是如何收集起来的我们放在后边再说，现在看看怎么查看这些统计信息。

MySQL 给我们提供了 `SHOW TABLE STATUS` 语句来查看表的统计信息，如果要看指定的某个表的统计信息，在该语句后加对应的 `LIKE` 语句就好了，比方说我们要查看 `order_exp` 这个表的统计信息可以这么写：

`SHOW TABLE STATUS LIKE 'order_exp'\G`

```
mysql> SHOW TABLE STATUS LIKE 'order_exp'\G
***** 1. row *****
      Name: order_exp
      Engine: InnoDB
      Version: 10
      Row_format: Dynamic
      Rows: 10350
      Avg_row_length: 153
      Data_length: 1589248
      Max_data_length: 0
      Index_length: 1130496
      Data_free: 4194304
      Auto_increment: 10819
      Create_time: 2021-03-31 15:39:43
      Update_time: 2021-04-05 18:43:17
      Check_time: NULL
      Collation: utf8_general_ci
      Checksum: NULL
      Create_options:
      Comment:
1 row in set (0.00 sec)
```

出现了很多统计选项，但我们目前只需要两个：

Rows

本选项表示表中的记录条数。对于使用 MyISAM 存储引擎的表来说，该值是准确的，对于使用 InnoDB 存储引擎的表来说，该值是一个估计值。从查询结果我们也可以看出来，由于我们的 order_exp 表是使用 InnoDB 存储引擎的，所以虽然实际上表中有 10567 条记录，但是 SHOW TABLE STATUS 显示的 Rows 值只有 10350 条记录。

Data_length

本选项表示表占用的存储空间字节数。使用 MyISAM 存储引擎的表来说，该值就是数据文件的大小，对于使用 InnoDB 存储引擎的表来说，该值就相当于聚簇索引占用的存储空间大小，也就是说可以这样计算该值的大小：

$\text{Data_length} = \text{聚簇索引的页面数量} \times \text{每个页面的大小}$

我们的 order_exp 使用默认 16KB 的页面大小，而上边查询结果显示 Data_length 的值是 1589248，所以我们可以反向来推导出聚簇索引的页面数量：

$\text{聚簇索引的页面数量} = 1589248 \div 16 \div 1024 = 97$

我们现在已经得到了聚簇索引占用的页面数量以及该表记录数的估计值，所以就可以计算全表扫描成本了。

现在可以看一下全表扫描成本的计算过程：

I/O 成本

$97 \times 1.0 + 1.1 = 98.1$

97 指的是聚簇索引占用的页面数，1.0 指的是加载一个页面的成本常数，后边的 1.1 是一个微调值。

TIPS: MySQL 在真实计算成本时会进行一些微调，这些微调的值是直接硬编码到代码里的，没有注释而且这些微调的值十分的小，并不影响我们分析。

CPU 成本

$10350 \times 0.2 + 1.0 = 2071$

10350 指的是统计数据中表的记录数，对于 InnoDB 存储引擎来说是一个估计值，0.2 指的是访问一条记录所需的成本常数，后边的 1.0 是一个微调值。

总成本：

$98.1 + 2071 = 2169.1$

综上所述，对于 order_exp 的全表扫描所需的总成本就是 2169.1。

TIPS: 我们前边说过表中的记录其实都存储在聚簇索引对应 B+ 树的叶子节点中，所以只要我们通过根节点获得了最左边的叶子节点，就可以沿着叶子节点组成的双向链表把所有记录都查看一遍。

也就是说全表扫描这个过程其实有的 B+ 树非叶子节点是不需要访问的，但是 MySQL 在计算全表扫描成本时直接使用聚簇索引占用的页面数作为计算 I/O 成本的依据，是不区分非叶子节点和叶子节点的。

3. 计算使用不同索引执行查询的代价

从第 1 步分析我们得到，上述查询可能使用到 idx_order_no, idx_expire_time 这两个索引，我们需要分别分析单独使用这些索引执行查询的成本，最后还要分析是否可能使用到索引合并。这里需要提一点的是，MySQL 查询优化器先分析使用唯一二级索引的成本，再分析使用普通索引的成本，我们这里两个索引都是普通索引，先算哪个都可以。我们也先分析 idx_expire_time 的成本，然后再看使用 idx_order_no 的成本。

使用 idx_expire_time 执行查询的成本分析

idx_expire_time 对应的搜索条件是：expire_time > '2021-03-22 18:28:28' AND expire_time <= '2021-03-22 18:35:09'，也就是说对应的范围区间就是：('2021-03-22 18:28:28', '2021-03-22 18:35:09')。

*思考题：*扫描区间怎么样从我们复杂的 SQL 语句里提取出来？前面已经讲过了，不记得的同学回看一下章节《3.2.3.深入思考索引在查询中的使用》。

使用 idx_expire_time 搜索会使用二级索引 + 回表方式的查询，MySQL 计算这种查询的成本依赖两个方面的数据：

1、范围区间数量

不论某个范围区间的二级索引到底占用了多少页面，查询优化器认为读取索引的一个范围区间的 I/O 成本和读取一个页面是相同的。本例中使用 idx_expire_time 的范围区间只有一个，所以相当于访问这个范围区间的二级索引付出的 I/O 成本就是： $1 \times 1.0 = 1.0$

2、需要回表的记录数

优化器需要计算二级索引的某个范围区间到底包含多少条记录，对于本例来说就是要计算 idx_expire_time 在('2021-03-22 18:28:28', '2021-03-22 18:35:09') 这个范围区间中包含多少二级索引记录，计算过程是这样的：

步骤 1：先根据 expire_time > '2021-03-22 18:28:28' 这个条件访问一下 idx_expire_time 对应的 B+ 树索引，找到满足 expire_time > '2021-03-22 18:28:28' 这个条件的第一条记录，我们把这条记录称之为区间最左记录。我们前头说过在 B+ 数树中定位一条记录的过程是很快的，是常数级别的，所以这个过程的性能消耗是可以忽略不计的。

步骤 2: 然后再根据 `expire_time <= '2021-03-22 18:35:09'` 这个条件继续从 `idx_expire_time` 对应的 B+树索引中找出最后一条满足这个条件的记录, 我们把这条记录称之为区间最右记录, 这个过程的性能消耗也可以忽略不计的。

步骤 3: 如果区间最左记录和区间最右记录相隔不太远 (在 MySQL 5.7 这个版本里, 只要相隔不大于 10 个页面即可), 那就可以精确统计出满足 `expire_time > '2021-03-22 18:28:28' AND expire_time <= '2021-03-22 18:35:09'` 条件的二级索引记录条数。否则只沿着区间最左记录向右读 10 个页面, 计算平均每个页面中包含多少记录, 然后用这个平均值乘以区间最左记录和区间最右记录之间的页面数量就可以了。那么问题又来了, 怎么估计区间最左记录和区间最右记录之间有多少个页面呢? 解决这个问题还得回到 B+树索引的结构中来。

我们假设区间最左记录在页 b 中, 区间最右记录在页 c 中, 那么我们想计算区间最左记录和区间最右记录之间的页面数量就相当于计算页 b 和页 c 之间有多少页面, 而它们父节点中记录的每一条目录项记录都对应一个数据页, 所以计算页 b 和页 c 之间有多少页面就相当于计算它们父节点 (也就是页 a) 中对应的目录项记录之间隔着几条记录。在一个页面中统计两条记录之间有几条记录的成本就很小了。

不过还有问题, 如果页 b 和页 c 之间的页面实在太多, 以至于页 b 和页 c 对应的目录项记录都不在一个父页面中怎么办? 既然是树, 那就继续递归, 之前我们说过一个 B+树有 4 层高已经很了不得了, 所以这个统计过程也不是很耗费性能。

知道了如何统计二级索引某个范围区间的记录数之后, 就需要回到现实问题中来, MySQL 根据上述算法测得 `idx_expire_time` 在区间 ('2021-03-22 18:28:28', '2021-03-22 18:35:09') 之间大约有 39 条记录。

explain SELECT * FROM order_exp WHERE expire_time > '2021-03-22 18:28:28' AND expire_time <= '2021-03-22 18:35:09';

```
mysql> explain SELECT * FROM order_exp WHERE expire_time > '2021-03-22 18:28:28' AND expire_time <= '2021-03-22 18:35:09';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | order_exp | NULL | range | idx_expire_time | idx_expire_time | 5 | NULL | 39 |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

读取这 39 条二级索引记录需要付出的 CPU 成本就是:

$$39 \times 0.2 + 0.01 = 7.81$$

其中 39 是需要读取的二级索引记录条数, 0.2 是读取一条记录成本常数, 0.01 是微调。

在通过二级索引获取到记录之后, 还需要干两件事儿:

1、根据这些记录里的主键值到聚簇索引中做回表操作

MySQL 评估回表操作的 I/O 成本依旧很简单粗暴, 他们认为每次回表操作都相当于访问一个页面, 也就是说二级索引范围区间有多少记录, 就需要进行多少次回表操作, 也就是需要进行多少次页面 I/O。我们上边统计了使用 `idx_expire_time` 二级索引执行查询时, 预计有 39 条二级索引记录需要进行回表操作, 所以回表操作带来的 I/O 成本就是:

$$39 \times 1.0 = 39.0$$

其中 39 是预计的二级索引记录数, 1.0 是一个页面的 I/O 成本常数。

2、回表操作后得到的完整用户记录，然后再检测其他搜索条件是否成立

回表操作的本质就是通过二级索引记录的主键值到聚簇索引中找到完整的用户记录，然后再检测除 `expire_time>'2021-03-22 18:28:28' AND expire_time<'2021-03-22 18:35:09'` 这个搜索条件以外的搜索条件是否成立。

因为我们通过范围区间获取到二级索引记录共 39 条，也就对应着聚簇索引中 39 条完整的用户记录，读取并检测这些完整的用户记录是否符合其余的搜索条件的 CPU 成本如下：

$$39 \times 0.2 = 7.8$$

其中 39 是待检测记录的条数, 0.2 是检测一条记录是否符合给定的搜索条件的成本常数。

所以本例中使用 `idx_expire_time` 执行查询的成本就如下所示:

I/O 成本:

$$1.0 + 39 \times 1.0 = 40.0 \text{ (范围区间的数量 + 预估的二级索引记录条数)}$$

CPU 成本:

$$39 \times 0.2 + 0.01 + 39 \times 0.2 = 15.61 \quad (\text{读取二级索引记录的成本} + \text{读取并检测回表后聚簇索引记录的成本})$$

综上所述，使用 idx expire time 执行查询的总成本就是：

$$40.0 + 15.61 = 55.61$$

使用 idx order no 执行查询的成本分析

idx_order_no 对应的搜索条件是: order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S'), 也就是说相当于 3 个单点区间。

与使用 `idx_expire_time` 的情况类似,我们也需要计算使用 `idx_order_no` 时需要访问的范围区间数量以及需要回表的记录数,计算过程与上面类似,我们不详列所有计算步骤和说明了。

范围区间数量

使用 `idx_order_no` 执行查询时很显然有 3 个单点区间, 所以访问这 3 个范围区间的二级索引付出的 I/O 成本就是:

$$3 \times 1.0 = 3.0$$

需要回表的记录数

由于使用 `idx_expire_time` 时有 3 个单点区间, 所以每个单点区间都需要查找一遍对应的二级索引记录数, 三个单点区间总共需要回表的记录数是 58。

```
explain SELECT * FROM order_exp WHERE order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S');
```

[illegible]

读取这些二级索引记录的 CPU 成本就是： $58 \times 0.2 + 0.01 = 11.61$

得到总共需要回表的记录数之后，就要考虑：

根据这些记录里的主键值到聚簇索引中做回表操作，所需的 I/O 成本就是：
 $58 \times 1.0 = 58.0$

回表操作后得到的完整用户记录，然后再比较其他搜索条件是否成立

此步骤对应的 CPU 成本就是：

$58 \times 0.2 = 11.6$

所以本例中使用 `idx_order_no` 执行查询的成本就如下所示：

I/O 成本：

$3.0 + 58 \times 1.0 = 61.0$ (范围区间的数量 + 预估的二级索引记录条数)

CPU 成本：

$58 \times 0.2 + 58 \times 0.2 + 0.01 = 23.21$ （读取二级索引记录的成本 + 读取并检测回表后聚簇索引记录的成本）

综上所述，使用 `idx_order_no` 执行查询的总成本就是：

$61.0 + 23.21 = 84.21$

是否有可能使用索引合并 (Index Merge)

本例中有关 `order_no` 和 `expire_time` 的搜索条件是使用 AND 连接起来的，而对于 `idx_order_no` 和 `idx_expire_time` 都是范围查询，也就是说查找到的二级索引记录并不是按照主键值进行排序的，并不满足使用 Intersection 索引合并的条件，所以并不会使用索引合并。而且 MySQL 查询优化器计算索引合并成本的算法也比较麻烦，所以我们也不会细说。

4. 对比各种方案，找出成本最低的那一个

下边把执行本例中的查询的各种可执行方案以及它们对应的成本列出来：

全表扫描的成本：2148.7

使用 `idx_expire_time` 的成本：55.61

使用 `idx_order_no` 的成本：84.21

很显然，使用 `idx_expire_time` 的成本最低，所以当然选择 `idx_expire_time` 来执行查询。

```
mysql> explain SELECT * FROM order_exp WHERE order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S') AND expire_time> '2021-03-22 18:24:00' AND order_note LIKE '%7排1%' AND order_status = 0;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | order_exp | NULL | range | idx_order_no,idx_expire_time | idx_expire_time | 5 | NULL | 39 |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

请注意：1、MySQL 的源码中对成本的计算实际要更复杂，但是基本思想和算法是没错的。

2、在 MySQL 的实际计算中，在和全文扫描比较成本时，使用索引的成本会去除读取并检测回表后聚簇索引记录的成本，也就是说，我们通过 MySQL 看到的成本将会是：`idx_expire_time` 为 47.81(55.61-7.8)，`idx_order_no` 为 72.61(84.21-11.6)。但是 MySQL 比较完成本后，会再计算一次使用索引的成本，

此时就会加上去除读取并检测回表后聚簇索引记录的成本，也就是我们计算出来的值。

5.3.2.2. 基于索引统计数据的成本计算

index dive

有时候使用索引执行查询时会有许多单点区间，比如使用 IN 语句就很容易产生非常多的单点区间，比如下边这个查询（下边查询语句中的...表示还有很多参数）：

```
SELECT * FROM order_exp WHERE order_no IN ('aa1', 'aa2', 'aa3', ... , 'zzz');
```

很显然，这个查询可能使用到的索引就是 idx_order_no，由于这个索引并不是唯一二级索引，所以并不能确定一个单点区间对应的二级索引记录的条数有多少，需要我们去计算。就是先获取索引对应的 B+树的区间最左记录和区间最右记录，然后再计算这两条记录之间有多少记录（记录条数少的时候可以做到精确计算，多的时候只能估算）。MySQL 把这种通过直接访问索引对应的 B+树来计算某个范围区间对应的索引记录条数的方式称之为 index dive。

有零星几个单点区间的话，使用 index dive 的方式去计算这些单点区间对应的记录数也不是什么问题，如果 IN 语句里 20000 个参数怎么办？

这就意味着 MySQL 的查询优化器为了计算这些单点区间对应的索引记录条数，要进行 20000 次 index dive 操作，这性能损耗就很大，搞不好计算这些单点区间对应的索引记录条数的成本比直接全表扫描的成本都大了。MySQL 考虑到了这种情况，所以提供了一个系统变量 eq_range_index_dive_limit，我们看一下在 MySQL 5.7.21 中这个系统变量的默认值：

```
show variables like '%dive%';
```

```
mysql> show variables like '%dive%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| eq_range_index_dive_limit | 200   |
+-----+-----+
1 row in set (0.00 sec)
```

也就是说如果我们的 IN 语句中的参数个数小于 200 个的话，将使用 index dive 的方式计算各个单点区间对应的记录条数，如果大于或等于 200 个的话，就不能使用 index dive 了，要使用所谓的索引统计数据来进行估算。怎么个估算法？

像会为每个表维护一份统计数据一样，MySQL 也会为表中的每一个索引维护一份统计数据，查看某个表中索引的统计数据可以使用 SHOW INDEX FROM 表名的语法，比如我们查看一下 order_exp 的各个索引的统计数据可以这么写：

```
show index from order_exp;
```

```
mysql> show index from order_exp;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| order_exp | 0 | PRIMARY | 1 | id | A | 9460 | NULL | NULL | | BTREE | |
| order_exp | 0 | u_idx_day_status | 1 | insert_time | A | 915 | NULL | NULL | | BTREE | |
| order_exp | 0 | u_idx_day_status | 2 | order_status | A | 915 | NULL | NULL | | BTREE | |
| order_exp | 0 | u_idx_day_status | 3 | expire_time | A | 9774 | NULL | NULL | | BTREE | |
| order_exp | 1 | idx_order_no | 1 | order_no | A | 9432 | NULL | NULL | | BTREE | |
| order_exp | 1 | idx_expire_time | 1 | expire_time | A | 9802 | NULL | NULL | | BTREE | |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

属性名 描述

Table 索引所属表的名称。

Non_unique 索引列的值是否是唯一的，聚簇索引和唯一二级索引的该列值为 0，普通二级索引该列值为 1。

Key_name 索引的名称。

Seq_in_index 索引列在索引中的位置，从 1 开始计数。比如对于联合索引 `u_idx_day_status`，来说，`'insert_time'`、`'order_status'`、`'expire_time'` 对应的位置分别是 1、2、3。

Column_name 索引列的名称。

Collation 索引列中的值是按照何种排序方式存放的，值为 A 时代表升序存放，为 NULL 时代表降序存放。

Cardinality 索引列中不重复值的数量。后边我们会重点看这个属性的。

Sub_part 对于存储字符串或者字节串的列来说，有时候我们只想对这些串的前 n 个字符或字节建立索引，这个属性表示的就是那个 n 值。如果对完整的列建立索引的话，该属性的值就是 NULL。

Packed 索引列如何被压缩，NULL 值表示未被压缩。这个属性我们暂时不了解，可以先忽略掉。

Null 该索引列是否允许存储 NULL 值。

Index_type 使用索引的类型，我们最常见的就是 BTREE，其实也就是 B+树索引。

Comment 索引列注释信息。

Index_comment 索引注释信息。

Cardinality 属性，**Cardinality** 直译过来就是基数的意思，表示索引列中不重复值的个数。比如对于一个一万行记录的表来说，某个索引列的 **Cardinality** 属性是 10000，那意味着该列中没有重复的值，如果 **Cardinality** 属性是 1 的话，就意味着该列的值全部是重复的。不过需要注意的是，对于 InnoDB 存储引擎来说，使用 **SHOW INDEX** 语句展示出来的某个索引列的 **Cardinality** 属性是一个估计值，并不是精确的。

前边说道，当 **IN** 语句中的参数个数大于或等于系统变量 `eq_range_index_dive_limit` 的值的话，就不会使用 `index dive` 的方式计算各个单点区间对应的索引记录条数，而是使用索引统计数据，这里所指的索引统计数据指的是这两个值：

使用 SHOW TABLE STATUS 展示出的 Rows 值,也就是一个表中有多少条记录。

使用 SHOW INDEX 语句展示出的 Cardinality 属性。

结合上一个 Rows 统计数据,我们可以针对索引列,计算出平均一个值重复多少次。

一个值的重复次数 $\approx \text{Rows} \div \text{Cardinality}$

以 order_exp 表的 idx_order_no 索引为例,它的 Rows 值是 10350,它对应的 Cardinality 值是 10220,所以我们可以计算 order_no 列平均单个值的重复次数就是:

$10350 \div 10220 \approx 1.012$ (条)

此时再看上边那条查询语句:

```
SELECT * FROM order_exp WHERE order_no IN ('aa1', 'aa2', 'aa3', ... , 'zzz');
```

假设 IN 语句中有 20000 个参数的话,就直接使用统计数据来估算这些参数需要单点区间对应的记录条数了,每个参数大约对应 1.012 条记录,所以总共需要回表的记录数就是:

$20000 \times 1.012 = 21,730$

使用统计数据来计算单点区间对应的索引记录条数比 index dive 的方式简单,但是它的致命弱点就是:不精确!。使用统计数据算出来的查询成本与实际所需的成本可能相差非常大。

大家需要注意一下,在 MySQL 5.7.3 以及之前的版本中,eq_range_index_dive_limit 的默认值为 10,之后的版本默认值为 200。所以如果大家采用的是 5.7.3 以及之前的版本的话,很容易采用索引统计数据而不是 index dive 的方式来计算查询成本。当你的查询中使用到了 IN 查询,但是却实际没有用到索引,就应该考虑一下是不是由于 eq_range_index_dive_limit 值太小导致的。

5.3.3. 再深入 Explain

5.3.3.1. EXPLAIN 输出成本

前面我们已经对 MySQL 查询优化器如何计算成本有了比较深刻的了解。但是 EXPLAIN 语句输出中缺少了一个衡量执行计划好坏的重要属性——成本。不过 MySQL 已经为我们提供了一种查看某个执行计划花费的成本的方式:

在 EXPLAIN 单词和真正的查询语句中间加上 FORMAT=JSON。

这样我们就可以得到一个 json 格式的执行计划,里边包含该计划花费的成本,比如这样:

```
explain format=json SELECT * FROM order_exp WHERE order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S') AND expire_time> '2021-03-22 18:28:28' AND expire_time<= '2021-03-22 18:35:09' AND insert_time> expire_time AND order_note LIKE '%7 排 1%' AND order_status = 0\G
```

```
***** 1. row *****
```

```
EXPLAIN: {
```

```
"query_block": {
  "select_id": 1, # 整个查询语句只有 1 个 SELECT 关键字, 该关键字对应的 id 号为 1
  "cost_info": {
    "query_cost": "55.61" # 整个查询的执行成本预计为 55.61
  },
  "table": {
    "table_name": "order_exp",
    "access_type": "range",
    "possible_keys": [
      "idx_order_no",
      "idx_expire_time"
    ],
    "key": "idx_expire_time",
    "used_key_parts": [
      "expire_time"
    ],
    "key_length": "5",
    "rows_examined_per_scan": 39,
    "rows_produced_per_join": 0,
    "filtered": "0.13",
    "index_condition": "(`mysqladv`.`order_exp`.`expire_time` > '2021-03-22 18:28:28')
and (`mysqladv`.`order_exp`.`expire_time` <= '2021-03-22 18:35:09'))",
    "cost_info": {
      "read_cost": "55.60",
      "eval_cost": "0.01",
      "prefix_cost": "55.61", #单独查询表的成本, 也就是: read_cost + eval_cost
      "data_read_per_join": "24" #和连接查询相关的数据量, 单位字节, 这里无用
    },
    "used_columns": [
      "id",
      "order_no",
      "order_note",
      "insert_time",
      "expire_duration",
      "expire_time",
      "order_status"
    ]
  }
}
```

```

],
    "attached_condition": "((`mysqladv`.`order_exp`.`order_status` = 0) and
(`mysqladv`.`order_exp`.`order_no` in ('DD00_6S','DD00_9S','DD00_10S')) and
(`mysqladv`.`order_exp`.`insert_time` > `mysqladv`.`order_exp`.`expire_time`) and
(`mysqladv`.`order_exp`.`order_note` like '%7 排 1%'))"
}
}
}
1 row in set, 1 warning (0.00 sec)

```

5.3.3.2. Optimizer Trace

对于 MySQL5.6 之前的版本来说，只能通过 EXPLAIN 语句查看到最后优化器决定使用的执行计划，却无法知道它为什么做这个决策。我们可能有这样的疑问：“我就觉得使用其他的执行方案比 EXPLAIN 输出的这种方案强，凭什么优化器做的决定和我想的不一样呢？为什么 MySQL 一定要全文扫描，不用索引呢？”

在 MySQL 5.6 以及之后的版本中，MySQL 提出了一个 optimizer trace 的功能，这个功能可以让我们方便的查看优化器生成执行计划的整个过程，这个功能的开启与关闭由系统变量 optimizer_trace 决定：

SHOW VARIABLES LIKE 'optimizer_trace';

```

mysql> SHOW VARIABLES LIKE 'optimizer_trace';
+-----+-----+
| Variable_name | Value                               |
+-----+-----+
| optimizer_trace | enabled=off,one_line=off          |
+-----+-----+
1 row in set (0.01 sec)

```

可以看到 enabled 值为 off，表明这个功能默认是关闭的。one_line 的值是控制输出格式的，如果为 on 那么所有输出都将在一行中展示，我们就保持其默认值为 off。

如果想打开这个功能，必须首先把 enabled 的值改为 on，就像这样：

SET optimizer_trace="enabled=on";

```

mysql> SET optimizer_trace="enabled=on";
Query OK, 0 rows affected (0.00 sec)

```

然后我们就可以输入我们想要查看优化过程的查询语句，当该查询语句执行完成后，就可以到 information_schema 数据库下的 OPTIMIZER_TRACE 表中查看完整的优化过程。这个 OPTIMIZER_TRACE 表有 4 个列，分别是：

QUERY：表示我们的查询语句。

TRACE：表示优化过程的 JSON 格式文本。

MISSING_BYTES_BEYOND_MAX_MEM_SIZE: 由于优化过程可能会输出很多，如果超过某个限制时，多余的文本将不会被显示，这个字段展示了被忽略的文本字节数。

INSUFFICIENT_PRIVILEGES: 表示是否没有权限查看优化过程，默认值是 0，只有某些特殊情况下才会是 1，我们暂时不关心这个字段的值。

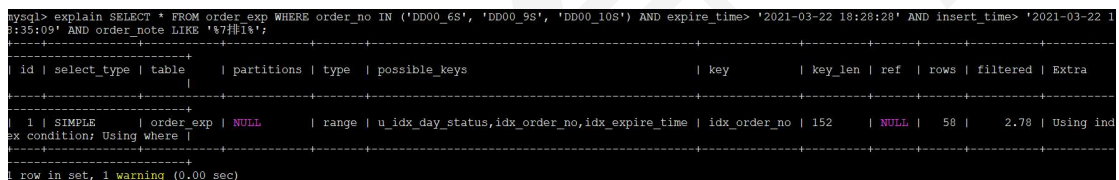
当停止查看语句的优化过程时，把 optimizer trace 功能关闭

SET optimizer_trace="enabled=off";

注意：开启 trace 会影响 mysql 性能，所以只能临时分析 sql 使用，用完之后立即关闭。

现在有一个搜索条件比较多的查询语句，它的执行计划如下：

explain SELECT * FROM order_exp WHERE order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S') AND expire_time> '2021-03-22 18:28:28' AND insert_time> '2021-03-22 18:35:09' AND order_note LIKE '%7排1%';



id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	order_exp		NULL	u_idx_day_status,idx_order_no,idx_expire_time	idx_order_no	152	NULL	58	2.78	Using index condition; Using where

row in set, 1 warning (0.00 sec)

可以看到该查询可能使用到的索引有 3 个 u_idx_day_status,idx_order_no,idx_expire_time，那么为什么优化器最终选择了 idx_order_no 而不选择其他的索引或者直接全表扫描呢？这时候就可以通过 optimizer trace 功能来查看优化器的具体工作过程：

#记得开启 optimizer trace 功能

SELECT * FROM order_exp WHERE order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S') AND expire_time> '2021-03-22 18:28:28' AND insert_time> '2021-03-22 18:35:09' AND order_note LIKE '%7排1%';

SELECT * FROM information_schema.OPTIMIZER_TRACE\G

展示的内容极多，我们选择优化过程中的一些比较重要的点关注一下：

***** 1. row *****

分析的查询语句是什么

QUERY: SELECT * FROM order_exp WHERE order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S') AND expire_time> '2021-03-22 18:28:28' AND insert_time> '2021-03-22 18:35:09' AND order_note LIKE '%7排1%'

优化的具体过程

TRACE: {

"steps": [

{

"join_preparation": { # prepare 阶段

"select#": 1,

"steps": [

```

{
    "IN_uses_bisection": true
},
{# 类似于查询优化器将查询语句重写后的语句
    "expanded_query": "/* select#1 */ select `order_exp`.`id` AS `id`,`order_exp`.`order_no` AS
`order_no`,`order_exp`.`order_note` AS `order_note`,`order_exp`.`insert_time` AS `insert_time`,`order_exp`.`expire_duration` AS
`expire_duration`,`order_exp`.`expire_time` AS `expire_time`,`order_exp`.`order_status` AS `order_status` from `order_exp` where
((`order_exp`.`order_no` in ('DD00_6S','DD00_9S','DD00_10S')) and (`order_exp`.`expire_time` > '2021-03-22 18:28:28') and
(`order_exp`.`insert_time` > '2021-03-22 18:35:09') and (`order_exp`.`order_note` like '%7 排 1%'))"
    }
]
}
},
{
    "join_optimization": { # optimize 阶段
        "select#": 1,
        "steps": [
            {
                "condition_processing": { # 处理搜索条件
                    "condition": "WHERE",
                    # 原始搜索条件
                    "original_condition": "((`order_exp`.`order_no` in ('DD00_6S','DD00_9S','DD00_10S')) and (`order_exp`.`expire_time` >
'2021-03-22 18:28:28') and (`order_exp`.`insert_time` > '2021-03-22 18:35:09') and (`order_exp`.`order_note` like '%7 排 1%'))",
                    "steps": [
                        {
                            "transformation": "equality_propagation", # 等值传递转换
                            "resulting_condition": "((`order_exp`.`order_no` in ('DD00_6S','DD00_9S','DD00_10S')) and
(`order_exp`.`expire_time` > '2021-03-22 18:28:28') and (`order_exp`.`insert_time` > '2021-03-22 18:35:09') and
(`order_exp`.`order_note` like '%7 排 1%'))"
                        },
                        {
                            "transformation": "constant_propagation", # 常量传递转换
                            "resulting_condition": "((`order_exp`.`order_no` in ('DD00_6S','DD00_9S','DD00_10S')) and
(`order_exp`.`expire_time` > '2021-03-22 18:28:28') and (`order_exp`.`insert_time` > '2021-03-22 18:35:09') and
(`order_exp`.`order_note` like '%7 排 1%'))"
                        }
                    ]
                }
            }
        ]
    }
}

```

```
"transformation": "trivial_condition_removal", # 去除没用的条件

"resulting_condition": "((`order_exp`.`order_no` in ('DD00_6S','DD00_9S','DD00_10S')) and
(`order_exp`.`expire_time` > '2021-03-22 18:28:28') and (`order_exp`.`insert_time` > '2021-03-22 18:35:09') and
(`order_exp`.`order_note` like '%7 排 1%'))"

    }

  }

}

},

{

  "substitute_generated_columns": { # 替换虚拟生成列

  }

},

{

  "table_dependencies": [ # 表的依赖信息

    {

      "table": "`order_exp`",

      "row_may_be_null": false,

      "map_bit": 0,

      "depends_on_map_bits": [

      ]

    }

  ]

},

{

  "ref_optimizer_key_uses": [

  ]

},

{

  "rows_estimation": [ # 预估不同单表访问方法的访问成本

    {

      "table": "`order_exp`",

      "range_analysis": {

        "table_scan": { # 全表扫描的行数以及成本

          "rows": 10345,

          "cost": 2168.1

        },


```

```
"potential_range_indexes": [ # 分析可能使用的索引
{
    "index": "PRIMARY", # 主键
    "usable": false,
    "cause": "not_applicable" # 主键不可用
},
{
    "index": "u_idx_day_status", # u_idx_day_status 可能被使用
    "usable": true,
    "key_parts": [
        "insert_time",
        "order_status",
        "expire_time"
    ]
},
{
    "index": "idx_order_no", # idx_order_no 可能被使用
    "usable": true,
    "key_parts": [
        "order_no",
        "id"
    ]
},
{
    "index": "idx_expire_time", # idx_expire_time 可能被使用
    "usable": true,
    "key_parts": [
        "expire_time",
        "id"
    ]
}
],
"setup_range_conditions": [
],
"group_index_range": {
```

```
"chosen": false,

"cause": "not_group_by_or_distinct"

},

"analyzing_range_alternatives": { # 分析各种可能使用的索引的成本

"range_scan_alternatives": [

{

"index": "u_idx_day_status", # 使用 u_idx_day_status 的成本分析

"ranges": [ # 使用 u_idx_day_status 的范围区间

"0x99a92d28c9 < insert_time"

],

"index_dives_for_eq_ranges": true, # 是否使用 index dive

"rowid_ordered": false, # 使用该索引获取的记录是否按照主键排序

"using_mrr": false, # 是否使用 mrr

"index_only": false, # 是否是索引覆盖访问

"rows": 5172, # 使用该索引获取的记录条数

"cost": 6207.4, # 使用该索引的成本

"chosen": false, # 是否选择该索引

"cause": "cost" # 因为成本太大所以不选择该索引

},

{

"index": "idx_order_no",

"ranges": [

"DD00_10S <= order_no <= DD00_10S",

"DD00_6S <= order_no <= DD00_6S",

"DD00_9S <= order_no <= DD00_9S"

],

"index_dives_for_eq_ranges": true,

"rowid_ordered": false,

"using_mrr": false,

"index_only": false,

"rows": 58,

"cost": 72.61,

"chosen": true

},

{
```

```
"index": "idx_expire_time",

"ranges": [

    "0x99a92d271c < expire_time"

],

"index_dives_for_eq_ranges": true,

"rowid_ordered": false,

"using_mrr": false,

"index_only": false,

"rows": 5172,

"cost": 6207.4,

"chosen": false,

"cause": "cost"

}

],

"analyzing_roworder_intersect": { # 分析使用索引合并的成本

    "usable": false,

    "cause": "too_few_roworder_scans"

}

},

"chosen_range_access_summary": { # 对于上述单表查询 s1 最优的访问方法

    "range_access_plan": {

        "type": "range_scan",

        "index": "idx_order_no",

        "rows": 58,

        "ranges": [

            "DD00_10S <= order_no <= DD00_10S",

            "DD00_6S <= order_no <= DD00_6S",

            "DD00_9S <= order_no <= DD00_9S"

        ]

    },

    "rows_for_plan": 58,

    "cost_for_plan": 72.61,

    "chosen": true

}

}
```

```

    }
  ]
},
{
  # 分析各种可能的执行计划,负责对比各可行计划的开销,并选择相对最优的执行计划。
  # (对多表查询这可能有很多种不同的方案,单表查询的方案上边已经分析过了,直接选取
  idx_order_no 即可)
  "considered_execution_plans": [
    {
      "plan_prefix": [
      ],
      "table": "`order_exp`",
      "best_access_path": {
        "considered_access_paths": [
          {
            "rows_to_scan": 58,
            "access_type": "range",
            "range_details": {
              "used_index": "idx_order_no"
            },
            "resulting_rows": 58,
            "cost": 84.21,
            "chosen": true
          }
        ]
      },
      "condition_filtering_pct": 100,
      "rows_for_plan": 58,
      "cost_for_plan": 84.21,
      "chosen": true
    }
  ],
  # 尝试给查询添加一些其他的查询条件,增加主要是为了便于 ICP (索引条件下推)
  "attaching_conditions_to_tables": {

```

```
"original_condition": "((`order_exp`.`order_no` in ('DD00_6S','DD00_9S','DD00_10S')) and (`order_exp`.`expire_time` >
'2021-03-22 18:28:28') and (`order_exp`.`insert_time` > '2021-03-22 18:35:09') and (`order_exp`.`order_note` like '%7 排 1%'))",

"attached_conditions_computation": [

],

"attached_conditions_summary": [

{

"table": "`order_exp`",

"attached": "((`order_exp`.`order_no` in ('DD00_6S','DD00_9S','DD00_10S')) and (`order_exp`.`expire_time` >
'2021-03-22 18:28:28') and (`order_exp`.`insert_time` > '2021-03-22 18:35:09') and (`order_exp`.`order_note` like '%7 排 1%'))"

}

]

},

{

# 再稍稍的改进一下执行计划

"refine_plan": [

{

"table": "`order_exp`",

"pushed_index_condition": "(`order_exp`.`order_no` in ('DD00_6S','DD00_9S','DD00_10S'))",

"table_condition_attached": "((`order_exp`.`expire_time` > '2021-03-22 18:28:28') and (`order_exp`.`insert_time` >
'2021-03-22 18:35:09') and (`order_exp`.`order_note` like '%7 排 1%'))"

}

]

}

},

{

"join_execution": { # execute 阶段

"select#": 1,

"steps": [

]

}

}

]

}
```

因优化过程文本太多而丢弃的文本字节大小，值为 0 时表示并没有丢弃

MISSING_BYTES_BEYOND_MAX_MEM_SIZE: 0

权限字段

INSUFFICIENT_PRIVILEGES: 0

1 row in set (0.00 sec)

优化过程大致分为了三个阶段：

prepare 阶段

optimize 阶段

execute 阶段

我们所说的基于成本的优化主要集中在 optimize 阶段，对于单表查询来说，我们主要关注 optimize 阶段的"rows_estimation"这个过程，这个过程深入分析了对单表查询的各种执行方案的成本；

对于多表连接查询来说，我们更多需要关注"considered_execution_plans"这个过程，这个过程里会写明各种不同的连接方式所对应的成本。反正优化器最终会选择成本最低的那种方案来作为最终的执行计划，也就是我们使用 EXPLAIN 语句所展现出的那种方案。

如果对使用 EXPLAIN 语句展示出的对某个查询的执行计划很不理解，就可以尝试使用 optimizer trace 功能来详细了解每一种执行方案对应的成本。

5.3.4. 连接查询的成本

5.3.4.1. Condition filtering 介绍

连接查询至少是要有两个表的，课程的讲述中可能使用 order_exp 表的派生表 s1、s2 和 order_exp2。

我们前边说过，MySQL 中连接查询采用的是嵌套循环连接算法，驱动表会被访问一次，被驱动表可能会被访问多次，所以对于两表连接查询来说，它的查询成本由下边两个部分构成：

单次查询驱动表的成本

多次查询被驱动表的成本（具体查询多少次取决于对驱动表查询的结果集中有多少条记录）

对驱动表进行查询后得到的记录条数称之为驱动表的**扇出**（英文名：fanout）。很显然驱动表的扇出值越小，对被驱动表的查询次数也就越少，连接查询的总成本也就越低。当查询优化器想计算整个连接查询所使用的成本时，就需要计算出驱动表的扇出值，有的时候扇出值的计算是很容易的，比如下边这两个查询：

查询一：

```
SELECT * FROM order_exp AS s1 INNER JOIN order_exp2 AS s2;
```

假设使用 s1 表作为驱动表，很显然对驱动表的单表查询只能使用全表扫描的方式执行，驱动表的扇出值也很明确，那就是驱动表中有多少记录，扇出值就

是多少。统计数据中 s1 表的记录行数是 10573，也就是说优化器就直接会把 10573 当作在 s1 表的扇出值。

查询二：

```
SELECT * FROM order_exp AS s1 INNER JOIN order_exp2 AS s2
WHERE s1.expire_time> '2021-03-22 18:28:28' AND s1.expire_time<=
'2021-03-22 18:35:09';
```

仍然假设 s1 表是驱动表的话，很显然对驱动表的单表查询可以使用 idx_expire_time 索引执行查询。此时范围区间('2021-03-22 18:28:28', '2021-03-22 18:35:09')中有多少条记录，那么扇出值就是多少。

但是有的时候扇出值的计算就变得更棘手，比方说下边几个查询：

查询三：

```
SELECT * FROM order_exp AS s1 INNER JOIN order_exp2 AS s2 WHERE
s1.order_note > 'xyz';
```

本查询和查询一类似，只不过对于驱动表 s1 多了一个 order_note > 'xyz' 的搜索条件。查询优化器又不会真正的去执行查询，所以它只能猜这 10573 记录里有多少条记录满足 order_note > 'xyz' 条件。

查询四：

```
SELECT * FROM order_exp AS s1 INNER JOIN order_exp2 AS s2 WHERE
s1.expire_time> '2021-03-22 18:28:28' AND s1.expire_time<= '2021-03-22 18:35:09'
AND s1.order_note > 'xyz';
```

本查询和查询二类似，只不过对于驱动表 s1 也多了一个 order_note > 'xyz' 的搜索条件。不过因为本查询可以使用 idx_expire_time 索引，所以只需要从符合二级索引范围区间的记录中猜有多少条记录符合 order_note > 'xyz' 条件，也就是只需要猜在 39 条记录中有多少符合 order_note > 'xyz' 条件。

```
mysql> explain SELECT * FROM order_exp AS s1 WHERE s1.expire_time> '2021-03-22 18:28:28' AND s1.expire_time<=
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible keys | key | key_len | ref | rows | filtered |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | s1 | NULL | range | idx_expire_time | idx_expire_time | 5 | NULL | 39 | 100 |
```

查询五：

```
SELECT * FROM order_exp AS s1 INNER JOIN order_exp2 AS s2 WHERE
s1.expire_time> '2021-03-22 18:28:28' AND s1.expire_time<= '2021-03-22 18:35:09'
AND s1.order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S') AND s1.order_note >
'xyz';
```

本查询和查询四类似，不过在驱动表 s1 选取 idx_expire_time 索引执行查询后，优化器需要从符合二级索引范围区间的记录中猜有多少条记录符合下边两个条件：

order_no IN ('DD00_6S', 'DD00_9S', 'DD00_10S')

order_note > 'xyz'

也就是优化器需要猜在 39 条记录中有多少符合上述两个条件的。

说了这么多，其实就是想表达在这两种情况下计算驱动表扇出值时需要靠猜：

如果使用的是全表扫描的方式执行的单表查询，那么计算驱动表扇出时需要猜满足搜索条件的记录到底有多少条。

如果使用的是索引执行的单表扫描，那么计算驱动表扇出的时候需要猜满足除使用到对应索引的搜索条件外的其他搜索条件的记录有多少条。

MySQL 把这个猜的过程称之为 **condition filtering**。当然，这个过程可能会使用到索引，也可能使用到统计数据，也可能就是 MySQL 单纯的瞎猜，整个评估过程非常复杂，所以我们不去细讲。

在 MySQL 5.7 之前的版本中，查询优化器在计算驱动表扇出时，如果是使用全表扫描的话，就直接使用表中记录的数量作为扇出值，如果使用索引的话，就直接使用满足范围条件的索引记录条数作为扇出值。

在 MySQL 5.7 中，MySQL 引入了这个 **condition filtering** 的功能，就是还要猜一猜剩余的那些搜索条件能把驱动表中的记录再过滤多少条，其实本质上就是为了让成本估算更精确。我们所说的纯粹瞎猜其实是很不严谨的，MySQL 称之为启发式规则。

5.3.4.2. 两表连接的成本分析

连接查询的成本计算公式是这样的：

连接查询总成本 = 单次访问驱动表的成本 + 驱动表扇出数 x 单次访问被驱动表的成本

对于左（外）连接和右（外）连接查询来说，它们的驱动表是固定的，所以想要得到最优的查询方案只需要分别为驱动表和被驱动表选择成本最低的访问方法。

可是对于内连接来说，驱动表和被驱动表的位置是可以互换的，所以需要考虑两个方面的问题：

不同的表作为驱动表最终的查询成本可能是不同的，也就是需要考虑最优的表连接顺序。然后分别为驱动表和被驱动表选择成本最低的访问方法。

很显然，计算内连接查询成本的方式更麻烦一些，下边我们就以内连接为例来看看如何计算出最优的连接查询方案。当然在某些情况下，左（外）连接和右（外）连接查询在某些特殊情况下可以被优化为内连接查询。

我们来看看内连接，比如对于下边这个查询来说：

```
SELECT * FROM order_exp AS s1 INNER JOIN order_exp2 AS s2 ON s1.order_no=
s2.order_note WHERE s1.expire_time> '2021-03-22 18:28:28' AND
s1.expire_time<= '2021-03-22 18:35:09' AND s2.expire_time> '2021-03-22 18:35:09'
AND s2.expire_time<= '2021-03-22 18:35:59';
```

可以选择的连接顺序有两种：

s1 连接 s2，也就是 s1 作为驱动表，s2 作为被驱动表。

s2 连接 s1，也就是 s2 作为驱动表，s1 作为被驱动表。

查询优化器需要分别考虑这两种情况下的最优查询成本，然后选取那个成本更低的连接顺序以及该连接顺序下各个表的最优访问方法作为最终的查询计划。我们定性的分析一下，不像分析单表查询那样定量的分析了：

使用 s1 作为驱动表的情况

分析对于驱动表的成本最低的执行方案，首先看一下涉及 s1 表的搜索条件有哪些：

`s1.expire_time > '2021-03-22 18:28:28' AND s1.expire_time <= '2021-03-22 18:35:09'`

所以这个查询可能使用到 `idx_expire_time` 索引，从全表扫描和使用 `idx_expire_time` 这两个方案中选出成本最低的那个，很显然使用 `idx_expire_time` 执行查询的成本更低些。

然后分析对于被驱动表的成本最低的执行方案，此时涉及被驱动表 s2 的搜索条件就是：

1、`s2.order_note = 常数`（这是因为对驱动表 s1 结果集中的每一条记录，都需要进行一次被驱动表 s2 的访问，此时那些涉及两表的条件现在相当于只涉及被驱动表 s2 了。）

2、`s2.expire_time > '2021-03-22 18:35:09' AND s2.expire_time <= '2021-03-22 18:35:59'`

很显然，第一个条件由于 `order_note` 没有用到索引，所以并没有什么用，此时访问 s2 表时可行的方案也是全表扫描和使用 `idx_expire_time` 两种，假设使用 `idx_expire_time` 的成本更小。

所以此时使用 s1 作为驱动表时的总成本就是（暂时不考虑使用 join buffer 对成本的影响）：

使用 `idx_expire_time` 访问 s1 的成本 + s1 的扇出 × 使用 `idx_expire_time` 访问 s2 的成本

使用 s2 作为驱动表的情况

分析对于驱动表的成本最低的执行方案

首先看一下涉及 s2 表的搜索条件有哪些：

`s2.expire_time > '2021-03-22 18:35:09' AND s2.expire_time <= '2021-03-22 18:35:59'`

所以这个查询可能使用到 `idx_expire_time` 索引，从全表扫描和使用 `idx_expire_time` 这两个方案中选出成本最低的那个，假设使用 `idx_expire_time` 执行查询的成本更低些。

然后分析对于被驱动表的成本最低的执行方案

此时涉及被驱动表 s1 的搜索条件就是：

1、`s1.order_no = 常数`

2、`s1.expire_time > '2021-03-22 18:28:28' AND s1.expire_time <= '2021-03-22 18:35:09'`

这时就很有趣了，使用 `idx_order_no` 可以进行 ref 方式的访问，使用 `idx_expire_time` 可以使用 range 方式的访问。

那么优化器需要从全表扫描、使用 `idx_order_no`、使用 `idx_expire_time` 这几个方案里选出一个成本最低的方案。

这里有个问题，因为 `idx_expire_time` 的范围区间是确定的，怎么计算使用 `idx_expire_time` 的成本我们上边已经说过了，可是在没有真正执行查询前，`s1.order_no = 常数` 中的常数值我们是不知道的，怎么衡量使用 `idx_order_no` 执行查询的成本呢？其实很简单，直接使用我们前面说过的索引统计数据就好了（就是索引列平均一个值重复多少次）。一般情况下，`ref` 的访问方式要比 `range` 成本更低，这里假设使用 `idx_order_no` 进行对 `s1` 的访问。

所以此时使用 `s2` 作为驱动表时的总成本就是：

使用 `idx_expire_time` 访问 `s2` 的成本 + `s2` 的扇出 × 使用 `idx_order_no` 访问 `s1` 的成本

最后优化器会比较这两种方式的最优访问成本，选取那个成本更低的连接顺序去真正的执行查询。从上边的计算过程也可以看出来，一般来讲，连接查询成本占大头的其实是驱动表扇出数 × 单次访问被驱动表的成本，所以我们的优化重点其实是下边这两个部分：

尽量减少驱动表的扇出

对被驱动表的访问成本尽量低

这一点对于我们实际书写连接查询语句时十分有用，我们需要尽量在被驱动表的连接列上建立索引，这样就可以使用 `ref` 访问方法来降低访问被驱动表的成本了。如果可以，被驱动表的连接列最好是该表的主键或者唯一二级索引列，这样就可以把访问被驱动表的成本降到更低了。

5.3.4.3. EXPLAIN 输出连接成本

连接查询在输出成本时和单表查询稍有不同，如下：

```
explain format=json SELECT * FROM order_exp AS s1 INNER JOIN order_exp2
AS s2 ON s1.order_no= s2.order_note WHERE s1.expire_time> '2021-03-22
18:28:28' AND s1.expire_time<= '2021-03-22 18:35:09' AND s2.expire_time>
'2021-03-22 18:35:09' AND s2.expire_time<= '2021-03-22 18:35:59'\G
```

```
***** 1. row *****
```

```
EXPLAIN: {
```

```
  "query_block": {
```

```
    "select_id": 1, # 整个查询语句只有 1 个 SELECT 关键字，该关键字对应的 id 号为 1
```

```
    "cost_info": {
```

```
      "query_cost": "840.51" # 整个查询的执行成本
```

```
    },
```

```
    "nested_loop": [ # 几个表之间采用嵌套循环连接算法执行
```

```
      {
```

```
        "table": {
```

```
          "table_name": "s2", # s2 表是驱动表
```

```
          "access_type": "range", # 访问方法为 range
```

```
"possible_keys": [
  "idx_expire_time"
],
"key": "idx_expire_time",
"used_key_parts": [
  "expire_time"
],
"key_length": "5",
"rows_examined_per_scan": 321, # 查询 s2 表大致需要扫描 321 条记录
"rows_produced_per_join": 321, # 驱动表 s2 的扇出是 321
"filtered": "100.00", # condition filtering 代表的百分比
"index_condition": "((`mysqladv`.`s2`.`expire_time` > '2021-03-22
18:35:09') and (`mysqladv`.`s2`.`expire_time` <= '2021-03-22 18:35:59'))",
"cost_info": {
  "read_cost": "386.21",
  "eval_cost": "64.20",
  "prefix_cost": "450.41", # 查询 s1 表总共的成本, read_cost + eval_cost
  "data_read_per_join": "152K" # 读取的数据量
},
"used_columns": [
  "id",
  "order_no",
  "order_note",
  "insert_time",
  "expire_duration",
  "expire_time",
  "order_status"
]
}
},
{
  "table": {
    "table_name": "s1", # s1 表是被驱动表
    "access_type": "ref",
    "possible_keys": [
      "idx_order_no",
```

```

        "idx_expire_time"
    ],
    "key": "idx_order_no",
    "used_key_parts": [
        "order_no"
    ],
    "key_length": "152",
    "ref": [
        "mysqladv.s2.order_note"
    ],
    "rows_examined_per_scan": 1, # 查询一次 s1 表大致需要扫描 1 条记录
    "rows_produced_per_join": 16, # 被驱动表 s2 的扇出是 16（由于没有多余的
    表进行连接，所以这个值无用）
    "filtered": "4.94", # condition filtering 代表的百分比
    "index_condition": "(`mysqladv`.`s1`.`order_no` =
`mysqladv`.`s2`.`order_note`)",
    "cost_info": {
        "read_cost": "325.08",
        "eval_cost": "3.21",
        "prefix_cost": "840.51", # 单次查询 s2、多次查询 s1 表总共的成本
        "data_read_per_join": "7K"
    },
    "used_columns": [
        "id",
        "order_no",
        "order_note",
        "insert_time",
        "expire_duration",
        "expire_time",
        "order_status"
    ],
    "attached_condition": "((`mysqladv`.`s1`.`expire_time` >
'2021-03-22 18:28:28') and (`mysqladv`.`s1`.`expire_time` <= '2021-03-22
18:35:09'))"
    }
}

```

```
    ]
  }
}
```

1 row in set, 1 warning (0.00 sec)

我们使用#后边跟随注释的形式为大家解释了 EXPLAIN FORMAT=JSON 语句的输出内容，s2 表的"cost_info"中 prefix_cost 就是单独查询 s2 表的成本。

对于 s1 表的"cost_info"中，由于 s1 表是被驱动表，所以可能被读取多次，这里的 read_cost 和 eval_cost 是访问多次 s2 表后累加起来的值，而 s1 表中的 prefix_cost 的值代表的是整个连接查询预计的成本。

看完了上面的执行计划的输出，可能大家有疑惑，驱动表 S2 的查询成本为 450.41，总查询成本为 840.51，也就是说对被驱动表 S1 的查询成本也就是 390 左右，看起来用 S1 做驱动表好像更省一点。真的这样吗？我们把 SQL 语句改造一下，将 INNER JOIN 替换为 STRAIGHT_JOIN：

```
explain format=json SELECT * FROM order_exp AS s1 STRAIGHT_JOIN
order_exp2 AS s2 ON s1.order_no= s2.order_note WHERE s1.expire_time>
'2021-03-22 18:28:28' AND s1.expire_time<= '2021-03-22 18:35:09' AND
s2.expire_time> '2021-03-22 18:35:09' AND s2.expire_time<= '2021-03-22
18:35:59'\G
```

大家可以自行看看所需要的成本是多少，并自行解释一下原因。

5.3.4.4. 多表连接的成本分析

首先要考虑一下多表连接时可能产生出多少种连接顺序：

对于两表连接，比如表 A 和表 B 连接

只有 AB、BA 这两种连接顺序。其实相当于 $2 \times 1 = 2$ 种连接顺序。

对于三表连接，比如表 A、表 B、表 C 进行连接

有 ABC、ACB、BAC、BCA、CAB、CBA 这么 6 种连接顺序。其实相当于 $3 \times 2 \times 1 = 6$ 种连接顺序。

对于四表连接的话，则会有 $4 \times 3 \times 2 \times 1 = 24$ 种连接顺序。

对于 n 表连接的话，则有 $n \times (n-1) \times (n-2) \times \dots \times 1$ 种连接顺序，就是 n 的阶乘种连接顺序，也就是 n!。

有 n 个表进行连接，MySQL 查询优化器要每一种连接顺序的成本都计算一遍么？那可是 n!种连接顺序呀。其实真的是要都算一遍，不过 MySQL 用了很多办法减少计算非常多种连接顺序的成本的方法：

提前结束某种顺序的成本评估

MySQL 在计算各种链接顺序的成本之前，会维护一个全局的变量，这个变量表示当前最小的连接查询成本。如果在分析某个连接顺序的成本时，该成本已经超过当前最小的连接查询成本，那就压根儿不对该连接顺序继续往下分析了。比方说 A、B、C 三个表进行连接，已经得到连接顺序 ABC 是当前的最小连接成本，

比方说 10.0，在计算连接顺序 BCA 时，发现 B 和 C 的连接成本就已经大于 10.0 时，就不再继续往后分析 BCA 这个连接顺序的成本了。

系统变量 optimizer_search_depth

为了防止无穷无尽的分析各种连接顺序的成本，MySQL 提出了 optimizer_search_depth 系统变量，如果连接表的个数小于该值，那么就继续穷举分析每一种连接顺序的成本，否则只对与 optimizer_search_depth 值相同数量的表进行穷举分析。很显然，该值越大，成本分析的越精确，越容易得到好的执行计划，但是消耗的时间也就越长，否则得到不是很好的执行计划，但可以省掉很多分析连接成本的时间。

根据某些规则压根儿就不考虑某些连接顺序

即使是有上边两条规则的限制，但是分析多个表不同连接顺序成本花费的时间还是会很长，所以 MySQL 干脆提出了一些所谓的启发式规则（就是根据以往经验指定的一些规则），凡是不满足这些规则的连接顺序压根儿就不分析，这样可以极大的减少需要分析的连接顺序的数量，但是也可能造成错失最优的执行计划。他们提供了一个系统变量 optimizer_prune_level 来控制到底是不是用这些启发式规则。

5.3.5. 调节成本常数

我们前边已经介绍了两个成本常数：

读取一个页面花费的成本默认是 1.0

检测一条记录是否符合搜索条件的成本默认是 0.2

其实除了这两个成本常数，MySQL 还支持很多，它们被存储到了 MySQL 数据库的两个表中：

SHOW TABLES FROM mysql LIKE '%cost%';

```
mysql> SHOW TABLES FROM mysql LIKE '%cost%';
+-----+
| Tables_in_mysql (%cost%) |
+-----+
| engine_cost               |
| server_cost               |
+-----+
2 rows in set (0.00 sec)
```

因为一条语句的执行其实是分为两层的：server 层、存储引擎层。

在 server 层进行连接管理、查询缓存、语法解析、查询优化等操作，在存储引擎层执行具体的数据存取操作。也就是说一条语句在 server 层中执行的成本是和它操作的表使用的存储引擎是没关系的，所以关于这些操作对应的成本常数就存储在了 server_cost 表中，而依赖于存储引擎的一些操作对应的成本常数就存储在了 engine_cost 表中。

5.3.5.1. mysql.server_cost 表

server_cost 表中在 server 层进行的一些操作对应的成本常数，具体内容如下：

```
SELECT * FROM mysql.server_cost;
```

```
mysql> SELECT * FROM mysql.server_cost;
```

cost_name	cost_value	last_update	comment
disk_temptable_create_cost	NULL	2021-03-19 16:19:15	NULL
disk_temptable_row_cost	NULL	2021-03-19 16:19:15	NULL
key_compare_cost	NULL	2021-03-19 16:19:15	NULL
memory_temptable_create_cost	NULL	2021-03-19 16:19:15	NULL
memory_temptable_row_cost	NULL	2021-03-19 16:19:15	NULL
row_evaluate_cost	NULL	2021-03-19 16:19:15	NULL

```
6 rows in set (0.00 sec)
```

我们先看一下 `server_cost` 各个列都分别是什么意思：

cost_name

表示成本常数的名称。

cost_value

表示成本常数对应的值。如果该列的值为 `NULL` 的话，意味着对应的成本常数会采用默认值。

last_update

表示最后更新记录的时间。

comment

注释。

从 `server_cost` 中的内容可以看出来，目前在 `server` 层的一些操作对应的成本常数有以下几种：

disk_temptable_create_cost 默认值 40.0 创建基于磁盘的临时表的成本，如果增大这个值的话会让优化器尽量少的创建基于磁盘的临时表。

disk_temptable_row_cost 默认值 1.0 向基于磁盘的临时表写入或读取一条记录的成本，如果增大这个值的话会让优化器尽量少的创建基于磁盘的临时表。

key_compare_cost 0.1 两条记录做比较操作的成本，多用在排序操作上，如果增大这个值的话会提升 `filesort` 的成本，让优化器可能更倾向于使用索引完成排序而不是 `filesort`。

memory_temptable_create_cost 默认值 2.0 创建基于内存的临时表的成本，如果增大这个值的话会让优化器尽量少的创建基于内存的临时表。

memory_temptable_row_cost 默认值 0.2 向基于内存的临时表写入或读取一条记录的成本，如果增大这个值的话会让优化器尽量少的创建基于内存的临时表。

row_evaluate_cost 默认值 0.2 这个就是我们之前一直使用的检测一条记录是否符合搜索条件的成本，增大这个值可能让优化器更倾向于使用索引而不是直接全表扫描。

MySQL 在执行诸如 `DISTINCT` 查询、分组查询、`Union` 查询以及某些特殊条件下的排序查询都可能在内部先创建一个临时表，使用这个临时表来辅助完成查询（比如对于 `DISTINCT` 查询可以建一个带有 `UNIQUE` 索引的临时表，直接把需要去重的记录插入到这个临时表中，插入完成之后的记录就是结果集了）。在数据量大的情况下可能创建基于磁盘的临时表，也就是为该临时表使用 `MyISAM`、`InnoDB`

等存储引擎，在数据量不大时可能创建基于内存的临时表，也就是使用 **Memory** 存储引擎。大家可以看到，创建临时表和对这个临时表进行写入和读取的操作代价还是很高的就行了。

这些成本常数在 `server_cost` 中的初始值都是 **NULL**，意味着优化器会使用它们的默认值来计算某个操作的成本，如果我们想修改某个成本常数的值的话，需要做两个步骤：

对我们感兴趣的成本常数做 **update** 更新操作，然后使用下边语句即可：

FLUSH OPTIMIZER_COSTS;

当然，在你修改完某个成本常数后想把它们再改回默认值的话，可以直接把 `cost_value` 的值设置为 **NULL**，再使用 **FLUSH OPTIMIZER_COSTS** 语句让系统重新加载。

5.3.5.2. *mysql.engine_cost* 表

`engine_cost` 表表中在存储引擎层进行的一些操作对应的成本常数，具体内容如下：

SELECT * FROM mysql.engine_cost;

```
mysql> SELECT * FROM mysql.engine_cost;
```

engine_name	device_type	cost_name	cost_value	last_update	comment
default	0	io_block_read_cost	NULL	2021-03-19 16:19:15	NULL
default	0	memory_block_read_cost	NULL	2021-03-19 16:19:15	NULL

2 rows in set (0.00 sec)

与 `server_cost` 相比，`engine_cost` 多了两个列：

engine_name 列

指成本常数适用的存储引擎名称。如果该值为 **default**，意味着对应的成本常数适用于所有的存储引擎。

device_type 列

指存储引擎使用的设备类型，这主要是为了区分常规的机械硬盘和固态硬盘，不过在 **MySQL 5.7.X** 这个版本中并没有对机械硬盘的成本和固态硬盘的成本作区分，所以该值默认是 **0**。

我们从 `engine_cost` 表中的内容可以看出来，目前支持的存储引擎成本常数只有两个：

io_block_read_cost 默认值 **1.0** 从磁盘上读取一个块对应的成本。请注意我使用的是块，而不是页这个词。对于 **InnoDB** 存储引擎来说，一个页就是一个块，不过对于 **MyISAM** 存储引擎来说，默认是以 **4096** 字节作为一个块的。增大这个值会加重 **I/O** 成本，可能让优化器更倾向于选择使用索引执行查询而不是执行全表扫描。

memory_block_read_cost 默认值 **1.0** 与上一个参数类似，只不过衡量的是从内存中读取一个块对应的成本。

怎么从内存中和从磁盘上读取一个块的默认成本是一样的？这主要是因为 **MySQL** 目前的实现中，并不能准确预测某个查询需要访问的块中有哪些块已

经加载到内存中，有哪些块还停留在磁盘上，所以 MySQL 简单的认为不管这个块有没有加载到内存中，使用的成本都是 1.0。

与更新 `server_cost` 表中的记录一样，我们也可以通过更新 `engine_cost` 表中的记录来更改关于存储引擎的成本常数，做法一样。

5.4. InnoDB 中的统计数据

我们前边唠叨查询成本的时候经常用到一些统计数据，比如通过 `SHOW TABLE STATUS` 可以看到关于表的统计数据，通过 `SHOW INDEX` 可以看到关于索引的统计数据，那么这些统计数据是怎么来的呢？它们是以什么方式收集的呢？

5.4.1. 统计数据存储方式

InnoDB 提供了两种存储统计数据的方式：

永久性的统计数据，这种统计数据存储在磁盘上，也就是服务器重启之后这些统计数据还在。

非永久性的统计数据，这种统计数据存储在内存中，当服务器关闭时这些统计数据就都被清除了，等到服务器重启之后，在某些适当的场景下才会重新收集这些统计数据。

MySQL 给我们提供了系统变量 `innodb_stats_persistent` 来控制到底采用哪种方式去存储统计数据。在 MySQL 5.6.6 之前，`innodb_stats_persistent` 的值默认是 OFF，也就是说 InnoDB 的统计数据默认是存储到内存的，之后的版本中 `innodb_stats_persistent` 的值默认是 ON，也就是统计数据默认被存储到磁盘中。

SHOW VARIABLES LIKE 'innodb_stats_persistent';

```
mysql> show variables like 'innodb_stats_persistent';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_stats_persistent | ON    |
+-----+-----+
1 row in set (0.01 sec)
```

不过最近的 MySQL 版本都基本不用基于内存的非永久性统计数据了，所以也就不深入研究。

不过 InnoDB 默认是以表为单位来收集和存储统计数据的，也就是说我们可以把某些表的统计数据（以及该表的索引统计数据）存储在磁盘上，把另一些表的统计数据存储在内存中。怎么做到的呢？我们可以在创建和修改表的时候通过指定 `STATS_PERSISTENT` 属性来指明该表的统计数据存储方式：

CREATE TABLE 表名 (...) Engine=InnoDB, STATS_PERSISTENT = (1|0);

ALTER TABLE 表名 Engine=InnoDB, STATS_PERSISTENT = (1|0);

当 `STATS_PERSISTENT=1` 时，表明我们想把该表的统计数据永久的存储到磁盘上，当 `STATS_PERSISTENT=0` 时，表明我们想把该表的统计数据临时的存储到内存中。如果我们在创建表时未指定 `STATS_PERSISTENT` 属性，那默认采用系统变量 `innodb_stats_persistent` 的值作为该属性的值。

5.4.2. 基于磁盘的永久性统计数据

当我们选择把某个表以及该表索引的统计数据存放到磁盘上时，实际上是把这些统计数据存储到了两个表里：

```
SHOW TABLES FROM mysql LIKE 'innodb%';
```

```
mysql> SHOW TABLES FROM mysql LIKE 'innodb%';
+-----+
| Tables_in_mysql (innodb%) |
+-----+
| innodb_index_stats         |
| innodb_table_stats         |
+-----+
2 rows in set (0.00 sec)
```

可以看到，这两个表都位于 mysql 系统数据库下边，其中：

innodb_table_stats 存储了关于表的统计数据，每一条记录对应着一个表的统计数据。

innodb_index_stats 存储了关于索引的统计数据，每一条记录对应着一个索引的一个统计项的统计数据。

5.4.2.1. innodb_table_stats

直接看一下这个 innodb_table_stats 表中的各个列都是干嘛的：

```
mysql> desc mysql.innodb_table_stats;
+-----+-----+
| Field                | Type                |
+-----+-----+
| database_name         | varchar(64)         |
| table_name            | varchar(199)        |
| last_update           | timestamp            |
| n_rows                | bigint(20)          |
| clustered_index_size  | bigint(20)          |
| sum_of_other_index_sizes | bigint(20)          |
+-----+-----+
6 rows in set (0.00 sec)
```

database_name 数据库名

table_name 表名

last_update 本条记录最后更新时间

n_rows 表中记录的条数

clustered_index_size 表的聚簇索引占用的页面数量

sum_of_other_index_sizes 表的其他索引占用的页面数量

我们直接看一下这个表里的内容：

```
SELECT * FROM mysql.innodb_table_stats;
```

```
mysql> SELECT * FROM mysql.innodb_table_stats;
```

database name	table name	last update	n_rows	clustered_index_size	sum_of_other_index_sizes
mysql	gtid_executed	2021-03-19 16:19:15	0	1	0
mysqladv	e1	2021-03-31 16:34:58	3	1	0
mysqladv	e2	2021-03-31 16:35:08	3	1	0
mysqladv	order_exp	2021-03-31 15:31:28	10248	97	81
mysqladv	s1	2021-03-26 21:42:50	10573	161	50
mysqladv	s2	2021-03-26 18:11:20	10315	97	69
mysqladv	score	2021-03-31 16:50:57	4	1	0
mysqladv	student	2021-03-31 16:48:12	4	1	0
mysqladv	teacher	2021-03-30 20:11:12	5	1	1
mysqladv	test	2021-03-22 14:31:58	1	1	0
mysqladv	undo_test	2021-03-29 21:53:16	0	1	1
sys	sys_config	2021-03-19 16:19:16	6	1	0

12 rows in set (0.00 sec)

几个重要统计信息项的值如下：

`n_rows` 的值是 10350，表明 `order_exp` 表中大约有 10350 条记录，注意这个数据是估计值。

`clustered_index_size` 的值是 97，表明 `order_exp` 表的聚簇索引占用 97 个页面，这个值也是也是一个估计值。

`sum_of_other_index_sizes` 的值是 81，表明 `order_exp` 表的其他索引一共占用 81 个页面，这个值也是也是一个估计值。

n_rows 统计项的收集

InnoDB 统计一个表中有多少行记录是这样的：

按照一定算法（并不是纯粹随机的）选取几个叶子节点页面，计算每个页面中主键值记录数量，然后计算平均一个页面中主键值的记录数量乘以全部叶子节点的数量就算是该表的 `n_rows` 值。

可以看出来这个 `n_rows` 值精确与否取决于统计时采样的页面数量，MySQL 用名为 `innodb_stats_persistent_sample_pages` 的系统变量来控制使用永久性的统计数据时，计算统计数据时采样的页面数量。该值设置的越大，统计出的 `n_rows` 值越精确，但是统计耗时也就最久；该值设置的越小，统计出的 `n_rows` 值越不精确，但是统计耗时特别少。所以在实际使用是需要我们去权衡利弊，该系统变量的默认值是 20。

InnoDB 默认是以表为单位来收集和存储统计数据的，我们也可以单独设置某个表的采样页面的数量，设置方式就是在创建或修改表的时候通过指定 `STATS_SAMPLE_PAGES` 属性来指明该表的统计数据存储方式：

`CREATE TABLE 表名 (...) Engine=InnoDB, STATS_SAMPLE_PAGES = 具体的采样页面数量;`

`ALTER TABLE 表名 Engine=InnoDB, STATS_SAMPLE_PAGES = 具体的采样页面数量;`

如果我们在创建表的语句中并没有指定 `STATS_SAMPLE_PAGES` 属性的话，将默认使用系统变量 `innodb_stats_persistent_sample_pages` 的值作为该属性的值。

`clustered_index_size` 和 `sum_of_other_index_sizes` 统计项的收集牵涉到很具体的 InnoDB 表空间的知识 and 存储页面数据的细节，我们就不深入讲解了。

5.4.2.2. innodb_index_stats

直接看一下这个 `innodb_index_stats` 表中的各个列都是干嘛的：

desc mysql.innodb_index_stats;

```
mysql> desc mysql.innodb_index_stats;
```

Field	Type	Null	Key	Default	Extra
database_name	varchar(64)	NO	PRI	NULL	
table_name	varchar(199)	NO	PRI	NULL	
index_name	varchar(64)	NO	PRI	NULL	
last_update	timestamp	NO		CURRENT_TIMESTAMP	on update CURRENT_TIMESTAMP
stat_name	varchar(64)	NO	PRI	NULL	
stat_value	bigint(20) unsigned	NO		NULL	
sample_size	bigint(20) unsigned	YES		NULL	
stat_description	varchar(1024)	NO		NULL	

8 rows in set (0.00 sec)

字段名 描述

database_name 数据库名

table_name 表名

index_name 索引名

last_update 本条记录最后更新时间

stat_name 统计项的名称

stat_value 对应的统计项的值

sample_size 为生成统计数据而采样的页面数量

stat_description 对应的统计项的描述

innodb_index_stats 表的每条记录代表着一个索引的一个统计项。可能这会大家有些懵逼这个统计项到底指什么，别着急，我们直接看一下关于 order_exp 表的索引统计数据都有些什么：

```
mysql> SELECT * FROM mysql.innodb_index_stats WHERE table_name = 'order_exp';
```

```
mysql> SELECT * FROM mysql.innodb_index_stats WHERE table_name = 'order_exp';
```

database_name	table_name	index_name	last_update	stat_name	stat_value	sample_size	stat_description
mysqladv	order_exp	PRIMARY	2021-03-25 23:32:02	n_diff_pfx01	9460	20	id
mysqladv	order_exp	PRIMARY	2021-03-25 23:32:02	n_leaf_pages	79	NULL	Number of leaf pages in the index
mysqladv	order_exp	PRIMARY	2021-03-25 23:32:02	size	97	NULL	Number of pages in the index
mysqladv	order_exp	idx_expire_time	2021-03-31 15:31:28	n_diff_pfx01	9802	13	expire_time
mysqladv	order_exp	idx_expire_time	2021-03-31 15:31:28	n_diff_pfx02	10562	13	expire_time,id
mysqladv	order_exp	idx_expire_time	2021-03-31 15:31:28	n_leaf_pages	13	NULL	Number of leaf pages in the index
mysqladv	order_exp	idx_expire_time	2021-03-31 15:31:28	size	14	NULL	Number of pages in the index
mysqladv	order_exp	idx_order_no	2021-03-25 23:32:02	n_diff_pfx01	9432	24	order_no
mysqladv	order_exp	idx_order_no	2021-03-25 23:32:02	n_diff_pfx02	9774	24	order_no,id
mysqladv	order_exp	idx_order_no	2021-03-25 23:32:02	n_leaf_pages	24	NULL	Number of leaf pages in the index
mysqladv	order_exp	idx_order_no	2021-03-25 23:32:02	size	25	NULL	Number of pages in the index
mysqladv	order_exp	u_idx_day_status	2021-03-25 23:32:02	n_diff_pfx01	915	27	insert_time
mysqladv	order_exp	u_idx_day_status	2021-03-25 23:32:02	n_diff_pfx02	915	27	insert_time,order_status
mysqladv	order_exp	u_idx_day_status	2021-03-25 23:32:02	n_diff_pfx03	9774	27	insert_time,order_status,expire_time
mysqladv	order_exp	u_idx_day_status	2021-03-25 23:32:02	n_leaf_pages	27	NULL	Number of leaf pages in the index
mysqladv	order_exp	u_idx_day_status	2021-03-25 23:32:02	size	28	NULL	Number of pages in the index

16 rows in set (0.00 sec)

先查看 index_name 列，这个列说明该记录是哪个索引的统计信息，从结果中我们可以看出来，PRIMARY 索引（也就是主键）占了 3 条记录，idx_expire_time 索引占了 6 条记录。

针对 index_name 列相同的记录，stat_name 表示针对该索引的统计项名称，stat_value 展示的是该索引在该统计项上的值，stat_description 指的是来描述该统计项的含义的。我们来具体看一下一个索引都有哪些统计项：

n_leaf_pages：表示该索引的叶子节点占用多少页面。

size：表示该索引共占用多少页面。

n_diff_pfxNN：表示对应的索引列不重复的值有多少。其中的 NN 长得有点儿怪呀，啥意思呢？

其实 NN 可以被替换为 01、02、03... 这样的数字。比如对于 u_idx_day_status 来说：

n_diff_pfx01 表示的是统计 insert_time 这单单一个列不重复的值有多少。

n_diff_pfx02 表示的是统计 insert_time,order_status 这两个列组合起来不重复的值有多少。

n_diff_pfx03 表示的是统计 insert_time,order_status,expire_time 这三个列组合起来不重复的值有多少。

n_diff_pfx04 表示的是统计 key_pare1、key_pare2、expire_time、id 这四个列组合起来不重复的值有多少。

对于普通的二级索引，并不能保证它的索引列值是唯一的，比如对于 idx_order_no 来说，key1 列就可能有很多值重复的记录。此时只有在索引列上加上主键值才可以区分两条索引列值都一样的二级索引记录。

对于主键和唯一二级索引则没有这个问题，它们本身就可以保证索引列值的不重复，所以也不需要再统计一遍在索引列后加上主键值的不重复值有多少。比如 u_idx_day_statu 和 idx_order_no。

在计算某些索引列中包含多少不重复值时，需要对一些叶子节点页面进行采样，sample_size 列就表明了采样的页面数量是多少。

对于有多个列的联合索引来说，采样的页面数量是：
innodb_stats_persistent_sample_pages × 索引列的个数。

```
mysql> show variables like 'innodb_stats_transient_sample_pages';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_stats_transient_sample_pages | 8      |
+-----+-----+
1 row in set (0.00 sec)
```

当需要采样的页面数量大于该索引的叶子节点数量的话，就直接采用全表扫描来统计索引列的不重复值数量了。所以大家可以在查询结果中看到不同索引对应的 size 列的值可能是不同的。

5.4.2.3. 定期更新统计数据

随着我们不断的对表进行增删改操作，表中的数据也一直在变化，innodb_table_stats 和 innodb_index_stats 表里的统计数据也在变化。MySQL 提供了如下两种更新统计数据的方式：

开启 innodb_stats_auto_recalc。

系统变量 innodb_stats_auto_recalc 决定着服务器是否自动重新计算统计数据，它的默认值是 ON，也就是该功能默认是开启的。每个表都维护了一个变量，该变量记录着对该表进行增删改的记录条数，如果发生变动的记录数量超过了表大小的 10%，并且自动重新计算统计数据的功能是打开的，那么服务器会重新进行一次统计数据的计算，并且更新 innodb_table_stats 和 innodb_index_stats 表。不过自动重新计算统计数据的过程是异步发生的，也就是即使表中变动的记录数

超过了 10%，自动重新计算统计数据也不会立即发生，可能会延迟几秒才会进行计算。

再一次强调，InnoDB 默认是以表为单位来收集和存储统计数据的，我们也可以单独为某个表设置是否自动重新计算统计数据的属性，设置方式就是在创建或修改表的时候通过指定 `STATS_AUTO_RECALC` 属性来指明该表的统计数据存储方式：

```
CREATE TABLE 表名 (...) Engine=InnoDB, STATS_AUTO_RECALC = (1|0);
```

```
ALTER TABLE 表名 Engine=InnoDB, STATS_AUTO_RECALC = (1|0);
```

当 `STATS_AUTO_RECALC=1` 时，表明我们想让该表自动重新计算统计数据，当 `STATS_AUTO_RECALC=0` 时，表明不想让该表自动重新计算统计数据。如果我们在创建表时未指定 `STATS_AUTO_RECALC` 属性，那默认采用系统变量 `innodb_stats_auto_recalc` 的值作为该属性的值。

手动调用 `ANALYZE TABLE` 语句来更新统计信息

如果 `innodb_stats_auto_recalc` 系统变量的值为 `OFF` 的话，我们也可以手动调用 `ANALYZE TABLE` 语句来重新计算统计数据，比如我们可以这样更新关于 `order_exp` 表的统计数据：

```
mysql> ANALYZE TABLE order_exp;
```

```
mysql> SELECT * FROM mysql.innodb_index_stats WHERE table_name = 'order_exp';
```

database_name	table_name	index_name	last_update	stat_name	stat_value	sample_size	stat_description
mysqladv	order_exp	PRIMARY	2021-03-25 23:32:02	n_diff_pfx01	9460	20	id
mysqladv	order_exp	PRIMARY	2021-03-25 23:32:02	n_leaf_pages	79	NULL	Number of leaf pages in the index
mysqladv	order_exp	PRIMARY	2021-03-25 23:32:02	size	97	NULL	Number of pages in the index
mysqladv	order_exp	idx_expire_time	2021-03-31 15:31:28	n_diff_pfx01	9802	13	expire_time
mysqladv	order_exp	idx_expire_time	2021-03-31 15:31:28	n_diff_pfx02	10562	13	expire_time,id
mysqladv	order_exp	idx_expire_time	2021-03-31 15:31:28	n_leaf_pages	13	NULL	Number of leaf pages in the index
mysqladv	order_exp	idx_expire_time	2021-03-31 15:31:28	size	14	NULL	Number of pages in the index
mysqladv	order_exp	idx_order_no	2021-03-25 23:32:02	n_diff_pfx01	9432	24	order_no
mysqladv	order_exp	idx_order_no	2021-03-25 23:32:02	n_diff_pfx02	9774	24	order_no,id
mysqladv	order_exp	idx_order_no	2021-03-25 23:32:02	n_leaf_pages	24	NULL	Number of leaf pages in the index
mysqladv	order_exp	idx_order_no	2021-03-25 23:32:02	size	25	NULL	Number of pages in the index
mysqladv	order_exp	u_idx_day_status	2021-03-25 23:32:02	n_diff_pfx01	915	27	insert_time
mysqladv	order_exp	u_idx_day_status	2021-03-25 23:32:02	n_diff_pfx02	915	27	insert_time,order_status
mysqladv	order_exp	u_idx_day_status	2021-03-25 23:32:02	n_diff_pfx03	9774	27	insert_time,order_status,expire_time
mysqladv	order_exp	u_idx_day_status	2021-03-25 23:32:02	n_leaf_pages	27	NULL	Number of leaf pages in the index
mysqladv	order_exp	u_idx_day_status	2021-03-25 23:32:02	size	28	NULL	Number of pages in the index

16 rows in set (0.00 sec)

```
mysql> ANALYZE TABLE order_exp;
```

Table	Op	Msg_type	Msg_text
mysqladv.order_exp	analyze	status	OK

1 row in set (0.02 sec)

```
mysql> SELECT * FROM mysql.innodb_index_stats WHERE table_name = 'order_exp';
```

database_name	table_name	index_name	last_update	stat_name	stat_value	sample_size	stat_description
mysqladv	order_exp	PRIMARY	2021-04-01 00:05:51	n_diff_pfx01	10345	20	id
mysqladv	order_exp	PRIMARY	2021-04-01 00:05:51	n_leaf_pages	86	NULL	Number of leaf pages in the index
mysqladv	order_exp	PRIMARY	2021-04-01 00:05:51	size	97	NULL	Number of pages in the index
mysqladv	order_exp	idx_expire_time	2021-04-01 00:05:51	n_diff_pfx01	9802	13	expire_time
mysqladv	order_exp	idx_expire_time	2021-04-01 00:05:51	n_diff_pfx02	10562	13	expire_time,id

`ANALYZE TABLE` 语句会立即重新计算统计数据，也就是这个过程是同步的，在表中索引多或者采样页面特别多时这个过程可能会特别慢最好在业务不是很繁忙的时候再运行。

5.4.2.4. 手动更新 `innodb_table_stats` 和 `innodb_index_stats` 表

其实 `innodb_table_stats` 和 `innodb_index_stats` 表就相当于一个普通的表一样，我们能对它们做增删改查操作。这也就意味着我们可以手动更新某个表或者索引的统计数据。比如说我们想把 `order_exp` 表关于行数的统计数据更改一下可以这么做：

步骤一：更新 `innodb_table_stats` 表。

步骤二：让 MySQL 查询优化器重新加载我们更改过的数据。

更新完 `innodb_table_stats` 只是单纯的修改了一个表的数据，需要让 MySQL 查询优化器重新加载我们更改过的数据，运行下边的命令就可以了：

```
FLUSH TABLE order_exp;
```

5.5. MySQL 的查询重写规则

对于一些执行起来十分耗费性能的语句，MySQL 还是依据一些规则，竭尽全力的把这个很糟糕的语句转换成某种可以比较高效执行的形式，这个过程也可以被称作查询重写。

5.5.1. 条件化简

我们编写的查询语句的搜索条件本质上是一个表达式，这些表达式可能比较繁杂，或者不能高效的执行，MySQL 的查询优化器会为我们简化这些表达式。

5.5.1.1. 移除不必要的括号

有时候表达式里有许多无用的括号，比如这样：

```
((a = 5 AND b = c) OR ((a > c) AND (c < 5)))
```

看着就很烦，优化器会把那些用不到的括号给干掉，就是这样：

```
(a = 5 and b = c) OR (a > c AND c < 5)
```

5.5.1.2. 常量传递 (*constant_propagation*)

有时候某个表达式是某个列和某个常量做等值匹配，比如这样：

```
a = 5
```

当这个表达式和其他涉及列 `a` 的表达式使用 `AND` 连接起来时，可以将其他表达式中的 `a` 的值替换为 `5`，比如这样：

```
a = 5 AND b > a
```

就可以被转换为：

```
a = 5 AND b > 5
```

等值传递 (*equality_propagation*)

有时候多个列之间存在等值匹配的关系，比如这样：

```
a = b and b = c and c = 5
```

这个表达式可以被简化为：

```
a = 5 and b = 5 and c = 5
```

5.5.1.3. 移除没用的条件 (*trivial_condition_removal*)

对于一些明显永远为 `TRUE` 或者 `FALSE` 的表达式，优化器会移除掉它们，比如这个表达式：

`(a < 1 and b = b) OR (a = 6 OR 5 != 5)`

很明显，`b = b` 这个表达式永远为 `TRUE`，`5 != 5` 这个表达式永远为 `FALSE`，所以简化后的表达式就是这样的：

`(a < 1 and TRUE) OR (a = 6 OR FALSE)`

可以继续被简化为

`a < 1 OR a = 6`

5.5.1.4. 表达式计算

在查询开始执行之前，如果表达式中只包含常量的话，它的值会被先计算出来，比如这个：

`a = 5 + 1`

因为 `5 + 1` 这个表达式只包含常量，所以就会被化简成：

`a = 6`

但是这里需要注意的是，如果某个列并不是以单独的形式作为表达式的操作数时，比如出现在函数中，出现在某个更复杂表达式中，就像这样：

`ABS(a) > 5`

或者：

`-a < -8`

优化器是不会尝试对这些表达式进行化简的。我们前边说过只有搜索条件中索引列和常数使用某些运算符连接起来才可能使用到索引，所以如果可以的话，最好让索引列以单独的形式出现在表达式中。

5.5.1.5. 常量表检测

MySQL 觉得下边这种查询运行的特别快：

使用主键等值匹配或者唯一二级索引列等值匹配作为搜索条件来查询某个表。

MySQL 觉得这两种查询花费的时间特别少，少到可以忽略，所以也把通过这两种方式查询的表称之为常量表（英文名：**constant tables**）。优化器在分析一个查询语句时，先首先执行常量表查询，然后把查询中涉及到该表的条件全部替换成常数，最后再分析其余表的查询成本，比方说这个查询语句：

```
SELECT * FROM table1 INNER JOIN table2
```

```
ON table1.column1 = table2.column2
```

```
WHERE table1.primary_key = 1;
```

很明显，这个查询可以使用主键和常量值的等值匹配来查询 `table1` 表，也就是在这个查询中 `table1` 表相当于常量表，在分析对 `table2` 表的查询成本之前，就会执行对 `table1` 表的查询，并把查询中涉及 `table1` 表的条件都替换掉，也就是上边的语句会被转换成这样：

SELECT table1 表记录的各个字段的常量值, table2.* FROM table1 INNER JOIN table2 ON table1 表 column1 列的常量值 = table2.column2;

5.5.2. 外连接消除

我们前边说过，内连接的驱动表和被驱动表的位置可以相互转换，而左（外）连接和右（外）连接的驱动表和被驱动表是固定的。这就导致内连接可能通过优化表的连接顺序来降低整体的查询成本，而外连接却无法优化表的连接顺序。

我们之前说过，外连接和内连接的本质区别就是：对于外连接的驱动表的记录来说，如果无法在被驱动表中找到匹配 ON 子句中的过滤条件的记录，那么该记录仍然会被加入到结果集中，对应的被驱动表记录的各个字段使用 NULL 值填充；而内连接的驱动表的记录如果无法在被驱动表中找到匹配 ON 子句中的过滤条件的记录，那么该记录会被舍弃。查询效果就是这样：

SELECT * FROM e1 INNER JOIN e2 ON e1.m1 = e2.m2;

```
mysql> SELECT * FROM e1 INNER JOIN e2 ON e1.m1 = e2.m2;
+-----+-----+-----+-----+
| m1    | n1    | m2    | n2    |
+-----+-----+-----+-----+
|      2 | b     |      2 | b     |
|      3 | c     |      3 | c     |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

SELECT * FROM e1 LEFT JOIN e2 ON e1.m1 = e2.m2;

```
mysql> SELECT * FROM e1 LEFT JOIN e2 ON e1.m1 = e2.m2;
+-----+-----+-----+-----+
| m1    | n1    | m2    | n2    |
+-----+-----+-----+-----+
|      2 | b     |      2 | b     |
|      3 | c     |      3 | c     |
|      1 | a     | NULL   | NULL   |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

对于上边例子中的（左）外连接来说，由于驱动表 e1 中 m1=1, n1='a'的记录无法在被驱动表 e2 中找到符合 ON 子句条件 e1.m1 = e2.m2 的记录，所以就直接把这条记录加入到结果集，对应的 e2 表的 m2 和 n2 列的值都设置为 NULL。

因为凡是不符合 WHERE 子句中条件的记录都不会参与连接。只要我们在搜索条件中指定关于被驱动表相关列的值不为 NULL，那么外连接中在被驱动表中找不到符合 ON 子句条件的驱动表记录也就被排除出最后的结果集了，也就是说：在这种情况下：外连接和内连接也就没有什么区别了！比方说这个查询：

mysql> SELECT * FROM e1 LEFT JOIN e2 ON e1.m1 = e2.m2 WHERE e2.n2 IS NOT NULL;


```
mysql> SELECT * FROM e1 LEFT JOIN e2 ON e1.m1 = e2.m2 WHERE e2.n2 IS NOT NULL;
```

m1	n1	m2	n2
2	b	2	b
3	c	3	c

```
2 rows in set (0.00 sec)
```

由于指定了被驱动表 `e2` 的 `n2` 列不允许为 `NULL`，所以上边的 `e1` 和 `e2` 表的左（外）连接查询和内连接查询是一样的。当然，我们也可以不用显式的指定被驱动表的某个列 `IS NOT NULL`，只要隐含的有这个意思就行了，比方说这样：

```
mysql> SELECT * FROM e1 LEFT JOIN e2 ON e1.m1 = e2.m2 WHERE e2.m2 = 2;
```

在这个例子中，我们在 `WHERE` 子句中指定了被驱动表 `e2` 的 `m2` 列等于 2，也就相当于间接的指定了 `m2` 列不为 `NULL` 值，所以上边的这个左（外）连接查询其实和下边这个内连接查询是等价的：

```
mysql> SELECT * FROM e1 INNER JOIN e2 ON e1.m1 = e2.m2 WHERE e2.m2 = 2;
```

我们把这种在外连接查询中，指定的 `WHERE` 子句中包含被驱动表中的列不为 `NULL` 值的条件称之为空值拒绝（英文名：`reject-NULL`）。在被驱动表的 `WHERE` 子句符合空值拒绝的条件后，外连接和内连接可以相互转换。这种转换带来的好处就是查询优化器可以通过评估表的不同连接顺序的成本，选出成本最低的那种连接顺序来执行查询。

5.5.3. 子查询优化

5.5.3.1. 子查询语法

在一个查询语句 `A` 里的某个位置也可以有另一个查询语句 `B`，这个出现在 `A` 语句的某个位置中的查询 `B` 就被称为子查询，`A` 也被称之为外层查询。子查询可以在一个外层查询的各种位置出现，比如：

SELECT 子句中

也就是我们平时说的查询列表中，比如这样：

```
mysql> SELECT (SELECT m1 FROM e1 LIMIT 1);
```

其中的 `(SELECT m1 FROM e1 LIMIT 1)` 就是子查询。

FROM 子句中

比如：

```
SELECT m, n FROM (SELECT m2 + 1 AS m, n2 AS n FROM e2 WHERE m2 > 2) AS t;
```

这个例子中的子查询是：`(SELECT m2 + 1 AS m, n2 AS n FROM e2 WHERE m2 > 2)`，这里可以把子查询的查询结果当作是一个表，子查询后边的 `AS t` 表明这个子查询的结果就相当于一个名称为 `t` 的表，这个名叫 `t` 的表的列就是子查询结果中的列，比如例子中表 `t` 就有两个列：`m` 列和 `n` 列。这个放在 `FROM` 子句中的子查询本质上相当于一个表，但又和我们平常使用的表有点儿不一样，MySQL 把这种由子查询结果集组成的表称之为派生表。

WHERE 或 ON 子句中

把子查询放在外层查询的 WHERE 子句或者 ON 子句中可能是我们最常用的一种使用子查询的方式了，比如这样：

```
mysql> SELECT * FROM e1 WHERE m1 IN (SELECT m2 FROM e2);
```

这个查询表明我们想要将(SELECT m2 FROM e2)这个子查询的结果作为外层查询的 IN 语句参数，整个查询语句的意思就是我们想找 e1 表中的某些记录，这些记录的 m1 列的值能在 e2 表的 m2 列找到匹配的值。

ORDER BY 子句、GROUP BY 子句中

虽然语法支持，但没啥意义。

按返回的结果集区分子查询

因为子查询本身也算是一个查询，所以可以按照它们返回的不同结果集类型而把这些子查询分为不同的类型：

标量子查询

那些只返回一个单一值的子查询称之为标量子查询，比如这样：

```
SELECT (SELECT m1 FROM e1 LIMIT 1);
```

或者这样：

```
SELECT * FROM e1 WHERE m1 = (SELECT MIN(m2) FROM e2);
```

```
SELECT * FROM e1 WHERE m1 < (SELECT MIN(m2) FROM e2);
```

这两个查询语句中的子查询都返回一个单一的值，也就是一个标量。这些标量子查询可以作为一个单一值或者表达式的一部分出现在查询语句的各个地方。

行子查询

顾名思义，就是返回一条记录的子查询，不过这条记录需要包含多个列（只包含一个列就成了标量子查询了）。比如这样：

```
SELECT * FROM e1 WHERE (m1, n1) = (SELECT m2, n2 FROM e2 LIMIT 1);
```

其中的(SELECT m2, n2 FROM e2 LIMIT 1)就是一个行子查询，整条语句的含义就是要从 e1 表中找一些记录，这些记录的 m1 和 n1 列分别等于子查询结果中的 m2 和 n2 列。

列子查询

列子查询自然就是查询出一个列的数据喽，不过这个列的数据需要包含多条记录（只包含一条记录就成了标量子查询了）。比如这样：

```
SELECT * FROM e1 WHERE m1 IN (SELECT m2 FROM e2);
```

其中的(SELECT m2 FROM e2)就是一个列子查询，表明查询出 e2 表的 m2 列的值作为外层查询 IN 语句的参数。

表子查询

顾名思义，就是子查询的结果既包含很多条记录，又包含很多个列，比如这样：

```
SELECT * FROM e1 WHERE (m1, n1) IN (SELECT m2, n2 FROM e2);
```

其中的(SELECT m2, n2 FROM e2)就是一个表子查询，这里需要和行子查询对比一下，行子查询中我们用了 LIMIT 1 来保证子查询的结果只有一条记录，表子查询中不需要这个限制。

按与外层查询关系来区分子查询

不相关子查询

如果子查询可以单独运行出结果，而不依赖于外层查询的值，我们就可以把这个子查询称之为不相关子查询。我们前边介绍的那些子查询全部都可以看作不相关子查询。

相关子查询

如果子查询的执行需要依赖于外层查询的值，我们就可以把这个子查询称之为相关子查询。比如：

```
SELECT * FROM e1 WHERE m1 IN (SELECT m2 FROM e2 WHERE n1 = n2);
```

例子中的子查询是(SELECT m2 FROM e2 WHERE n1 = n2)，可是这个查询中有一个搜索条件是 n1 = n2，别忘了 n1 是表 e1 的列，也就是外层查询的列，也就是说子查询的执行需要依赖于外层查询的值，所以这个子查询就是一个相关子查询。

[NOT] IN/ANY/SOME/ALL 子查询

对于列子查询和表子查询来说，它们的结果集中包含很多条记录，这些记录相当于是一个集合，所以就不能单纯的和另外一个操作数使用操作符来组成布尔表达式了，MySQL 通过下面的语法来支持某个操作数和一个集合组成一个布尔表达式：

IN 或者 NOT IN

具体的语法形式如下：

操作数 [NOT] IN (子查询)

这个布尔表达式的意思是用来判断某个操作数在不在由子查询结果集组成的集合中，比如下边的查询的意思是找出 e1 表中的某些记录，这些记录存在于子查询的结果集中：

```
SELECT * FROM e1 WHERE (m1, n1) IN (SELECT m2, n2 FROM e2);
```

ANY/SOME (ANY 和 SOME 是同义词)

具体的语法形式如下：

操作数 比较符 ANY/SOME(子查询)

这个布尔表达式的意思是只要子查询结果集中存在某个值和给定的操作数做比较操作，比较结果为 TRUE，那么整个表达式的结果就为 TRUE，否则整个表达式的结果就为 FALSE。比方说下边这个查询：

```
SELECT * FROM e1 WHERE m1 > ANY(SELECT m2 FROM e2);
```

这个查询的意思就是对于 e1 表的某条记录的 m1 列的值来说，如果子查询(SELECT m2 FROM e2)的结果集中存在一个小于 m1 列的值，那么整个布尔表达式

的值就是 TRUE，否则为 FALSE，也就是说只要 m1 列的值大于子查询结果集中最小的值，整个表达式的结果就是 TRUE，所以上边的查询本质上等价于这个查询：

```
SELECT * FROM e1 WHERE m1 > (SELECT MIN(m2) FROM e2);
```

另外，=ANY 相当于判断子查询结果集中是否存在某个值和给定的操作数相等，它的含义和 IN 是相同的。

ALL

具体的语法形式如下：

操作数 比较操作 ALL(子查询)

这个布尔表达式的意思是子查询结果集中所有的值和给定的操作数做比较操作比较结果为 TRUE，那么整个表达式的结果就为 TRUE，否则整个表达式的结果就为 FALSE。比方说下边这个查询：

```
SELECT * FROM e1 WHERE m1 > ALL(SELECT m2 FROM e2);
```

这个查询的意思就是对于 e1 表的某条记录的 m1 列的值来说，如果子查询(SELECT m2 FROM e2)的结果集中的所有值都小于 m1 列的值，那么整个布尔表达式的值就是 TRUE，否则为 FALSE，也就是说只要 m1 列的值大于子查询结果集中最大的值，整个表达式的结果就是 TRUE，所以上边的查询本质上等价于这个查询：

```
SELECT * FROM e1 WHERE m1 > (SELECT MAX(m2) FROM e2);
```

EXISTS 子查询

有的时候我们仅仅需要判断子查询的结果集中是否有记录，而不在乎它的记录具体是个啥，可以使用把 EXISTS 或者 NOT EXISTS 放在子查询语句前边，就像这样：

```
SELECT * FROM e1 WHERE EXISTS (SELECT 1 FROM e2);
```

对于子查询(SELECT 1 FROM e2)来说，我们并不关心这个子查询最后到底查询出的结果是什么，所以查询列表里填*、某个列名，或者其他啥东西都无所谓，我们真正关心的是子查询的结果集中是否存在记录。也就是说只要(SELECT 1 FROM e2)这个查询中有记录，那么整个 EXISTS 表达式的结果就为 TRUE。

子查询语法注意事项

子查询必须用小括号扩起来。

在 SELECT 子句中的子查询必须是标量子查询，如果子查询结果集中有多个列或者多个行，都不允许放在 SELECT 子句中，在想要得到标量子查询或者行子查询，但又不能保证子查询的结果集只有一条记录时，应该使用 LIMIT 1 语句来限制记录数量。

对于[NOT] IN/ANY/SOME/ALL 子查询来说，子查询中不允许有 LIMIT 语句，而且这类子查询中 ORDER BY 子句、DISTINCT 语句、没有聚集函数以及 HAVING 子句的 GROUP BY 子句没有什么意义。因为子查询的结果其实就相当于一个集合，集合里的值排不排序等一点儿都不重要。

不允许在一条语句中增删改某个表的记录时同时还对该表进行子查询。

5.5.3.2. 子查询在 MySQL 中是怎么执行的

想象子查询的执行方式

想象中子查询的执行方式是这样的：

如果该子查询是不相关子查询，比如下边这个查询：

```
SELECT * FROM s1 WHERE order_note IN (SELECT order_note FROM s2);
```

先单独执行(SELECT order_note FROM s2)这个子查询。然后在将上一步子查询得到的结果当作外层查询的参数再执行外层查询 SELECT * FROM s1 WHERE order_note IN (...)。

如果该子查询是相关子查询，比如下边这个查询：

```
SELECT * FROM s1 WHERE order_note IN (SELECT order_note FROM s2 WHERE  
s1.order_no= s2.order_no);
```

这个查询中的子查询中出现了 s1.order_no= s2.order_no 这样的条件，意味着该子查询的执行依赖着外层查询的值，先从外层查询中获取一条记录，本例中也就是先从 s1 表中获取一条记录，然后执行子查询。

最后根据子查询的查询结果来检测外层查询 WHERE 子句的条件是否成立，如果成立，就把外层查询的那条记录加入到结果集，否则就丢弃。

再次执行第一步，获取第二条外层查询中的记录，依次类推。

但真的是这样吗？其实 MySQL 用了一系列的办法来优化子查询的执行，大部分情况下这些优化措施其实挺有效的，下边我们来看看各种不同类型的子查询具体是怎么执行的。

标量子查询、行子查询的执行方式

对于不相关标量子查询或者行子查询来说，它们的执行方式很简单，比方说下边这个查询语句：

```
SELECT * FROM s1 WHERE order_note = (SELECT order_note FROM s2 WHERE  
key3 = 'a' LIMIT 1);
```

它的执行方式和我们前面想象的一样：先单独执行(SELECT order_note FROM s2 WHERE key3 = 'a' LIMIT 1)这个子查询。然后在将上一步子查询得到的结果当作外层查询的参数再执行外层查询 SELECT * FROM s1 WHERE order_note= ...。

也就是说，对于包含不相关的标量子查询或者行子查询的查询语句来说，MySQL 会分别独立的执行外层查询和子查询，就当作两个单表查询就好了。

对于相关的标量子查询或者行子查询来说，比如下边这个查询：

```
SELECT * FROM s1 WHERE  
order_note = (SELECT order_note FROM s2 WHERE s1.order_no=  
s2.order_no LIMIT 1);
```

事情也和我们前面想象的一样，它的执行方式就是这样的：

先从外层查询中获取一条记录，本例中也就是先从 s1 表中获取一条记录。

然后从上一步骤中获取的那条记录中找出子查询中涉及到的值，本例中就是从 s1 表中获取的那条记录中找出 s1.order_no 列的值，然后执行子查询。

最后根据子查询的查询结果来检测外层查询 WHERE 子句的条件是否成立，如果成立，就把外层查询的那条记录加入到结果集，否则就丢弃。

再次执行第一步，获取第二条外层查询中的记录，依次类推。

也就是说对于两种使用标量子查询以及行子查询的场景中，MySQL 优化器的执行方式并没有什么新鲜的。

MySQL 对 IN 子查询的优化

物化表

对于不相关的 IN 子查询，比如这样：

```
SELECT * FROM s1 WHERE order_note IN (SELECT order_note FROM s2 WHERE order_no = 'a');
```

我们最开始的感觉就是这种不相关的 IN 子查询和不相关的标量子查询或者行子查询是一样一样的，都是把外层查询和子查询当作两个独立的单表查询来对待。但是 MySQL 为了优化 IN 子查询下了很大力气，所以整个执行过程并不像我们想象的那么简单。

对于不相关的 IN 子查询来说，如果子查询的结果集中的记录条数很少，那么把子查询和外层查询分别看成两个单独的单表查询效率很高，但是如果单独执行子查询后的结果集太多的话，就会导致这些问题：

- 1、结果集太多，可能内存中都放不下。

- 2、对于外层查询来说，如果子查询的结果集太多，那就意味着 IN 子句中的参数特别多，这就导致：无法有效的使用索引，只能对外层查询进行全表扫描。

在对外层查询执行全表扫描时，由于 IN 子句中的参数太多，这会导致检测一条记录是否符合和 IN 子句中的参数匹配花费的时间太长。

比如说 IN 子句中的参数只有两个：

```
SELECT * FROM tbl_name WHERE column IN (a, b);
```

这样相当于需要对 tbl_name 表中的每条记录判断一下它的 column 列是否符合 column = a OR column = b。在 IN 子句中的参数比较少时这并不是什么问题，如果 IN 子句中的参数比较多时，比如这样：

```
SELECT * FROM tbl_name WHERE column IN (a, b, c ..., ...);
```

那么这样每条记录需要判断一下它的 column 列是否符合 column = a OR column = b OR column = c OR ...，这样性能耗费可就多了。

MySQL 的改进是不直接将不相关子查询的结果集当作外层查询的参数，而是将该结果集写入一个临时表里。写入临时表的过程是这样的：

- 1、该临时表的列就是子查询结果集中的列。

- 2、写入临时表的记录会被去重，临时表也是个表，只要为表中记录的所有列建立主键或者唯一索引。

一般情况下子查询结果集不会大的离谱，所以会为它建立基于内存的使用 Memory 存储引擎的临时表，而且会为该表建立哈希索引。

如果子查询的结果集非常大，超过了系统变量 `tmp_table_size` 或者 `max_heap_table_size`，临时表会转而使用基于磁盘的存储引擎来保存结果集中的记录，索引类型也对应转变为 B+ 树索引。

MySQL 把这个将子查询结果集中的记录保存到临时表的过程称之为**物化**（英文名：Materialize）。为了方便起见，我们就把那个存储子查询结果集的临时表称之为物化表。正因为物化表中的记录都建立了索引（基于内存的物化表有哈希索引，基于磁盘的有 B+ 树索引），通过索引执行 IN 语句判断某个操作数在不在子查询结果集中变得非常快，从而提升了子查询语句的性能。

物化表转连接

事情到这就完了？我们还得重新审视一下最开始的那个查询语句：

```
SELECT * FROM s1 WHERE order_note IN (SELECT order_note FROM s2 WHERE order_no = 'a');
```

当我们把子查询进行物化之后，假设子查询物化表的名称为 `materialized_table`，该物化表存储的子查询结果集的列为 `m_val`，那么这个查询就相当于表 `s1` 和子查询物化表 `materialized_table` 进行内连接：

```
SELECT s1.* FROM s1 INNER JOIN materialized_table ON order_note = m_val;
```

转化成内连接之后就有意思了，查询优化器可以评估不同连接顺序需要的成本是多少，选取成本最低的那种查询方式执行查询。我们分析一下上述查询中使用外层查询的表 `s1` 和物化表 `materialized_table` 进行内连接的成本都是由哪几部分组成的：

1、如果使用 `s1` 表作为驱动表的话，总查询成本由下边几个部分组成：

物化子查询时需要的成本

扫描 `s1` 表时的成本

`s1` 表中的记录数量 × 通过 `m_val = xxx` 对 `materialized_table` 表进行单表访问的成本（我们前边说过物化表中的记录是不重复的，并且为物化表中的列建立了索引，所以这个步骤显然是非常快的）。

2、如果使用 `materialized_table` 表作为驱动表的话，总查询成本由下边几个部分组成：

物化子查询时需要的成本

扫描物化表时的成本

物化表中的记录数量 × 通过 `order_note = xxx` 对 `s1` 表进行单表访问的成本（如果 `order_note` 列上建立了索引，这个步骤还是非常快的）。

MySQL 查询优化器会通过运算来选择上述成本更低的方案来执行查询。

将子查询转换为 semi-join

虽然将子查询进行物化之后再执行查询都会有建立临时表的成本，但是不管怎么说，我们见识到了将子查询转换为连接的强大作用，MySQL 继续开脑洞：能

不能不进行物化操作直接把子查询转换为连接呢？让我们重新审视一下上边的查询语句：

```
SELECT * FROM s1 WHERE order_note IN (SELECT order_note FROM s2 WHERE order_no = 'a');
```

我们可以把这个查询理解成：对于 s1 表中的某条记录，如果我们能在 s2 表（准确的说是执行完 WHERE s2.order_no= 'a'之后的结果集）中找到一条或多条记录，这些记录的 order_note 的值等于 s1 表记录的 order_note 列的值，那么该条 s1 表的记录就会被加入到最终的结果集。这个过程其实和把 s1 和 s2 两个表连接起来的效果很像：

```
SELECT s1.* FROM s1 INNER JOIN s2
    ON s1.order_note = s2.order_note
    WHERE s2.order_no= 'a';
```

只不过我们不能保证对于 s1 表的某条记录来说，在 s2 表（准确的说是执行完 WHERE s2.order_no= 'a'之后的结果集）中有多少条记录满足 s1.order_no = s2.order_no 这个条件，不过我们可以分三种情况讨论：

情况一：对于 s1 表的某条记录来说，s2 表中没有任何记录满足 s1.order_note = s2.order_note 这个条件，那么该记录自然也不会加入到最后的结果集。

情况二：对于 s1 表的某条记录来说，s2 表中有且只有 1 条记录满足 s1.order_note = s2.order_note 这个条件，那么该记录会被加入最终的结果集。

情况三：对于 s1 表的某条记录来说，s2 表中至少有 2 条记录满足 s1.order_note = s2.order_note 这个条件，那么该记录会被多次加入最终的结果集。

对于 s1 表的某条记录来说，由于我们只关心 s2 表中是否存在记录满足 s1.order_no = s2.order_note 这个条件，而不关心具体有多少条记录与之匹配，又因为情况三的存在，我们上边所说的 IN 子查询和两表连接之间并不完全等价。但是将子查询转换为连接又真的可以充分发挥优化器的作用，所以 MySQL 在这里提出了一个新概念 --- **半连接**（英文名：semi-join）。

将 s1 表和 s2 表进行半连接的意思就是：对于 s1 表的某条记录来说，我们只关心在 s2 表中是否存在与之匹配的记录，而不关心具体有多少条记录与之匹配，最终的结果集中只保留 s1 表的记录。为了让大家有更直观的感受，我们假设 MySQL 内部是这么改写上边的子查询的：

```
SELECT s1.* FROM s1 SEMI JOIN s2
    ON s1.order_note = s2.order_note
    WHERE order_no= 'a';
```

注意：semi-join 只是在 MySQL 内部采用的一种执行子查询的方式，MySQL 并没有提供面向用户的 semi-join 语法。

概念是有了，怎么实现这种所谓的半连接呢？MySQL 准备了好几种办法。

Table pullout (子查询中的表上拉)

当子查询的查询列表处只有主键或者唯一索引列时，可以直接把子查询中的表上拉到外层查询的 FROM 子句中，并把子查询中的搜索条件合并到外层查询的搜索条件中，比如假设 s2 中存在这个一个 key2 列，列上有唯一性索引：

```
SELECT * FROM s1
```

```
WHERE key2 IN (SELECT key2 FROM s2 WHERE key3 = 'a');
```

由于 key2 列是 s2 表的唯一二级索引列，所以我们可以直接把 s2 表上拉到外层查询的 FROM 子句中，并且把子查询中的搜索条件合并到外层查询的搜索条件中，上拉之后的查询就是这样的：

```
SELECT s1.* FROM s1 INNER JOIN s2
```

```
ON s1.key2 = s2.key2
```

```
WHERE s2.key3 = 'a';
```

为啥当子查询的查询列表处只有主键或者唯一索引列时，就可以直接将子查询转换为连接查询呢？因为主键或者唯一索引列中的数据本身就是不重复的嘛！所以对于同一条 s1 表中的记录，你不可能找到两条以上的符合 s1.key2 = s2.key2 的记录。

DuplicateWeedout execution strategy (重复值消除)

对于这个查询来说：

```
SELECT * FROM s1 WHERE order_note IN (SELECT order_note FROM s2 WHERE order_no= 'a');
```

转换为半连接查询后，s1 表中的某条记录可能在 s2 表中有多条匹配的记录，所以该条记录可能多次被添加到最后的结果集中，为了消除重复，我们可以建立一个临时表，比方说这个临时表长这样：

```
CREATE TABLE tmp (  
    id PRIMARY KEY  
);
```

这样在执行连接查询的过程中，每当某条 s1 表中的记录要加入结果集时，就首先把这条记录的 id 值加入到这个临时表里，如果添加成功，说明之前这条 s1 表中的记录并没有加入最终的结果集，现在把该记录添加到最终的结果集；如果添加失败，说明之前这条 s1 表中的记录已经加入过最终的结果集，这里直接把它丢弃就好了，这种使用临时表消除 semi-join 结果集中的重复值的方式称之为 DuplicateWeedout。

LooseScan execution strategy (松散扫描)

大家看这个查询：

```
SELECT * FROM s1 WHERE order_note IN (SELECT order_no FROM s2 WHERE order_no> 'a' AND order_no< 'b');
```

在子查询中，对于 s2 表的访问可以使用到 order_no 列的索引，而恰好子查询的查询列表处就是 order_no 列，这样在将该查询转换为半连接查询后，如果将 s2 作为驱动表执行查询的话，那么执行过程就是这样：

在 s2 表的 idx_order_no 索引中，值为'aa'的二级索引记录一共有 3 条，那么只需要取第一条的值到 s1 表中查找 s1.order_note='aa'的记录，如果能在 s1 表中找到对应的记录，那么就把对应的记录加入到结果集。依此类推，其他值相同的二级索引记录，也只需要取第一条记录的值到 s1 表中找匹配的记录，这种虽然是扫描索引，但只取值相同的记录的第一条去做匹配操作的方式称之为松散扫描。

当然除了我们上面所说的，MySQL 中的半连接方式还有好几种，比如 Semi-join Materialization 半连接物化、FirstMatch execution strategy（首次匹配）等等，我们就不更深入的讨论了。

semi-join 的适用条件

当然，并不是所有包含 IN 子查询的查询语句都可以转换为 semi-join，只有形如这样的查询才可以被转换为 semi-join：

```
SELECT ... FROM outer_tables
    WHERE expr IN (SELECT ... FROM inner_tables ...) AND ...
```

或者这样的形式也可以：

```
SELECT ... FROM outer_tables
    WHERE (oe1, oe2, ...) IN (SELECT ie1, ie2, ... FROM inner_tables ...) AND ...
```

用文字总结一下，只有符合下边这些条件的子查询才可以被转换为 semi-join：

该子查询必须是和 IN 语句组成的布尔表达式，并且在外层查询的 WHERE 或者 ON 子句中出现。

外层查询也可以有其他的搜索条件，只不过和 IN 子查询的搜索条件必须使用 AND 连接起来。

该子查询必须是一个单一的查询，不能是由若干查询由 UNION 连接起来的形式。

该子查询不能包含 GROUP BY 或者 HAVING 语句或者聚集函数。

MySQL 对不能转为 semi-join 查询的子查询优化

1、对于不相关子查询来说，可以尝试把它们物化之后再参与查询

比如我们上边提到的这个查询：

```
SELECT * FROM s1 WHERE order_note NOT IN (SELECT order_note FROM s2
WHERE order_no= 'a')
```

先将子查询物化，然后再判断 order_note 是否在物化表的结果集中可以加快查询执行的速度。

2、不管子查询是相关的还是不相关的，都可以把 IN 子查询尝试转为 EXISTS 子查询

其实对于任意一个 IN 子查询来说，都可以被转为 EXISTS 子查询，通用的例子如下：

```
outer_expr IN (SELECT inner_expr FROM ... WHERE subquery_where)
```

可以被转换为：

```
EXISTS (SELECT inner_expr FROM ... WHERE subquery_where AND
outer_expr=inner_expr)
```

为啥要转换呢？这是因为不转换的话可能用不到索引，比方说下边这个查询：

```
SELECT * FROM s1 WHERE order_no IN (SELECT order_no FROM s2 where
s1.order_note = s2.order_note) OR insert_time > '2021-03-22 18:28:28';
```

这个查询中的子查询是一个相关子查询，而且子查询执行的时候不能使用到索引，但是将它转为 EXISTS 子查询后却可以使用到索引：

```
SELECT * FROM s1 WHERE EXISTS (SELECT 1 FROM s2 where s1.order_note =
s2.order_note AND s2.order_no= s1.order_no) OR insert_time > '2021-03-22
18:28:28'00;
```

转为 EXISTS 子查询时便可能使用到 s2 表的 idx_order_no 索引了。

需要注意的是，如果 IN 子查询不满足转换为 semi-join 的条件，又不能转换为物化表或者转换为物化表的成本太大，那么它就会被转换为 EXISTS 查询。

在 MySQL5.5 以及之前的版本没有引进 semi-join 和物化的方式优化子查询时，优化器都会把 IN 子查询转换为 EXISTS 子查询，所以当时好多声音都是建议大家把子查询转为连接，不过随着 MySQL 的发展，最近的版本中引入了非常多的子查询优化策略，内部的转换工作优化器会为大家自动实现。

小结

如果 IN 子查询符合转换为 semi-join 的条件，查询优化器会优先把该子查询转换为 semi-join，然后再考虑下边 5 种执行半连接的策略中哪个成本最低：

Table pullout

DuplicateWeedout

LooseScan

Materialization

FirstMatch

选择成本最低的那种执行策略来执行子查询。

如果 IN 子查询不符合转换为 semi-join 的条件，那么查询优化器会从下边两种策略中找出一种成本更低的方式执行子查询：

先将子查询物化之后再执行查询

执行 IN to EXISTS 转换。

ANY/ALL 子查询优化

如果 ANY/ALL 子查询是不相关子查询的话，它们在很多场合都能转换成我们熟悉的方式去执行，比方说：

原始表达式 转换为

< ANY (SELECT inner_expr ...) < (SELECT MAX(inner_expr) ...)

> ANY (SELECT inner_expr ...) > (SELECT MIN(inner_expr) ...)

< ALL (SELECT inner_expr ...) < (SELECT MIN(inner_expr) ...)

> ALL (SELECT inner_expr ...) > (SELECT MAX(inner_expr) ...)

[NOT] EXISTS 子查询的执行

如果[NOT] EXISTS 子查询是不相关子查询，可以先执行子查询，得出该[NOT] EXISTS 子查询的结果是 TRUE 还是 FALSE，并重写原先的查询语句，比如对这个查询来说：

```
SELECT * FROM s1 WHERE EXISTS (SELECT 1 FROM s2 WHERE expire_time= 'a')  
OR order_no> '2021-03-22 18:28:28';
```

因为这个语句里的子查询是不相关子查询，所以优化器会首先执行该子查询，假设该 EXISTS 子查询的结果为 TRUE，那么接着优化器会重写查询为：

```
SELECT * FROM s1 WHERE TRUE OR order_no> '2021-03-22 18:28:28';
```

进一步简化后就变成了：

```
SELECT * FROM s1 WHERE TRUE;
```

对于相关的[NOT] EXISTS 子查询来说，比如这个查询：

```
SELECT * FROM s1 WHERE EXISTS (SELECT 1 FROM s2 WHERE s1.order_note =  
s2.order_note);
```

很不幸，这个查询只能按照我们想象中的那种执行相关子查询的方式来执行。不过如果[NOT] EXISTS 子查询中如果可以使用索引的话，那查询速度也会加快不少，比如：

```
SELECT * FROM s1 WHERE EXISTS (SELECT 1 FROM s2 WHERE  
s1.order_note = s2.order_no);
```

上边这个 EXISTS 子查询中可以使用 idx_order_no 来加快查询速度。

5.6. 再深入查询优化

5.6.1. 全局考虑性能优化

5.6.3.1. 为什么查询速度会慢

在尝试编写快速的查询之前，需要清楚一点，真正重要的是响应时间。如果把查询看作是一个任务，那么它由一系列子任务组成，每个子任务都会消耗一定的时间。如果要优化查询，实际上要优化其子任务，要么消除其中一些子任务，要么减少子任务的执行次数，要么让子任务运行得更快。

MySQL 查询的生命周期大致可以按照顺序来看：从客户端，到服务器，然后在服务器上进行解析，生成执行计划，执行，并返回结果给客户端。其中“执行”可以认为是整个生命周期中最重要阶段，这其中包括了大量为了检索数据到存储引擎的调用以及调用后的数据处理，包括排序、分组等。

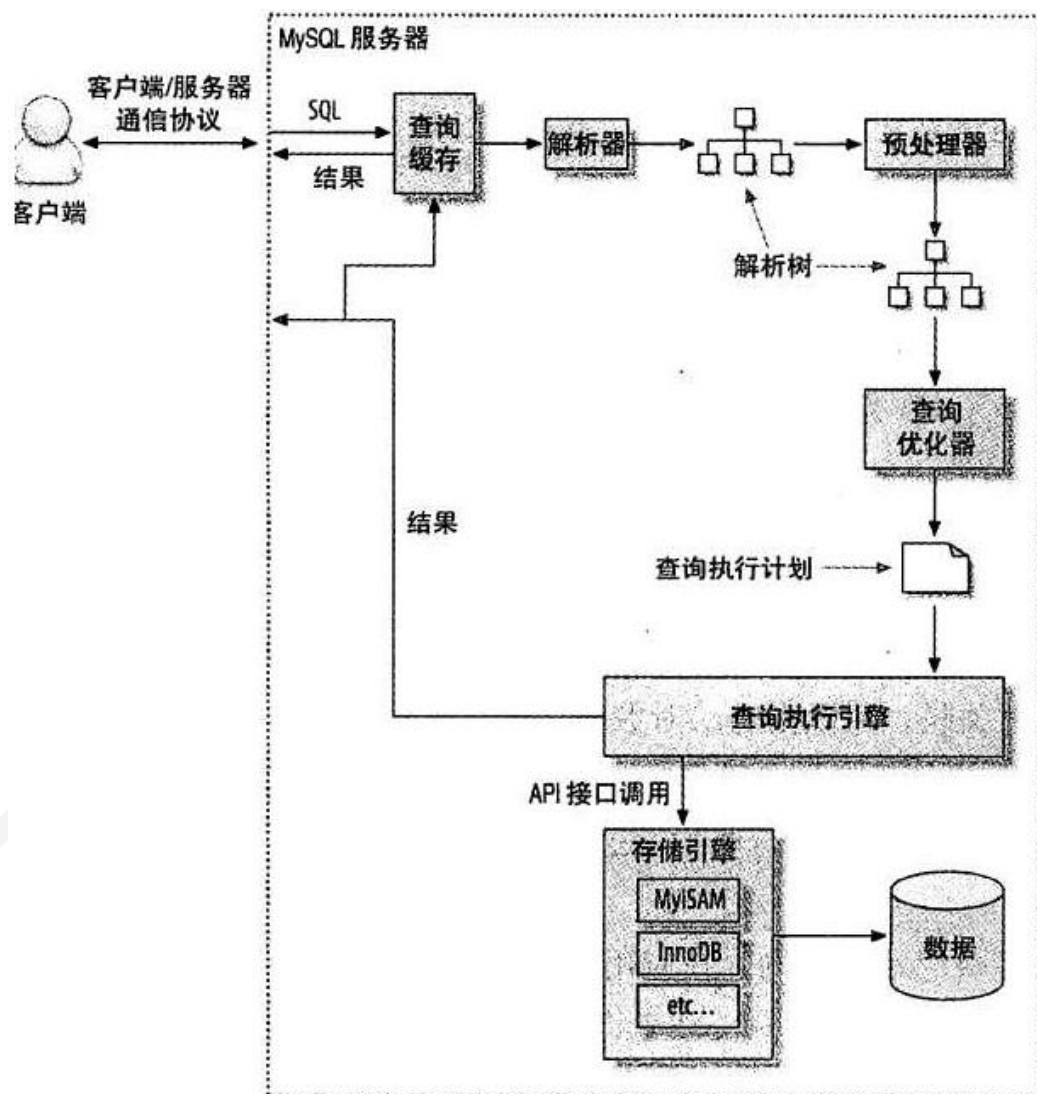
在完成这些任务的时候，查询需要在不同的地方花费时间，包括网络，CPU 计算，生成统计信息和执行计划、锁等待（互斥等待）等操作，尤其是向底层存储引擎检索数据的调用操作，这些调用需要在内存操作，CPU 操作和内存不足时导致的 IO 操作上消耗时间。根据存储引擎不同，可能还会产生大量的上下文切换以及系统调用。

优化查询的目的就是减少和消除这些操作所花费的时间。

5.6.3.2. 查询执行的流程再回顾

当希望 MySQL 能够以更高的性能运行查询时，最好的办法就是弄清楚 MySQL 是如何优化和执行查询的。一旦理解这一点，很多查询优化工作实际上就是遵循一些原则让优化器能够按照预想的合理的方式运行。

换句话说，是时候回头看看我们前面讨论的内容了:MySQL 执行一个查询的过程。根据下图，我们可以看到当向 MySQL 发送一个请求的时候，MySQL 到底做了些什么：



1. 客户端发送一条查询给服务器。
2. 服务器先检查查询缓存，如果命中了缓存，则立刻返回存储在缓存中的结果。否则进入下一阶段。
3. 服务器端进行 SQL 解析、预处理，再由优化器生成对应的执行计划。
4. MySQL 根据优化器生成的执行计划，调用存储引擎的 API 来执行查询。
5. 将结果返回给客户端。

上面的每一步都比想象的复杂，查询优化器固然是其中特别复杂也特别难理解的部分。但是其他部分就对查询的性能毫无影响？

5.6.3.3. MySQL 客户端/服务器通信协议

一般来说，不需要去理解 MySQL 通信协议的内部实现细节，只需要大致理解通信协议是如何工作的。MySQL 客户端和服务端之间的通信协议是“半双工”的，这意味着，在任何一个时刻，要么是由服务器向客户端发送数据，要么是由客户端向服务器发送数据，这两个动作不能同时发生。所以，我们无法也无须将一个消息切成小块独立来发送。

这种协议让 MySQL 通信简单快速，但是也从很多地方限制了 MySQL。一个明显的限制是，这意味着没法进行流量控制。一旦一端开始发送消息，另一端要接收完整个消息才能响应它。这就像来回抛球的游戏；在任何时刻，只有一个人能控制球，而且只有控制球的人才能将球抛回去（发送消息）。

客户端用一个单独的数据包将查询传给服务器。这也是为什么当查询的语句很长的时候，参数 `max_allowed_packet` 就特别重要了。一旦客户端发送了请求，它能做的事情就只是等待结果了。

相反的，一般服务器响应给用户的数据通常很多，由多个数据包组成。当服务器开始响应客户端请求时，客户端必须完整地接收整个返回结果，而不能简单地只取前面几条结果，然后让服务器停止发送数据。这种情况下，客户端若接收完整的结果，然后取前面几条需要的结果，或者接收完几条结果后就“粗暴”地断开连接，都不是好主意。这也是在必要的时候一定要在查询中加上 `LIMIT` 限制的原因。

换一种方式解释这种行为：当客户端从服务器取数据时，看起来是一个拉数据的过程，但实际上是 MySQL 在向客户端推送数据的过程。客户端不断地接收从服务器推送的数据，客户端也没法让服务器停下来。

多数连接 MySQL 的库函数都可以获得全部结果集并缓存到内存里，还可以逐行获取需要的数据。默认一般是获得全部结果集并缓存到内存中。MySQL 通常需要等所有的数据都已经发送给客户端才能释放这条查询所占用的资源，所以接收全部结果并缓存通常可以减少服务器的压力，让查询能够早点结束、早点释放相应的资源。

当使用库函数从 MySQL 获取数据时，其结果看起来都是从 MySQL 服务器获取数据，而实际上都是从这个库函数的缓存获取数据。多数情况下这没什么问题，但是如果需要返回一个很大的结果集的时候，这样做并不好，因为库函数会花很多时间和内存来存储所有的结果集。

对于 Java 程序来说，很有可能发生 OOM，所以 MySQL 的 JDBC 里提供了 `setFetchSize()` 之类的功能，来解决这个问题：

1、当 `statement` 设置以下属性时，采用的是流数据接收方式，每次只从服务器接收部份数据，直到所有数据处理完毕，不会发生 JVM OOM。

```
setResultSetType(ResultSet.TYPE_FORWARD_ONLY);  
setFetchSize(Integer.MIN_VALUE);
```

2、调用 `statement` 的 `enableStreamingResults` 方法，实际上 `enableStreamingResults` 方法内部封装的就是第 1 种方式。

3、设置连接属性 `useCursorFetch=true` (5.0 版驱动开始支持), `statement` 以 `TYPE_FORWARD_ONLY` 打开, 再设置 `fetch size` 参数, 表示采用服务器端游标, 每次从服务器取 `fetch_size` 条数据。

比如:

```
con = DriverManager.getConnection(url);
ps = (PreparedStatement)
con.prepareStatement(sql,ResultSet.TYPE_FORWARD_ONLY,
    ResultSet.CONCUR_READ_ONLY);
ps.setFetchSize(Integer.MIN_VALUE);
ps.setFetchDirection(ResultSet.FETCH_REVERSE);
rs = ps.executeQuery();
while (rs.next()) {.....实际的业务处理}
```

5.6.3.4. 查询状态

对于一个 MySQL 连接, 或者说一个线程, 任何时刻都有一个状态, 该状态表示了 MySQL 当前正在做什么。在一个查询的生命周期中, 状态会变化很多次。

5.6.3.5. 查询优化处理

查询的生命周期的下一步是将一个 SQL 转换成一个执行计划, MySQL 再依照这个执行计划和存储引擎进行交互。这包括多个子阶段:解析 SQL、预处理、优化 SQL 执行计划。这个过程中任何错误(例如语法错误)都可能终止查询。在实际执行中, 这几部分可能一起执行也可能单独执行。

我们通过前面的学习, 可以看到 MySQL 的查询优化器是一个非常复杂的部件, 它使用了很多优化策略来生成一个最优的执行计划。优化策略可以简单地分为两种, 一种是静态优化, 一种是动态优化。静态优化可以直接对解析树进行分析, 并完成优化。例如, 优化器可以通过一些简单的代数变换将 **WHERE** 条件转换成另一种等价形式。静态优化不依赖于特别的数值, 如 **WHERE** 条件中带入的一些常数等。静态优化在第一次完成后就一直有效, 即使使用不同的参数重复执行查询也不会发生变化。可以认为这是一种“编译时优化”。

相反, 动态优化则和查询的上下文有关, 也可能和很多其他因素有关, 例如 **WHERE** 条件中的取值、索引中条目对应的数据行数等。这需要在每次查询的时候都重新评估, 可以认为这是“运行时优化”。

优化器是相当复杂性和智能的。建议大家“不要自以为比优化器更聪明”。如果没有必要, 不要去干扰优化器的工作, 让优化器按照它的方式工作。尽量按照优化器的提示去优化我们的表、索引和 SQL 语句, 比如写查询, 或者重新设计更优的库表结构, 或者添加更合适的索引。但是请尽可能的保持 SQL 语句的简洁, SQL 语句变得很复杂的情况下, 请相信我, 维护会成为一个地狱。而带来的最终的收益微乎其微。

当然, 虽然优化器已经很智能了, 但是有时候也无法给出最优的结果。有时候你可能比优化器更了解数据, 例如, 由于应用逻辑使得某些条件总是成立; 还

有时，优化器缺少某种功能特性，如哈希索引；再如前面提到的，从优化器的执行成本角度评估出来的最优执行计划，实际运行中可能比其他的执行计划更慢。

如果能够确认优化器给出的不是最佳选择，并且清楚优化背后的原理，那么也可以帮助优化器做进一步的优化。

MySQL 架构由多个层次组成。在服务器层有查询优化器，却没有保存数据和索引的统计信息。统计信息由存储引擎实现，不同的存储引擎可能会存储不同的统计信息（也可以按照不同的格式存储统计信息）。某些引擎，例如 Archive 引擎，则根本就没有存储任何统计信息！

因为服务器层没有任何统计信息，所以 MySQL 查询优化器在生成查询的执行计划时，需要向存储引擎获取相应的统计信息。存储引擎则提供给优化器对应的统计信息，包括：每个表或者索引有多少个页面、每个表的每个索引的基数是多少、数据行和索引长度、索引的分布信息等。优化器根据这些信息来选择一个最优的执行计划。

当出现不理想的 SQL 查询时，我们就需要知道查询优化器是如何工作的，以便有针对性的进行改进，不管是 SQL 语句本身还是表结构相关，比如索引。这个时候请仔细耐心的对慢查询进行分析。

5.6.3.6. 查询执行引擎

在解析和优化阶段，MySQL 将生成查询对应的执行计划，MySQL 的查询执行引擎则根据这个执行计划来完成整个查询。相对于查询优化阶段，查询执行阶段不是那么复杂：MySQL 只是简单地根据执行计划给出的指令逐步执行。

5.6.3.7. 返回结果给客户端

查询执行的最后一个阶段是将结果返回给客户端。即使查询不需要返回结果集给客户端，MySQL 仍然会返回这个查询的一些信息，如该查询影响到的行数。

如果查询可以被缓存，那么 MySQL 在这个阶段也会将结果存放到查询缓存中。

MySQL 将结果集返回客户端是一个增量、逐步返回的过程。一旦服务器开始生成第一条结果时，MySQL 就可以开始向客户端逐步返回结果集了。

这样处理有两个好处：服务器端无须存储太多的结果，也就不会因为要返回太多结果而消耗太多内存。另外，这样的处理也让 MySQL 客户端第一时间获得返回的结果。结果集中的每一行都会以一个满足 MySQL 客户端/服务器通信协议的封包发送，再通过 TCP 协议进行传输，在 TCP 传输的过程中，可能对 MySQL 的封包进行缓存然后批量传输。

5.6.3.8. 通过 show profile 分析 SQL

通过上面的描述可知，当我们通过应用程序访问 MySQL 服务时，有时候性能不一定全部卡在语句的执行上。当然通过慢查询日志定位那些执行效率较低的 SQL 语句时候我们常用的手段，但是：

一、慢查询日志在查询结束以后才记录，在应用反映执行效率出现问题的时候查询未必执行完成；

二、有时候问题的产生不一定是语句的执行，有可能是其他原因导致的。慢查询日志并不能定位问题。

show processlist

这个时候通过 **show processlist** 查看线程状态非常有用,这可以让我们很快地了解当前 MySQL 正在进行的线程,包括线程的状态、是否锁表等，可以实时地查看 SQL 的执行情况，同时对一些锁表操作进行优化。在一个繁忙的服务器上，可能会看到大量的不正常的状态，例如 **statistics** 正占用大量的时间。这通常表示，某个地方有异常了。线程常见的状态有很多，比如

statistics

The server is calculating statistics to develop a query execution plan. If a thread is in this state for a long time, the server is probably disk-bound performing other work.

服务器正在计算统计信息以研究一个查询执行计划。如果线程长时间处于此状态，则服务器可能是磁盘绑定执行其他工作。

Creating tmp table

The thread is creating a temporary table in memory or on disk. If the table is created in memory but later is converted to an on-disk table, the state during that operation is Copying to tmp table on disk.

该线程正在内存或磁盘上创建临时表。如果表在内存中创建但稍后转换为磁盘表，则该操作期间的状态将为 Copying to tmp table on disk

Sending data

The thread is reading and processing rows for a SELECT statement, and sending data to the client. Because operations occurring during this state tend to perform large amounts of disk access (reads), it is often the longest-running state over the lifetime of a given query.

线程正在读取和处理 SELECT 语句的行，并将数据发送到客户端。由于在此状态期间发生的操作往往会执行大量磁盘访问（读取），因此它通常是给定查询生命周期中运行时间最长的状态。

其余的可以参考：

<https://dev.mysql.com/doc/refman/5.7/en/general-thread-states.html>

通过 show profile 分析

对于每个线程到底时间花在哪里，可以通过 show profile 来分析。

1、首先检查当前 MySQL 是否支持 profile

select @@have_profiling;

```
mysql> select @@have_profiling;
+-----+
| @@have_profiling |
+-----+
| YES              |
+-----+
1 row in set, 1 warning (0.00 sec)
```

2、默认 profiling 是关闭的，可以通过 set 语句在 Session 级别开启 profiling:

select @@profiling;

set profiling=1;

```
mysql> select @@profiling;
+-----+
| @@profiling |
+-----+
| 0          |
+-----+
1 row in set, 1 warning (0.00 sec)

mysql> set profiling=1;
Query OK, 0 rows affected, 1 warning (0.00 sec)
```

3、执行一个 SQL 查询

select count(*) from order_exp;

4、通过 show profiles 语句，看到当前 SQL 的 Query ID

show profiles;

```
mysql> show profiles;
+-----+-----+-----+
| Query_ID | Duration | Query |
+-----+-----+-----+
| 1        | 0.00212775 | select count(*) from order_exp |
+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

5、通过 show profile for query 语句能够看到执行过程中线程的每个状态和消耗的时间

show profile for query 1;

```
mysql> show profile for query 1;
+-----+-----+
| Status                               | Duration |
+-----+-----+
| starting                             | 0.000084 |
| checking permissions                  | 0.000012 |
| checking permissions                  | 0.000004 |
| Opening tables                        | 0.000025 |
| init                                 | 0.000041 |
| System lock                           | 0.000010 |
| optimizing                            | 0.000007 |
| statistics                            | 0.000031 |
| preparing                             | 0.000019 |
| executing                             | 0.000003 |
| Sending data                          | 0.001846 |
| end                                   | 0.000005 |
| query end                             | 0.000009 |
| closing tables                        | 0.000007 |
| freeing items                         | 0.000013 |
| cleaning up                           | 0.000015 |
+-----+-----+
16 rows in set, 1 warning (0.00 sec)
```

通过仔细检查 `show profile for query` 的输出,能够发现在执行 `COUNT(*)`的过程中,时间主要消耗在 `Sending data` 这个状态上。

6、在获取到最消耗时间的线程状态后,MySQL 支持进一步选择 `all`、`cpu`、`block io`、`contextswitch`、`page faults` 等明细类型来查看 MySQL 在使用什么资源上耗费了过高的时间:

show profile all for query 1\G

```
***** 11. row *****
      Status: Sending data
      Duration: 0.001846
      CPU_user: 0.001846
      CPU_system: 0.000000
      Context_voluntary: 0
      Context_involuntary: 0
      Block_ops_in: 0
      Block_ops_out: 0
      Messages_sent: 0
      Messages_received: 0
      Page_faults_major: 0
      Page_faults_minor: 0
      Swaps: 0
      Source_function: exec
      Source_file: sql_executor.cc
      Source_line: 202
```

能够发现 `Sending data` 状态下,时间主要消耗在 `CPU` 上了。

所以 `show profile` 能够在做 SQL 优化时帮助我们了解时间都耗费到哪里去了,同时如果 MySQL 源码感兴趣,还可以通过 `show profile source for query` 查看 SQL 解析执行过程中每个步骤对应的源码的文件、函数名以及具体的源文件行数。

6. InnoDB 引擎底层解析

MySQL 对于我们来说还是一个黑盒,我们只负责使用客户端发送请求并等待服务器返回结果,表中的数据到底存到了哪里? 以什么格式存放的? MySQL 是以什么方式来访问的这些数据? 这些问题我们统统不知道。要搞明白查询优化背后的原理,就必须深入 MySQL 的底层去一探究竟,而且事务、锁等的原理也要求我们必须深入底层。

6.1. InnoDB 记录存储结构和索引页结构

InnoDB 是一个将表中的数据存储在磁盘上的存储引擎,所以即使关机后重启我们的数据还是存在的。而真正处理数据的过程是发生在内存中的,所以需要把磁盘中的数据加载到内存中,如果是处理写入或修改请求的话,还需要把内存中的内容刷新到磁盘上。而我们知道读写磁盘的速度非常慢,和内存读写差了几个数量级,所以当我们想从表中获取某些记录时,InnoDB 存储引擎需要一条一条的把记录从磁盘上读出来么?

InnoDB 采取的方式是:将数据划分为若干个页,以页作为磁盘和内存之间交互的基本单位,InnoDB 中页的大小一般为 16 KB。也就是在一般情况下,一次最少从磁盘中读取 16KB 的内容到内存中,一次最少把内存中的 16KB 内容刷新到磁盘中。

我们平时是以记录为单位来向表中插入数据的,这些记录在磁盘上的存放方式也被称为行格式或者记录格式。InnoDB 存储引擎设计了 4 种不同类型的行格式,分别是 Compact、Redundant、Dynamic 和 Compressed 行格式。

6.1.1. 行格式

我们可以在创建或修改表的语句中指定行格式:

CREATE TABLE 表名 (列的信息) ROW_FORMAT=行格式名称

6.1.1.1. COMPACT

记录的额外信息			记录的真实数据			
变长字段长度列表	NULL值列表	记录头信息	列1的值	列2的值		列N的值

我们知道 MySQL 支持一些变长的数据类型,比如 VARCHAR(M)、VARBINARY(M)、各种 TEXT 类型,各种 BLOB 类型,我们也可以把拥有这些数据类型的列称为变长字段,变长字段中存储多少字节的数据是不固定的,所以我们在存储真实数据的时候需要顺便把这些数据占用的字节数也存起来。如果该可变字段允许存储的最大字节数 ($M \times W$) 超过 255 字节并且真实存储的字节数 (L) 超过 127 字节,则使用 2 个字节,否则使用 1 个字节。

表中的某些列可能存储 NULL 值,如果把这些 NULL 值都放到记录的真实数据中存储会很占地方,所以 Compact 行格式把这些值为 NULL 的列统一管理起来,

存储到 NULL 值列表。每个允许存储 NULL 的列对应一个二进制位，二进制位的值为 1 时，代表该列的值为 NULL。二进制位的值为 0 时，代表该列的值不为 NULL。

还有一个用于描述记录的记录头信息，它是由固定的 5 个字节组成。5 个字节也就是 40 个二进制位，不同的位代表不同的意思。

预留位 1 1 没有使用

预留位 2 1 没有使用

delete_mask 1 标记该记录是否被删除

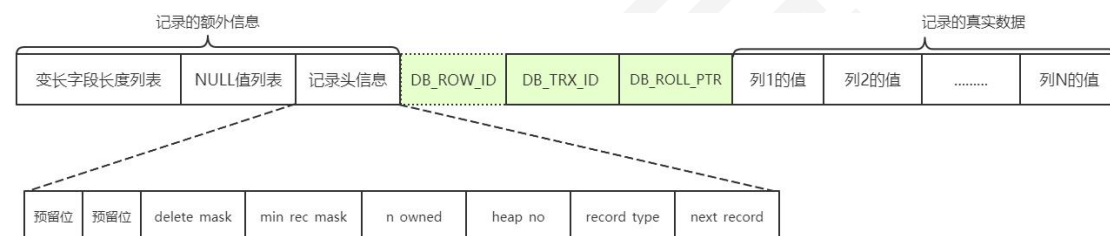
min_rec_mask 1 B+树的每层非叶子节点中的最小记录都会添加该标记

n_owned 4 表示当前记录拥有的记录数

heap_no 13 表示当前记录在页的位置信息

record_type3 表示当前记录的类型，0 表示普通记录，1 表示 B+树非叶子节点记录，2 表示最小记录，3 表示最大记录

next_record16 表示下一条记录的相对位置



记录的真实数据除了我们自己定义的列的数据以外，MySQL 会为每个记录默认的增加一些列（也称为隐藏列），包括：

DB_ROW_ID(row_id): 非必须，6 字节，表示行 ID，唯一标识一条记录

DB_TRX_ID: 必须，6 字节，表示事务 ID

DB_ROLL_PTR: 必须，7 字节，表示回滚指针

InnoDB 表对主键的生成策略是：优先使用用户自定义主键作为主键，如果用户没有定义主键，则选取一个 Unique 键作为主键，如果表中连 Unique 键都没有定义的话，则 InnoDB 会为表默认添加一个名为 row_id 的隐藏列作为主键。

DB_TRX_ID（也可以称为 trx_id）和 DB_ROLL_PTR（也可以称为 roll_ptr）这两个列是必有的，但是 row_id 是可选的（在没有自定义主键以及 Unique 键的情况下才会添加该列）。

其他的行格式和 Compact 行格式差别不大。

6.1.1.2. Redundant 行格式

Redundant 行格式是 MySQL5.0 之前用的一种行格式，不予深究。

6.1.1.3. Dynamic 和 Compressed 行格式

MySQL5.7 的默认行格式就是 Dynamic，Dynamic 和 Compressed 行格式和 Compact 行格式挺像，只不过在处理行溢出数据时有所不同。Compressed 行格

式和 Dynamic 不同的一点是，Compressed 行格式会采用压缩算法对页面进行压缩，以节省空间。

6.1.1.4. 数据溢出

如果我们定义一个表，表中只有一个 VARCHAR 字段，如下：

```
CREATE TABLE test_varchar( c VARCHAR(60000) )
```

然后往这个字段插入 60000 个字符，会发生什么？

前边说过，MySQL 中磁盘和内存交互的基本单位是页，也就是说 MySQL 是以页为基本单位来管理存储空间的，我们的记录都会被分配到某个页中存储。而一个页的大小一般是 16KB，也就是 16384 字节，而一个 VARCHAR(M) 类型的列就最多可以存储 65532 个字节，这样就可能造成一个页存放不了一条记录的情况。

在 Compact 和 Redundant 行格式中，对于占用存储空间非常大的列，在记录的真实数据处只会存储该列的前 768 个字节的数据，然后把剩余的数据分散存储在几个其他的页中，记录的真实数据处用 20 个字节存储指向这些页的地址。这个过程也叫做行溢出，存储超出 768 字节的那些页面也被称为溢出页。

Dynamic 和 Compressed 行格式，不会在记录的真实数据处存储字段真实数据的前 768 个字节，而是把所有的字节都存储到其他页面中，只在记录的真实数据处存储其他页面的地址。

那发生行溢出的临界点是什么呢？也就是说在列存储多少字节的数据时就会发生行溢出？

MySQL 中规定一个页中至少存放两行记录，以上边的 test_varchar 表为例，它只有一个列 c，我们往这个表中插入两条记录，每条记录最少插入多少字节的数据才会行溢出现象呢？这得分析一下页中的空间都是如何利用的。

每个页除了存放我们的记录以外，也需要存储一些页本身的信息，加起来需要 132 个字节的空间，其他的空间都可以被用来存储记录。每个记录需要的额外信息是 27 字节。

假设一个列中存储的数据字节数为 n，MySQL 规定如果该列不发生溢出现象，就需要满足下边这个式子：

$$132 + 2 \times (27 + n) < 16384$$

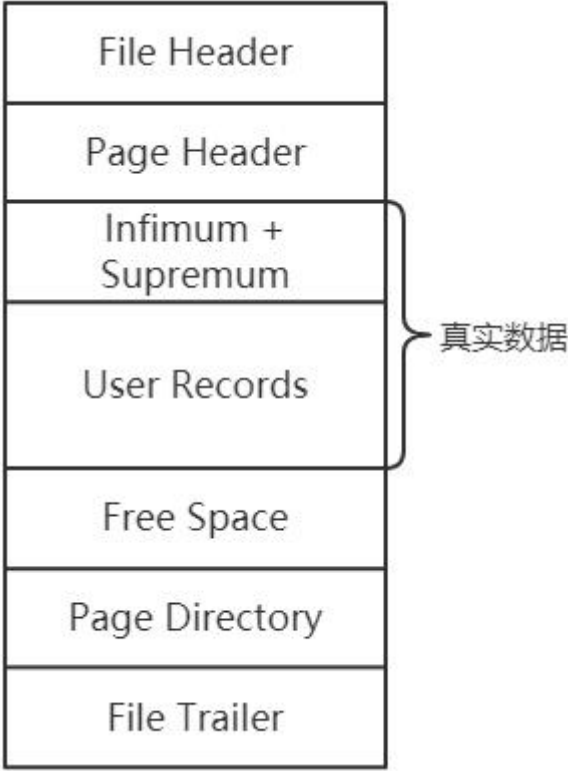
求解这个式子得出的解是：n < 8099。也就是说如果一个列中存储的数据小于 8099 个字节，那么该列就不会成为溢出列，否则该列就需要成为溢出列。不过这个 8099 个字节的结论只是针对只有一个列的 test_varchar 表来说的，如果表中有多个列，那上边的式子和结论都需要改。所以重点就是：这个临界点具体值无关紧要，只要知道如果我们一条记录的某个列中存储的数据占用的字节数非常多时，该列就可能成为溢出列。

6.1.2. 索引页格式

前边我们简单提了一下页的概念，它是 InnoDB 管理存储空间的基本单位，一个页的大小一般是 16KB。

InnoDB 为了不同的目的而设计了许多种不同类型的页，存放我们表中记录的那种类型的页自然也是其中的一员，官方称这种存放记录的页为索引（INDEX）页，不过要理解成数据页也没问题，毕竟存在着聚簇索引这种索引和数据混合的东西。

6.1.2.1. 数据页结构



一个 InnoDB 数据页的存储空间大致被划分成了 7 个部分：

- File Header 文件头部 38 字节 页的一些通用信息
- Page Header 页面头部 56 字节 数据页专有的一些信息
- Infimum + Supremum 最小记录和最大记录 26 字节 两个虚拟的行记录
- User Records 用户记录 大小不确定 实际存储的行记录内容
- Free Space 空闲空间 大小不确定 页中尚未使用的空间
- Page Directory 页面目录 大小不确定 页中的某些记录的相对位置
- File Trailer 文件尾部 8 字节 校验页是否完整

User Records

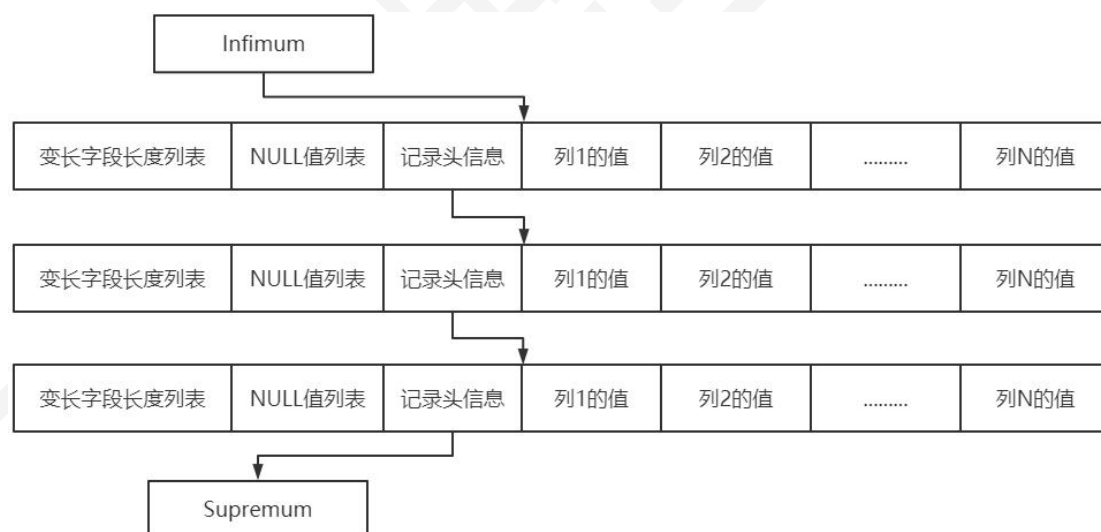
我们自己存储的记录会按照我们指定的行格式存储到 User Records 部分。但是在一开始生成页的时候，其实并没有 User Records 这个部分，每当我们插入一条记录，都会从 Free Space 部分，也就是尚未使用的存储空间中申请一个记录大小的空间划分到 User Records 部分，当 Free Space 部分的空间全部被 User Records 部分替代掉之后，也就意味着这个页使用完了，如果还有新的记录插入的话，就需要去申请新的页了。

当前记录被删除时，则会修改记录头信息中的 `delete_mask` 为 1，也就是说被删除的记录还在页中，还在真实的磁盘上。这些被删除的记录之所以不立即从磁盘上移除，是因为移除它们之后把其他的记录在磁盘上重新排列需要性能消耗。

所以只是打一个删除标记而已，所有被删除掉的记录都会组成一个所谓的垃圾链表，在这个链表中的记录占用的空间称之为所谓的可重用空间，之后如果有新记录插入到表中的话，可能把这些被删除的记录占用的存储空间覆盖掉。

同时我们插入的记录在会记录自己在本页中的位置，写入了记录头信息中 `heap_no` 部分。`heap_no` 值为 0 和 1 的记录是 InnoDB 自动给每个页增加的两个记录，称为伪记录或者虚拟记录。这两个伪记录一个代表最小记录，一个代表最大记录，这两条存放在页的 **User Records** 部分，他们被单独放在一个称为 **Infimum + Supremum** 的部分。

记录头信息中 `next_record` 记录了从当前记录的真实数据到下一条记录的真实数据的地址偏移量。这其实是个链表，可以通过一条记录找到它的下一条记录。但是需要注意注意再注意的一点是，下一条记录指得并不是按照我们插入顺序的下一条记录，而是按照主键值由小到大的顺序的下一条记录。而且规定 **Infimum** 记录（也就是最小记录）的下一条记录就是本页中主键值最小的用户记录，而本页中主键值最大的用户记录的下一条记录就是 **Supremum** 记录（也就是最大记录）



我们的记录按照主键从小到大的顺序形成了一个单链表，记录被删除，则从这个链表上摘除。

Page Directory

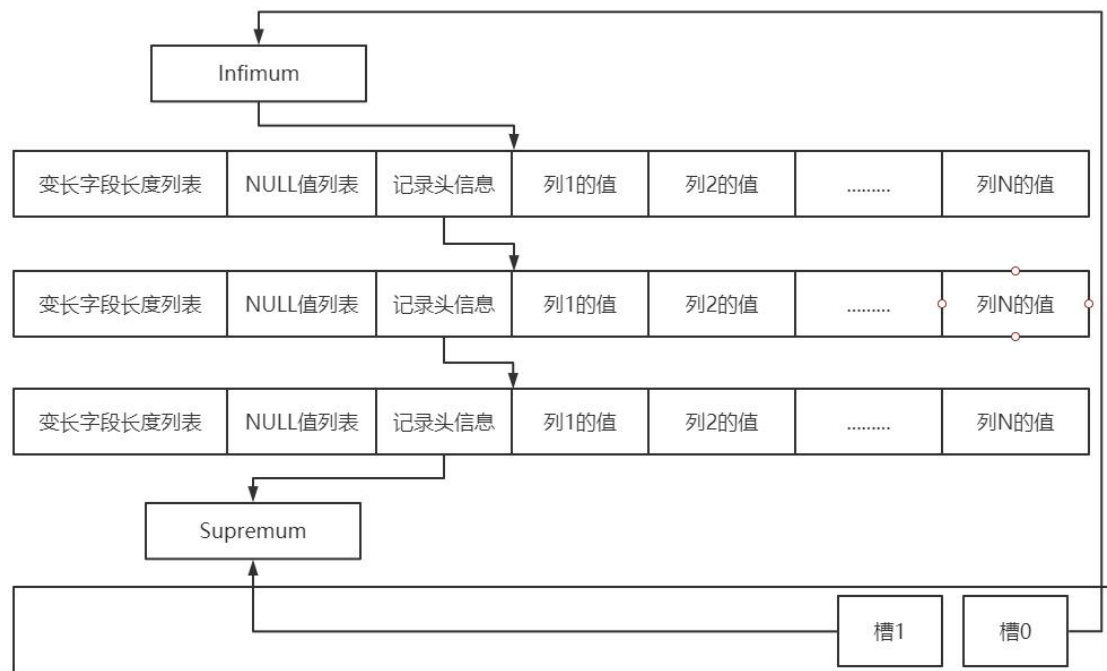
Page Directory 主要是解决记录链表的查找问题。如果我们想根据主键值查找页中的某条记录该咋办？按链表查找的办法：从 **Infimum** 记录（最小记录）开始，沿着链表一直往后找，总会找到或者找不到。

InnoDB 的改进是，为页中的记录再制作了一个目录，他们的制作过程是这样的：

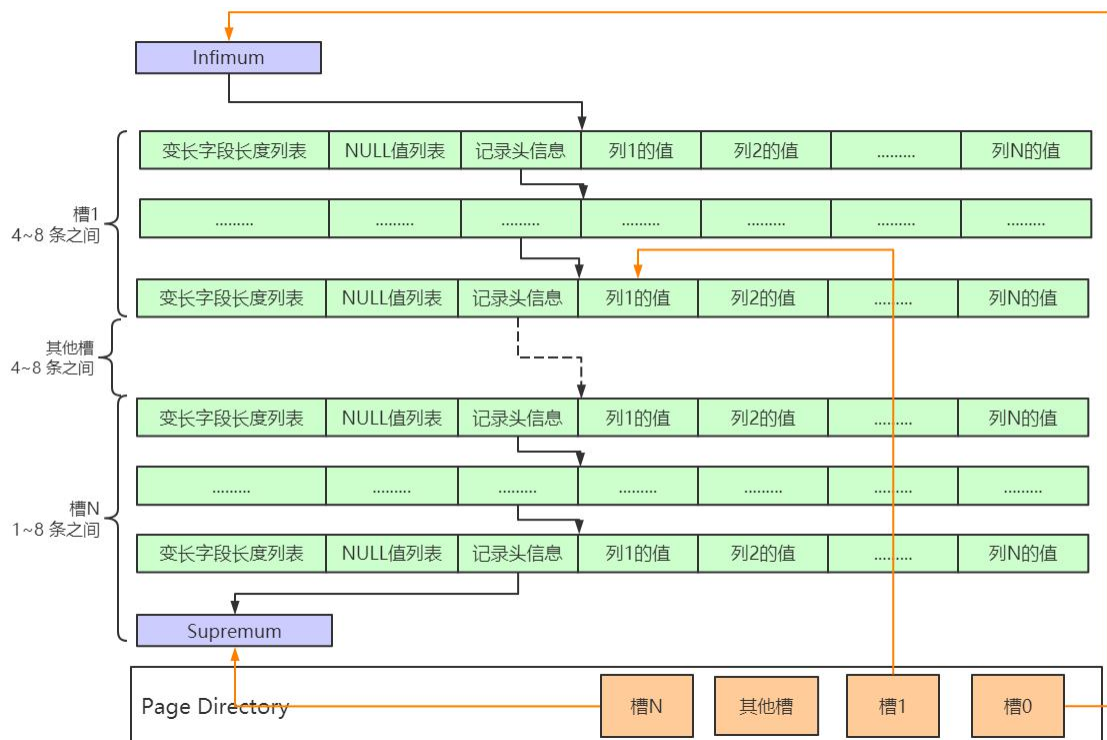
- 1、将所有正常的记录（包括最大和最小记录，不包括标记为已删除的记录）划分为几个组。

2、每个组的最后一条记录（也就是组内最大的那条记录）的头信息中的 **n_owned** 属性表示该记录拥有多少条记录，也就是该组内共有几条记录。

3、将每个组的最后一条记录的地址偏移量单独提取出来按顺序存储到靠近页的尾部的地方，这个地方就是所谓的 **Page Directory**，也就是页目录页面目录中的这些地址偏移量被称为槽（英文名：**Slot**），所以这个页面目录就是由槽组成的。



4、每个分组中的记录条数是有规定的：对于最小记录所在的分组只能有 **1** 条记录，最大记录所在的分组拥有的记录条数只能在 **1~8** 条之间，剩下的分组中记录的条数范围只能是在 **4~8** 条之间。如下图：



这样，一个数据页中查找指定主键值的记录的过程分为两步：

通过二分法确定该记录所在的槽，并找到该槽所在分组中主键值最小的那条记录。

通过记录的 `next_record` 属性遍历该槽所在的组中的各个记录。

Page Header

InnoDB 为了能得到一个数据页中存储的记录的状态信息，比如本页中已经存储了多少条记录，第一条记录的地址是什么，页目录中存储了多少个槽等等，特意在页中定义了一个叫 **Page Header** 的部分，它是页结构的第二部分，这个部分占用固定的 56 个字节，专门存储各种状态信息。

File Header

File Header 针对各种类型的页都通用，也就是说不同类型的页都会以 File Header 作为第一个组成部分，它描述了一些针对各种页都通用的一些信息，比方说页的类型，这个页的编号是多少，它的上一个页、下一个页是谁，页的校验和等等，这个部分占用固定的 38 个字节。

页的类型，包括 Undo 日志页、段信息节点、Insert Buffer 空闲列表、Insert Buffer 位图、系统页、事务系统数据、表空间头部信息、扩展描述页、溢出页、索引页，有些页会在后面的课程看到。

同时通过上一个页、下一个页建立一个双向链表把许许多多的页就串联起来，而无需这些页在物理上真正连着。但是并不是所有类型的页都有上一个和下一个页的属性，数据页是有这两个属性的，所以所有的数据页其实是一个双向链表。

File Trailer

我们知道 InnoDB 存储引擎会把数据存储到磁盘上，但是磁盘速度太慢，需要以页为单位把数据加载到内存中处理，如果该页中的数据在内存中被修改了，

那么在修改后的某个时间需要把数据同步到磁盘中。但是在同步了一半的时候中断电了咋办？

为了检测一个页是否完整（也就是在同步的时候有没有发生只同步一半的尴尬情况），InnoDB 每个页的尾部都加了一个 File Trailer 部分，这个部分由 8 个字节组成，可以分成 2 个小部分：

前 4 个字节代表页的校验和

这个部分是和 File Header 中的校验和相对应的。每当一个页面在内存中修改了，在同步之前就要把它的校验和算出来，因为 File Header 在页面的前边，所以校验和会被首先同步到磁盘，当完全写完时，校验和也会被写到页的尾部，如果完全同步成功，则页的首部和尾部的校验和应该是一致的。如果写了一半儿断电了，那么在 File Header 中的校验和就代表着已经修改过的页，而在 File Trailer 中的校验和代表着原先的页，二者不同则意味着同步中间出了错。

后 4 个字节代表页面被最后修改时对应的日志序列位置（LSN），这个也和校验页的完整性有关。

这个 File Trailer 与 File Header 类似，都是所有类型的页通用的。

6.2.InnoDB 的表空间

表空间是一个抽象的概念，对于系统表空间来说，对应着文件系统中一个或多个实际文件；对于每个独立表空间来说，对应着文件系统中一个名为表名.ibd 的实际文件。大家可以把表空间想象成被切分为许许多多多个页的池子，当我们想为某个表插入一条记录的时候，就从池子中捞出一个对应的页来把数据写进去。

再回忆一次，InnoDB 是以页为单位管理存储空间的，我们的聚簇索引（也就是完整的表数据）和其他的二级索引都是以 B+树的形式保存到表空间的，而 B+树的节点就是数据页。

任何类型的页都有 File Header 这个部分，File Header 中专门的地方（FIL_PAGE_ARCH_LOG_NO_OR_SPACE_ID）保存页属于哪个表空间，同时表空间中的每一个页都对应着一个页号（FIL_PAGE_OFFSET），这个页号由 4 个字节组成，也就是 32 个比特位，所以一个表空间最多可以拥有 2^{32} 个页，如果按照页的默认大小 16KB 来算，一个表空间最多支持 64TB 的数据。

6.2.1. 独立表空间结构

6.2.1.1. 区 (extent)

表空间中的页可以达到 2^{32} 个页，实在是太多了，为了更好的管理这些页面，InnoDB 中还有一个区（英文名：extent）的概念。对于 16KB 的页来说，连续的 64 个页就是一个区，也就是说一个区默认占用 1MB 空间大小。

不论是系统表空间还是独立表空间，都可以看成是由若干个区组成的，每 256 个区又被划分成一个组。

第一个组最开始的 3 个页面的类型是固定的：用来登记整个表空间的一些整体属性以及本组所有的区被称为 FSP_HDR，也就是 extent 0 ~ extent 255 这 256 个区，整个表空间只有一个 FSP_HDR。

其余各组最开始的 2 个页面的类型是固定的，一个 XDES 类型，用来登记本组 256 个区的属性，FSP_HDR 类型的页面其实和 XDES 类型的页面的作用类似，只不过 FSP_HDR 类型的页面还会额外存储一些表空间的属性。

引入区的主要目的是什么？我们每向表中插入一条记录，本质上就是向该表的聚簇索引以及所有二级索引代表的 B+树的节点中插入数据。而 B+树的每一层中的页都会形成一个双向链表，如果是以页为单位来分配存储空间的话，双向链表相邻的两个页之间的物理位置可能离得非常远。

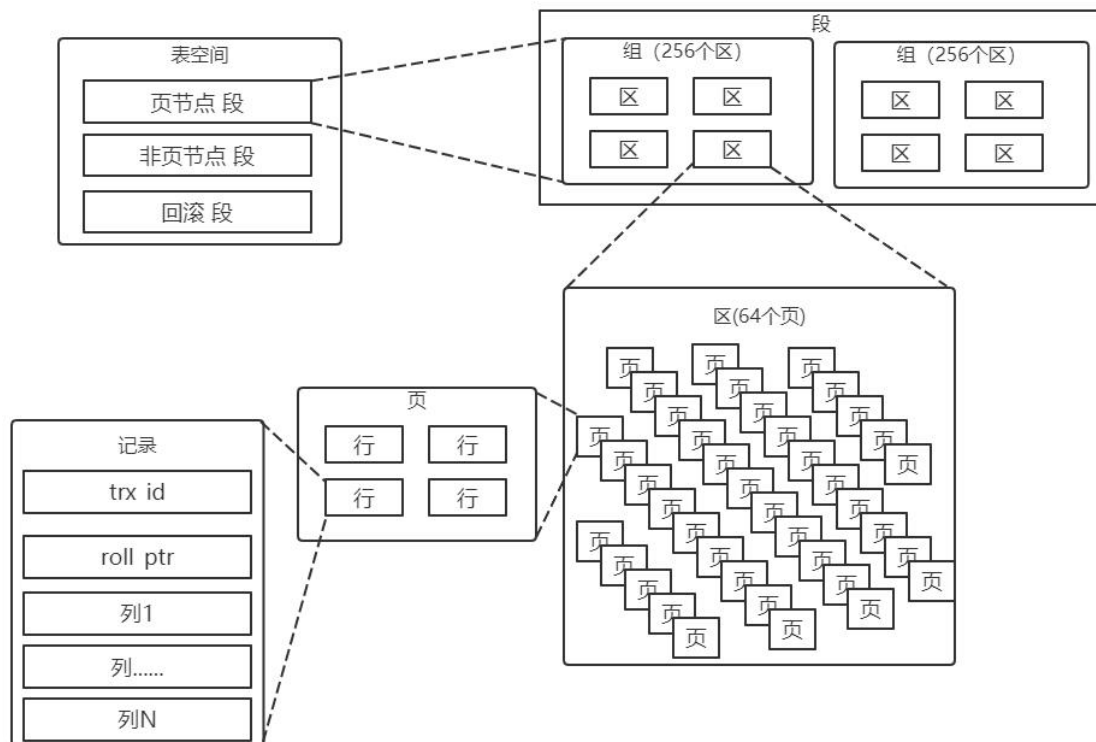
我们介绍 B+树索引的适用场景的时候特别提到范围查询只需要定位到最左边的记录和最右边的记录，然后沿着双向链表一直扫描就可以了，而如果链表中相邻的两个页物理位置离得非常远，就是所谓的随机 I/O。再一次强调，磁盘的速度和内存的速度差了好几个数量级，随机 I/O 是非常慢的，所以我们应该尽量让链表中相邻的页的物理位置也相邻，这样进行范围查询的时候才可以使用所谓的顺序 I/O。

一个区就是在物理位置上连续的 64 个页。在表中数据量大的时候，为某个索引分配空间的时候就不再按照页为单位分配了，而是按照区为单位分配，甚至在表中的数据十分非常特别多的时候，可以一次性分配多个连续的区，从性能角度看，可以消除很多的随机 I/O。

6.2.1.2. 段 (segment)

我们提到的范围查询，其实是对 B+树叶子节点中的记录进行顺序扫描，而如果不区分叶子节点和非叶子节点，统统把节点代表的页面放到申请到的区中的话，进行范围扫描的效果就大打折扣了。所以 InnoDB 对 B+树的叶子节点和非叶子节点进行了区别对待，也就是说叶子节点有自己独有的区，非叶子节点也有自己独有的区。存放叶子节点的区的集合就算是一个段 (segment)，存放非叶子节点的区的集合也算是一个段。也就是说一个索引会生成 2 个段，一个叶子节点段，一个非叶子节点段。

段其实不对应表空间中某一个连续的物理区域，而是一个逻辑上的概念。



6.2.2. 系统表空间

6.2.2.1. 整体结构

系统表空间的结构和独立表空间基本类似，只不过由于整个 MySQL 进程只有一个系统表空间，在系统表空间中会额外记录一些有关整个系统信息的页面，所以会比独立表空间多出一些记录这些信息的页面，相当于是表空间之首，所以它的表空间 ID（Space ID）是 0。

系统表空间和独立表空间的前三个页面的类型是一致的，只是页号为 3~7 的页面是系统表空间特有的。分别包括：

页号	页面类型	英文描述	描述
3	SYS Insert Buffer Header	存储 Insert Buffer 的头部信息	
4	INDEX Insert Buffer Root	存储 Insert Buffer 的根页面	
5	TRX_SYS Transaction System	事务系统的相关信息	
6	SYS First Rollback Segment	第一个回滚段的页面	
7	SYS Data Dictionary Header	数据字典头部信息	

系统表空间的 extent 1 和 extent 2 这两个区，也就是页号从 64~191 这 128 个页面被称为 Doublewrite buffer，也就是双写缓冲区。

双写缓冲区/双写机制

双写缓冲区/双写机制是 InnoDB 的三大特性之一，还有两个是 Buffer Pool、自适应 Hash 索引。

它是一种特殊文件 flush 技术，带给 InnoDB 存储引擎的是数据页的可靠性。它的作用是，在把页写到数据文件之前，InnoDB 先把它们写到一个叫 **doublewrite buffer**（双写缓冲区）的连续区域内，在写 **doublewrite buffer** 完成后，InnoDB 才会把页写到数据文件的适当的位置。如果在写页的过程中发生意外崩溃，InnoDB 在稍后的恢复过程中在 **doublewrite buffer** 中找到完好的 **page** 副本用于恢复。

所以，虽然叫双写缓冲区，但是这个缓冲区不仅在内存中有，更多的是属于 MySQL 的系统表空间，属于磁盘文件的一部分。那为什么要引入一个双写机制呢？

InnoDB 的页大小一般是 16KB，其数据校验也是针对这 16KB 来计算的，将数据写入到磁盘是以页为单位进行操作的。而操作系统写文件是以 4KB 作为单位的，那么每写一个 InnoDB 的页到磁盘上，操作系统需要写 4 个块。

而计算机硬件和操作系统，在极端情况下（比如断电）往往并不能保证这一操作的原子性，16K 的数据，写入 4K 时，发生了系统断电或系统崩溃，只有一部分写是成功的，这种情况下会产生 **partial page write**（部分页写入）问题。这时页数据出现不一样的情形，从而形成一个“断裂”的页，使数据产生混乱。在 InnoDB 存储引擎未使用 **doublewrite** 技术前，曾经出现过因为部分写失效而导致数据丢失的情况。

doublewrite buffer 是 InnoDB 在表空间上的 128 个页（2 个区，**extend1** 和 **extend2**），大小是 2MB。为了解决部分页写入问题，当 MySQL 将脏数据 flush 到数据文件的时候，先使用 **memcpy** 将脏数据复制到内存中的一个区域（也是 2M），之后通过这个内存区域再分 2 次，每次写入 1MB 到系统表空间，然后马上调用 **fsync** 函数，同步到磁盘上。在这个过程中是顺序写，开销并不大，在完成 **doublewrite** 写入后，再将数据写入各数据文件文件，这时是离散写入。

所以在正常的情况下，MySQL 写数据页时，会写两遍到磁盘上，第一遍是写到 **doublewrite buffer**，第二遍是写到真正的数据文件中。如果发生了极端情况（断电），InnoDB 再次启动后，发现了一个页数据已经损坏，那么此时就可以从 **doublewrite buffer** 中进行数据恢复了。

前面说过，位于系统表空间上的 **doublewrite buffer** 实际上也是一个文件，写系统表空间会导致系统有更多的 **fsync** 操作，而硬盘的 **fsync** 性能因素会降低 MySQL 的整体性能。不过在存储上，**doublewrite** 是在一个连续的存储空间，所以硬盘在写数据的时候是顺序写，而不是随机写，这样性能影响不大，相比不双写，降低了大概 5-10% 左右。

所以，在一些情况下可以关闭 **doublewrite** 以获取更高的性能。比如在 **slave** 上可以关闭，因为即使出现了 **partial page write** 问题，数据还是可以从中继日志中恢复。比如某些文件系统 ZFS 本身有些文件系统本身就提供了部分写失效的防范机制，也可以关闭。

在数据库异常关闭的情况下启动时，都会做数据库恢复（redo）操作，恢复的过程中，数据库都会检查页面是不是合法（校验等等），如果发现一个页面校验结果不一致，则此时会用到双写这个功能。

有经验的同学也许会想到，如果发生写失效，可以通过重做日志（Redo Log）进行恢复啊！但是要注意，重做日志中记录的是对页的物理操作，如偏移量 800，写 'aaaa' 记录，而不是页面的全量记录，而如果发生 **partial page write**（部分页写

入) 问题时, 出现问题的是未修改过的数据, 此时重做日志(Redo Log)无能为力。写 doublewrite buffer 成功了, 这个问题就不用担心了。

如果是写 doublewrite buffer 本身失败, 那么这些数据不会被写到磁盘, InnoDB 此时会从磁盘载入原始的数据, 然后通过 InnoDB 的事务日志来计算出正确的数据, 重新写入到 doublewrite buffer, 这个速度就比较慢了。如果 doublewrite buffer 写成功的话, 但是写数据文件失败, innodb 就不用通过事务日志来计算了, 而是直接用 doublewrite buffer 的数据再写一遍, 速度上会快很多。

总体来说, doublewrite buffer 的作用有两个: 提高 innodb 把缓存的数据写到硬盘这个过程的安全性; 间接的好处就是, innodb 的事务日志不需要包含所有数据的前后映像, 而是二进制变化量, 这可以节省大量的 IO。

6.2.2.2. InnoDB 数据字典(Data Dictionary Header)

我们平时使用 INSERT 语句向表中插入的那些记录称之为用户数据, MySQL 只是作为一个软件来为我们来保管这些数据, 提供方便的增删改查接口而已。但是每当我们向一个表中插入一条记录的时候, MySQL 先要校验一下插入语句对应的表存不存在, 插入的列和表中的列是否符合, 如果语法没有问题的话, 还需要知道该表的聚簇索引和所有二级索引对应的根页面是哪个表空间的哪个页面, 然后把记录插入对应索引的 B+树中。所以说, MySQL 除了保存着我们插入的用户数据之外, 还需要保存许多额外的信息, 比方说:

某个表属于哪个表空间, 表里边有多少列, 表对应的每一个列的类型是什么, 该表有多少索引, 每个索引对应哪几个字段, 该索引对应的根页面在哪个表空间的哪个页面, 该表有哪些外键, 外键对应哪个表的哪些列, 某个表空间对应文件系统上文件路径是什么。

上述这些数据并不是我们使用 INSERT 语句插入的用户数据, 实际上是为了更好的管理我们这些用户数据而不得已引入的一些额外数据, 这些数据也称为元数据。InnoDB 存储引擎特意定义了一些列的内部系统表 (internal system table) 来记录这些元数据:

表名	描述
----	----

SYS_TABLES	整个 InnoDB 存储引擎中所有的表的信息
------------	------------------------

SYS_COLUMNS	整个 InnoDB 存储引擎中所有的列的信息
-------------	------------------------

SYS_INDEXES	整个 InnoDB 存储引擎中所有的索引的信息
-------------	-------------------------

SYS_FIELDS	整个 InnoDB 存储引擎中所有的索引对应的列的信息
------------	-----------------------------

SYS_FOREIGN	整个 InnoDB 存储引擎中所有的外键的信息
-------------	-------------------------

SYS_FOREIGN_COLS	整个 InnoDB 存储引擎中所有的外键对应列的信息
------------------	----------------------------

SYS_TABLESPACES	整个 InnoDB 存储引擎中所有的表空间信息
-----------------	-------------------------

SYS_DATAFILES	整个 InnoDB 存储引擎中所有的表空间对应文件系统的文件路径信息
---------------	------------------------------------

SYS_VIRTUAL	整个 InnoDB 存储引擎中所有的虚拟生成列的信息
-------------	----------------------------

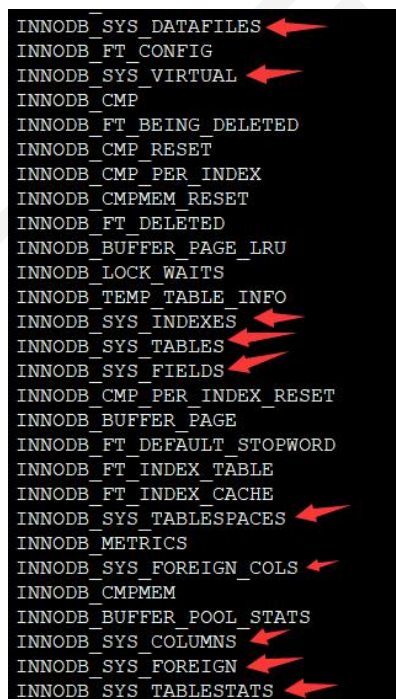
这些系统表也被称为数据字典，它们都是以 B+树的形式保存在系统表空间的某些页面中，其中 `SYS_TABLES`、`SYS_COLUMNS`、`SYS_INDEXES`、`SYS_FIELDS` 这四个表尤其重要，称之为基本系统表。

这 4 个表是表中之表，那这 4 个表的元数据去哪里获取呢？只能把这 4 个表的元数据，就是它们有哪些列、哪些索引等信息硬编码到代码中，然后 InnoDB 的又拿出一个固定的页面来记录这 4 个表的聚簇索引和二级索引对应的 B+树位置，这个页面就是页号为 7 的页面 **Data Dictionary Header**，类型为 `SYS`，记录了数据字典的头部信息。除了这 4 个表的 5 个索引的根页面信息外，这个页号为 7 的页面还记录了整个 InnoDB 存储引擎的一些全局属性，比如 `Row ID`。

数据字典头部信息中有个 `Max Row ID` 字段，我们说过如果我们不显式的为表定义主键，而且表中也没有 `UNIQUE` 索引，那么 InnoDB 存储引擎会默认为我们生成一个名为 `row_id` 的列作为主键。因为它是主键，所以每条记录的 `row_id` 列的值不能重复。

原则上只要一个表中的 `row_id` 列不重复就可以了，也就是说表 a 和表 b 拥有一样的 `row_id` 列也没啥关系，不过 InnoDB 只提供了这个 `Max Row ID` 字段，不论哪个拥有 `row_id` 列的表插入一条记录时，该记录的 `row_id` 列的值就是 `Max Row ID` 对应的值，然后再把 `Max Row ID` 对应的值加 1，也就是说这个 `Max Row ID` 是全局共享的。

用户是不能直接访问 InnoDB 的这些内部系统表的，除非你直接去解析系统表空间对应文件系统上的文件。不过 InnoDB 考虑到查看这些表的内容可能有助于大家分析问题，所以在系统数据库 `information_schema` 中提供了一些以 `innodb_sys` 开头的表：



```
INNO_DB_SYS_DATAFILES
INNO_DB_FT_CONFIG
INNO_DB_SYS_VIRTUAL
INNO_DB_CMP
INNO_DB_FT_BEING_DELETED
INNO_DB_CMP_RESET
INNO_DB_CMP_PER_INDEX
INNO_DB_CMPMEM_RESET
INNO_DB_FT_DELETED
INNO_DB_BUFFER_PAGE_LRU
INNO_DB_LOCK_WAITS
INNO_DB_TEMP_TABLE_INFO
INNO_DB_SYS_INDEXES
INNO_DB_SYS_TABLES
INNO_DB_SYS_FIELDS
INNO_DB_CMP_PER_INDEX_RESET
INNO_DB_BUFFER_PAGE
INNO_DB_FT_DEFAULT_STOPWORD
INNO_DB_FT_INDEX_TABLE
INNO_DB_FT_INDEX_CACHE
INNO_DB_SYS_TABLESPACES
INNO_DB_METRICS
INNO_DB_SYS_FOREIGN_COLS
INNO_DB_CMPMEM
INNO_DB_BUFFER_POOL_STATS
INNO_DB_SYS_COLUMNS
INNO_DB_SYS_FOREIGN
INNO_DB_SYS_TABLESTATS
```

在 `information_schema` 数据库中的这些以 `INNODB_SYS` 开头的表并不是真正的内部系统表（内部系统表就是我们上边唠叨的以 `SYS` 开头的那些表），而是在存储引擎启动时读取这些以 `SYS` 开头的系统表，然后填充到这些以 `INNODB_SYS` 开头的表中。

6.3.InnoDB 的 Buffer Pool

6.3.1. 缓存的重要性

我们知道，对于使用 InnoDB 作为存储引擎的表来说，不管是用于存储用户数据的索引（包括聚簇索引和二级索引），还是各种系统数据，都是以页的形式存放在表空间中的，而所谓的表空间只不过是 InnoDB 对文件系统上一个或几个实际文件的抽象，也就是说我们的数据说到底还是存储在磁盘上的。

但是磁盘的速度慢，所以 InnoDB 存储引擎在处理客户端的请求时，当需要访问某个页的数据时，就会把完整的页的数据全部加载到内存中，也就是说即使我们只需要访问一个页的一条记录，那也需要先把整个页的数据加载到内存中。将整个页加载到内存中后就可以进行读写访问了，在进行完读写访问之后并不着急将该页对应的内存空间释放掉，而是将其缓存起来，这样将来有请求再次访问该页面时，就可以省去磁盘 IO 的开销了。

6.3.2. Buffer Pool

InnoDB 为了缓存磁盘中的页，在 MySQL 服务器启动的时候就向操作系统申请了一片连续的内存，他们给这片内存起了个名，叫做 Buffer Pool（中文名是缓冲池）。那它有多大呢？这个其实看我们机器的配置，默认情况下 Buffer Pool 只有 128M 大小，这个值其实是偏小的。

show variables like 'innodb_buffer_pool_size';

```
mysql> show variables like 'innodb_buffer_pool_size';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_buffer_pool_size | 134217728 |
+-----+-----+
1 row in set (0.01 sec)
```

可以在启动服务器的时候配置 innodb_buffer_pool_size 参数的值，它表示 Buffer Pool 的大小，就像这样：

[server]

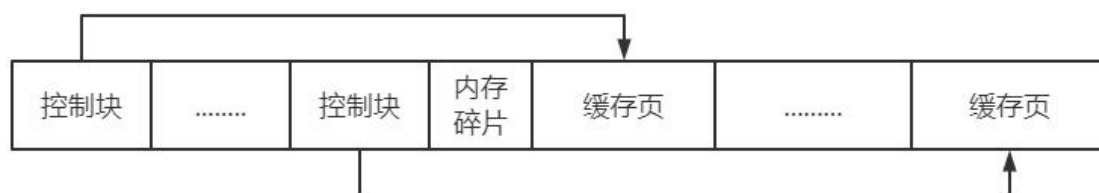
innodb_buffer_pool_size = 268435456

其中，268435456 的单位是字节，也就是指定 Buffer Pool 的大小为 256M。需要注意的是，Buffer Pool 也不能太小，最小值为 5M(当小于该值时会自动设置成 5M)。

Buffer Pool 内部组成

Buffer Pool 中默认的缓存页大小和在磁盘上默认的页大小是一样的，都是 16KB。为了更好的管理这些在 Buffer Pool 中的缓存页，InnoDB 为每一个缓存页都创建了一些所谓的控制信息，这些控制信息包括该页所属的表空间编号、页号、缓存页在 Buffer Pool 中的地址、链表节点信息、一些锁信息以及 LSN 信息，当然还有一些别的控制信息。

每个缓存页对应的控制信息占用的内存大小是相同的，我们称为控制块。控制块和缓存页是一一对应的，它们都被存放到 Buffer Pool 中，其中控制块被存放到 Buffer Pool 的前边，缓存页被存放到 Buffer Pool 后边，所以整个 Buffer Pool 对应的内存空间看起来就是这样的：

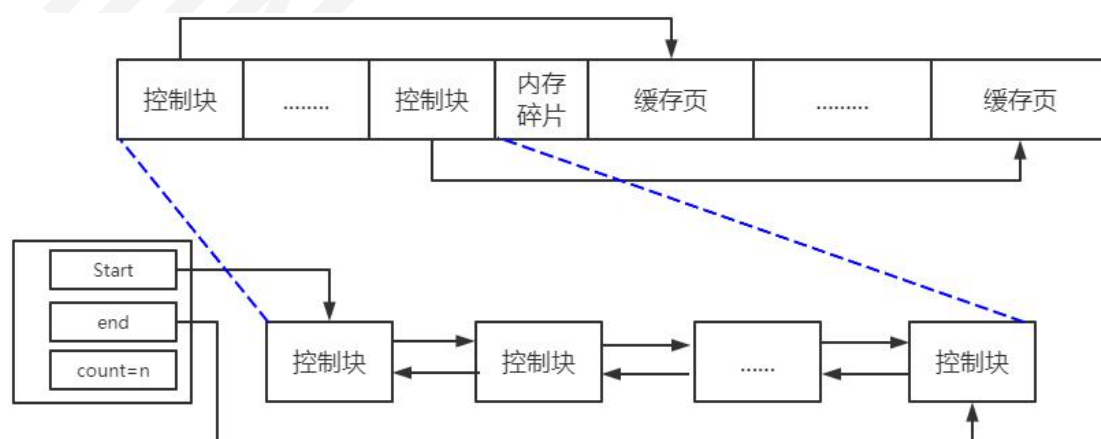


每个控制块大约占用缓存页大小的 5%，而我们设置的 `innodb_buffer_pool_size` 并不包含这部分控制块占用的内存空间大小，也就是说 InnoDB 在为 Buffer Pool 向操作系统申请连续的内存空间时，这片连续的内存空间一般会比 `innodb_buffer_pool_size` 的值大 5% 左右。

6.3.2.1. free 链表的管理

最初启动 MySQL 服务器的时候，需要完成对 Buffer Pool 的初始化过程，就是先向操作系统申请 Buffer Pool 的内存空间，然后把它划分成若干对控制块和缓存页。但是此时并没有真实的磁盘页被缓存到 Buffer Pool 中（因为还没有用到），之后随着程序的运行，会不断的有磁盘上的页被缓存到 Buffer Pool 中。

那么问题来了，从磁盘上读取一个页到 Buffer Pool 中的时候该放到哪个缓存页的位置呢？或者说怎么区分 Buffer Pool 中哪些缓存页是空闲的，哪些已经被使用了呢？最好在某个地方记录一下 Buffer Pool 中哪些缓存页是可用的，这个时候缓存页对应的控制块就派上大用场了，我们可以把所有空闲的缓存页对应的控制块作为一个节点放到一个链表中，这个链表也可以被称作 free 链表（或者说空闲链表）。刚刚完成初始化的 Buffer Pool 中所有的缓存页都是空闲的，所以每一个缓存页对应的控制块都会被加入到 free 链表中，假设该 Buffer Pool 中可容纳的缓存页数量为 n ，那增加了 free 链表的效果图就是这样的：



有了这个 free 链表之后，每当需要从磁盘中加载一个页到 Buffer Pool 中时，就从 free 链表中取一个空闲的缓存页，并且把该缓存页对应的控制块的信息填上（就是该页所在的表空间、页号之类的信息），然后把该缓存页对应的 free 链表节点从链表中移除，表示该缓存页已经被使用了。

6.3.2.2. 缓存页的哈希处理

我们前边说过，当我们需要访问某个页中的数据时，就会把该页从磁盘加载到 Buffer Pool 中，如果该页已经在 Buffer Pool 中的话直接使用就可以了。那么问题也就来了，我们怎么知道该页在不在 Buffer Pool 中呢？难不成需要依次遍历 Buffer Pool 中各个缓存页么？

我们其实是根据表空间号 + 页号来定位一个页的，也就相当于表空间号 + 页号是一个 key，缓存页就是对应的 value，怎么通过一个 key 来快速找着一个 value 呢？

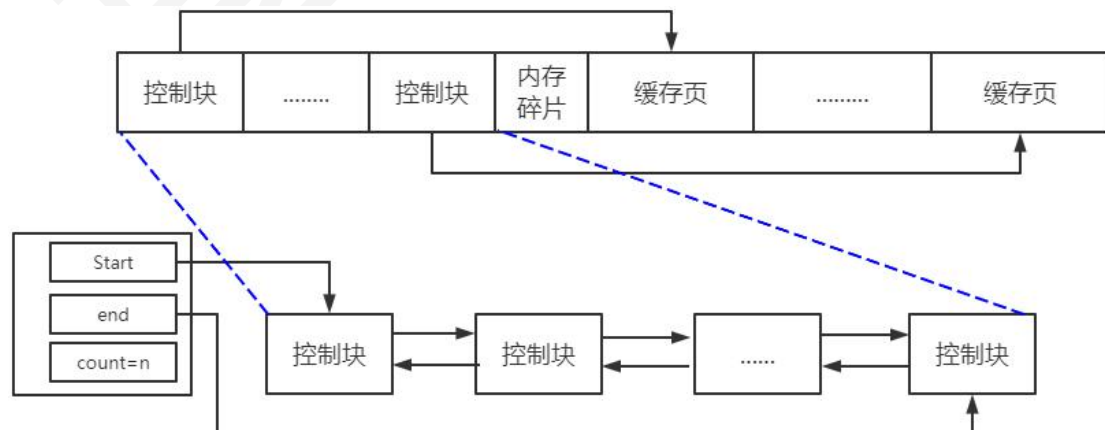
所以我们可以用表空间号 + 页号作为 key，缓存页作为 value 创建一个哈希表，在需要访问某个页的数据时，先从哈希表中根据表空间号 + 页号看看有没有对应的缓存页，如果有，直接使用该缓存页就好，如果没有，那就从 free 链表中选一个空闲的缓存页，然后把磁盘中对应的页加载到该缓存页的位置。

6.3.2.3. flush 链表的管理

如果我们修改了 Buffer Pool 中某个缓存页的数据，那它就和磁盘上的页不一致了，这样的缓存页也被称为脏页（英文名：dirty page）。当然，最简单的做法就是每发生一次修改就立即同步到磁盘上对应的页上，但是频繁的往磁盘上写数据会严重的影响程序的性能。所以每次修改缓存页后，我们并不着急立即把修改同步到磁盘上，而是在未来的某个时间点进行同步。

但是如果立即同步到磁盘的话，那之后再同步的时候我们怎么知道 Buffer Pool 中哪些页是脏页，哪些页从来没被修改过呢？总不能把所有的缓存页都同步到磁盘上吧，假如 Buffer Pool 被设置的很大，比方说 300G，那一次性同步会非常慢。

所以，需要再创建一个存储脏页的链表，凡是修改过的缓存页对应的控制块都会作为一个节点加入到一个链表中，因为这个链表节点对应的缓存页都是需要被刷新到磁盘上的，所以也叫 flush 链表。链表的构造和 free 链表差不多。



6.3.2.4. LRU 链表的管理

缓存不够的窘境

Buffer Pool 对应的内存大小毕竟是有限的，如果需要缓存的页占用的内存大小超过了 Buffer Pool 大小，也就是 free 链表中已经没有多余的空闲缓存页的时候该咋办？当然是把某些旧的缓存页从 Buffer Pool 中移除，然后再把新的页放进来，那么问题来了，移除哪些缓存页呢？

为了回答这个问题，我们还需要回到我们设立 Buffer Pool 的初衷，我们就是想减少和磁盘的 IO 交互，最好每次在访问某个页的时候它都已经被缓存到 Buffer Pool 中了。假设我们一共访问了 n 次页，那么被访问的页已经在缓存中的次数除以 n 就是所谓的缓存命中率，我们的期望就是让缓存命中率越高越好。

从这个角度出发，回想一下我们的微信聊天列表，排在前边的都是最近很频繁使用的，排在后边的自然就是最近很少使用的，假如列表能容纳下的联系人有限，你是会把最近很频繁使用的留下还是最近很少使用的留下呢？当然是留下最近很频繁使用的了。

简单的 LRU 链表

管理 Buffer Pool 的缓存页其实也是这个道理，当 Buffer Pool 中不再有空闲的缓存页时，就需要淘汰掉部分最近很少使用的缓存页。不过，我们怎么知道哪些缓存页最近频繁使用，哪些最近很少使用呢？

再创建一个链表，由于这个链表是为了按照最近最少使用的原则去淘汰缓存页的，所以这个链表可以被称为 LRU 链表（LRU 的英文全称：Least Recently Used）。当我们需要访问某个页时，可以这样处理 LRU 链表：

如果该页不在 Buffer Pool 中，在把该页从磁盘加载到 Buffer Pool 中的缓存页时，就把该缓存页对应的控制块作为节点塞到 LRU 链表的头部。

如果该页已经缓存在 Buffer Pool 中，则直接把该页对应的控制块移动到 LRU 链表的头部。

也就是说：只要我们使用到某个缓存页，就把该缓存页调整到 LRU 链表的头部，这样 LRU 链表尾部就是最近最少使用的缓存页。所以当 Buffer Pool 中的空闲缓存页使用完时，到 LRU 链表的尾部找些缓存页淘汰就行了。

划分区域的 LRU 链表

但是这种实现存在两种比较尴尬的情况：

情况一：InnoDB 提供了预读（英文名：read ahead）。所谓预读，就是 InnoDB 认为执行当前的请求可能之后会读取某些页面，就预先把它们加载到 Buffer Pool 中。根据触发方式的不同，预读又可以细分为下边两种：

线性预读

InnoDB 提供了一个系统变量 `innodb_read_ahead_threshold`，如果顺序访问了某个区（extent）的页面超过这个系统变量的值，就会触发一次异步读取下一个区中全部的页面到 Buffer Pool 的请求。

这个 `innodb_read_ahead_threshold` 系统变量的值默认是 56，我们可以在服务器启动时通过启动参数或者服务器运行过程中直接调整该系统变量的值。

随机预读

如果 Buffer Pool 中已经缓存了某个区的 13 个连续的页面，不论这些页面是不是顺序读取的，都会触发一次异步读取本区中所有其他的页面到 Buffer Pool 的请求。InnoDB 同时提供了 `innodb_random_read_ahead` 系统变量，它的默认值为 OFF。

如果预读到 Buffer Pool 中的页成功的被使用到，那就可以极大的提高语句执行的效率。可是如果用不到呢？这些预读的页都会放到 LRU 链表的头部，但是如果此时 Buffer Pool 的容量不太大而且很多预读的页面都没有用到的话，这就会导致处在 LRU 链表尾部的一些缓存页会很快的被淘汰掉，也就是所谓的劣币驱逐良币，会大大降低缓存命中率。

情况二：应用程序可能会写一些需要扫描全表的查询语句（比如没有建立合适的索引或者压根儿没有 WHERE 子句的查询）。

扫描全表意味着什么？意味着将访问到该表所在的所有页！假设这个表中记录非常多的话，那该表会占用特别多的页，当需要访问这些页时，会把它们统统都加载到 Buffer Pool 中，这也就意味着 Buffer Pool 中的所有页都被换了一次血，其他查询语句在执行时又得执行一次从磁盘加载到 Buffer Pool 的操作。而这种全表扫描的语句执行的频率也不高，每次执行都要把 Buffer Pool 中的缓存页换一次血，这严重的影响到其他查询对 Buffer Pool 的使用，从而大大降低了缓存命中率。

总结一下上边说的可能降低 Buffer Pool 的两种情况：

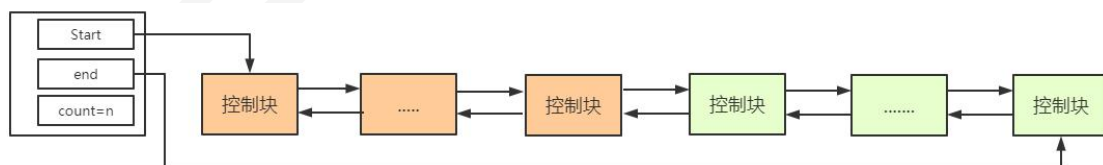
加载到 Buffer Pool 中的页不一定被用到。

如果非常多的使用频率偏低的页被同时加载到 Buffer Pool 时，可能会把那些使用频率非常高的页从 Buffer Pool 中淘汰掉。

因为有两种情况的存在，所以 InnoDB 把这个 LRU 链表按照一定比例分成两截，分别是：

一部分存储使用频率非常高的缓存页，所以这一部分链表也叫做热数据，或者称 young 区域。

另一部分存储使用频率不是很高的缓存页，所以这一部分链表也叫做冷数据，或者称 old 区域。



我们是按照某个比例将 LRU 链表分成两半的，不是某些节点固定是 young 区域的，某些节点固定是 old 区域的，随着程序的运行，某个节点所属的区域也可能发生变化。那这个划分成两截的比例怎么确定呢？对于 InnoDB 存储引擎来说，我们可以通过查看系统变量 `innodb_old_blocks_pct` 的值来确定 old 区域在 LRU 链表中所占的比例，比方说这样：

```
SHOW VARIABLES LIKE 'innodb_old_blocks_pct';
```

```
mysql> SHOW VARIABLES LIKE 'innodb_old_blocks_pct';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_old_blocks_pct | 37    |
+-----+-----+
1 row in set (0.01 sec)
```

从结果可以看出来，默认情况下，old 区域在 LRU 链表中所占的比例是 37%，也就是说 old 区域大约占 LRU 链表的 3/8。这个比例我们是设置的，我们可以在启动时修改 innodb_old_blocks_pct 参数来控制 old 区域在 LRU 链表中所占的比例。在服务器运行期间，我们也可以修改这个系统变量的值，不过需要注意的是，这个系统变量属于全局变量。

有了这个被划分成 young 和 old 区域的 LRU 链表之后，InnoDB 就可以针对我们上边提到的两种可能降低缓存命中率的情况进行优化了：

针对预读的页面可能不进行后续访问情况的优化：

InnoDB 规定，当磁盘上的某个页面在初次加载到 Buffer Pool 中的某个缓存页时，该缓存页对应的控制块会被放到 old 区域的头部。这样针对预读到 Buffer Pool 却不进行后续访问的页面就会被逐渐从 old 区域逐出，而不会影响 young 区域中被使用比较频繁的缓存页。

针对全表扫描时，短时间内访问大量使用频率非常低的页面情况的优化：

在进行全表扫描时，虽然首次被加载到 Buffer Pool 的页被放到了 old 区域的头部，但是后续会被马上访问到，每次进行访问的时候又会把该页放到 young 区域的头部，这样仍然会把那些使用频率比较高的页面给顶下去。

有同学会想：可不可以在第一次访问该页面时不将其从 old 区域移动到 young 区域的头部，后续访问时再将其移动到 young 区域的头部。回答是：行不通！因为 InnoDB 规定每次去页面中读取一条记录时，都算是访问一次页面，而一个页面中可能会包含很多条记录，也就是说读取完某个页面的记录就相当于访问了这个页面好多次。

全表扫描有一个特点，那就是它的执行频率非常低，出现了全表扫描的语句也是我们应该尽快优化的对象。而且在执行全表扫描的过程中，即使某个页面中有很多条记录，也就是去多次访问这个页面所花费的时间也是非常少的。

所以在对某个处在 old 区域的缓存页进行第一次访问时就在它对应的控制块中记录下来这个访问时间，如果后续的访问时间与第一次访问的时间在某个时间间隔内，那么该页面就不会被从 old 区域移动到 young 区域的头部，否则将它移动到 young 区域的头部。上述的这个间隔时间是由系统变量 innodb_old_blocks_time 控制的：

```
SHOW VARIABLES LIKE 'innodb_old_blocks_time';
```

```
mysql> SHOW VARIABLES LIKE 'innodb_old_blocks_time';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_old_blocks_time | 1000 |
+-----+-----+
1 row in set (0.01 sec)
```

这个 `innodb_old_blocks_time` 的默认值是 1000，它的单位是毫秒，也就意味着对于从磁盘上被加载到 LRU 链表的 old 区域的某个页来说，如果第一次和最后一次访问该页面的时间间隔小于 1s（很明显在一次全表扫描的过程中，多次访问一个页面中的时间不会超过 1s），那么该页是不会被加入到 `young` 区域的，当然，像 `innodb_old_blocks_pct` 一样，我们也可以在服务器启动或运行时设置 `innodb_old_blocks_time` 的值，这里需要注意的是，如果我们把 `innodb_old_blocks_time` 的值设置为 0，那么每次我们访问一个页面时就会把该页面放到 `young` 区域的头部。

综上所述，正是因为将 LRU 链表划分为 `young` 和 `old` 区域这两个部分，又添加了 `innodb_old_blocks_time` 这个系统变量，才使得预读机制和全表扫描造成的缓存命中率降低的问题得到了遏制，因为用不到的预读页面以及全表扫描的页面都只会被放到 `old` 区域，而不影响 `young` 区域中的缓存页。

更进一步优化 LRU 链表

对于 `young` 区域的缓存页来说，我们每次访问一个缓存页就要把它移动到 LRU 链表的头部，这样开销是不是太大？

毕竟在 `young` 区域的缓存页都是热点数据，也就是可能被经常访问的，这样频繁的对 LRU 链表进行节点移动操作也会拖慢速度？为了解决这个问题，MySQL 中还有一些优化策略，比如只有被访问的缓存页位于 `young` 区域的 1/4 的后边，才会被移动到 LRU 链表头部，这样就可以降低调整 LRU 链表的频率，从而提升性能

还有没有什么别的针对 LRU 链表的优化措施呢？当然还有，我们这里不继续说了，更多的需要看 MySQL 的源码，但是不论怎么优化，出发点就是：尽量高效的提高 Buffer Pool 的缓存命中率。

其他的一些链表

为了更好的管理 Buffer Pool 中的缓存页，除了我们上边提到的一些措施，InnoDB 们还引进了其他的一些链表，比如 `unzip LRU` 链表用于管理解压页，`zip clean` 链表用于管理没有被解压的压缩页，`zip free` 数组中每一个元素都代表一个链表，它们组成所谓的伙伴系统来为压缩页提供内存空间等等。

6.3.2.5. 刷新脏页到磁盘

后台有专门的线程每隔一段时间负责把脏页刷新到磁盘，这样可以不影响用户线程处理正常的请求。主要有两种刷新路径：

- 1、从 LRU 链表的冷数据中刷新一部分页面到磁盘。

后台线程会定时从 LRU 链表尾部开始扫描一些页面，扫描的页面数量可以通过系统变量 `innodb_lru_scan_depth` 来指定，如果从里边儿发现脏页，会把它们刷新到磁盘。这种刷新页面的方式被称之为 `BUF_FLUSH_LRU`。

2、从 flush 链表中刷新一部分页面到磁盘。

后台线程也会定时从 flush 链表中刷新一部分页面到磁盘，刷新的速率取决于当时系统是不是很忙。这种刷新页面的方式被称之为 `BUF_FLUSH_LIST`。

有时候后台线程刷新脏页的进度比较慢，导致用户线程在准备加载一个磁盘页到 Buffer Pool 时没有可用的缓存页，这时就会尝试看看 LRU 链表尾部有没有可以直接释放掉的未修改页面，如果没有的话会不得不将 LRU 链表尾部的一个脏页同步刷新到磁盘（和磁盘交互是很慢的，这会降低处理用户请求的速度）。这种刷新单个页面到磁盘中的刷新方式被称之为 `BUF_FLUSH_SINGLE_PAGE`。

当然，有时候系统特别繁忙时，也可能出现用户线程批量的从 flush 链表中刷新脏页的情况，很显然在处理用户请求过程中去刷新脏页是一种严重降低处理速度的行为，这属于一种迫不得已的情况。

6.3.2.6. 多个 Buffer Pool 实例

我们上边说过，Buffer Pool 本质是 InnoDB 向操作系统申请的一块连续的内存空间，在多线程环境下，访问 Buffer Pool 中的各种链表都需要加锁处理，在 Buffer Pool 特别大而且多线程并发访问特别高的情况下，单一的 Buffer Pool 可能会影响请求的处理速度。所以在 Buffer Pool 特别大的时候，我们可以把它们拆分成若干个小的 Buffer Pool，每个 Buffer Pool 都称为一个实例，它们都是独立的，独立的去申请内存空间，独立的管理各种链表，所以在多线程并发访问时并不会相互影响，从而提高并发处理能力。

我们可以在服务器启动的时候通过设置 `innodb_buffer_pool_instances` 的值来修改 Buffer Pool 实例的个数

那每个 Buffer Pool 实例实际占多少内存空间呢？其实使用这个公式算出来的： $\text{innodb_buffer_pool_size} / \text{innodb_buffer_pool_instances}$

也就是总共的大小除以实例的个数，结果就是每个 Buffer Pool 实例占用的大小。

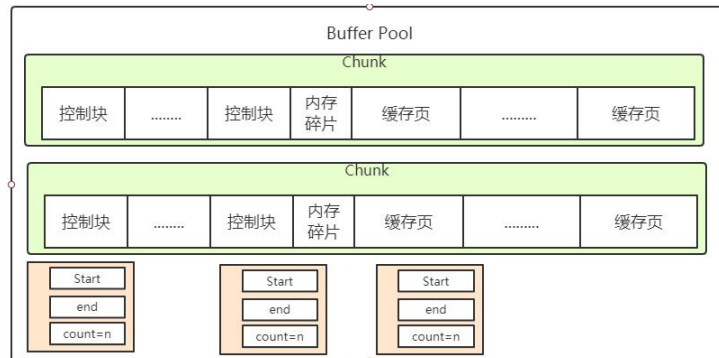
不过也不是说 Buffer Pool 实例创建的越多越好，分别管理各个 Buffer Pool 也是需要性能开销的，InnoDB 规定：当 `innodb_buffer_pool_size`（默认 128M）的值小于 1G 的时候设置多个实例是无效的，InnoDB 会默认把 `innodb_buffer_pool_instances` 的值修改为 1。所以 Buffer Pool 大于或等于 1G 的时候设置应该多个 Buffer Pool 实例。

6.3.2.7. innodb_buffer_pool_chunk_size

在 MySQL 5.7.5 之前，Buffer Pool 的大小只能在服务器启动时通过配置 `innodb_buffer_pool_size` 启动参数来调整大小，在服务器运行过程中是不允许调整该值的。不过 MySQL 在 5.7.5 以及之后的版本中支持了在服务器运行过程中调整 Buffer Pool 大小的功能，

但是有一个问题，就是每次当我们要重新调整 Buffer Pool 大小时，都需要重新向操作系统申请一块连续的内存空间，然后将旧的 Buffer Pool 中的内容复制到

这一块新空间，这是极其耗时的。所以 MySQL 决定不再一次性为某个 Buffer Pool 实例向操作系统申请一大片连续的内存空间，而是以一个所谓的 chunk 为单位向操作系统申请空间。也就是说一个 Buffer Pool 实例其实是由若干个 chunk 组成的，一个 chunk 就代表一片连续的内存空间，里边儿包含了若干缓存页与其对应的控制块：



正是因为发明了这个 chunk 的概念，我们在服务器运行期间调整 Buffer Pool 的大小时就是以 chunk 为单位增加或者删除内存空间，而不需要重新向操作系统申请一片大的内存，然后进行缓存页的复制。这个所谓的 chunk 的大小是我们在启动操作 MySQL 服务器时通过 `innodb_buffer_pool_chunk_size` 启动参数指定的，它的默认值是 134217728，也就是 128M。不过需要注意的是，`innodb_buffer_pool_chunk_size` 的值只能在服务器启动时指定，在服务器运行过程中是不可以修改的。

Buffer Pool 的缓存页除了用来缓存磁盘上的页面以外，还可以存储锁信息、自适应哈希索引等信息。

6.3.2.8. 查看 Buffer Pool 的状态信息

MySQL 给我们提供了 `SHOW ENGINE INNODB STATUS` 语句来查看关于 InnoDB 存储引擎运行过程中的一些状态信息，其中就包括 Buffer Pool 的一些信息，我们看一下（为了突出重点，我们只把输出中关于 Buffer Pool 的部分提取了出来）：

SHOW ENGINE INNODB STATUS\G

```
-----
BUFFER POOL AND MEMORY
-----
Total large memory allocated 137428992
Dictionary memory allocated 183405
Buffer pool size      8191
Free buffers          6958
Database pages        1196
Old database pages    421
Modified db pages     0
Pending reads         0
Pending writes: LRU 0, flush list 0, single page 0
Pages made young 0, not young 0
0.00 youngs/s, 0.00 non-youngs/s
Pages read 292, created 904, written 4224
0.00 reads/s, 0.00 creates/s, 0.00 writes/s
No buffer pool page gets since the last printout
Pages read ahead 0.00/s, evicted without access 0.00/s, Random read ahead 0.00/s
LRU len: 1196, unzip LRU len: 0
I/O sum[0]:cur[0], unzip sum[0]:cur[0]
-----
```

这里边的每个值都代表什么意思如下，知道即可：

Total memory allocated: 代表 Buffer Pool 向操作系统申请的连续内存空间大小，包括全部控制块、缓存页、以及碎片的大小。

Dictionary memory allocated: 为数据字典信息分配的内存空间大小，注意这个内存空间和 Buffer Pool 没啥关系，不包括在 Total memory allocated 中。

Buffer pool size: 代表该 Buffer Pool 可以容纳多少缓存页，注意，单位是页！

Free buffers: 代表当前 Buffer Pool 还有多少空闲缓存页，也就是 free 链表中还有多少个节点。

Database pages: 代表 LRU 链表中的页的数量，包含 young 和 old 两个区域的节点数量。

Old database pages: 代表 LRU 链表 old 区域的节点数量。

Modified db pages: 代表脏页数量，也就是 flush 链表中节点的数量。

Pending reads: 正在等待从磁盘上加载到 Buffer Pool 中的页面数量。

当准备从磁盘中加载某个页面时，会先为这个页面在 Buffer Pool 中分配一个缓存页以及它对应的控制块，然后把这个控制块添加到 LRU 的 old 区域的头部，但是这个时候真正的磁盘页并没有被加载进来，Pending reads 的值会跟着加 1。

Pending writes LRU: 即将从 LRU 链表中刷新到磁盘中的页面数量。

Pending writes flush list: 即将从 flush 链表中刷新到磁盘中的页面数量。

Pending writes single page: 即将以单个页面的形式刷新到磁盘中的页面数量。

Pages made young: 代表 LRU 链表中曾经从 old 区域移动到 young 区域头部的节点数量。

Page made not young: 在将 innodb_old_blocks_time 设置的值大于 0 时，首次访问或者后续访问某个处在 old 区域的节点时由于不符合时间间隔的限制而不能将其移动到 young 区域头部时，Page made not young 的值会加 1。

young/s: 代表每秒从 old 区域被移动到 young 区域头部的节点数量。

non-young/s: 代表每秒由于不满足时间限制而不能从 old 区域移动到 young 区域头部的节点数量。

Pages read、created、written: 代表读取，创建，写入了多少页。后边跟着读取、创建、写入的速率。

Buffer pool hit rate: 表示在过去某段时间，平均访问 1000 次页面，有多少次该页面已经被缓存到 Buffer Pool 了。

young-making rate: 表示在过去某段时间，平均访问 1000 次页面，有多少次访问使页面移动到 young 区域的头部了。

not (young-making rate): 表示在过去某段时间，平均访问 1000 次页面，有多少次访问没有使页面移动到 young 区域的头部。

LRU len: 代表 LRU 链表中节点的数量。

unzip_LRU: 代表 unzip_LRU 链表中节点的数量。

I/O sum: 最近 50s 读取磁盘页的总数。

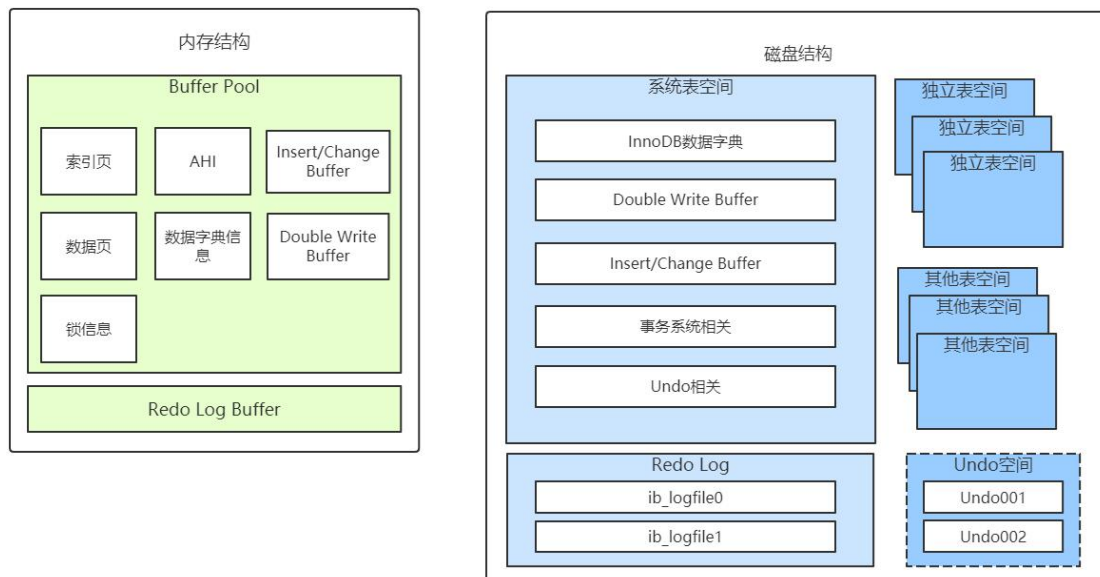
I/O cur: 现在正在读取的磁盘页数量。

I/O unzip sum: 最近 50s 解压的页面数量。

I/O unzip cur: 正在解压的页面数量。

6.3.3. InnoDB 的内存结构总结

InnoDB 的内存结构和磁盘存储结构图总结如下：



其中的 Insert/Change Buffer 主要是用于对二级索引的写入优化，Undo 空间则是 undo 日志一般放在系统表空间，但是通过参数配置后，也可以用独立表空间存放，所以用虚线表示。

7.事务的底层原理和 MVCC

在事务的实现机制上，MySQL 采用的是 WAL（Write-ahead logging，预写式日志）机制来实现的。

在使用 WAL 的系统中，所有的修改都先被写入到日志中，然后再被应用到系统中。通常包含 redo 和 undo 两部分信息。

为什么需要使用 WAL，然后包含 redo 和 undo 信息呢？举个例子，如果一个系统直接将变更应用到系统状态中，那么在机器掉电重启之后系统需要知道操作是成功了，还是只有部分成功或者是失败了（为了恢复状态）。如果使用了 WAL，那么在重启之后系统可以通过比较日志和系统状态来决定是继续完成操作还是撤销操作。

redo log 称为重做日志，每当有操作时，在数据变更之前将操作写入 redo log，这样当发生掉电之类的情況时系统可以在重启后继续操作。

undo log 称为撤销日志，当一些变更执行到一半无法完成时，可以根据撤销日志恢复到变更之间的状态。

MySQL 中用 redo log 来在系统 Crash 重启之类的情況时修复数据（事务的持久性），而 undo log 来保证事务的原子性。

7.1. redo 日志

7.1.1. redo 日志的作用

InnoDB 存储引擎是以页为单位来管理存储空间的，我们进行的增删改查操作其实本质上都是在访问页面（包括读页面、写页面、创建新页面等操作）。在 Buffer Pool 的时候说过，在真正访问页面之前，需要把在磁盘上的页缓存到内存中的 Buffer Pool 之后才可以访问。但是在事务的时候又强调过一个称之为持久性的特性，就是说对于一个已经提交的事务，在事务提交后即使系统发生了崩溃，这个事务对数据库中所做的更改也不能丢失。

如果我们只在内存的 Buffer Pool 中修改了页面，假设在事务提交后突然发生了某个故障，导致内存中的数据都失效了，那么这个已经提交了的事务对数据库中所做的更改也就跟着丢失了，这是我们所不能忍受的。那么如何保证这个持久性呢？一个很简单的做法就是在事务提交完成之前把该事务所修改的所有页面都刷新到磁盘，但是这个简单粗暴的做法有些问题：

刷新一个完整的数据页太浪费了

有时候我们仅仅修改了某个页面中的一个字节，但是我们知道在 InnoDB 中是以页为单位来进行磁盘 IO 的，也就是说我们在该事务提交时不得不将一个完整的页面从内存中刷新到磁盘，我们又知道一个页面默认是 16KB 大小，只修改一个字节就要刷新 16KB 的数据到磁盘上显然是太浪费了。

随机 IO 刷起来比较慢

一个事务可能包含很多语句，即使是一条语句也可能修改许多页面，该事务修改的这些页面可能并不相邻，这就意味着在将某个事务修改的 Buffer Pool 中的页面刷新到磁盘时，需要进行很多的随机 IO，随机 IO 比顺序 IO 要慢，尤其对于传统的机械硬盘来说。

怎么办呢？我们只是想让已经提交了的事务对数据库中数据所做的修改永久生效，即使后来系统崩溃，在重启后也能把这种修改恢复出来。所以我们其实没有必要在每次事务提交时就把该事务在内存中修改过的全部页面刷新到磁盘，只需要把修改了哪些东西记录一下就好，比方说某个事务将系统表空间中的第 100 号页面中偏移量为 1000 处的那个字节的值 1 改成 2 我们只需要记录一下：

将第 0 号表空间的 100 号页面的偏移量为 1000 处的值更新为 2。

这样我们在事务提交时，把上述内容刷新到磁盘中，即使之后系统崩溃了，重启之后只要按照上述内容所记录的步骤重新更新一下数据页，那么该事务对数据库中所做的修改又可以被恢复出来，也就意味着满足持久性的要求。因为在系统崩溃重启时需要按照上述内容所记录的步骤重新更新数据页，所以上述内容也被称之为重做日志，英文名为 redo log，也可以称之为 redo 日志。与在事务提交时将所有修改过的内存中的页面刷新到磁盘中相比，只将该事务执行过程中产生的 redo 日志刷新到磁盘的好处如下：

1、redo 日志占用的空间非常小

存储表空间 ID、页号、偏移量以及需要更新的值所需的存储空间是很小的。

2、redo 日志是顺序写入磁盘的

在执行事务的过程中，每执行一条语句，就可能产生若干条 redo 日志，这些日志是按照产生的顺序写入磁盘的，也就是使用顺序 IO。

7.1.2. redo 日志格式

通过上边的内容我们知道，redo 日志本质上只是记录了一下事务对数据库做了哪些修改。InnoDB 们针对事务对数据库的不同修改场景定义了多种类型的 redo 日志，但是绝大部分类型的 redo 日志都有下边这种通用的结构：

type	space ID	page Number	data
------	----------	-------------	------

各个部分的详细释义如下：

type: 该条 redo 日志的类型，redo 日志设计大约有 53 种不同的类型日志。

space ID: 表空间 ID。

page number: 页号。

data: 该条 redo 日志的具体内容。

7.1.2.1. 简单的 redo 日志类型

我们用一个简单的例子来说明最基本的 redo 日志类型。我们前边介绍 InnoDB 的记录行格式的时候说过，如果我们没有为某个表显式的定义主键，并且表中也没有定义 Unique 键，那么 InnoDB 会自动的为表添加一个称之为 row_id 的隐藏列作为主键。为这个 row_id 隐藏列赋值的方式如下：

服务器会在内存中维护一个全局变量，每当向某个包含隐藏的 row_id 列的表中插入一条记录时，就会把该变量的值当作新记录的 row_id 列的值，并且把该变量自增 1。

每当这个变量的值为 256 的倍数时，就会将该变量的值刷新到系统表空间的页号为 7 的页面中一个称之为 Max Row ID 的属性处。

当系统启动时，会将上边提到的 Max Row ID 属性加载到内存中，将该值加上 256 之后赋值给我们前边提到的全局变量。

这个 Max Row ID 属性占用的存储空间是 8 个字节，当某个事务向某个包含 row_id 隐藏列的表插入一条记录，并且为该记录分配的 row_id 值为 256 的倍数时，就会向系统表空间页号为 7 的页面的相应偏移量处写入 8 个字节的值。但是我们要知道，这个写入实际上是在 Buffer Pool 中完成的，我们需要为这个页面的修改记录一条 redo 日志，以便在系统崩溃后能将已经提交的该事务对该页面所做的修改恢复出来。这种情况下对页面的修改是极其简单的，redo 日志中只需要记录一下在某个页面的某个偏移量处修改了几个字节的值，具体被修改的内容是啥就好了，InnoDB 把这种极其简单的 redo 日志称之为物理日志，并且根据在页面中写入数据的多少划分了几种不同的 redo 日志类型：

MLOG_1BYTE (type 字段对应的十进制数字为 1)：表示在页面的某个偏移量处写入 1 个字节的 redo 日志类型。

MLOG_2BYTE (type 字段对应的十进制数字为 2)：表示在页面的某个偏移量处写入 2 个字节的 redo 日志类型。

MLOG_4BYTE (type 字段对应的十进制数字为 4)：表示在页面的某个偏移量处写入 4 个字节的 redo 日志类型。

MLOG_8BYTE (type 字段对应的十进制数字为 8)：表示在页面的某个偏移量处写入 8 个字节的 redo 日志类型。

MLOG_WRITE_STRING (type 字段对应的十进制数字为 30)：表示在页面的某个偏移量处写入一串数据。

我们上边提到的 Max Row ID 属性实际占用 8 个字节的存储空间，所以在修改页面中的该属性时，会记录一条类型为 MLOG_8BYTE 的 redo 日志，MLOG_8BYTE 的 redo 日志结构如下所示：

offset 代表在页面中的偏移量。

type	space ID	page Number	offset	data
------	----------	-------------	--------	------

其余 MLOG_1BYTE、MLOG_2BYTE、MLOG_4BYTE 类型的 redo 日志结构和 MLOG_8BYTE 的类似，只不过具体数据中包含对应个字节的数据罢了。

MLOG_WRITE_STRING 类型的 redo 日志表示写入一串数据，但是因为不能确定写入的具体数据占用多少字节，所以需要在日志结构中还会多一个 len 字段。

7.1.2.2. 复杂一些的 redo 日志类型

有时候执行一条语句会修改非常多的页面，包括系统数据页面和用户数据页面（用户数据指的就是聚簇索引和二级索引对应的 B+树）。以一条 INSERT 语句为例，它除了要向 B+树的页面中插入数据，也可能更新系统数据 Max Row ID 的值，不过对于我们用户来说，平时更关心的是语句对 B+树所做更新：

表中包含多少个索引，一条 INSERT 语句就可能更新多少棵 B+树。

针对某一棵 B+树来说，既可能更新叶子节点页面，也可能更新非叶子节点页面，也可能创建新的页面（在该记录插入的叶子节点的剩余空间比较少，不足以存放该记录时，会进行页面的分裂，在非叶子节点页面中添加目录项记录）。

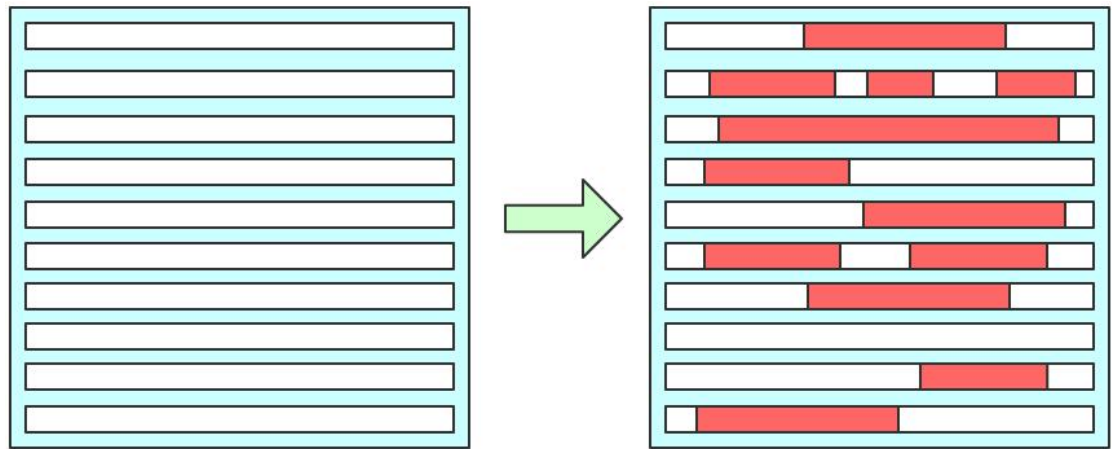
在语句执行过程中，INSERT 语句对所有页面的修改都得保存到 redo 日志中去。实现起来是非常麻烦的，比方说将记录插入到聚簇索引中时，如果定位到的叶子节点的剩余空间足够存储该记录时，那么只更新该叶子节点页面就好，那么只记录一条 MLOG_WRITE_STRING 类型的 redo 日志，表明在页面的某个偏移量处增加了哪些数据就好了么？

别忘了在一个数据页中除了存储实际的记录之后，还有什么 File Header、Page Header、Page Directory 等等部分，所以每往叶子节点代表的数据页里插入一条记录时，还有其他很多地方会跟着更新，比如说：

可能更新 Page Directory 中的槽信息、Page Header 中的各种页面统计信息，比如槽数量可能会更改，还未使用的空间最小地址可能会更改，本页面中的记录数量可能会更改，各种信息都可能会被修改，同时数据页里的记录是按照索引列

从小到大的顺序组成一个单向链表的，每插入一条记录，还需要更新上一条记录的记录头信息中的 `next_record` 属性来维护这个单向链表。

画一个简易的示意图就像是这样：



其实说到底，把一条记录插入到一个页面时需要更改的地方非常多。这时我们如果使用上边介绍的简单的物理 redo 日志来记录这些修改时，可以有两种解决方案：

方案一：在每个修改的地方都记录一条 redo 日志。

也就是如上图所示，有多少个加粗的块，就写多少条物理 redo 日志。这样子记录 redo 日志的缺点是显而易见的，因为被修改的地方是在太多了，可能记录的 redo 日志占用的空间都比整个页面占用的空间都多了。

方案二：将整个页面的第一个被修改的字节到最后一个修改的字节之间所有的数据当成是一条物理 redo 日志中的具体数据。

从图中也可以看出来，第一个被修改的字节到最后一个修改的字节之间仍然有许多没有修改过的数据，我们把这些没有修改的数据也加入到 redo 日志中去依然很浪费。

正因为上述两种使用物理 redo 日志的方式来记录某个页面中做了哪些修改比较浪费，InnoDB 中就有非常多的 redo 日志类型来做记录。

这些类型的 redo 日志既包含物理层面的意思，也包含逻辑层面的意思，具体指：

物理层面看，这些日志都指明了对哪个表空间的哪个页进行了修改。

逻辑层面看，在系统崩溃重启时，并不能直接根据这些日志里的记载，将页面内的某个偏移量处恢复成某个数据，而是需要调用一些事先准备好的函数，执行完这些函数后才可以将页面恢复成系统崩溃前的样子。

简单来说，一个 redo 日志类型而只是把在本页面中变动（比如插入、修改）一条记录所有必备的要素记了下来，之后系统崩溃重启时，服务器会调用相关向某个页面变动（比如插入、修改）一条记录的那个函数，而 redo 日志中的那些数据就可以被当成是调用这个函数所需的参数，在调用完该函数后，页面中的相关值也就都被恢复到系统崩溃前的样子了。这就是所谓的逻辑日志的意思。

当然，如果不是为了写一个解析 redo 日志的工具或者自己开发一套 redo 日志系统的话，那就不需要去研究 InnoDB 中的 redo 日志具体格式。

大家只要记住：**redo** 日志会把事务在执行过程中对数据库所做的所有修改都记录下来，在之后系统崩溃重启后可以把事务所做的任何修改都恢复出来。

7.1.3. Mini-Transaction

7.1.3.1. 以组的形式写入 redo 日志

语句在执行过程中可能修改若干个页面。比如我们前边说的一条 **INSERT** 语句可能修改系统表空间页号为 7 的页面的 **Max Row ID** 属性（当然也可能更新别的系统页面，只不过我们没有都列举出来而已），还会更新聚簇索引和二级索引对应 **B+** 树中的页面。由于对这些页面的更改都发生在 **Buffer Pool** 中，所以在修改完页面之后，需要记录一下相应的 **redo** 日志。

在这个执行语句的过程中产生的 **redo** 日志被 **InnoDB** 人为的划分成了若干个不可分割的组，比如：

- 1、更新 **Max Row ID** 属性时产生的 **redo** 日志是不可分割的。
- 2、向聚簇索引对应 **B+** 树的页面中插入一条记录时产生的 **redo** 日志是不可分割的。
- 3、向某个二级索引对应 **B+** 树的页面中插入一条记录时产生的 **redo** 日志是不可分割的。
- 4、还有其他的一些对页面的访问操作时产生的 **redo** 日志是不可分割的....。

怎么理解这个不可分割的意思呢？我们以向某个索引对应的 **B+** 树插入一条记录为例，在向 **B+** 树中插入这条记录之前，需要先定位到这条记录应该被插入到哪个叶子节点代表的数据页中，定位到具体的数据页之后，有两种可能的情况：

情况一：该数据页的剩余的空闲空间充足，足够容纳这一条待插入记录，那么事情很简单，直接把记录插入到这个数据页中，记录一条 **redo** 日志就好了，我们把这种情况称之为**乐观插入**。

情况二：该数据页剩余的空闲空间不足，那么事情就很麻烦了，遇到这种情况要进行所谓的页分裂操作：

- 1、新建一个叶子节点；
- 2、然后把原先数据页中的一部分记录复制到这个新的数据页中；
- 3、然后再把记录插入进去，把这个叶子节点插入到叶子节点链表中；
- 4、非叶子节点中添加一条目录项记录指向这个新创建的页面；
- 5、非叶子节点空间不足，继续分裂。

很显然，这个过程要对多个页面进行修改，也就意味着会产生很多条 **redo** 日志，我们把这种情况称之为**悲观插入**。

另外，这个过程中，由于需要新申请数据页，还需要改动一些系统页面，比方说要修改各种段、区的统计信息信息，各种链表的统计信息，也会产生 **redo** 日志。

当然在乐观插入时也可能产生多条 **redo** 日志。

InnoDB 认为向某个索引对应的 B+树中插入一条记录的这个过程必须是原子的，不能说插了一半之后就停止了。比方说在悲观插入过程中，新的页面已经分配好了，数据也复制过去了，新的记录也插入到页面中了，可是没有向非叶子节点中插入一条目录项记录，这个插入过程就是不完整的，这样会形成一棵不正确的 B+树。

我们知道 redo 日志是为了在系统崩溃重启时恢复崩溃前的状态，如果在悲观插入的过程中只记录了一部分 redo 日志，那么在系统崩溃重启时会将索引对应的 B+树恢复成一种不正确的状态。

所以规定在执行这些需要保证原子性的操作时必须以组的形式来记录的 redo 日志，在进行系统崩溃重启恢复时，针对某个组中的 redo 日志，要么把全部的日志都恢复掉，要么一条也不恢复。在实现上，根据多个 redo 日志的不同，使用了特殊的 redo 日志类型作为组的结尾，来表示一组完整的 redo 日志。

7.1.3.2. Mini-Transaction 的概念

所以 MySQL 把对底层页面中的一次原子访问的过程称之为一个 Mini-Transaction，比如上边所说的修改一次 Max Row ID 的值算是一个 Mini-Transaction，向某个索引对应的 B+树中插入一条记录的过程也算是一个 Mini-Transaction。

一个所谓的 Mini-Transaction 可以包含一组 redo 日志，在进行崩溃恢复时这一组 redo 日志作为一个不可分割的整体。

一个事务可以包含若干条语句，每一条语句其实是由若干个 Mini-Transaction 组成，每一个 Mini-Transaction 又可以包含若干条 redo 日志，最终形成了一个树形结构。

7.1.4. redo 日志的写入过程

7.1.4.1. redo log block 和日志缓冲区

InnoDB 为了更好的进行系统崩溃恢复，把通过 Mini-Transaction 生成的 redo 日志都放在了大小为 512 字节的块（block）中。。

我们前边说过，为了解决磁盘速度过慢的问题而引入了 Buffer Pool。同理，写入 redo 日志时也不能直接写到磁盘上，实际上在服务器启动时就向操作系统申请了一大片称之为 redo log buffer 的连续内存空间，翻译成中文就是 redo 日志缓冲区，我们也可以简称为 log buffer。这片内存空间被划分成若干个连续的 redo log block，我们可以通过启动参数 innodb_log_buffer_size 来指定 log buffer 的大小，该启动参数的默认值为 16MB。

向 log buffer 中写入 redo 日志的过程是顺序的，也就是先往前边的 block 中写，当该 block 的空闲空间用完之后再往下一个 block 中写。

我们前边说过一个 Mini-Transaction 执行过程中可能产生若干条 redo 日志，这些 redo 日志是一个不可分割的组，所以其实并不是每生成一条 redo 日志，就将其插入到 log buffer 中，而是每个 Mini-Transaction 运行过程中产生的日志先暂

时存到一个地方，当该 Mini-Transaction 结束的时候，将过程中产生的一组 redo 日志再全部复制到 log buffer 中。

7.1.4.2. redo 日志刷盘时机

我们前边说 Mini-Transaction 运行过程中产生的一组 redo 日志在 Mini-Transaction 结束时会被复制到 log buffer 中，可是这些日志总在内存里呆着也不是个办法，在一些情况下它们会被刷新到磁盘里，比如：

1、log buffer 空间不足时，log buffer 的大小是有限的（通过系统变量 `innodb_log_buffer_size` 指定），如果不停的往这个有限大小的 log buffer 里塞入日志，很快它就会被填满。InnoDB 认为如果当前写入 log buffer 的 redo 日志量已经占满了 log buffer 总容量的大约一半左右，就需要把这些日志刷新到磁盘上。

2、事务提交时，我们前边说过之所以使用 redo 日志主要是因为它占用的空间少，还是顺序写，在事务提交时可以不把修改过的 Buffer Pool 页面刷新到磁盘，但是为了保证持久性，必须要把修改这些页面对应的 redo 日志刷新到磁盘。

3、后台有一个线程，大约每秒都会刷新一次 log buffer 中的 redo 日志到磁盘。

4、正常关闭服务器时等等。

7.1.4.3. redo 日志文件组

MySQL 的数据目录（使用 `SHOW VARIABLES LIKE 'datadir'` 查看）下默认有两个名为 `ib_logfile0` 和 `ib_logfile1` 的文件，log buffer 中的日志默认情况下就是刷新到这两个磁盘文件中。如果我们对默认的 redo 日志文件不满意，可以通过下边几个启动参数来调节：

`innodb_log_group_home_dir`，该参数指定了 redo 日志文件所在的目录，默认值就是当前的数据目录。

`innodb_log_file_size`，

该参数指定了每个 redo 日志文件的大小，默认值为 48MB，

`innodb_log_files_in_group`，该参数指定 redo 日志文件的个数，默认值为 2，最大值为 100。

所以磁盘上的 redo 日志文件可以不只有一个，而是以一个日志文件组的形式出现的。这些文件以 `ib_logfile[数字]`（数字可以是 0、1、2...）的形式进行命名。在将 redo 日志写入日志文件组时，是从 `ib_logfile0` 开始写，如果 `ib_logfile0` 写满了，就接着 `ib_logfile1` 写，同理，`ib_logfile1` 写满了就去写 `ib_logfile2`，依此类推。如果写到最后一个文件该咋办？那就重新转到 `ib_logfile0` 继续写。

7.1.4.4. redo 日志文件格式

我们前边说过 log buffer 本质上是一片连续的内存空间，被划分成了若干个 512 字节大小的 block。将 log buffer 中的 redo 日志刷新到磁盘的本质就是把 block 的镜像写入日志文件中，所以 redo 日志文件其实也是由若干个 512 字节大小的 block 组成。

redo 日志文件组中的每个文件大小都一样，格式也一样，都是由两部分组成：前 2048 个字节，也就是前 4 个 block 是用来存储一些管理信息的。

从第 2048 字节往后是用来存储 log buffer 中的 block 镜像的。

7.1.5. Log Sequence Number

自系统开始运行，就不断的在修改页面，也就意味着会不断的生成 redo 日志。redo 日志的量在不断的递增，就像人的年龄一样，自打出生起就不断递增，永远不可能缩减了。

InnoDB 为记录已经写入的 redo 日志量，设计了一个称之为 Log Sequence Number 的全局变量，翻译过来就是：日志序列号，简称 LSN。规定初始的 lsn 值为 8704（也就是一条 redo 日志也没写入时，LSN 的值为 8704）。

我们知道在向 log buffer 中写入 redo 日志时不是一条一条写入的，而是以一个 Mini-Transaction 生成的一组 redo 日志为单位进行写入的。从上边的描述中可以看出，每一组由 Mini-Transaction 生成的 redo 日志都有一个唯一的 LSN 值与其对应，LSN 值越小，说明 redo 日志产生的越早。

7.1.5.1. flushed_to_disk_lsn

redo 日志是首先写到 log buffer 中，之后才会被刷新到磁盘上的 redo 日志文件。InnoDB 中有一个称之为 buf_next_to_write 的全局变量，标记当前 log buffer 中已经有哪些日志被刷新到磁盘中了。

我们前边说 lsn 是表示当前系统中写入的 redo 日志量，这包括了写到 log buffer 而没有刷新到磁盘的日志，相应的，InnoDB 也有一个表示刷新到磁盘中的 redo 日志量的全局变量，称之为 flushed_to_disk_lsn。系统第一次启动时，该变量的值和初始的 lsn 值是相同的，都是 8704。随着系统的运行，redo 日志被不断写入 log buffer，但是并不会立即刷新到磁盘，lsn 的值就和 flushed_to_disk_lsn 的值拉开了差距。我们演示一下：

系统第一次启动后，向 log buffer 中写入了 mtr_1、mtr_2、mtr_3 这三个 mtr 产生的 redo 日志，假设这三个 mtr 开始和结束时对应的 lsn 值分别是：

mtr_1: 8716 ~ 8916

mtr_2: 8916 ~ 9948

mtr_3: 9948 ~ 10000

此时的 lsn 已经增长到了 10000，但是由于没有刷新操作，所以此时 flushed_to_disk_lsn 的值仍为 8704。

随后进行将 log buffer 中的 block 刷新到 redo 日志文件的操作，假设将 mtr_1 和 mtr_2 的日志刷新到磁盘，那么 flushed_to_disk_lsn 就应该增长 mtr_1 和 mtr_2 写入的日志量，所以 flushed_to_disk_lsn 的值增长到了 9948。

综上所述，当有新的 redo 日志写入到 log buffer 时，首先 lsn 的值会增长，但 flushed_to_disk_lsn 不变，随后随着不断有 log buffer 中的日志被刷新到磁盘上，flushed_to_disk_lsn 的值也跟着增长。如果两者的值相同时，说明 log buffer 中的所有 redo 日志都已经刷新到磁盘中了。

Tips: 应用程序向磁盘写入文件时其实是先写到操作系统的缓冲区中去，如果某个写入操作要等到操作系统确认已经写到磁盘时才返回，那需要调用一下操作系统提供的 `fsync` 函数。其实只有当系统执行了 `fsync` 函数后，`flushed_to_disk_lsn` 的值才会跟着增长，当仅仅把 `log buffer` 中的日志写入到操作系统缓冲区却没有显式的刷新到磁盘时，另外的一个称之为 `write_lsn` 的值跟着增长。

当然系统的 `LSN` 值远不止我们前面描述的 `lsn`，还有很多。

7.1.5.2. 查看系统中的各种 LSN 值

我们可以使用 `SHOW ENGINE INNODB STATUS` 命令查看当前 InnoDB 存储引擎中的各种 LSN 值的情况，比如：

SHOW ENGINE INNODB STATUS\G

```
---
LOG
---
Log sequence number 12708714
Log flushed up to   12708714
Pages flushed up to 12708714
Last checkpoint at  12708705
0 pending log flushes, 0 pending chkp writes
10 log i/o's done, 0.07 log i/o's/second
```

其中：

Log sequence number: 代表系统中的 `lsn` 值，也就是当前系统已经写入的 redo 日志量，包括写入 `log buffer` 中的日志。

Log flushed up to: 代表 `flushed_to_disk_lsn` 的值，也就是当前系统已经写入磁盘的 redo 日志量。

Pages flushed up to: 代表 flush 链表中被最早修改的那个页面对应的 `oldest_modification` 属性值。

Last checkpoint at: 当前系统的 `checkpoint_lsn` 值。

7.1.6. innodb_flush_log_at_trx_commit 的用法

我们前边说为了保证事务的持久性，用户线程在事务提交时需要将该事务执行过程中产生的所有 redo 日志都刷新到磁盘上。会很明显的降低数据库性能。如果对事务的持久性要求不是那么强烈的话，可以选择修改一个称为 `innodb_flush_log_at_trx_commit` 的系统变量的值，该变量有 3 个可选的值：

0: 当该系统变量值为 0 时，表示在事务提交时不立即向磁盘中同步 redo 日志，这个任务是交给后台线程做的。

这样很明显会加快请求处理速度，但是如果事务提交后服务器挂了，后台线程没有及时将 redo 日志刷新到磁盘，那么该事务对页面的修改会丢失。

1: 当该系统变量值为 1 时, 表示在事务提交时需要将 redo 日志同步到磁盘, 可以保证事务的持久性。1 也是 innodb_flush_log_at_trx_commit 的默认值。

```
mysql> show variables like 'innodb_flush_log_at_trx_commit';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_flush_log_at_trx_commit | 1     |
+-----+-----+
1 row in set (0.00 sec)
```

2: 当该系统变量值为 2 时, 表示在事务提交时需要将 redo 日志写到操作系统的缓冲区中, 但并不需要保证将日志真正的刷新到磁盘。

这种情况下如果数据库挂了, 操作系统没挂的话, 事务的持久性还是可以保证的, 但是操作系统也挂了的话, 那就不能保证持久性了。

7.1.7. 崩溃后的恢复

7.1.7.1. 恢复机制

在服务器不挂的情况下, redo 日志简直就是个大累赘, 不仅没用, 反而让性能变得更差。但是万一数据库挂了, 就可以在重启时根据 redo 日志中的记录就可以将页面恢复到系统崩溃前的状态。

MySQL 可以根据 redo 日志中的各种 LSN 值, 来确定恢复的起点和终点。然后将 redo 日志中的数据, 以哈希表的形式, 将一个页面下的放到哈希表的一个槽中。之后就可以遍历哈希表, 因为对同一个页面进行修改的 redo 日志都放在了一个槽里, 所以可以一次性将一个页面修复好(避免了很多读取页面的随机 IO)。并且通过各种机制, 避免无谓的页面修复, 比如已经刷新的页面, 进而提升崩溃恢复的速度。

7.1.7.2. 崩溃后的恢复为什么不用 binlog ?

1、这两者使用方式不一样

binlog 会记录表所有更改操作, 包括更新删除数据, 更改表结构等等, 主要用于人工恢复数据, 而 redo log 对于我们是不可见的, 它是 InnoDB 用于保证 crash-safe 能力的, 也就是在事务提交后 MySQL 崩溃的话, 可以保证事务的持久性, 即事务提交后其更改是永久性的。

一句话概括: binlog 是用作人工恢复数据, redo log 是 MySQL 自己使用, 用于保证在数据库崩溃时的事务持久性。

2、redo log 是 InnoDB 引擎特有的, binlog 是 MySQL 的 Server 层实现的, 所有引擎都可以使用。

3、redo log 是物理日志, 记录的是“在某个数据页上做了什么修改”, 恢复的速度更快; binlog 是逻辑日志, 记录的是这个语句的原始逻辑, 比如“给 ID=2 这的 c 字段加 1 ”;

4、redo log 是“循环写”的日志文件，redo log 只会记录未刷盘的日志，已经刷入磁盘的数据都会从 redo log 这个有限大小的日志文件里删除。binlog 是追加日志，保存的是全量的日志。

5、最重要的是，当数据库 crash 后，想要恢复未刷盘但已经写入 redo log 和 binlog 的数据到内存时，binlog 是无法恢复的。虽然 binlog 拥有全量的日志，但没有一个标志让 innoDB 判断哪些数据已经入表(写入磁盘)，哪些数据还没有。

比如，binlog 记录了两条日志：

给 ID=2 这一行的 c 字段加 1

给 ID=2 这一行的 c 字段加 1

在记录 1 入表后，记录 2 未入表时，数据库 crash。重启后，只通过 binlog 数据库无法判断这两条记录哪条已经写入磁盘，哪条没有写入磁盘，不管是两条都恢复至内存，还是都不恢复，对 ID=2 这行数据来说，都不对。

但 redo log 不一样，只要刷入磁盘的数据，都会从 redo log 中抹掉，数据库重启后，直接把 redo log 中的数据都恢复至内存就可以了。

7.2. undo 日志

7.2.1. 事务回滚的需求

我们说过事务需要保证原子性，也就是事务中的操作要么全部完成，要么什么也不做。但是偏偏有时候事务执行到一半会出现一些情况，比如：

情况一：事务执行过程中可能遇到各种错误，比如服务器本身的错误，操作系统错误，甚至是突然断电导致的错误。

情况二：程序员可以在事务执行过程中手动输入 ROLLBACK 语句结束当前的事务的执行。

这两种情况都会导致事务执行到一半就结束，但是事务执行过程中可能已经修改了很多东西，为了保证事务的原子性，我们需要把东西改回原先的样子，这个过程就称之为回滚（英文名：rollback），这样就可以造成这个事务看起来什么都没做，所以符合原子性要求。

每当我们要对一条记录做改动时（这里的改动可以指 INSERT、DELETE、UPDATE），都需要把回滚时所需的東西都给记下来。比方说：

你插入一条记录时，至少要把这条记录的主键值记下来，之后回滚的时候只需要把这个主键值对应的记录删掉。

你删除了一条记录，至少要把这条记录中的内容都记下来，这样之后回滚时再把由这些内容组成的记录插入到表中。

你修改了一条记录，至少要把修改这条记录前的旧值都记录下来，这样之后回滚时再把这条记录更新为旧值。

这些为了回滚而记录的这些东西称之为撤销日志，英文名为 undo log/undo 日志。这里需要注意的一点是，由于查询操作（SELECT）并不会修改任何用户记录，所以在查询操作执行时，并不需要记录相应的 undo 日志。

当然，在真实的 InnoDB 中，undo 日志其实并不像我们上边所说的那么简单，不同类型的操作产生的 undo 日志的格式也是不同的。

7.2.2. 事务 id

7.2.2.1. 给事务分配 id 的时机

一个事务可以是一个只读事务，或者是一个读写事务：

我们可以通过 `START TRANSACTION READ ONLY` 语句开启一个只读事务。

在只读事务中不可以对普通的表（其他事务也能访问到的表）进行增、删、改操作，但可以对用户临时表做增、删、改操作。

我们可以通过 `START TRANSACTION READ WRITE` 语句开启一个读写事务，或者使用 `BEGIN`、`START TRANSACTION` 语句开启的事务默认也算是读写事务。

在读写事务中可以对表执行增删改查操作。

如果某个事务执行过程中对某个表执行了增、删、改操作，那么 InnoDB 存储引擎就会给它分配一个独一无二的事务 id，分配方式如下：

对于只读事务来说，只有在它第一次对某个用户创建的临时表执行增、删、改操作时才会为这个事务分配一个事务 id，否则的话是不分配事务 id 的。

我们前边说过对某个查询语句执行 `EXPLAIN` 分析它的查询计划时，有时候在 `Extra` 列会看到 `Using temporary` 的提示，这个表明在执行该查询语句时会用到内部临时表。这个所谓的内部临时表和我们手动用 `CREATE TEMPORARY TABLE` 创建的用户临时表并不一样，在事务回滚时并不需要把执行 `SELECT` 语句过程中用到的内部临时表也回滚，在执行 `SELECT` 语句用到内部临时表时并不会为它分配事务 id。

对于读写事务来说，只有在它第一次对某个表（包括用户创建的临时表）执行增、删、改操作时才会为这个事务分配一个事务 id，否则的话也是不分配事务 id 的。

有的时候虽然我们开启了一个读写事务，但是在这个事务中全是查询语句，并没有执行增、删、改的语句，那也就意味着这个事务并不会被分配一个事务 id。

上边描述的事务 id 分配策略是针对 MySQL 5.7 来说的，前边的版本的分配方式可能不同。

7.2.2.2. 事务 id 生成机制

这个事务 id 本质上就是一个数字，它的分配策略和我们前边提到的对隐藏列 `row_id`（当用户没有为表创建主键和 `UNIQUE` 键时 InnoDB 自动创建的列）的分配策略大抵相同，具体策略如下：

服务器会在内存中维护一个全局变量，每当需要为某个事务分配一个事务 id 时，就会把该变量的值当作事务 id 分配给该事务，并且把该变量自增 1。

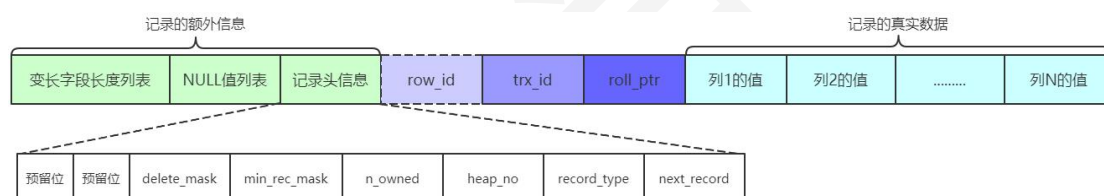
每当这个变量的值为 256 的倍数时，就会将该变量的值刷新到系统表空间的页号为 5 的页面中一个称之为 **Max Trx ID** 的属性处，这个属性占用 8 个字节的存储空间。

当系统下一次重新启动时，会将上边提到的 **Max Trx ID** 属性加载到内存中，将该值加上 256 之后赋值给我们前边提到的全局变量（因为在上次关机时该全局变量的值可能大于 **Max Trx ID** 属性值）。

这样就可以保证整个系统中分配的事务 id 值是一个递增的数字。先被分配 id 的事务得到的是较小的事务 id，后被分配 id 的事务得到的是较大的事务 id。

7.2.3. trx_id 隐藏列

我们在学习 InnoDB 记录行格式的时候重点强调过：聚簇索引的记录除了会保存完整的用户数据以外，而且还会自动添加名为 **trx_id**、**roll_pointer** 的隐藏列，如果用户没有在表中定义主键以及 **UNIQUE** 键，还会自动添加一个名为 **row_id** 的隐藏列。



其中的 **trx_id** 列就是某个对这个聚簇索引记录做改动的语句所在的事务对应的事务 id 而已（此处的改动可以是 **INSERT**、**DELETE**、**UPDATE** 操作）。至于 **roll_pointer** 隐藏列我们后边分析。

7.2.4. undo 日志的格式

为了实现事务的原子性，InnoDB 存储引擎在实际进行增、删、改一条记录时，都需要先把对应的 **undo** 日志记下来。一般每对一条记录做一次改动，就对应着一条 **undo** 日志，但在某些更新记录的操作中，也可能会对对应着 2 条 **undo** 日志。

一个事务在执行过程中可能新增、删除、更新若干条记录，也就是说需要记录很多条对应的 **undo** 日志，这些 **undo** 日志会被从 0 开始编号，也就是说根据生成的顺序分别被称为第 0 号 **undo** 日志、第 1 号 **undo** 日志、...、第 n 号 **undo** 日志等，这个编号也被称之为 **undo no**。

这些 **undo** 日志是被记录到类型为 **FIL_PAGE_UNDO_LOG** 的页面中。这些页面可以从系统表空间中分配，也可以从一种专门存放 **undo** 日志的表空间，也就是所谓的 **undo tablespace** 中分配。先来看看不同操作都会产生什么样子的 **undo** 日志。

7.2.4.1. INSERT 操作对应的undo 日志

当我们向表中插入一条记录时最终导致的结果就是这条记录被放到了一个数据页中。如果希望回滚这个插入操作，那么把这条记录删除就好了，也就是说

在写对应的 undo 日志时，主要是把这条记录的主键信息记上。InnoDB 的设计了一个类型为 TRX_UNDO_INSERT_REC 的 undo 日志。

如果记录中的主键只包含一个列，那么在类型为 TRX_UNDO_INSERT_REC 的 undo 日志中只需要把该列占用的存储空间大小和真实值记录下来，如果记录中的主键包含多个列，那么每个列占用的存储空间大小和对应的真实值都需要记录下来。

当我们向某个表中插入一条记录时，实际上需要向聚簇索引和所有的二级索引都插入一条记录。不过记录 undo 日志时，我们只需要考虑向聚簇索引插入记录时的情况就好了，因为其实聚簇索引记录和二级索引记录是一一对应的，我们在回滚插入操作时，只需要知道这条记录的主键信息，然后根据主键信息做对应的删除操作，做删除操作时就会顺带着把所有二级索引中相应的记录也删除掉。后边说到的 DELETE 操作和 UPDATE 操作对应的 undo 日志也都是针对聚簇索引记录而言的。

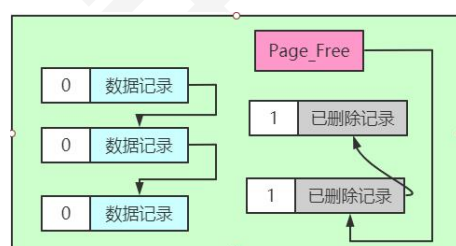
roll_pointer 的作用

roll_pointer 本质上就是一个指向记录对应的 undo 日志的一个指针。比方说我们向表里插入了 2 条记录，每条记录都有与其对应的一条 undo 日志。记录被存储到了类型为 FIL_PAGE_INDEX 的页面中（就是我们前边一直所说的数据页），undo 日志被存放到了类型为 FIL_PAGE_UNDO_LOG 的页面中。roll_pointer 本质就是一个指针，指向记录对应的 undo 日志。

7.2.4.2. DELETE 操作对应的 undo 日志

我们知道插入到页面中的记录会根据记录头信息中的 next_record 属性组成一个单向链表，我们把这个链表称之为正常记录链表；被删除的记录其实也会根据记录头信息中的 next_record 属性组成一个链表，只不过这个链表中的记录占用的存储空间可以被重新利用，所以也称这个链表为垃圾链表。Page Header 部分有一个称之为 PAGE_FREE 的属性，它指向由被删除记录组成的垃圾链表中的头节点。

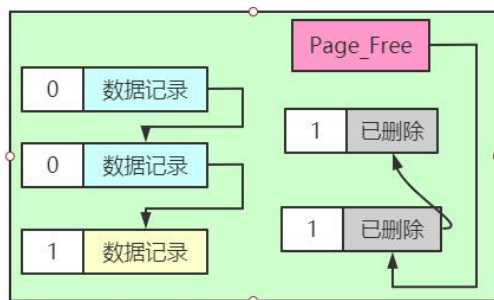
假设此刻某个页面中的记录分布情况是这样的



我们只把记录的 delete_mask 标志位展示了出来。从图中可以看出，正常记录链表中包含了 3 条正常记录，垃圾链表里包含了 2 条已删除记录。页面的 Page Header 部分的 PAGE_FREE 属性的值代表指向垃圾链表头节点的指针。

假设现在我们准备使用 DELETE 语句把正常记录链表中的最后一条记录给删除掉，其实这个删除的过程需要经历两个阶段：

阶段一：将记录的 delete_mask 标识位设置为 1，这个阶段称之为 delete mark。

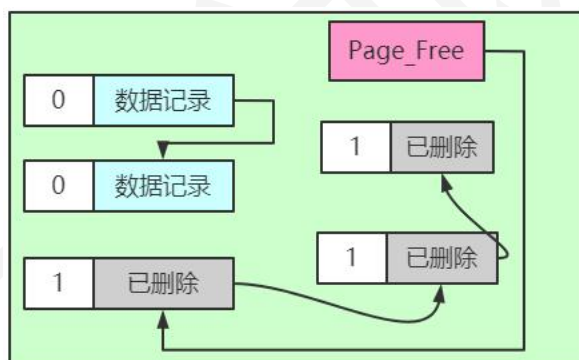


可以看到，正常记录链表中的最后一条记录的 `delete_mask` 值被设置为 1，但是并没有被加入到垃圾链表。也就是此时记录处于一个中间状态。在删除语句所在的事务提交之前，被删除的记录一直都处于这种所谓的中间状态。

为啥会有这种奇怪的中间状态呢？其实主要是为了实现一个称之为 **MVCC** 的功能，稍后再介绍。

阶段二：当该删除语句所在的事务提交之后，会有专门的线程后来真正的把记录删除掉。所谓真正的删除就是把该记录从正常记录链表中移除，并且加入到垃圾链表中，然后还要调整一些页面的其他信息，比如页面中的用户记录数量 `PAGE_N_RECS`、上次插入记录的位置 `PAGE_LAST_INSERT`、垃圾链表头节点的指针 `PAGE_FREE`、页面中可重用的字节数量 `PAGE_GARBAGE`、还有页目录的一些信息等等。这个阶段称之为 **purge**。

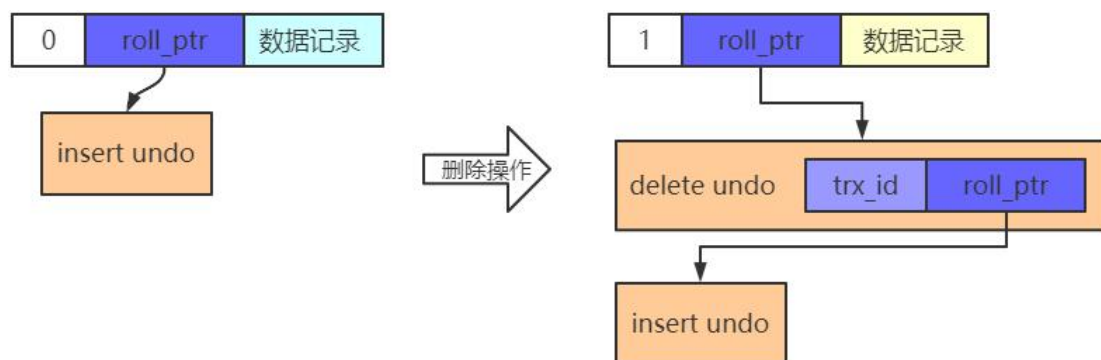
把阶段二执行完了，这条记录就算是真正的被删除掉了。这条已删除记录占用的存储空间也可以被重新利用了。



从上边的描述中我们也可以看出来，在删除语句所在的事务提交之前，只会经历阶段一，也就是 **delete mark** 阶段（提交之后我们就不用回滚了，所以只需考虑对删除操作的阶段一做的影响进行回滚）。InnoDB 中就会产生一种称之为 `TRX_UNDO_DEL_MARK_REC` 类型的 **undo** 日志。

版本链

同时，在对一条记录进行 **delete mark** 操作前，需要把该记录的旧的 `trx_id` 和 `roll_pointer` 隐藏列的值都给记到对应的 **undo** 日志中来，就是我们图中显示的 `old trx_id` 和 `old roll_pointer` 属性。这样有一个好处，那就是可以通过 **undo** 日志的 `old roll_pointer` 找到记录在修改之前对应的 **undo** 日志。比方说在一个事务中，我们先插入了一条记录，然后又执行对该记录的删除操作，这个过程的示意图就是这样：



从图中可以看出, 执行完 **delete mark** 操作后, 它对应的 **undo** 日志和 **INSERT** 操作对应的 **undo** 日志就串成了一个链表。这个链表就称之为版本链。

7.2.4.3. UPDATE 操作对应的 undo 日志

在执行 **UPDATE** 语句时, InnoDB 对更新主键和不更新主键这两种情况有截然不同的处理方案。

不更新主键的情况

在不更新主键的情况下, 又可以细分为被更新的列占用的存储空间不发生变化和发生变化的情况。

就地更新 (in-place update)

更新记录时, 对于被更新的每个列来说, 如果更新后的列和更新前的列占用的存储空间都一样大, 那么就可以进行就地更新, 也就是直接在原记录的基础上修改对应列的值。再次强调一边, 是每个列在更新前后占用的存储空间一样大, 有任何一个被更新的列更新前比更新后占用的存储空间大, 或者更新前比更新后占用的存储空间小都不能进行就地更新。

先删除掉旧记录, 再插入新记录

在不更新主键的情况下, 如果有任何一个被更新的列更新前和更新后占用的存储空间大小不一致, 那么就需要先把这条旧的记录从聚簇索引页面中删除掉, 然后再根据更新后列的值创建一条新的记录插入到页面中。

请注意一下, 我们这里所说的删除并不是 **delete mark** 操作, 而是真正的删除掉, 也就是把这条记录从正常记录链表中移除并加入到垃圾链表中, 并且修改页面中相应的统计信息 (比如 **PAGE_FREE**、**PAGE_GARBAGE** 等这些信息)。由用户线程同步执行真正的删除操作, 真正删除之后紧接着就要根据各个列更新后的值创建的新记录插入。

这里如果新创建的记录占用的存储空间大小不超过旧记录占用的空间, 那么可以直接重用被加入到垃圾链表中的旧记录所占用的存储空间, 否则的话需要在页面中新申请一段空间以供新记录使用, 如果本页面内已经没有可用的空间的话, 那就需要进行页面分裂操作, 然后再插入新记录。

针对 **UPDATE** 不更新主键的情况 (包括上边所说的就地更新和先删除旧记录再插入新记录), InnoDB 设计了一种类型为 **TRX_UNDO_UPD_EXIST_REC** 的 **undo** 日志。

更新主键的情况

在聚簇索引中，记录是按照主键值的大小连成了一个单向链表的，如果我们更新了某条记录的主键值，意味着这条记录在聚簇索引中的位置将会发生改变，比如你将记录的主键值从 1 更新为 10000，如果还有非常多的记录的主键值分布在 1~10000 之间的话，那么这两条记录在聚簇索引中就有可能离得非常远，甚至中间隔了好多个页面。针对 UPDATE 语句中更新了记录主键值的情况，InnoDB 在聚簇索引中分了两步处理：

将旧记录进行 delete mark 操作

也就是说在 UPDATE 语句所在的事务提交前，对旧记录只做一个 delete mark 操作，在事务提交后才由专门的线程做 purge 操作，把它加入到垃圾链表中。这里一定要和我们上边所说的在不更新记录主键值时，先真正删除旧记录，再插入新记录的方式区分开！

之所以只对旧记录做 delete mark 操作，是因为别的事务同时也可能访问这条记录，如果把它真正的删除加入到垃圾链表后，别的事务就访问不到了。这个功能就是所谓的 MVCC。

创建一条新记录

根据更新后各列的值创建一条新记录，并将其插入到聚簇索引中（需重新定位插入的位置）。

由于更新后的记录主键值发生了改变，所以需要重新从聚簇索引中定位这条记录所在的位置，然后把它插进去。

针对 UPDATE 语句更新记录主键值的情况，在对该记录进行 delete mark 操作前，会记录一条类型为 TRX_UNDO_DEL_MARK_REC 的 undo 日志；之后插入新记录时，会记录一条类型为 TRX_UNDO_INSERT_REC 的 undo 日志，也就是说每对一条记录的主键值做改动时，会记录 2 条 undo 日志。

7.2.5. FIL_PAGE_UNDO_LOG 页面

我们前边说明表空间的时候说过，表空间其实是由许许多多的页面构成的，页面默认大小为 16KB。这些页面有不同的类型，比如类型为 FIL_PAGE_INDEX 的页面用于存储聚簇索引以及二级索引，类型为 FIL_PAGE_TYPE_FSP_HDR 的页面用于存储表空间头部信息的，还有其他各种类型的页面，其中有一种称之为 FIL_PAGE_UNDO_LOG 类型的页面是专门用来存储 undo 日志的。

7.3. MVCC

全称 Multi-Version Concurrency Control，即多版本并发控制，主要是为了提高数据库的并发性能。

同一行数据平时发生读写请求时，会上锁阻塞住。但 MVCC 用更好的方式去处理读一写请求，做到在发生读一写请求冲突时不用加锁。

这个读是指的快照读，而不是当前读，当前读是一种加锁操作，是悲观锁。

那它到底是怎么做到读一写不用加锁的，快照读和当前读是指什么？我们后面都会学到。

7.3.1. MVCC 原理

8.3.1.1. 复习事务隔离级别

事务并发执行遇到的问题

脏读（Dirty Read）

如果一个事务读到了另一个未提交事务修改过的数据，那就意味着发生了脏读。

不可重复读（Non-Repeatable Read）

如果一个事务只能读到另一个已经提交的事务修改过的数据，并且其他事务每对该数据进行一次修改并提交后，该事务都能查询得到最新值，那就意味着发生了不可重复读。

幻读（Phantom）

如果一个事务先根据某些条件查询出一些记录，之后另一个事务又向表中插入了符合这些条件的记录，原先的事务再次按照该条件查询时，能把另一个事务插入的记录也读出来，那就意味着发生了幻读。

幻读强调的是在一个事务按照某个相同条件多次读取记录时，后读取时读到了之前没有读到的记录。

那对于先前已经读到的记录，之后又读取不到这种情况，其实这相当于对每一条记录都发生了不可重复读的现象。幻读只是重点强调了读取到了之前读取没有获取到的记录。

前面已经说过，不同的数据库厂商对 SQL 标准中规定的四种隔离级别支持不一样，比方说 Oracle 就只支持 READ COMMITTED 和 SERIALIZABLE 隔离级别。MySQL 虽然支持 4 种隔离级别，但与 SQL 标准中所规定的各级隔离级别允许发生的问题却有些出入，具体的情况如下表：

隔离级别	脏读	不可重复读	幻读
READ UNCOMMITTED 未提交读	可能	可能	可能
READ COMMITTED 已提交读	-	可能	可能
REPEATABLE READ 可重复读	-	-	-
SERIALIZABLE 可串行化	-	-	-

MySQL 在 REPEATABLE READ 隔离级别下，是可以很大程度避免幻读问题的发生的，MySQL 是怎么做到的？

7.3.1.2. 版本链

我们知道，对于使用 InnoDB 存储引擎的表来说，它的聚簇索引记录中都包含两个必要的隐藏列(row_id 并不是必要的，我们创建的表中有主键或者非 NULL 的 UNIQUE 键时都不会包含 row_id 列)：

trx_id: 每次一个事务对某条聚簇索引记录进行改动时，都会把该事务的事务 id 赋值给 trx_id 隐藏列。

roll_pointer: 每次对某条聚簇索引记录进行改动时，都会把旧的版本写入到 undo 日志中，然后这个隐藏列就相当于一个指针，可以通过它来找到该记录修改前的信息。

为了说明这个问题，我们创建一个演示表

```
CREATE TABLE teacher (  
    number INT,  
    name VARCHAR(100),  
    domain varchar(100),  
    PRIMARY KEY (number)  
) Engine=InnoDB CHARSET=utf8;
```

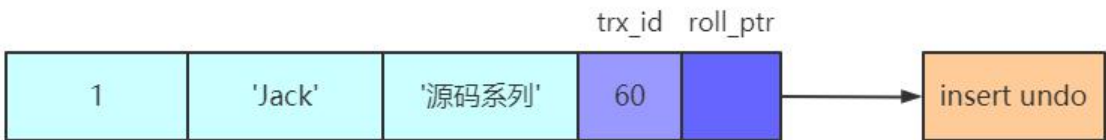
然后向这个表里插入一条数据：

```
INSERT INTO teacher VALUES(1, 'Jack', '源码系列');
```

现在表里的数据就是这样的：

```
mysql> select * from teacher;  
+-----+-----+-----+  
| number | name  | domain |  
+-----+-----+-----+  
|      1 | Jack | 源码系列 |  
+-----+-----+-----+  
1 row in set (0.00 sec)
```

假设插入该记录的事务 id 为 60，那么此刻该条记录的示意图如下所示：



假设之后两个事务 id 分别为 80、120 的事务对这条记录进行 UPDATE 操作，操作流程如下：

Trx 80	Trx 120
BEGIN	
	BEGIN
UPDATE teacher SET name = 'Mark' WHERE number = 1;	

UPDATE teacher SET name = 'James' WHERE number = 1;	
COMMIT	
	UPDATE teacher SET name = 'King' WHERE number = 1;
	UPDATE teacher SET name = '大飞' WHERE number = 1;
	COMMIT

每次对记录进行改动，都会记录一条 undo 日志，每条 undo 日志也都有一个 roll_pointer 属性（INSERT 操作对应的 undo 日志没有该属性，因为该记录并没有更早的版本），可以将这些 undo 日志都连起来，串成一个链表，所以现在的情况就像下图一样：



对该记录每次更新后，都会将旧值放到一条 undo 日志中，就算是该记录的一个旧版本，随着更新次数的增多，所有的版本都会被 roll_pointer 属性连接成一个链表，我们把这个链表称之为**版本链**，版本链的头节点就是当前记录最新的值。另外，每个版本中还包含生成该版本时对应的事务 id。于是可以利用这个记录的版本链来控制并发事务访问相同记录的行为，那么这种机制就被称之为**多版本并发控制(Mult-Version Concurrency Control MVCC)**。

7.3.1.3. ReadView

隔离级别	脏读	不可重复读	幻读
READ UNCOMMITTED 未提交读	可能	可能	可能
READ COMMITTED 已提交读	-	可能	可能
REPEATABLE READ 可重复读	-	-	—
SERIALIZABLE 可串行化	-	-	-

对于使用 READ UNCOMMITTED 隔离级别的事务来说，由于可以读到未提交事务修改过的记录，所以直接读取记录的最新版本就好了。

对于使用 **SERIALIZABLE** 隔离级别的事务来说，InnoDB 使用加锁的方式来访问记录。

对于使用 **READ COMMITTED** 和 **REPEATABLE READ** 隔离级别的事务来说，都必须保证读到已经提交了的事务修改过的记录，也就是说假如另一个事务已经修改了记录但是尚未提交，是不能直接读取最新版本的记录的，核心问题就是：**READ COMMITTED** 和 **REPEATABLE READ** 隔离级别在不可重复读和幻读上的区别是从哪里来的，其实结合前面的知识，这两种隔离级别关键是需要判断一下版本链中的哪个版本是当前事务可见的。

为此，InnoDB 提出了一个 **ReadView** 的概念，这个 **ReadView** 中主要包含 4 个比较重要的内容：

m_ids：表示在生成 **ReadView** 时当前系统中**活跃的读写事务**的事务 id 列表。

min_trx_id：表示在生成 **ReadView** 时当前系统中活跃的读写事务中最小的事务 id，也就是 **m_ids** 中的最小值。

max_trx_id：表示生成 **ReadView** 时系统中应该分配给下一个事务的 id 值。注意 **max_trx_id** 并不是 **m_ids** 中的最大值，事务 id 是递增分配的。比方说现在有 id 为 1，2，3 这三个事务，之后 id 为 3 的事务提交了。那么一个新的读事务在生成 **ReadView** 时，**m_ids** 就包括 1 和 2，**min_trx_id** 的值就是 1，**max_trx_id** 的值就是 4。

creator_trx_id：表示生成该 **ReadView** 的事务的事务 id。

有了这个 **ReadView**，这样在访问某条记录时，只需要按照下边的步骤判断记录的某个版本是否可见：

1、如果被访问版本的 **trx_id** 属性值与 **ReadView** 中的 **creator_trx_id** 值相同，意味着当前事务在访问它自己修改过的记录，所以该版本可以被当前事务访问。

2、如果被访问版本的 **trx_id** 属性值小于 **ReadView** 中的 **min_trx_id** 值，表明生成该版本的事务在当前事务生成 **ReadView** 前已经提交，所以该版本可以被当前事务访问。

3、如果被访问版本的 **trx_id** 属性值大于或等于 **ReadView** 中的 **max_trx_id** 值，表明生成该版本的事务在当前事务生成 **ReadView** 后才开启，所以该版本不可以被当前事务访问。

4、如果被访问版本的 **trx_id** 属性值在 **ReadView** 的 **min_trx_id** 和 **max_trx_id** 之间($\text{min_trx_id} < \text{trx_id} < \text{max_trx_id}$)，那就需要判断一下 **trx_id** 属性值是不是在 **m_ids** 列表中，如果在，说明创建 **ReadView** 时生成该版本的事务还是活跃的，该版本不可以被访问；如果不在，说明创建 **ReadView** 时生成该版本的事务已经被提交，该版本可以被访问。

5、如果某个版本的数据对当前事务不可见的话，那就顺着版本链找到下一个版本的数据，继续按照上边的步骤判断可见性，依此类推，直到版本链中的最后一个版本。如果最后一个版本也不可见的话，那么就意味着该条记录对该事务完全不可见，查询结果就不包含该记录。

在 MySQL 中，**READ COMMITTED** 和 **REPEATABLE READ** 隔离级别的一个非常大的区别就是它们生成 **ReadView** 的时机不同。

我们还是以表 `teacher` 为例，假设现在表 `teacher` 中只有一条由事务 id 为 60 的事务插入的一条记录，接下来看一下 `READ COMMITTED` 和 `REPEATABLE READ` 所谓的生成 `ReadView` 的时机不同到底不同在哪里。

`READ COMMITTED` —— 每次读取数据前都生成一个 `ReadView`

比方说现在系统里有两个事务 id 分别为 80、120 的事务在执行：

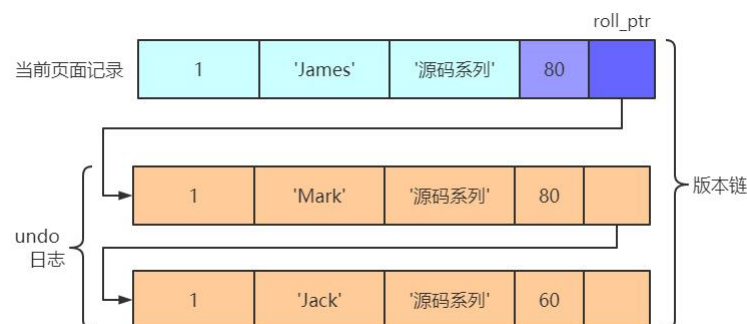
Transaction 80

`UPDATE teacher SET name = 'Mark' WHERE number = 1;`

`UPDATE teacher SET name = 'James' WHERE number = 1;`

...

此刻，表 `teacher` 中 `number` 为 1 的记录得到的版本链表如下所示：



假设现在有一个使用 `READ COMMITTED` 隔离级别的事务开始执行：

使用 `READ COMMITTED` 隔离级别的事务

`BEGIN;`

SELECE1: Transaction 80、120 未提交

`SELECT * FROM teacher WHERE number = 1;` # 得到的列 `name` 的值为 'Jack'

这个 SELECE1 的执行过程如下：

在执行 `SELECT` 语句时会先生成一个 `ReadView`，`ReadView` 的 `m_ids` 列表的内容就是 `[80, 120]`，`min_trx_id` 为 80，`max_trx_id` 为 121，`creator_trx_id` 为 0。

然后从版本链中挑选可见的记录，从图中可以看出，最新版本的列 `name` 的内容是 'James'，该版本的 `trx_id` 值为 80，在 `m_ids` 列表内，所以不符合可见性要求（4、如果被访问版本的 `trx_id` 属性值在 `ReadView` 的 `min_trx_id` 和 `max_trx_id` 之间 ($min_trx_id < trx_id < max_trx_id$)，那就需要判断一下 `trx_id` 属性值是不是在 `m_ids` 列表中，如果在，说明创建 `ReadView` 时生成该版本的事务还是活跃的，该版本不可以被访问；如果不在，说明创建 `ReadView` 时生成该版本的事务已经被提交，该版本可以被访问），根据 `roll_pointer` 跳到下一个版本。

下一个版本的列 `name` 的内容是 'Mark'，该版本的 `trx_id` 值也为 80，也在 `m_ids` 列表内，所以也不符合要求，继续跳到下一个版本。

下一个版本的列 `name` 的内容是 "Jack"，该版本的 `trx_id` 值为 60，小于 `ReadView` 中的 `min_trx_id` 值，所以这个版本是符合要求的（2、如果被访问版本的 `trx_id` 属性值小于 `ReadView` 中的 `min_trx_id` 值，表明生成该版本的事务在当

前事务生成 ReadView 前已经提交，所以该版本可以被当前事务访问），最后返回给用户的版本就是这条列 name 为"Jack"的记录。

之后，我们把事务 id 为 80 的事务提交一下，

然后再到事务 id 为 120 的事务中更新一下表 teacher 中 number 为 1 的记录：

```
# Transaction120
```

```
BEGIN;
```

```
# 更新了一些别的表的记录
```

```
UPDATE teacher SET name = 'King' WHERE number = 1;
```

```
UPDATE teacher SET name = '大飞' WHERE number = 1;
```

此刻，表 teacher 中 number 为 1 的记录的版本链就长这样：



然后再到刚才使用 READ COMMITTED 隔离级别的事务中继续查找这个 number 为 1 的记录，如下：

```
# 使用 READ COMMITTED 隔离级别的事务
```

```
BEGIN;
```

```
# SELECE1: Transaction 80、120 均未提交
```

```
SELECT * FROM teacher WHERE number = 1; # 得到的列 name 的值为"Jack"
```

```
# SELECE2: Transaction 80 提交，Transaction 120 未提交
```

```
SELECT * FROM teacher WHERE number = 1; # 得到的列 name 的值为"James"
```

这个 SELECE2 的执行过程如下：

在执行 SELECT 语句时会又会单独生成一个 ReadView，该 ReadView 的 m_ids 列表的内容就是[120]（事务 id 为 80 的那个事务已经提交了，所以再次生成快照时就没有它了），min_trx_id 为 120，max_trx_id 为 121，creator_trx_id 为 0。

然后从版本链中挑选可见的记录，从图中可以看出，最新版本的列 name 的内容是'大飞'，该版本的 trx_id 值为 120，在 m_ids 列表内，所以不符合可见性要求，根据 roll_pointer 跳到下一个版本。

下一个版本的列 name 的内容是'King'，该版本的 trx_id 值为 120，也在 m_ids 列表内，所以也不符合要求，继续跳到下一个版本。

下一个版本的列 name 的内容是'James', 该版本的 trx_id 值为 80, 小于 ReadView 中的 min_trx_id 值 120, 所以这个版本是符合要求的, 最后返回给用户的版本就是这条列 name 为"James"的记录。

以此类推, 如果之后事务 id 为 120 的记录也提交了, 再次在使用 READ COMMITTED 隔离级别的事务中查询表 teacher 中 number 值为 1 的记录时, 得到的结果就是'大飞'了, 具体流程我们就不分析了。总结一下就是: 使用 READ COMMITTED 隔离级别的事务在每次查询开始时都会生成一个独立的 ReadView。

REPEATABLE READ —— 在第一次读取数据时生成一个 ReadView

对于使用 REPEATABLE READ 隔离级别的事务来说, 只会在第一次执行查询语句时生成一个 ReadView, 之后的查询就不会重复生成了。我们还是用例子看一下是什么效果。

比方说现在系统里有两个事务 id 分别为 80、120 的事务在执行:

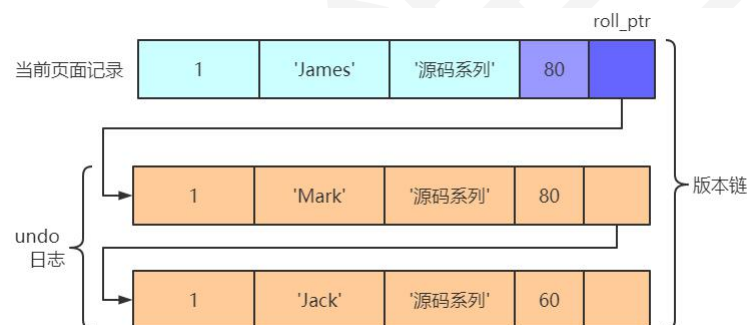
Transaction 80

UPDATE teacher SET name = 'Mark' WHERE number = 1;

UPDATE teacher SET name = 'James' WHERE number = 1;

...

此刻, 表 teacher 中 number 为 1 的记录得到的版本链表如下所示:



假设现在有一个使用 REPEATABLE READ 隔离级别的事务开始执行:

使用 REPEATABLE READ 隔离级别的事务

BEGIN;

SELECE1: Transaction 100、200 未提交

SELECT * FROM teacher WHERE number = 1; # 得到的列 name 的值为'Jack'

这个 SELECE1 的执行过程如下:

在执行 SELECT 语句时会先生成一个 ReadView, ReadView 的 m_ids 列表的内容就是[80, 120], min_trx_id 为 80, max_trx_id 为 121, creator_trx_id 为 0。

然后从版本链中挑选可见的记录, 从图中可以看出, 最新版本的列 name 的内容是'James', 该版本的 trx_id 值为 80, 在 m_ids 列表内, 所以不符合可见性要求, 根据 roll_pointer 跳到下一个版本。

下一个版本的列 name 的内容是'Mark', 该版本的 trx_id 值也为 80, 也在 m_ids 列表内, 所以也不符合要求, 继续跳到下一个版本。

下一个版本的列 name 的内容是"Jack", 该版本的 trx_id 值为 60, 小于 ReadView 中的 min_trx_id 值, 所以这个版本是符合要求的, 最后返回给用户的版本就是这条列 name 为"Jack"的记录。

之后, 我们把事务 id 为 80 的事务提交一下,

然后再到事务 id 为 120 的事务中更新一下表 teacher 中 number 为 1 的记录:

```
BEGIN;
```

```
# 更新了一些别的表的记录
```

```
UPDATE teacher SET name = 'King' WHERE number = 1;
```

```
UPDATE teacher SET name = '大飞' WHERE number = 1;
```

此刻, 表 teacher 中 number 为 1 的记录的版本链就长这样:



然后再到刚才使用 REPEATABLE READ 隔离级别的事务中继续查找这个 number 为 1 的记录, 如下:

```
# 使用 REPEATABLE READ 隔离级别的事务
```

```
BEGIN;
```

```
# SELECE1: Transaction 80、120 均未提交
```

```
SELECT * FROM teacher WHERE number = 1; # 得到的列 name 的值为"Jack"
```

```
# SELECE2: Transaction 80 提交, Transaction 120 未提交
```

```
SELECT * FROM teacher WHERE number = 1; # 得到的列 name 的值为"Jack"
```

这个 SELECE2 的执行过程如下:

因为当前事务的隔离级别为 REPEATABLE READ, 而之前在执行 SELECE1 时已经生成过 ReadView 了, 所以此时直接复用之前的 ReadView, 之前的 ReadView 的 m_ids 列表的内容就是[80, 120], min_trx_id 为 80, max_trx_id 为 121, creator_trx_id 为 0。

然后从版本链中挑选可见的记录, 从图中可以看出, 最新版本的列 name 的内容是'大飞', 该版本的 trx_id 值为 120, 在 m_ids 列表内, 所以不符合可见性要求, 根据 roll_pointer 跳到下一个版本。

下一个版本的列 name 的内容是'King', 该版本的 trx_id 值为 120, 也在 m_ids 列表内, 所以也不符合要求, 继续跳到下一个版本。

下一个版本的列 name 的内容是'James', 该版本的 `trx_id` 值为 80, 而 `m_ids` 列表中是包含值为 80 的事务 id 的, 所以该版本也不符合要求, 同理下一个列 name 的内容是'Mark'的版本也不符合要求。继续跳到下一个版本。

下一个版本的列 name 的内容是'Jack', 该版本的 `trx_id` 值为 60, 小于 `ReadView` 中的 `min_trx_id` 值 80, 所以这个版本是符合要求的, 最后返回给用户的版本就是这条列 c 为"Jack"的记录。

也就是说两次 `SELECT` 查询得到的结果是重复的, 记录的列 c 值都是"Jack", 这就是可重复读的含义。如果我们之后再吧事务 id 为 120 的记录提交了, 然后再到刚才使用 `REPEATABLE READ` 隔离级别的事务中继续查找这个 number 为 1 的记录, 得到的结果还是'Jack', 具体执行过程大家可以自己分析一下。

7.3.1.4. MVCC 下的幻读解决和幻读现象

前面我们已经知道了, `REPEATABLE READ` 隔离级别下 `MVCC` 可以解决不可重复读问题, 那么幻读呢? `MVCC` 是怎么解决的? 幻读是一个事务按照某个相同条件多次读取记录时, 后读取时读到了之前没有读到的记录, 而这个记录来自另一个事务添加的新记录。

我们可以想想, 在 `REPEATABLE READ` 隔离级别下的事务 T1 先根据某个搜索条件读取到多条记录, 然后事务 T2 插入一条符合相应搜索条件的记录并提交, 然后事务 T1 再根据相同搜索条件执行查询。结果会是什么? 按照 `ReadView` 中的比较规则:

3、如果被访问版本的 `trx_id` 属性值大于或等于 `ReadView` 中的 `max_trx_id` 值, 表明生成该版本的事务在当前事务生成 `ReadView` 后才开启, 所以该版本不可以被当前事务访问。

4、如果被访问版本的 `trx_id` 属性值在 `ReadView` 的 `min_trx_id` 和 `max_trx_id` 之间($\text{min_trx_id} < \text{trx_id} < \text{max_trx_id}$), 那就需要判断一下 `trx_id` 属性值是不是在 `m_ids` 列表中, 如果在, 说明创建 `ReadView` 时生成该版本的事务还是活跃的, 该版本不可以被访问; 如果不在, 说明创建 `ReadView` 时生成该版本的事务已经被提交, 该版本可以被访问。

不管事务 T2 比事务 T1 是否先开启, 事务 T1 都是看不到 T2 的提交的。请自行按照上面介绍的版本链、`ReadView` 以及判断可见性的规则来分析一下。

但是, 在 `REPEATABLE READ` 隔离级别下 InnoDB 中的 `MVCC` 可以很大程度地避免幻读现象, 而不是完全禁止幻读。怎么回事呢? 我们来看下面的情况:

```
mysql> select * from teacher;
+-----+-----+-----+
| number | name  | domain |
+-----+-----+-----+
|      1 | Jack  | 源码系列 |
|      3 | Mark  | 并发编程 |
|      9 | James | Redis   |
|     15 | King  | JVM     |
|     21 | DaFei | MySQL   |
+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from teacher where number = 30;
Empty set (0.00 sec)
```

我们首先在事务 T1 中：

```
select * from teacher where number = 30;
```

很明显，这个时候是找不到 number = 30 的记录。

我们在事务 T2 中，执行：

```
mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into teacher values(30,'Luffy','ELK');
Query OK, 1 row affected (0.00 sec)

mysql> commit;
Query OK, 0 rows affected (0.00 sec)
```

通过执行 insert into teacher values(30,'Luffy','ELK');，我们往表中插入了一条 number = 30 的记录。

此时回到事务 T1，执行：

```
mysql> update teacher set domain='RabbitMQ' where number=30;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from teacher where number = 30;
+-----+-----+-----+
| number | name  | domain |
+-----+-----+-----+
|      30 | Luffy | RabbitMQ |
+-----+-----+-----+
1 row in set (0.00 sec)
```

```
update teacher set domain='RabbitMQ' where number=30;
```

```
select * from teacher where number = 30;
```

嗯，怎么回事？事务 T1 很明显出现了幻读现象。

在 REPEATABLE READ 隔离级别下，T1 第一次执行普通的 SELECT 语句时生成了一个 ReadView，之后 T2 向 teacher 表中新插入一条记录并提交。

ReadView 并不能阻止 T1 执行 UPDATE 或者 DELETE 语句来改动这个新插入的记录（由于 T2 已经提交，因此改动该记录并不会造成阻塞），但是这样一来，这条新记录的 `trx_id` 隐藏列的值就变成了 T1 的事务 id。之后 T1 再使用普通的 SELECT 语句去查询这条记录时就可以看到这条记录了，也就可以把这条记录返回给客户端。因为这个特殊现象的存在，我们也可以认为 MVCC 并不能完全禁止幻读。

7.3.1.5. MVCC 小结

从上边的描述中我们可以看出来，所谓的 MVCC（Multi-Version Concurrency Control，多版本并发控制）指的就是在使用 READ COMMITTED、REPEATABLE READ 这两种隔离级别的事务在执行普通的 SELECT 操作时访问记录的版本链的过程，这样子可以使不同事务的读-写、写-读操作并发执行，从而提升系统性能。

READ COMMITTED、REPEATABLE READ 这两个隔离级别的一个很大不同就是：生成 ReadView 的时机不同，READ COMMITTED 在每一次进行普通 SELECT 操作前都会生成一个 ReadView，而 REPEATABLE READ 只在第一次进行普通 SELECT 操作前生成一个 ReadView，之后的查询操作都重复使用这个 ReadView 就好了，从而基本上可以避免幻读现象。

我们之前说执行 DELETE 语句或者更新主键的 UPDATE 语句并不会立即把对应的记录完全从页面中删除，而是执行一个所谓的 delete mark 操作，相当于只是对记录打上了一个删除标志位，这主要就是为 MVCC 服务的。

另外，所谓的 MVCC 只是在我们进行普通的 SELECT 查询时才生效，截止到目前我们所见的所有 SELECT 语句都算是普通的查询，至于什么是个不普通的查询，我们马上就会讲到。

8. MySQL 中的锁

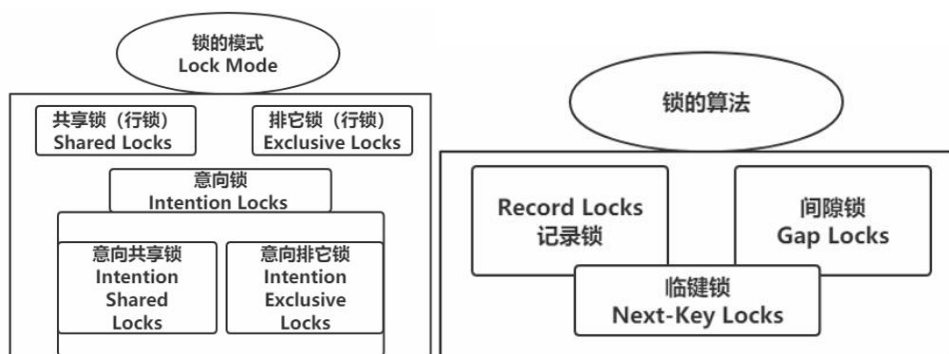
按照 MySQL 官方的说法，InnoDB 中锁可以分为：

14.7.1 InnoDB Locking

This section describes lock types used by InnoDB.

- Shared and Exclusive Locks 乐观/共享锁、互斥/排他锁
- Intention Locks 意向锁
- Record Locks 行锁（表锁）
- Gap Locks 间隙锁
- Next-Key Locks Record Locks+Gap Locks
- Insert Intention Locks 插入意向锁
- AUTO-INC Locks 自增锁
- Predicate Locks for Spatial Indexes 空间索引预测锁

可见，InnoDB 中锁非常多，总的来说，可以如下分类：



这些锁都是做什么的？具体含义是什么？我们现在来一一学习。

8.1. 解决并发事务问题

我们已经知道事务并发执行时可能带来的各种问题，最大的一个难点是：一方面要最大程度地利用数据库的并发访问，另外一方面还要确保每个用户能以一致的方式读取和修改数据，尤其是一个事务进行读取操作，另一个同时进行改动操作的情况下。

8.1.1. 复习并发事务问题

一个事务进行读取操作，另一个进行改动操作,我们前边说过，这种情况下可能发生脏读、不可重复读、幻读的问题。

SQL 标准规定不同隔离级别下可能发生的问题不一样：

在 READ UNCOMMITTED 隔离级别下，脏读、不可重复读、幻读都可能发生。

在 READ COMMITTED 隔离级别下，不可重复读、幻读可能发生，脏读不可以发生。

在 REPEATABLE READ 隔离级别下，幻读可能发生，脏读和不可重复读不可以发生。

在 SERIALIZABLE 隔离级别下，上述问题都不可以发生。

不过各个数据库厂商对 SQL 标准的支持都可能不一样，与 SQL 标准不同的一点就是，MySQL 在 REPEATABLE READ 隔离级别实际上就基本解决了幻读问题。

怎么解决脏读、不可重复读、幻读这些问题呢？其实有两种可选的解决方案：

8.1.2. 方案一：读操作 MVCC，写操作进行加锁

所谓的 MVCC 我们在前一章有过详细的描述，就是通过生成一个 ReadView，然后通过 ReadView 找到符合条件的记录版本(历史版本是由 undo 日志构建的)，其实就像是在生成 ReadView 的那个时刻做了一个快照，查询语句只能读到在生成 ReadView 之前已提交事务所做的更改，在生成 ReadView 之前未提交的事务或者之后才开启的事务所做的更改是看不到的。而写操作肯定针对的是最新版本的记录，读记录的历史版本和改动记录的最新版本本身并不冲突，也就是采用 MVCC 时，读-写操作并不冲突。

我们说过普通的 `SELECT` 语句在 `READ COMMITTED` 和 `REPEATABLE READ` 隔离级别下会使用到 `MVCC` 读取记录。在 `READ COMMITTED` 隔离级别下，一个事务在执行过程中每次执行 `SELECT` 操作时都会生成一个 `ReadView`，`ReadView` 的存在本身就保证了事务不可以读取到未提交的事务所做的更改，也就是避免了脏读现象；`REPEATABLE READ` 隔离级别下，一个事务在执行过程中只有第一次执行 `SELECT` 操作才会生成一个 `ReadView`，之后的 `SELECT` 操作都复用这个 `ReadView`，这样也就避免了不可重复读和很大程度上避免了幻读的问题。

8.1.3. 一致性读 (Consistent Reads) /快照读

事务利用 `MVCC` 进行的读取操作称之为一致性读，或者一致性无锁读，也称之为快照读，但是往往读取的是历史版本数据。所有普通的 `SELECT` 语句（`plain SELECT`）在 `READ COMMITTED`、`REPEATABLE READ` 隔离级别下都算是一致性读。上面的这句话中，普通的 `SELECT` 语句是指不加锁的 `select` 语句在非串行化事务隔离级别下。

一致性读并不会对表中的任何记录做加锁操作，其他事务可以自由的对表中的记录做改动。

很明显，采用 `MVCC` 方式的话，读-写操作彼此并不冲突，性能更高，采用加锁方式的话，读-写操作彼此需要排队执行，影响性能。一般情况下我们当然愿意采用 `MVCC` 来解决读-写操作并发执行的问题，但是业务在某些情况下，要求必须采用加锁的方式执行。

8.1.4. 方案二：读、写操作都采用加锁的方式

如果我们的一些业务场景不允许读取记录的旧版本，而是每次都必须去读取记录的最新版本，比方在银行存款的事务中，你需要先把账户的余额读出来，然后将其加上本次存款的数额，最后再写到数据库中。在将账户余额读取出来后，就不想让别的事务再访问该余额，直到本次存款事务执行完成，其他事务才可以访问账户的余额。这样在读取记录的时候也就需要对其进行加锁操作，这样也就意味着读操作和写操作也像写-写操作那样排队执行。

我们说脏读的产生是因为当前事务读取了另一个未提交事务写的一条记录，如果另一个事务在写记录的时候就给这条记录加锁，那么当前事务就无法继续读取该记录了，所以也就不会有脏读问题的产生了。

不可重复读的产生是因为当前事务先读取一条记录，另外一个事务对该记录做了改动之后并提交之后，当前事务再次读取时会获得不同的值，如果在当前事务读取记录时就给该记录加锁，那么另一个事务就无法修改该记录，自然也不会发生不可重复读了。

我们说幻读问题的产生是因为当前事务读取了一个范围的记录，然后另外的事务向该范围内插入了新记录，当前事务再次读取该范围的记录时发现了新插入的新记录，我们把新插入的那些记录称之为幻影记录。采用加锁的方式解决幻读问题就有不太容易了，因为当前事务在第一次读取记录时那些幻影记录并不存在，所以读取的时候加锁就有点麻烦 —— 因为并不知道给谁加锁。InnoDB 中是如何解决的，我们后面会讲到。

8.2. 锁定读 (Locking Reads) /LBCC

也称当前读, 读取的是最新版本, 并且对读取的记录加锁, 阻塞其他事务同时改动相同记录, 避免出现安全问题。

哪些是当前读呢? `select lock in share mode` (共享锁)、`select for update` (排他锁)、`update` (排他锁)、`insert` (排他锁)、`delete` (排他锁)、串行化事务隔离级别都是当前读。

当前读这种实现方式, 也可以称之为 LBCC (基于锁的并发控制, Lock-Based Concurrency Control), 怎么做到的?

8.2.1.1. 共享锁和独占锁

在使用加锁的方式解决问题时, 由于既要允许读-读情况不受影响, 又要使写-写、读-写或写-读情况中的操作相互阻塞, MySQL 中的锁有好几类:

共享锁, 英文名: **Shared Locks**, 简称 **S 锁**。在事务要读取一条记录时, 需要先获取该记录的 **S 锁**。

独占锁, 也常称排他锁, 英文名: **Exclusive Locks**, 简称 **X 锁**。在事务要改动一条记录时, 需要先获取该记录的 **X 锁**。

假如事务 **E1** 首先获取了一条记录的 **S 锁** 之后, 事务 **E2** 接着也要访问这条记录:

如果事务 **E2** 想要再获取一个记录的 **S 锁**, 那么事务 **E2** 也会获得该锁, 也就意味着事务 **E1** 和 **E2** 在该记录上同时持有 **S 锁**。

如果事务 **E2** 想要再获取一个记录的 **X 锁**, 那么此操作会被阻塞, 直到事务 **E1** 提交之后将 **S 锁** 释放掉。

如果事务 **E1** 首先获取了一条记录的 **X 锁** 之后, 那么不管事务 **E2** 接着想获取该记录的 **S 锁** 还是 **X 锁** 都会被阻塞, 直到事务 **E1** 提交。

所以我们说 **S 锁** 和 **S 锁** 是兼容的, **S 锁** 和 **X 锁** 是不兼容的, **X 锁** 和 **X 锁** 也是不兼容的, 画个表表示一下就是这样:

X	不兼容 X	不兼容 S
S	不兼容 X	兼容 S

8.2.1.2. 锁定读的 SELECT 语句

MySQL 有两种比较特殊的 **SELECT** 语句格式:

对读取的记录加 **S 锁**:

```
SELECT ... LOCK IN SHARE MODE;
```

也就是在普通的 **SELECT** 语句后边加 **LOCK IN SHARE MODE**, 如果当前事务执行了该语句, 那么它会为读取到的记录加 **S 锁**, 这样允许别的事务继续获取这些记录的 **S 锁** (比方说别的事务也使用 **SELECT ... LOCK IN SHARE MODE** 语句来读取这些记录), 但是不能获取这些记录的 **X 锁** (比方说使用 **SELECT ... FOR UPDATE** 语句来读取这些记录, 或者直接修改这些记录)。

如果别的事务想要获取这些记录的 X 锁，那么它们会阻塞，直到当前事务提交之后将这些记录上的 S 锁释放掉。

对读取的记录加 X 锁：

`SELECT ... FOR UPDATE;`

也就是在普通的 `SELECT` 语句后边加 `FOR UPDATE`，如果当前事务执行了该语句，那么它会为读取到的记录加 X 锁，这样既不允许别的事务获取这些记录的 S 锁（比方说别的事务使用 `SELECT ... LOCK IN SHARE MODE` 语句来读取这些记录），也不允许获取这些记录的 X 锁（比如说使用 `SELECT ... FOR UPDATE` 语句来读取这些记录，或者直接修改这些记录）。

如果别的事务想要获取这些记录的 S 锁或者 X 锁，那么它们会阻塞，直到当前事务提交之后将这些记录上的 X 锁释放掉。

8.2.1.3. 写操作的锁

平常所用到的写操作无非是 `DELETE`、`UPDATE`、`INSERT` 这三种：

`DELETE`：

对一条记录做 `DELETE` 操作的过程其实是先在 B+ 树中定位到这条记录的位置，然后获取一下这条记录的 X 锁，然后再执行 `delete mark` 操作。我们也可以把这个定位待删除记录在 B+ 树中位置的过程看成是一个获取 X 锁的锁定读。

`INSERT`：

一般情况下，新插入一条记录的操作并不加锁，InnoDB 通过一种称之为隐式锁来保护这条新插入的记录在本事务提交前不被别的事务访问。当然，在一些特殊情况下 `INSERT` 操作也是会获取锁的，具体情况我们后边再说。

`UPDATE`：

在对一条记录做 `UPDATE` 操作时分为三种情况：

如果未修改该记录的键值并且被更新的列占用的存储空间在修改前后未发生变化，则先在 B+ 树中定位到这条记录的位置，然后再获取一下记录的 X 锁，最后在原记录的位置进行修改操作。其实我们也可以把这个定位待修改记录在 B+ 树中位置的过程看成是一个获取 X 锁的锁定读。

如果未修改该记录的键值并且至少有一个被更新的列占用的存储空间在修改前后发生变化，则先在 B+ 树中定位到这条记录的位置，然后获取一下记录的 X 锁，将该记录彻底删除掉（就是把记录彻底移入垃圾链表），最后再插入一条新记录。这个定位待修改记录在 B+ 树中位置的过程看成是一个获取 X 锁的锁定读，新插入的记录由 `INSERT` 操作提供的隐式锁进行保护。

如果修改了该记录的键值，则相当于在原记录上做 `DELETE` 操作之后再进行一次 `INSERT` 操作，加锁操作就需要按照 `DELETE` 和 `INSERT` 的规则进行了。

8.3. 锁的粒度

我们前边提到的锁都是针对记录的，也可以被称之为行级锁或者行锁，对一条记录加锁影响的也只是这条记录而已，我们就说这个锁的粒度比较细；其实一个事务也可以在表级别进行加锁，自然就被称之为表级锁或者表锁，对一个表加

锁影响整个表中的记录，我们就说这个锁的粒度比较粗。给表加的锁也可以分为共享锁（S 锁）和独占锁（X 锁）

8.3.1.1. 表锁与行锁的比较

锁定粒度：表锁 > 行锁

加锁效率：表锁 > 行锁

冲突概率：表锁 > 行锁

并发性能：表锁 < 行锁

8.3.1.2. 给表加 S 锁

如果一个事务给表加了 S 锁，那么：

别的事务可以继续获得该表的 S 锁

别的事务可以继续获得该表中的某些记录的 S 锁

别的事务不可以继续获得该表的 X 锁

别的事务不可以继续获得该表中的某些记录的 X 锁

8.3.1.3. 给表加 X 锁

如果一个事务给表加了 X 锁（意味着该事务要独占这个表），那么：

别的事务不可以继续获得该表的 S 锁

别的事务不可以继续获得该表中的某些记录的 S 锁

别的事务不可以继续获得该表的 X 锁

别的事务不可以继续获得该表中的某些记录的 X 锁。

为了更好的理解这个表级别的 S 锁和 X 锁和后面的意向锁，我们举一个现实生活中的例子。我们用曾经很火爆的互联网风口项目共享 Office 来说明加锁：

共享 Office 有栋大楼，楼自然有很多层。办公室都是共享的，客户可以随便选办公室办公。每层楼可以容纳客户同时办公，每当一个客户进去办公，就相当于在每层的入口处挂了一把 S 锁，如果很多客户进去办公，相当于每层的入口处挂了很多把 S 锁（类似行级别的 S 锁）。

有的时候楼层会进行检修，比方说换地板，换天花板，检查水电啥的，这些维修项目并不能同时开展。如果楼层针对某个项目进行检修，就不允许客户来办公，也不允许其他维修项目进行，此时相当于楼层门口会挂一把 X 锁（类似行级别的 X 锁）。

上边提到的这两种锁都是针对楼层而言的，不过有时候我们会有一些特殊的需求：

A、有投资人要来考察 Office 的环境。

投资人和公司并不想影响客户进去办公，但是此时不能有楼层进行检修，所以可以在大楼门口放置一把 S 锁（类似表级别的 S 锁）。此时：

来办公的客户们看到大楼门口有 S 锁，可以继续进入大楼办公。

修理工看到大楼门口有 S 锁，则先在大楼门口等着，啥时候投资人走了，把大楼的 S 锁撤掉再进入大楼维修。

B、公司要和房东谈条件。

此时不允许大楼中有正在办公的楼层，也不允许对楼层进行维修。所以可以在大楼门口放置一把 X 锁（类似表级别的 X 锁）。此时：

来办公的客户们看到大楼门口有 X 锁，则需要在大楼门口等着，啥时候条件谈好，把大楼的 X 锁撤掉再进入大楼办公。

修理工看到大楼门口有 X 锁，则先在大楼门口等着，啥时候考试结束，把大楼的 X 锁撤掉再进入大楼维修。

8.4. 意向锁

但是在上面的例子这里头有两个问题：

如果我们想对大楼整体上 S 锁，首先需要确保大楼中的没有正在维修的楼层，如果有正在维修的楼层，需要等到维修结束才可以对大楼整体上 S 锁。

如果我们想对大楼整体上 X 锁，首先需要确保大楼中的没有办公的楼层以及正在维修的楼层，如果有办公的楼层或者正在维修的楼层，需要等到全部办公的同学都办公离开，以及维修工维修完楼层离开后才可以对大楼整体上 X 锁。

我们在对大楼整体上锁（表锁）时，怎么知道大楼中有没有楼层已经被上锁（行锁）了呢？依次检查每一楼层门口有没有上锁？那这效率也太慢了吧！于是 InnoDB 提出了一种意向锁（英文名：Intention Locks）：

意向共享锁，英文名：Intention Shared Lock，简称 IS 锁。当事务准备在某条记录上加 S 锁时，需要先在表级别加一个 IS 锁。

意向独占锁，英文名：Intention Exclusive Lock，简称 IX 锁。当事务准备在某条记录上加 X 锁时，需要先在表级别加一个 IX 锁。

视角回到大楼和楼层上来：

如果有客户到楼层中办公，那么他先在整栋大楼门口放一把 IS 锁（表级锁），然后再到楼层门口放一把 S 锁（行锁）。

如果有维修工到楼层中维修，那么它先在整栋大楼门口放一把 IX 锁（表级锁），然后再到楼层门口放一把 X 锁（行锁）。

之后：

如果有投资人要参观大楼，也就是想在大楼门口前放 S 锁（表锁）时，首先要看一下大楼门口有没有 IX 锁，如果有，意味着有楼层在维修，需要等到维修结束把 IX 锁撤掉后才可以在整栋大楼上加 S 锁。

如果有谈条件要占用大楼，也就是想在大楼门口前放 X 锁（表锁）时，首先要看一下大楼门口有没有 IS 锁或 IX 锁，如果有，意味着有楼层在办公或者维修，需要等到客户们办完公以及维修结束把 IS 锁和 IX 锁撤掉后才可以在整栋大楼上加 X 锁。

注意：客户在大楼门口加 IS 锁时，是不关心大楼门口是否有 IX 锁的，维修工在大楼门口加 IX 锁时，是不关心大楼门口是否有 IS 锁或者其他 IX 锁的。IS 和

IX 锁只是为了判断当前时间大楼里有没有被占用的楼层用的，也就是在对大楼加 S 锁或者 X 锁时才会用到。

总结一下：IS、IX 锁是表级锁，它们的提出仅仅为了在之后加表级别的 S 锁和 X 锁时可以快速判断表中的记录是否被上锁，以避免用遍历的方式来查看表中有没有上锁的记录。就是说其实 IS 锁和 IX 锁是兼容的，IX 锁和 IX 锁是兼容的。我们画个表来看一下**表级别**的各种锁的兼容性：

兼容性	X	IX	S	IS
X	不兼容	不兼容	不兼容	不兼容
IX	不兼容		不兼容	
S	不兼容	不兼容		
IS	不兼容			

锁的组合性：

组合性	X	IX	S	IS
表锁	有	有	有	有
行锁	有		有	

8.5.MySQL 中的行锁和表锁

MySQL 支持多种存储引擎，不同存储引擎对锁的支持也是不一样的。当然，我们重点还是讨论 InnoDB 存储引擎中的锁，其他的存储引擎只是稍微看看。

8.5.1. 其他存储引擎中的锁

对于 MyISAM、MEMORY、MERGE 这些存储引擎来说，它们只支持表级锁，而且这些引擎并不支持事务，所以使用这些存储引擎的锁一般都是针对当前会话来说的。比方说在 Session 1 中对一个表执行 SELECT 操作，就相当于为这个表加了一个表级别的 S 锁，如果在 SELECT 操作未完成时，Session 2 中对这个表执行 UPDATE 操作，相当于要获取表的 X 锁，此操作会被阻塞，直到 Session 1 中的 SELECT 操作完成，释放掉表级别的 S 锁后，Session 2 中对这个表执行 UPDATE 操作才能继续获取 X 锁，然后执行具体的更新语句。

因为使用 MyISAM、MEMORY、MERGE 这些存储引擎的表在同一时刻只允许一个会话对表进行写操作，所以这些存储引擎实际上最好用在只读，或者大部分都是读操作，或者单用户的情景下。另外，在 MyISAM 存储引擎中有一个称之为 Concurrent Inserts 的特性，支持在对 MyISAM 表读取时同时插入记录，这样可以提升一些插入速度。关于更多 Concurrent Inserts 的细节，详情可以参考文档。

8.5.2. InnoDB 存储引擎中的锁

InnoDB 存储引擎既支持表锁，也支持行锁。表锁实现简单，占用资源较少，不过粒度很粗，有时候你仅仅需要锁住几条记录，但使用表锁的话相当于为表中

的所有记录都加锁，所以性能比较差。行锁粒度更细，可以实现更精准的并发控制。下边我们详细看一下。

8.5.2.1. InnoDB 中的表级锁

表级别的 S 锁、X 锁

在对某个表执行 `SELECT`、`INSERT`、`DELETE`、`UPDATE` 语句时，InnoDB 存储引擎是不会为这个表添加表级别的 S 锁或者 X 锁的。

另外，在对某个表执行一些诸如 `ALTER TABLE`、`DROP TABLE` 这类的 DDL 语句时，其他事务对这个表并发执行诸如 `SELECT`、`INSERT`、`DELETE`、`UPDATE` 的语句会发生阻塞，同理，某个事务中对某个表执行 `SELECT`、`INSERT`、`DELETE`、`UPDATE` 语句时，在其他会话中对这个表执行 DDL 语句也会发生阻塞。这个过程其实是通过在 server 层使用一种称之为元数据锁（英文名：Metadata Locks，简称 MDL）来实现的，一般情况下也不会使用 InnoDB 存储引擎自己提供的表级别的 S 锁和 X 锁。

其实这个 InnoDB 存储引擎提供的表级 S 锁或者 X 锁是相当鸡肋，只会有一些特殊情况下，比方说崩溃恢复过程中用到。不过我们还是可以手动获取一下的，比方说在系统变量 `autocommit=0`，`innodb_table_locks = 1` 时，手动获取 InnoDB 存储引擎提供的表 t 的 S 锁或者 X 锁可以这么写：

`LOCK TABLES t READ`：InnoDB 存储引擎会对表 t 加表级别的 S 锁。

`LOCK TABLES t WRITE`：InnoDB 存储引擎会对表 t 加表级别的 X 锁。

不过请尽量避免在使用 InnoDB 存储引擎的表上使用 `LOCK TABLES` 这样的手动锁表语句，它们并不会提供什么额外的保护，只是会降低并发能力而已。

表级别的 IS 锁、IX 锁

当我们在对使用 InnoDB 存储引擎的表的某些记录加 S 锁之前，那就需要先在表级别加一个 IS 锁，当我们在对使用 InnoDB 存储引擎的表的某些记录加 X 锁之前，那就需要先在表级别加一个 IX 锁。

IS 锁和 IX 锁的使命只是为了后续在加表级别的 S 锁和 X 锁时判断表中是否有已经被加锁的记录，以避免用遍历的方式来查看表中有没有上锁的记录。我们并不能手动添加意向锁，只能由 InnoDB 存储引擎自行添加。

表级别的 AUTO-INC 锁

在使用 MySQL 过程中，我们可以为表的某个列添加 `AUTO_INCREMENT` 属性，之后在插入记录时，可以不指定该列的值，系统会自动为它赋上递增的值系统实现这种自动给 `AUTO_INCREMENT` 修饰的列递增赋值的原理主要是两个：

1、采用 AUTO-INC 锁，也就是在执行插入语句时就在表级别加一个 AUTO-INC 锁，然后为每条待插入记录的 `AUTO_INCREMENT` 修饰的列分配递增的值，在该语句执行结束后，再把 AUTO-INC 锁释放掉。这样一个事务在持有 AUTO-INC 锁的过程中，其他事务的插入语句都要被阻塞，可以保证一个语句中分配的递增值是连续的。

如果我们的插入语句在执行前不可以确定具体要插入多少条记录（无法预计即将插入记录的数量），比方说使用 `INSERT ... SELECT`、`REPLACE ... SELECT` 或者

LOAD DATA 这种插入语句，一般是使用 AUTO-INC 锁为 AUTO_INCREMENT 修饰的列生成对应的值。

2、采用一个轻量级的锁，在为插入语句生成 AUTO_INCREMENT 修饰的列的值时获取一下这个轻量级锁，然后生成本次插入语句需要用到的 AUTO_INCREMENT 列的值之后，就把该轻量级锁释放掉，并不需要等到整个插入语句执行完才释放锁。

如果我们的插入语句在执行前就可以确定具体要插入多少条记录，比方说我们上边举的关于表 t 的例子中，在语句执行前就可以确定要插入 2 条记录，那么一般采用轻量级锁的方式对 AUTO_INCREMENT 修饰的列进行赋值。这种方式可以避免锁定表，可以提升插入性能。

InnoDB 提供了一个称之为 innodb_autoinc_lock_mode 的系统变量来控制到底使用上述两种方式中的哪种来为 AUTO_INCREMENT 修饰的列进行赋值，当 innodb_autoinc_lock_mode 值为 0 时，一律采用 AUTO-INC 锁；当 innodb_autoinc_lock_mode 值为 2 时，一律采用轻量级锁；当 innodb_autoinc_lock_mode 值为 1 时，两种方式混着来（也就是在插入记录数量确定时采用轻量级锁，不确定时使用 AUTO-INC 锁）。

不过当 innodb_autoinc_lock_mode 值为 2 时，可能会造成不同事务中的插入语句为 AUTO_INCREMENT 修饰的列生成的值是交叉的，在有主从复制的场景中是不安全的。

MySQL5.7.X 中缺省为 1。

8.5.2.2. InnoDB 中的行级锁

行锁，也称为记录锁，顾名思义就是在记录上加的锁。但是要注意，这个记录指的是通过给索引上的索引项加锁。InnoDB 这种行锁实现特点意味着：只有通过索引条件检索数据，InnoDB 才使用行级锁，否则，InnoDB 将使用表锁。

不论是使用主键索引、唯一索引或普通索引，InnoDB 都会使用行锁来对数据加锁。

只有执行计划真正使用了索引，才能使用行锁：即便在条件中使用了索引字段，但是否使用索引来检索数据是由 MySQL 通过判断不同执行计划的代价来决定的，如果 MySQL 认为全表扫描效率更高，比如对一些很小的表，它就不会使用索引，这种情况下 InnoDB 将使用表锁，而不是行锁。

同时当我们用范围条件而不是相等条件检索数据，并请求锁时，InnoDB 会给符合条件的已有数据记录的索引项加锁。

不过即使是行锁，InnoDB 里也是分成了各种类型的。换句话说即使对同一条记录加行锁，如果类型不同，起到的功效也是不同的。我们使用前面的 teacher，增加一个索引，并插入几条记录。

```
INDEX `idx_name`(`name`)
```



```
mysql> select * from teacher;
+-----+-----+-----+
| number | name  | domain |
+-----+-----+-----+
|      1 | Jack  | 源码系列 |
|      3 | Mark  | 并发编程 |
|      9 | James | Redis   |
|     15 | King  | JVM     |
|     21 | DaFei | MySQL   |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

我们来看看都有哪些常用的行锁类型。

Record Locks

也叫记录锁，就是仅仅把一条记录锁上，官方的类型名称为：**LOCK_REC_NOT_GAP**。比方说我们把 **number** 值为 9 的那条记录加一个记录锁的示意图如下：

1	3	9	15	21
'Jack'	'Mark'	'James'	'King'	'DaFei'
'源码系列'	'并发编程'	'Redis'	'JVM'	'MySQL'

记录锁是有 S 锁和 X 锁之分的，当一个事务获取了一条记录的 S 型记录锁后，其他事务也可以继续获取该记录的 S 型记录锁，但不可以继续获取 X 型记录锁；当一个事务获取了一条记录的 X 型记录锁后，其他事务既不可以继续获取该记录的 S 型记录锁，也不可以继续获取 X 型记录锁；

Gap Locks

我们说 MySQL 在 REPEATABLE READ 隔离级别下是可以解决幻读问题的，解决方案有两种，可以使用 MVCC 方案解决，也可以采用加锁方案解决。但是在使用加锁方案解决时有问题，就是事务在第一次执行读取操作时，那些幻影记录尚不存在，我们无法给这些幻影记录加上记录锁。InnoDB 提出了一种称之为 Gap Locks 的锁，官方的类型名称为：**LOCK_GAP**，我们也可以简称为 gap 锁。

间隙锁实质上是对索引前后的间隙上锁，不对索引本身上锁。

会话 1 开启一个事务，执行

```
begin;
```

```
update teacher set domain ='Spring' where name='James';
```

```
mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> update teacher set domain ='Spring' where name='James';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

会对(['Jack',1],['James',9])之间,(['James',9], ['King',15])之间进行上锁。

'DaFei'	'Jack'		'James'		'King'	'Mark'
1	3		9		15	21

(上图有错误, Jack 下应该为 1)

如图中为['James',9]的记录加了 gap 锁,意味着不允许别的事务在这条记录前后间隙插入新记录。

begin;

insert into teacher value(22,'Jahes','docker');

```
mysql> begin;
Query OK, 0 rows affected (0.01 sec)

mysql> insert into teacher value(22,'Jahes','docker');
ERROR 1205 (HY000): Lock wait timeout exceeded; try restarting transaction
```

为什么不能插入? 因为记录 (22,'Jahes') 要插入的话,在索引 idx_name 上,刚好落在(['James',9], ['King',15])之间,是有锁的,当然不允许插入。

但是当 SQL 语句变为: insert into teacher value(70,'Kings','docker');能插入吗?

当然能,因为 (70,'Kings') 这条记录不在被锁的区间内。

```
mysql> insert into teacher values(70,'King','docker');
Query OK, 1 row affected (0.00 sec)
```

思考题

现在有表,表中有记录如下:

```
CREATE TABLE test1 (
id int(1) NOT NULL AUTO_INCREMENT,
number int(1) NOT NULL COMMENT '数字',
PRIMARY KEY (id),
KEY number (number) USING BTREE
) ENGINE=InnoDB AUTO_INCREMENT=1 DEFAULT CHARSET=utf8;

INSERT INTO test1 VALUES (1, 1);
INSERT INTO test1 VALUES (5, 3);
INSERT INTO test1 VALUES (7, 8);
INSERT INTO test1 VALUES (11, 12);

开启一个事务
SELECT * FROM test1 WHERE number = 3 FOR UPDATE;

开启另外一个事务:
INSERT INTO test1 (id, number) VALUES (2, 1); # 阻塞
```

```
INSERT INTO test1 (id, number) VALUES (3, 2); # 阻塞
INSERT INTO test1 (id, number) VALUES (6, 8); # 阻塞
INSERT INTO test1 (id, number) VALUES (8, 8); # 正常执行
INSERT INTO test1 (id, number) VALUES (9, 9); # 正常执行
INSERT INTO test1 (id, number) VALUES (10, 12); #
```

问题：为什么（6，8）不能执行，（8，8）可以执行？

解决思路：画一个 `number` 的索引数据存放的图，然后根据间隙锁的加锁方式，把锁加上，就能很快明白答案。更详细的的解答，看 <http://bbs.xiangxueketang.cn/question/1118>。

Next-Key Locks

有时候我们既想锁住某条记录，又想阻止其他事务在该记录前边的间隙插入新记录，所以 InnoDB 就提出了一种称之为 **Next-Key Locks** 的锁，官方的类型名称为：`LOCK_ORDINARY`，我们也可以简称为 **next-key** 锁。**next-key** 锁的本质就是一个记录锁和一个 **gap** 锁的合体。

默认情况下，InnoDB 以 `REPEATABLE READ` 隔离级别运行。在这种情况下，InnoDB 使用 **Next-Key Locks** 锁进行搜索和索引扫描，这可以防止幻读的发生。

Insert Intention Locks

我们说一个事务在插入一条记录时需要判断一下插入位置是不是被别的事务加了所谓的 **gap** 锁（**next-key** 锁也包含 **gap** 锁，后边就不强调了），如果有的话，插入操作需要等待，直到拥有 **gap** 锁的那个事务提交。

但是 InnoDB 规定事务在等待的时候也需要在内存中生成一个锁结构，表明有事务想在某个间隙中插入新记录，但是现在处于等待状态。这种类型的锁命名为 **Insert Intention Locks**，官方的类型名称为：`LOCK_INSERT_INTENTION`，我们也可以称为插入意向锁。

可以理解为插入意向锁是一种锁的等待队列，让等锁的事务在内存中进行排队等待，当持有锁的事务完成后，处于等待状态的事务就可以获得锁继续事务了。

隐式锁

锁的维护是需要成本的，为了节约资源，MySQL 在设计提出了一个隐式锁的概念。一般情况下 `INSERT` 操作是不加锁的，当然真的有并发冲突的情况下，还是会导致问题的。

所以 MySQL 中，一个事务对新插入的记录可以不显式的加锁，但是别的事务在对这条记录加 **S** 锁或者 **X** 锁时，会去检查索引记录中的 `trx_id` 隐藏列，然后进行各种判断，会先帮助当前事务生成一个锁结构，然后自己再生成一个锁结构后进入等待状态。但是由于事务 `id` 的存在，相当于加了一个隐式锁。

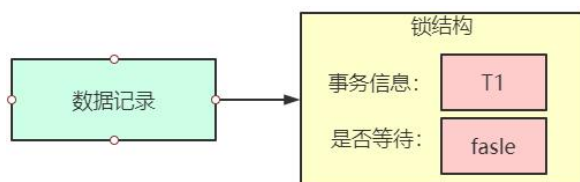
这样的话，隐式锁就起到了延迟生成锁的用处。这个过程，我们无法干预，是由引擎自动处理的，对我们是完全透明的，我们知道下就行了。

8.5.2.3. 锁的内存结构

所谓的锁其实是一个内存中的结构，在事务执行前本来是没有锁的，也就是说一开始是没有锁结构和记录进行关联的，当一个事务想对这条记录做改动时，首先会看看内存中有没有与这条记录关联的锁结构，当没有的时候就会在内存中生成一个锁结构与之关联。比方说事务 E1 要对记录做改动，就需要生成一个锁结构与之关联。锁结构里至少要有两个比较重要的属性：

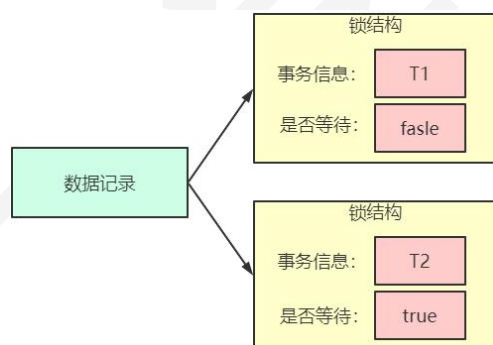
trx 信息：代表这个锁结构是哪个事务生成的。

is_waiting：代表当前事务是否在等待。



当事务 T1 改动了条记录后，就生成了一个锁结构与该记录关联，因为之前没有别的事务为这条记录加锁，所以 is_waiting 属性就是 false，我们把这个场景就称之为获取锁成功，或者加锁成功，然后就可以继续执行操作了。

在事务 T1 提交之前，另一个事务 T2 也想对该记录做改动，那么先去看看有没有锁结构与这条记录关联，发现有一个锁结构与之关联后，然后也生成了一个锁结构与这条记录关联，不过锁结构的 is_waiting 属性值为 true，表示当前事务需要等待，我们把这个场景就称之为获取锁失败，或者加锁失败，或者没有成功的获取到锁：



在事务 T1 提交之后，就会把该事务生成的锁结构释放掉，然后看看还有没有别的事务在等待获取锁，发现了事务 T2 还在等待获取锁，所以把事务 T2 对应的锁结构的 is_waiting 属性设置为 false，然后把该事务对应的线程唤醒，让它继续执行，此时事务 T2 就算获取到锁了。

这种实现方式非常像并发编程里的 CLH 队列。

对一条记录加锁的本质就是在内存中创建一个锁结构与之关联。那么，一个事务对多条记录加锁时，是不是就要创建多个锁结构呢。

比如 `SELECT * FROM teacher LOCK IN SHARE MODE;`

很显然，这条语句需要为 teacher 表中的所有记录进行加锁。那么，是不是需要为每条记录都生成一个锁结构呢？其实理论上创建多个锁结构没有问题，反

而更容易理解。但是如果一个事务要获取 10,000 条记录的锁，要生成 10,000 个这样的结构，不管是执行效率还是空间效率来说都是很不划算的，所以实际上，并不是一个记录一个锁结构。

当然锁结构实际是很复杂的，我们大概了解下里面包含哪些元素。

锁所在的事务信息：无论是表级锁还是行级锁，一个锁属于一个事务，这里记载着该锁对应的事务信息。

索引信息：对于行级锁来说，需要记录一下加锁的记录属于哪个索引。

表锁/行锁信息：表级锁结构和行级锁结构在这个位置的内容是不同的。

具体表现为表级锁记载着这是对哪个表加的锁，还有其他的一些信息；

而行级锁记载了记录所在的表空间、记录所在的页号、区分到底是为哪一条记录加了锁的数据结构。

锁模式：锁是 IS，IX，S，X 中的哪一种。

锁类型：表锁还是行锁，行锁的具体类型。

其他一些和锁管理相关的数据结构，比如哈希表和链表等。

基本上来说，同一个事务里，同一个数据页面，同一个加锁类型的锁会保存在一起。

8.6. 查看事务加锁的情况

可以通过

show engine innodb status\G

查看事务加锁的情况，不过一般情况下，看不到哪个事务对哪些记录加了那些锁，需要修改系统变量 `innodb_status_output_locks`（MySQL5.6.16 引入），缺省是 OFF。

show variables like 'innodb_status_output_locks';

```
mysql> show variables like 'innodb_status_output_locks';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_status_output_locks | OFF   |
+-----+-----+
1 row in set (0.00 sec)
```

我们需要设置为 ON，

set global innodb_status_output_locks = ON;

```
mysql> set global innodb_status_output_locks = ON;
Query OK, 0 rows affected (0.00 sec)

mysql> show variables like 'innodb_status_output_locks';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_status_output_locks | ON    |
+-----+-----+
1 row in set (0.01 sec)
```

然后开启事务，并执行语句

begin;

select * from teacher where name > 'King' and name < 'Mark' and domain != 'MySQL' order by name desc for update;

```
mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from teacher where name > 'King' and name < 'Mark' and domain != 'MySQL' order by name desc for update;
Empty set (0.00 sec)

mysql>
```

再执行 show engine innodb status\G，会显示：

```
-----
TRANSACTIONS
-----
Trx id counter 12852
Purge done for trx's n:o < 12847 undo n:o < 0 state: running but idle
History list length 0
LIST OF TRANSACTIONS FOR EACH SESSION:
---TRANSACTION 421182911889920, not started
0 lock struct(s), heap size 1136, 0 row lock(s)
---TRANSACTION 421182911889008, not started
0 lock struct(s), heap size 1136, 0 row lock(s)
---TRANSACTION 421182911888096, not started
0 lock struct(s), heap size 1136, 0 row lock(s)
---TRANSACTION 12851, ACTIVE 19 sec
2 lock struct(s), heap size 1136, 3 row lock(s)
MySQL thread id 8, OS thread handle 139707640215296, query id 143 localhost root
TABLE LOCK table `mysqladv`.`teacher` trx id 12851 lock mode IX
RECORD LOCKS space id 33 page no 3 n bits 72 index PRIMARY of table `mysqladv`.`teacher` trx id 12851 lock_mode X
Record lock, heap no 1 PHYSICAL RECORD: n_fields 1; compact format; info bits 0
 0: len 8; hex 73757072656d756d; asc supremum;;

Record lock, heap no 5 PHYSICAL RECORD: n_fields 5; compact format; info bits 0
 0: len 4; hex 8000000f; asc ;;
 1: len 6; hex 000000003223; asc 2#;;
 2: len 7; hex be000001450110; asc E ;;
 3: len 4; hex 4b696e67; asc King;;
 4: len 3; hex 4a564d; asc JVM;;

Record lock, heap no 6 PHYSICAL RECORD: n_fields 5; compact format; info bits 0
 0: len 4; hex 80000015; asc ;;
 1: len 6; hex 000000003228; asc 2(;;
 2: len 7; hex a1000001120110; asc ;;
 3: len 5; hex 4461466569; asc DaFei;;
 4: len 5; hex 4d7953514c; asc MySQL;;
```

这样就显示的很清楚了。

1、

```
TABLE LOCK table `mysqladv`.`teacher` trx id 12851 lock mode IX
```

表示事务 ID 为 12851 对 mysqladv 下的 teacher 表加了表级意向独占锁。

2、

```
RECORD LOCKS space id 33 page no 4 n bits 72 index idx_name of table `mysqladv`.`teacher` trx id 12852 lock_mode X locks gap before rec
```

RECORD LOCKS space id 33 page no 4 n bits 72 index idx_name of table
`mysqladv`.`teacher` trx id 12852 lock_mode X locks gap before rec

表示一个内存中的锁结构，

space id 33: 表空间 id 为 33;

page no 3: 页编号为 4;

index PRIMARY: 对应的索引是 idx_name;

lock_mode X locks gap before rec: 存放的是一个 X 型的 gap 锁

```
Record lock, heap no 6 PHYSICAL RECORD: n_fields 2; compact format; info bits 0
 0: len 4; hex 4d61726b; asc Mark;;
 1: len 4; hex 80000003; asc ;;
```

表示的加锁记录的详细信息

3、

```
RECORD LOCKS space id 33 page no 4 n bits 72 index idx_name of table `mysqladv`.`teacher` trx id 12852 lock_mode X
```

RECORD LOCKS space id 33 page no 4 n bits 72 index idx_name of table
`mysqladv`.`teacher` trx id 12852 lock_mode X

表示一个内存中的锁结构，

space id 33: 表空间 id 为 33;

page no 3: 页编号为 4;

index PRIMARY: 对应的索引是 idx_name;

lock_mode X: 存放的是一个 X 型的 next-key 锁

4、如果是记录锁，则会显示 “lock_mode X locks rec but not gap”

8.7. 死锁和空间锁

一般来说，只要有并发和加锁这两种情况的共同加持下，都会有死锁的身影。死锁的具体成因，借用我们在并发编程中的内容：

8.7.1. 死锁

8.7.1.1. 概念

是指两个或两个以上的进程在执行过程中，由于竞争资源或者由于彼此通信而造成的一种阻塞的现象，若无外力作用，它们都将无法推进下去。此时称系统处于死锁状态或系统产生了死锁。

举个例子：A 和 B 去按摩洗脚，都想在洗脚的时候，同时顺便做个头部按摩，13 技师擅长足底按摩，14 擅长头部按摩。

这个时候 A 先抢到 14，B 先抢到 13，两个人都想同时洗脚和头部按摩，于是就互不相让，扬言我死也不让你，这样的话，A 抢到 14，想要 13，B 抢到 13，想要 14，在这个想同时洗脚和头部按摩的事情上 A 和 B 就产生了死锁。怎么解决这个问题呢？

第一种，假如这个时候，来了个 15，刚好也是擅长头部按摩的，A 又没有两个脑袋，自然就归了 B，于是 B 就美滋滋的洗脚和做头部按摩，剩下 A 在旁边气鼓鼓的，这个时候死锁这种情况就被打破了，不存在了。

第二种，C 出场了，用武力强迫 A 和 B，必须先做洗脚，再头部按摩，这种情况下，A 和 B 谁先抢到 13，谁就可以进行下去，另外一个没抢到的，就等着，这种情况下，也不会产生死锁。

所以总结一下：

死锁是必然发生在多操作者 ($M \geq 2$ 个) 情况下，争夺多个资源 ($N \geq 2$ 个，且 $N \leq M$) 才会发生这种情况。很明显，单线程自然不会有死锁，只有 B 一个去，不要 2 个，打十个都没问题；单资源呢？只有 13，A 和 B 也只会产生激烈竞争，打得不可开交，谁抢到就是谁的，但不会产生死锁。同时，死锁还有几个要求，1、争夺资源的顺序不对，如果争夺资源的顺序是一样的，也不会产生死锁；

2、争夺者拿到资源不放手。

8.7.1.2. 学术化的定义

死锁的发生必须具备以下四个必要条件。

1) 互斥条件：指进程对所分配到的资源进行排它性使用，即在一段时间内某资源只由一个进程占用。如果此时还有其它进程请求资源，则请求者只能等待，直至占有资源的进程用毕释放。

2) 请求和保持条件：指进程已经保持至少一个资源，但又提出了新的资源请求，而该资源已被其它进程占有，此时请求进程阻塞，但又对自己已获得的其它资源保持不放。

3) 不剥夺条件：指进程已获得的资源，在未使用完之前，不能被剥夺，只能在使用完时由自己释放。

4) 环路等待条件：指在发生死锁时，必然存在一个进程——资源的环形链，即进程集合{P0, P1, P2, ..., Pn}中的 P0 正在等待一个 P1 占用的资源；P1 正在等待 P2 占用的资源，……，Pn 正在等待已被 P0 占用的资源。

理解了死锁的原因，尤其是产生死锁的四个必要条件，就可以最大可能地避免、预防和解除死锁。

只要打破四个必要条件之一就能有效预防死锁的发生。

打破互斥条件：改造独占性资源为虚拟资源，大部分资源已无法改造。

打破不可抢占条件：当一进程占有一独占性资源后又申请一独占性资源而无法满足，则退出原占有的资源。

打破占有且申请条件：采用资源预先分配策略，即进程运行前申请全部资源，满足则运行，不然就等待，这样就不会占有且申请。

打破循环等待条件：实现资源有序分配策略，对所有设备实现分类编号，所有进程只能采用按序号递增的形式申请资源。

避免死锁常见的算法有有序资源分配法、银行家算法。

8.7.1.3. MySQL 中的死锁

所以 MySQL 中的死锁的成因是一样的。

会话 1:

begin;

select * from teacher where number = 1 for update;

```
mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from teacher where number = 1 for update;
+-----+-----+-----+
| number | name  | domain |
+-----+-----+-----+
|      1 | Jack | 源码系列 |
+-----+-----+-----+
1 row in set (0.00 sec)
```

会话 2:

begin;

select * from teacher where number = 3 for update;

```
mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from teacher where number = 3 for update;
+-----+-----+-----+
| number | name  | domain |
+-----+-----+-----+
|      3 | Mark | 并发编程 |
+-----+-----+-----+
1 row in set (0.00 sec)
```

会话 1:

select * from teacher where number = 3 for update;

可以看到这个语句的执行将会被阻塞

会话 2:

select * from teacher where number = 1 for update;

```
mysql> select * from teacher where number = 1 for update;
ERROR 1213 (40001): Deadlock found when trying to get lock; try restarting transaction
```

MySQL 检测到了死锁，并结束了会话 2 中事务的执行，此时，切回会话 1，发现原本阻塞的 SQL 语句执行完成了。

```
mysql> select * from teacher where number = 3 for update;
+-----+-----+-----+
| number | name  | domain |
+-----+-----+-----+
|      3 | Mark | 并发编程 |
+-----+-----+-----+
1 row in set (15.07 sec)
```

同时通过

show engine innodb status\G

可以看见死锁的详细情况:


```

LATEST DETECTED DEADLOCK
-----
2021-04-14 12:19:37 0x7fadbde65700
*** (1) TRANSACTION:
TRANSACTION 14037, ACTIVE 41 sec starting index read
mysql tables in use 1, locked 1
LOCK WAIT 3 lock struct(s), heap size 1136, 2 row lock(s)
MySQL thread id 124, OS thread handle 140385570518784, query id 5876 localhost root statis
select * from teacher where number = 3 for update
*** (1) WAITING FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 33 page no 3 n bits 80 index PRIMARY of table `mysqldadv`.`teacher` t
Record lock, heap no 3 PHYSICAL RECORD: n_fields 5; compact format; info bits 0
 0: len 4; hex 80000003; asc      ;;
 1: len 6; hex 00000000321d; asc      2 ;;
 2: len 7; hex ba0000012f0110; asc      / ;;
 3: len 4; hex 4d61726b; asc Mark;;
 4: len 12; hex e5b9b6e58f91e7bc96e7a88b; asc      ;;

*** (2) TRANSACTION:
TRANSACTION 14038, ACTIVE 29 sec starting index read
mysql tables in use 1, locked 1
3 lock struct(s), heap size 1136, 2 row lock(s)
MySQL thread id 123, OS thread handle 140385265800960, query id 5877 localhost root statis
select * from teacher where number = 1 for update
*** (2) HOLDS THE LOCK(S):
RECORD LOCKS space id 33 page no 3 n bits 80 index PRIMARY of table `mysqldadv`.`teacher` t
Record lock, heap no 3 PHYSICAL RECORD: n_fields 5; compact format; info bits 0
 0: len 4; hex 80000003; asc      ;;
 1: len 6; hex 00000000321d; asc      2 ;;
 2: len 7; hex ba0000012f0110; asc      / ;;
 3: len 4; hex 4d61726b; asc Mark;;
 4: len 12; hex e5b9b6e58f91e7bc96e7a88b; asc      ;;

*** (2) WAITING FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 33 page no 3 n bits 80 index PRIMARY of table `mysqldadv`.`teacher` t
Record lock, heap no 2 PHYSICAL RECORD: n_fields 5; compact format; info bits 0
 0: len 4; hex 80000001; asc      ;;
 1: len 6; hex 00000000321c; asc      2 ;;
 2: len 7; hex b90000012d0110; asc      - ;;
 3: len 4; hex 4a61636b; asc Jack;;
 4: len 12; hex e6ba90e7a081e7b3bbe58897; asc      ;;

*** WE ROLL BACK TRANSACTION (2)

```

8.7.2. Predicate Locks for Spatial Indexes

从 MySQL5.7 开始 MySQL 整合了 boost.geometry 库以更好的支持空间数据类型，并支持在在 Spatial 数据类型的列上构建索引，在 InnoDB 内，这个索引和普通的索引有所不同，基于 R-TREE 的结构，目前支持对 2D 数据的描述，暂不支持 3D。

R-TREE 和 BTREE 不同，它能够描述多维空间，而多维数据并没有明确的数据顺序，因此无法在 RR 隔离级别下构建 NEXT-KEY 锁以避免幻读，因此 InnoDB 使用称为 Predicate Lock 的锁模式来加锁，会锁住一块查询用到的被称为 MBR(minimum boundingrectangle/box)的数据区域。因此这个锁不是锁到某个具体的记录之上的，可以理解作为一种 Page 级别的锁。

Predicate Lock 和普通的记录锁或者表锁（如上所述）存储在不同的 lock hash 中，其相互之间不会产生冲突。

9.常见面试题

MySQL 中有哪些存储引擎？

InnoDB 存储引擎

InnoDB 是 MySQL 的默认事务型引擎，也是最重要、使用最广泛的存储引擎。它被设计用来处理大量的短期(short-lived)事务，应该优先考虑 InnoDB 引擎。

MyISAM 存储引擎

在 MySQL 5.1 及之前的版本，MyISAM 是默认的存储引擎。MyISAM 提供了大量的特性，包括全文索引、压缩、空间函数（GIS）等，但 MyISAM 不支持事务和行级锁，而且崩溃后无法安全恢复。MyISAM 对整张表加锁，很容易因为表锁的问题导致典型的性能问题。

Mrg_MyISAM

Merge 存储引擎，是一组 MyISAM 的组合，也就是说，他将 MyISAM 引擎的多个表聚合起来，但是他的内部没有数据，真正的数据依然是 MyISAM 引擎的表中，但是可以直接进行查询、删除更新等操作。

Archive 引擎

Archive 存储引擎只支持 INSERT 和 SELECT 操作，会缓存所有的写并利用 zlib 对插入的行进行压缩，所以比 MyISAM 表的磁盘 I/O 更少。但是每次 SELECT 查询都需要执行全表扫描。所以 Archive 表适合日志和数据采集类应用，Archive 引擎是一个针对高速插入和压缩做了优化的简单引擎。

Blackhole 引擎

Blackhole 引擎没有实现任何的存储机制，它会丢弃所有插入的数据，不做任何保存。可以在一些特殊的复制架构和日志审核时发挥作用。但这种引擎在应用方式上有很多问题，因此并不推荐。

CSV 引擎

CSV 引擎可以将普通的 CSV 文件(逗号分割值的文件)作为 MySQL 的表来处理，但这种表不支持索引。因此 CSV 引擎可以作为一种数据交换的机制，非常有用。

Federated 引擎

Federated 引擎是访问其他 MySQL 服务器的一个代理，它会创建一个到远程 MySQL 服务器的客户端连接，并将查询传输到远程服务器执行，然后提取或者发送需要的数据。默认是禁用的。

Memory 引擎

Memory 表至少比 MyISAM 表要快一个数量级，数据文件是存储在内存中。Memory 表的结构在重启以后还会保留，但数据会丢失。

Memory 表在很多场景可以发挥好的作用：

用于查找(lookup)或者映射(mapping)表，例如将邮编和州名映射的表。

用于缓存周期性聚合数据(periodically aggregated data)的结果。

用于保存数据分析中产生的中间数据。

Memory 表支持 Hash 索引，因此查找操作非常快。Memory 表是表级锁，因此并发写入的性能较低，每行的长度是固定的，可能导致部分内存的浪费。

NDB 集群引擎

使用 MySQL 服务器、NDB 集群存储引擎，以及分布式的、share-nothing 的、容灾的、高可用的 NDB 数据库的组合，被称为 MySQL 集群 (MySQL Cluster)。

MyISAM 和 InnoDB 的区别是什么？

MyISAM 引擎是 5.1 版本之前的默认引擎，支持全文检索、压缩、空间函数等，但是不支持事务和行级锁，所以一般用于有大量查询少量插入的场景来使用，而且 MyISAM 不支持外键，并且索引和数据是分开存储的。

InnoDB 是基于聚簇索引建立的，和 MyISAM 相反它支持事务、外键，并且通过 MVCC 来支持高并发，索引和数据存储在一起。

请概述下数据库的范式设计

目前关系数据库有六种范式，常见范式：第一范式：1NF 是对属性的原子性约束，要求属性具有原子性，不可再分解；第二范式：2NF 是对记录的惟一性约束，要求记录有惟一标识，即实体的惟一性；第三范式：3NF 是对字段冗余性的约束，即任何字段不能由其他字段派生出来，它要求字段没有冗余。。

范式化设计优缺点：

优点：可以尽量得减少数据冗余，使得更新快，体积小；缺点：对于查询需要多个表进行关联，减少写得效率增加读得效率，更难进行索引优化

反范式化：

优点：可以减少表得关联，可以更好得进行索引优化；缺点：数据冗余以及数据异常，数据得修改需要更多的成本，常见的反范式设计有缓存、冗余等等。

数据库表设计时，字段你会如何选择？

更小的通常更好，应该尽量使用可以正确存储数据的最小数据类型。更小的数据类型通常更快，因为它们占用更少的磁盘、内存和 CPU 缓存，并且处理时需要的 CPU 周期也更少。但是要确保没有低估需要存储的值的范围。

简单就好，简单数据类型的操作通常需要更少的 CPU 周期。例如，整型比字符操作代价更低，比如应该使用 MySQL 内建的类型而不是字符串来存储日期和时间。

尽量避免 NULL，如果查询中包含可为 NULL 的列，对 MySQL 来说更难优化，因为可为 NULL 的列使得索引、索引统计和值比较都更复杂。可为 NULL 的列会使用更多的存储空间，在 MySQL 里也需要特殊处理。当可为 NULL 的列被索引时，每个索引记录需要一个额外的字节。

mysql 里记录货币用什么字段类型好？

MySQL 既支持精确类型的存储 DECIMAL 类型，也支持不精确类型存储 FLOAT 和 DOUBLE 类型。对于货币记录，应该选择 DECIMAL 类型，但是 DECIMAL 类型是以字符串形式存放的，所以性能会有影响。

作为替代方案，可以在数据量比较大的而且要求精度时，虑使用 **BIGINT** 代替 **DECIMAL**，将需要存储的货币单位根据小数的位数乘以相应的倍数即可。

谈谈 MySQL 里的字符串类型

MySQL 里的字符串类型有：**SET**、**BLOB**、**ENUM**、**VARCHAR**、**CHAR**、**TEXT**。**VARCHAR** 和 **CHAR** 是两种最主要的字符串类型。**VARCHAR** 类型用于存储可变长字符串，大部分的业务情况下比定长类型更节省空间，**CHAR** 类型是定长的，**CHAR** 适合存储很短的字符串，或者所有值定长或都接近同一个长度。

使用 **BLOB** 和 **TEXT** 则要慎重，一般把 **BLOB** 或 **TEXT** 列分离到单独的表中，还可以对 **BLOB** 或 **TEXT** 列使用合成的(**Synthetic**)索引，就是根据大文本字段的内容建立一个散列值并单独存储在数据列中，可以通过检索散列值找到数据行。如果表中的字段的取值是固定几个字符串，可以使用枚举列代替常用的字符串类型。

VARCHAR(M)最多能存储多少数据？

对于 **VARCHAR(M)**类型的列最多可以定义 65535 个字节。其中的 **M** 代表该类型最多存储的字符数量，但在实际存储时并不能放这么多。

MySQL 对一条记录占用的最大存储空间是有限制的，除了 **BLOB** 或者 **TEXT** 类型的列之外，其他所有的列（不包括隐藏列和记录头信息）占用的字节长度加起来不能超过 65535 个字节。所以 MySQL 服务器建议我们把存储类型改为 **TEXT** 或者 **BLOB** 的类型。这个 65535 个字节除了列本身的数据之外，还包括一些其他的数据，从行记录格式我们可以得知，为了存储一个 **VARCHAR(M)**类型的列，实际需要占用 3 部分存储空间：真实数据、真实数据占用字节的长度、**NULL** 值标识，如果该列有 **NOT NULL** 属性则可以没有这部分存储空间。

我们假设表中只有一个 **VARCHAR** 字段的情况：

如果该 **VARCHAR** 类型的列没有 **NOT NULL** 属性，那最多只能存储 65532 个字节的数据，因为真实数据的长度可能占用 2 个字节，**NULL** 值标识需要占用 1 个字节。

如果 **VARCHAR** 类型的列有 **NOT NULL** 属性，那最多只能存储 65533 个字节的数据，因为真实数据的长度可能占用 2 个字节，不需要 **NULL** 值标识。

如果 **VARCHAR(M)**类型的列使用的不是 **ascii** 字符集，那 **M** 的最大取值取决于该字符集表示一个字符最多需要的字节数。在列的值允许为 **NULL** 的情况下，**gbk** 字符集表示一个字符最多需要 2 个字节，那在该字符集下，**M** 的最大取值就是 32766（也就是：65532/2），也就是说最多能存储 32766 个字符；**utf8** 字符集表示一个字符最多需要 3 个字节，那在该字符集下，**M** 的最大取值就是 21844，就是说最多能存储 21844（也就是：65532/3）个字符。

不管如何，请牢记：**MySQL** 一个行中的所有列（不包括隐藏列和记录头信息）占用的字节长度加起来不能超过 65535 个字节。

什么是虚拟生成列？

虚拟生成列又叫 **Generated Column**，是 MySQL 5.7 引入的新特性，就是数据库中这一列由其他列计算而得。在 MySQL 5.7 中，支持两种 **Generated Column**，

即 Virtual Generated Column（虚拟生成的列）和 Stored Generated Column（存储生成的列），二者含义如下：

1、Virtual Generated Column（虚拟生成的列）：不存储该列值，即 MySQL 只是将这一列的元信息保存在数据字典中，并不会将这一列数据持久化到磁盘上，而是当读取该行时，触发触发器对该列进行计算显示。

2、Stored Generated Column（存储生成的列）：存储该列值，即该列值在插入或更新行时进行计算和存储。所以相对于 Virtual Column 列需要更多的磁盘空间，与 Virtual Column 相比并没有优势。因此，MySQL 5.7 中，不指定 Generated Column 的类型，默认是 Virtual Column

在表中允许 Virtual Column 和 Stored Column 的混合使用

提高效率：由于 mysql 在普通索引上加函数会造成索引失效，造成查询性能下降，Generated Column（函数索引）刚好可以解决这个问题，可以在 Generated Column 加上索引来提高效率。但是不能建立虚拟列和真实列的联合索引，同时虚拟列是不允许创建主键索引和全文索引。

创建虚拟生成列的语法：

```
CREATE TABLE `triangle` (  
  `a` double DEFAULT NULL,  
  `b` double DEFAULT NULL,  
  `sided` double GENERATED ALWAYS AS (SQRT(a * a + b * b))  
);
```

```
alter table triangle add column sided tinyint(1) generated always as (a*b)  
virtual;
```

请说下事务的基本特性

事务应该具有 4 个属性：原子性、一致性、隔离性、持久性。这四个属性通常称为 ACID 特性。

原子性指的是一个事务中的操作要么全部成功，要么全部失败。

一致性指的是数据库总是从一个一致性的状态转换到另外一个一致性的状态。比如 A 转账给 B 100 块钱，假设中间 sql 执行过程中系统崩溃 A 也不会损失 100 块，因为事务没有提交，修改也就不会保存到数据库。

隔离性指的是一个事务的修改在最终提交前，对其他事务是不可见的。

持久性指的是一旦事务提交，所做的修改就会永久保存到数据库中。

事务并发可能引发什么问题？

当一个事务读取到了另外一个事务修改但未提交的数据，被称为脏读。

当事务内相同的记录被检索两次，且两次得到的结果不同时，此现象称为不可重复读。

在事务执行过程中，事务 2 将新记录添加到正在读取的事务 1 中，导致事务 1 按照某个相同条件多次读取记录时，后读取时读到了之前没有读到的记录，发生幻读。

事务 2 中是删除了符合的记录而不是插入新记录，那事务 1 中之后再根据条件读取的记录变少了，在 MySQL 中这种现象不属于幻读，相当于对每一条记录都发生了不可重复读的现象。

请描述下 MySQL 中 InnoDB 支持的四种事务隔离和区别

read uncommitted: 未提交读，可能发生脏读、不可重复读和幻读问题。

read committed: 提交读，可能发生不可重复读和幻读问题，但是不会发生脏读问题。

repeatable read: 可重复读，在 SQL 标准中可能发生幻读问题，但是不会发生脏读和不可重复读的问题，但是 MySQL 通过 MVCC 基本解决了幻读问题。这也是 MySQL 的缺省隔离级别。

serializable: 串行化读，脏读、不可重复读和幻读问题都不会发生。

MySQL 有哪些索引类型

从数据结构角度可分为 B+ 树索引、哈希索引、以及 FULLTEXT 索引（现在 MyISAM 和 InnoDB 引擎都支持了）和 R-Tree 索引（用于对 GIS 数据类型创建 SPATIAL 索引）；

从物理存储角度可分为聚集索引（clustered index）、非聚集索引（non-clustered index）；

从逻辑角度可分为主键索引、普通索引，或者单列索引、多列索引、唯一索引、非唯一索引等等。

简单描述 MySQL 各个索引的区别

索引是一种特殊的文件(InnoDB 数据表上的索引是表空间的一个组成部分)，它们包含着对数据表里所有记录的引用指针。

普通索引(由关键字 KEY 或 INDEX 定义的索引)的唯一任务是加快对数据的访问速度。

普通索引允许被索引的数据列包含重复的值。如果能确定某个数据列将只包含彼此各不相同的值，在为这个数据列创建索引的时候就应该用关键字 UNIQUE 把它定义为一个唯一索引。也就是说，唯一索引可以保证数据记录的唯一性。

主键，是一种特殊的唯一索引，在一张表中只能定义一个主键索引，主键用于唯一标识一条记录，使用关键字 PRIMARY KEY 来创建。

索引可以覆盖多个数据列，如像 INDEX(columnA, columnB)索引，这就是联合索引。

MySQL 的索引对数据库的性能有什么影响

索引 (Index) 是帮助 MySQL 高效获取数据的数据结构, 所以索引可以极大的提高数据的查询速度。

但是每建立一个索引都要为它建立一棵 B+树, 一棵很大的 B+树由许多数据页组成会占据很多的存储空间。

而且每次对表中的数据进行增、删、改操作时, 都需要去修改各个 B+树索引, 同时这些操作可能会对节点和记录的排序造成破坏, 所以存储引擎需要额外的时间进行一些记录移位, 页面分裂、页面回收的操作来维护好节点和记录的排序。如果我们建了许多索引, 每个索引对应的 B+树都要进行相关的维护操作, 这必然会对性能造成影响。

为什么 MySQL 的索引要使用 B+树而不是 B 树?

答案见下一小节

InnoDB 一棵 B+树可以存放多少行数据?

当然在实际的数据库中, 一个节点可以存储的数据可以很多, 为什么?

计算机在存储数据的时候, 有最小存储单元, 这就好比我们今天进行现金的流通最小单位是一毛。在计算机中磁盘存储数据最小单元是扇区, 一个扇区的大小是 512 字节, 而文件系统 (例如 XFS/EXT4) 他的最小单元是块, 一个块的大小是 4k, 而对于我们的 InnoDB 存储引擎也有自己的最小储存单元——页 (Page), 一个页的大小是 16K。InnoDB 的所有数据文件 (后缀为 ibd 的文件), 他的大小始终都是 16384 (16k) 的整数倍。

```
[mysql@izwz9j203ithc4guluvb2wZ mysqladv]$ ls -l
total 432
-rw-r----- 1 mysql mysql    65 Mar 20 17:07 db.opt
-rw-r----- 1 mysql mysql   9015 Mar 22 18:43 order_exp.frm
-rw-r----- 1 mysql mysql  294912 Mar 22 18:43 order_exp.ibd
-rw-r----- 1 mysql mysql   8556 Mar 20 17:50 test.frm
-rw-r----- 1 mysql mysql   98304 Mar 22 14:35 test.ibd
-rw-r----- 1 mysql mysql   8556 Mar 20 23:27 test_myisam.frm
-rw-r----- 1 mysql mysql     0 Mar 20 23:27 test_myisam.MYD
-rw-r----- 1 mysql mysql  1024 Mar 20 23:27 test_myisam.MYI
```

数据表中的数据都是存储在页中的, 所以一个页中能存储多少行数据呢? 假设一行数据的大小是 1k, 那么一个页可以存放 16 行这样的数据。

对于 B+树而言, 只有叶子节点存放数据, 非叶子节点存放的是只保存索引信息和下一层节点的指针信息。一个非叶子节点能存放多少指针?

其实这也很好算, 我们假设主键 ID 为 常用的 bigint 类型, 长度为 8 字节, 而指针大小在 InnoDB 源码中设置为 6 字节, 这样一共 14 字节, 我们一个页中能存放多少这样的单元, 其实就代表有多少指针, 即 $16384/14=1170$ 个。

那么可以算出一棵高度为 2 的 B+树, 存在一个根节点和若干个叶子节点能存放 $1170*16=18720$ 条这样的数据记录。

根据同样的原理我们可以算出一个高度为 3 的 B+ 树可以存放:
 $1170*1170*16=21902400$ 条这样的记录。

所以在 InnoDB 中 B+ 树高度一般为 1-3 层，就能满足千万级的数据存储。

那么为什么 MySQL 的索引要使用 B+树而不是 B 树？

而 B 树和 B+树的最大区别就是，B 树不管叶子节点还是非叶子节点，都会保存数据，这样导致在非叶子节点中能保存的指针数量变少（有些资料也称为扇出），指针少的情况下要保存大量数据，只能增加树的高度，导致 IO 操作变多，查询性能变低。

HashMap 适合做数据库索引吗？

- 1、hash 表只能匹配是否相等，不能实现范围查找；
- 2、当需要按照索引进行 order by 时，hash 值没办法支持排序；
- 3、组合索引可以支持部分索引查询，如(a,b,c)的组合索引，查询中只用到了 A 和 b 也可以查询的，如果使用 hash 表，组合索引会将几个字段合并 hash，没办法支持部分索引；
- 4、当数据量很大时，hash 冲突的概率也会非常大。

InnoDB 中只有 B+树索引吗？

InnoDB 存储引擎不仅仅有 B+树索引，它还支持全文索引、哈希索引。

InnoDB 存储引擎内部自己去监控索引表，如果监控到某个索引经常用，那么就认为是热数据，然后内部自己创建一个 hash 索引，称之为自适应哈希索引 (Adaptive Hash Index, AHI)。使用的哈希函数采用除法散列方式，其冲突机制采用链表方式。我们对这个自适应哈希索引能够干预的地方很少，只能设定是否启用和分区个数。

从 MySQL 5.6.x 开始，InnoDB 开始支持全文检索，内部的实现机制就是倒排索引。但是 MySQL 整体架构上对全文检索支持并不好而且限制很多，比如每张表只能有一个全文检索的索引，不支持没有单词界定符 (delimiter) 的语言，所以如果有大批量或者专门的全文检索需求，还是应该选择专门的全文检索引擎。

什么是密集索引和稀疏索引？

密集索引的定义：叶子节点保存的不只是键值，还保存了位于同一行记录里的其他列的信息，由于密集索引决定了表的物理排列顺序，一个表只有一个物理排列顺序，所以一个表只能创建一个密集索引。

稀疏索引：叶子节点仅保存了键位信息以及该行数据的地址，有的稀疏索引只保存了键位信息机器主键。

mysam 存储引擎，不管是主键索引，唯一键索引还是普通索引都是稀疏索引，innodb 存储引擎：有且只有一个密集索引。

所以，密集索引就是 innodb 存储引擎里的聚簇索引，稀疏索引就是 innodb 存储引擎里的普通二级索引。

为什么要用自增列作为主键？

1、如果我们定义了主键(PRIMARY KEY)，那么 InnoDB 会选择主键作为聚集索引。

如果没有显式定义主键，则 InnoDB 会选择第一个不包含有 NULL 值的唯一索引作为主键索引。

如果也没有这样的唯一索引，则 InnoDB 会选择内置 6 字节长的 ROWID 作为隐含的聚集索引(ROWID 随着行记录的写入而主键递增，这个 ROWID 不像 ORACLE 的 ROWID 那样可引用，是隐含的)。

2、数据记录本身被存于主索引（一颗 B+Tree）的叶子节点上，这就要求同一个叶子节点内（大小为一个内存页或磁盘页）的各条数据记录按主键顺序存放

因此每当有一条新的记录插入时，MySQL 会根据其主键将其插入适当的节点和位置，如果页面达到装载因子（InnoDB 默认为 15/16），则开辟一个新的页（节点）

3、如果表使用自增主键，那么每次插入新的记录，记录就会顺序添加到当前索引节点的后续位置，当一页写满，就会自动开辟一个新的页

4、如果使用非自增主键（如果身份证号或学号等），由于每次插入主键的值近似于随机，因此每次新纪录都要被插到现有索引页得中间某个位置。此时 MySQL 不得不为了将新记录插到合适位置而移动数据，甚至目标页面可能已经被回写到磁盘上而从缓存中清掉，此时又要从磁盘上读回来，这增加了很多开销同时频繁的移动、分页操作造成了大量的碎片，得到了不够紧凑的索引结构，后续不得不通过 OPTIMIZE TABLE 来重建表并优化填充页面。

主键和唯一键有什么区别？

主键不能重复，不能为空，唯一键不能重复，可以为空。

建立主键的目的是让外键来引用。

一个表最多只有一个主键，但可以有很多唯一键

说说对 SQL 语句优化有哪些方法？（选择几条）

(1) Where 子句中：where 表之间的连接必须写在其他 Where 条件之前，那些可以过滤掉最大数量记录的条件必须写在 Where 子句的末尾.HAVING 最后。

(2) 用 EXISTS 替代 IN、用 NOT EXISTS 替代 NOT IN。

(3) 避免在索引列上使用计算

(4) 避免在索引列上使用 IS NULL 和 IS NOT NULL

(5)对查询进行优化，应尽量避免全表扫描，首先应考虑在 where 及 order by 涉及的列上建立索引。

(6) 应尽量避免在 where 子句中对字段进行 null 值判断，否则将导致引擎放弃使用索引而进行全表扫描

(7) 应尽量避免在 where 子句中对字段进行表达式操作，这将导致引擎放弃使用索引而进行全表扫描。

如何提高 insert 的性能？

答：有如下方法：

a)合并多条 insert 为一条，即：`insert into t values(a,b,c), (d,e,f), ,,`

原因分析：主要原因是多条 insert 合并后日志量（MySQL 的 binlog 和 innodb 的事务日志）减少了，降低日志刷盘的数据量和频率，从而提高效率。通过合并 SQL 语句，同时也能减少 SQL 语句解析的次数，减少网络传输的 IO。

b)修改参数 `bulk_insert_buffer_size`，调大批量插入的缓存；

c)设置 `innodb_flush_log_at_trx_commit = 0`，相对于 `innodb_flush_log_at_trx_commit = 1` 可以十分明显的提升导入速度；

（备注：`innodb_flush_log_at_trx_commit` 参数对 InnoDB Log 的写入性能有非常关键的影响。该参数可以设置为 0，1，2，解释如下：

0: log buffer 中的数据将以每秒一次的频率写入到 log file 中，且同时会进行文件系统到磁盘的同步操作，但是每个事务的 commit 并不会触发任何 log buffer 到 log file 的刷新或者文件系统到磁盘的刷新操作；

1: 在每次事务提交的时候将 log buffer 中的数据都会写入到 log file，同时也会触发文件系统到磁盘的同步；

2: 事务提交会触发 log buffer 到 log file 的刷新，但并不会触发磁盘文件系统到磁盘的同步。此外，每秒会有一次文件系统到磁盘同步操作。）

d) 手动使用事务

因为 mysql 默认是 autocommit 的，这样每插入一条数据，都会进行一次 commit；所以，为了减少创建事务的消耗，我们可用手工使用事务，即 `START TRANSACTION;insert ...;insert... commit;` 即执行多个 insert 后再一起提交；一般 1000 条 insert 提交一次。

什么是覆盖索引？什么是回表查询？

InnoDB 存储引擎有两大类索引聚集索引和普通（辅助/二级）索引，聚簇索引的叶子节点存储行记录，因此 InnoDB 必须要有聚簇索引且仅有一个聚簇索引，而普通索引的叶子节点只存储索引值和主键值，所以，通过聚簇索引一次性能获取所有列的数据，普通索引一般不行。

当我们 SQL 语句的中列无法在普通索引中获得时，就需要主键值到聚簇索引中获取相关的数据，这个过程就被称为回表。

而如果我们使用联合索引，使得 SQL 所需的所有列数据在这个索引上就能获得时，我们称为发生了索引覆盖或者覆盖索引。

什么是三星索引？

对于一个查询而言，一个三星索引，可能是其最好的索引。

如果查询使用三星索引，一次查询通常只需要进行一次磁盘随机读以及一次窄索引片的扫描，因此其相应时间通常比使用一个普通索引的响应时间少几个数量级。

一个查询相关的索引行是相邻的或者至少相距足够靠近的则获得一星；

如果索引中的数据顺序和查找中的排列顺序一致则获得二星；

如果索引中的列包含了查询中需要的全部列则获得三星。

三星索引在实际的业务中如果无法同时达到，一般我们认为第三颗星最重要，第一和第二颗星重要性差不多，根据业务情况调整这两颗星的优先度。

大表关联查询优化

一个 6 亿的表 a，一个 3 亿的表 b，通过 tid 关联，你如何最快的查询出满足条件的第 50000 到第 50200 中的这 200 条数据记录。

1、如果 A 表 TID 是自增长,并且是连续的,B 表的 ID 为索引

```
select * from a,b where a.tid = b.id and a.tid>500000 limit 200;
```

2、如果 A 表的 TID 不是连续的,那么就需要使用覆盖索引.TID 要么是主键,要么是辅助索引,B 表 ID 也需要有索引。

```
select * from b , (select tid from a limit 50000,200) a where b.id = a .tid;
```

[SELECT *] 和[SELECT 全部字段]有何优缺点？

1>.前者要解析数据字典，后者不需要

2>.结果输出顺序，前者与建表列顺序相同，后者按指定字段顺序。

3>.表字段改名，前者不需要修改，后者需要改

4>.后者可以建立索引进行优化，前者无法优化

5>.后者的可读性比前者要高

请概述下什么是 MySQL 的分区表

表分区，是指根据一定规则，将数据库中的一张表分解成多个更小的，容易管理的部分。从逻辑上看，只有一张表，但是底层却是由多个物理分区组成。

1、表分区与分表的区别

分表：指的是通过一定规则，将一张表分解成多张不同的表。比如将用户订单记录根据时间成多个表。

分表与分区的区别在于：分区从逻辑上来讲只有一张表，而分表则是将一张表分解成多张表。

2、表分区的好处？

1) 存储更多数据。分区表的数据可以分布在不同的物理设备上，从而高效地利用多个硬件设备。和单个磁盘或者文件系统相比，可以存储更多数据

2) 优化查询。在 where 语句中包含分区条件时，可以只扫描一个或多个分区表来提高查询效率；涉及 sum 和 count 语句时，也可以在多个分区上并行处理，最后汇总结果。

3) 分区表更容易维护。例如：想批量删除大量数据可以清除整个分区。

3、分区表的限制因素

一个表最多只能有 1024 个分区

如果分区字段中有主键或者唯一索引的列，那么多有主键列和唯一索引列都必须包含进来。即：分区字段要么不包含主键或者索引列，要么包含全部主键和索引列。

分区表中无法使用外键约束

MySQL 的分区适用于一个表的所有数据和索引，不能只对表数据分区而不对索引分区，也不能只对索引分区而不对表分区，也不能只对表的一部分数据分区。

4、分区表的类型

RANGE 分区： 这种模式允许将数据划分不同范围。例如可以将一个表通过年份划分成若干个分区

LIST 分区： 这种模式允许系统通过预定义的列表的值来对数据进行分割。按照 List 中的值分区，与 RANGE 的区别是，range 分区的区间范围值是连续的。

HASH 分区： 这种模式允许通过对表的一个或多个列的 Hash Key 进行计算，最后通过这个 Hash 码不同数值对应的数据区域进行分区。例如可以建立一个对表主键进行分区的表。

KEY 分区： 上面 Hash 模式的一种延伸，这里的 Hash Key 是 MySQL 系统产生的。

Column 分区： 需要和 RANGE 和 List 结合，支持字符串和日期的分区，也支持多列分区。

复合分区/子分区：分区之下还可以再分区。

5、在实际工作中用分区表比较少

- 1) 分区表，分区键设计不太灵活，如果不走分区键，很容易出现全表锁
- 2) 自己分库分表，自己掌控业务场景与访问模式，可控。分区表，研发写了一个 sql，都不确定 mysql 是怎么操作的，不太可控
- 3) 分区表无论怎么分，都是在一台机器上，天然就有性能的上限。

说几条 MySQL 对 SQL 的执行做的优化手段

1、对 SQL 语句的优化，MySQL 会对我们的 SQL 语句做重写，包括条件化简，比如常量传递、除没用的条件等等；还会将一些外连接转换为内连接，然后选择成本最低的方式执行；对 IN 子查询会进行物化、物化表转连接查询、转换为半连接等方式进行。

2、在 SQL 语句的执行过程中，MySQL 引入了索引条件下推。比如 where 后面的多个搜索条件都使用到了一个二级索引列，这些搜索条件中虽然出现了索引列，有些却不能使用到索引，像 like '%...' 查询，MySQL 为了避免不必要的回表，从二级索引取得的索引记录，先做条件判断，如果条件不满足，则该二级索引记录不会去回表，这样可以大量的减少回表操作的随机 IO 成本。

3、在回表操作上，因为每次执行回表操作时都相当于要随机读取一个聚簇索引页面，而这些随机 IO 带来的性能开销比较大。MySQL 中提出了一个名为 Disk-Sweep Multi-Range Read (MRR，多范围读取)的优化措施，即先读取一部分二级索引记录，将它们的主键值排好序之后再统一执行回表操作。

4、MySQL 在一般情况下执行一个查询时最多只会用到单个二级索引，但存

在有特殊情况，也可能在一个查询中使用到多个二级索引，称之为：索引合并，比如 Intersection 交集合并、Union 索引合并等等。

InnoDB 引擎的三大特性是什么？

InnoDB 的三大特性是：Buffer Pool、自适应 Hash 索引、双写缓冲区。

自适应 Hash 索引，InnoDB 存储引擎内部自己去监控索引表，如果监控到某个索引经常用，那么就认为是热数据，然后内部自己创建一个 hash 索引，称之为自适应哈希索引 (Adaptive Hash Index, AHI)，创建以后，如果下次又查询到这个索引，那么直接通过 hash 算法推导出记录的地址，直接一次就能查到数据。

InnoDB 存储引擎使用的哈希函数采用除法散列方式，其冲突机制采用链表方式。

Buffer Pool，为了提高访问速度，MySQL 预先就分配/准备了许多这样的空间，为的就是与 MySQL 数据文件中的页做交换，来把数据文件中的页放到事先准备好的内存中。数据的访问是按照页（默认为 16KB）的方式从数据文件中读取到 buffer pool 中。Buffer Pool 按照最少使用算法 (LRU)，来管理内存中的页。

Buffer Pool 实例允许有多个，每个实例都有一个专门的 mutex 保护。Buffer Pool 中缓存的数据页类型有：索引页、数据页、undo 页、插入缓冲 (insert buffer)、自适应哈希索引、InnoDB 存储的锁信息、数据字典信息 (data dictionary) 等等。

双写缓冲区，是一个位于系统表空间的存储区域，在写入时，InnoDB 先从缓冲池中的得到的 page 写入系统表空间的双写缓冲区。之后，再把 page 写到 .ibd 数据文件中相应的位置。如果在 page 写入数据文件的过程中发生意外崩溃，InnoDB 在稍后的恢复过程中在 doublewrite buffer 中找到完好的 page 副本用于恢复。

doublewrite 是顺序写，开销比较小。所以在正常的情况下，MySQL 写数据 page 时，会写两遍到磁盘上，第一遍是写到 doublewrite buffer，第二遍是从 doublewrite buffer 写到真正的数据文件中。

它的主要作用是为了避免 partial page write（部分页写入）的问题。因为 InnoDB 的 page size 一般是 16KB，校验和写入到磁盘是以 page 为单位进行的。而操作系统写文件是以 4KB 作为单位的，每写一个 page，操作系统需要写 4 个块，中间发生了系统断电或系统崩溃，只有一部分页面是写入成功的。这时 page 数据出现不一样的情形，从而形成一个“断裂”的 page，使数据产生混乱。

redolog 和 binlog 的区别是什么？

答案见下一题

MySQL 崩溃后的恢复为什么不用 binlog？

1、这两者使用方式不一样

binlog 会记录表所有更改操作，包括更新删除数据，更改表结构等等，主要用于人工恢复数据，而 redo log 对于我们是不可见的，它是 InnoDB 用于保证 crash-safe 能力的，也就是在事务提交后 MySQL 崩溃的话，可以保证事务的持久性，即事务提交后其更改是永久性的。

一句话概括：binlog 是用作人工恢复数据，redo log 是 MySQL 自己使用，用于保证在数据库崩溃时的事务持久性。

2、redo log 是 InnoDB 引擎特有的，binlog 是 MySQL 的 Server 层实现的，所有引擎都可以使用。

3、redo log 是物理日志，记录的是“在某个数据页上做了什么修改”，恢复的速度更快；binlog 是逻辑日志，记录的是这个语句的原始逻辑，比如“给 ID=2 这的 c 字段加 1”，binlog 有三种日志记录格式 Row、SQL、混合模式。

4、redo log 是“循环写”的日志文件，redo log 只会记录未刷盘的日志，已经刷入磁盘的数据都会从 redo log 这个有限大小的日志文件里删除。binlog 是追加日志，保存的是全量的日志。

5、最重要的是，当数据库 crash 后，想要恢复未刷盘但已经写入 redo log 和 binlog 的数据到内存时，binlog 是无法恢复的。虽然 binlog 拥有全量的日志，但没有一个标志让 innoDB 判断哪些数据已经入表(写入磁盘)，哪些数据还没有。

比如，binlog 记录了两条日志：

给 ID=2 这一行的 c 字段加 1

给 ID=2 这一行的 c 字段加 1

在记录 1 入表后，记录 2 未入表时，数据库 crash。重启后，只通过 binlog 数据库无法判断这两条记录哪条已经写入磁盘，哪条没有写入磁盘，不管是两条都恢复至内存，还是都不恢复，对 ID=2 这行数据来说，都不对。

但 redo log 不一样，只要刷入磁盘的数据，都会从 redo log 中抹掉，数据库重启后，直接把 redo log 中的数据都恢复至内存就可以了。

MySQL 如何实现事务的 ACID？

参见下个问题。

InnoDB 事务是如何通过日志来实现的？

总的来说，事务的原子性是通过 undo log 来实现的，事务的持久性是通过 redo log 来实现的，事务的隔离性是通过读写锁+MVCC 来实现的。

事务的一致性通过原子性、隔离性、持久性来保证。也就是说 ACID 四大特性之中，C(一致性)是目的，A(原子性)、I(隔离性)、D(持久性)是手段，是为了保证一致性，数据库提供的手段。数据库必须要实现 AID 三大特性，才有可能实现一致性。同时一致性也需要应用程序的支持，应用程序在事务里故意写出违反约束的代码，一致性还是无法保证的，例如，转账代码里从 A 账户扣钱而不给 B 账户加钱，那一一致性还是无法保证。

至于 InnoDB 事务是如何通过日志来实现的，简单来说，因为事务在修改页时，要先记 undo，在记 undo 之前要记 undo 的 redo，然后修改数据页，再记数据页修改的 redo。Redo（里面包括 undo 的修改）一定要比数据页先持久化到磁盘。

当事务需要回滚时，因为有 undo，可以把数据页回滚到前镜像的状态，崩溃恢复时，如果 redo log 中事务没有对应的 commit 记录，那么需要用 undo

把该事务的修改回滚到事务开始之前。如果有 `commit` 记录,就用 `redo` 前滚到该事务完成时并提交掉。

更详细的回答是:

`redo` 通常是物理日志,记录的是页的物理修改操作,用来恢复提交事务修改的页操作。而 `undo` 是逻辑日志,根据每行记录进行记录,用来回滚记录到某个特定的版本。

当事务提交之后会把所有修改信息都会存到 `redo` 日志中。`redo` 日志由两部分组成,一个是在内存里的 `redo log buffer`,另一个是在磁盘里的 `redo log` 文件。

`mysql` 为了提升性能不会把每次的修改都实时同步到磁盘,而是会先存到 `Buffer Pool`(缓冲池)里头,把这个当作缓存来用。然后使用后台线程去做缓冲池和磁盘之间的同步。

系统重启后读取 `redo log` 恢复最新数据。虽然 `redo log` 会在事务提交前做一次磁盘写入,但是这种 `IO` 操作相比于 `buffer pool` 这种以页(16kb)为管理单位的随机写入,它做的是几个字节的顺序写入,效率要高得多。

`redo log buffer` 中的数据,会在一个合适的时间点刷入到磁盘中。

这个合适的时间点包括:

- 1、MySQL 正常关闭的时候;
- 2、MySQL 的后台线程每隔一段时间定时的讲 `redo log buffer` 刷入到磁盘,默认是每隔 1s 刷一次;
- 3、当 `redo log buffer` 中的日志写入量超过 `redo log buffer` 内存的一半时,即超过 8MB 时,会触发 `redo log buffer` 的刷盘;
- 4、当事务提交时,根据配置的参数 `innodb_flush_log_at_trx_commit` 来决定是否刷盘。要严格保证数据不丢失,必须得保证 `innodb_flush_log_at_trx_commit` 配置为 1。

`redo log` 在进行数据重做时,只有读到了 `commit` 标识,才会认为这条 `redo log` 日志是完整的,才会进行数据重做,否则会认为这个 `redo log` 日志不完整,不会进行数据重做。

`undo log` 和 `redo log` 记录物理日志不一样,它是逻辑日志。可以认为当 `delete` 一条记录时,`undo log` 中会记录一条对应的 `insert` 记录,反之亦然,当 `update` 一条记录时,它记录一条对应相反的 `update` 记录。当执行回滚时,就可以从 `undo log` 中的逻辑记录读取到相应的内容并进行回滚。

而事务的隔离性,也可以通过 `undo log` 来实现的:当读取的某一行被其他事务锁定时,它可以从 `undo log` 中分析出该行记录以前的数据是什么,从而提供该行版本信息,帮助用户实现一致性非锁定读取,这也是 `MVCC` 的实现机制的组成部分。

什么是当前读和快照读?

当前读

像 `select lock in share mode`(共享锁), `select for update` ; `update`, `insert` ,`delete`(排他锁)这些操作都是一种当前读,就是它读取的是记录的最新版本,

读取时还要保证其他并发事务不能修改当前记录，会对读取的记录进行加锁。是一种悲观锁的实现。

快照读

像不加锁的 `select` 操作就是快照读，即不加锁的非阻塞读；快照读的前提是隔离级别不是串行级别，串行级别下的快照读会退化成当前读；之所以出现快照读的情况，是基于提高并发性能的考虑，快照读的实现是基于多版本并发控制，即 MVCC。

什么是 MVCC ?

MVCC (Multi-Version Concurrency Control) ，叫做基于多版本的并发控制协议。他是和 LBCC (Lock-Based Concurrency Control) 基于锁的并发控制概念是相对的。MVCC 是乐观锁的一种实现方式，它在很多情况下，避免了加锁操作，降低了开销；既然是基于多版本，即快照读可能读到的并不一定是数据的最新版本，而有可能是之前的历史版本。

MVCC 最大的好处：读不加锁，读写不冲突。在读多写少的 OLTP 应用中，读写不冲突是非常重要的，极大的增加了系统的并发性能，现阶段几乎所有的 RDBMS 包括 MySQL，都支持了 MVCC。

MVCC 的底层实现原理是什么？

MVCC 实现原理主要是依赖记录中的隐式字段，undo 日志，Read View 来实现的。

MySQL 中每行记录除了我们自定义的字段外，还有数据库隐式定义的 `DB_TRX_ID`, `DB_ROLL_PTR`, `DB_ROW_ID` 等字段。`DB_TRX_ID` 是最近修改(修改/插入)事务 ID，记录创建这条记录/最后一次修改该记录的事务 ID。`DB_ROLL_PTR`，回滚指针，用于配合 undo 日志，指向这条记录的上一个版本。

不同事务或者相同事务的对同一记录的修改，会导致该记录的 undo log 成为一条记录版本线性表，也就是版本链。

事务进行快照读操作的时候产生一个 Read View，记录并维护系统当前活跃事务的 ID，因为当每个事务开启时，都会被分配一个 ID，这个 ID 是递增的，所以最新的事务，ID 值越大。

Read View 主要是将要被修改的数据的最新记录中的 `DB_TRX_ID`（即当前事务 ID）取出来，与系统当前其他活跃事务的 ID 去对比（由 Read View 维护），如果 `DB_TRX_ID` 跟 Read View 的属性做了某些比较，不符合可见性，那就通过 `DB_ROLL_PTR` 回滚指针去取出 Undo Log 中的 `DB_TRX_ID` 再比较，即遍历链表的 `DB_TRX_ID`（从链首到链尾，即从最近的一次修改查起），直到找到满足特定条件的 `DB_TRX_ID`，那么这个 `DB_TRX_ID` 所在的旧记录就是当前事务能看见的最新老版本。

RC,RR 级别下 Read View 生成时机的不同，造成 RC,RR 级别下快照读的结果的不同。RC 隔离级别下，是每个快照读都会生成并获取最新的 Read View，也就是说事务中，每次快照读都会新生成一个快照和 Read View，这就是我们在 RC 级别下的事务中可以看到别的事务提交的更新的原因；而在 RR 隔离级别下，则是同一个事务中的第一个快照读才会创建 Read View，之后的快照读获取的都是同

一个 Read View，快照读生成 Read View 时，Read View 会记录此时所有其他活动事务的快照，这些事务的修改对于当前事务都是不可见的。而早于 Read View 创建的事务所做的修改均是可见。

什么是锁？MySQL 中提供了几类锁？

锁是实现数据库并发控制的重要手段，可以保证数据库在多人同时操作时能够正常运行。MySQL 提供了全局锁、行级锁、表级锁。其中 InnoDB 支持表级锁和行级锁，MyISAM 只支持表级锁。

什么是全局锁、共享锁、排它锁？

全局锁就是对整个数据库实例加锁，它的典型使用场景就是做全库逻辑备份。这个命令可以使整个库处于只读状态。使用该命令之后，数据更新语句、数据定义语句、更新类事务的提交语句等操作都会被阻塞。

共享锁又称读锁 (read lock)，是读取操作创建的锁。其他用户可以并发读取数据，但任何事务都不能对数据进行修改（获取数据上的排他锁），直到已释放所有共享锁。当如果事务对读锁进行修改操作，很可能造成死锁。

排他锁 exclusive lock（也叫 writer lock）又称写锁。

若某个事物对某一行加上了排他锁，只能这个事务对其进行读写，在此事务结束之前，其他事务不能对其进行加任何锁，其他进程可以读取，不能进行写操作，需等待其释放。排它锁是悲观锁的一种实现。

若事务 1 对数据对象 A 加上 X 锁，事务 1 可以读 A 也可以修改 A，其他事务不能再对 A 加任何锁，直到事物 1 释放 A 上的锁。这保证了其他事务在事物 1 释放 A 上的锁之前不能再读取和修改 A。排它锁会阻塞所有的排它锁和共享锁。

MySQL 中的表锁有哪些？

MySQL 里表级锁有两种：普通表级锁、元数据锁 (meta data lock) 简称 MDL 和 AUTO-INC 锁。表锁的语法是 lock tables t read/write。

可以用 unlock tables 主动释放锁，也可以在客户端断开的时候自动释放。lock tables 语法除了会限制别的线程的读写外，也限定了本线程接下来的操作对象。

对于 InnoDB 这种支持行锁的引擎，一般不使用 lock tables 命令来控制并发，毕竟锁住整个表的影响面还是太大。

MDL：不需要显式使用，在访问一个表的时候会被自动加上。

MDL 的作用：保证读写的正确性。

在对一个表做增删改查操作的时候，加 MDL 读锁；当要对表做结构变更操作的时候，加 MDL 写锁。

读锁之间不互斥，读写锁之间，写锁之间是互斥的，用来保证变更表结构操作的安全性。

AUTO-INC 锁，也就是在执行插入语句时就在表级别加一个 AUTO-INC 锁，然后为每条待插入记录的 AUTO_INCREMENT 修饰的列分配递增的值。

InnoDB 引擎的行锁是怎么实现的？

InnoDB 是基于索引来完成行锁，在锁的算法实现上有三种：

- Record lock: 单个行记录上的锁
- Gap lock: 间隙锁，锁定一个范围，不包括记录本身
- Next-key lock: record+gap 锁定一个范围，包含记录本身

Gap 锁设计的目的是为了阻止多个事务将记录插入到同一范围内，而这会导致幻读问题的产生，innodb 对于行的查询使用 next-key lock，Next-locking keying 是 Record lock 和 Gap lock 的组合。当查询的索引含有唯一属性时，将 next-key lock 降级为 record key。

有两种方式显式关闭 gap 锁，第一种. 将事务隔离级别设置为 RC；第二种. 将参数 innodb_locks_unsafe_for_binlog 设置为 1。

谈一下 MySQL 中的死锁

死锁是指两个或两个以上的进程在执行过程中，因争夺资源而造成的一种互相等待的现象,若无外力作用,它们都将无法推进下去。此时称系统处于死锁状态或系统产生了死锁。

如何查看死锁？

使用命令 `show engine innodb status` 查看最近的一次死锁。

InnoDB Lock Monitor 打开锁监控，每 15s 输出一次日志。使用完毕后建议关闭，否则会影响数据库性能。

对待死锁常见的两种策略：

通过 `innodb_lock_wait_timeout` 来设置超时时间，一直等待直到超时；

发起死锁检测，发现死锁之后，主动回滚死锁中的某一个事务，让其它事务继续执行。

简述下 MySQL8 中的新增特性有哪些

MySQL8 在功能上的我们需要关注增强主要有：1、账户与安全；2、索引 3、InnoDB 增强。

主要表现在：

1、用户的创建与授权需要两条单独的 SQL 语句执行。认证插件更新。密码管理和角色管理发生变化。

2、隐藏索引，被隐藏的索引不会被优化器使用，但依然真实存在，主要用于软删除，可以根据需要后续真正删除或者重新可视化。

开始真正支持降序索引，以往的 MySQL 虽然支持降序索引，但是写盘的时候依然是升序保存。MySQL8.0 中则是真正的按降序保存。

不再对 `group by` 操作进行隐式排序。

索引中可以使用函数表达式，创建表时创建一个函数索引，查询的时候使用同样的函数就可以利用索引了。

3、原子 ddl 操作，MySQL5.7 执行 drop 命令 drop table t1,t2; 如果 t1 存在，t2 不存在，会提示 t2 表不存在，但是 t1 表仍然会被删除，MySQL8.0 执行同样的 drop 命令，会提示 t2 表不存在，而且 t1 表不会被删除，保证了原子性。

自增列持久化，解决了之前的版本，主键重复的问题。MySQL5.7 及其以前的版本，MySQL 服务器重启，会重新扫描表的主键最大值，如果之前已经删除过 id=100 的数据，但是表中当前记录的最大值如果是 99，那么经过扫描，下一条记录的 id 是 100，而不是 101。MySQL8.0 则是每次在变化的时候，都会将自增计数器的最大值写入 redo log，同时在每次检查点将其写入引擎私有的系统表。则不会出现自增主键重复的问题。

10. 实战

10.1. 小实战

现在有三个表：商品表 goods、订单表 goods_order、订单详情表 goods_order_detail。三个表中的数据有两种方式导入：

1、执行 goods_stru.sql、goods_order_stru.sql、goods_order_detail_stru.sql 建立原始表，然后执行存储过程 create_goods 产生原始数据，再执行存储过程 randon_detail 进行数据更新；

2、执行 SQL 脚本 goods_all.sql、 goods_order_all.sql、 goods_order_detail_all.sql，同时导入表结构和数据。

三个表目前都没有索引：

对象 goods @mysqladv (aliyun57) - 表

开始事务 文本 筛选 排序

id	goods_name
1	goods_1
2	goods_2
3	goods_3
4	goods_4

对象 goods @mysqladv (aliyun57) - 表 goods_order @mysqladv (aliyu...

开始事务 文本 筛选 排序 导入 导出

id	order_no
1	order_1
2	order_2
3	order_3
4	order_4
5	order_5

对象 goods_order_detail @mysqladv...

开始事务 文本 筛选 排序 导入 导出

id	goods_id	order_id	price
1	1	1	369
2	470	1	202
3	1	1	902
4	561	1	906
5	51	1	822
6	96	1	395
7	470	1	507

三个表目前的记录数如下：

```
mysql> select count(*) from goods;
+-----+
| count(*) |
+-----+
|      1000 |
+-----+
1 row in set (0.00 sec)

mysql> select count(*) from goods_order;
+-----+
| count(*) |
+-----+
|    998863 |
+-----+
1 row in set (0.38 sec)

mysql> select count(*) from goods_order_detail;
+-----+
| count(*) |
+-----+
|    9988621 |
+-----+
1 row in set (2.76 sec)
```

现在有这么个业务场景：查找包含赠送商品且商品编号是 666 的订单

```
select o.* from goods_order o where o.id in(select order_id from
goods_order_detail od where od.goods_id=666 and od.price=0);
```

```
mysql> select o.* from goo
+-----+-----+
| id      | order_no |
+-----+-----+
| 663865  | order_663865 |
| 663972  | order_663972 |
| 664066  | order_664066 |
| 664435  | order_664435 |
| 664556  | order_664556 |
| 664802  | order_664802 |
+-----+-----+
6 rows in set (5.07 sec)
```

花了 5 秒多，有没有办法改进呢？先看执行计划如下：

```
mysql> explain select o.* from goods_order o where o.id in(select order_id from goods_order_detail od where od.goods_id=666 ar
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible keys | key | key_len | ref | rows |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | <subquery2> | NULL | ALL | NULL | NULL | NULL | NULL | NULL |
| 1 | SIMPLE | o | NULL | eq_ref | PRIMARY | PRIMARY | 4 | <subquery2>.order_id | 1 |
| 2 | MATERIALIZED | od | NULL | ALL | NULL | NULL | NULL | NULL | 9730497 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

从执行计划可以看出，MySQL 对这个查询的处理是：

1、建立了物化表；2、将子查询改为了利用物化表进行连接查询；3、只利用到了主键；4、全表扫描的情况很严重。

为了验证这一点，我们查看下 MySQL 对这个查询语句的改写：

```
mysql> show warnings;
```

```
/* select#1 */ select `mysqladv`.`o`.`id` AS `id`,`mysqladv`.`o`.`order_no` AS
`order_no` from `mysqladv`.`goods_order` `o` semi join
(`mysqladv`.`goods_order_detail` `od`) where ((`mysqladv`.`o`.`id` =
`<subquery2>`.`order_id`) and (`mysqladv`.`od`.`price` = 0) and
(`mysqladv`.`od`.`goods_id` = 666))
```



```
mysql> show warnings\G
+-----+
Level: Note
Code: 1003
Message: /* select#1 */ select 'mysqladv`.`o`.`id' AS 'id', 'mysqladv`.`o`.`order_no' AS 'order_no' from 'mysqladv`.`goods_order' `o` semi join ('mysqladv`.`goods_order_detail' `od`) where (('mysqladv`.`o`.`id' = '<subquery2>`.`order_id') and ('mysqladv`.`od`.`price' = 0) and ('mysqladv`.`od`.`goods_id' = 666))
1 row in set (0.00 sec)
```

可以看到 MySQL 将这个子查询改造为了半连接 semi join。那么我们要对这个 SQL 语句的执行进行优化。

分析我们的 SQL 语句，在 in 子查询中，用到了 goods_order_detail 中的 goods_id 字段和 price 字段作为条件，我们考虑增加一个 goods_id 字段和 price 字段的联合索引。

```
ALTER TABLE `mysqladv`.`goods_order_detail` ADD INDEX
`idx_price_price`(`goods_id`, `price`);
```

```
mysql> ALTER TABLE `mysqladv`.`goods_order_detail`
-> ADD INDEX `idx_price_price`(`goods_id`, `price`);
Query OK, 0 rows affected (55.89 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

再来看看执行效果和执行计划：

```
mysql> select o.* from goods_order
+-----+-----+
| id      | order_no |
+-----+-----+
| 663865  | order_663865 |
| 663972  | order_663972 |
| 664066  | order_664066 |
| 664435  | order_664435 |
| 664556  | order_664556 |
| 664802  | order_664802 |
+-----+-----+
6 rows in set (0.01 sec)
```

```
mysql> explain select o.* from goods_order o where o.id in(select order_id from goods_order_detail od where od.goods_id=666 and od.pr
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table      | partitions | type | possible keys | key | key_len | ref | rows |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE      | <subquery2> | NULL       | ALL  | NULL         | NULL | NULL    | NULL | NULL |
| 1 | SIMPLE      | o           | NULL       | eq_ref | PRIMARY      | PRIMARY | 4      | <subquery2>.order_id | 1 |
| 2 | MATERIALIZED | od          | NULL       | ref   | idx_price_price | idx_price_price | 12     | const,const | 6 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
3 rows in set, 1 warning (0.01 sec)
```

可以看到，虽然还有物化表，但是对物化表的扫描行数，已经由 9713504 降到了 6，对物化表也不再是全表扫描，速度提升到了 0.01 秒。

能不能再做提升呢？去除物化的过程和全表扫描呢？审视我们的子查询：
select order_id from goods_order_detail od where od.goods_id=666 and od.price=0;

可以利用索引覆盖，在索引扫描的时候直接把 order_id 查找出来，所以我们修改下索引 idx_price_price，再增加一个字段 order_id。

```
ALTER TABLE `mysqladv`.`goods_order_detail` DROP INDEX `idx_price_price`,
ADD INDEX `idx_price_price_order`(`goods_id`, `price`, `order_id`)
```

```
mysql> ALTER TABLE `mysqladv`.`goods_order_detail`
-> DROP INDEX `idx_price_price`,
-> ADD INDEX `idx_price_price_order`(`goods_id`, `price`, `order_id`);
Query OK, 0 rows affected (1 min 25.36 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

再来看看执行效果和执行计划：


```
mysql> select o.* from goods_order
+-----+-----+
| id      | order_no |
+-----+-----+
| 663865  | order_663865 |
| 663972  | order_663972 |
| 664066  | order_664066 |
| 664435  | order_664435 |
| 664556  | order_664556 |
| 664802  | order_664802 |
+-----+-----+
6 rows in set (0.00 sec)
```

```
mysql> explain select o.* from goods_order o where o.id in(select order_id from goods_order_detail od where od.goods_id=666 and od.price=0)
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | od | NULL | ref | idx_price_price_order | idx_price_price_order | 12 | const,const | 6 |
| 1 | SIMPLE | o | NULL | eq_ref | PRIMARY | PRIMARY | 4 | mysqladv.od.order_id | 1 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set, 1 warning (0.00 sec)
```

可以看到，已经完全去除了物化的过程和全表扫描。

当然，SQL 语句的执行中，上面的时间没有精确到毫秒，具体执行时长不知道是多少，没关系，我们用 `show profiles;` 来观察。

```
set profiling=1;
```

```
select o.* from goods_order o where o.id in(select order_id from
goods_order_detail od where od.goods_id=666 and od.price=0);
```

```
show profiles;
```

```
18 | 0.00075550 | select o.* from goods_order o where o.id in
```

我们将这个 SQL 语句由 5.07 秒优化到 0.01 秒，提升了 507 倍，再优化到 0.00075 秒，再次提升了 13 倍，和最初相比执行速度提升了 6760 倍。

总结：

- 1.建立索引很重要
- 2.合适的索引更重要
- 3、覆盖索引是个利器
- 4.学会分析 sql 执行计划，mysql 会对 sql 进行优化，所以分析执行计划很重要。

10.2. 啤酒和尿布

有这么个故事：全球零售业巨头沃尔玛在对消费者购物行为分析时发现，男性顾客在购买婴儿尿片时，常常会顺便搭配几瓶啤酒来犒劳自己，于是尝试推出了将啤酒和尿布摆在一起的促销手段。没想到这个举措居然使尿布和啤酒的销量都大幅增加了。如今，“啤酒+尿布”的数据分析成果早已成了大数据技术应用的经典案例。不过这个故事的真实性是存在着很大的疑问的。

但是这个故事本身反映的是销售商品相关性分析，其中的关联规则是反映一个事物与其他事物之间的相互依存性和关联性，常用于实体商店或在线电商的推荐系统：通过对顾客的购买记录数据库进行关联规则挖掘，最终目的是发现顾客群体的购买习惯的内在共性，例如购买产品 A 的同时也连带购买产品 B 的概率，根据挖掘结果，调整货架的布局陈列、设计促销组合方案，实现销量的提升。当然这个属于数据挖掘和大数据的领域，比如实际中使用 ClickHouse 来做这些工作。

那么我们用 MySQL 能做吗？在数据量不是特别大的情况下也是可以的。依然使用我们前面所用的三张表商品表 goods、订单表 goods_order、订单详情表 goods_order_detail 来试试。

现在的业务场景如下，对我们销售的商品找到关联度最高的商品，也就是在一张订单表中总是相伴出现的商品，不过考虑到实际情况，购买次数少的商品没有太大分析的必要。思路如下：

第一步，找到订单详情表中出现的商品明细，按商品在详情表中出现的次数排序：

```
select order_id,goods_id from goods_order_detail where
goods_id in (select odo.goods_id
from goods_order_detail odo
GROUP BY odo.goods_id
ORDER BY count(odo.goods_id));
```

第二步，通过第一步查询后的临时表作为主表，通过单号为关键字段，做连接（外连接和内连接均可，对结果基本没有影响）：

```
select aa.goods_id as src_goods,bb.goods_id as relative_goods,count(*) from
(select order_id,goods_id from goods_order_detail where
goods_id in (select odo.goods_id from goods_order_detail odo GROUP BY
odo.goods_id ORDER BY count(odo.goods_id))) aa
left join goods_order_detail bb
on aa.order_id=bb.order_id
where aa.goods_id != bb.goods_id
group by aa.goods_id,bb.goods_id
order by count(*) desc;
```

这个 SQL 语句的执行是很慢的，我们不去具体执行，看看他的执行计划和执行成本。

```
"cost_info": {
  "read_cost": "7535455.36",
  "eval_cost": "1698338806134.54",
  "prefix_cost": "18870441495094.36",
  "data_read_per_join": "185T"
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	goods_order_detail	NULL	index	NULL	idx_price_price_order	16	NULL	9713504	100.00	Using where; Using index; Using temporary; Using filesort
1	PRIMARY	bb	NULL	index	NULL	idx_price_price_order	16	NULL	9713504	9.00	Using where; Using index; Using join buffer (Block Nested Loop)
3	SUBQUERY	odo	NULL	range	idx_price_price_order	idx_price_price_order	4	NULL	14384	100.00	Using index for group-by

3 rows in set, 1 warning (0.00 sec)

执行成本为 1.88E13，从执行计划来看，扫描的行数和访问类型都让人不满意，需要改进一下。怎么改？

仔细分析我们的语句和业务，有必要去寻找所有商品的关联商品吗？购买次数很低的商品没必要进行这种统计与分析，所以我们先把找到订单详情表中出现的商品明细这个步骤提取出来，限定只寻找售出数量前 10 位的商品。

```
select odo.goods_id,count(odo.goods_id)
from goods_order_detail odo
GROUP BY odo.goods_id
ORDER BY count(odo.goods_id) desc
limit 10;
```

```
mysql> select odo.goods_id,count(odo.goods_id)
-> from goods_order_detail odo
-> GROUP BY odo.goods_id
-> ORDER BY count(odo.goods_id) desc
-> limit 10;
```

goods_id	count(odo.goods_id)
470	430034
202	356208
51	341176
561	339427
96	289927
164	279889
239	276058
702	269554
872	132594
941	45340

10 rows in set (2.93 sec)

这个速度还是可以接受的，我们把查询出来的商品在应用程序中缓存起来，然后，一个个商品去处理。比如先处理 id 为 470 的商品：

```
select bb.goods_id as relative_goods,count(*) from
(select order_id from goods_order_detail where
goods_id=470) aa
left join goods_order_detail bb
on aa.order_id=bb.order_id
where bb.goods_id != 470
group by bb.goods_id
order by count(*) desc;
```

```
mysql> explain select bb.goods_id as relative_goods,count(*) from
-> (select order_id from goods_order_detail where
-> goods_id=470) aa
-> left join goods_order_detail bb
-> on aa.order_id=bb.order_id
-> where bb.goods_id != 470
-> group by bb.goods_id
-> order by count(*) desc;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	goods_order_detail	NULL	ref	idx_price_price_order	idx_price_price_order	4	const	866494	100.00	Using index; Using temporary; Using filesort
1	SIMPLE	bb	NULL	index	idx_price_price_order	idx_price_price_order	16	NULL	9713504	10.00	Using where; Using index; Using join buffer (Block Nested Loop)

2 rows in set, 1 warning (0.00 sec)

成本变为:

```
"cost_info": {
  "read_cost": "695275.96",
  "eval_cost": "168333861207.89",
  "prefix_cost": "1683339457684.36",
  "data_read_per_join": "18T"
},
```

相比原来的 1.88E13，现在的 1.68E12，已经降低了一个数量级，但依然很大，最重要的是驱动表的扇出数和被驱动表的查询记录数都非常大，需要继续优化。虽然驱动表 aa 显示了“Using tempopray,Using filesort”，但是驱动表 aa 的优化余地很小，发力点还在对被驱动表 bb 的优化上，为什么？请同学自行思考。

目前这个 sql 语句利用的是 idx_price_price_order 索引，看起来对我们当前的业务不是特别适合，从执行计划来看，对被驱动表 bb 的扫描行数达到了 9713504，太大了。我们需要对索引进行调整，分析我们的 SQL 语句：

```
select bb.goods_id as relative_goods,count(*) from (select order_id from
goods_order_detail where goods_id=470) aa left join goods_order_detail bb
on aa.order_id=bb.order_id where bb.goods_id != 470 group by bb.goods_id
order by count(*) desc;
```

所以，我们考虑，在连接列 order_id 上建立一个索引：

```
ALTER TABLE `mysqladv`.`goods_order_detail` ADD INDEX
`idx_order`(`order_id`);
```

```
mysql> ALTER TABLE `mysqladv`.`goods_order_detail`
-> ADD INDEX `idx_order`(`order_id`);
Query OK, 0 rows affected (35.40 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> explain select bb.goods_id as relative_goods,count(*) from (select order_id from goods_order_detail where goods_id=470) aa left join goods_order_detail bb
on aa.order_id=bb.order_id where bb.goods_id != 470 group by bb.goods_id order by count(*) desc;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	goods_order_detail	NULL	ref	idx_price_price_order,idx_order	idx_price_price_order	4	const	866494	100.00	Using index; Using temporary; Using filesort
1	SIMPLE	bb	NULL	ref	idx_price_price_order,idx_order	idx_order	16	NULL	9	100.00	Using where

2 rows in set, 1 warning (0.00 sec)

可以看见，被驱动表 bb 的扫描行数降到了 9，成本变为了：


```

"cost_info": {
  "read_cost": "8425362.89",
  "eval_cost": "1685072.58",
  "prefix_cost": "10285848.67",
  "data_read_per_join": "192M"
},

```

相比原来的 1.88E13、1.68E12，现在降到了 1.02E7。

继续考虑，我们对被驱动表的访问，能不能再降低点成本？比如回表的成本？因为我们现在的索引 idx_order 是会引发回表的。所以我们再建立了一个联合索引：

```

ALTER TABLE `mysqladv`.`goods_order_detail`
DROP INDEX `idx_order`,
ADD INDEX `idx_order_goods`(`order_id`, `goods_id`)

```

为什么我们要删除 idx_order，建立 idx_order_goods，保留 idx_order 不好吗？请同学们自行思考。

```

mysql> ALTER TABLE `mysqladv`.`goods_order_detail`
-> DROP INDEX `idx_order`,
-> ADD INDEX `idx_order_goods`(`order_id`, `goods_id`);
Query OK, 0 rows affected (41.29 sec)
Records: 0 Duplicates: 0 Warnings: 0

```

```

mysql> explain select bb.goods_id as relative_goods,count(*) from (select order_id from goods_order_detail
on aa.order_id=bb.order_id where bb.goods_id != 470 group by bb.goods_id order by count(*) desc;
+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | goods_order_detail | NULL | ref | idx_price_price_order,idx_order | idx_price_order |
| 866494 | 100.00 | Using index; Using temporary; Using filesort |
| 1 | SIMPLE | bb | NULL | ref | idx_price_price_order,idx_order | idx_order_goods |
| 9 | 100.00 | Using where; Using index |
+-----+-----+-----+-----+-----+-----+-----+
2 rows in set, 1 warning (0.00 sec)

```

```

"cost_info": {
  "read_cost": "877582.02",
  "eval_cost": "1687923.00",
  "prefix_cost": "2740918.22",
  "data_read_per_join": "193M"
},

```

可以看见，被驱动表 bb 的扫描利用了覆盖索引，成本变为了 2.74E6，实际的查询时间：

```

+-----+-----+-----+
| 962 | 282 |
+-----+-----+-----+
960 rows in set (7.87 sec)

```

基本上在可以接受的范围内了。如果我们继续增加一个索引：


```
ALTER TABLE `mysqladv`.`goods_order_detail`
ADD INDEX `idx_goods_order`(`goods_id`,`order_id`);
```

虽然执行计划中的成本没有太大变化，但是查询时间还能降到 4.70S，基本上时间再次减半。

```
|          452 |          2196 |
|          962 |          282 |
+-----+-----+
960 rows in set (4.70 sec)
```

为什么？我们不是已经有了一个 idx_goods_price_order (`goods\_id`, `price`, `order\_id`) 吗？请同学们自行思考。

最后考虑到业务和网络传输量，960 个结果其实是没有必要的，所以最终的 SQL 如下：

```
select bb.goods_id as relative_goods,count(*) from
(select order_id from goods_order_detail where
goods_id=470) aa
left join goods_order_detail bb
on aa.order_id=bb.order_id
where bb.goods_id != 470
group by bb.goods_id
order by count(*) desc
limit 10;
```

索引如下：

字段	索引	外键	触发器	选项	注释	SQL 预览			
名	字段						索引类型	索引方法	注
▶ idx_goods_price_order	`goods_id`, `price`, `order_id`						NORMAL	BTREE	
idx_goods_order	`goods_id`, `order_id`						NORMAL	BTREE	
idx_order_goods	`order_id`, `goods_id`						NORMAL	BTREE	

当然上面的 SQL 如果去除排序，比如变为

```
Select .....
group by bb.goods_id
order by null;
```

执行成本还可降低到 2.55E6 同时执行计划里不再出现 using_filesort，不过对整体的影响并不大，从实际的执行时间来看，也确实是如此。

在这种情况下，执行成本相比原来的 1.88E13、1.68E12，现在降到了 1.02E7，再到 2.74E6，实际的一个商品的查询时间为 4.7 秒，算上我们要查询十个商品，总时间为 4.7*10+2.93=50 秒，基本上查询成本从 1.88E13 变为 2.74E6*10，降低了 10 万倍，对于我们这种属于非实时统计的业务来说，这个速度相对还是不错的。

的。当然如果你对上面的业务还有更好的优化措施，并且实验通过了，请和 Mark 老师分享。

以上，就是我们 SQL 优化的全部内容，如果觉得在这门课上觉得 Mark 老师讲的还不错，学到了很多不错的知识，请在以下链接给 Mark 老师一个五星好评，，你的鼓励是 Mark 老师和享学课堂不断前行和给大家带来更优秀课程的动力。

<https://ke.qq.com/comment/index.html?cid=287404>